Gareth James • Daniela Witten • Trevor Hastie
Robert Tibshirani • Jonathan Taylor

An Introduction to Statistical Learning

with Applications in Python

First Printing: July 5, 2023

To our parents:

Alison and Michael James

Chiara Nappi and Edward Witten

Valerie and Patrick Hastie

Vera and Sami Tibshirani

John and Brenda Taylor

and to our families:

Michael, Daniel, and Catherine

Tessa, Theo, Otto, and Ari

Samantha, Timothy, and Lynda

Charlie, Ryan, Julie, and Cheryl

Lee-Ann and Isobel

前言

统计学习指的是一套用于*making sense of complex datasets*的工具。近年来，我们见证了几乎所有科学和工业领域数据收集规模和范围的惊人增长。因此，统计学习已成为任何希望理解数据的人的关键工具包——而随着当今越来越多的工作涉及数据，这意味着统计学习正迅速成为*everyone*的关键工具包。

最早关于统计学习的书籍之一——*The Elements of Statistical Learning* (ESL，由Hastie、Tibshirani和Friedman合著)——于2001年出版，并于2009年推出了第二版。ESL不仅在统计学领域，也在相关领域成为了一本广受欢迎的教材。ESL受欢迎的原因之一是其相对易于理解的风格。然而，ESL最适合那些在数学科学方面受过高级训练的人士。

An Introduction to Statistical Learning, With Applications in R (ISLR)——首次出版于2013年，并于2021年推出第二版——源于对统计学习关键主题进行更广泛且技术性较弱的处理的明确需求。除了回顾线性回归外，ISLR还涵盖了许多当今最重要的统计和机器学习方法，包括重采样、分类和回归的稀疏方法、广义加性模型、基于树的方法、支持向量机、深度学习、生存分析、聚类以及多重检验。

自2013年出版以来，ISLR已成为全球本科生和研究生课堂的主要内容，同时也是数据科学家的重要参考书。其成功的关键之一是，从第2章开始，每章都包含一个`R`实验，说明如何实现该章中介绍的统计学习方法，为读者提供宝贵的实践经验。

然而，近年来`Python`已成为数据科学中日益流行的语言，并且对`Python`的需求也在不断增加。

基于ISLR的替代方案。因此，本书，*An Introduction to Statistical Learning, With Applications in Python* (ISLP)，涵盖了与ISLR相同的材料，但实验部分是用Python实现的——这一壮举得益于新增了一位合著者，Jonathan Taylor。其中几个实验利用了ISLP Python包，我们编写该包是为了方便在Python中执行每一章所涵盖的统计学习方法。这些实验对于Python新手以及经验丰富的用户都将非常有用。

ISLP（以及ISLR）的意图是更侧重于方法的应用，而较少关注数学细节，因此它适合统计学或相关定量领域的高年级本科生或硕士生，也适合希望使用统计学习工具分析数据的其他学科的人。它可以用作跨越两个学期的课程的教科书。

我们感谢以下读者对ISLR第一版提供了宝贵意见：帕拉维·巴苏、亚历珊德拉·乔德乔娃、帕特里克·丹纳赫、威尔·菲西安、卢埃拉·傅、萨姆·格罗斯、马克斯·格雷泽·G'Sell、考特尼·保尔森、乔兴浩、伊莉莎·盛、诺亚·西蒙、陈家明、陈欣露。我们感谢以下读者对ISLR第二版提供的有益投入：艾伦·阿格雷斯蒂、伊恩·卡迈克尔、陈一坤、艾琳·克雷格、丁黛西、高露西、伊斯梅尔·勒姆哈德里、布莱恩·马丁、安娜·纽菲尔德、杰夫·蒂姆斯、卡斯滕·沃尔克曼、史蒂夫·亚德洛夫斯基和詹姆斯·邹。我们对巴拉苏布拉马尼亚姆·“纳拉斯”·纳拉辛汉在ISLR和ISLP上的协助表示万分感激。

我们深感荣幸能够见证ISLR在学术内外对统计学习实践方式所产生的重大影响。我们期望这个新的Python版本能继续为当今及未来的应用统计学家和数据科学家提供他们在数据驱动世界中取得成功所需的工具。

It's tough to make predictions, especially about the future.

-Yogi Berra

目录

前言 vii

1 引言 1

2 统计学习 15
2.1 什么是统计学习? 15
2.1.1 为什么要估计 f？ . . . . . 17 2.1.2 我们如何估计 f？ . . . . . 20 2.1.3 预测准确性与模型可解释性之间的权衡 . . . . . 23 2.1.4 监督学习与无监督学习 . . . . . 25 2.1.5 回归与分类问题 27

2.2 评估模型准确性 27
2.2.1 测量拟合质量 . . . . . 28 2.2.2 偏差-方差权衡 . . . . . 31 2.2.3 分类设置 34

2.3 实验：Python 入门 40
2.3.1 入门 . . . . . 40 2.3.2 基本命令 . . . . . 40 2.3.3 数值 Python 简介 . . . . . 42 2.3.4 图形 . . . . . 48 2.3.5 序列与切片表示法 . . . . . 51 2.3.6 索引数据 . . . . . 51 2.3.7 加载数据 . . . . . 55 2.3.8 For 循环 . . . . . 59 2.3.9 附加的图形与数值摘要 61

2.4 练习 . . . . . 63 3 线性回归 69

3.1 简单线性回归 . . . . . 70 3.1.1 估计系数 . . . . . 71 3.1.2 评估系数估计的准确性 . . . . . 72 3.1.3 评估模型的准确性 . . . . . 77 3.2 多元线性回归 . . . . . 80 3.2.1 估计回归系数 81

3.2.2 一些重要问题 . . . . . 83 3.3 回归模型中的其他考虑因素 . . . . . 91 3.3.1 定性预测变量 . . . . . 91 3.3.2 线性模型的扩展 . . . . . 94 3.3.3 潜在问题 . . . . . 100 3.4 营销计划 . . . . . 109 3.5 线性回归与 K 最近邻的比较 . . . . . 111 3.6 实验：线性回归 . . . . . 116 3.6.1 导入包 . . . . . 116 3.6.2 简单线性回归 . . . . . 117 3.6.3 多元线性回归 . . . . . 122 3.6.4 多变量拟合优度 . . . . . 123 3.6.5 交互项 . . . . . 124 3.6.6 预测变量的非线性变换 . . . . . 125 3.6.7 定性预测变量 . . . . . 126 3.7 练习 127

4 分类 135
4.1 分类概述 . . . . . 135 4.2 为什么不使用线性回归? . . . . . 136 4.3 逻辑回归 138
4.3.1 逻辑回归模型 . . . . . 139 4.3.2 估计回归系数 . . . . . 140 4.3.3 进行预测 . . . . . 141 4.3.4 多元逻辑回归 . . . . . 142 4.3.5 多项逻辑回归 144
4.4 用于分类的生成模型 146
4.4.1 $p = 1$ 的线性判别分析 . . . . . 147 4.4.2 $p >1$ 的线性判别分析 . . . . . 150 4.4.3 二次判别分析 . . . . . 156 4.4.4 朴素贝叶斯 158
4.5 分类方法的比较 161
4.5.1 分析性比较 . . . . . 161 4.5.2 经验性比较 164
4.6 广义线性模型 167
4.6.1 Bikeshare 数据上的线性回归 . . . . . 167 4.6.2 Bikeshare 数据上的泊松回归 . . . . . 169 4.6.3 更一般化的广义线性模型 . 172
4.7 实验：逻辑回归、LDA、QDA 和 KNN 173
4.7.1 股票市场数据 . . . . . 173 4.7.2 逻辑回归 . . . . . 174 4.7.3 线性判别分析 . . . . . 179 4.7.4 二次判别分析 . . . . . 181 4.7.5 朴素贝叶斯 . . . . . 182 4.7.6 K最近邻 183 4.7.7 自行车共享数据集上的线性回归与泊松回归188
4.8 练习 193

5 重采样方法 201 5.1 交叉验证 . . . . . 202 5.1.1 验证集方法 . . . . . 202 5.1.2 留一交叉验证 . . . . . 204 5.1.3 k-折交叉验证 . . . . . 206 5.1.4 k-折交叉验证的偏差-方差权衡 . . . . . 208 5.1.5 分类问题上的交叉验证 . . . . . 209 5.2 自助法 . . . . . 212 5.3 实验：交叉验证与自助法 . . . . . 215 5.3.1 验证集方法 . . . . . 216 5.3.2 交叉验证 . . . . . 217 5.3.3 自助法 . . . . . 220 5.4 练习 224

6 线性模型选择与正则化 229 6.1 子集选择 . . . . . 231 6.1.1 最优子集选择 . . . . . 231 6.1.2 逐步选择 . . . . . 233 6.1.3 选择最优模型 . . . . . 235 6.2 收缩方法 . . . . . 240 6.2.1 岭回归 . . . . . 240 6.2.2 Lasso . . . . . 244 6.2.3 选择调节参数 . . . . . 252 6.3 降维方法 . . . . . 253 6.3.1 主成分回归 . . . . . 254 6.3.2 偏最小二乘 . . . . . 260 6.4 高维情形下的考量 . . . . . 262 6.4.1 高维数据 . . . . . 262 6.4.2 高维情形下会出现什么问题？ . . . . . 263 6.4.3 高维中的回归 . . . . . 265 6.4.4 解释高维中的结果 . . . . . 266 6.5 实验：线性模型与正则化方法 . . . . . 267 6.5.1 子集选择方法 . . . . . 268 6.5.2 岭回归与Lasso . . . . . 273 6.5.3 PCR与PLS回归 . . . . . 280 6.6 练习 283

7 超越线性 289 7.1 多项式回归 . . . . . 290 7.2 阶梯函数 . . . . . 292 7.3 基函数 . . . . . 293 7.4 回归样条 . . . . . 294 7.4.1 分段多项式 . . . . . 294 7.4.2 约束与样条 . . . . . 296 7.4.3 样条基表示 . . . . . 296 7.4.4 选择节点的数量和位置 . . . . . 297 7.4.5 与多项式回归的比较 299

7.5 平滑样条 . . . . . 300 7.5.1 平滑样条概述 . . . . . 300 7.5.2 选择平滑参数 λ . . . . . 301 7.6 局部回归 . . . . . 303 7.7 广义加性模型 . . . . . 305 7.7.1 用于回归问题的GAM . . . . . 306 7.7.2 用于分类问题的GAM . . . . . 308 7.8 实验：非线性建模 309

7.8.1 多项式回归与阶梯函数 . . . . . 310 7.8.2 样条 . . . . . 315 7.8.3 平滑样条与GAMs . . . . . 317 7.8.4 局部回归 324

7.9 练习 325

8 基于树的方法 331
8.1 决策树基础 . . . . . 331 8.1.1 回归树 . . . . . 331 8.1.2 分类树 . . . . . 337 8.1.3 树与线性模型 . . . . . 341 8.1.4 树的优缺点 . . . . . 341 8.2 Bagging、随机森林、Boosting 与贝叶斯加性回归树 . . . . . 343 8.2.1 Bagging . . . . . 343 8.2.2 随机森林 . . . . . 346 8.2.3 Boosting . . . . . 347 8.2.4 贝叶斯加性回归树 . . . . . 350 8.2.5 树集成方法总结 . . . . . 353 8.3 实验：基于树的方法 . . . . . 354 8.3.1 拟合分类树 . . . . . 355 8.3.2 拟合回归树 . . . . . 358 8.3.3 Bagging 与随机森林 . . . . . 360 8.3.4 Boosting . . . . . 361 8.3.5 贝叶斯加性回归树 362

8.4 练习 363

9 支持向量机 367
9.1 最大间隔分类器 367

9.1.1 什么是超平面? . . . . . 368 9.1.2 使用分离超平面进行分类 . . . 368 9.1.3 最大间隔分类器 . . . . . 370 9.1.4 最大间隔分类器的构建 . . 372 9.1.5 不可分情形 372

9.2 支持向量分类器 . . . . . 373 9.2.1 支持向量分类器概述 . . . . . 373 9.2.2 支持向量分类器的细节 . . . . . 374 9.3 支持向量机 . . . . . 377 9.3.1 非线性决策边界的分类 . . . . . 378 9.3.2 支持向量机 379

9.3.3 一个应用于心脏病数据的案例 . . . . . 382 9.4 多类支持向量机 . . . . . 383 9.4.1 一对一分类 . . . . . 384 9.4.2 一对多分类 . . . . . 384 9.5 与逻辑回归的关系 . . . . . 384 9.6 实验：支持向量机 . . . . . 387 9.6.1 支持向量分类器 . . . . . 387 9.6.2 支持向量机 . . . . . 390 9.6.3 ROC曲线 . . . . . 392 9.6.4 多类SVM . . . . . 393 9.6.5 应用于基因表达数据 . . . . . 394 9.7 练习 395

10 深度学习 399
10.1 单层神经网络 . . . . . 400 10.2 多层神经网络 . . . . . 402 10.3 卷积神经网络 . . . . . 406 10.3.1 卷积层 . . . . . 407 10.3.2 池化层 . . . . . 410 10.3.3 卷积神经网络的结构 . . . 410 10.3.4 数据增强 . . . . . 411 10.3.5 使用预训练分类器的结果 . . . . . 412 10.4 文档分类 . . . . . 413 10.5 循环神经网络 . . . . . 416 10.5.1 用于文档分类的序列模型 . . 418 10.5.2 时间序列预测 . . . . . 420 10.5.3 RNN 总结 424
10.6 何时使用深度学习 . . . . . 425 10.7 拟合神经网络 427
10.7.1 反向传播 . . . . . 428 10.7.2 正则化与随机梯度下降 . . 429 10.7.3 Dropout学习 . . . . . 431 10.7.4 网络调优 431
10.8 插值与双重下降 432
10.9 实验：深度学习 . . . . . 435 10.9.1 Hitters数据上的单层网络 . . . . . 437 10.9.2 MNIST数字数据上的多层网络 . . . . . 444 10.9.3 卷积神经网络 . . . . . 448 10.9.4 使用预训练的CNN模型 . . . . . 452 10.9.5 IMDB文档分类 . . . . . 454 10.9.6 循环神经网络 . . . . . 458 10.10 习题 465

11 生存分析与删失数据 469
11.1 生存时间与删失时间 . . . . . 470 11.2 深入探讨删失 . . . . . 470 11.3 Kaplan–Meier 生存曲线 . . . . . 472 11.4 对数秩检验 . . . . . 474 11.5 带有生存响应的回归模型 476

11.5.1 风险函数 . . . . . 476 11.5.2 比例风险 . . . . . 478 11.5.3 示例：脑癌数据 . . . . . 482 11.5.4 示例：发表数据 482

11.6 Cox模型的收缩 484
11.7 附加主题 486
11.7.1 生存分析中的曲线下面积 . . . . 486 11.7.2 时间尺度的选择 . . . . . 487 11.7.3 时间依赖协变量 . . . . . 488 11.7.4 检验比例风险假设 . . 488 11.7.5 生存树 488

11.8 实验：生存分析 489
11.8.1 脑癌数据 . . . . . 489 11.8.2 出版物数据 . . . . . 493 11.8.3 呼叫中心数据 494
11.9 练习 498

12 无监督学习 503
12.1 无监督学习的挑战 . . . . . 503 12.2 主成分分析 . . . . . 504 12.2.1 什么是主成分？ . . . . . 505 12.2.2 主成分的另一种解释 . . . . . 508 12.2.3 方差解释比例 . . . . . 510 12.2.4 关于主成分分析的更多内容 . . . . . 512 12.2.5 主成分的其他用途 515
12.3 缺失值与矩阵补全 515
12.4 聚类方法 520
12.4.1 K-均值聚类 . . . . . 521 12.4.2 层次聚类 . . . . . 525 12.4.3 聚类中的实际问题 532

12.5 实验：无监督学习 535
12.5.1 主成分分析 . . . . . 535 12.5.2 矩阵补全 . . . . . 539 12.5.3 聚类 542 12.5.4 NCI60数据示例 546

12.6 练习 552

13 多重检验 557
13.1 假设检验快速回顾 . . . . . 558 13.1.1 检验一个假设 . . . . . 558 13.1.2 第一类错误与第二类错误 562
13.2 多重检验的挑战 563
13.3 族系错误率 565
13.3.1 什么是族系错误率？ . . . . . 565 13.3.2 控制族系错误率的方法 567 13.3.3 族系错误率与功效之间的权衡 572

13.4 错误发现率 573
13.4.1 错误发现率的直观理解 . . . . . 573 13.4.2 Benjamini–Hochberg 方法 575

13.5 p 值的重抽样方法与错误发现率 . . . . . 577 13.5.1 p 值的重抽样方法 . . . . . 578 13.5.2 错误发现率的重抽样方法 . . . . . 579 13.5.3 重抽样方法何时有用? . . . . . 581 13.6 实验：多重检验 . . . . . 583 13.6.1 假设检验回顾 . . . . . 583 13.6.2 族错误率 . . . . . 585 13.6.3 错误发现率 . . . . . 588 13.6.4 重抽样方法 . . . . . 590 13.7 习题 593

Index **597**

1
简介



统计学习概述

Statistical learning 是指用于 *understanding data* 的大量工具集。这些工具可以分类为 *supervised* 或 *unsupervised*。广义而言，有监督统计学习涉及建立一个统计模型，用于基于一个或多个 *inputs* 来预测或估计 *output*。这类问题出现在商业、医学、天体物理学和公共政策等多个领域。在无监督统计学习中，有输入但没有监督输出；尽管如此，我们可以从这些数据中学习关系和结构。为了说明统计学习的一些应用，我们简要讨论本书中考虑的三个真实世界数据集。

Wage Data

在本应用中（在本书中我们称之为 {v*} 数据集），我们考察了与美国大西洋地区一组男性工资相关的若干因素。特别地，我们希望理解员工的 `age` 和 `education`，以及日历 `year` 对其 `wage` 的关联。例如，考虑图1.1左侧面板，它展示了数据集中每个个体的 `wage` 与 `age` 的关系。有证据表明，`wage` 随 `age` 增加，但在大约60岁之后又下降。蓝线给出了给定 `age` 时平均 `wage` 的估计，使这一趋势更加清晰。给定员工的 `age`，我们可以利用这条曲线来 *predict* 其 `wage`。然而，从图1.1也可以清楚地看出，与这个平均值相关的变异性相当大，因此仅凭 `age` 不太可能准确预测某个特定男性的 `wage`。

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_1

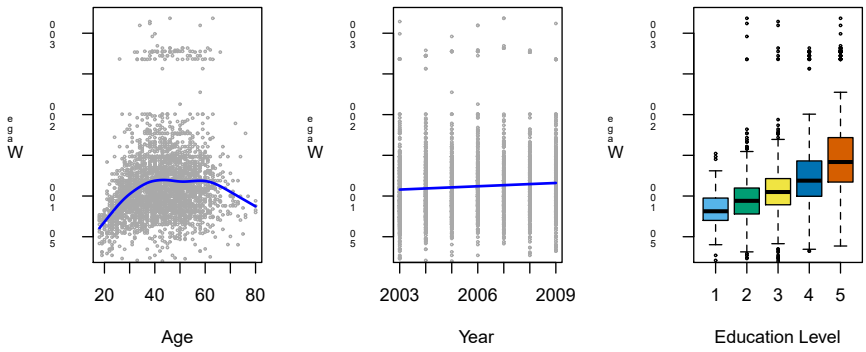


图1.1。`Wage` *data, which contains income survey information for men from the central Atlantic region of the United States.* 左：`wage` *as a function of* `age`*. On average,* `wage` *increases with* `age` *until about* 60 *years of age, at which point it begins to decline.* 中：`wage` *as a function of* `year`*. There is a slow but steady increase of approximately* $10,000 *in the average* `wage` *between* 2003 *and* 2009*.* 右：*Boxplots displaying* `wage` *as a function of* `education`*, with* 1 *indicating the lowest level (no high school diploma) and* 5 *the highest level (an advanced graduate degree). On average,* `wage` *increases with the level of education.*

我们还有关于每位员工教育水平以及`wage`获得所在`year`的信息。图1.1的中部和右侧面板展示了`wage`作为`year`和`education`的函数，表明这两个因素都与`wage`有关。在2003年至2009年间，工资以大约10,000美元的幅度增长，呈现出大致线性（或直线）的趋势，尽管这一增长相对于数据的变异性来说非常微小。此外，教育水平较高的个体通常工资也更高：教育水平最低（1）的男性往往比教育水平最高（5）的男性工资低得多。显然，要最准确地预测某个男性的`wage`，需要结合他的`age`、`education`以及`year`。在第3章中，我们将讨论线性回归，它可以用来根据这个数据集预测`wage`。理想情况下，我们预测`wage`时应考虑到`wage`与`age`之间的非线性关系。在第7章中，我们将讨论一类解决这一问题的方法。

Stock Market Data

`Wage` 数据涉及预测一个 *continuous* 或 *quantitative* 输出值。这通常被称为 *regression* 问题。然而，在某些情况下，我们可能希望预测一个非数值——即 *categorical* 或 *qualitative* 输出。例如，在第4章中，我们研究了一个股票市场数据集，其中包含2001年至2005年5年间标准普尔500（S&P）股票指数的每日变动。我们将此称为 `Smarket` 数据。目标是利用过去5天指数的百分比变化，预测指数在某一天会 *increase* 还是 *decrease*。这里，统计学习问题不涉及预测数值。相反，它涉及预测给定的

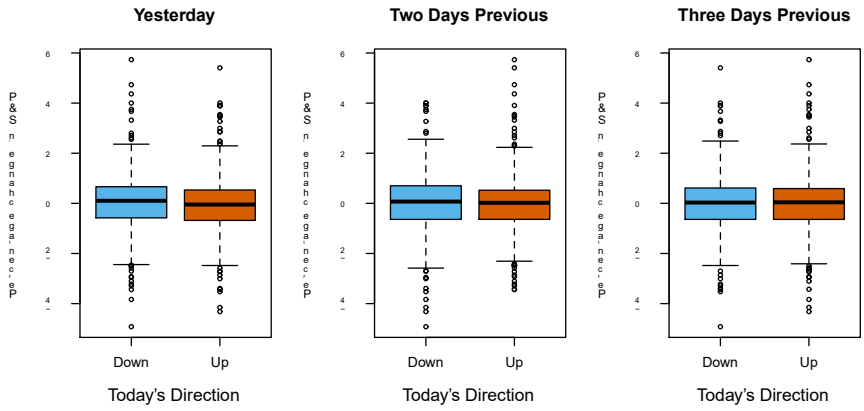


图1.2。左：*Boxplots of the previous day's percentage change in the S&P index for the days for which the market increased or decreased, obtained from the* `Smarket` *data.* 中右：*Same as left panel, but the percentage changes for 2 and 3 days previous are shown.*

当天的股市表现将归入 `Up` 桶或 `Down` 桶。这被称为 *classification* 问题。一个能够准确预测市场走向的模型将非常有用！

图1.2的左侧面板展示了股票指数前一日百分比变化的两个箱线图：一个对应随后一天市场上涨的648天，另一个对应市场下跌的602天。这两个图几乎完全相同，表明没有简单的策略可以利用标普昨日走势来预测今日回报。其余面板显示了今日之前2天和3天的百分比变化箱线图，同样表明过去与当前回报之间几乎没有关联。当然，这种缺乏模式的现象是意料之中的：如果连续几天的回报之间存在强相关性，那么人们可以采用简单的交易策略从市场获利。尽管如此，在第4章中，我们使用几种不同的统计学习方法探索这些数据。有趣的是，数据中存在一些弱趋势的迹象，表明至少在这个5年期间，有可能在大约60%的情况下正确预测市场运动的方向（图1.3）。

Gene Expression Data

前两个应用展示了同时包含输入变量和输出变量的数据集。然而，另一类重要问题涉及我们仅能观察到输入变量而无对应输出的情况。例如，在营销场景中，我们可能拥有众多现有或潜在客户的人口统计信息。我们可能希望通过根据观察到的特征对个体进行分组，来理解哪些类型的客户彼此相似。这被称为

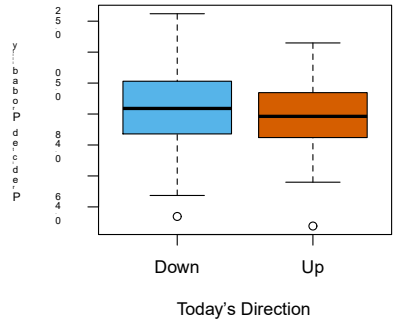


图 1.3. *We fit a quadratic discriminant analysis model to the subset of the* `Smarket` *data corresponding to the 2001–2004 time period, and predicted the probability of a stock market decrease using the 2005 data. On average, the predicted probability of decrease is higher for the days in which the market does decrease. Based on these results, we are able to correctly predict the direction of movement in the market 60% of the time.*

clustering 与之前的例子不同，这里我们并不试图预测一个输出变量。

我们将第12章专门讨论在不存在自然输出变量的问题中的统计学习方法。我们考虑`NCI60`数据集，它由64个癌细胞系中每个细胞系的6,830个基因表达测量值组成。我们不是预测某个特定的输出变量，而是有兴趣根据基因表达测量值确定这些细胞系之间是否存在分组或聚类。这是一个难以回答的问题，部分原因是每个细胞系有数千个基因表达测量值，使得数据难以可视化。

图1.4左侧面板通过仅使用两个数字Z_1和Z_2来表示64个细胞系中的每一个，从而解决了这一问题。它们是数据的前两个*principal components*，将每个细胞系的6,830个表达测量值汇总为两个数字或*dimensions*。虽然这种降维很可能导致了一些信息损失，但现在可以直观地检查数据以寻找聚类证据。确定聚类数量通常是一个难题。但图1.4的左侧面板至少提示了四组细胞系，我们用不同的颜色表示了它们。

在这个特定的数据集中，事实证明细胞系对应于14种不同类型的癌症。（然而，这一信息并未用于创建图1.4左侧面板。）图1.4右侧面板与左侧面板完全相同，只是用不同颜色的符号显示了这14种癌症类型。有明确的证据表明，具有相同癌症类型的细胞系在这个二维表示中往往彼此靠近。此外，尽管癌症信息并未用于生成左侧面板，但所得到的聚类与右侧面板中观察到的一些实际癌症类型确实有一定相似性。这为我们聚类分析的准确性提供了一定的独立验证。

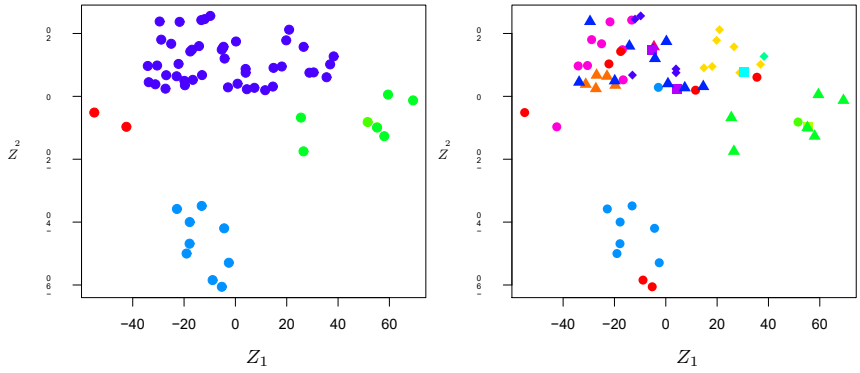


图1.4。左：*Representation of the* NCI60 *gene expression data set in a two-dimensional space,* Z_1 *and* Z_2*. Each point corresponds to one of the* 64 *cell lines. There appear to be four groups of cell lines, which we have represented using different colors.* 右：*Same as left panel except that we have represented each of the* 14 *different types of cancer using a different colored symbol. Cell lines corresponding to the same cancer type tend to be nearby in the two-dimensional space.*

统计学习简史

尽管*statistical learning*这个术语相当新，但该领域背后的许多概念早在很久以前就已发展起来。在十九世纪初，*least squares*方法被开发出来，实现了现在被称为*linear regression*的最早形式。这种方法首次成功应用于天文学问题。线性回归用于预测定量值，例如个人的工资。为了预测定性值，例如患者是存活还是死亡，或者股市是上涨还是下跌，*linear discriminant analysis*于1936年被提出。在20世纪40年代，多位作者提出了另一种方法，*logistic regression*。在20世纪70年代初，*generalized linear model*这个术语被开发出来，用以描述一整类统计学习方法，其中包括线性回归和逻辑回归作为特例。

到20世纪70年代末，已有更多从数据中学习的技术可用。然而，由于当时拟合*non-linear*关系在计算上颇为困难，这些技术几乎全是*linear*方法。到了20世纪80年代，计算技术终于进步到足以使非线性方法不再受计算能力限制。80年代中期，*classification and regression trees*被开发出来，紧随其后的是*generalized additive models*。*Neural networks*在80年代流行起来，而*support vector machines*则在90年代兴起。

从那时起，统计学习作为统计学的一个新子领域出现，专注于监督和无监督建模与预测。近年来，统计学习的进展以越来越强大的、相对用户友好的软件的可用性为标志，例如流行且免费提供的 {v*} 系统。这有可能继续将该领域从一组使用的技术和

由统计学家和计算机科学家开发，成为了更广泛群体不可或缺的工具包。

本书

The Elements of Statistical Learning (Hastie、Tibshirani和Friedman合著的ESL)首次出版于2001年。自那时起，它已成为统计机器学习基础领域的重要参考文献。其成功源于对统计学习中众多重要主题的全面而详尽的阐述，以及相对于许多高级统计学教科书而言，它对广泛读者群体的可及性。然而，ESL成功背后的最大因素在于其话题性。在其出版之际，对统计学习领域的兴趣正开始激增。ESL提供了该主题最早且易于理解、内容全面的入门介绍之一。

自《ESL》首次出版以来，统计学习领域持续蓬勃发展。该领域的扩张呈现两种形态。最显著的增长涉及开发新的和改进的统计学习方法，旨在解答跨越多个领域的一系列科学问题。然而，统计学习领域也扩大了其受众范围。在20世纪90年代，计算能力的提升激发了非统计学家的浓厚兴趣，他们渴望使用前沿的统计工具来分析自己的数据。遗憾的是，这些方法的高度技术性意味着用户群体主要局限于统计、计算机科学及相关领域的专家，他们具备理解和实施这些方法所需的训练（及时间）。

近年来，新的和改进的软件包显著减轻了许多统计学习方法的实施负担。与此同时，从商业到医疗保健、遗传学、社会科学等众多领域，人们越来越认识到统计学习是一种具有重要实际应用价值的强大工具。因此，该领域已从主要具有学术兴趣的领域转变为一个拥有巨大潜在受众的主流学科。随着大量数据及其分析软件的日益普及，这一趋势必将持续下去。

An Introduction to Statistical Learning (ISL)的目的是促进统计学习从学术领域向主流领域的过渡。ISL并不旨在取代ESL，后者在考虑的方法数量和探索的深度上都是一本更为全面的著作。我们认为ESL是专业人士（拥有统计学、机器学习或相关领域的研究生学位）的重要伴侣，他们需要理解统计学习方法背后的技术细节。然而，统计学习技术的用户群体已扩大到包括具有更广泛兴趣和背景的个人。因此，为ESL提供一个技术性较低且更易于理解的版本是有其必要性的。

多年来，在教授这些主题的过程中，我们发现它们引起了来自工商管理、生物学和计算机科学等不同领域的硕士和博士研究生的兴趣，同时也吸引了注重定量分析的高年级本科生。对于这一多元化的群体来说，能够理解各种方法的模型、直觉以及优缺点至关重要。然而，对于这些受众而言，统计学习方法背后的许多技术细节，如优化算法和理论性质，并非他们主要关注的内容。我们相信，这些学生无需对这些方面有深入的理解，便能成为各种方法论的有识之用户，并通过使用统计学习工具为他们所选领域做出贡献。

ISL基于以下四个前提。

我们相信，许多当代的统计学习程序应该并且将会变得像当前经典方法（如线性回归）一样广泛可用和使用。因此，我们没有试图考虑每一种可能的方法（这是一项不可能完成的任务），而是集中于介绍我们认为最广泛适用的方法。

2. *Statistical learning should not be viewed as a series of black boxes.* 没有任何一种方法能在所有可能的应用中都表现良好。如果不了解盒子内部所有的齿轮，或者这些齿轮之间的相互作用，就不可能选出最佳的盒子。因此，我们试图仔细描述我们所考虑的每种方法背后的模型、直觉、假设和权衡。

3. *While it is important to know what job is performed by each cog, it is not necessary to have the skills to construct the machine inside the box!* 因此，我们尽量减少了对拟合程序和相关理论性质技术细节的讨论。我们假设读者熟悉基本的数学概念，但并不要求具备数学科学的研究生学位。例如，我们几乎完全避免了使用矩阵代数，即使没有矩阵和向量的详细知识，也完全可以理解整本书。

4. *We presume that the reader is interested in applying statistical learning methods to real-world problems.* 为了方便这一点，并激发所讨论的技术，我们在每章中都专门安排了一节计算机实验。在每个实验中，我们引导读者实际应用该章所考虑的方法。当我们在课程中教授这些材料时，我们将大约三分之一的课堂时间用于完成实验，并且我们发现它们极其有用。许多最初对实验感到畏惧、不太擅长计算的学生在学期或学季的过程中逐渐掌握了要领。本书最初出版（2013年，第二版2021年）

有计算机实验室用R语言编写。自那时起，对统计学习中重要技术的Python实现需求日益增长。因此，本版本包含了Python的实验室。可用的Python包数量迅速增加，通过检查每个实验室开头的导入内容，读者会看到我们精心挑选并使用了最合适的包。我们还在我们的包ISLP中提供了一些额外的代码和功能。然而，ISL中的实验室是自包含的，如果读者希望使用不同的软件包或不想将讨论的方法应用于实际问题，可以选择跳过。

谁应该阅读这本书?

本书面向任何有兴趣使用现代统计方法进行数据建模和预测的人。这一群体包括科学家、工程师、数据分析师、数据科学家和量化分析师，但也包括拥有社会科学或商业等非定量领域学位的技术背景较弱的个人。我们期望读者至少学过一门基础统计学课程。线性回归方面的背景知识也很有用，但并非必需，因为我们会在第3章回顾线性回归背后的关键概念。本书的数学水平适中，不需要具备矩阵运算的详细知识。本书介绍了Python。此前接触过编程语言（如MATLAB或R）会有所帮助，但并非必需。

本教材第一版已被用于教授商学、经济学、计算机科学、生物学、地球科学、心理学以及物理与社会科学诸多领域的研究生。它也被用于教授已修读过线性回归课程的资深本科生。在将ESL作为主要教材、数学上更为严谨的课程中，ISL可作为辅助教材，用于讲授各种方法的计算方面。

符号表示法与简单矩阵代数

为教科书选择符号总是一项艰巨的任务。我们大体上采用与ESL相同的符号约定。

我们将使用 n 来表示样本中不同数据点或观测值的数量。我们将让 p 表示可用于进行预测的变量数量。例如，`Wage` 数据集由针对 3,000 人的 11 个变量组成，因此我们有 $n = 3{,}000$ 个观测值和 $p =$ 个变量（例如 `year`、`age`、`race` 等）。请注意，在本书中，我们用彩色字体表示变量名：`Variable Name`。

在一些示例中，p 可能相当大，比如达到数千甚至数百万的数量级；这种情况经常出现，例如在现代生物数据分析或基于网络的广告数据分析中。

一般来说，我们将令x_{ij}表示第i次观测中第j个变量的值，其中$i = 1, 2, \ldots, n$ 且$j = 1, 2, \ldots, p$。在本书中，i将用于索引样本或观测（从1到n），而j将用于索引变量（从1到p）。我们令$\mathbf{X}$表示一个$n \times p$矩阵，其第（i, j）个元素为x_{ij}。即，

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \ldots & x_{1p} \\ x_{21} & x_{22} & \ldots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \ldots & x_{np} \end{pmatrix}.$$

对于不熟悉矩阵的读者，将$\mathbf{X}$可视化为一个包含n行和p列的数字电子表格是很有用的。

有时我们会关注 $\mathbf{X}$ 的行，记作 $x_1, x_2, \ldots, x_n$。这里 x_i 是一个长度为 p 的向量，包含第 i 个观测的 p 个变量测量值。即，

$$x_i = \begin{pmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{ip} \end{pmatrix}. \tag{1.1}$$

(向量默认表示为列。) 例如，对于 `Wage` 数据，x_i 是一个长度为 11 的向量，由 `year`、`age`、`race` 以及第 i 个个体的其他值组成。在其他时候，我们反而会关注 $\mathbf{X}$ 的列，我们将其写作 $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_p$。每个都是长度为 n 的向量。即，

$$\mathbf{x}_j = \begin{pmatrix} x_{1j} \\ x_{2j} \\ \vdots \\ x_{nj} \end{pmatrix}.$$

F例如，对于 `Wage` 数据，$\mathbf{x}_1$ 包含了 $n =$ 的 3,000 个值，用于 `year`。使用这种表示法，矩阵 $\mathbf{X}$ 可以写为

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_p \end{pmatrix},$$

或

$$\mathbf{X} = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{pmatrix}.$$

符号 ^{T} 表示矩阵或向量的 *transpose*。例如，

$$\mathbf{X}^T = \begin{pmatrix} x_{11} & x_{21} & \ldots & x_{n1} \\ x_{12} & x_{22} & \ldots & x_{n2} \\ \vdots & \vdots & & \vdots \\ x_{1p} & x_{2p} & \ldots & x_{np} \end{pmatrix},$$

当

$$x_i^T = \begin{pmatrix} x_{i1} & x_{i2} & \cdots & x_{ip} \end{pmatrix}.$$

我们使用y_i表示我们希望对其进行预测的变量的第i次观测，例如wage。因此，我们将所有n次观测的集合写成向量形式为

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}.$$

那么我们的观测数据由 $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ 组成，其中每个 x_i 是一个长度为 p 的向量。（如果 $p =$ 为 1，那么 x_i 就是一个标量。）

在本文中，长度为n的向量将始终用*lower case bold*表示；例如

$$\mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}.$$

然而，长度不为n (的向量，例如长度为p的特征向量，如(1.1))所示，将用*lower case normal font*表示，例如a。标量也将用*lower case normal font*表示，例如a。在极少数情况下，当小写正体字体的这两种用途导致歧义时，我们将澄清所指的用途。矩阵将使用*bold capitals*表示，例如$\mathbf{A}$。随机变量将使用*capital normal font*表示，例如A，无论其维度如何。

偶尔我们会想要指明某个特定对象的维度。为了表明一个对象是标量，我们将使用符号 $a \in \mathbb{R}$。为了表明它是一个长度为 k 的向量，我们将使用 $a \in \mathbb{R}^k$ (，或者如果其长度为 n)，则使用 $\mathbf{a} \in \mathbb{R}^n$。我们将使用 $\mathbf{A} \in \mathbb{R}^{r \times s}$ 来表示一个对象是一个 $r \times s$ 矩阵。

我们尽可能避免了使用矩阵代数。然而，在少数情况下，完全避免它会变得过于繁琐。在这些罕见的情况下，理解两个矩阵相乘的概念很重要。假设$\mathbf{A} \in \mathbb{R}^{r \times d}$和$\mathbf{B} \in \mathbb{R}^{d \times s}$。那么$\mathbf{A}$和$\mathbf{B}$的乘积记为$\mathbf{AB}$。$\mathbf{AB}$的第$(i, j)$个元素通过将$\mathbf{A}$的第$i$行的每个元素与$\mathbf{B}$的第$j$列的对应元素相乘来计算。即，$(\mathbf{AB})_{ij} = \sum_{k=1}^{d} a_{ik} b_{kj}$。作为一个例子，考虑

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

然后

$$\mathbf{AB} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 1 \times 5 + 2 \times 7 & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

请注意，此操作生成一个 $r \times s$ 矩阵。仅当 $\mathbf{A}$ 的列数与 $\mathbf{B}$ 的行数相同时，才能计算 $\mathbf{AB}$。

本书的组织结构

第2章介绍统计学习背后的基本术语和概念。本章还介绍了*K-nearest neighbor*分类器，这是一种非常简单的方法，在许多问题上却出奇地有效。第3章和第4章涵盖经典的回归和分类线性方法。特别地，第3章回顾了*linear regression*，这是所有回归方法的根本出发点。在第4章中，我们讨论两个最重要的经典分类方法，*logistic regression*和*linear discriminant analysis*。

在所有统计学习情境中，一个核心问题涉及为特定应用选择最佳方法。因此，在第5章中，我们介绍*cross-validation*和*bootstrap*，它们可用于估计多种不同方法的准确性，以便选择最佳方法。

近期统计学习研究大多集中于非线性方法。然而，线性方法在可解释性乃至准确性上往往优于其非线性竞争对手。因此，在第六章中我们探讨了一系列线性方法，既有经典也有更现代的，它们相较于标准线性回归提供了潜在的改进。这些包括*stepwise selection*、*ridge regression*、*principal components regression*以及*lasso*。

剩余章节进入非线性统计学习的领域。我们首先在第7章介绍一些对单输入变量问题效果良好的非线性方法。然后我们展示如何将这些方法用于拟合具有多个输入的非线性 *additive* 模型。在第8章中，我们研究基于 *tree* 的方法，包括 *bagging*、*boosting* 和 *random forests*。第9章讨论了 *Support vector machines*，这是一组用于执行线性和非线性分类的方法。我们在第10章涵盖 *deep learning*，这是一种近年来受到广泛关注的非线性回归和分类方法。第11章探讨 *survival analysis*，这是一种专门适用于输出变量为 *censored*（即未被完全观测）情形的回归方法。

在第12章中，我们探讨了*unsupervised*设定，其中包含输入变量但无输出变量。特别地，我们介绍了*princi-pal components analysis*、*K-means clustering*和*hierarchical clustering*。最后，在第13章，我们涵盖了多重假设检验这一极其重要的主题。

在每章末尾，我们会呈现一个或多个`Python`实验室环节，在这些环节中，我们系统地探讨该章讨论的各种方法的应用。这些实验室展示了不同方法的优势与不足，并为实现各种方法所需的语法提供了实用参考。读者可以根据自己的进度完成实验，或者在课堂环境中将实验作为小组讨论的重点。在每个`Python`实验中，我们展示了编写本书时执行实验所得到的结果。然而，新版本的`Python`不断发布，随着时间推移，实验中调用的包将会更新。因此，未来可能会出现所示结果与

Name	Description
Auto	Gas mileage, horsepower, and other information for cars.
Bikeshare	Hourly usage of a bike sharing program in Washington, DC.
Boston	Housing values and other information about Boston census tracts.
BrainCancer	Survival times for patients diagnosed with brain cancer.
Caravan	Information about individuals offered caravan insurance.
Carseats	Information about car seat sales in 400 stores.
College	Demographic characteristics, tuition, and more for USA colleges.
Credit	Information about credit card debt for 400 customers.
Default	Customer default records for a credit card company.
Fund	Returns of 2,000 hedge fund managers over 50 months.
Hitters	Records and salaries for baseball players.
Khan	Gene expression measurements for four cancer types.
NCI60	Gene expression measurements for 64 cancer cell lines.
NYSE	Returns, volatility, and volume for the New York Stock Exchange.
OJ	Sales information for Citrus Hill and Minute Maid orange juice.
Portfolio	Past values of financial assets, for use in portfolio allocation.
Publication	Time to publication for 244 clinical trials.
Smarket	Daily percentage returns for S&P 500 over a 5-year period.
USArrests	Crime statistics per 100,000 residents in 50 states of USA.
Wage	Income survey data for men in central Atlantic region of USA.
Weekly	1,089 weekly stock market returns for 21 years.

表 1.1. *A list of data sets needed to perform the labs and exercises in this textbook. All data sets are available in the* ISLP *package, with the exception of* USArrests*, which is part of the base* R *distribution, but accessible from* Python
。

实验部分可能不再完全对应于执行实验的读者所获得的结果。如有必要，我们将在书籍网站上发布实验室的更新。

我们使用符号来表示包含更具挑战性概念的部分或练习。这些内容可轻松跳过，适合不希望深入探究材料或缺乏数学背景的读者。

实验室和练习中使用的数据集

在本教材中，我们通过市场营销、金融、生物学等领域的应用来说明统计学习方法。ISLP 包包含了许多数据集，这些数据集是执行与本书相关的实验和练习所必需的。另一个数据集是基础 R 发行版的一部分（USArrests 数据），我们在第 12.5.1 节中展示如何从 Python 访问它。表 1.1 包含了执行实验和练习所需数据集的摘要。其中几个数据集也可在书籍网站上以文本文件形式获取，供第 2 章使用。

书籍网站

本书的网站位于

www.statlearning.com

它包含许多资源，包括与本书相关的Python包，以及一些额外的数据集。

致谢

本书中有少数几幅图取自ESL：图6.7、8.3和12.14。所有其他图都是为ISL的R版本制作的，除了图13.10因Python软件支持该图而有所不同。

2
统计学习



2.1 什么是统计学习?

为了激发我们对统计学习的研究，我们从一个简单的例子开始。假设我们是被客户聘请的统计顾问，目的是调查广告与特定产品销售量之间的关联。`Advertising`数据集包含了该产品在200个不同市场的`sales`，以及每个市场中针对三种不同媒体（`TV`、`radio`和`newspaper`）的广告预算。数据展示在图2.1中。我们的客户无法直接增加产品的销售量。另一方面，他们可以控制三种媒体各自的广告支出。因此，如果我们确定了广告与销售量之间存在关联，就可以指导客户调整广告预算，从而间接提高销售量。换言之，我们的目标是开发一个准确的模型，能够基于这三种媒体预算来预测销售量。

在此设置中，广告预算为*input variables*，而`sales`是*output variable*。输入变量通常使用符号 X，用下标来区分它们。因此 X_1 可能是 `TV` 预算，X_2 是 `radio` 预算，而 X_3 是 `newspaper` 预算。输入有不同的名称，例如 *predictors*、*independent variables*、*features*，或者有时只是*variables*。输出变量——在本例中是`sales`——是通常称为 *response* 或 *dependent variable*，并且通常表示为使用符号Y。在本书中，我们将互换使用所有这些术语。

输入变量
输出变量
预测变量 自变量 特征
变量
响应因变量

更一般地，假设我们观察到一个定量响应Y和p个不同的预测变量 $X_1, X_2, \ldots, X_p$。我们假设Y和$X = (X_1, X_2, \ldots, X_p)$之间存在某种关系，可以用非常一般的形式写成

$$Y = f(X) + \epsilon. \tag{2.1}$$

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_2

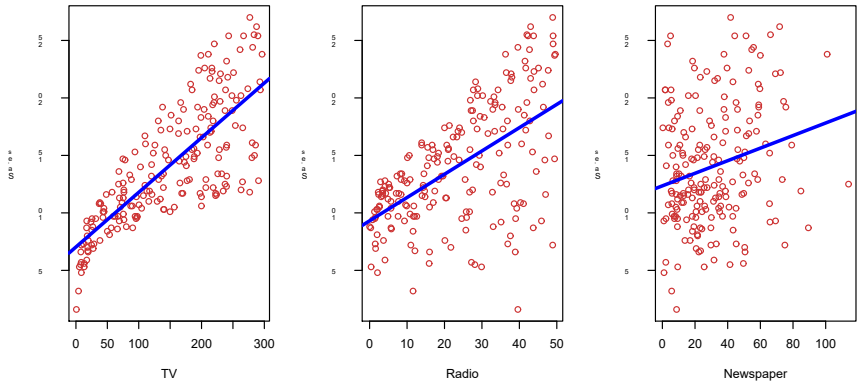


图2.1。*The* `Advertising` *data set. The plot displays* `sales`*, in thousands of units, as a function of* `TV`*,* `radio`*, and* `newspaper` *budgets, in thousands of dollars, for* 200 *different markets. In each plot we show the simple least squares fit of* `sales` *to that variable, as described in Chapter 3. In other words, each blue line represents a simple model that can be used to predict* `sales` *using* `TV`*,* `radio`*, and* `newspaper`*, respectively.*

这里 f 是 $X_1, \ldots, X_p$ 的某个固定但未知的函数，而 ϵ 是一个随机的 *error term*，它独立于 X 并且均值为零。在这个公式中——，f 表示 X 提供的关于 Y 的 *systematic* 信息。

error term

systematic

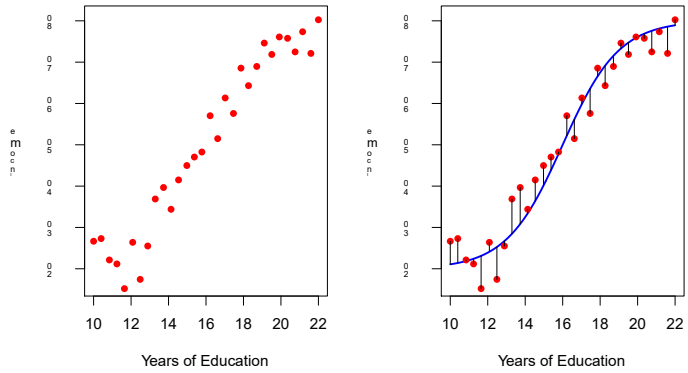


图2.2。*The* `Income` *data set. 左：The red dots are the observed values of* `income` *(in thousands of dollars) and* `years of education` *for* 30 *individuals. 右：The blue curve represents the true underlying relationship between* `income` *and* `years of education`*, which is generally unknown (but is known in this case because the data were simulated). The black lines represent the error associated with each observation. Note that some errors are positive (if an observation lies above the blue curve) and some are negative (if an observation lies below the curve). Overall, these errors have approximately mean zero.*

再举一个例子，考虑图2.2左侧的面板，该面板展示了`Income`数据集中30名个体的`income`与`years of education`的关系图。该图表明，或许可以利用`years of education`来预测`income`。然而，将输入变量与

输出变量通常是未知的。在这种情况下，必须基于观测点来估计f。由于Income是模拟数据集，f是已知的，并在图2.2右侧面板中用蓝色曲线显示。垂直线代表误差项ϵ。我们注意到，30个观测值中有些位于蓝色曲线上方，有些位于其下方；总体而言，误差的均值近似为零。

一般来说，函数 f 可能包含多个输入变量。在图2.3中，我们将 income 表示为 years of education 和 seniority 的函数。此处 f 是一个二维曲面，必须基于观测数据进行估计。

本质上，统计学习是指一系列用于估计f的方法。在本章中，我们将概述估计f时出现的一些关键理论概念，以及评估所获得估计的工具。

2.1.1 Why Estimate f?

我们希望估计 f 主要有两个原因：*prediction* 和 *inference*。我们依次讨论每一个。

预测

在许多情况下，一组输入 X 是现成可用的，但输出 Y 却不易获得。在这种设定下，由于误差项平均为零，我们可以使用以下公式预测 Y:

$$\hat{Y} = \hat{f}(X), \tag{2.2}$$

其中 $\hat{f}$ 表示我们对 f 的估计，而 $\hat{Y}$ 表示对 Y 的预测结果。在此设定下，$\hat{f}$ 常被视为 *black box*，因为人们通常并不关心 $\hat{f}$ 的确切形式，只要它能对 Y 给出准确的预测即可。

例如，假设$X_1, \ldots, X_p$是患者血液样本的特征，这些特征可以在实验室中轻松测量，而Y是编码患者对特定药物严重不良反应风险的变量。自然会寻求使用X来预测Y，因为这样我们就可以避免给有高不良反应风险的患者使用该药物——即那些Y估计值高的患者。

$\hat{Y}$作为Y的预测的准确性取决于两个量，我们将称之为*reducible error*和*irreducible error*。一般来说，
$\hat{f}$ 将不会是 f 的完美估计，这种不准确性会引入一些误差。这个误差是 *reducible*，因为我们可以通过使用最合适的统计学习技术来估计 f，从而有可能提高 $\hat{f}$ 的准确性。然而，即使能够为 f 形成一个完美的估计，使得我们的估计响应呈现为 $\hat{Y} = f(X)$ 的形式，我们的预测仍然会有一些误差！这是因为 Y 也是 ϵ 的函数，而根据定义，ϵ 无法使用 X 来预测。因此，与 ϵ 相关的变异性也会影响我们预测的准确性。这被称为 *irreducible* 误差，因为无论我们对 f 的估计有多好，我们都无法减少由 ϵ 引入的误差。

reducible error
irreducible error

为什么不可约误差大于零？量ϵ可能包含有助于预测Y的未测量变量：因为我们不

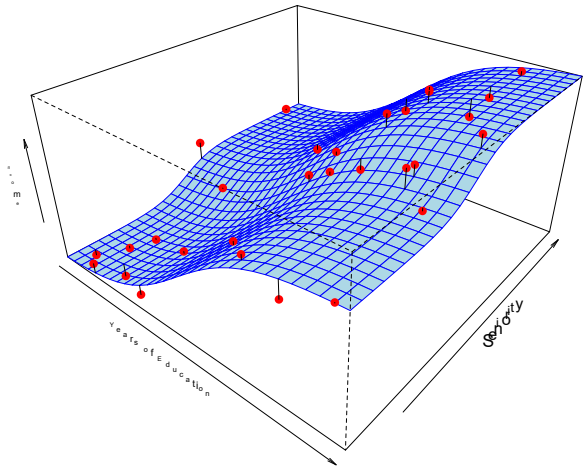


图2.3. *The plot displays* `income` *as a function of* `years of education` *and* `seniority` *in the* `Income` *data set. The blue surface represents the true underlying relationship between* `income` *and* `years of education` *and* `seniority`*, which is known since the data are simulated. The red dots indicate the observed values of these quantities for* 30 *individuals.*

测量它们，f不能将其用于预测。量ϵ也可能包含不可测量的变异。例如，对于某一天的一个给定患者，不良反应的风险可能因药物本身的制造变异或患者当天的总体健康状况而异。

考虑给定的估计值$\hat{f}$ 和一组预测因子 X，这会产生预测$\hat{Y} = \hat{f}(X)$。暂时假设$\hat{f}$ 和 X 都是固定的，因此唯一的变化来自于 ϵ。那么，很容易证明

$$\begin{aligned} \mathrm{E}(Y - \hat{Y})^2 &= \mathrm{E}[f(X) + \epsilon - \hat{f}(X)]^2 \\ &= \underbrace{[f(X) - \hat{f}(X)]^2}_{\text{Reducible}} + \underbrace{\mathrm{Var}(\epsilon)}_{\text{Irreducible}}, \end{aligned} \tag{2.3}$$

其中 $\mathrm{E}(Y - \hat{Y})^2$ 表示平方的平均值，即 *expected value* Y的预测值与实际值之间的差异，而 $\mathrm{Var}(\epsilon)$ 表示与误差项 ϵ 相关的 *variance*。

expected value

variance

本书的重点在于估计 f 的技术，旨在最小化可减少的误差。重要的是要记住，不可减少的误差将始终为我们对 Y的预测准确性提供一个上限。这个界限在实践中几乎总是未知的。

推理

我们通常感兴趣的是理解Y与$X_1, \ldots, X_p$之间的关联。在这种情况下，我们希望估计f，但我们的目标不一定是对Y进行预测。此时，$\hat{f}$ 不能被当作黑盒，因为我们需要知道其确切形式。在这一设定下，人们可能对回答以下问题感兴趣：

- *Which predictors are associated with the response?* 通常情况是，只有一小部分可用的预测变量与Y有显著关联。根据应用的不同，从大量可能的变量中识别出少数几个*important*预测变量极为有用。

- *What is the relationship between the response and each predictor?* 一些预测变量可能与Y呈正相关关系，即预测变量的较大值对应于Y的较大值。其他预测变量可能具有相反的关系。根据f的复杂性，响应变量与给定预测变量之间的关系也可能取决于其他预测变量的值。

- *Can the relationship between Y and each predictor be adequately summarized using a linear equation, or is the relationship more complicated?* 历史上，大多数估算f的方法都采用线性形式。在某些情况下，这样的假设是合理的，甚至是可取的。但往往真实关系更为复杂，此时线性模型可能无法准确表示输入与输出变量之间的关系。

在本书中，我们将看到许多属于预测设置、推断设置或两者结合的例子。

例如，考虑一家有兴趣开展直接营销活动的公司。其目标是根据对每个人测量的人口统计变量的观察，识别可能对邮件作出积极回应的人。在这种情况下，人口统计变量作为预测变量，而对营销活动的回应（无论是积极还是消极）作为结果。该公司并不想深入理解每个单独预测变量与回应之间的关系；相反，该公司只是想使用这些预测变量准确预测回应。这是为预测而建模的一个例子。

相反，考虑图2.1中所示的`Advertising`数据。人们可能对回答以下问题感兴趣：

– *Which media are associated with sales?*

– *Which media generate the biggest boost in sales?* 或

– *How large of an increase in sales is associated with a given increase in TV advertising?*

这种情况属于推断范式。另一个例子涉及根据价格、商店位置、折扣水平、竞争价格等变量，对客户可能购买的产品的品牌进行建模。在这种情况下，人们可能真正最感兴趣的是每个变量与购买概率之间的关联。例如，*to what extent is the product's price associated with sales?* 这是为推断而建模的一个例子。

最后，一些建模可以同时用于预测和推断。例如，在房地产环境中，人们可能试图将价值

房屋对犯罪率、 zoning、距河流的距离、空气质量、学校、社区收入水平、房屋面积等输入因素的反应。在这种情况下，人们可能对每个单独的输入变量与房价之间的关联感兴趣——例如，*how much extra will a house be worth if it has a view of the river?* 这是一个推断问题。或者，人们可能只是对根据房屋的特征来预测其价值感兴趣：*is this house under- or over-valued?* 这是一个预测问题。

根据我们的最终目标是预测、推断还是两者的结合，估计*f*的不同方法可能是合适的。例如，*linear models*允许相对简单和

linear model

可解释的推理，但可能不如其他一些方法得出的预测准确。相比之下，我们在本书后续章节中讨论的一些高度非线性方法或许能为*Y*提供相当准确的预测，但这是以模型可解释性较差、推理更具挑战性为代价的。

2.1.2 How Do We Estimate f?

在本书中，我们探索了许多线性和非线性方法来估计*f*。然而，这些方法通常具有某些共同特征。我们在本节中概述这些共同特征。我们将始终假设我们观察到了一组n不同的数据点。例如，在图2.2中，我们观察到了n =30个数据点。这些观察值被称为*training data*，因为我们将使用这些

训练数据

观测数据用于训练或教导我们的方法如何估计 f。令 x_{ij} 表示第 j 个预测变量或输入在第 i 个观测中的值，其中 $i = 1, 2, \ldots, n$ 和 $j = 1, 2, \ldots, p$。相应地，令 y_i 表示第 i 个观测的响应变量。那么我们的训练数据由 $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ 组成，其中 $x_i = (x_{i1}, x_{i2}, \ldots, x_{ip})^T$。

我们的目标是应用统计学习方法对训练数据进行处理，以估计未知函数*f*。换句话说，我们想要找到一个函数$\hat{f}$，使得对于任何观测(X, Y)，都有 $Y \approx \hat{f}(X)$。广义上讲，用于此任务的大多数统计学习方法可以归类为*parametric*或*non-parametric*。我们现在简要讨论这些

参数 非参数

两种方法。

参数方法

P参数方法涉及一种基于模型的两步逼近法 oach。

1. 首先，我们对*f*的函数形式或形状做出假设。例如，一个非常简单的假设是*f*关于*X*是线性的：

$$f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p. \tag{2.4}$$

这是一个 *linear model*，这将在第3章中进行广泛讨论。一旦我们假设 f 是线性的，估计 f 的问题就大大简化了。不再需要估计一个完全任意的 p 维函数 $f(X)$，而只需要估计 $p + 1$ 个系数 $\beta_0, \beta_1, \ldots, \beta_p$。

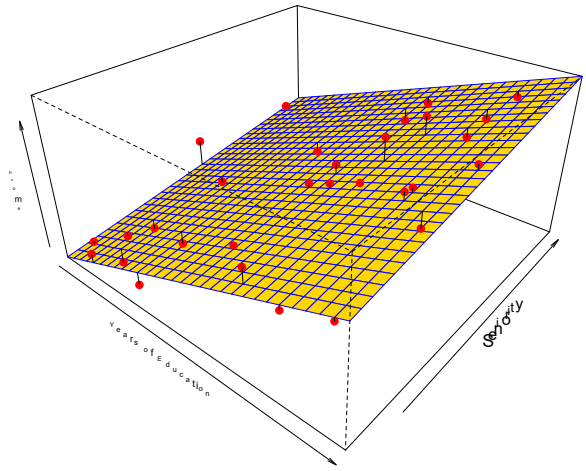


图2.4。*A linear model fit by least squares to the* `Income` *data from Figure 2.3. The observations are shown in red, and the yellow plane indicates the least squares fit to the data.*

2. 选择模型后，我们需要一个使用训练数据来*fit*或*train*该模型的过程。在线性模型fi的情况下
(2.4)，我们需要估计参数 $\beta_0, \beta_1, \ldots, \beta_p$。也就是说，我们希望找到这些参数的值，使得 t t雨

$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p.$$

拟合模型(2.4)最常用的方法被称为*(ordinary) least squares*，我们将在第3章讨论。然而，
最小二乘法是拟合线性模型的众多可能方法之一。在第6章中，我们将讨论估计(2.4)中参数的其他方法。 最小二乘法

所述的基于模型的方法被称为*parametric*；它将估计*f*的问题简化为估计一组参数的问题。假设*f*具有参数形式简化了估计*f*的问题，因为通常估计一组参数（例如线性模型(2.4)中的$\beta_0, \beta_1, \ldots, \beta_p$）比拟合一个完全任意的函数*f*要容易得多。参数方法的一个潜在缺点是，我们选择的模型通常不会匹配*f*的真实未知形式。如果所选模型与真实的*f*相差太远，那么我们的估计将会很差。我们可以尝试通过选择能够拟合许多不同可能函数形式的*flexible*模型来解决这个问题。

对于*f*。但一般来说，拟合更灵活的模型需要估计更多的参数。这些更复杂的模型可能导致一种称为*overfitting*数据的现象，这实质上意味着它们 灵活

过于关注错误或*noise*。这些问题通过—— overfitting
读完这本书。 noise

图2.4展示了将参数方法应用于图2.3中`Income`数据的一个示例。我们拟合了一个形式为的线性模型

$$\texttt{income} \approx \beta_0 + \beta_1 \times \texttt{education} + \beta_2 \times \texttt{seniority}.$$

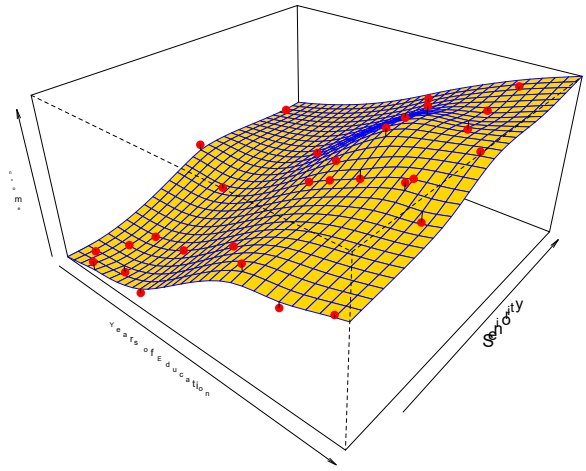


图 2.5. *A smooth thin-plate spline fit to the* `Income` *data from Figure 2.3 is shown in yellow; the observations are displayed in red. Splines are discussed in Chapter 7.*

由于我们已经假设响应与两个预测变量之间存在线性关系，整个拟合问题就简化为估计β_0、β_1和β_2，我们使用最小二乘线性回归来完成这一估计。比较图2.3和图2.4，我们可以看到图2.4中给出的线性拟合并不完全正确：真实的f具有一些线性拟合未能捕捉到的曲率。然而，线性拟合在捕捉`years of education`与`income`之间的正相关关系，以及`seniority`与`income`之间略微较弱正相关关系方面，似乎仍然表现合理。可能由于观测数量如此之少，这已经是我们能做到的最好结果了。

非参数方法

非参数方法不对 f 的函数形式做出明确假设。相反，它们寻求对 f 的估计，使其尽可能接近数据点，同时不过于粗糙或波动。这类方法相比参数方法具有显著优势：通过避免对 f 的特定函数形式做出假设，它们有可能准确拟合 f 的更广泛可能形状。任何参数方法都伴随着这样一种可能性：用于估计 f 的函数形式与真实的 f 大不相同，在这种情况下，所得模型将无法很好地拟合数据。相比之下，非参数方法完全避免了这一风险，因为基本上没有对 f 的形式做出假设。但非参数方法确实存在一个主要缺点：由于它们没有将估计 f 的问题简化为少量参数，因此需要非常大量的观测值（远多于参数方法通常所需的数量）才能获得对 f 的准确估计。

图2.5展示了拟合`Income`数据的非参数方法的一个示例。使用*thin-plate spline*来估计f。这种方该方法并未对f施加任何预先设定的模型。它转而尝试

thin-plate spline

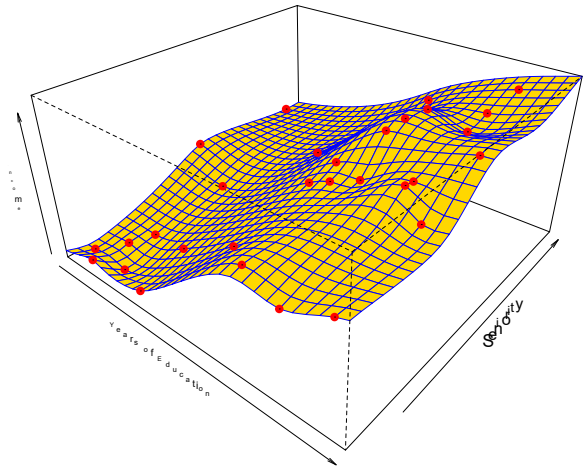


图2.6。*A rough thin-plate spline fit to the* `Income` *data from Figure 2.3. This fit makes zero errors on the training data.*

为了生成对 f 的估计，使其尽可能接近观测数据，且受限于拟合——即图2.5中的黄色曲面——为 *smooth*。在这种情况下，非参数拟合产生了对图2.3所示真实 f 的极为准确的估计。为了拟合薄板样条，数据分析师必须选择平滑程度。图2.6展示了使用较低平滑程度的相同薄板样条拟合，允许更粗糙的拟合。由此得到的估计完美地拟合了观测数据！然而，图2.6所示的样条拟合比图2.3中的真实函数 f 变化要大得多。这是过拟合数据的一个例子，我们之前已经讨论过。这是一种不理想的情况，因为所得到的拟合将无法对不属于原始训练数据集的新观测值产生准确的响应估计。我们在第5章讨论选择 *correct* 平滑程度的方法。样条在第7章讨论。

正如我们所见，参数方法和非参数方法在统计学习中各有优缺点。本书将全面探讨这两种类型的方法。

2.1.3 The Trade-Off Between Prediction Accuracy and Model Interpretability

在本书中我们考察的众多方法中，有些方法灵活性较低，或者限制性更强，因为它们只能产生相对较小范围的形状来估计 f。例如，线性回归是一种相对不灵活的方法，因为它只能生成线性函数，如图2.1所示的直线或图2.4所示的平面。其他方法，如图2.5和图2.6所示的薄板样条，则灵活性要大得多，因为它们可以生成范围广泛得多的可能形状来估计 f。

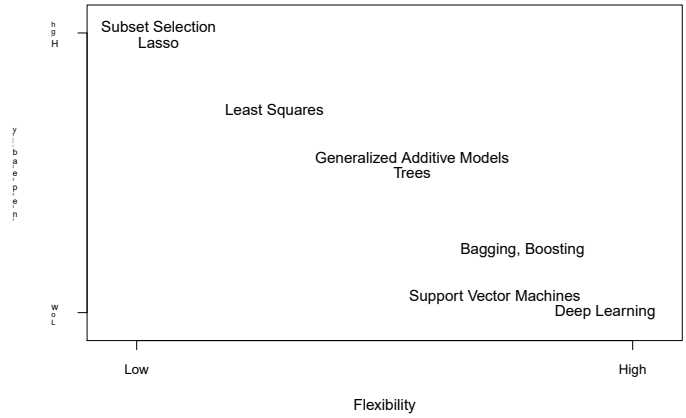


图2.7。 *A representation of the tradeoff between flexibility and interpretability, using different statistical learning methods. In general, as the flexibility of a method increases, its interpretability decreases.*

人们可能会合理地提出以下问题：*why would we ever choose to use a more restrictive method instead of a very flexible approach?* 我们可能更倾向于限制性更强的模型有若干原因。如果我们主要关注的是推断，那么限制性更强的模型更具可解释性。例如，当推断是目标时，线性模型可能是一个好的选择，因为理解 Y 与 $X_1, X_2, \ldots, X_p$ 之间的关系会相当容易。相比之下，非常灵活的方法，例如第7章讨论并在图2.5和图2.6中展示的样条，以及第8章讨论的提升方法，可能会导致对 f 的估计过于复杂，以至于难以理解任何单个预测变量与响应之间的关系。

图 2.7 展示了本书涵盖的一些方法在灵活性和可解释性之间的权衡。第 3 章讨论的最小二乘线性回归相对不灵活，但可解释性很强。第 6 章讨论的 *lasso* 依赖于

lasso

线性模型(2.4)，但采用了另一种拟合方法来估计系数$\beta_0, \beta_1, \ldots, \beta_p$。新方法在估计系数时更为严格，并将许多系数精确设置为零。因此，从这个意义上说，套索方法比线性回归的灵活性更低。同时，它也比线性回归更具可解释性，因为在最终模型中，响应变量仅与预测变量的一个小子集相关——即那些系数估计非零的预测变量。*Generalizedadditive models*（广义加性模型(GAMs)）则在第7章讨论，它们扩展了线

广义加性模型

耳模型（2.4）以允许某些非线性关系。因此，GAMs比线性回归更灵活。它们也比线性回归稍微不那么易于解释，因为现在每个预测变量与响应之间的关系是用曲线建模的。最后，完全非线性的方法如*bagging*、*boosting*、*support vector machines*

Bagging
Boosting

使用非线性核和*neural networks*（深度学习），讨论于第8章、第9章和第10章是高度灵活的方法，较难解释。

支持向量机

我们已经确立，当推断是目标时，使用简单且相对不灵活的统计学习方法具有明显优势。然而，在某些情况下，我们仅对预测感兴趣，而预测模型的可解释性并不重要。例如，如果我们旨在开发一种预测股票价格的算法，我们对该算法的唯一要求是预测准确——可解释性并非考虑因素。在这种情况下，我们可能会预期使用最灵活的模型将是最佳选择。令人惊讶的是，事实并非总是如此！我们常常通过使用较不灵活的方法获得更准确的预测。这一乍看之下可能显得反直觉的现象，与高度灵活方法中潜在的过拟合倾向有关。我们在图2.6中看到了过拟合的一个例子。我们将在第2.2节及本书后续部分进一步讨论这一非常重要的概念。

2.1.4 Supervised Versus Unsupervised Learning

大多数统计学习问题都属于以下两类之一：*supervised* 或 *unsupervised*。到目前为止，我们在本章中讨论的例子——都落入监督学习领域。对于预测变量测量值 x_i、$i = 1, \ldots, n$ 的每个观测，都有一个相关的响应测量值 y_i。我们希望对一个将响应与预测变量联系起来的模型进行拟合，目的是准确预测未来观测的响应（预测），或更好地理解响应与预测变量之间的关系（推断）。许多经典统计学习方法，如线性回归和 *logistic regression* (第4章)，作为

监督式
无监督

逻辑回归

以及更现代的方法，如GAM、提升法和支持向量机，都运作于监督学习领域。本书的绝大部分内容都致力于这一设定。

相比之下，无监督学习描述了一种更具挑战性的情况，即对于每个观测 $i = 1, \ldots, n$，我们观察到一个测量向量 x_i，但没有相关的响应 y_i。由于没有响应变量可预测，因此无法拟合线性回归模型。在这种情况下，我们在某种意义上是盲目的；这种情况被称为 *unsupervised*，因为我们缺乏一个可以监督分析的响应变量。那么，哪种统计分析是可能的呢？我们可以尝试理解变量之间或观测之间的关系。在这种情况下，我们可以使用的一种统计学习工具是 *cluster analysis*，或称聚类。聚类分析的目标

聚类分析

是在$x_1, \ldots, x_n$的基础上确定观测值是否属于相对不同的组。例如，在市场细分研究中，我们可能会观察到潜在客户的多个特征（变量），如邮政编码、家庭收入和购物习惯。我们可能认为客户分为不同的组，如高消费群体与低消费群体。如果每个客户消费模式的信息可用，那么监督分析将是可能的。然而，这些信息并不可用——也就是说，我们不知道每个潜在客户是否是高消费群体。在这种情况下，我们可以尝试根据测量的变量对客户进行聚类，以识别

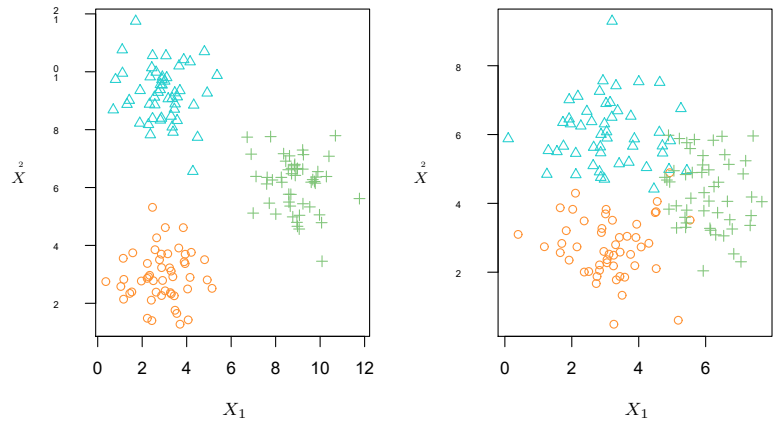


图2.8. *A clustering data set involving three groups. Each group is shown using a different colored symbol.* 左： *The three groups are well-separated. In this setting, a clustering approach should successfully identify the three groups.* 右： *There is some overlap among the groups. Now the clustering task is more challenging.*

不同的潜在客户群体。识别这些群体可能很有意义，因为它们在诸如消费习惯等某些感兴趣的属性方面可能存在差异。

图2.8提供了一个关于聚类问题的简单说明。我们绘制了150个观测值，这些观测值在两个变量X_1和X_2上进行了测量。每个观测值对应三个不同组别中的一个。为了说明目的，我们使用不同的颜色和符号来绘制每个组的成员。然而，在实践中，组别归属是未知的，目标是确定每个观测值所属的组。在图2.8的左侧面板中，这是一项相对容易的任务，因为各组分离得很好。相比之下，右侧面板展示了一个更具挑战性的情况，其中组与组之间存在一些重叠。不能期望聚类方法将所有重叠的点分配到它们正确的组（蓝色、绿色或橙色）。

在图2.8所示的例子中，只有两个变量，因此只需通过直观检查观测值的散点图即可识别聚类。然而，在实际应用中，我们经常遇到包含远多于两个变量的数据集。在这种情况下，我们无法轻易地绘制观测值。例如，如果数据集中有p个变量，那么可以制作$p(p-1)/2$个不同的散点图，而视觉检查根本不是一个可行的识别聚类的方法。因此，自动聚类方法显得尤为重要。我们将在第12章讨论聚类及其他无监督学习方法。

许多问题自然归属于监督学习或无监督学习范式。然而，有时一个分析应被视为监督还是无监督的问题并不那么明确。例如，假设我们有一组 n 观测值。对于其中的 m 个观测值，其中 $m < n$，我们既有预测变量测量值又有响应

测量。对于剩余的 $n - m$ 个观测，我们有预测变量测量值，但没有响应测量值。如果预测变量可以相对便宜地测量，而相应的响应收集起来要昂贵得多，就会出现这种情况。我们将这种设定称为 *semi-supervised learning* 问题。在这种设定下，我们希望使用一个 sta-

半监督学习

一种统计学习方法，它可以结合那些有响应测量值的 m 观察结果以及那些没有响应测量值的 $n - m$ 观察结果。尽管这是一个有趣的话题，但它超出了本书的范围。

2.1.5 Regression Versus Classification Problems

变量可以被描述为 *quantitative* 或 *qualitative*（也称为*categorical*）。定量变量取数值。例-

定量 定性

例如，一个人的年龄、身高或收入，房屋的价值，以及股票的价格。相反，定性变量取值于 K 个不同的 *classes* 或类别之一。定性变量的例子

类别

包括一个人的婚姻状况（已婚或未婚）、购买产品的品牌（品牌A、B或C）、一个人是否拖欠债务（是或否），或癌症诊断（急性髓性白血病、急性淋巴细胞白血病或无白血病）。我们倾向于将具有定量响应的问题称为*regression*问题，而那些涉及

类

定性响应通常被称为*classification*问题。如何——

回归

然而，这一区分并不总是那么清晰。最小二乘线性回归（第3章）适用于定量响应变量，而逻辑回归（第4章）通常用于定性（二分类，或*binary*）响

分类

响应。因此，尽管名为逻辑回归，它实际上是一种分类方法。但由于它估计的是类别概率，也可以被视为一种回归方法。一些统计方法，如 K 最近邻（第2章和第4章）和提升法（第8章），可用于定量或定性响应的情况。

二进制

我们倾向于根据响应变量是定量还是定性来选择统计学习方法；即当响应变量为定量时，我们可能使用线性回归，而当其为定性时，则使用逻辑回归。然而，预测变量*predictors*是定性还是定量通常被认为不那么重要。本书中讨论的大多数统计学习方法，只要在进行分析前对任何定性预测变量进行适当的*coded*，就可以应用于任何类型的预测变量。这一点在第3章中讨论。

2.2 评估模型准确度

本书的关键目标之一是向读者介绍广泛的统计学习方法，这些方法远远超出了标准的线性回归方法。为什么有必要引入如此多不同的统计学习方法，而不仅仅是单一的*best*方法？*Thereis no free lunch in statistics:*没有任何一种方法能在所有可能的数据集上超越其他所有方法。在特定的数据集上，某一种特定的方法可能有效。

最好的方法，但在类似但不同的数据集上，其他方法可能效果更好。因此，对于任何给定的数据集，决定哪种方法产生最佳结果是一项重要任务。选择最佳方法可能是实践中进行统计学习最具挑战性的部分之一。

在本节中，我们将探讨针对特定数据集选择统计学习程序时出现的一些最重要概念。随着本书的深入，我们将解释如何将这些概念应用于实践中。

2.2.1 Measuring the Quality of Fit

为了在给定数据集上评估统计学习方法的性能，我们需要某种方式来衡量其预测与实际观测数据的匹配程度。也就是说，我们需要量化对于给定观测的预测响应值与其真实响应值接近的程度。在回归设置中，最常用的度量是*mean squared error* (MSE)，由下式给出：

均方误差

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{f}(x_i))^2, \tag{2.5}$$

其中$\hat{f}(x_i)$是$\hat{f}$对第i个观测值的预测。如果预测响应非常接近真实响应，MSE将很小；如果对于某些观测值，预测响应与真实响应差异很大，MSE将很大。

(2.5) 中的 MSE 是使用用于拟合模型的训练数据计算的，因此更准确地应称为 *training MSE*。但一般来说，我们并不真正关心该方法的效果有多好在训练数据上。相反，*we are interested in the accuracy of the predictions that we obtain when we apply our method to previously unseen test data*。为什么这是我们关心的？假设我们感兴趣在开发一种基于先前股票收益预测股票价格的算法时，我们可以利用过去6个月的股票收益来训练该方法。但我们真正关心的并非该方法对上周股票价格的预测准确度，而是它对明天或下个月价格的预测能力。类似地，假设我们拥有多位患者的临床测量数据（如体重、血压、身高、年龄、疾病家族史）以及每位患者是否患有糖尿病的相关信息。我们可以利用这些患者来训练一个统计学习方法，以根据临床测量预测糖尿病风险。实际上，我们希望这种方法能基于*futurepatients*的临床测量数据准确预测其糖尿病风险。我们对于该方法是否能准确预测用于训练模型的患者的糖尿病风险并不十分关注，因为我们已经知道这些患者中哪些患有糖尿病。

training MSE

test data

更数学地表述，假设我们在训练观测值$\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$上拟合我们的统计学习方法，并得到估计值$\hat{f}$。然后我们可以计算$\hat{f}(x_1), \hat{f}(x_2), \ldots, \hat{f}(x_n)$。

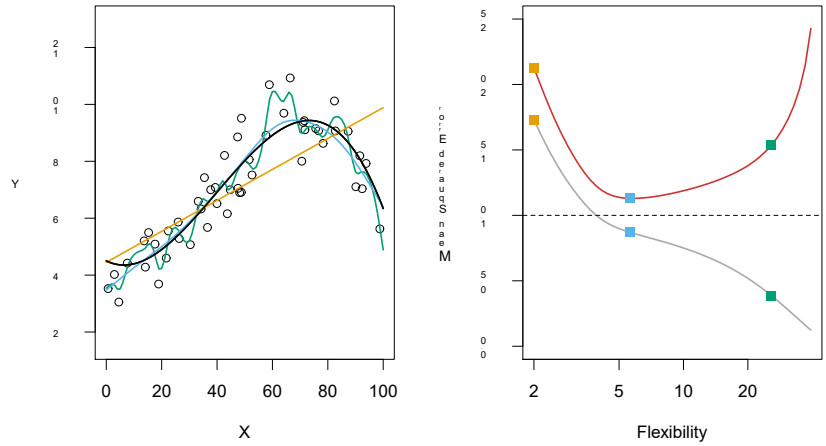


图 2.9. 左： *Data simulated from f, shown in black. Three estimates of f are shown: the linear regression line (orange curve), and two smoothing spline fits (blue and green curves).* 右： *Training MSE (grey curve), test MSE (red curve), and minimum possible test MSE over all methods (dashed line). Squares represent the training and test MSEs for the three fits shown in the left-hand panel.*

如果这些近似等于$y_1, y_2, \ldots, y_n$，那么由(2.5)给出的训练MSE很小。然而，我们实际上并不关心$\hat{f}(x_i) \approx y_i$是否成立；相反，我们想知道$\hat{f}(x_0)$是否近似等于y_0，其中(x_0, y_0)是一个
previously unseen test observation not used to train the statistical learning method。我们要选择使*test MSE*最低的方法，而不是训练MSE最低的方法。换句话说，

test MSE

如果我们有大量的测试观测值，我们可以计算

$$\text{Ave}(y_0 - \hat{f}(x_0))^2, \tag{2.6}$$

这些测试观测值（x_0, y_0）的平均平方预测误差。我们希望选择使该量尽可能小的模型。

我们如何着手尝试选择一种使测试MSE最小化的方法呢？在某些情况下，我们可能有可用的测试数据集——也就是说，我们可以获得一组未用于训练统计学习方法的观测数据。然后，我们只需在测试观测上评估(2.6)，并选择测试MSE最小的学习方法。但如果没有可用的测试观测呢？在这种情况下，人们可能会想象简单地选择一种使训练MSE (2.5)最小化的统计学习方法。这似乎可能是一种明智的方法，因为训练MSE和测试MSE似乎密切相关。不幸的是，这种策略存在一个根本问题：不能保证训练MSE最低的方法也会具有最低的测试MSE。粗略地说，问题在于许多统计方法专门估计系数以最小化训练集MSE。对于这些方法，训练集MSE可能相当小，但测试MSE通常要大得多。

图2.9通过一个简单的例子说明了这一现象。在图2.9的左侧面板中，我们根据(2.1)式生成了观测值，其中

由黑色曲线给出的真实 f。橙色、蓝色和绿色曲线展示了使用灵活性递增的方法得到的 f 的三种可能估计。橙色线是线性回归拟合，相对不灵活。蓝色和绿色曲线是使用第7章讨论的 *smoothing splines* 生成的，其平滑程度不同。这是

smoothing spline

很明显，随着灵活度的增加，曲线更加贴合观测数据。绿色曲线最为灵活，与数据匹配得非常好；然而，我们注意到它对黑色显示的真正f ()拟合得很差，因为它过于波动。通过调整平滑样条拟合的灵活度，我们可以为这些数据生成许多不同的拟合曲线。

现在转向图2.9右侧面板。灰色曲线展示了平均训练MSE随灵活性变化的函数关系，更正式地说，是*degrees of freedom*，对于一系列平滑样条而言。

degrees of freedom

自由度是一个概括曲线灵活性的量；第7章将对此进行更充分的讨论。橙色、蓝色和绿色方块表示与左侧面板中相应曲线相关的MSE。一个更受限因而更平滑的曲线比一条波动曲线具有更少的自由度——注意在图2.9中，线性回归处于最受限的一端，具有两个自由度。训练MSE随着灵活性的增加而单调下降。在这个例子中，真实的f是非线性的，因此橙色的线性拟合不够灵活，无法很好地估计f。绿色曲线在三种方法中具有最低的训练MSE，因为它对应于左侧面板中拟合的三条曲线中最灵活的一条。

在这个例子中，我们知道真实的函数f，因此我们也可以计算在一个非常大的测试集上的测试MSE，作为灵活度的函数。（当然，一般情况下f是未知的，所以这是不可能的。）测试MSE在图2.9右侧面板中用红色曲线显示。与训练MSE一样，测试MSE最初随着灵活度的增加而下降。然而，在某个点上，测试MSE趋于平缓，然后再次开始上升。因此，橙色和绿色曲线都有很高的测试MSE。蓝色曲线使测试MSE最小化，考虑到在图2.9左侧面板中它视觉上似乎最好地估计了f，这并不令人惊讶。水平虚线表示Var(ϵ)，即（2.3）中的不可约误差，它对应于所有可能方法中可实现的最低测试MSE。因此，由蓝色曲线表示的平滑样条接近最优。

在图2.9的右侧面板中，随着统计学习方法灵活性的增加，我们观察到训练MSE单调递减，而测试MSE则出现*U-shape*。这是统计学习的一个基本属性，无论手头的数据集如何，也无论使用何种统计方法，这一属性都成立。随着模型灵活性的增加，训练MSE会下降，但测试MSE未必如此。当某种方法产生较小的训练MSE但较大的测试MSE时，我们称其为对数据的*overfitting*。这种情况发生的原因是，我们的统计学习过程过于努力地在训练数据中寻找模式，可能会捕捉到一些仅仅由随机 chance 而非未知函数f的真实属性所导致的模式。当我们过度拟合训练数据时，测试MSE将会非常大，因为所谓的

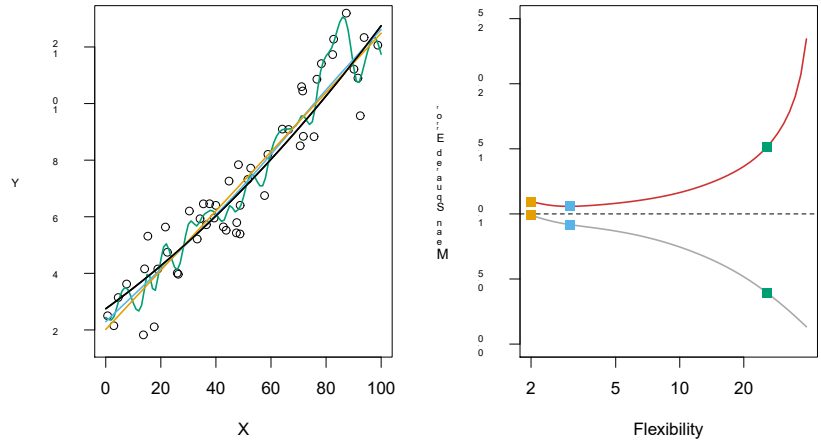


图2.10。 *Details are as in Figure 2.9, using a different true f that is much closer to linear. In this setting, linear regression provides a very good fit to the data.*

该方法在训练数据中发现的模式在测试数据中根本不存在。需要注意的是，无论是否发生了过拟合，我们几乎总是预期训练MSE小于测试MSE，因为大多数统计学习方法都直接或间接地寻求最小化训练MSE。过拟合特指这样一种情况：一个灵活性较低的模型本可以产生更小的测试MSE。

图2.10提供了另一个示例，其中真实的f近似线性。我们再次观察到，随着模型灵活性的增加，训练MSE单调递减，并且测试MSE呈现U形。然而，由于真实关系接近线性，测试MSE在再次上升之前仅略微下降，因此橙色最小二乘拟合显著优于高度灵活的绿色曲线。最后，图2.11展示了一个f高度非线性的例子。训练和测试MSE曲线仍然表现出相同的一般模式，但现在两条曲线都迅速下降，之后测试MSE才开始缓慢上升。

在实践中，通常可以相对容易地计算训练MSE，但估计测试MSE则要困难得多，因为通常没有可用的测试数据。正如前三个例子所示，对应于具有最小测试MSE的模型的灵活性水平在不同的数据集之间可能会有很大差异。在本书中，我们讨论了多种可用于实践中估计这一最小值点的方法。一个重要方法是*cross-validation* (第5章)，这是一种

使用训练数据估计测试MSE的方法。

cross-validation

2.2.2 The Bias-Variance Trade-Off

在测试MSE曲线（图2.9–2.11）中观察到的U形，结果是统计学习方法中两种相互竞争性质的结果。

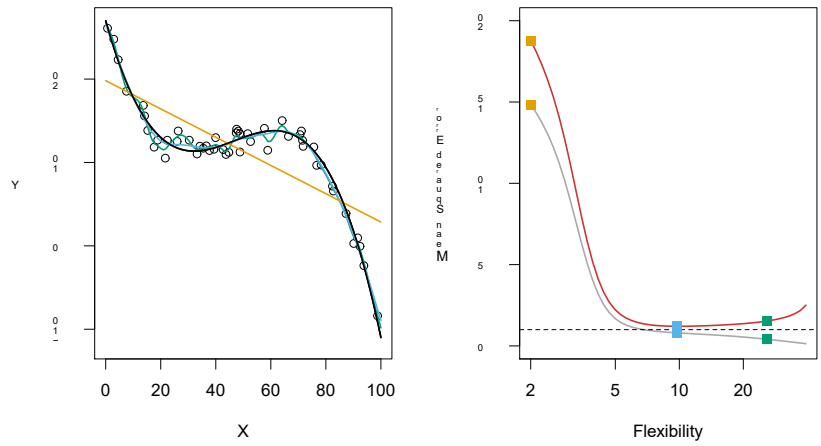


图 2.11. *Details are as in Figure 2.9, using a different f that is far from linear. In this setting, linear regression provides a very poor fit to the data.*

尽管数学证明超出了本书的范围，但可以证明，对于给定的值x_0，预期的测试MSE总是可以分解为三个基本量的和：$\hat{f}(x_0)$的*variance*、$\hat{f}(x_0)$的平方*bias*以及误差的方差项 ϵ。也就是说，

方差偏差

$$E\left(y_0 - \hat{f}(x_0)\right)^2 = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\epsilon). \qquad (2.7)$$

这里记号 {v*} 定义了 x_0 处的 *expected test MSE*，exp指的是如果我们重复使用大量训练集估计f，并在x_0处测试每个模型，所得到的平均测试MSE。整体预期测试MSE可以通过对测试集中所有可能的x_0值求$E\left(y_0 - \hat{f}(x_0)\right)^2$的平均值来计算。

已选择测试均方误差

公式2.7告诉我们，为了最小化预期的测试误差，我们需要选择一种同时达到*low variance*和*low bias*的统计学习方法。需要注意的是，方差本质上是一个非负量，而偏差平方同样是非负的。因此，我们可以看出，预期的测试MSE永远不会低于Var(ϵ)，即来自(2.3)的不可约误差。

在统计学习方法中，*variance*和*bias*指的是什么？*Variance*指的是如果使用不同的训练数据集来估计，$\hat{f}$会发生多大的变化。由于训练数据用于拟合统计学习方法，不同的训练数据集会导致不同的$\hat{f}$。但理想情况下，f的估计值在不同训练集之间不应有太大变化。然而，如果一种方法具有高方差，那么训练数据中的微小变化可能导致$\hat{f}$的大幅变化。一般来说，更灵活的统计方法具有更高的方差。考虑图2.9中的绿色和橙色曲线。灵活的绿色曲线非常紧密地跟随观测值。它具有高方差，因为改变这些数据点中的任何一个都可能导致估计值$\hat{f}$发生显著变化。

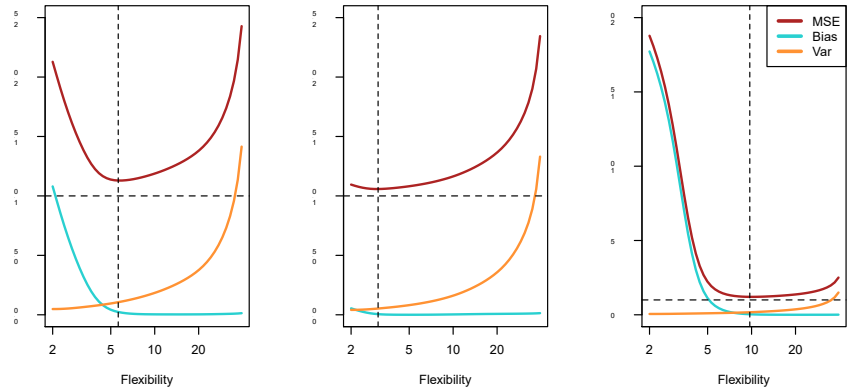


图2.12。 *Squared bias (blue curve), variance (orange curve), $Var(\epsilon)$ (dashed line), and test MSE (red curve) for the three data sets in Figures 2.9–2.11. The vertical dotted line indicates the flexibility level corresponding to the smallest test MSE.*

相反，橙色的最小二乘线相对不灵活，方差较小，因为移动任何单个观测值通常只会导致直线位置的小幅变动。

另一方面，*bias*指的是通过用一个更为简单的模型近似一个可能极其复杂的现实问题而引入的误差。例如，线性回归假设Y与$X_1, X_2, \ldots, X_p$之间存在线性关系。现实生活中几乎不存在如此简单的线性关系，因此进行线性回归无疑会导致f的估计产生一些偏差。在图2.11中，真实的f实质上是非线性的，所以无论我们给定多少训练观测值，都无法使用线性回归产生准确的估计。换言之，在这个例子中，线性回归会导致高偏差。然而，在图2.10中，真实的f非常接近线性，因此给定足够的数据，线性回归应该能够产生准确的估计。一般来说，更灵活的方法会导致更小的偏差。

通常，当我们使用更灵活的方法时，方差会增加，偏差会减少。这两个量的相对变化率决定了测试MSE是增加还是减少。当我们增加一类方法的灵活性时，偏差最初往往会比方差增加得更快地减少。因此，期望测试MSE下降。然而，在某个点上，增加灵活性对偏差几乎没有影响，但开始显著增加方差。当这种情况发生时，测试MSE增加。注意，我们在图2.9-2.11的右侧面板中观察到了这种测试MSE先下降后增加的模式。 注:{v*}保持不变。

图2.12中的三个图展示了方程2.7在图2.9至2.11中的示例应用。每种情况下，蓝色实线曲线代表不同灵活性水平下的偏差平方，而橙色曲线则对应方差。水平虚线表示Var({v*})，即不可减少的误差。最后，红色曲线对应于测试集MSE，其值为两者之和。

在这三个量中。在所有三种情况下，随着方法灵活性的增加，方差增大而偏差减小。然而，对应于最优测试MSE的灵活性水平在三个数据集中差异很大，因为在每个数据集中，平方偏差和方差的变化速率不同。在图2.12的左侧面板中，偏差最初迅速减小，导致预期测试MSE最初急剧下降。另一方面，在图2.12的中间面板中，真实的f接近于线性，因此随着灵活性的增加，偏差只有小幅减小，测试MSE仅略有下降，随后随着方差的增加而迅速上升。最后，在图2.12的右侧面板中，随着灵活性的增加，偏差显著下降，因为真实的f非常非线性。同时，随着灵活性的增加，方差几乎没有增加。因此，在模型灵活性增加的过程中，测试MSE大幅下降，之后仅出现小幅上升。

偏差、方差和测试集 MSE 之间的关系，如方程 2.7 所示并在图 2.12 中展示，被称为 *bias-variance trade-off*。一种统计学习方法良好的测试集表现 re-
需要低方差和低平方偏差。这被称为权衡，因为很容易获得一种具有极低偏差但高方差的方法（例如，通过绘制一条经过每个训练观测点的曲线）或一种具有极低方差但高偏差的方法（通过拟合一条水平线到数据）。挑战在于找到一种方法，使得方差和平方偏差都较低。这种权衡是本书中最重要的反复出现的主题之一。

bias-variance trade-off

在现实生活中，当f未被观测时，通常无法明确计算统计学习方法的测试MSE、偏差或方差。然而，我们始终应牢记偏差-方差权衡。在本书中，我们探索的方法极其灵活，因此基本上可以消除偏差。但这并不保证它们会优于如线性回归这样简单得多的方法。举一个极端的例子，假设真实的f是线性的。在这种情况下，线性回归将没有偏差，使得更灵活的方法很难与之竞争。相反，如果真实的f是高度非线性的，并且我们有大量的训练观测值，那么我们可能使用高度灵活的方法会做得更好，如图2.11所示。在第5章中，我们将讨论交叉验证，这是一种使用训练数据估计测试MSE的方法。

2.2.3 The Classification Setting

到目前为止，我们对模型准确性的讨论主要集中在回归设置上。但是，我们遇到的许多概念，例如偏差-方差权衡，只需稍作修改即可转移到分类设置中，这是由于 y_i 不再是定量的。假设我们寻求基于训练观测 $\{(x_1, y_1), \ldots, (x_n, y_n)\}$ 来估计 f，其中现在 $y_1, \ldots, y_n$ 是定性的。量化我们估计 $\hat{f}$ 准确性的最常见方法是训练 *error rate*，即如果应用我们

error rate

我们对训练观测的估计 $\hat{f}$：

$$\frac{1}{n}\sum_{i=1}^{n} I(y_i \neq \hat{y}_i). \tag{2.8}$$

这里，$\hat{y}_i$ 是使用 $\hat{f}$ 对第 i 个观测的预测类别标签。而 $I(y_i \neq \hat{y}_i)$ 是一个 *indicator variable*，当 $y_i \neq \hat{y}_i$ 时等于 1，当 $y_i = \hat{y}_i$ 时等于零。
如果 $I(y_i \neq \hat{y}_i) =$ 为 0，那么第 i 个观测被我们的分类方法正确分类；否则它被错误分类。因此，公式 2.8 计算了错误分类的比例。

指示变量

方程2.8被称为*training error*速率，因为它
基于用于训练分类器的数据进行计算。与回归设置中一样，我们最感兴趣的是将分类器应用于未用于训练的测试观察结果时所产生的错误率。与一组形式为

训练误差

test error

$(x_0, y0)$ 由下式给出

$$\text{Ave}\left(I(y_0 \neq \hat{y}_0)\right), \tag{2.9}$$

其中 $\hat{y}_0$ 是将分类器应用于具有预测变量 x_0 的测试观测所得到的预测类别标签。*good* 分类器是指其测试误差（2.9）最小的分类器。

贝叶斯分类器

可以证明（尽管证明超出了本书的范围），(2.9) 中给出的测试错误率，在平均意义上，由一个非常简单的分类器
assigns each observation to the most likely class,given its predictor values
来最小化。换句话说，我们只需将预测向量为x_0的测试观测分配给类别j，使得

$$\Pr(Y = j|X = x_0) \tag{2.10}$$

是最大的。注意(2.10)是一个*conditional probability*：它是概率
也就是说，$Y = j$，给定观测到的预测变量向量x_0。这个非常简单的分类器被称为*Bayes classifier*。在一个两类问题中，其中有
只有两个可能的响应值，比如*class 1*或*class 2*，贝叶斯分类器对应于如果P r($Y = 1|X = x_0$) > 0.5，则预测为一类，否则预测为二类。

条件概率

贝叶斯分类器

图2.13提供了一个在由预测变量X_1和X_2构成的二维空间中使用模拟数据集的示例。橙色和蓝色圆圈对应于属于两个不同类别的训练观测值。对于每个X_1和X_2的值，响应为橙色或蓝色的概率各不相同。由于这是模拟数据，我们知道数据是如何生成的，并且可以计算每个X_1和X_2值的条件概率。橙色阴影区域反映了Pr（Y =橙色$|X$）大于50%的点集，而蓝色阴影区域表示概率低于50%的点集。紫色虚线表示概率恰好为50%的点。这被称为*Bayes decision boundary*。贝叶斯

贝叶斯决策边界

分类器的预测由贝叶斯决策边界决定；落在边界橙色一侧的观测值将被分配给

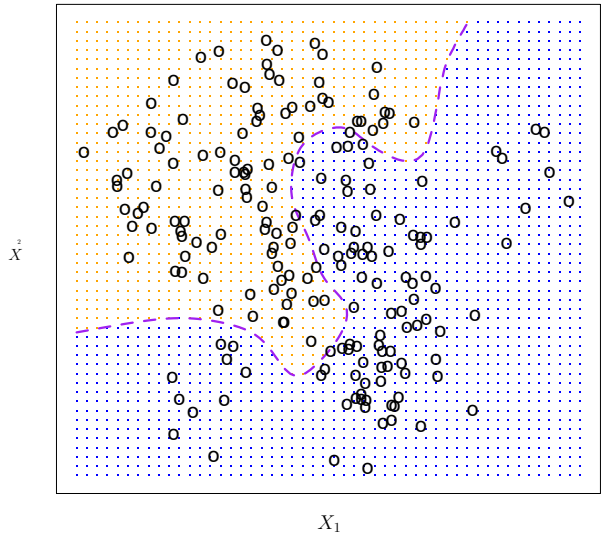


图2.13. *A simulated data set consisting of* 100 *observations in each of two groups, indicated in blue and in orange. The purple dashed line represents the Bayes decision boundary. The orange background grid indicates the region in which a test observation will be assigned to the orange class, and the blue background grid indicates the region in which a test observation will be assigned to the blue class.*

对于橙色类别也是如此，而边界蓝色一侧的观测同样会被分配到蓝色类别。

贝叶斯分类器产生尽可能低的测试错误率，称为 *Bayes error rate*。因为贝叶斯分类器总是会选择贝叶斯

错误

对一

于 (2.10) 最大的速率，错误率将在 $X = x_0$ 处为 $1-\max_j \Pr(Y = j|X = x_0)$。般来说，整体贝叶斯错误率由下式给出：

$$1 - E\left(\max_j \Pr(Y = j|X)\right), \tag{2.11}$$

其中，期望是对所有可能的X值上的概率进行平均。对于我们的模拟数据，贝叶斯错误率为0.133。它大于零，因为类别在真实总体中存在重叠，这意味着对于某些x_0值，$\max_j \Pr(Y = j|X = x_0) < 1$。贝叶斯错误率类似于之前讨论的不可约误差。

K-最近邻

理论上，我们总是希望使用贝叶斯分类器来预测定性响应。但对于实际数据，我们不知道给定 X 时 Y的条件分布，因此计算贝叶斯分类器是不可能的。所以，贝叶斯分类器作为一个无法达到的黄金标准，用来与其他方法进行比较。许多方法尝试估计给定 X 时 Y的条件分布，然后将给定的观测分类到具有最高 *estimated* 概率的类别。其中一种方法是 *K-nearest neighbors* (KNN) 分类器。给定一个正整

K-nearest neighbors

对于整数 K 和一个测试观察 x_0，KNN 分类器首先识别训练数据中与 x_0 最接近的 K 个点，表示为 $\mathcal{N}_0$。然后，它将类别 j 的条件概率估计为 $\mathcal{N}_0$ 中响应值等于 j 的点的比例：

$$\Pr(Y = j|X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = j). \tag{2.12}$$

最后，KNN将测试观测值 {v*} 分类到由（2.12）得出的概率最大的类别中。

图2.14给出了KNN方法的一个示例。在左侧面板中，我们绘制了一个由六个蓝色和六个橙色观测值组成的小型训练数据集。我们的目标是对由黑色十字标记的点进行预测。假设我们选择 $K = 3$。那么KNN将首先识别出离十字最近的三个观测值。这个邻域显示为一个圆圈。它由两个蓝色点和一个橙色点组成，从而得到蓝色类别的估计概率为2/3，橙色类别的估计概率为1/3。因此，KNN将预测黑色十字属于蓝色类别。在图2.14的右侧面板中，我们对 X_1 和 X_2 的所有可能值应用了 $K = 3$ 的KNN方法，并绘制了相应的KNN决策边界。

尽管方法非常简单，但 KNN 常常能产生令人惊讶地接近最优贝叶斯分类器的分类器。图 2.15 展示了当 $K = 10$ 时，应用于图 2.13 中更大的模拟数据集的 KNN 决策边界。注意，即使 KNN 分类器不知道真实分布，KNN 决策边界也非常接近贝叶斯分类器的决策边界。使用 KNN 的测试错误率为 0.1363，接近贝叶斯错误率 0.1304。

K 的选择对所得到的 KNN 分类器有显著影响。图 2.16 展示了对图 2.13 中模拟数据的两个 KNN 拟合，分别使用了 $K = 1$ 和 $K = 100$。当 $K = 1$ 时，决策边界过于灵活，在数据中寻找到与贝叶斯决策边界不符的模式。这对应于一个低偏差但极高方差的分类器。随着 K 增大，方法变得不那么灵活，并产生接近线性的决策边界。这对应于一个低方差但高偏差的分类器。在这个模拟数据集上，无论是 $K = 1$ 还是 $K = 100$ 都没有给出良好的预测：它们的测试错误率分别为 0.1695 和 0.1925。

正如在回归设置中一样，训练错误率与测试错误率之间并没有强烈的关系。当 K 为 1 时，KNN 的训练错误率为 0，但测试错误率可能相当高。一般来说，当我们使用更灵活的分类方法时，训练错误率会下降，但测试错误率可能不会。在图 2.17 中，我们绘制了 KNN 测试误差和训练误差作为 $1/K$ 的函数。随着 $1/K$ 增加，方法变得更加灵活。与回归设置一样，随着灵活性的增加，训练错误率持续下降。然而，测试错误呈现出特征性的 U 形，先下降（在约 $K = 10$ 处达到最小值），然后当方法变得过于灵活并过拟合时再次上升。

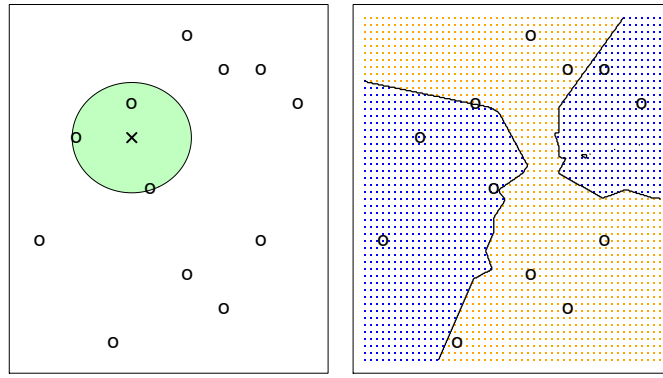


图 2.14. *The KNN approach, using $K = 3$, is illustrated in a simple situation with six blue observations and six orange observations.* 左： *a test observation at which a predicted class label is desired is shown as a black cross. The three closest points to the test observation are identified, and it is predicted that the test observation belongs to the most commonly-occurring class, in this case blue.* 右： *The KNN decision boundary for this example is shown in black. The blue grid indicates the region in which a test observation will be assigned to the blue class, and the orange grid indicates the region in which it will be assigned to the orange class.*

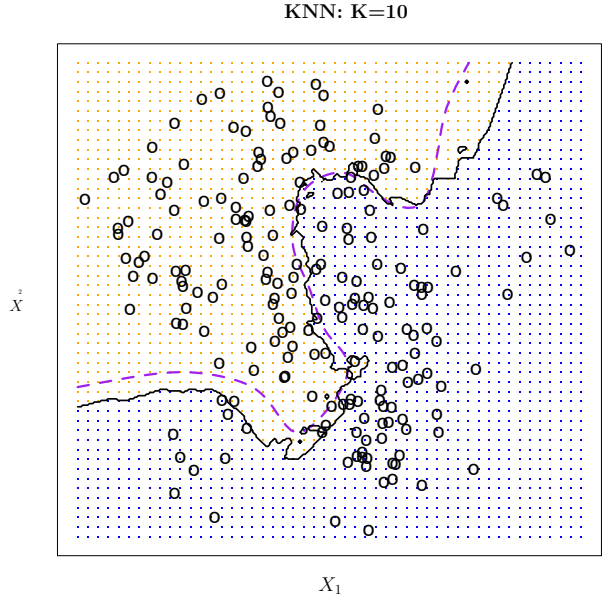


图2.15。 *The black curve indicates the KNN decision boundary on the data from Figure 2.13, using $K = 10$. The Bayes decision boundary is shown as a purple dashed line. The KNN and Bayes decision boundaries are very similar.*

在回归和分类设置中，选择正确的灵活性水平对于任何统计学习方法的成功都至关重要。偏差-方差权衡以及由此导致的测试误差中的U型曲线，会使这项任务变得困难。在第5章中，我们将回到这个主题并讨论

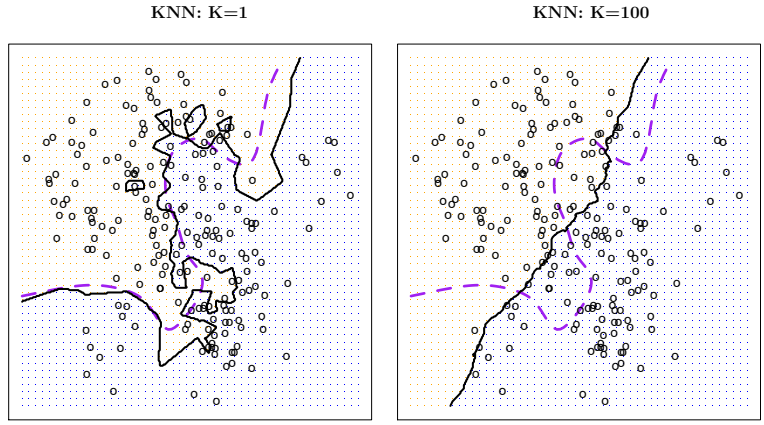


图 2.16. *A comparison of the KNN decision boundaries (solid black curves) obtained using* $K = 1$ *and* $K = 100$ *on the data from Figure 2.13. With* $K = 1$*, the decision boundary is overly flexible, while with* $K = 100$ *it is not sufficiently flexible. The Bayes decision boundary is shown as a purple dashed line.*

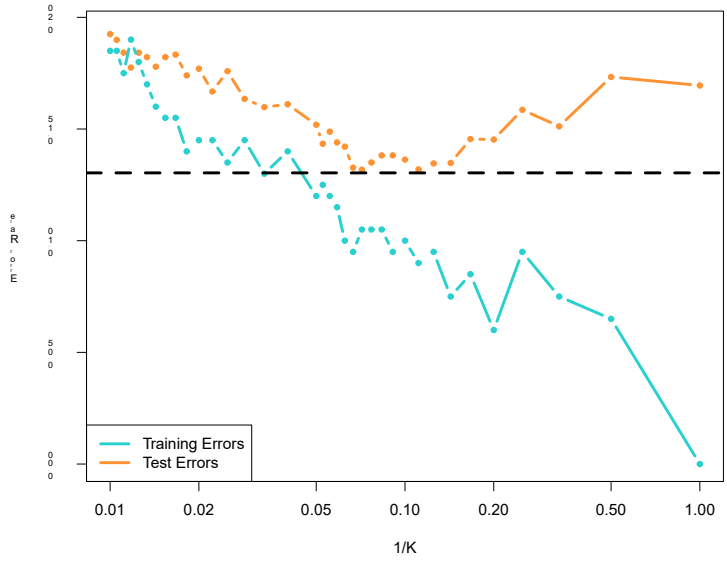


图2.17。*The KNN training error rate (blue, 200 observations) and test error rate (orange, 5,000 observations) on the data from Figure 2.13, as the level of flexibility (assessed using* $1/K$ *on the log scale) increases, or equivalently as the number of neighbors* K *decreases. The black dashed line indicates the Bayes error rate. The jumpiness of the curves is due to the small size of the training data set.*

用于估计测试错误率并据此为给定的统计学习方法选择最优灵活度水平的各种方法。

2.3 实验：Python入门

2.3.1 Getting Started

要运行本书中的实验，你需要两样东西：

1. 安装 Python3，这是实验室中使用的 Python 的特定版本。2. 访问 Jupyter，这是一个非常流行的 Python 接口，它通过一个名为 *notebook* 的文件运行代码。

笔记本

您可以按照 anaconda.com 上提供的说明下载并安装 Python3。

有多种方法可以访问Jupyter。这里仅列举几种：

1. 使用谷歌的 Colaboratory 服务：colab.research.google.com/。
2. 使用 JupyterHub，可在 jupyter.org/hub 获取。
3. 使用您自己的 jupyter 安装。安装说明可在 jupyter.org/install 获取。

请参阅书籍网站statlearning.com上的Python资源页面，以获取有关在您的计算机上使Python和Jupyter正常工作的最新信息。

你需要安装 ISLP 包，它提供了对我们所提供数据集和自定义函数的访问权限。在 macOS 或 Linux 终端中，输入 pip install ISLP；这也会安装实验中最需要的多数其他软件包。Python 资源页面上有一个指向 ISLP 文档网站的链接。

要运行此实验，请从 Python 资源页面下载文件 Ch2-statlearn-lab.ipynb 。现在在命令行中运行以下代码：jupyter lab Ch2-statlearn-lab.ipynb 。

如果您使用的是 Windows，您可以使用 start menu 来访问 anaconda，并按照链接操作。例如，要安装 ISLP 并运行此实验，您可以在 anaconda shell 中运行上述相同的代码。

2.3.2 Basic Commands

在本实验中，我们将介绍一些简单的 Python 命令。关于 Python 的更多一般资源，读者可以参考 docs.python.org/3/tutorial/ 上的教程。

与大多数编程语言一样，Python 使用 *functions* 来执行操作。
操作。要运行一个名为fun的函数，我们输入fun(input1,input2)，其中输入（或*arguments*）input1和input2告诉Python如何运行该函数。一个函数可以有任意数量的输入。例如，

function

argument

print()

print() 函数将其所有参数的文本表示输出到控制台。

```
In [1]: print('fit a model with', 11, 'variables')
```

```
fit a model with 11 variables
```

以下命令将提供有关 `print()` 函数的信息。

```
In [2]: print?
```

在 Python 中加法两个整数是非常直观的。

```
In [3]: 3 + 5
```

```
Out[3]: 8
```

在 Python 中，文本数据使用 *strings* 处理。例如，`"hello"` 和字符串 `'hello'` 都是字符串。我们可以使用加法 `+` 符号将它们连接起来。

```
In [4]: "hello" + " " + "world"
```

```
Out[4]: 'hello world'
```

字符串实际上是一种 *sequence*：这是有序序列列表的通用术语。最重要的三种序列类型是列表、元组和字符串。我们现在介绍列表。

以下命令指示 Python 将数字 3、4 和 5 连接在一起，并将它们保存为一个名为 `x` 的 *list*。当我们输入 `x` 时，它会将列表返回给我们。

```
In [5]: x = [3, 4, 5]
        x
```

```
Out[5]: [3, 4, 5]
```

请注意，我们使用了方括号 `[]` 来构建此列表。我们经常需要将两组数字相加。尝试以下代码是合理的，尽管它不会产生预期的结果。

```
In [6]: y = [4, 9, 7]
        x + y
```

```
Out[6]: [3, 4, 5, 4, 9, 7]
```

结果可能看起来略显反直觉：为什么 Python 没有逐元素地将列表的条目相加？在 Python 中，列表持有 *arbitrary* 对象，并使用 *concatenation* 进行相加。实际上，拼接是
我们之前输入 `"hello" + " " + "world"` 时看到的行为。

拼接

这个例子反映了Python是一种通用编程语言。Python的许多数据特定功能来自其他包，尤其是numpy和pandas。在下一节中，我们将介绍numpy包。关于numpy的更多信息，请参见docs.scipy.org/doc/numpy/user/quickstart.html。

2.3.3 Introduction to Numerical Python

如前所述，本书利用了包含在 numpy *library* 或 *package* 中的功能。包是模块的集合，这些模块
不一定包含在基础 Python 发行版中。名称 numpy 是 *numerical Python* 的缩写。

numpy
包

要访问 numpy，我们必须先 import 它。

import

```
In [7]: import numpy as np
```

在上一行中，我们将 numpy *module* np 命名为；其缩写为
更容易引用。

模块

在 numpy 中，*array* 是多维数字集合的通用术语。数组 我们使用 np.array() 函数来定义 x 和 y，它们是一维
数组，即向量。

np.arr ay()

```
In [8]: x = np.array([3, 4, 5])
        y = np.array([4, 9, 7])
```

请注意，如果之前忘记运行 import numpy as np 命令，那么在上一条命令行中调用 np.array() 函数时将会遇到错误。语法 np.array() 表示所调用的函数是 numpy 包的一部分，我们已将其简写为 np。

由于 x 和 y 已通过 np.array() 定义，将它们相加时会得到一个合理的结果。将其与上一节中的结果进行比较，那时我们尝试在不使用 numpy 的情况下将两个列表相加。

```
In [9]: x + y
```

```
Out[9]: array([ 7, 13, 12])
```

在 numpy 中，矩阵通常表示为二维数组，向量表示为一维数组。[1] 我们可以按如下方式创建二维数组。

```
In [10]: x = np.array([[1, 2], [3, 4]])
         x
```

```
Out[10]: array([[1, 2],
                [3, 4]])
```

对象 x 拥有多个 *attributes*，即关联对象。要访问一个
x 的属性，我们输入 x.attribute，其中我们将 attribute 替换为属性的名称。例如，我们可以访问 x 的 ndim 属性
如下。

属性

ndim

```
In [11]: x.ndim
```

```
Out[11]: 2
```

输出表明 x 是一个二维数组。类似地，x.dtype 是对象 x 的 *data type* 属性。这表明 x 由
64 位整数：

data type

[1] While it is also possible to create matrices using np.matrix(), we will use np.array() throughout the labs in this book.

```
In [12]: x.dtype
```

```
Out[12]: dtype('int64')
```

为什么x由整数组成？这是因为我们通过向np.array()函数传入纯整数来创建了x。如果我们传入了任何小数，那么我们将得到一个*floating point numbers*浮点数数组。
（即实数值）。

浮点

```
In [13]: np.array([[1, 2], [3.0, 4]]).dtype
```

```
Out[13]: dtype('float64')
```

输入 fun? 将导致 Python 显示与函数 fun 相关的文档（如果存在）。我们可以对 np.array() 尝试此操作。

```
In [14]: np.array?
```

本文档表明，我们可以通过将 dtype 参数传入 np.array() 来创建一个浮点数组。

dtype

```
In [15]: np.array([[1, 2], [3, 4]], float).dtype
```

```
Out[15]: dtype('float64')
```

数组 x 是二维的。我们可以通过查看其 shape 属性来找出它的行数和列数。

shape

```
In [16]: x.shape
```

```
Out[16]: (2, 2)
```

method 是一个与对象关联的函数。例如，方法给定一个数组 x，表达式 x.sum() 使用数组的 sum() 方法对其所有元素求和。调用 x.sum() 会自动将 x 作为其 sum() 方法的 .sum() 第一个参数提供。

```
In [17]: x = np.array([1, 2, 3, 4])
         x.sum()
```

```
Out[17]: 10
```

我们还可以通过将 x 作为参数传递给 np.sum() 函数来对 x 的元素求和。

np.sum()

```
In [18]: x = np.array([1, 2, 3, 4])
         np.sum(x)
```

```
Out[18]: 10
```

再举一个例子，reshape()方法返回一个新数组，其 .reshape() 元素与x相同，但形状不同。我们通过传入一个tuple tuple来实现这一点。

在对 reshape() 的调用中，此处为 (2, 3)。此元组指定我们想要创建一个具有 2 行 3 列的二维数组。[2] 在接下来的内容中，\n 字符创建了一个 *new line*。

In [19]:
```
x = np.array([1, 2, 3, 4, 5, 6])
print('beginning x:\n', x)
x_reshape = x.reshape((2, 3))
print('reshaped x:\n', x_reshape)
```

```
beginning x:
 [1 2 3 4 5 6]
reshaped x:
 [[1 2 3]
 [4 5 6]]
```

之前的输出揭示了 numpy 数组被指定为 *rows* 的序列。这被称为 *row-major ordering*，与 *column-major ordering* 相对。

Python（因此numpy)使用从0开始的索引。这意味着要访问x_reshape的左上角元素，我们输入x_reshape[0,0]。

In [20]:
```
x_reshape[0, 0]
```

Out[20]: 1

类似地，x_reshape[1,2] 得出 x_reshape 中第二行第三列的元素。

In [21]:
```
x_reshape[1, 2]
```

Out[21]: 6

同样地，x[2]产生了x的第三项。现在，让我们修改x_reshape左上角的元素。令我们惊讶的是，我们发现x的第一个元素也已经被修改了！

In [22]:
```
print('x before we modify x_reshape:\n', x)
print('x_reshape before we modify x_reshape:\n', x_reshape)
x_reshape[0, 0] = 5
print('x_reshape after we modify its top left element:\n',
    x_reshape)
print('x after we modify top left element of x_reshape:\n', x)
```

Out[22]:
```
x before we modify x_reshape:
 [1 2 3 4 5 6]
x_reshape before we modify x_reshape:
 [[1 2 3]
 [4 5 6]]
x_reshape after we modify its top left element:
 [[5 2 3]
```

[2] Like lists, tuples represent a sequence of objects. Why do we need more than one way to create a sequence? There are a few differences between tuples and lists, but perhaps the most important is that elements of a tuple cannot be modified, whereas elements of a list can be.

```
 [4 5 6]]
x after we modify top left element of x_reshape:
 [5 2 3 4 5 6]
```

修改 x_reshape 也修改了 x，因为这两个对象在内存中占据相同的空间。

我们刚刚看到可以修改数组的元素。我们也可以修改元组吗？事实证明我们不能——尝试这样做会引入一个*exception*，即错误。

异常

```
In [23]: my_tuple = (3, 4, 5)
         my_tuple[0] = 2
```

```
TypeError: 'tuple' object does not support item assignment
```

我们现在简要提及一些将派上用场的数组属性。数组的shape属性包含了它的维度；这始终是一个元组。ndim属性给出了维度的数量，而T提供了它的转置。

```
In [24]: x_reshape.shape, x_reshape.ndim, x_reshape.T
```

```
Out[24]: ((2, 3),
          2,
          array([[5, 4],
                 [2, 5],
                 [3, 6]]))
```

请注意，三个单独的输出 (2,3)、2 和 array([[5, 4],[2, 5], [3,6]]) 本身作为一个元组输出。我们经常希望将函数应用于数组。例如，我们可以使用 np.sqrt() 函数计算各元素的平方根：

np.sqrt()

```
In [25]: np.sqrt(x)
```

```
Out[25]: array([2.24, 1.41, 1.73, 2., 2.24, 2.45])
```

我们也可以对元素进行平方：

```
In [26]: x**2
```

```
Out[26]: array([25, 4, 9, 16, 25, 36])
```

我们可以使用相同的符号来计算平方根，即提高到1/2的幂，而不是2。

```
In [27]: x**0.5
```

```
Out[27]: array([2.24, 1.41, 1.73, 2., 2.24, 2.45])
```

在本书中，我们经常需要生成随机数据。np.random.normal()函数生成一个随机正态变量向量。

我们可以通过调用 np.random.normal? 查看帮助页面来进一步了解这个函数。帮助页面的第一行显示 normal(loc=0.0, scale=1.0, size=None)。这一行 *signature* 告诉我们这个函数的参数是……

np.random.normal()

结构

参数为loc、scale和size。这些是*keyword*参数，其键
意味着当它们被传入函数时，可以通过名称（以任意顺序）引用。[3] 默认
情况下，此函数将生成均值为(loc) 0 且标准差为(scale) 1 的随机正态变量
；此外，除非更改size的参数，否则将生成单个随机变量。

单词

我们现在从 $N(0,1)$ 分布生成50个独立的随机变量。

```
In [28]: x = np.random.normal(size=50)
         x
```

```
Out[28]: array([-1.19,  0.41,  0.9 , -0.44, -0.9 , -0.38,  0.13,  1.87,
                -0.35,  1.16,  0.79, -0.97, -1.21,  0.06, -1.62, -0.6 ,
                -0.77, -2.12,  0.38, -1.22, -0.06, -1.97, -1.74, -0.56,
                 1.7 , -0.95,  0.56,  0.35,  0.87,  0.88, -1.66, -0.32,
                -0.3 , -1.36,  0.92, -0.31,  1.28, -1.94,  1.07,  0.07,
                 0.79, -0.46,  2.19, -0.27, -0.64,  0.85,  0.13,  0.46,
                -0.09,  0.7 ])
```

我们通过向 x 的每个元素添加一个独立的 $N(50,1)$ 随机变量，创建一个数
组 y。

```
In [29]: y = x + np.random.normal(loc=50, scale=1, size=50)
```

np.corrcoef() 函数计算 x 与 之间的相关矩阵
y非对角元素给出了x与y之间的相关性。

np.corrcoef()

```
In [30]: np.corrcoef(x, y)
```

```
Out[30]: array([[1.  , 0.69],
                [0.69, 1.  ]])
```

如果你在自己的 Jupyter notebook 中跟着操作，那么你可能已经注意到，
运行过去几条命令时得到了一组不同的结果。特别是，每次我们调用
np.random.normal() 时，都会得到不同的答案，如下例所示。

```
In [31]: print(np.random.normal(scale=5, size=2))
         print(np.random.normal(scale=5, size=2))
```

```
Out[31]: [4.28 2.59]
         [4.62 -2.54]
```

为了确保我们的代码在每次运行时提供完全相同的结果，我们可以使用
np.random.default_rng()设置一个*random seed*。
函数。此函数接受任意的用户指定整数参数。如果我们在生成随机数据前
设置了随机种子，那么重新运行代码将得到相同的结果。对象 rng 基本上
拥有所有

随机种子
np.random.
default_rng()

[3] Python also uses *positional* arguments. Positional arguments do not need to use a keyword. To see an example, type in np.sum?. We see that a is a positional argument, i.e. this function assumes that the first unnamed argument that it receives is the array to be summed. By contrast, axis and dtype are keyword arguments: the position in which these arguments are entered into np.sum() does not matter.

在 np.random 中发现的随机数生成方法。因此，为了生成正态数据，我们使用 rng.normal()。

In [32]:
```
rng = np.random.default_rng(1303)
print(rng.normal(scale=5, size=2))
rng2 = np.random.default_rng(1303)
print(rng2.normal(scale=5, size=2))
```

Out[32]:
```
[4.09  -1.07 ]
[4.09  -1.07 ]
```

在本书的整个实验环节中，每当我们在numpy内进行涉及随机量的计算时，我们都会使用np.random.default_rng()。理论上，这应能让读者精确复现所述结果。然而，随着numpy新版本的推出，实验室输出与numpy输出之间可能会出现一些细微差异。

np.mean()、np.var() 和 np.std() 函数可用于计算数组的均值、方差和标准差。这些函数也可以作为数组上的方法使用。

np.mean()
np.var()
np.std()

In [33]:
```
rng = np.random.default_rng(3)
y = rng.standard_normal(10)
np.mean(y), y.mean()
```

Out[33]:
```
(-0.11, -0.11)
```

In [34]:
```
np.var(y), y.var(), np.mean((y - y.mean())**2)
```

Out[34]:
```
(2.72, 2.72, 2.72)
```

请注意，默认情况下 np.var() 除以样本大小 n，而不是 n –1；请参阅 np.var? 中的 ddof 参数。

In [35]:
```
np.sqrt(np.var(y)), np.std(y)
```

Out[35]:
```
(1.65, 1.65)
```

np.mean()、np.var() 和 np.std() 函数也可以应用于矩阵的行和列。为了说明这一点，我们构造一个由 $N(0, 1)$ 随机变量组成的 10×3 矩阵，并考虑计算其行和。

In [36]:
```
X = rng.standard_normal((10, 3))
X
```

Out[36]:
```
array([[ 0.23, -0.35, -0.28],
       [-0.67, -1.06, -0.39],
       [ 0.48, -0.24,  0.96],
       [-0.2 ,  0.02,  1.55],
       [ 0.55, -0.51, -0.18],
       [ 0.54,  1.94, -0.27],
       [-0.24,  1.  , -0.89],
       [-0.29,  0.88,  0.58],
       [ 0.09,  0.67, -2.83],
       [ 1.02, -0.96, -1.67]])
```

由于数组按行优先顺序排列，第一个轴，即 axis=0，指的是其行。我们将此参数传递给对象 X 的 mean() 方法。

.mean()

```
In [37]: X.mean(axis=0)
```

```
Out[37]: array([0.15, 0.14, -0.34])
```

以下产生相同的结果。

```
In [38]: X.mean(0)
```

```
Out[38]: array([0.15, 0.14, -0.34])
```

2.3.4 Graphics

在Python中，常见做法是使用库matplotlib进行图形处理。
然而，由于Python在编写时并未考虑数据分析，绘图的概念并非该语言固有。我们将使用matplotlib.pyplot中的subplots()函数来创建图形以及用于绘制数据的坐标轴。关于如何在Python中制作图表的更多示例，鼓励读者访问matplotlib.org/stable/gallery/。

matplotlib

在matplotlib中，一个图表由一个*figure*和一个或多个*axes*组成。你可以将图形视为空白画布，其上将显示一个或多个图形：它是整个绘图窗口。*axes* 包含有关每个图形的重要信息，例如其 x 轴和 y 轴标签、标题等。（请注意，在 matplotlib 中，单词 *axes* 不是 *axis* 的复数形式：图形的 *axes* 包含的信息远不止 x 轴和 y 轴。）

图轴

我们首先从matplotlib导入subplots()函数。我们创建图形时始终使用此函数。该函数返回一个长度为二的元组：一个图形对象以及相关的轴对象。我们通常将 figsize 作为关键字参数传递。创建轴后，我们使用其 plot() 方法尝试第一次绘图。要了解更多关于它，输入 ax.plot?。

subplots()

.plot()

```
In [39]: from matplotlib.pyplot import subplots
         fig, ax = subplots(figsize=(8, 8))
         x = rng.standard_normal(100)
         y = rng.standard_normal(100)
         ax.plot(x, y);
```

我们在此暂停一下，注意我们已经将subplots()返回的长度为二的元组*unpacked*解包到了两个不同的变量fig和ax中。通常更倾向于使用解包，而不是以下等价但略为冗长的代码：

```
In [40]: output = subplots(figsize=(8, 8))
         fig = output[0]
         ax = output[1]
```

我们看到之前的单元格生成了一个线图，这是默认设置。要创建散点图，我们向 ax.plot() 提供一个额外参数，指示应显示圆形。

In [41]:
```
fig, ax = subplots(figsize=(8, 8))
ax.plot(x, y, 'o');
```

Different values of this additional argument can be used to produce different colored lines as well as different linestyles.

As an alternative, we could use the `ax.scatter()` function to create a scatterplot. `.scatter()`

In [42]:
```
fig, ax = subplots(figsize=(8, 8))
ax.scatter(x, y, marker='o');
```

Notice that in the code blocks above, we have ended the last line with a semicolon. This prevents `ax.plot(x, y)` from printing text to the notebook. However, it does not prevent a plot from being produced. If we omit the trailing semi-colon, then we obtain the following output:

In [43]:
```
fig, ax = subplots(figsize=(8, 8))
ax.scatter(x, y, marker='o')
```

Out[43]:
```
<matplotlib.collections.PathCollection at 0x7fb3d9c8f310>
Figure(432x288)
```

In what follows, we will use trailing semicolons whenever the text that would be output is not germane to the discussion at hand.

To label our plot, we make use of the `set_xlabel()`, `set_ylabel()`, and `set_title()` methods of `ax`. `.set_xlabel()` `.set_ylabel()` `.set_title()`

In [44]:
```
fig, ax = subplots(figsize=(8, 8))
ax.scatter(x, y, marker='o')
ax.set_xlabel("this is the x-axis")
ax.set_ylabel("this is the y-axis")
ax.set_title("Plot of X vs Y");
```

Having access to the figure object `fig` itself means that we can go in and change some aspects and then redisplay it. Here, we change the size from `(8, 8)` to `(12, 3)`.

```
fig.set_size_inches(12,3)
fig
```

Occasionally we will want to create several plots within a figure. This can be achieved by passing additional arguments to `subplots()`. Below, we create a 2×3 grid of plots in a figure of size determined by the `figsize` argument. In such situations, there is often a relationship between the axes in the plots. For example, all plots may have a common x-axis. The `subplots()` function can automatically handle this situation when passed the keyword argument `sharex=True`. The `axes` object below is an array pointing to different plots in the figure.

In [45]:
```
fig, axes = subplots(nrows=2,
                     ncols=3,
                     figsize=(15, 5))
```

We now produce a scatter plot with `'o'` in the second column of the first
第二行第三列中有一个带有 `'+'` 的散点图。

In [46]:
```
axes[0,1].plot(x, y, 'o')
axes[1,2].scatter(x, y, marker='+')
fig
```

输入 subplots? 以了解有关 subplots() 的更多信息。

要保存 fig 的输出，我们调用其 savefig() 方法。参数 dpi 是每英寸点数，用于确定图形将以像素为单位的大小。

.savefig()

In [47]:
```
fig.savefig("Figure.png", dpi=400)
fig.savefig("Figure.pdf", dpi=200);
```

我们可以继续使用逐步更新来修改fig；例如，我们可以修改x轴的范围、重新保存图形，甚至重新显示它。

In [48]:
```
axes[0,1].set_xlim([-1,1])
fig.savefig("Figure_updated.jpg")
fig
```

我们现在创建一些更复杂的图形。方法ax.contour()生成一个*contour plot*，用以表示三维数据，类似于地形图的等高线图。它接受三个参数：

.contour()

- 一个由x值组成的向量（第一维），
- 一个包含y值的向量（第二维度），并且
- 一个矩阵，其元素对应于每对(x,y)坐标的z值（第三个维度）。

要创建x和y，我们将使用命令np.linspace(a, b, n)，它会返回一个由n个数字组成的向量，起始于a，终止于b。

In [49]:
```
fig, ax = subplots(figsize=(8, 8))
x = np.linspace(-np.pi, np.pi, 50)
y = x
f = np.multiply.outer(np.cos(y), 1 / (1 + x**2))
ax.contour(x, y, f);
```

W可以通过增加更多级别来提高分辨率，e 图像。

In [50]:
```
fig, ax = subplots(figsize=(8, 8))
ax.contour(x, y, f, levels=45);
```

要微调 ax.contour() 函数的输出，请通过输入 ?plt.contour 查看帮助文件。

ax.imshow() 方法类似于 ax.contour()，不同之处在于它生成一个颜色编码的图，其颜色取决于 z 值。这被称为 *heatmap*，有时用于在天气预报中绘制温度的热图。

In [51]:
```
fig, ax = subplots(figsize=(8, 8))
ax.imshow(f);
```

2.3.5 Sequences and Slice Notation

如上所示，函数np.linspace()可用于创建数字序列。

```
In [52]: seq1 = np.linspace(0, 10, 11)
         seq1
```

```
Out[52]: array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])
```

函数 np.arange() 返回一个以 step 间隔的数字序列。如果没有指定 step ，则使用默认值1。让我们创建一个从0开始到10结束的序列。

np.arange()

```
In [53]: seq2 = np.arange(0, 10)
         seq2
```

```
Out[53]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

为什么上面没有输出 10？这与 Python 中的 *slice* 记法有关。slice 切片记法用于对列表、元组和数组等序列进行索引。假设我们要检索字符串的第四到第六个（含）条目。我们使用索引记法 [3:6] 获取该字符串的一个切片。

```
In [54]: "hello world"[3:6]
```

```
Out[54]: 'lo '
```

在上面的代码块中，当在[]内部使用时，记号3:6是slice(3,6)的简写。

```
In [55]: "hello world"[slice(3,6)]
```

```
Out[55]: 'lo '
```

你原本可能以为 slice(3,6) 会输出文本字符串中的第四个到第七个字符（记得 Python 的索引是从零开始的），但实际上它只输出了第四个到第六个。这也解释了为什么之前的 np.arange(0, 10) 命令只输出了从 0 到 9 的整数。请参阅文档 slice?，了解创建切片时的有用选项。

2.3.6 Indexing Data

首先，我们创建一个二维 numpy 数组。

```
In [56]: A = np.array(np.arange(16)).reshape((4, 4))
         A
```

```
Out[56]: array([[ 0,  1,  2,  3],
                [ 4,  5,  6,  7],
                [ 8,  9, 10, 11],
                [12, 13, 14, 15]])
```

输入 A[1,2] 可以获取第二行第三列对应的元素。（照例，Python 从0.开始索引）

```
In [57]: A[1,2]
```

```
Out[57]: 6
```

左括号符号 `[` 后面的第一个数字表示行，第二个数字表示列。

索引行、列和子矩阵

要一次选择多行，我们可以传入一个列表来指定选择。例如，`[1,3]` 将获取第二行和第四行：

```
In [58]: A[[1,3]]
```

```
Out[58]: array([[ 4,  5,  6,  7],
                [12, 13, 14, 15]])
```

要选择第一列和第三列，我们在方括号的第二个参数中传入`[0,2]`。在这种情况下，我们需要提供第一个参数`:`，它选择所有行。

```
In [59]: A[:,[0,2]]
```

```
Out[59]: array([[ 0,  2],
                [ 4,  6],
                [ 8, 10],
                [12, 14]])
```

现在，假设我们想要选取由第二行和第四行以及第一列和第三列组成的子矩阵。这就是索引变得稍微有点棘手的地方。人们自然而然地会尝试使用列表来选取行和列：

```
In [60]: A[[1,3],[0,2]]
```

```
Out[60]: array([ 4, 14])
```

糟糕——发生了什么？我们得到了一个长度为一维数组，其长度为二，与

```
In [61]: np.array([A[1,0],A[3,2]])
```

```
Out[61]: array([ 4, 14])
```

类似地，以下代码无法提取由第二行和第四行以及第一、第三和第四列组成的子矩阵：

```
In [62]: A[[1,3],[0,2,3]]
```

```
IndexError: shape mismatch: indexing arrays could not be broadcast
    together with shapes (2,) (3,)
```

我们可以看到这里出了什么问题。当提供两个索引列表时，numpy 解释是它们为一系列条目提供了成对的 i, j 索引。这就是为什么这对列表必须具有相同的长度。然而，这并非我们的本意，因为我们正在寻找一个子矩阵。

一种简单的方法如下。我们首先通过取 {v3} 的行的子集来创建一个子矩阵，然后动态地通过取列的子集进一步创建另一个子矩阵。

```
In [63]: A[[1,3]][:,[0,2]]
```

```
Out[63]: array([[ 4,  6],
                [12, 14]])
```

有更高效的方法来实现相同的结果。*convenience function* np.ix_() 允许我们使用便捷函数提取子矩阵
列表，通过创建一个中间 *mesh* 对象。

np.ix_()
网格

```
In [64]: idx = np.ix_([1,3],[0,2,3])
         A[idx]
```

```
Out[64]: array([[ 4,  6,  7],
                [12, 14, 15]])
```

另外，我们可以使用切片有效地对矩阵进行子集化。切片1:4:2捕获序列中的第二项和第四项，而切片0:3:2捕获第一项和第三项（切片序列中的第三个元素是步长）。

```
In [65]: A[1:4:2,0:3:2]
```

```
Out[65]: array([[ 4,  6],
                [12, 14]])
```

为什么我们可以直接使用切片来获取子矩阵，却不能使用列表呢？这是因为它们是不同的Python类型，并且被numpy区别对待。切片可以用于从任意序列中提取对象，如字符串、列表和元组，而使用列表进行索引则更为有限。

布尔索引

在numpy中，*Boolean*是一种类型，它等同于True或False（，也分别表示为1和0的布尔表示）。下一行创建了一个由0组成的向量，以布尔值表示，长度等于A的第一维。

```
In [66]: keep_rows = np.zeros(A.shape[0], bool)
         keep_rows
```

```
Out[66]: array([False,  False, False,  False])
```

我们现在将其中两个元素设置为 True。

```
In [67]: keep_rows[[1,3]] = True
         keep_rows
```

```
Out[67]: array([False,  True, False,  True])
```

注意，keep_rows 的元素在视为整数时，与 np.array([0,1,0,1]) 的值相同。下面，我们使用 == 来验证它们的相等性。当应用于两个数组时，== 操作按元素应用。

```
In [68]: np.all(keep_rows == np.array([0,1,0,1]))
```

```
Out[68]: True
```

(这里，函数 np.all() 检查了数组 np.all() 的所有条目是否都是 True。一个类似的函数 np.any() 可以用来检查数组是否有任何 np.any() 条目是 True。)

然而，尽管根据==，np.array([0,1,0,1])和keep_rows是相等的，但它们索引不同的行集！前者检索了A的第一、第二、第一和第二行。

```
In [69]: A[np.array([0,1,0,1])]
```

```
Out[69]: array([[0, 1, 2, 3],
                [4, 5, 6, 7],
                [0, 1, 2, 3],
                [4, 5, 6, 7]])
```

相比之下，keep_rows仅检索A的第二行和第四行——即布尔值等于TRUE的行。

```
In [70]: A[keep_rows]
```

```
Out[70]: array([[ 4,  5,  6,  7],
                [12, 13, 14, 15]])
```

这个示例表明，numpy 对布尔值和整数的处理方式不同。

我们再次利用 np.ix_() 函数来创建一个包含第二行和第四行，以及第一列、第三列和第四列的网格。这次，我们将该函数应用于布尔值，而非列表。

```
In [71]: keep_cols = np.zeros(A.shape[1], bool)
         keep_cols[[0, 2, 3]] = True
         idx_bool = np.ix_(keep_rows, keep_cols)
         A[idx_bool]
```

```
Out[71]: array([[ 4,  6, 7],
                [12, 14, 15]])
```

我们还可以在传递给 np.ix_() 的参数中，将列表与布尔数组混合使用：

```
In [72]: idx_mixed = np.ix_([1,3], keep_cols)
         A[idx_mixed]
```

```
Out[72]: array([[ 4,  6, 7],
                [12, 14, 15]])
```

有关在 numpy 中索引的更多详情，读者可参阅之前提到的 numpy 教程。

2.3.7 Loading Data

数据集通常包含不同类型的数据，并且可能带有与行或列相关的名称。因此，它们通常最适合使用 *data frame* 来容纳。我们可以将数据框视为一个由相同长度的数组组成的序列；这些数组即列。不同数组中的条目可以组合成行。pandas 库可用于创建和操作数据框对象。

数据框

读入数据集

大多数分析的第一步是将数据集导入到 Python 中。在尝试加载数据集之前，我们必须确保 Python 知道在哪里找到包含该数据集的文件。如果文件与这个 notebook 文件位于同一位置，那就没问题了。否则，命令 os.chdir() 可以用于*change directory*。（在调用os.chdir()之前，你需要先调用import os。）

os.chdir()

我们将首先读取Auto.csv，它可在书籍网站上获取。这是一个逗号分隔的文件，可以使用pd.read_csv()读取：

pd.read_csv()

In [73]:
```
import pandas as pd
Auto = pd.read_csv('Auto.csv')
Auto
```

该书籍网站还提供了该数据的以空白字符分隔的版本，称为Auto.data。可以按以下方式读取：

In [74]:
```
Auto = pd.read_csv('Auto.data', delim_whitespace=True)
```

Auto.csv 和 Auto.data 都只是文本文件。在将数据加载到 Python 之前，最好使用文本编辑器或其他软件（例如 Microsoft Excel）查看一下。

我们现在来看与变量horsepower相对应的Auto列：

In [75]:
```
Auto['horsepower']
```

Out[75]:
```
0      130.0
1      165.0
2      150.0
3      150.0
4      140.0
       ...
392     86.00
393     52.00
394     84.00
395     79.00
396     82.00
Name: horsepower, Length: 397, dtype: object
```

我们看到这一列的 dtype 是 object。事实证明，在读取数据时，horsepower 列的所有值都被解释为字符串。我们可以通过查看唯一值来找出原因。

In [76]:
```
np.unique(Auto['horsepower'])
```

为了节省空间，我们省略了上一代码块的输出。我们看到问题出在值?上，它被用来编码缺失值。

为了解决这个问题，我们必须为 pd.read_csv() 提供一个名为 na_values 的参数。现在，文件中的每个 ? 实例都被替换为值 np.nan，这意味着 *not a number*：

```
In [77]: Auto = pd.read_csv('Auto.data',
                            na_values=['?'],
                            delim_whitespace=True)
         Auto['horsepower'].sum()
```

```
Out[77]: 40952.0
```

Auto.shape 属性告诉我们数据有 397 个观测值（即行）和九个变量（即列）。

```
In [78]: Auto.shape
```

```
Out[78]: (397, 9)
```

处理缺失数据有多种方法。在这种情况下，由于只有五行包含缺失观测值，我们选择使用Auto.dropna()方法来简单地删除这些行。

.dropna()

```
In [79]: Auto_new = Auto.dropna()
         Auto_new.shape
```

```
Out[79]: (392, 9)
```

选择行和列的基础知识

我们可以使用Auto.columns来检查变量名。

```
In [80]: Auto = Auto_new # overwrite the previous value
         Auto.columns
```

```
Out[80]: Index(['mpg', 'cylinders', 'displacement', 'horsepower',
                'weight', 'acceleration', 'year', 'origin', 'name'],
                dtype='object')
```

访问数据帧的行和列与访问数组的行和列类似，但不完全相同。回想一下，方法[]的第一个参数总是应用于数组的行。类似地，向方法[]传递一个切片会创建一个数据帧，其*rows*由该切片确定：

```
In [81]: Auto[:3]
```

```
Out[81]:       mpg   cylinders   displacement   horsepower   weight     ...
         0    18.0           8          307.0        130.0   3504.0     ...
         1    15.0           8          350.0        165.0   3693.0     ...
         2    18.0           8          318.0        150.0   3436.0     ...
```

类似地，可以使用布尔值数组来筛选行：

```
In [82]: idx_80 = Auto['year'] > 80
         Auto[idx_80]
```

然而，如果我们向[]方法传入一个字符串列表，那么我们将获得一个包含相应*columns*集合的数据框。

```
In [83]: Auto[['mpg', 'horsepower']]
```

```
Out[83]:         mpg     horsepower
        0       18.0    130.0
        1       15.0    165.0
        2       18.0    150.0
        3       16.0    150.0
        4       17.0    140.0
        ...     ...     ...
        392     27.0    86.0
        393     44.0    52.0
        394     32.0    84.0
        395     28.0    79.0
        396     31.0    82.0
        392 rows x 2 columns
```

由于我们加载数据框时未指定 *index* 列，因此行使用 0 到 396 的整数标注。

```
In [84]: Auto.index
```

```
Out[84]: Int64Index([  0,   1,   2,   3,   4,   5,   6,   7,   8,   9,
                     ...
                     387, 388, 389, 390, 391, 392, 393, 394, 395, 396],
                    dtype='int64', length=392)
```

我们可以使用 `set_index()` 方法利用内容 `.set_index` 重命名行 `()` `Auto['name']`的。

```
In [85]: Auto_re = Auto.set_index('name')
         Auto_re
```

```
Out[85]:                              mpg   cylinders   displacement   ...
                            name

        chevrolet chevelle malibu   18.0           8          307.0   ...
                  buick skylark 32  15.0           8          350.0   ...
                 plymouth satellite 18.0           8          318.0   ...
                      amc rebel sst 16.0           8          304.0   ...
```

```
In [86]: Auto_re.columns
```

```
Out[86]: Index(['mpg', 'cylinders', 'displacement', 'horsepower',
                'weight', 'acceleration', 'year', 'origin'],
               dtype='object')
```

我们看到列 `'name'` 不再存在。既然索引已设置为 `name`，我们现在可以通过 `Auto` 的 `loc[]` 方法使用 `name` 访问数据框的行：

`.loc[]`

```
In[87]: rows = ['amc rebel sst', 'ford torino']
        Auto_re.loc[rows]
```

```
Out[87]:                 mpg   cylinders   displacement horsepower   ...
              name
        amc rebel sst   16.0           8          304.0      150.0   ...
          ford torino   17.0           8          302.0      140.0   ...
```

作为使用索引名称的替代方法，我们可以使用 `iloc[]` 方法来检索 `Auto` 的第 4 行和第 5 行：

.iloc[]

```
In[88]: Auto_re.iloc[[3,4]]
```

我们还可以使用它来检索Auto_re的第1列、第3列和第4列：

```
In[89]: Auto_re.iloc[:,[0,2,3]]
```

我们可以使用一次对 `iloc[]` 的调用来提取第4行和第5行，以及第1列、第3列和第4列：

```
In[90]: Auto_re.iloc[[3,4],[0,2,3]]
```

```
Out[90]:                 mpg   displacement   horsepower
              name
        amc rebel sst   16.0          304.0        150.0
          ford torino   17.0          302.0        140.0
```

索引条目不必唯一：数据框中有几辆名为 `ford galaxie 500` 的汽车。

```
In[91]: Auto_re.loc['ford galaxie 500', ['mpg', 'origin']]
```

```
Out[91]:                      mpg    origin
                  name
        ford galaxie 500    15.0         1
        ford galaxie 500    14.0         1
        ford galaxie 500    14.0         1
```

关于选择行和列的更多内容

现在假设我们想要创建一个数据框，其中包含 `year` 大于 80 的汽车子集中的 `weight` 和 `origin`，即那些 1980 年后制造的汽车。为此，我们首先创建一个布尔数组来索引行。`loc[]` 方法既允许布尔条目，也允许字符串：

```
In[92]: idx_80 = Auto_re['year'] > 80
        Auto_re.loc[idx_80, ['weight', 'origin']]
```

为了更简洁地完成此操作，我们可以使用一个称为 `lambda` 函数。

```
n[93]: Auto_re.loc[lambda df: df['year'] > 80, ['weight', 'origin']]
```

`lambda` 调用创建了一个函数，该函数接受一个参数，这里是 `df`，并返回 `df['year']>80`。由于它是在 `loc[]` 方法内部创建的，用于

数据框 Auto_re，该数据框将作为提供的参数。作为使用 lambda 的另一个例子，假设我们想要所有1980年以后制造且每加仑燃油行驶里程超过30英里的汽车：

In [94]:
```
Auto_re.loc[lambda df: (df['year'] > 80) & (df['mpg'] > 30),
            ['weight', 'origin']
           ]
```

符号 & 计算逐元素的 *and* 操作。再举一个例子，假设我们想要检索所有 displacement 小于 300 的 Ford 和 Datsun 汽车。我们使用 str.contains() 方法检查每个 name 条目是否包含字符串 ford 或 datsun。

.str.contains()

index 数据框的属性：

In [95]:
```
Auto_re.loc[lambda df: (df['displacement'] < 300)
                       & (df.index.str.contains('ford')
                       | df.index.str.contains('datsun')),
            ['weight', 'origin']
           ]
```

这里，符号 | 计算逐元素的 *or* 操作。

总之，可以使用一组强大的操作来索引数据帧的行和列。对于基于整数的查询，请使用iloc[]方法。对于字符串和布尔选择，请使用loc[]方法。对于筛选行的函数式查询，请在rows参数中使用带有函数（通常是lambda）的loc[]方法。

2.3.8 For Loops

for 循环是许多语言中的标准工具，它在改变代码内部不同值的同时反复评估 for 某段代码。例如，假设我们遍历列表中的元素并计算它们的总和。

In [96]:
```
total = 0
for value in [3,2,19]:
    total += value
print('Total is: {0}'.format(total))
```

```
Total is: 24
```

在包含 for 语句的行下方缩进的代码，会针对 for 语句中指定的序列中的每个值运行。循环会在单元格结束或代码缩进与原始 for 语句处于同一级别时结束。我们看到，上方最后一行打印总数的代码仅在 for 循环终止后执行一次。循环可以通过额外的缩进来嵌套。

In [97]:
```
total = 0
for value in [2,3,19]:
    for weight in [3, 2, 1]:
        total += value * weight
print('Total is: {0}'.format(total))
```

```
Total is: 144
```

上文中，我们对value和weight的每种组合进行了求和。我们还利用了Python中的*increment*表示法：表达式a += b等同于 a = a + b。除了作为便捷的表示法外，这在计算密集型任务中也能节省时间，因为无需显式创建 a+b 的中间值。

增量

也许更常见的任务是求和{v*}对。例如，要计算一个随机变量的平均值，该变量分别以0.2、0.3、0.5的概率取值2、3或19，我们将计算加权和。诸如此类的任务通常可以使用zip()函数来完成，该函数遍历一系列元组。

zip()

In [98]:
```
total = 0
for value, weight in zip([2,3,19],
                         [0.2,0.3,0.5]):
    total += weight * value
print('Weighted average is: {0}'.format(total))
```

```
Weighted average is: 10.8
```

字符串格式化

在上面的代码块中，我们还打印了一个显示总和的字符串。然而，对象 total 是一个整数而非字符串。将某个值插入字符串是一项常见任务，借助 Python 中一些强大的字符串格式化工具可以轻松完成。许多数据清理任务都涉及对字符串进行操作和以编程方式生成字符串。

例如，我们可能希望遍历数据框的列并打印每列中缺失值的百分比。让我们创建一个数据框 D，其中列中的20%条目的缺失，即设置为 np.nan。

np.nan

我们将使用 rng.standard_normal() 从均值为 0、方差为 1 的正态分布中生成 D 中的值，然后使用 rng.choice() 覆盖一些随机条目。

In [99]:
```
rng = np.random.default_rng(1)
A = rng.standard_normal((127, 5))
M = rng.choice([0, np.nan], p=[0.8,0.2], size=A.shape)
A += M
D = pd.DataFrame(A, columns=['food',
                             'bar',
                             'pickle',
                             'snack',
                             'popcorn'])
D[:3]
```

Out[99]:

	food	bar	pickle	snack	popcorn
0	0.345584	0.821618	0.330437	-1.303157	NaN
1	NaN	-0.536953	0.581118	0.364572	0.294132
2	NaN	0.546713	NaN	-0.162910	-0.482119

In [100]:
```
for col in D.columns:
    template = 'Column "{0}" has {1:.2%} missing values'
    print(template.format(col,
          np.isnan(D[col]).mean()))
```

```
Column "food" has 16.54% missing values
Column "bar" has 25.98% missing values
Column "pickle" has 29.13% missing values
Column "snack" has 21.26% missing values
Column "popcorn" has 22.83% missing values
```

我们看到 template.format() 方法期望两个参数 {0} 和 {1:.2%}，后者包含一些格式信息。具体来说，它指定第二个参数应以带有两位小数的百分比表示。

参考文档docs.python.org/3/library/string.html包含了许多有益且更复杂的示例。

2.3.9 Additional Graphical and Numerical Summaries

我们可以使用 ax.plot() 或 ax.scatter() 函数来显示定量变量。然而，简单输入变量名会产生错误信息，因为 Python 不知道在 Auto 数据集中查找这些变量。

In [101]:
```
fig, ax = subplots(figsize=(8, 8))
ax.plot(horsepower, mpg, 'o');
```

```
NameError: name 'horsepower' is not defined
```

我们可以通过直接访问列来解决这个问题：

In [102]:
```
fig, ax = subplots(figsize=(8, 8))
ax.plot(Auto['horsepower'], Auto['mpg'], 'o');
```

或者，我们可以使用plot()方法并通过调用Auto.plot()来实现。
使用此方法，可以通过名称访问变量。数据框的绘图方法返回一个熟悉的 .plot()
物体：一个坐标轴。我们可以像之前那样使用它来更新图表：

In [103]:
```
ax = Auto.plot.scatter('horsepower', 'mpg');
ax.set_title('Horsepower vs. MPG')
```

如果我们想要保存包含给定坐标轴的图形，可以通过访问figure属性来找到相关的图形：

In [104]:
```
fig = ax.figure
fig.savefig('horsepower_mpg.png');
```

我们可以进一步指示数据框绘制到特定的坐标轴对象。在这种情况下，对应的 plot() 方法将返回我们作为参数传入的修改后的坐标轴。请注意，当我们请求一维网格的图表时，对象 axes 同样是一维的。我们将散点图放置在图中一行三个图表的中间图表中。

In [105]:
```
fig, axes = subplots(ncols=3, figsize=(15, 5))
Auto.plot.scatter('horsepower', 'mpg', ax=axes[1]);
```

另请注意，数据框的列可以作为属性访问：尝试输入Auto.horsepower。

我们现在考虑 cylinders 变量。输入 Auto.cylinders.dtype 显示它被当作定量变量处理。然而，由于该变量可能的值数量很少，我们可能希望将其视为定性变量。下面，我们用 Auto.cylinders 的分类版本替换 cylinders 列。函数 pd.Series() 归功于其名称源于以下事实：pandas 常用于时间序列应用。

pd.Series()

In [106]:
```
Auto.cylinders = pd.Series(Auto.cylinders, dtype='category')
Auto.cylinders.dtype
```

Now that cylinders is qualitative, we can display it using the boxplot() method.

.boxplot()

In [107]:
```
fig, ax = subplots(figsize=(8, 8))
Auto.boxplot('mpg', by='cylinders', ax=ax);
```

The hist() method can be used to plot a *histogram*.

.hist()

In [108]:
```
fig, ax = subplots(figsize=(8, 8))
Auto.hist('mpg', ax=ax);
```

The color of the bars and the number of bins can be changed:

In [109]:
```
fig, ax = subplots(figsize=(8, 8))
Auto.hist('mpg', color='red', bins=12, ax=ax);
```

See Auto.hist? for more plotting options.

We can use the pd.plotting.scatter_matrix() function to create a *scatterplot matrix* to visualize all of the pairwise relationships between the columns in a data frame.

pd.plotting.scatter_matrix()

In [110]:
```
pd.plotting.scatter_matrix(Auto);
```

We can also produce scatterplots for a subset of the variables.

In [111]:
```
pd.plotting.scatter_matrix(Auto[['mpg',
                                 'displacement',
                                 'weight']]);
```

The describe() method produces a numerical summary of each column in a data frame.

.describe()

In [112]:
```
Auto[['mpg', 'weight']].describe()
```

We can also produce a summary of just a single column.

In [113]:
```
Auto['cylinders'].describe()
Auto['mpg'].describe()
```

To exit Jupyter, select File / Close and Halt.

2.4 练习

Conceptual

1. 对于(a)至(d)各部分，指出我们通常预期灵活统计学习方法的性能会比不灵活方法更好还是更差。证明你的答案。

(a) 样本量 n 非常大，而预测变量的数量 p 很少。 (b) 预测变量的数量 p 非常大，而观测值的数量 n 很少。 (c) 预测变量与响应之间的关系高度非线性。 (d) 误差项的方差，即 $\sigma^2 = \text{Var}(\epsilon)$，非常高。

2. 解释每种情景是分类问题还是回归问题，并指出我们最关注的是推断还是预测。最后，提供n和p。

(a) 我们收集了美国排名前500家公司的数据。对于每家公司，我们记录了利润、员工人数、行业和CEO薪资。我们感兴趣的是了解哪些因素影响CEO薪资。(b) 我们正在考虑推出一款新产品，并希望知道它会是$success$还是$failure$。我们收集了20款先前推出的类似产品的数据。对于每款产品，我们记录了它是成功还是失败、产品定价、营销预算、竞争价格以及其他十个变量。(c) 我们感兴趣的是预测美元/欧元汇率相对于全球股市周变化的百分比变化。因此，我们收集了2012年全年的每周数据。对于每一周，我们记录了美元/欧元的百分比变化、美国市场的百分比变化、英国市场的百分比变化以及德国市场的百分比变化。

3. 我们现在重新审视偏差-方差分解。

(a) 在同一图表中，给出典型（平方）偏差、方差、训练误差、测试误差和贝叶斯（或不可约）误差曲线的草图，随着我们从较不灵活的统计学习方法过渡到更灵活的方法。x轴应表示方法的灵活性程度，y轴应表示每条曲线的值。应包含五条曲线。确保对每一条进行标注。

(b) 解释为何这五条曲线在(a)部分中呈现出所示形状。

4. 现在，你将思考一些统计学习的现实生活应用。

(a) 描述*classification*可能有用的三个现实生活应用。描述响应以及预测变量。每个应用的目标是推断还是预测？解释你的答案。(b) 描述*regression*可能有用的三个现实生活应用。描述响应以及预测变量。每个应用的目标是推断还是预测？解释你的答案。(c) 描述*cluster analysis*可能有用的三个现实生活应用。

5. 对于回归或分类而言，非常灵活（相对于不那么灵活）的方法有哪些优点和缺点？在什么情况下，更灵活的方法可能比不那么灵活的方法更受青睐？又何时可能更倾向于采用不那么灵活的方法？

6. 描述参数与非参数统计学习方法之间的差异。参数方法在回归或分类中的优势（相对于非参数方法）是什么？其劣势又是什么？

7. 下表提供了一个包含六个观测值、三个预测因子和一个定性响应变量的训练数据集。

Obs.	X_1	X_2	X_3	Y
1	0	3	0	Red
2	2	0	0	Red
3	0	1	3	Red
4	0	1	2	Green
5	−1	0	1	Green
6	1	1	1	Red

假设我们希望使用该数据集在$X_1 = X_2 = X_3 = 0$时利用K近邻对Y进行预测。

(a) 计算每个观测值与测试点 $X_1 = X_2 = X_3 = 0$ 之间的欧几里得距离。

(b) 当 $K = 1$ 时，我们的预测是什么？为什么？

(c) 我们对$K = 3$的预测是什么？为什么？

(d) 如果此问题中的贝叶斯决策边界高度非线性，那么我们会预期K的*best*值是大还是小？为什么？

Applied

8. 本练习涉及College数据集，该数据集可在书籍网站的College.csv文件中找到。它包含了美国777所不同大学和学院的多个变量。这些变量是

- Private： 公共/私有指示符
- Apps ： 收到的申请数量
- Accept ：接受的申请人数
- Enroll ： 新入学学生人数
- Top10perc ：高中班级前10%的新生
- Top25perc ： 来自高中班级前25%的新生
- F.Undergrad ：全日制本科生人数
- P.Undergrad ： 非全日制本科生人数
- Outstate ： {v*}州外学费
- Room.Board ： 食宿费用
- Books ： 预估书籍费用
- Personal ： 估计个人支出
- PhD ： 拥有博士学位的教职员工百分比
- Terminal ： 拥有最终学位的教师百分比
- S.F.Ratio ：学生/教师比例
- perc.alumni 校友捐赠百分比
- Expend ：每名学生的教学支出
- Grad.Rate ：毕业率

在将数据读入 Python 之前，可以在 Excel 或文本编辑器中查看。

(a) 使用 pd.read_csv() 函数将数据读入 Python。将加载的数据称为 college。确保已将目录设置为数据的正确位置。

(b) 通过创建一个新单元格并仅输入代码 college 来查看笔记本中使用的数据。你应该会注意到第一列只是每所大学的名称，列名类似于 Unnamed: 0。我们实际上并不希望 pandas 将其视为数据。然而，将这些名称保留下来以备后续使用可能会很方便。尝试以下命令，并同样查看生成的数据框：

```
college2 = pd.read_csv('College.csv', index_col=0)
college3 = college.rename({'Unnamed: 0': 'College'},
                          axis=1)
college3 = college3.set_index('College')
```

这已将文件中的第一列用作数据框的index。这意味着pandas为每一行赋予了与相应大学相对应的名称。现在您应该看到第一个数据列是Private。请注意，学院名称出现在表格的左侧。我们还在上面引入了一个新的Python对象：一个*dictionary*，其指定方式为

(key，value) 对。将您修改后的数据与以下内容保持一致：

dictionary

```
college = college3
```

(使用{c)}的describe()方法生成数据集中变量的数值摘要。

(d) 使用 pd.plotting.scatter_matrix() 函数生成前几列 [Top10perc，Apps，Enroll] 的散点图矩阵。回想一下，你可以使用 A[C] 来引用数据框 A 的列列表 C。

(e) 使用college的boxplot()方法来生成Outstate与Private的并排箱线图。

(f) 创建一个新的定性变量，称为Elite，通过将Top10perc变量根据来自高中班级前10%的学生比例是否超过50%分为两组来进行*binning*。

```
college['Elite'] = pd.cut(college['Top10perc'],
                          [0,0.5,1],
                          labels=['No', 'Yes'])
```

使用college['Elite']的value_counts()方法来查看有多少精英大学。最后，再次使用boxplot()方法生成Outstate与Elite的并列箱线图。

(g) 使用 college 的 plot.hist() 方法，对几个定量变量生成一些具有不同箱数的直方图。命令 plt.subplots(2，2) 可能有用：它会把绘图窗口分成四个区域，以便可以同时绘制四幅图。通过更改参数，你可以按其他组合来划分屏幕。

（h）继续探索数据，并简要总结你的发现。

9. 本练习涉及实验室中研究的 Auto 数据集。确保已从数据中移除缺失值。

(a) 哪些预测变量是定量的，哪些是定性的？(b) 每个定量预测变量的*range*是多少？你可以使用numpy中的min()和max()方法来回答这个问题。

.min()
.max()

(c) 每个定量预测变量的均值和标准差是多少？

(d) 现在删除第10到第85个观测值。在剩余的数据子集中，每个预测变量的范围、均值和标准差分别是多少？（e）使用完整数据集，通过散点图或您选择的其他工具，以图形方式研究预测变量。创建一些突出显示预测变量之间关系的图表。对您的发现进行评论。（f）假设我们希望基于其他变量来预测燃气里程（`mpg`）。您的图表是否表明其他任何变量可能有助于预测`mpg`？请证明您的答案。

10. 本练习涉及`Boston`房屋数据集。

(a) 首先，加载 `Boston` 数据集，它是 `ISLP` 库的一部分。(b) 这个数据集有多少行？多少列？行和列分别代表什么？(c) 绘制该数据集中预测变量（列）之间的一些成对散点图。描述你的发现。(d) 是否有任何预测变量与人均犯罪率相关？如果有，请解释这种关系。(e) 波士顿的郊区中是否有犯罪率特别高的？税率？师生比？评论每个预测变量的范围。(f) 该数据集中有多少郊区与查尔斯河相邻？(g) 该数据集中城镇的师生比的中位数是多少？(h) 波士顿哪个郊区的自住房屋中位数价值最低？该郊区的其他预测变量的值是多少，这些值与那些预测变量的总体范围相比如何？评论你的发现。(i) 在该数据集中，有多少郊区的平均每户房间数超过七间？超过八间？评论平均每户房间数超过八间的郊区。

3
线性回归



本章讨论的是*linear regression*，一种非常简单的监督学习方法。特别地，线性回归是预测定量响应的有用工具。它已经存在很长时间，是无尽教科书的话题。尽管与本书后面章节中描述的某些更现代的统计学习方法相比，它可能显得有些枯燥，但线性回归仍然是一种有用且广泛使用的统计学习方法。此外，它还是新方法的良好起点：正如我们将在后面章节中看到的，许多花哨的统计学习方法可以看作是线性回归的概括或扩展。因此，在学习更复杂的学习方法之前，对线性回归有良好的理解的重要性怎么强调都不为过。在本章中，我们回顾线性回归模型背后的一些关键思想，以及最常用于拟合该模型的最小二乘法。

回忆第2章中的`Advertising`数据。图2.1展示了某一特定产品的`sales`(以千单位计)作为广告预算（以千美元计）的函数关系，涉及`TV`、`radio`和`newspaper`媒体。假设作为统计顾问，我们被要求基于这些数据为下一年提出一个能带来高产品销量的营销计划。为了提供这样的建议，哪些信息会是有用的？以下是我们可能试图解决的一些重要问题：

1. *Is there a relationship between advertising budget and sales?*
   Our first goal should be to determine whether the data provide evidence of an association between advertising expenditure and sales. If the evidence is weak, then one might argue that no money should be spent on advertising!

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_3

2. *How strong is the relationship between advertising budget and sales?* 假设广告与销售额之间存在关系，我们想知道这种关系的强度。了解广告预算是否能提供大量关于产品销售额的信息?

3. *Which media are associated with sales?* 电视、广播和报纸这三种媒体都与销售额相关，还是只有一两种媒体相关？要回答这个问题，我们必须找到一种方法，在同时在三种媒体上投入资金的情况下，分离出每种媒体对销售额的单独贡献。

4. *How large is the association between each medium and sales?* 在特定媒体上每花费一美元广告费，销售额将增加多少？我们能多准确地预测这一增长量?

5. *How accurately can we predict future sales?* 对于任何给定的电视、广播或报纸广告水平，我们对销售额的预测是什么，这个预测的准确性如何?

6. *Is the relationship linear?* 如果在各种媒体上的广告支出与销售额之间大致存在直线关系，那么线性回归是一种合适的工具。如果不是，那么仍然可能可以通过转换预测变量或响应变量来使用线性回归。

7. *Is there synergy among the advertising media?* 也许在电视广告上花费$50,000，在广播广告上花费$50,000，与单独将$100,000分配给电视或广播相比，会带来更高的销售额。在市场营销中，这被称为*synergy*效应，而在统计学中，它被称为*interaction*效应。

synergy
interaction

事实证明，线性回归可用于解答这些问题中的每一个。我们将首先在一般背景下讨论所有这些问题，然后在第3.4节中回到这个具体背景下来探讨它们。

3.1 简单线性回归

Simple linear regression 名副其实：它非常直接
基于单个预测变量 X 预测定量响应 Y 的方法。它假设 X 和 Y 之间近似存在线性关系。在数学上，我们可以将这种线性关系写作

简单线性回归

$$Y \approx \beta_0 + \beta_1 X. \tag{3.1}$$

你可能会把“≈”读作*“is approximately modeled as”*。我们有时会通过说我们在*regressing* Y *on* X (或Y *onto* X)来描述(3.1)。

例如，X 可能代表 TV 广告，而 Y 可能代表 sales。然后，我们可以通过拟合模型将 sales 回归到 TV 上

$$\texttt{sales} \approx \beta_0 + \beta_1 \times \texttt{TV}.$$

在公式3.1中，β_0和β_1是表示线性模型中*intercept*和*slope*项的两个未知常数。β_0和β_1一起是
被称为模型 *coefficients* 或 *parameters*。一旦我们使用了我们的
训练数据以生成模型系数的估计值$\hat{\beta}_0$和$\hat{\beta}_1$，我们可以通过计算特定电视广告值的基础来预测未来的销售。

截距斜率

系数参数

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x, \tag{3.2}$$

其中 $\hat{y}$ 表示基于 $X = x$ 对 Y的预测。这里我们使用一个 *hat* 符号 来表示未知参数或系数的估计值，或表示响应变量的预测值。

3.1.1 Estimating the Coefficients

在实践中，β_0 和 β_1 是未知的。因此，在我们能够使用 (3.1) 进行预测之前，必须先利用数据来估计这些系数。让

$$(x_1, y_1),\ (x_2, y_2), \ldots,\ (x_n, y_n)$$

表示 n 个观测对，其中每一对由 X 的一个测量值和 Y的一个测量值组成。在 Advertising 示例中，该数据集由 $n =$ 个不同市场中的电视广告预算和产品销售额组成。（回想一下，数据展示在图2.1中。）我们的目标是获得系数估计 $\hat{\beta}_0$ 和 $\hat{\beta}_1$，使得线性模型 (3.1) 能够很好地拟合可用数据——也就是说，使得对于 $i = 1, \ldots, n$，有 $y_i \approx \hat{\beta}_0 + \hat{\beta}_1 x_i$。换句话说，我们希望找到一个截距 $\hat{\beta}_0$ 和一个斜率 $\hat{\beta}_1$，使得由此得到的直线尽可能接近这 $n =$ 个数据点。衡量 *closeness* 的方式有很多种。然而，迄今为止最常见的方法涉及最小化 *least squares* 准则，我们取

最小二乘法

本章将讨论该方法。其他方法将在第6章中考虑。

设 $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$ 为基于 X 的第 i 个值对 Y的预测。则 $e_i = y_i - \hat{y}_i$ 表示第 i 个 *residual*——这是以下两者之间的差：
第i个观测响应值和第i个由我们的线性模型预测的响应值。我们将
residual sum of squares (RSS)定义为

残差

residual sum of squares

$$\text{RSS} = e_1^2 + e_2^2 + \cdots + e_n^2,$$

或等价地表示为

$$\text{RSS} = (y_1 - \hat{\beta}_0 - \hat{\beta}_1 x_1)^2 + (y_2 - \hat{\beta}_0 - \hat{\beta}_1 x_2)^2 + \cdots + (y_n - \hat{\beta}_0 - \hat{\beta}_1 x_n)^2. \tag{3.3}$$

最小二乘法通过选择$\hat{\beta}_0$和$\hat{\beta}_1$来最小化RSS。利用一些微积分知识，可以证明最小化点是

$$\begin{aligned} \hat{\beta}_1 &= \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}, \\ \hat{\beta}_0 &= \bar{y} - \hat{\beta}_1 \bar{x}, \end{aligned} \tag{3.4}$$

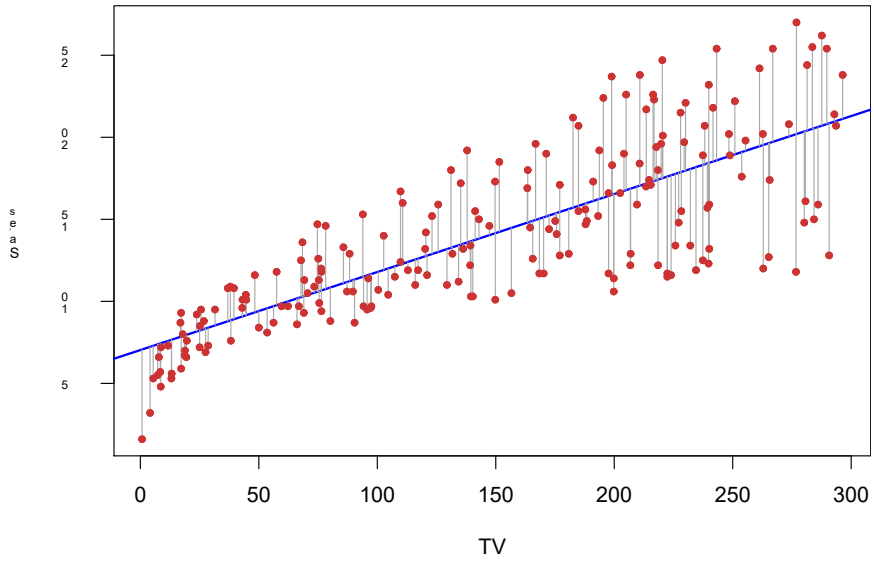


图3.1。*For the* `Advertising` *data, the least squares fit for the regression of* `sales` *onto* `TV` *is shown. The fit is found by minimizing the residual sum of squares. Each grey line segment represents a residual. In this case a linear fit captures the essence of the relationship, although it overestimates the trend in the left of the plot.*

其中 $\bar{y} \equiv \frac{1}{n}\sum_{i=1}^{n} y_i$ 和 $\bar{x} \equiv \frac{1}{n}\sum_{i=1}^{n} x_i$ 是样本均值。换句话说，(3.4) 定义了简单线性回归的 *least squares coefficient estimates*。

图3.1展示了针对 `Advertising` 数据的简单线性回归拟合，其中 $\hat{\beta}_0 = 7.03$ ，$\hat{\beta}_1 = 0.0475$。换言之，根据这一近似，额外花费 $1,000 于电视广告与销售大约额外 47.5 个单位的产品相关联。在图3.2中，我们使用以 `sales` 为响应变量、`TV` 为预测变量的广告数据，计算了 β_0 和 β_1 多个取值下的RSS。在每个图中，红点表示由(3.4)给出的最小二乘估计对$(\hat{\beta}_0, \hat{\beta}_1)$。这些值显然使RSS最小化。

3.1.2 Assessing the Accuracy of the Coefficient Estimates

回想(2.1)中的假设，我们假定X与Y之间的关系形式为$Y = f(X) + \epsilon$，其中f为某个未知函数，ϵ是均值为零的随机误差项。若要用线性函数来近似f，则我们可以将此关系表示为：

$$Y = \beta_0 + \beta_1 X + \epsilon. \tag{3.5}$$

这里 β_0 是截距项——即当 $X =$ 为 0 时 Y 的期望值，而 β_1 是斜率——X 每增加一个单位时 Y 的平均增加量。误差项是对这个简单模型所遗漏内容的统称：真实关系很可能不是线性的，可能存在其他导致 Y 变化的变量，并且可能存在测量误差。我们通常假设误差项独立于 X。

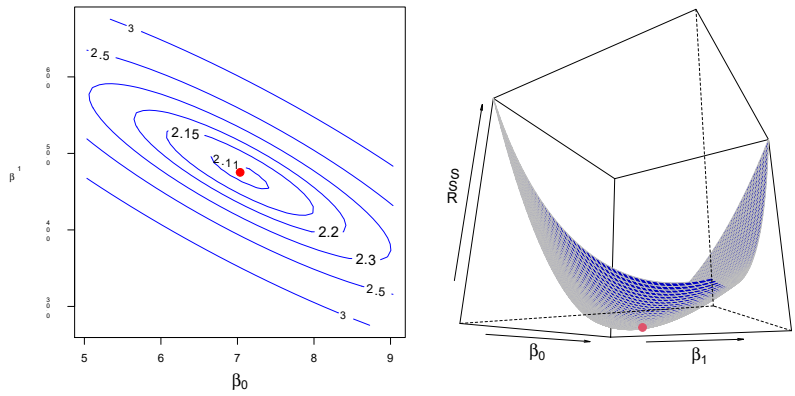


图3.2。*Contour and three-dimensional plots of the RSS on the* `Advertising` *data, using* `sales` *as the response and* `TV` *as the predictor. The red dots correspond to the least squares estimates* $\hat{\beta}_0$ *and* $\hat{\beta}_1$*, given by (3.4).*

由(3.5)给出的模型定义了*population regression line*，该是 X 和 Y 之间真实关系的最佳线性近似。[1] 最小二乘回归系数估计 (3.4) 刻画了 *least squares line* (3.2)。图 3.3 的左侧面板展示了这些

总体回归线

在简单模拟示例中的两行。我们创建了100个随机的X，并从模型中生成了100个相应的Y。

最小二乘直线

$$Y = 2 + 3X + \epsilon, \tag{3.6}$$

其中 ϵ 是从均值为零的正态分布中生成的。图3.3左侧面板中的红线显示了 *true* 关系，$f(X) = 2 + 3X$，而蓝线是基于观测数据的最小二乘估计。对于真实数据，真实关系通常是未知的，但最小二乘线始终可以使用(3.4)中给出的系数估计来计算。换句话说，在实际应用中，我们可以获得一组观测值，从中计算最小二乘线；然而，总体回归线是不可观测的。在图3.3的右侧面板中，我们从(3.6)给出的模型生成了十个不同的数据集，并绘制了相应的十条最小二乘线。请注意，从同一真实模型生成的不同数据集会产生略有不同的最小二乘线，但未观测到的总体回归线不会改变。

乍一看，总体回归线与最小二乘线之间的差异可能显得微妙且令人困惑。我们仅有一个数据集，因此两条不同的线描述预测变量与响应变量之间的关系意味着什么？从根本上说，这两条线的概念是使用样本信息来估计大规模总体特征的标准统计方法的自然延伸。例如，假设我们想知道

[1]The assumption of linearity is often a useful working model. However, despite what many textbooks might tell us, we seldom believe that the true relationship is linear.

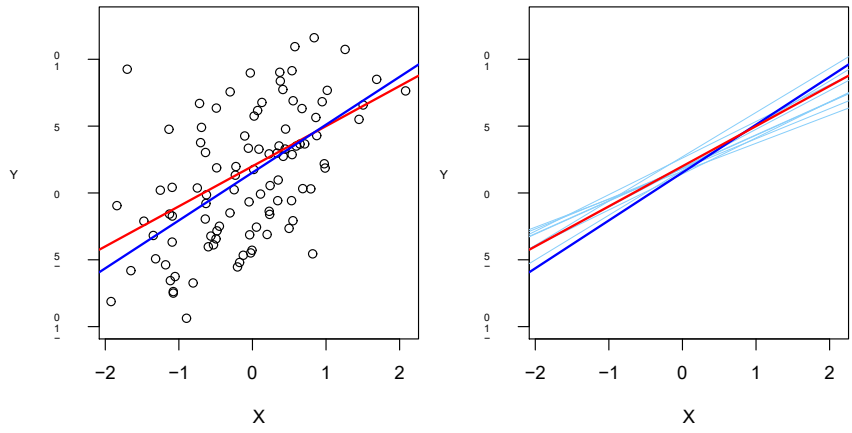


图3.3。 *A simulated data set.* 左： *The red line represents the true relationship,* $f(X) = 2 + 3X$, *which is known as the population regression line. The blue line is the least squares line; it is the least squares estimate for* $f(X)$ *based on the observed data, shown in black.* 右： *The population regression line is again shown in red, and the least squares line in dark blue. In light blue, ten least squares lines are shown, each computed on the basis of a separate random set of observations. Each least squares line is different, but on average, the least squares lines are quite close to the population regression line.*

某个随机变量 Y的总体均值 μ。遗憾的是，μ 是未知的，但我们确实可以获得来自 Y的 n 个观测值，$y_1, \ldots, y_n$，我们可以用它来估计 μ。一个合理的估计是 $\hat{\mu} = \bar{y}$，其中 $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$ 是样本均值。样本均值和总体均值不同，但一般来说，样本均值会提供总体均值的一个良好估计。同样，线性回归中未知系数 β_0 和 β_1 定义了总体回归线。我们试图使用 (3.4) 中给出的 $\hat{\beta}_0$ 和 $\hat{\beta}_1$ 来估计这些未知系数。这些系数估计定义了最小二乘线。

基于 *bias* 的概念，线性回归与随机变量均值估计之间的类比是恰当的。如果我们使用
样本均值$\hat{\mu}$来估计μ，该估计量为*unbiased*，其意义在于
平均而言，我们期望$\hat{\mu}$等于μ。这究竟意味着什么？它意味着基于某一特定观测集$y_1, \ldots, y_n$，$\hat{\mu}$可能高估μ，而基于另一观测集，$\hat{\mu}$可能低估μ。但如果我们能对从大量观测集中获得的大量μ的估计值求平均，那么这个平均值将*exactly*等于μ。因此，无偏估计量不会*systematically*高估或低估真实参数。最小二乘系数估计（由(3.4)给出）也具有无偏性：如果我们基于特定数据集估计β_0和β_1，那么我们的估计值不会恰好等于β_0和β_1。但如果我们能对从大量数据集中获得的估计值求平均，那么这些估计值的平均值将完全准确！事实上，从图3.3的右侧面板可以看出，许多最小二乘线的平均值，每条

bias
unbiased

从单独的数据集估计得到的回归线非常接近真实的总体回归线。

我们继续类比随机变量 Y 的总体均值 μ 的估计。一个自然的问题如下：样本均值 $\hat{\mu}$ 作为 μ 的估计有多准确？我们已经确定，在许多数据集上 $\hat{\mu}$ 的平均值将非常接近 μ，但单个估计 $\hat{\mu}$ 可能大幅低估或高估 μ。那个对 $\hat{\mu}$ 的单个估计会偏离多远？一般来说，我们通过计算 $\hat{\mu}$ 的 *standard error* 来回答这个问题，记作 SE($\hat{\mu}$)。我们有

著名的公式

standard error

$$\text{Var}(\hat{\mu}) = \text{SE}(\hat{\mu})^2 = \frac{\sigma^2}{n}, \tag{3.7}$$

其中 σ 是 Y 的每个实现 y_i 的标准差。[2] 粗略地说，标准误告诉我们这个估计值 $\hat{\mu}$ 与 μ 的实际值平均相差多少。方程3.7还告诉我们这种偏差如何随 n 缩小——我们拥有的观测越多，$\hat{\mu}$ 的标准误就越小。类似地，我们可以想知道 $\hat{\beta}_0$ 和 $\hat{\beta}_1$ 与真实值 β_0 和 β_1 有多接近。为了计算与 $\hat{\beta}_0$ 和 $\hat{\beta}_1$ 相关的标准误，我们使用以下公式：

$$\text{SE}(\hat{\beta}_0)^2 = \sigma^2 \left[\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2} \right], \quad \text{SE}(\hat{\beta}_1)^2 = \frac{\sigma^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}, \tag{3.8}$$

其中 $\sigma^2 = \text{Var}(\epsilon)$。为了使这些公式严格成立，我们需要假设每个观测的误差 ϵ_i 具有相同的方差 σ^2 且互不相关。在图3.1中这显然不成立，但该公式仍然是一个很好的近似。注意公式中，当 x_i 越分散时，SE($\hat{\beta}_1$) 越小；直观地说，在这种情况下我们有更多的 *leverage* 来估计斜率。我们还看到，如果 $\bar{x}$ 为零（此时 $\hat{\beta}_0$ 将等于 $\bar{y}$），那么 SE($\hat{\beta}_0$) 将与 SE($\hat{\mu}$) 相同。一般来说，σ^2 是未知的，但可以从数据中估计。这个 σ 的估计被称为 *residual standard error*，由以下公式给出

残差标准误

$\text{RSE} = \sqrt{\text{RSS}/(n-2)}$。严格来说，当$\sigma^2$是从数据中估计得出时，我们应当写作 $\widehat{\text{SE}}(\hat{\beta}_1)$，以表明已进行了估计，但为简化记号，我们将省略这个额外的“帽子”。

标准误可用于计算 *confidence intervals*。95% 置信区间被定义为一系列值，使得有95%的概率，这个范围将包含参数的真实未知值。该范围是根据从数据样本计算出的下限和上限来定义的。一个95%的置信区间具有以下性质：如果我们重复抽样并为每个样本构建置信区间，那么95%的区间将包含参数的真实未知值。对于线性回归，β_1的95%置信区间大约呈以下形式：

confidence interval

$$\hat{\beta}_1 \pm 2 \cdot \text{SE}(\hat{\beta}_1). \tag{3.9}$$

[2] This formula holds provided that the n observations are uncorrelated.

T也就是说，大约有95%的概率，在 区间

$$\left[\hat{\beta}_1 - 2 \cdot \text{SE}(\hat{\beta}_1),\ \hat{\beta}_1 + 2 \cdot \text{SE}(\hat{\beta}_1)\right] \tag{3.10}$$

将包含 β_1 的真值。[3] 同样，β_0 的置信区间大约形式为

$$\hat{\beta}_0 \pm 2 \cdot \text{SE}(\hat{\beta}_0). \tag{3.11}$$

在广告数据的情况下，β_0的95%置信区间为[6.130, 7.935]，而β_1的95%置信区间为[0.042, 0.053]。因此，我们可以得出结论，在没有任何广告的情况下，销售额平均将下降在6,130到7,935个单位之间。此外，电视广告每增加$1,000，销售额平均将增加42到53个单位之间。

标准误也可用于对
系数。最常见的假设检验涉及检验 *null hypothesis* 的

hypothesis test

null hypothesis

$$H_0 : \text{There is no relationship between } X \text{ and } Y \tag{3.12}$$

相对于 *alternative hypothesis*

备择假设

$$H_a : \text{There is some relationship between } X \text{ and } Y. \tag{3.13}$$

数学上，这对应于测试

$$H_0 : \beta_1 = 0$$

与

$$H_a : \beta_1 \neq 0,$$

因为如果 $\beta_1 = 0$，则模型 (3.5) 简化为 $Y = \beta_0 + \epsilon$，且 X 与 Y不相关。为了检验零假设，我们需要确定$\hat{\beta}_1$——我们对 β_1 的估计——是否足够远离零，从而使我们有信心认为 β_1 非零。多远才算足够远? 这当然取决于$\hat{\beta}_1$ 的准确性——也就是说，取决于 SE($\hat{\beta}_1$)。如果 SE($\hat{\beta}_1$) 很小，那么即使$\hat{\beta}_1$ 的值相对较小，也可能提供强有力的证据表明 $\beta_1 \neq 0$，从而表明 X 与 Y之间存在关系。相反，如果 SE($\hat{\beta}_1$) 很大，那么$\hat{\beta}_1$ 的绝对值必须很大，我们才能拒绝零假设。在实践中，我们计算一个 *t-statistic*，

t-statistic

由以下给出

$$t = \frac{\hat{\beta}_1 - 0}{\text{SE}(\hat{\beta}_1)}, \tag{3.14}$$

[3] *Approximately* for several reasons. Equation 3.10 relies on the assumption that the errors are Gaussian. Also, the factor of 2 in front of the SE($\hat{\beta}_1$) term will vary slightly depending on the number of observations n in the linear regression. To be precise, rather than the number 2, (3.10) should contain the 97.5 % quantile of a t-distribution with $n-2$ degrees of freedom. Details of how to compute the 95 % confidence interval precisely in R will be provided later in this chapter.

	Coefficient	Std. error	t-statistic	p-value
`Intercept`	7.0325	0.4578	15.36	< 0.0001
`TV`	0.0475	0.0027	17.67	< 0.0001

表 3.1. *For the* `Advertising` *data, coefficients of the least squares model for the regression of number of units sold on TV advertising budget. An increase of* \$1,000 *in the TV advertising budget is associated with an increase in sales by around 50 units. (Recall that the* `sales` *variable is in thousands of units, and the* `TV` *variable is in thousands of dollars.)*

它衡量$\hat{\beta}_1$偏离0的标准差数量。如果X与Y之间确实没有关系，那么我们预期（3.14）将具有自由度为$n-$的t分布。t分布呈钟形，当n值大于约30时，它与标准正态分布相当相似。因此，在假定β_1 ==0的情况下，计算观察到绝对值等于$|t|$或更大的任何数值的概率是一件简单的事。我们将此概率称为*p-value*。粗略地说，我们对p值的解释如下：较小的

p-值表明，如果预测变量与响应变量之间不存在任何真实关联，那么由于偶然性而观察到预测变量与响应变量之间存在如此显著关联的可能性很小。因此，如果我们看到一个较小的 p-值，那么我们可以推断预测变量与响应变量之间存在关联。我们 *reject the null hypothesis*——也就是说，我们宣布 X 与 Y 之间存在关系——如果 p-值足够小。拒绝原假设的典型 p-值截断值为5%或1%，尽管这一主题将在第13章中进行更详细的探讨。当 $n = 30$ 时，这些分别对应于大约为2和2.75的 t-统计量（3.14）。

p-value

表3.1提供了根据`Advertising`数据中电视广告预算对销售单位数量进行回归的最小二乘模型详情。注意，$\hat{\beta}_0$和$\hat{\beta}_1$的系数相对于其标准误来说非常大，因此t统计量也很大；若H_0为真，观察到如此大的值的概率几乎为零。因此，我们可以得出结论，$\beta_0 \neq 0$且$\beta_1 \neq 0$。[4]

3.1.3 Assessing the Accuracy of the Model

一旦我们拒绝了原假设（3.12），转而支持备择假设（3.13），自然地，我们希望对*the extent to which themodel fits the data*进行量化。线性回归拟合的质量通常通过两个相关的量来评估：*residual standard error* (RSE)和R^2
统计量。

R^2

[4]In Table 3.1, a small p-value for the intercept indicates that we can reject the null hypothesis that $\beta_0 = 0$, and a small p-value for `TV` indicates that we can reject the null hypothesis that $\beta_1 = 0$. Rejecting the latter null hypothesis allows us to conclude that there is a relationship between `TV` and `sales`. Rejecting the former allows us to conclude that in the absence of `TV` expenditure, `sales` are non-zero.

Quantity	Value
Residual standard error	3.26
R^2	0.612
F-statistic	312.1

表 3.2. *For the* `Advertising` *data, more information about the least squares model for the regression of number of units sold on TV advertising budget.*

表3.2展示了售出单位数量对电视广告预算的线性回归的RSE、R^2统计量和F统计量（将在第3.2.2节中描述）。

残差标准误

回忆模型（3.5），每个观测值都与一个误差项 ϵ 相关联。由于这些误差项的存在，即使我们知道了真实的回归线（即即使 β_0 和 β_1 已知），我们也无法从 X 完美预测 Y。RSE 是 ϵ 标准差的估计。粗略地说，它是响应变量偏离真实回归线的平均量。它使用以下公式计算：

$$\text{RSE} = \sqrt{\frac{1}{n-2}\text{RSS}} = \sqrt{\frac{1}{n-2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}. \tag{3.15}$$

注意，RSS在3.1.1节中定义，并由公式给出：

$$\text{RSS} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2. \tag{3.16}$$

在广告数据的情况下，我们从表3.2的线性回归输出中看到，RSE为3.26。换句话说，每个市场的实际销售额与真实回归线的平均偏差约为3,260个单位。另一种思考方式是，即使模型是正确的，并且未知系数β_0和β_1的真实值完全已知，任何基于电视广告的销售额预测平均仍会偏差约3,260个单位。当然，3,260个单位是否是可接受的预测误差取决于问题的背景。在广告数据集中，所有市场的`sales`平均值约为14,000个单位，因此百分比误差为3,260/14,000 = 23%。

RSE 被视为模型 (3.5) 对数据的 *lack of fit* 的一种度量。如果使用该模型得到的预测值与真实结果值非常接近——也就是说，如果对于 $i = 1, \ldots, n$ 有 $\hat{y}_i \approx y_i$——那么 (3.15) 将会很小，我们可以得出结论：该模型对数据拟合得非常好。另一方面，如果一个或多个观测值的 $\hat{y}_i$ 与 y_i 相差很远，那么 RSE 可能相当大，表明该模型对数据拟合得不好。

$R2$ 统计量

RSE提供了模型（3.5）与数据拟合不足的绝对度量。但由于它是用Y的单位来衡量的，因此并不总是

明确什么构成良好的RSE。R^2统计量提供了一种替代的拟合度量。它表现为*proportion*——解释的方差比例——因此其值始终介于0和1之间，且与Y的尺度无关。

为了计算 R^2，我们使用公式

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}} \tag{3.17}$$

其中 $\text{TSS} = \sum(y_i - \bar{y})^2$ 是 *total sum of squares*，而 RSS 定义为在(3.16)中。TSS衡量响应变量Y的总方差，可以视为在进行回归之前响应变量固有的变异性量。相比之下，RSS衡量执行回归后仍未解释的变异性量。因此，TSS − RSS衡量了通过执行回归所解释（或移除）的响应变量变异性量，而R^2衡量了*proportion of variability in* Y *that can be explained using* X。接近1的R^2统计量表明，响应变量中的大部分变异性被回归所解释。接近0的数值表明回归并未解释响应变量中的大部分变异性；这可能是由于线性模型错误，或误差方差σ^2较高，或两者兼有。在表3.2中，R^2为0.61，因此`sales`中略低于三分之二的变异性可由`TV`的线性回归解释。

平方和总和

R^2 统计量 (3.17) 相比 RSE (3.15) 具有解释上的优势，因为与 RSE 不同，它始终介于 0 和 1 之间。然而，确定什么是 *good* R^2 值仍然具有挑战性，一般而言，这取决于具体应用。例如，在物理学的某些问题中，我们可能知道数据确实来自一个具有小残差的线性模型。在这种情况下，我们会期望看到一个极其接近 1 的 R^2 值，而显著较小的 R^2 值可能表明生成数据的实验存在严重问题。另一方面，在生物学、心理学、市场营销和其他领域的典型应用中，线性模型 (3.5) 充其量只是对数据的极其粗略的近似，而由于其他未测量因素导致的残差误差通常非常大。在这种情况下，我们会期望只有很小一部分响应方差能被预测变量解释，而远低于 0.1 的 R^2 值可能更为现实！

R^2 统计量是 X 与 Y之间线性关系的度量。回想一下，*correlation* 定义为

correlation

$$\text{Cor}(X, Y) = \frac{\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \overline{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \overline{y})^2}}, \tag{3.18}$$

也是衡量 X 和 Y之间线性关系的一种度量。[5] 这表明我们或许可以使用 $r = \text{Cor}(X, Y)$ 来代替 R^2，以评估线性模型的拟合情况。事实上，可以证明，在简单

[5]We note that in fact, the right-hand side of (3.18) is the sample correlation; thus, it would be more correct to write $\widehat{\text{Cor}}(X, Y)$; however, we omit the "hat" for ease of notation.

sales对radio的简单回归

	Coefficient	Std. error	t-statistic	p-value
Intercept	9.312	0.563	16.54	< 0.0001
radio	0.203	0.020	9.92	< 0.0001

sales对newspaper的简单回归

	Coefficient	Std. error	t-statistic	p-value
Intercept	12.351	0.621	19.88	< 0.0001
newspaper	0.055	0.017	3.30	0.00115

表3.3. *More simple linear regression models for the* Advertising *data. Coefficients of the simple linear regression model for number of units sold on* 顶部：*radio advertising budget and* 底部：*newspaper advertising budget. A* $1,000 *increase in spending on radio advertising is associated with an average increase in sales by around 203 units, while the same increase in spending on newspaper advertising is associated with an average increase in sales by around 55 units. (Note that the* sales *variable is in thousands of units, and the* radio *and* newspaper *variables are in thousands of dollars.)*

线性回归设置，$R^2 = r^2$。换句话说，平方相关性和R^2统计量是相同的。然而，在下一节中我们将讨论多元线性回归问题，其中我们同时使用多个预测变量来预测响应。预测变量与响应之间的相关性概念不会自动扩展到这种设置，因为相关性量化的是单对变量之间的关联，而不是更多变量之间的关联。我们将看到R^2填补了这一角色。

3.2 多元线性回归

简单线性回归是一种基于单个预测变量预测响应的有用方法。然而，在实践中，我们通常有不止一个预测变量。例如，在Advertising数据中，我们已经研究了销售额与电视广告之间的关系。我们还有关于广播和报纸广告支出的数据，我们可能想知道这两种媒体中的任何一种是否与销售额相关。我们如何扩展对广告数据的分析，以纳入这两个额外的预测变量？

一种选择是分别进行三个简单的线性回归，每个回归使用不同的广告媒介作为预测变量。例如，我们可以基于广播广告支出金额来拟合一个简单的线性回归以预测销售额。结果如表3.3（上表）所示。我们发现，广播广告支出每增加{1000}美元，销售额大约增加203个单位。表3.3（下表）包含了将销售额对报纸广告预算进行简单线性回归所得的最小二乘系数。报纸广告预算每增加{1000}美元，销售额大约增加55个单位。

然而，为每个预测变量分别拟合一个简单的线性回归模型的方法并不完全令人满意。首先，如何根据三种广告媒体预算给出销售额的单一预测尚不清楚，因为每个预算都与一个单独的回归方程相关联。其次，这三个回归方程在为回归系数形成估计时都忽略了另外两种媒体。我们很快会看到，如果在我们的数据集中，这200个市场中的媒体预算之间存在相关性，那么这可能导致对每种媒体预算与销售额之间关联的估计极具误导性。

与其为每个预测变量分别拟合一个简单的线性回归模型，更好的方法是对简单线性回归模型（3.5）进行扩展，使其能够直接容纳多个预测变量。我们可以通过在单一模型中为每个预测变量赋予一个单独的斜率系数来实现这一点。一般来说，假设我们有p个不同的预测变量。那么多重线性回归模型的形式为

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon, \tag{3.19}$$

其中 X_j 表示第 j 个预测变量，β_j 量化了该变量与响应变量之间的关联。我们将 β_j 解释为 X_j 每增加一个单位时对 Y 的 *average* 效应，即 *holding all other predictors fixed*。在广告示例中，(3.19) 变为

$$\texttt{sales} = \beta_0 + \beta_1 \times \texttt{TV} + \beta_2 \times \texttt{radio} + \beta_3 \times \texttt{newspaper} + \epsilon. \tag{3.20}$$

3.2.1 Estimating the Regression Coefficients

与简单线性回归设置中的情况一样，(3.19) 中的回归系数 $\beta_0, \beta_1, \ldots, \beta_p$ 是未知的，必须进行估计。给定估计值 $\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_p$，我们可以使用以下公式进行预测：

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p. \tag{3.21}$$

参数估计采用与简单线性回归中相同的最小二乘法。我们选择 $\beta_0, \beta_1, \ldots, \beta_p$ 来最小化残差平方和

$$\begin{aligned} \text{RSS} &= \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \\ &= \sum_{i=1}^{n} (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - \hat{\beta}_2 x_{i2} - \cdots - \hat{\beta}_p x_{ip})^2. \end{aligned} \tag{3.22}$$

使 (3.22) 最小化的值 $\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_p$ 是多元最小二乘回归系数估计。与 (3.4) 中给出的简单线性回归估计不同，多元回归系数估计的形式有些复杂，最容易用矩阵代数表示。因此，我们在此不提供它们。任何统计软件包都可以用来计算这些系数估计，本章后面我们将展示如何在 R 中完成。图 3.4

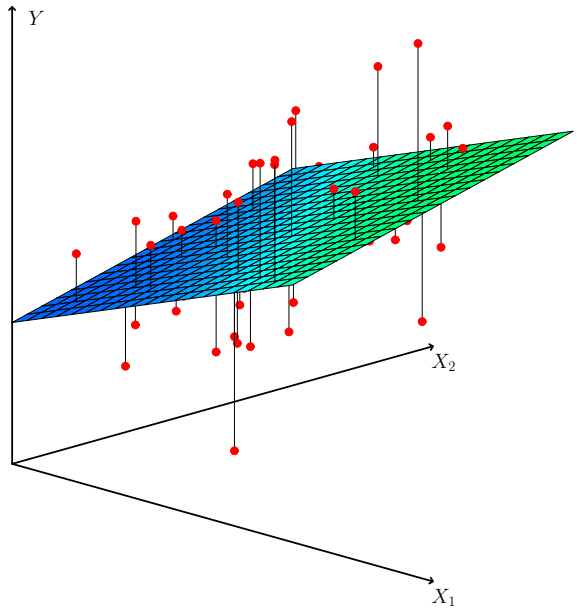


图3.4。*In a three-dimensional setting, with two predictors and one response, the least squares regression line becomes a plane. The plane is chosen to minimize the sum of the squared vertical distances between each observation (shown in red) and the plane.*

举例说明对包含 $p = 2$ 个预测变量的玩具数据集进行最小二乘拟合的示例。

表3.4展示了当使用TV、广播和报纸广告预算来通过`Advertising`数据预测产品销售时的多元回归系数估计值。我们解释这些结果如下：在给定TV和报纸广告量的情况下，在广播广告上额外花费$1,000与大约189个额外销售单位相关联。将这些系数估计值与表3.1和3.3中显示的进行比较，我们注意到`TV`和`radio`的多元回归系数估计值与简单线性回归系数估计值相当相似。然而，虽然表3.3中的`newspaper`回归系数估计值显著非零，但多元回归模型中`newspaper`的系数估计值接近零，相应的p值不再显著，其值约为0.86。这说明简单回归和多元回归系数可能大不相同。这种差异源于以下事实：在简单回归情况下，斜率项表示与报纸广告增加$1,000相关的产品销售平均增长，忽略了诸如`TV`和`radio`等其他预测变量。相比之下，在多元回归设置中，`newspaper`的系数表示在保持`TV`和`radio`固定的情况下，报纸支出增加$1,000时产品销售的平均增长。

多元回归表明`sales`和`newspaper`之间没有关系，而简单线性回归却暗示了这种关系，这是否合理?

	Coefficient	Std. error	t-statistic	p-value
Intercept	2.939	0.3119	9.42	< 0.0001
TV	0.046	0.0014	32.81	< 0.0001
radio	0.189	0.0086	21.89	< 0.0001
newspaper	−0.001	0.0059	−0.18	0.8599

表 3.4. *For the* Advertising *data, least squares coefficient estimates of the multiple linear regression of number of units sold on TV, radio, and newspaper advertising budgets.*

	TV	radio	newspaper	sales
TV	1.0000	0.0548	0.0567	0.7822
radio		1.0000	0.3541	0.5762
newspaper			1.0000	0.2283
sales				1.0000

表 3.5. *Correlation matrix for* TV, radio, newspaper, *and* sales *for the* Advertising *data.*

相反？事实上确实如此。考虑表3.5中显示的三个预测变量和响应变量的相关矩阵。注意，radio与newspaper之间的相关性为0.35。这表明报纸广告投入高的市场往往也有较高的广播广告投入。现在假设多元回归是正确的，即报纸广告与销售额无关，但广播广告与销售额相关。那么，在我们对广播投入更多广告的市场中，销售额往往会更高，而正如相关矩阵所示，在这些相同的市场中，我们往往也会在报纸广告上投入更多。因此，在仅考察sales与newspaper的简单线性回归中，我们会观察到newspaper的较高值往往与sales的较高值相关联，尽管报纸广告与销售额并无直接关联。所以newspaper广告是radio广告的替代变量；newspaper因radio与sales之间的关联而获得了“功劳”。

这个略显反直觉的结果在现实生活的许多情境中非常常见。举一个荒谬的例子来说明这一点。对某一海滩社区在一段时间内收集的数据进行鲨鱼袭击与冰淇淋销量的回归分析，会显示出正相关关系，类似于在sales和newspaper之间所看到的关系。当然，还没有人（迄今为止）建议在海滩禁止冰淇淋以减少鲨鱼袭击。实际上，更高的气温导致更多人前往海滩，这反过来又导致更多的冰淇淋销量和更多的鲨鱼袭击。将鲨鱼袭击对冰淇淋销量和气温进行多元回归分析表明，正如直觉所暗示的那样，在调整气温后，冰淇淋销量不再是一个显著的预测因素。

3.2.2 Some Important Questions

当我们进行多元线性回归时，我们通常有兴趣回答几个重要问题。

1. *Is at least one of the predictors $X_1, X_2, \ldots, X_p$ useful in predicting the response?*

2. *Do all the predictors help to explain Y, or is only a subset of the predictors useful?*

3. *How well does the model fit the data?*

4. *Given a set of predictor values, what response value should we predict, and how accurate is our prediction?*

我们现在依次处理这些问题。

一：响应变量与预测变量之间是否存在关系?

回想一下，在简单线性回归设定中，为了判断响应变量与预测变量之间是否存在关系，我们只需检查 $\beta_1 = 0$。在包含 p 个预测变量的多元回归设定中，我们需要询问所有回归系数是否都为零，即 $\beta_1 = \beta_2 = \cdots = \beta_p = 0$ 是否成立。与简单线性回归设定一样，我们使用假设检验来回答这个问题。我们检验原假设:

$$H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0$$

与替代方案相比

$$H_a : \text{ at least one } \beta_j \text{ is non-zero.}$$

通过计算 *F-statistic* 来执行该假设检验，

F-statistic

$$F = \frac{(\text{TSS} - \text{RSS})/p}{\text{RSS}/(n-p-1)}, \tag{3.23}$$

其中，与简单线性回归一样，$\text{TSS} = \sum(y_i - \bar{y})^2$ 和 $\text{RSS} = \sum(y_i - \hat{y}_i)^2$。如果线性模型假设正确，可以证明

$$E\{\text{RSS}/(n-p-1)\} = \sigma^2$$

并且，在H_0为真的情况下，

$$E\{(\text{TSS} - \text{RSS})/p\} = \sigma^2.$$

因此，当响应变量与预测变量之间没有关系时，人们会期望F统计量的值接近1。另一方面，如果H_a为真，则$E\{(\text{TSS} - \text{RSS})/p\} > \sigma^2$，所以我们期望$F$大于1。

通过将 `sales` 对 `radio`、`TV` 和 `newspaper` 进行回归得到的多元线性回归模型的 F 统计量如表 3.6 所示。在此例中，F 统计量为 570。由于该值远大于 1，它为拒绝原假设 H_0 提供了有力证据。换言之，较大的 F 统计量表明至少有一种广告媒体必须与 `sales` 相关。然而，如果 F 统计量接近 1 呢？F 统计量需要多大才能拒绝 H_0，并且

Quantity	Value
Residual standard error	1.69
R^2	0.897
F-statistic	570

表 3.6. *More information about the least squares model for the regression of number of units sold on TV, newspaper, and radio advertising budgets in the* `Advertising` *data. Other information about this model was displayed in Table 3.4.*

得出结论是否存在关系？事实证明，答案取决于 n 和 p 的值。当 n 很大时，一个略大于1的 F 统计量可能仍然提供了反对 H_0 的证据。相反，如果 n 很小，则需要更大的 F 统计量来拒绝 H_0。当 H_0 为真且误差 ϵ_i 呈正态分布时，F 统计量遵循 F 分布。[6] 对于任何给定的 n 和 p 值，可以使用任何统计软件包来计算与 F 统计量相关的 p 值。基于这个 p 值，我们可以决定是否拒绝 H_0。对于广告数据，表3.6中与 F 统计量相关的 p 值基本上为零，因此我们有极其强烈的证据表明至少有一种媒体与增加的 `sales` 有关。

在(3.23)中，我们正在检验H_0所有系数均为零。有时我们想要检验q系数的一个特定子集是否为零。这对应于一个零假设

$$H_0 : \quad \beta_{p-q+1} = \beta_{p-q+2} = \cdots = \beta_p = 0,$$

为了方便起见，我们将选择省略的变量置于列表末尾。在此情况下，我们拟合一个包含所有变量*except*及最后q个变量的第二个模型。假设该模型的残差平方和为RSS_0，那么相应的F统计量为

$$F = \frac{(\text{RSS}_0 - \text{RSS})/q}{\text{RSS}/(n-p-1)}. \tag{3.24}$$

请注意，在表3.4中，针对每个单独的预测变量，报告了一个t统计量和一个p值。这些提供了关于在调整其他预测变量后，每个单独预测变量是否与响应变量相关的信息。事实证明，这些F检验中的每一个都恰好等同于从模型中省略该单个变量而保留其他所有变量的检验——即(3.24)中的q=1。因此，它报告了将该变量添加到模型中的*partial effect*。例如，如我们之前所讨论的，这些p值表明，`TV`和`radio`与`sales`有关，但当`TV`和`radio`保持不变时，没有证据表明`newspaper`与`sales`有关联。

给定每个变量的这些个别p值，我们为何还需要查看整体的F统计量呢？毕竟，如果个别变量中的任一p值非常小，那么*at least one of the predictors is related to the response*似乎是理所当然的。然而，这种逻辑是有缺陷的，尤其是在预测变量数量p很大时。

---

[6] Even if the errors are not normally-distributed, the F-statistic approximately follows an F-distribution provided that the sample size n is large.

[7] The square of each t-statistic is the corresponding F-statistic.

例如，考虑一个例子，其中$p = 100$且H_0：$\beta_1 = \beta_2 = \cdots = \beta_p = 0$为真，因此没有变量真正与响应变量相关。在这种情况下，与每个变量相关的p值（如表3.4所示类型）中约有5%会因偶然性而低于0.05。换言之，即使预测变量与响应变量之间不存在任何真实关联，我们预期仍会观察到大约五个*small* p值。[8]事实上，我们很可能会因偶然性而至少观察到一个低于0.05的p值！因此，如果我们使用各个t统计量及相关的p值来判断变量与响应变量之间是否存在关联，我们极有可能错误地得出结论认为存在关系。然而，F统计量不存在这个问题，因为它对预测变量的数量进行了调整。因此，如果H_0为真，则无论预测变量的数量或观测数量如何，F统计量导致p值低于0.05的概率仅为5%。

使用F统计量来检验预测变量与响应变量之间是否存在任何关联的方法在p相对较小时有效，当然在p与n相比较小时更是如此。然而，有时我们会遇到非常大量的变量。如果$p > n$，那么需要估计的系数β_j数量将多于用于估计它们的观测值数量。在这种情况下，我们甚至无法使用最小二乘法拟合多元线性回归模型，因此F统计量无法使用，本章到目前为止所见的大多数其他概念也同样无法使用。当p较大时，可以使用下一节讨论的一些方法，例如*forward selection*。这种*high-dimensional*情形将在第6章中更详细地讨论。

high-dimensional

二：决定重要变量

如上一节所述，多元回归分析的第一步是计算F统计量并检查相关的p值。如果我们基于该p值得出结论，认为至少有一个预测变量与响应变量有关，那么自然会想知道*which*是有影响的变量！我们可以像表3.4中那样查看各个p值，但正如所讨论的（并在第13章进一步探讨的），如果p很大，我们很可能做出一些错误发现。

有可能所有预测变量都与响应变量相关，但更常见的情况是响应变量仅与预测变量的一个子集相关。确定哪些预测变量与响应变量相关，以便仅使用这些预测变量拟合单一模型的任务被称为*variable selection*。变量选择这个问题在第6章中得到了广泛研究，因此这里我们仅简要概述一些经典方法。

variable selection

理想情况下，我们希望通过尝试大量不同的模型来进行变量选择，每个模型包含不同的预测变量子集。例如，如果 $p = 2$，那么我们可以考虑四个模型：（1）不包含任何变量的模型，（2）仅包含 X_1 的模型，（3）包含

[8]This is related to the important concept of *multiple testing*, which is the focus of Chapter 13.

X仅2，以及（4）一个同时包含X_1和X_2的模型。我们便可以从已考虑的所有模型中选出*best*模型。我们如何确定哪个模型最好？可以使用多种统计量来判断模型的质量。这些包括*Mallow's* C_p、*Akaike information criterion* (AIC)、*Bayesian information criterion* (BIC)和*adjusted* $R2$. 这些在第六章中有更详细的讨论。我们还可以确定通过绘制各种模型输出（例如残差）来挖掘哪个模型最佳，以寻找模式。

马洛的 C_p
赤池信息量准则 贝叶斯信息量准则

遗憾的是，总共存在2^p个包含p个变量子集的模型。这意味着即使对于中等大小的p，尝试预测变量的每一个可能子集都是不可行的。例如，我们看到如果$p = 2$，那么有$2^2 = 4$个模型需要考虑。但如果$p = 30$，那么我们必须考虑$2^{30} = 1{,}073{,}741{,}824$个模型！这并不实际。因此，除非$p$非常小，我们不能考虑所有$2^p$个模型，而是需要一种自动化且高效的方法来选择一组较小的模型进行考虑。对于这项任务，有三种经典方法：

调整后的 R^2

- *Forward selection*。我们从*null model*开始——这是一个模型，包含截距但没有预测变量。然后我们拟合p个简单线性回归，并将导致最低RSS的变量添加到零模型。然后我们向该模型添加导致新二变量模型最低RSS的变量。这一过程持续进行，直到满足某个停止规则。

前向选择
零模型

- *Backward selection*. 我们从模型中的所有变量开始，并且移除具有最大 p 值的变量——也就是统计显著性最低的变量。拟合新的（$p - 1$）变量模型，然后移除具有最大 p 值的变量。此过程持续进行，直到达到停止规则为止。例如，当所有剩余变量的 p 值都低于某个阈值时，我们可能会停止。

后向选择

- *Mixed selection*. 这是正向和反向搜索的组合-择。模型中没有变量，我们从以下步骤开始，如同向前选择一样，我们添加能提供最佳拟合的变量。我们不断逐一添加变量。当然，正如我们在`Advertising`示例中指出的，随着新预测变量被添加到模型中，变量的p值可能会增大。因此，如果在任何时候模型中某个变量的p值超过了某个阈值，我们就从模型中移除该变量。我们持续执行这些向前和向后步骤，直到模型中的所有变量都具有足够低的p值，并且模型外的所有变量如果添加到模型中，都会有较大的p值。

混合选择

如果$p > n$，则不能使用后向选择，而前向选择始终可以使用。前向选择是一种贪心方法，可能会过早纳入后来变得冗余的变量。混合选择可以弥补这一点。

三：模型拟合

模型拟合的两个最常见的数值度量是RSE和R^2，即解释的方差比例。这些量的计算和解释方式与简单线性回归相同。

回想一下，在简单回归中，R^2是响应变量与变量之间相关系数的平方。在多元线性回归中，它等于$\text{Cor}(Y, \hat{Y})^2$，即响应变量与拟合线性模型之间相关系数的平方；实际上，拟合线性模型的一个性质就是它能在所有可能的线性模型中最大化这种相关性。

一个接近1的 R^2 值表明模型解释了响应变量中很大一部分的方差。例如，我们在表3.6中看到，对于 `Advertising` 数据，使用所有三种广告媒体来预测 `sales` 的模型其 R^2 为0.8972。另一方面，仅使用 `TV` 和 `radio` 来预测 `sales` 的模型其 R^2 值为0.89719。换句话说，如果我们向已包含电视和广播广告的模型中添加报纸广告，R^2 会有 $small$ 的增加，尽管我们之前看到表3.4中报纸广告的 p 值并不显著。事实证明，当向模型中添加更多变量时，R^2 总是会增加，即使这些变量与响应变量仅有微弱的关联。这是因为添加另一个变量总是会导致训练数据上残差平方和的减少（尽管不一定是测试数据）。因此，同样在训练数据上计算的 R^2 统计量必然增加。将报纸广告添加到仅包含电视和广播广告的模型中仅导致 R^2 的微小增加，这一事实提供了额外的证据，表明 `newspaper` 可以从模型中删除。基本上，`newspaper` 对训练样本的模型拟合没有真正的改善，并且由于过拟合，将其纳入很可能导致在独立测试样本上表现不佳。

相比之下，仅包含 `TV` 作为预测变量的模型，其 R^2 为 0.61（表3.2）。将 `radio` 加入模型后，R^2 显著提升。这意味着，使用电视和广播支出预测销售额的模型，明显优于仅使用电视广告支出的模型。我们可以通过考察仅以 `TV` 和 `radio` 作为预测变量的模型中 `radio` 系数的 p 值，进一步量化这一改进。

仅包含 `TV` 和 `radio` 作为预测变量的模型，其 RSE 为 1.681，而同时包含 `newspaper` 作为预测变量的模型，其 RSE 为 1.686（表 3.6）。相比之下，仅包含 `TV` 的模型，其 RSE 为 3.26（表 3.2）。这证实了我们之前的结论：使用电视和广播支出预测销售额的模型（在训练数据上）比仅使用电视支出的模型准确得多。此外，鉴于电视和广播支出已被用作预测变量，在模型中再使用报纸支出作为预测变量已无意义。细心的读者可能会好奇，既然 RSS 必然下降，为何在模型中添加 `newspaper` 后 RSE 反而可能增加。通常，RSE 定义为

$$\text{RSE} = \sqrt{\frac{1}{n-p-1}\text{RSS}}, \tag{3.25}$$

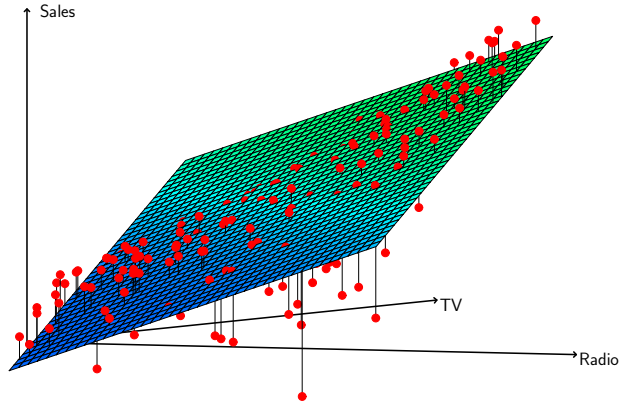


图 3.5. *For the* `Advertising` *data, a linear regression fit to* `sales` *using* `TV` *and* `radio` *as predictors. From the pattern of the residuals, we can see that there is a pronounced non-linear relationship in the data. The positive residuals (those visible above the surface), tend to lie along the 45-degree line, where TV and Radio budgets are split evenly. The negative residuals (most not visible), tend to lie away from this line, where budgets are more lopsided.*

对于简单线性回归，这简化为 (3.15)。因此，如果 RSS 的减少相对于 p 的增加较小，那么变量更多的模型可能具有更高的 RSE。

除刚才讨论的RSE和R^2统计量外，绘制数据图也可能很有用。图形摘要可以揭示数值统计量无法显现的模型问题。例如，图3.5展示了`TV`和`radio`对`sales`的三维图。我们看到有些观测点位于最小二乘回归平面上方，有些位于下方。特别是，对于大部分广告费用仅花在`TV`或`radio`上的情况，线性模型似乎高估了`sales`。而对于预算在两个媒体之间分配的情况，它低估了`sales`。这种明显的非线性模式表明在

interaction

广告媒体，将媒体组合在一起使用比使用任何单一媒体都能带来更大的销售提升。在3.3.2节中，我们将讨论通过使用交互项来扩展线性模型以适应这种协同效应。

四：预测

一旦我们拟合了多元回归模型，就可以直接应用(3.21)来根据预测变量$X_1, X_2, \ldots, X_p$的一组值预测响应Y。然而，这种预测存在三种不确定性。

1. 系数估计值$\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_p$是$\beta_0, \beta_1, \ldots, \beta_p$的估计值。也就是说，*least squares plane*

$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \cdots + \hat{\beta}_p X_p$$

只是对 *true population regression pl*的估计 *ane*

$$f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p.$$

系数估计中的不准确性涉及第2章中的 *reducible error*。我们可以计算 *confidence interval* 来判断$\hat{Y}$ 将有多接近 $f(X)$。

2. 当然，在实际应用中，假设$f(X)$为线性模型几乎总是对现实的一种近似，因此存在一个额外的潜在可减少误差源，我们称之为*model bias*。因此，当我们使用线性模型时，实际上是在估计真实曲面的最佳线性近似。然而，在此我们将忽略这一差异，并假设线性模型是正确的。

3. 即使我们知道了$f(X)$——也就是说，即使我们知道了$\beta_0, \beta_1, \ldots, \beta_p$的真实值——由于模型(3.20)中的随机误差$\epsilon$，响应值也无法被完美预测。在第二章中，我们将此称为*irreducible error*。Y与$\hat{Y}$之间会有多大差异？我们使用*prediction intervals*来回答这个问题。预测区间总是比置信区间宽，因为它们既包含了$f(X)$ (可减少误差)估计中的误差，也包含了关于单个点与总体回归平面差异多少的不确定性（不可减少的误差）。

我们使用*confidence interval*来量化围绕
在大量城市中，*average* `sales`。例如，假设每个城市在`TV`广告上花费$100,000，在`radio`广告上花费$20,000，则95%置信区间为[10,985, 11,528]。我们将其解释为，95%的这种形式的区间将包含$f(X)$的真实值。[9]另一方面，可以使用*prediction interval*来量化
关于*particular*城市中`sales`的不确定性。假设该城市在`TV`广告上花费$100,000，在`radio`广告上花费$20,000，则95%预测区间为[7,930, 14,580]。我们将其解释为：这种形式的区间中有95%将包含该城市Y的真实值。请注意，两个区间都以11,256为中心，但预测区间比置信区间宽得多，这反映了给定城市的`sales`相对于许多地点上平均`sales`的不确定性增加。

置信区间

预测区间

[9]In other words, if we collect a large number of data sets like the **Advertising** data set, and we construct a confidence interval for the average **sales** on the basis of each data set (given $100,000 in **TV** and $20,000 in **radio** advertising), then 95 % of these confidence intervals will contain the true value of average **sales**.

3.3 回归模型中的其他考虑因素

3.3.1 Qualitative Predictors

到目前为止的讨论中，我们假设线性回归模型中的所有变量都是*quantitative*。但在实际操作中，情况未必如此；常有某些预测变量为*qualitative*。

例如，图3.6中展示的`Credit`数据集记录了一些信用卡持有者的变量。响应变量是`balance`（每个个体的平均信用卡债务），并且有几个定量预测变量：`age`、`cards`（信用卡数量）、`education`（教育年数）、`income`（以千美元计）、`limit`（信用额度）和`rating`（信用评级）。图3.6的每个面板都是一对变量的散点图，其身份由相应的行和列标签给出。例如，直接位于“Balance”一词右侧的散点图描绘了`balance`与`age`的关系，而直接位于“Age”右侧的图则对应于`age`与`cards`的关系。除了这些定量变量外，我们还有四个定性变量：`own`（房屋所有权）、`student`（学生身份）、`status`（婚姻状况）以及`region`（东部、西部或南部）。

只有两个水平的预测变量

假设我们希望研究拥有房屋者和未拥有房屋者之间信用卡余额的差异，暂时忽略其他变量。如果一个定性预测变量（也称为*factor*）

只有两个*levels*或可能的值，然后将其纳入一个回归
sion 模型非常简单。我们只需创建一个指示器或 *dummy variable*
其取两个可能的数值。[10] 例如，基于 `own` 变量，我们可以创建一个新变量
，其形式为

factor
level
dummy variable

$$x_i = \begin{cases} 1 & \text{if } i\text{th person owns a house} \\ 0 & \text{if } i\text{th person does not own a house,} \end{cases} \tag{3.26}$$

并将其作为预测变量用于回归方程中。这样得到的模型

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i & \text{if } i\text{th person owns a house} \\ \beta_0 + \epsilon_i & \text{if } i\text{th person does not.} \end{cases} \tag{3.27}$$

现在，β_0 可以解释为那些没有自己住房的人的平均信用卡余额，$\beta_0 + \beta_1$ 解释为那些拥有自己住房的人的平均信用卡余额，而 β_1 解释为所有者与非所有者之间信用卡余额的平均差异。

表3.7显示了与模型（3.27）相关的系数估计和其他信息。非持卡人的平均信用卡债务估计为509.80美元，而持卡人估计额外承担19.73美元的债务，总计509.80美元 + 19.73美元 = 529.53美元。然而，我们

[10] In the machine learning community, the creation of dummy variables to handle qualitative predictors is known as “one-hot encoding”.

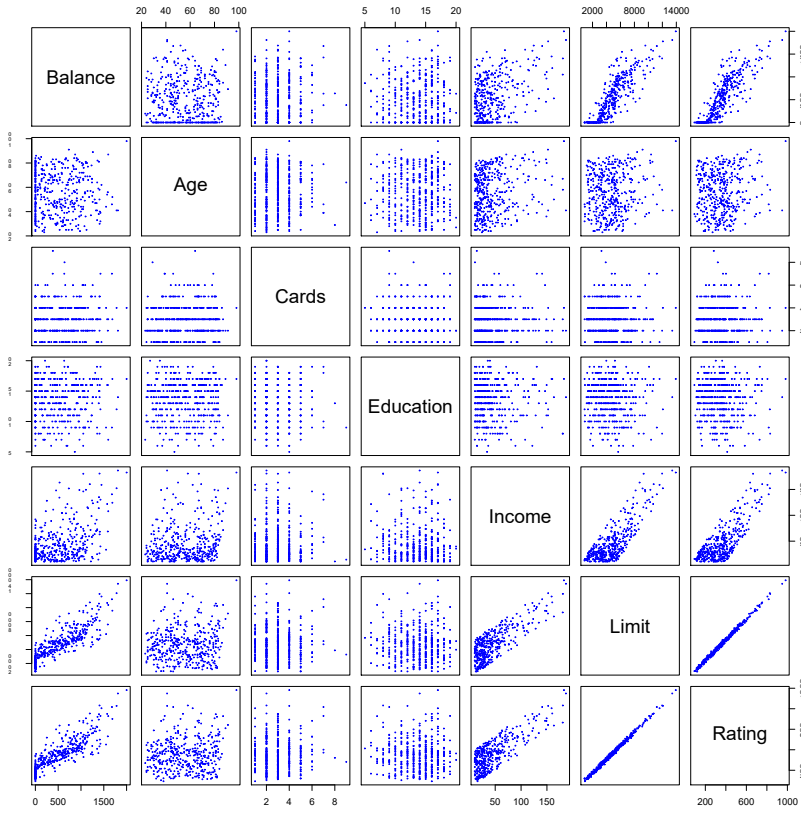


图3.6。*The* `Credit` *data set contains information about* `balance`, `age`, `cards`, `education`, `income`, `limit`, *and* `rating` *for a number of potential customers.*

请注意，虚拟变量的p值非常高。这表明没有统计证据表明基于房屋所有权的平均信用卡余额存在差异。

在(3.27)中将有卡者编码为1、无卡者编码为0的决定是任意的，对回归拟合没有影响，但会改变系数的解释。如果我们把无卡者编码为1、有卡者编码为0，那么β_0和β_1的估计值将分别为529.53和-19.73，从而再次得出无卡者的信用卡债务预测值为\$529.53 $-$ \$19.73 $=$ \$509.80，而有卡者的预测值为\$529.53。或者，除了0/1编码方案之外，我们还可以创建一个虚拟变量

$$x_i = \begin{cases} 1 & \text{if } i\text{th person owns a house} \\ -1 & \text{if } i\text{th person does not own a house} \end{cases}$$

并在回归方程中使用该变量。由此得到模型

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i & \text{if } i\text{th person owns a house} \\ \beta_0 - \beta_1 + \epsilon_i & \text{if } i\text{th person does not own a house.} \end{cases}$$

	Coefficient	Std. error	t-statistic	p-value
`Intercept`	509.80	33.13	15.389	< 0.0001
`own[Yes]`	19.73	46.05	0.429	0.6690

表 3.7. *Least squares coefficient estimates associated with the regression of* `balance` *onto* `own` *in the* `Credit` *data set. The linear model is given in (3.27). That is, ownership is encoded as a dummy variable, as in (3.26).*

现在，β_0可以解释为总体平均信用卡余额（忽略房屋所有权效应），而β_1分别是房主和非房主信用卡余额高于和低于平均值的金额。[11]在此示例中，β_0的估计值为\$519.665，介于非房主和房主平均值\$509.80和\$529.53之间。β_1的估计值为\$9.865，这是\$19.73的一半，即房主和非房主之间的平均差异。需要注意的是，无论使用何种编码方案，房主和非房主信用余额的最终预测值都将相同。唯一的区别在于系数的解释方式。

具有两个以上水平的定性预测变量

当一个定性预测变量有两个以上的水平时，单一的虚拟变量无法代表所有可能的值。在这种情况下，我们可以创建额外的虚拟变量。例如，对于`region`变量，我们创建了两个虚拟变量。第一个可以是

$$x_{i1} = \begin{cases} 1 & \text{if } i\text{th person is from the South} \\ 0 & \text{if } i\text{th person is not from the South,} \end{cases} \tag{3.28}$$

第二个可能是

$$x_{i2} = \begin{cases} 1 & \text{if } i\text{th person is from the West} \\ 0 & \text{if } i\text{th person is not from the West.} \end{cases} \tag{3.29}$$

然后，这两个变量都可以用于回归方程中，以获得模型

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i & \text{if } i\text{th person is from the South} \\ \beta_0 + \beta_2 + \epsilon_i & \text{if } i\text{th person is from the West} \\ \beta_0 + \epsilon_i & \text{if } i\text{th person is from the East.} \end{cases} \tag{3.30}$$

现在，β_0 可以解释为来自东部的个人的平均信用卡余额，β_1 可以解释为来自南部与东部的人的平均余额差异，而 β_2 可以解释为来自西部与东部的人的平均余额差异。虚拟变量的数量总是比水平数少一个。没有虚拟变量的水平——在这个例子中是东部——被称为 *baseline*。

baseline

[11] Technically β_0 is half the sum of the average debt for house owners and the average debt for non-house owners. Hence, β_0 is exactly equal to the overall average only if the two groups have an equal number of members.

	Coefficient	Std. error	t-statistic	p-value
`Intercept`	531.00	46.32	11.464	< 0.0001
`region[South]`	−12.50	56.68	−0.221	0.8260
`region[West]`	−18.69	65.02	−0.287	0.7740

表 3.8. *Least squares coefficient estimates associated with the regression of* `balance` *onto* `region` *in the* `Credit` *data set. The linear model is given in (3.30). That is, region is encoded via two dummy variables (3.28) and (3.29).*

从表3.8中，我们看到基线组，即东部地区的估计`balance`为$531.00。据估计，南部地区人群的债务将比东部地区少$18.69，而西部地区人群的债务将比东部地区少$12.50。然而，与两个虚拟变量系数估计相关的p值非常大，表明没有统计证据显示南部与东部之间或西部与东部之间的平均信用卡余额存在真实差异。[12]再次强调，选择作为基线类别的水平是任意的，无论这一选择如何，各组最终的预测结果都将相同。但是，系数及其p值确实取决于虚拟变量编码的选择。与其依赖个别系数，我们可以使用F检验来检验H_0：$\beta_1 = \beta_2 = 0$；这不依赖于编码方式。此F检验的p值为0.96，表明我们不能拒绝原假设，即`balance`与`region`之间不存在关系。

使用这种虚拟变量方法时，将定量和定性预测变量同时纳入并不会带来任何困难。例如，为了对信用卡余额回归`balance`，同时考虑如`income`这样的定量变量和如`student`这样的定性变量，我们只需为`student`创建一个虚拟变量，然后使用`income`和该虚拟变量作为预测变量来拟合多元回归模型。

除了此处采用的虚拟变量方法之外，还有许多不同的编码定性变量的方法。所有这些方法都能得到等价的模型拟合，但系数不同且具有不同的解释，旨在测量特定的*contrasts*。这一主题

contrast

这超出了本书的范围。

3.3.2 Extensions of the Linear Model

标准线性回归模型（3.19）提供了可解释的结果，并且在许多现实世界的问题上表现良好。然而，它做出了几个经常在实践中被违反的严格限制性假设。其中最重要的两个假设指出预测变量与响应变量之间的关系是*additive*和*linear*。可加性假设

additive
linear

意味着预测变量 X_j 与响应变量 Y 之间的关联不依赖于其他预测变量的值。线性假设表明，与 X_j 变化一个单位相关的响应 Y 的变化是恒定的，无论 X_j 的值如何。在本书后续章节中，我们将探讨一些放宽这两个假设的复杂方法。

[12]There could still in theory be a difference between South and West, although the data here does not suggest any difference.

假设。这里，我们简要考察一些扩展线性模型的常见经典方法。

去除加法假设

在我们之前对Advertising数据的分析中，我们得出结论，TV和radio似乎都与sales有关。作为这一结论基础的线性模型假设，增加一种广告媒体对sales的影响与其他媒体上的花费无关。例如，线性模型(3.20)表明，与TV增加一个单位相关的sales平均增加量始终为β_1，无论radio上的花费是多少。

然而，这个简单的模型可能并不正确。假设将资金投入到广播广告实际上会提高电视广告的效果，因此当radio增加时，TV的斜率项应当增大。在这种情况下，给定100,000美元的固定预算，将一半用于radio，一半用于TV，可能会比将全部金额分配给TV或radio中的任意一个带来更大的sales增长。在市场营销中，这被称为*synergy*效应，而在统计学中，它被称为*interaction*效应。图3.5表明，这种效应可能存在于广告数据中。请注意，当TV或radio的水平较低时，真实的sales低于线性模型的预测值。但是，当广告在两个媒体之间分配时，模型往往会低估sales。

考虑包含两个变量的标准线性回归模型，

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon.$$

根据该模型，X_1每增加一个单位，Y平均增加β_1个单位。注意，X_2的存在并不会改变这一陈述——也就是说，无论X_2的值如何，X_1每增加一个单位，Y就会增加β_1个单位。扩展该模型的一种方法是纳入第三个预测变量，称为*interaction term*，它是通过计算X_1与X_2的乘积构建的。这得到模型

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2 + \epsilon. \tag{3.31}$$

包含此交互项如何放宽可加性假设？注意(3.31)可以重写为

$$\begin{aligned} Y &= \beta_0 + (\beta_1 + \beta_3 X_2) X_1 + \beta_2 X_2 + \epsilon \\ &= \beta_0 + \tilde{\beta}_1 X_1 + \beta_2 X_2 + \epsilon \end{aligned} \tag{3.32}$$

其中$\tilde{\beta}_1 = \beta_1 + \beta_3 X_2$。由于$\tilde{\beta}_1$现在是$X_2$的函数，$X_1$与$Y$之间的关联不再是恒定的：$X_2$值的变化将改变$X_1$与$Y$之间的关联。类似的论证表明，$X_1$值的变化会改变$X_2$与$Y$之间的关联。

例如，假设我们感兴趣的是研究一家工厂的生产率。我们希望根据生产线的数量和生产工人的总数来预测生产的产品数量。增加生产线数量似乎很可能会影响

	Coefficient	Std. error	t-statistic	p-value
Intercept	6.7502	0.248	27.23	< 0.0001
TV	0.0191	0.002	12.70	< 0.0001
radio	0.0289	0.009	3.24	0.0014
TV×radio	0.0011	0.000	20.73	< 0.0001

表3.9. *For the* Advertising *data, least squares coefficient estimates associated with the regression of* sales *onto* TV *and* radio*, with an interaction term, as in (3.33).*

这将取决于工人的数量，因为如果没有工人来操作生产线，那么增加生产线数量并不会增加产量。这表明，在预测 units 的线性模型中，加入 lines 和 workers 之间的交互项是合适的。假设当我们拟合该模型时，得到

$$\begin{aligned} \texttt{units} &\approx 1.2 + 3.4 \times \texttt{lines} + 0.22 \times \texttt{workers} + 1.4 \times (\texttt{lines} \times \texttt{workers}) \\ &= 1.2 + (3.4 + 1.4 \times \texttt{workers}) \times \texttt{lines} + 0.22 \times \texttt{workers}. \end{aligned}$$

换句话说，增加一条额外的生产线将增加$3.4 + 1.4 \times$ workers的产量。因此，我们拥有的workers越多，lines的效果就越强。

现在我们回到 Advertising 的例子。一个使用 radio、TV 以及两者之间的交互作用来预测 sales 的线性模型形式为

$$\begin{aligned} \texttt{sales} &= \beta_0 + \beta_1 \times \texttt{TV} + \beta_2 \times \texttt{radio} + \beta_3 \times (\texttt{radio} \times \texttt{TV}) + \epsilon \\ &= \beta_0 + (\beta_1 + \beta_3 \times \texttt{radio}) \times \texttt{TV} + \beta_2 \times \texttt{radio} + \epsilon. \end{aligned} \tag{3.33}$$

我们可以将β_3解释为与广播广告增加一个单位相关的电视广告效果增加（反之亦然）。拟合模型(3.33)所得到的系数见表3.9。

表3.9中的结果强烈表明，包含交互项的模型优于仅包含*main effects*的模型。

main effect

交互项 p 值，即 TV×radio，极低，表明有强有力的证据支持 H_a： $\beta_3 \neq 0$。换句话说，很明显真实关系不是加性的。模型 (3.33) 的 R^2 为 96.8%，而仅使用 TV 和 radio 预测 sales 且不含交互项的模型仅为 89.7%。这意味着 $(96.8 - 89.7)/(100 - 89.7) = 69\%$ 的拟合加性模型后剩余的 sales 变异已被交互项解释。表 3.9 中的系数估计表明，电视广告支出增加 \$1,000 与销售额增加 $(\hat{\beta}_1 + \hat{\beta}_3 \times \texttt{radio}) \times 1{,}000 = 19 + 1.1 \times \texttt{radio}$ 个单位相关。而广播广告支出增加 \$1,000 将与销售额增加 $(\hat{\beta}_2 + \hat{\beta}_3 \times \texttt{TV}) \times 1{,}000 = 29 + 1.1 \times \texttt{TV}$ 个单位相关。

在这个例子中，与TV、radio以及交互项相关的p值在统计上均显著（表3.9），因此显然所有三个变量都应纳入模型。然而，有时会出现交互项的p值非常小，但相关的主效应（在此情况下为TV和radio）却不显著的情况。*hier-archical principle*指出*if we include an interaction in a model, we*

hierarchical principle

should also include the main effects, even if the p-values associated with their coefficients are not significant. 换句话说，如果X_1和X_2之间的交互作用看起来很显著，那么我们应当将X_1和X_2都纳入模型中，即便它们的系数估计值具有较大的p值。这一原则的理由在于，如果$X_1 \times X_2$与响应变量有关联，那么X_1或X_2的系数是否恰好为零就无关紧要了。此外，$X_1 \times X_2$通常与X_1和X_2相关联，因此忽略它们往往会改变交互作用的含义。

在前面的例子中，我们考虑了 `TV` 和 `radio` 之间的交互作用，这两个都是定量变量。然而，交互作用的概念同样适用于定性变量，或定量与定性变量的组合。事实上，定性变量与定量变量之间的交互作用有一种特别好的解释。考虑第3.3.1节中的 `Credit` 数据集，并假设我们希望使用 `income` (定量) 和 `student` (定性) 变量来预测 `balance`。在没有交互项的情况下，模型的形式为

$$\begin{aligned}\texttt{balance}_i &\approx \beta_0 + \beta_1 \times \texttt{income}_i + \begin{cases}\beta_2 & \text{if } i\text{th person is a student}\\ 0 & \text{if } i\text{th person is not a student}\end{cases}\\ &= \beta_1 \times \texttt{income}_i + \begin{cases}\beta_0 + \beta_2 & \text{if } i\text{th person is a student}\\ \beta_0 & \text{if } i\text{th person is not a student.}\end{cases}\end{aligned} \tag{3.34}$$

注意，这相当于对数据拟合两条平行线，一条针对学生，另一条针对非学生。学生和非学生的直线具有不同的截距，$\beta_0 + \beta_2$ 与 β_0，但斜率相同，均为 β_1。这一点在图3.7左侧面板中得到了说明。直线平行的事实意味着，`income` 增加一个单位对 `balance` 的平均效应并不取决于个体是否为学生。这代表了模型可能存在的一个严重局限性，因为实际上 `income` 的变化对学生与非学生的信用卡余额可能产生截然不同的影响。

这个问题可以通过添加一个交互变量来解决，该变量通过将`income`与`student`的虚拟变量相乘来创建。我们的模型现在变为:

$$\begin{aligned}\texttt{balance}_i &\approx \beta_0 + \beta_1 \times \texttt{income}_i + \begin{cases}\beta_2 + \beta_3 \times \texttt{income}_i & \text{if student}\\ 0 & \text{if not student}\end{cases}\\ &= \begin{cases}(\beta_0 + \beta_2) + (\beta_1 + \beta_3) \times \texttt{income}_i & \text{if student}\\ \beta_0 + \beta_1 \times \texttt{income}_i & \text{if not student.}\end{cases}\end{aligned} \tag{3.35}$$

我们再一次为学生和非学生得到两条不同的回归线。但现在这些回归线有不同的截距，$\beta_0+\beta_2$ 对 β_0，以及不同的斜率，$\beta_1+\beta_3$ 对 β_1。这允许收入变化可能对学生和非学生的信用卡余额产生不同影响的可能性。图3.7的右侧面板

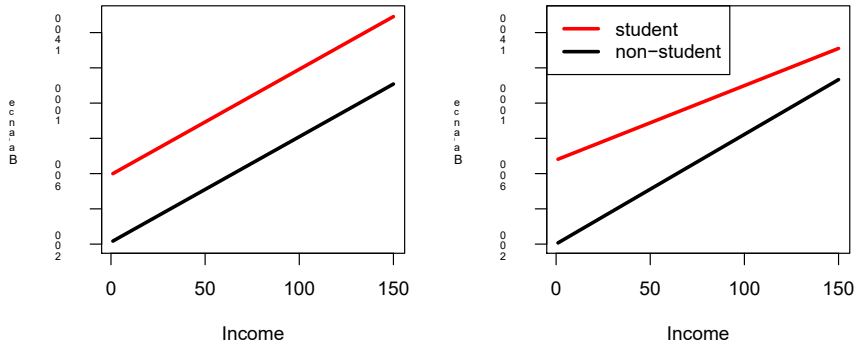


图3.7. *For the* `Credit` *data, the least squares lines are shown for prediction of* `balance` *from* `income` *for students and non-students.* 左：*The model (3.34) was fit. There is no interaction between* `income` *and* `student`. 右：*The model (3.35) was fit. There is an interaction term between* `income` *and* `student`.

展示了模型（3.35）中学生与非学生之间`income`和`balance`的估计关系。我们注意到，学生的斜率低于非学生的斜率。这表明，与非学生相比，学生收入的增加与信用卡余额的较小增长相关联。

非线性关系

如前所述，线性回归模型（3.19）假设响应变量与预测变量之间存在线性关系。但在某些情况下，响应变量与预测变量之间的真实关系可能是非线性的。这里我们介绍一种非常简单的直接扩展线性模型的方法，使用*polynomial regression*来适应非线性关系。在

polynomial regression

在后续章节中，我们将介绍在更一般设置下执行非线性拟合的更复杂方法。

考虑图3.8，其中展示了`Auto`数据集中多辆汽车的`mpg` (每加仑英里数的燃油效率)与`horsepower`的关系。橙色线代表线性回归拟合。`mpg`与`horsepower`之间存在明显的关系，但显然这种关系实际上是非线性的：数据暗示了一种曲线关系。在 linear 模型中纳入非线性关联的一种简单方法是包含预测变量的变换版本。例如，图3.8中的点似乎呈*quadratic*形状，这表明一个

quadratic

表单

$$\texttt{mpg} = \beta_0 + \beta_1 \times \texttt{horsepower} + \beta_2 \times \texttt{horsepower}^2 + \epsilon \tag{3.36}$$

可能会提供更好的拟合。方程3.36涉及使用`horsepower`的非线性函数来预测`mpg`。*But it is still a linear model!*即，(3.36)式就是一个包含$X_1 = \texttt{horsepower}$和$X_2 = \texttt{horsepower}^2$的简单多元线性回归模型。因此，我们可以利用标准的线性回归软件来估计β_0, β_1和β_2，从而生成非线性拟合。图3.8中的蓝色曲线显示了数据得到的二次拟合结果。该二次

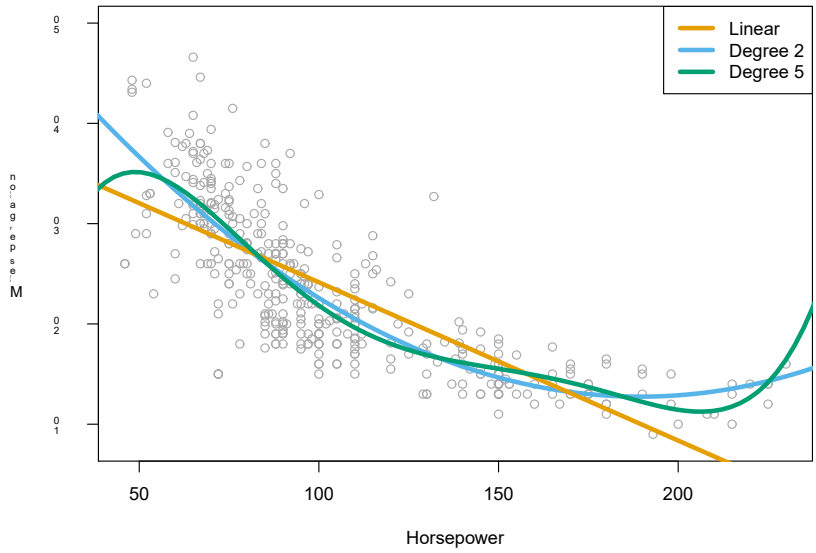


图3.8. *The* `Auto` *data set. For a number of cars,* `mpg` *and* `horsepower` *are shown. The linear regression fit is shown in orange. The linear regression fit for a model that includes* $\texttt{horsepower}^2$ *is shown as a blue curve. The linear regression fit for a model that includes all polynomials of* `horsepower` *up to fifth-degree is shown in green.*

	Coefficient	Std. error	t-statistic	p-value
`Intercept`	56.9001	1.8004	31.6	< 0.0001
`horsepower`	−0.4662	0.0311	−15.0	< 0.0001
$\texttt{horsepower}^2$	0.0012	0.0001	10.1	< 0.0001

表3.10。 *For the* `Auto` *data set, least squares coefficient estimates associated with the regression of* `mpg` *onto* `horsepower` *and* $\texttt{horsepower}^2$.

拟合效果似乎明显优于仅包含线性项时的拟合。二次拟合的 R^2 为 0.688，而线性拟合为 0.606，且表3.10中二次项的 p 值高度显著。

如果包含$\texttt{horsepower}^2$使模型有如此大的改进，那为什么不包括$\texttt{horsepower}^3$、$\texttt{horsepower}^4$，甚至$\texttt{horsepower}^5$呢？图3.8中的绿色曲线显示了在模型(3.36)中包含直到五次多项式的所有项后所得到的拟合。结果的拟合似乎不必要地波动——也就是说，不清楚包含额外项是否真的导致了对数据更好的拟合。

我们刚刚描述的将线性模型扩展以适应非线性关系的方法被称为 *polynomial regres-sion*，因为我们已经在回归模型中包含了预测变量的多项式函数。我们将在第7章进一步探讨这种方法以及线性模型的其他非线性扩展。

3.3.3 Potential Problems

当我们将线性回归模型拟合到特定数据集时，可能会出现许多问题。其中最常见的是以下几点：

1. *Non-linearity of the response-predictor relationships.*
2. *Correlation of error terms.*
3. *Non-constant variance of error terms.*
4. *Outliers.* 5.
High-leverage points. 6.
Collinearity.

在实践中，识别并克服这些问题既是一门科学，也是一门艺术。无数书籍中的许多页面都围绕这一主题进行了阐述。由于线性回归模型并非我们此处的主要焦点，我们将仅提供一些关键点的简要概述。

1. 数据的非线性

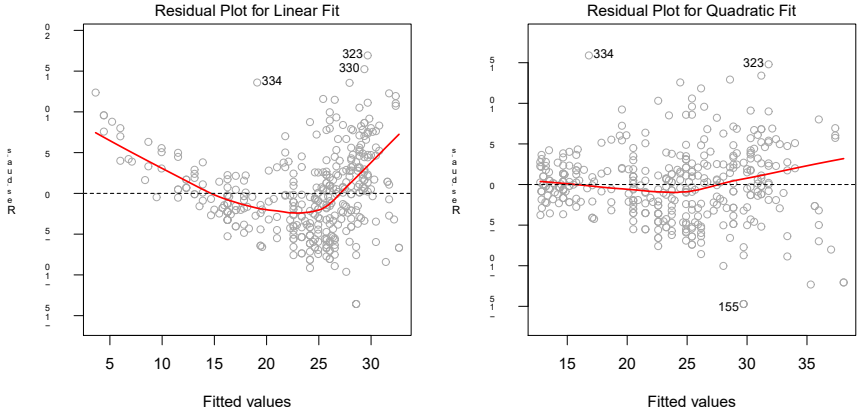


图3.9。 *Plots of residuals versus predicted (or fitted) values for the* `Auto` *data set. In each plot, the red line is a smooth fit to the residuals, intended to make it easier to identify a trend.* 左： *A linear regression of* `mpg` *on* `horsepower`. *A strong pattern in the residuals indicates non-linearity in the data.* 右： *A linear regression of* `mpg` *on* `horsepower` *and* $\texttt{horsepower}^2$. *There is little pattern in the residuals.*

线性回归模型假设预测变量与响应变量之间存在直线关系。如果真实关系远非线性，那么我们几乎所有的拟合结论都值得怀疑。此外，模型的预测精度可能会显著降低。

Residual plots 是识别非线性的有用图形工具。
给定一个简单线性回归模型，我们可以绘制残差，{v*}

residual plot

$y_i - \hat{y}_i$，与预测变量 x_i 相比。在多元回归模型的情况下，由于存在多个预测变量，我们改为绘制残差与预测（或 *fitted*）值 $\hat{y}_i$ 的关系图。理想情况下，残差图将显示无
可识别的模式。模式的存在可能表明线性模型的某些方面存在问题。

fitted

图3.9左侧面板展示了在3.8图中所示的`Auto`数据集上，从线性回归模型将`mpg`对`horsepower`进行回归后的残差图。红色线条是对残差的平滑拟合，其目的是便于识别任何趋势。残差呈现出明显的U形，这强烈表明数据中存在非线性关系。相比之下，图3.9右侧面板显示了包含二次项模型（3.36）的残差图。残差中似乎几乎没有模式，这表明二次项改善了对数据的拟合。

如果残差图表明数据中存在非线性关联，那么一种简单的方法是在回归模型中使用预测变量的非线性变换，例如对数X、$\sqrt{X}$和X^2。在本书后面的章节中，我们将讨论其他更先进的非线性方法来解决这个问题。

2. 误差项的相关性

线性回归模型的一个重要假设是误差项$\epsilon_1, \epsilon_2, \ldots, \epsilon_n$不相关。这意味着什么呢？例如，如果误差不相关，那么ϵ_i为正的事实对于ϵ_{i+1}的符号提供的信息很少或没有信息。为估计的回归系数或拟合值计算的标准误是基于误差项不相关的假设。如果实际上误差项之间存在相关性，那么估计的标准误往往会低估真实的标准误。因此，置信区间和预测区间会比它们应该的更窄。例如，一个95%置信区间实际上包含参数真实值的概率可能远低于0.95。此外，与模型相关的p值会低于它们应该的值；这可能导致我们错误地得出结论，认为某个参数在统计上是显著的。简而言之，如果误差项相关，我们可能对模型抱有不合理的信心。

作为一个极端的例子，假设我们意外地将数据加倍，导致观测值和误差项成对相同。如果我们忽略这一点，我们的标准误计算将会如同我们有一个大小为2n的样本，而实际上我们只有n个样本。对于2n个样本，我们估计的参数将与n个样本相同，但置信区间将因因子$\sqrt{2}$而变窄！

–

为什么误差项之间会出现相关性？这种相关性在*time series*数据的背景下频繁出现，该数据由观-
在离散时间点上获得测量值的观测。在许多情况下，相邻时间点获得的观测值会有正相关的误差。为了确定给定数据集是否如此，我们可以将模型的残差绘制为时间的函数。如果误差不相关，那么应该没有可辨别的模式。

time series

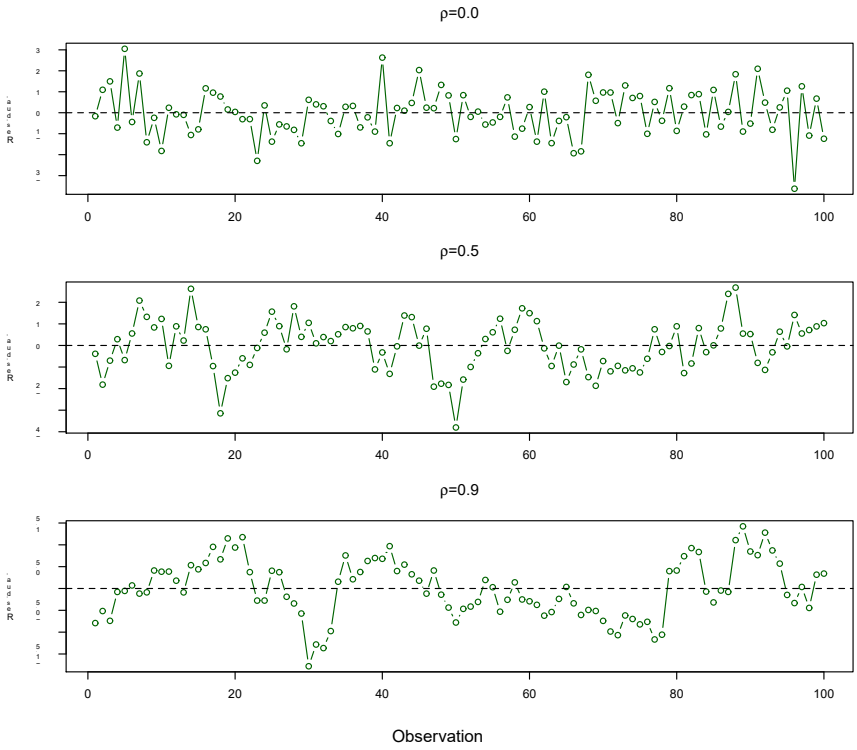


图3.10。*Plots of residuals from simulated time series data sets generated with differing levels of correlation ρ between error terms for adjacent time points.*

tern. 另一方面，如果误差项正相关，那么我们可能会在残差中看到*tracking*——也就是说，相邻残差可能
相似的数值。图3.10提供了一个示例。在顶部面板中，我们看到了对未相关误差生成的数据进行线性回归拟合后的残差。残差中没有与时间相关的趋势的证据。相比之下，底部面板中的残差来自一个相邻误差相关性为0.9的数据集。现在，残差中有一个明显的模式——相邻的残差倾向于取相似的值。最后，中间面板展示了一个更温和的情况，其中残差的相关性为0.5。仍有跟踪的证据，但模式不那么清晰。

tracking

许多方法已经被开发出来，以恰当考虑时间序列数据中误差项的相关性。误差项之间的相关性也可能出现在时间序列数据之外。例如，考虑一项研究，其中根据个体的体重来预测其身高。如果研究中的某些个体来自同一家庭、饮食相同或暴露于相同的环境因素，那么误差不相关的假设可能会被违反。一般来说，误差不相关的假设对于线性回归以及其他统计方法都极其重要，而良好的实验设计对于降低这种相关性的风险至关重要。

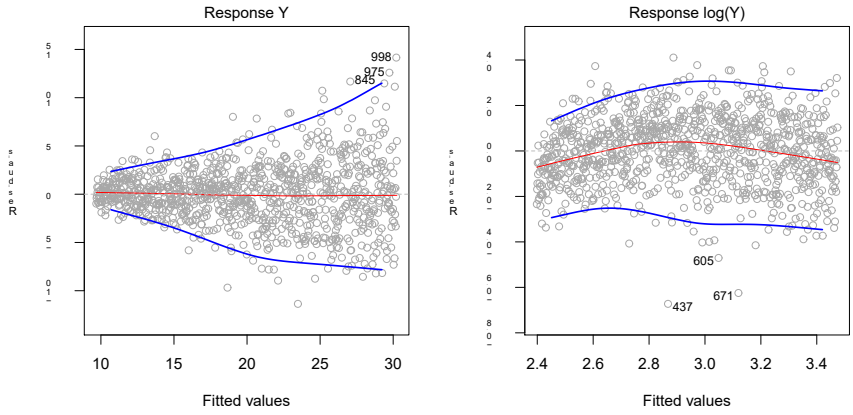


图3.11。 *Residual plots. In each plot, the red line is a smooth fit to the residuals, intended to make it easier to identify a trend. The blue lines track the outer quantiles of the residuals, and emphasize patterns.* 左： *The funnel shape indicates heteroscedasticity.* 右： *The response has been log transformed, and there is now no evidence of heteroscedasticity.*

3. 误差项的非恒定方差

线性回归模型的另一个重要假设是误差项具有恒定的方差，即 $\text{Var}(\epsilon_i) = \sigma^2$。与线性模型相关的标准误、置信区间和假设检验都依赖于这一假设。

不幸的是，误差项的方差往往非恒定。例如，误差项的方差可能随着响应值的增加而增大。可以通过在残差图。图3.11左侧面板展示了一个例子，其中残差的幅度倾向于随拟合值的增加而增大。面对这一问题时，一种可能的解决方案是使用凹函数，如对数Y或$\sqrt{Y}$，来转换响应变量Y。这样的转换导致对较大响应值的更大程度的收缩，从而减少了异方差性。图3.11右侧面板显示了使用对数Y转换响应变量后的残差图。现在，残差似乎具有恒定的方差，尽管数据中存在一些轻微非线性关系的证据。

异方差性

有时我们对每个响应的方差有很好的了解。例如，第i个响应可能是n_i次原始观测的平均值。如果这些原始观测彼此不相关，且方差为σ^2，那么它们的平均值的方差为$\sigma_i^2 = \sigma^2/n_i$。在这种情况下，一个简单的补救措施是通过*weighted least squares*来拟合我们的模型，权重与方差的倒数成正比——即本例中的 $w_i = n_i$。大多数线性回归软件都允许设置观测权重。

weighted least squares

4. 异常值

一个*outlier*是指y_i与由异常值预测的值相差甚远的点

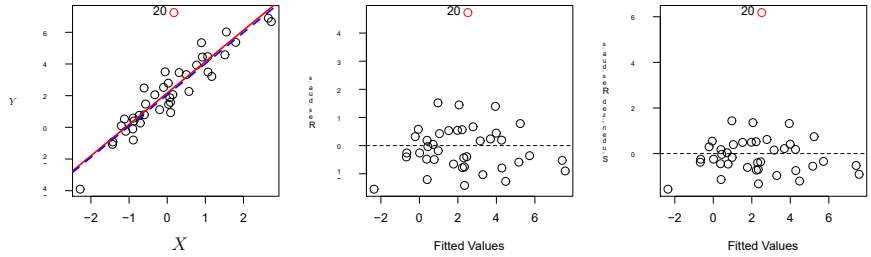


图3.12。左：*The least squares regression line is shown in red, and the regression line after removing the outlier is shown in blue.* 中：*The residual plot clearly identifies the outlier.* 右：*The outlier has a studentized residual of* 6 *; typically we expect values between* -3 *and* 3.

异常值可能由多种原因产生，例如数据收集过程中对某个观测值的错误记录。

图3.12左侧面板中的红点（观测20）展示了一个典型的离群点。红色实线是最小二乘回归拟合线，而蓝色虚线是移除离群点后的最小二乘拟合线。在这种情况下，移除离群点对最小二乘线影响很小：斜率几乎不变，截距仅有微小减少。对于预测值并无异常的离群点而言，对最小二乘拟合影响甚微是常见现象。然而，即便离群点对最小二乘拟合影响不大，也可能引发其他问题。例如，在本例中，当离群点包含在回归中时，RSE为1.09，而移除离群点后仅为0.77。由于RSE用于计算所有置信区间和p值，单个数据点导致的如此显著增加可能对拟合的解释产生影响。同样，包含离群点导致R^2从0.892下降至0.805。

残差图可用于识别异常值。在本例中，异常值在图3.12中间面板所示的残差图中清晰可见。但在实践中，很难确定残差需要多大才能将某个点视为异常值。为了解决这个问题，我们可以绘制*studentized residuals*，而不是绘制残差。*residuals*通过将每个残差e_i除以其估计标准差来计算

studentized residual

错误。学生化残差的绝对值大于3的观测值可能是异常值。在图3.12的右侧面板中，异常值的学生化残差超过6，而所有其他观测值的学生化残差介于-2和2之间。

如果我们认为异常值是由于数据收集或记录错误而产生的，那么一个简单的解决方法就是删除该观测值。然而，应该谨慎行事，因为异常值可能反而表明模型存在缺陷，例如缺少某个预测变量。

5. 高杠杆点

我们刚刚看到，离群点是那些在给定预测变量 x_i 时响应变量 y_i 异常观测值。相反，具有 *high leverage* 的观测值
有x_i的不寻常值。例如，左侧的第41个观测值

high leverage

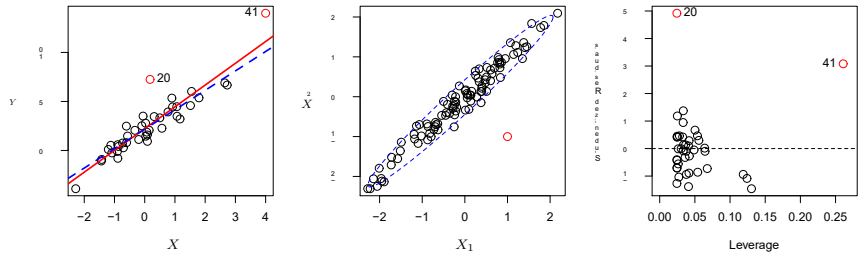


图3.13. 左：*Observation 41 is a high leverage point, while 20 is not. The red line is the fit to all the data, and the blue line is the fit with observation 41 removed.* 中：*The red observation is not unusual in terms of its* X_1 *value or its* X_2 *value, but still falls outside the bulk of the data, and hence has high leverage.* 右：*Observation* 41 *has a high leverage and a high residual.*

图3.13的面板具有高杠杆效应，因为该观测值的预测变量值相对于其他观测值较大。（注意，图3.13中显示的数据与图3.12中显示的数据相同，但添加了一个高杠杆观测值。）红色实线是对数据的最小二乘拟合，而蓝色虚线是去除观测41后产生的拟合。比较图3.12和3.13的左侧面板，我们观察到去除高杠杆观测值对最小二乘线的影响远大于去除离群值。事实上，高杠杆观测值往往对估计的回归线有相当大的影响。如果最小二乘线仅受几个观测值的严重影响，这值得担忧，因为这些点的任何问题都可能使整个拟合失效。因此，识别高杠杆观测值非常重要。

在简单线性回归中，高杠杆点相对容易识别，因为我们只需查找预测变量值超出观测值正常范围的观测点。但在包含众多预测变量的多元线性回归中，可能存在这样一个观测点：它在每个单独预测变量的值域内都处于正常范围，但从全部预测变量的整体来看却显得异常。图3.13的中间面板给出了一个示例，数据集包含两个预测变量X_1和X_2。大多数观测点的预测变量值落在蓝色虚线椭圆内，但红色观测点明显超出这一范围。然而，无论是X_1的值还是X_2的值，都不算异常。因此，如果我们仅考察X_1或仅考察X_2，就无法发现这个高杠杆点。这个问题在包含两个以上预测变量的多元回归场景中更为突出，因为此时没有简单的方法可以同时绘制数据的所有维度。

为了量化一个观测值的杠杆作用，我们计算 *leverage statistic*。该统计量的较大值表明一个观测值具有高
杠杆作用。对于简单线性回归，

leverage statistic

$$h_i = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum_{i'=1}^{n}(x_{i'} - \bar{x})^2}. \tag{3.37}$$

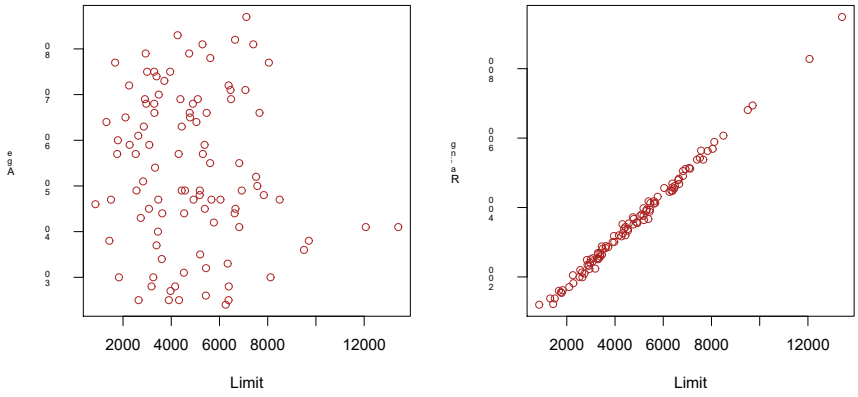


图3.14。*Scatterplots of the observations from the* `Credit` *data set.* 左：*A plot of* `age` *versus* `limit`. *These two variables are not collinear.* 右：*A plot of* `rating` *versus* `limit`. *There is high collinearity.*

从这个方程可以清楚地看出，h_i 随着 x_i 到 $\bar{x}$ 的距离增加而增大。h_i 可以简单扩展到多个预测变量的情形，不过我们在此不提供公式。杠杆统计量 h_i 始终介于 $1/n$ 和 1 之间，并且所有观测值的平均杠杆始终等于 $(p+1)/n$。因此，如果某个给定观测值的杠杆统计量远远超过 $(p+1)/n$，那么我们可能会怀疑对应点具有高杠杆。

图3.13右侧面板展示了图3.13左侧面板数据的学生化残差与h_i的关系图。观测41因其极高的杠杆统计量及较高的学生化残差而显得突出。换言之，它既是一个异常值，也是一个高杠杆观测点。这是一种特别危险的组合！该图还揭示了观测20对图3.12中最小二乘拟合影响相对较小的原因：其杠杆率较低。

6. 共线性

collinearity

Collinearity 指的是两个或更多预测变量所处的情形彼此密切相关。共线性的概念在图3.14中使用`Credit`数据集进行了说明。在图3.14的左侧面板中，两个预测变量`limit`和`age`似乎没有明显的关系。相比之下，在图3.14的右侧面板中，预测变量`limit`和`rating`彼此高度相关，我们说它们是*collinear*。共线性的存在在回归背景下可能会带来问题，因为很难分离出共线变量对响应的个体效应。换句话说，由于`limit`和`rating`倾向于同增同减，因此很难确定它们各自与响应`balance`的关联方式。

图3.15展示了共线性可能导致的一些困难。图3.15左侧面板是RSS（3.22）的等高线图，该RSS与`balance`对`limit`和`age`回归的不同可能系数估计相关联。每个椭圆代表一组系数

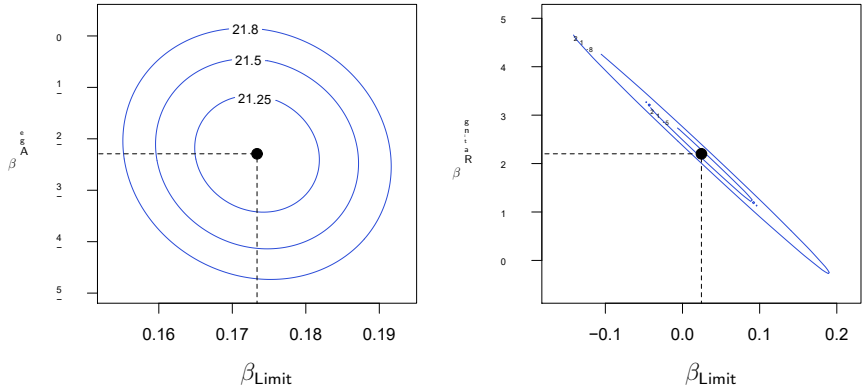


图3.15。*Contour plots for the RSS values as a function of the parameters* β *for various regressions involving the* `Credit` *data set. In each plot, the black dots represent the coefficient values corresponding to the minimum RSS. 左: A contour plot of RSS for the regression of* `balance` *onto* `age` *and* `limit`*. The minimum value is well defined. 右: A contour plot of RSS for the regression of* `balance` *onto* `rating` *and* `limit`*. Because of the collinearity, there are many pairs* $(\beta_{\text{Limit}}, \beta_{\text{Rating}})$ *with a similar value for RSS.*

对应于相同RSS的椭圆，越靠近中心的椭圆RSS值越低。黑点及相关的虚线代表使RSS尽可能最小的系数估计——换句话说，这些是最小二乘估计。`limit`和`age`的坐标轴已按比例调整，使得图中包含的系数估计值在最小二乘估计值两侧各达四个标准误。因此，该图包含了系数的所有可能合理值。例如，我们可以看到真实的`limit`系数几乎肯定在0.15到0.20之间。

相反，图3.15右侧面板展示了将`balance`对`limit`和`rating`进行回归时，与可能的系数估计值相关的RSS等高线图，我们知道`limit`和`rating`高度共线。此时，等高线沿着一条狭窄的山谷延伸；存在一个广泛的系数估计值范围，这些值会导致相同的RSS值。因此，数据中的微小变化都可能导致产生最小RSS的系数值对——即最小二乘估计——沿此山谷任意移动。这导致系数估计值存在极大的不确定性。注意，`limit`系数的尺度现在大约从−0.2到0.2；与包含`age`的回归中`limit`系数的合理范围相比，这增加了八倍。有趣的是，尽管`limit`和`rating`系数现在各自具有更大的不确定性，但它们几乎肯定位于此等高线山谷的某处。例如，我们不会期望`limit`和`rating`系数的真实值分别为−0.1和1，尽管这样的值对每个系数单独而言是合理的。

由于共线性降低了回归系数估计的准确性，它导致 $\hat{\beta}_j$ 的标准误增大。回想一下，每个预测变量的 t 统计量是通过将 $\hat{\beta}_j$ 除以其标准

		Coefficient	Std. error	t-statistic	p-value
Model 1	Intercept	−173.411	43.828	−3.957	< 0.0001
	age	−2.292	0.672	−3.407	0.0007
	limit	0.173	0.005	34.496	< 0.0001
Model 2	Intercept	−377.537	45.254	−8.343	< 0.0001
	rating	2.202	0.952	2.312	0.0213
	limit	0.025	0.064	0.384	0.7012

表3.11。 *The results for two multiple regression models involving the* Credit *data set are shown. Model 1 is a regression of* balance *on* age *and* limit*, and Model 2 a regression of* balance *on* rating *and* limit*. The standard error of* $\hat{\beta}_{\text{limit}}$ *increases 12-fold in the second regression, due to collinearity.*

错误。因此，共线性导致t统计量下降。结果是，在存在共线性的情况下，我们可能无法拒绝H_0：$\beta_j = 0$。这意味着假设检验的*power*——正确检测到一个 *non-zero* 系数——因共线性而减少。

电力

表3.11比较了两个单独多元回归模型得到的系数估计值。第一个是balance对age和limit的回归，第二个是balance对rating和limit的回归。在第一个回归中，age和limit都非常显著，p值非常小。在第二个回归中，limit和rating之间的共线性导致limit系数估计值的标准误增加了12倍，p值增加到0.701。换句话说，由于共线性的存在，limit变量的重要性被掩盖了。为避免这种情况，在拟合模型时，最好识别并解决潜在的共线性问题。

检测共线性的一种简单方法是查看预测变量的相关矩阵。该矩阵中绝对值较大的元素表明存在一对高度相关的变量，因此数据中存在共线性问题。不幸的是，并非所有共线性问题都能通过检查相关矩阵来检测：即使没有任何一对变量具有特别高的相关性，三个或更多变量之间也可能存在共线性。我们将这种情况称为 *multicollinearity*。

多重共线性

与其检查相关矩阵，更好的评估多重共线性的方法是计算 *variance inflation factor* (VIF)。VIF 是拟合完整模型时$\hat{\beta}_j$的方差与单独拟合$\hat{\beta}_j$时的方差的比值。VIF的最小可能值为1，这表明完全不存在共线性。通常在实践中，预测变量之间存在少量共线性。根据经验法则，VIF值超过5或10表明存在有问题的共线性程度。每个变量的VIF可以使用以下公式计算

方差膨胀因子

$$\text{VIF}(\hat{\beta}_j) = \frac{1}{1 - R^2_{X_j|X_{-j}}},$$

其中 $R^2_{X_j|X_{-j}}$ 是来自将 X_j 回归到所有其他预测变量上的 R^2。如果 $R^2_{X_j|X_{-j}}$ 接近于一，则存在共线性，因此 VIF 将会很大。

在 Credit 数据中，将 balance 对 age、rating 和 limit 进行回归，结果表明预测变量的 VIF 值分别为 1.01、160.67 和 160.59。正如我们所怀疑的，数据中存在相当严重的共线性！

当面临共线性问题时，有两种简单的解决方法。第一种是从回归中剔除一个有问题的变量。这通常可以在不对回归拟合造成太大影响的情况下完成，因为共线性的存在意味着该变量在存在其他变量的情况下所提供的关于响应的信息是冗余的。例如，如果我们用age和limit对balance进行回归，而不包含rating预测变量，那么得到的VIF值接近最小值1，并且R^2从0.754降至0.75。因此，从预测变量集中剔除rating有效地解决了共线性问题，而没有损害拟合效果。第二种解决方法是将共线变量合并为一个单一的预测变量。例如，我们可以取limit和rating标准化版本的平均值，以创建一个衡量*credit worthiness*的新变量。

3.4 营销计划

我们现在简要地回到本章开头提出的关于Advertising数据的七个问题。

1. *Is there a relationship between sales and advertising budget?*
   这个问题可以通过拟合一个将 sales 对 TV、radio 和 newspaper 进行多元回归的模型来回答，如（3.20）所示，并检验假设 H_0: $\beta_{\texttt{TV}} = \beta_{\texttt{radio}} = \beta_{\texttt{newspaper}} = 0$。在第3.2.2节中，我们展示了$F$统计量可用于判断是否应拒绝这一原假设。在此情况下，表3.6中与F统计量对应的p值非常低，明确表明广告与销售额之间存在关系。

2. *How strong is the relationship?*
   我们在第 3.1.3 节中讨论了模型准确性的两种度量。首先，RSE 估计响应值相对于总体回归线的标准差。对于 Advertising 数据，RSE 为 1.69 个单位，而响应值的均值为 14.022，表明百分比误差约为 12%。其次，R^2 统计量记录了由预测变量解释的响应值变异性的百分比。预测变量解释了 sales 中近 90% 的方差。RSE 和 R^2 统计量见表 3.6。

3. *Which media are associated with sales?*
   为了回答这个问题，我们可以检查与每个预测变量的t统计量相关的p值（第3.1.2节）。在表3.4中展示的多元线性回归中，TV和radio的p值较低，但newspaper的p值并不低。这表明只有TV和radio与sales相关。在第6章中，我们将更详细地探讨这个问题。

4。*How large is the association between each medium and sales?*
我们在第3.1.2节中看到，$\hat{\beta}_j$的标准误差可用于构建β_j的置信区间。对于Advertising数据，我们可以利用表3.4中的结果，使用所有三个媒体预算作为预测变量，计算多元回归模型中系数的95%置信区间。置信区间如下：TV为(0.043, 0.049)，radio为(0.172, 0.206)，newspaper为$(-0.013, 0.011)$。TV和radio的置信区间较窄且远离零点，提供了这些媒体与sales相关的证据。但newspaper的区间包含零，表明在给定TV和radio值的情况下，该变量在统计上不显著。

我们在第3.3.3节看到，共线性可能导致非常宽泛的标准误差。newspaper相关的置信区间如此之宽，是否是由共线性引起的呢？对于TV、radio和newspaper，VIF得分分别为1.005、1.145和1.145，这表明没有共线性的证据。

为了单独评估每种媒体与销售额的关联，我们可以进行三个单独的简单线性回归。结果如表3.1和3.3所示。有证据表明TV与sales之间以及radio与sales之间存在极强的关联。当忽略TV和radio的值时，有证据表明newspaper与sales之间存在轻微关联。

5. *How accurately can we predict future sales?*
可以使用（3.21）来预测响应。与此估计相关的准确性取决于我们是要预测单个响应 $Y = f(X) + \epsilon$，还是平均响应 $f(X)$ (第3.2.2节)。如果是前者，我们使用预测区间；如果是后者，我们使用置信区间。预测区间总是比置信区间更宽，因为它们考虑了与 ϵ（不可约误差）相关的不确定性。

6. *Is the relationship linear?*
在第3.3.3节中，我们看到残差图可用于识别非线性。如果关系是线性的，那么残差图应显示无模式。在 Advertising 数据的情况下，我们在图3.5中观察到非线性效应，尽管这种效应也可以在残差图中观察到。在第3.3.2节中，我们讨论了在线性回归模型中包含预测变量的变换，以适应非线性关系。

7. *Is there synergy among the advertising media?*
标准线性回归模型假设预测变量与响应变量之间存在可加关系。可加模型易于解释，因为每个预测变量与响应变量之间的关联与其他预测变量的取值无关。然而，对于某些数据集，可加假设可能不切实际。在第3.3.2节中，我们展示了如何包含一个交互项

在回归模型中，为了适应非可加关系。与交互项相关的较小p值表明存在这种关系。图3.5提示Advertising数据可能不是可加的。在模型中包含交互项导致R^2显著增加，从大约90%到近97%。

3.5 线性回归与 K 最近邻的比较

如第 2 章所述，线性回归是一种 *parametric* 方法的示例，因为它假设 $f(X)$ 具有线性函数形式。参数方法有几个优点。它们通常易于拟合，因为只需估计少量系数。在线性回归的情况下，系数具有简单的解释，并且可以轻松进行统计显著性检验。但参数方法确实有一个缺点：根据其构造，它们对 $f(X)$ 的形式做出了强假设。如果指定的函数形式与真实情况相差甚远，而预测准确性又是我们的目标，那么参数方法将表现不佳。例如，如果我们假设 X 和 Y 之间存在线性关系，但真实关系远非线性，那么所得模型将无法很好地拟合数据，并且从中得出的任何结论都将令人怀疑。

相反，*non-parametric*方法并不显式地假设$f(X)$的参数形式，从而为执行回归提供了一种替代且更灵活的方法。我们在本书中讨论各种非参数方法。在这里，我们考虑最简单且最著名的非参数方法之一，*K-nearest neighbors regression* (KNN回归)。

K-nearest neighbors regression

KNN回归方法与第2章讨论的KNN分类器密切相关。给定K的值和一个预测点x_0，KNN回归首先识别出最接近x_0的K个训练观测值，用$\mathcal{N}_0$表示。然后，它使用$\mathcal{N}_0$中所有训练响应的平均值来估计$f(x_0)$。换句话说，

$$\hat{f}(x_0) = \frac{1}{K} \sum_{x_i \in \mathcal{N}_0} y_i.$$

图3.16展示了在一个含有p =个预测变量的数据集上两种KNN拟合的情况。当K =为1时的拟合显示在左侧面板，而右侧面板对应于K =为9。我们看到，当K =为1时，KNN拟合完美地插值了训练观察值，因此呈现为阶梯函数的形式。当K =为9时，KNN拟合仍然是阶梯函数，但对九个观察值进行平均导致恒定预测的区域要小得多，从而得到更平滑的拟合。通常，K的最优值取决于我们在第2章介绍的*bias-variance tradeoff*。K的较小值提供了最灵活的拟合，这将具有低偏差但高方差。这种方差是由于给定区域内的预测完全依赖于一个观察值。

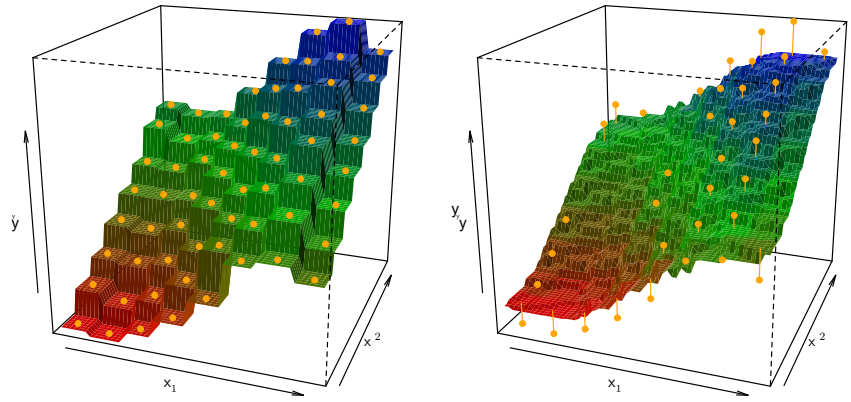


图3.16。 *Plots of* $\hat{f}(X)$ *using KNN regression on a two-dimensional data set with* 64 *observations (orange dots). 左：* $K = 1$ *results in a rough step function fit. 右：* $K = 9$ *produces a much smoother fit.*

相反，K 的较大值提供了更平滑且变化更小的拟合；一个区域的预测是多个点的平均值，因此改变一个观测值的影响较小。然而，平滑可能会通过掩盖 $f(X)$ 中的某些结构而导致偏差。在第五章中，我们介绍了几种估计测试错误率的方法。这些方法可用于识别 KNN 回归中 K 的最优值。

在何种情况下，诸如最小二乘线性回归这样的参数方法会优于如KNN回归这类非参数方法？答案很简单：
the parametric approach will outperform the non-parametric approach if the parametric form that has been selected is close to the true form of f。图3.17提供了一个例子，数据源自一维线性回归模型。黑色实线代表$f(X)$，而蓝色曲线对应使用$K = 1$和$K = 9$的KNN拟合。在这个情形下，$K = 1$的预测变化太大，而更平滑的$K = 9$拟合则更接近$f(X)$。然而，由于真实关系是线性的，非参数方法很难与线性回归相抗衡：非参数方法在方差上付出的代价无法通过偏差的减少来抵消。图3.18左侧面板中的蓝色虚线表示对相同数据的线性回归拟合，几乎完美。图3.18右侧面板显示，对于这批数据，线性回归优于KNN。以$1/K$为函数绘制的绿色实线表示KNN的测试集均方误差(MSE)。KNN的误差远高于黑色虚线，即线性回归的测试MSE。当K值较大时，KNN在MSE方面仅比最小二乘回归略差。而当K较小时，其表现则差得多。

在实践中，X 与 Y 之间的真实关系很少是精确线性的。图 3.19 考察了在 X 与 Y 之间关系非线性程度不断增加的情况下，最小二乘回归与 KNN 的相对表现。在最上面一行中，真实关系几乎是线性的。在这种情况下，我们看到线性回归的测试 MSE 仍然更优

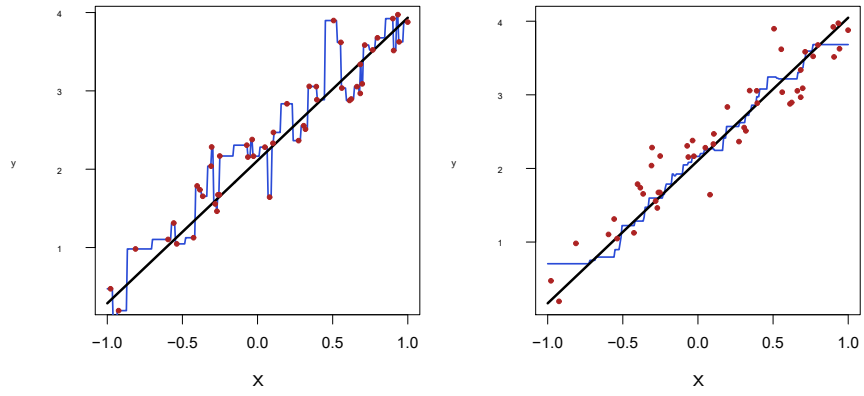


图3.17. *Plots of $\hat{f}(X)$ using KNN regression on a one-dimensional data set with* 5 0 *observations. The true relationship is given by the black solid line.* 左图: *The blue curve corresponds to $K = 1$ and interpolates (i.e. passes directly through) the training data.* 右图: *The blue curve corresponds to $K = 9$, and represents a smoother fit.*

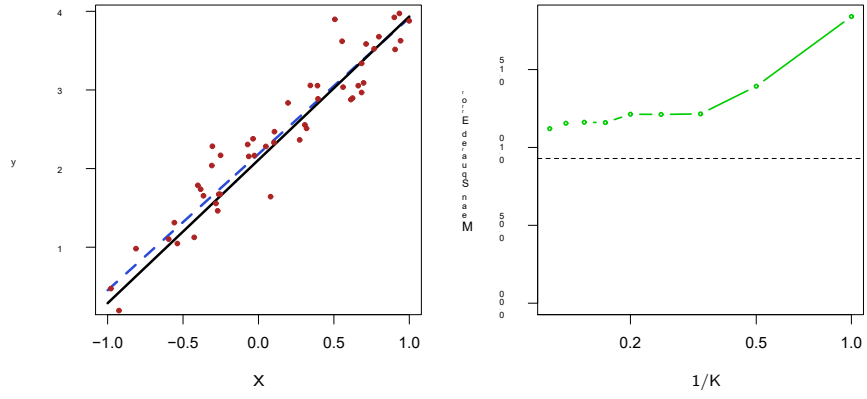


图3.18. *The same data set shown in Figure 3.17 is investigated further.* 左: *The blue dashed line is the least squares fit to the data. Since $f(X)$ is in fact linear (displayed as the black line), the least squares regression line provides a very good estimate of $f(X)$.* 右: *The dashed horizontal line represents the least squares test set MSE, while the green solid line corresponds to the MSE for KNN as a function of $1/K$ (on the log scale). Linear regression achieves a lower test MSE than does KNN regression, since $f(X)$ is in fact linear. For KNN regression, the best results occur with a very large value of K, corresponding to a small value of $1/K$.*

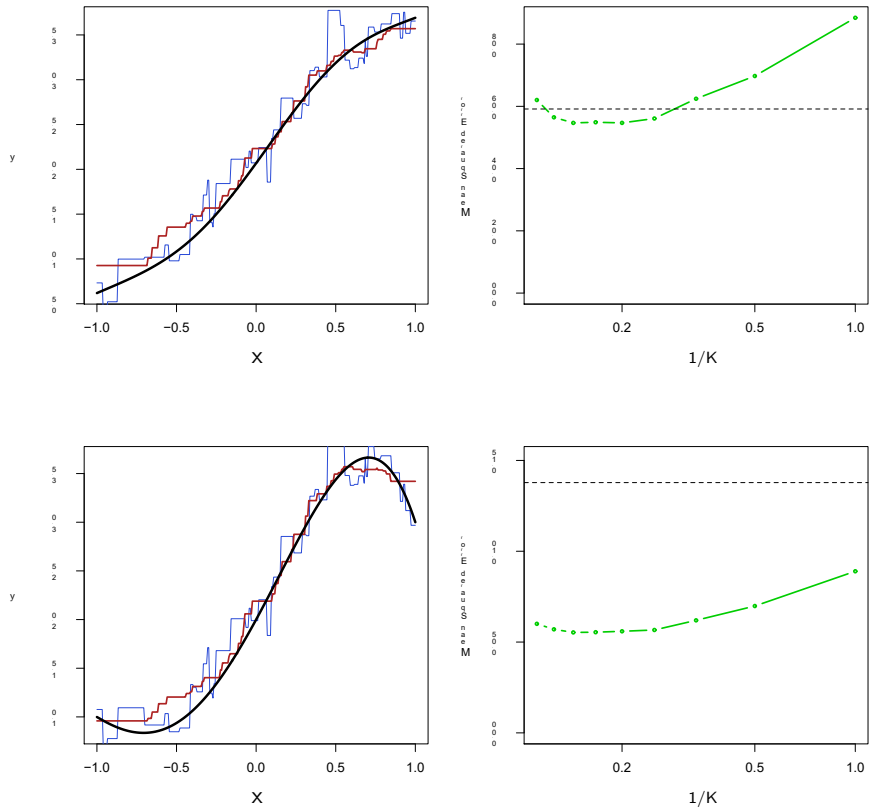


图3.19。左上：*In a setting with a slightly non-linear relationship between X and Y (solid black line), the KNN fits with $K = 1$ (blue) and $K = 9$ (red) are displayed.* 右上：*For the slightly non-linear data, the test set MSE for least squares regression (horizontal black) and KNN with various values of $1/K$ (green) are displayed.* 左下和右下：*As in the top panel, but with a strongly non-linear relationship between X and Y.*

与低 K 值的 KNN 相比。然而，对于 $K \geq 4$，KNN 优于线性回归。第二行展示了与线性的更显著偏离。在这种情况下，对于所有 K 值，KNN 都显著优于线性回归。注意，随着非线性程度的增加，非参数 KNN 方法的测试集 MSE 几乎没有变化，但线性回归的测试集 MSE 大幅增加。

图3.18和3.19展示了当关系为线性时，KNN的表现略逊于线性回归，但在非线性情况下，KNN远优于线性回归。在真实场景中，若真实关系未知，人们可能会倾向于认为KNN应优于线性回归，因为如果真实关系是线性的，KNN最多只会略差于线性回归；而如果真实关系是非线性的，KNN可能带来显著更好的结果。然而实际上，即使真实关系高度非线性，KNN仍可能提供比线性回归更差的结果。特别是，图3.18

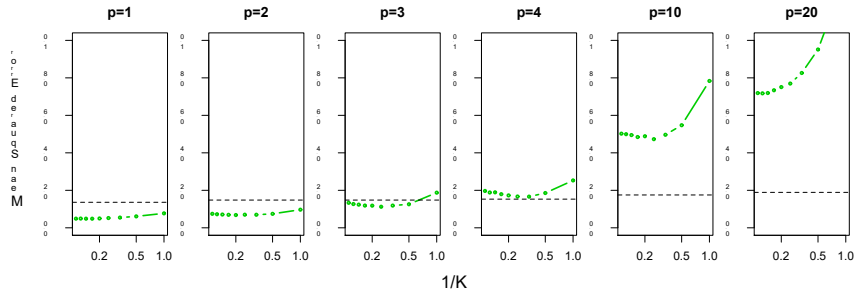


图3.20。 *Test MSE for linear regression (black dashed lines) and KNN (green curves) as the number of variables p increases. The true function is non-linear in the first variable, as in the lower panel in Figure 3.19, and does not depend on the additional variables. The performance of linear regression deteriorates slowly in the presence of these additional noise variables, whereas KNN's performance degrades much more quickly as p increases.*

并且3.19展示了p =1个预测变量的设置。但在更高维度中，KNN通常比线性回归表现更差。

图3.20考虑了与图3.19第二行相同的强非线性情况，只是我们添加了额外的*noise*个与响应无关的预测变量。当p =为1或p =为2时，KNN优于线性回归。但对于p =为3，结果好坏参半，而对于$p \geq$为4，线性回归优于KNN。事实上，维度的增加仅导致线性回归测试集MSE的小幅恶化，但却导致KNN的MSE增加了十倍以上。这种随着维度增加而性能下降是KNN的一个常见问题，其原因在于在高维情况下，样本量实际上减少了。在这个数据集中有50个训练观测值；当p =为1时，这提供了足够的信息来准确估计$f(X)$。然而，将50个观测值分散在p =为20个维度上会导致一种现象，即给定观测值没有*nearby neighbors*——这就是所谓的*curse of dimensionality*。即，

curse of dimensionality

{n19} 个观测值中最接近给定测试观测值 x_0 的那些，在 p 较大时，在 p 维空间中可能距离 x_0 很远，导致对 $f(x_0)$ 的预测非常差，因此 KNN 拟合效果不佳。一般来说，当每个预测变量的观测值数量较少时，参数方法往往优于非参数方法。

即使在维度较小的情况下，从可解释性的角度来看，我们可能更倾向于线性回归而非KNN。如果KNN的测试MSE仅略低于线性回归的测试MSE，我们可能愿意牺牲一点预测准确性，以换取一个仅用几个系数就能描述的简单模型，并且该模型还提供了p值。

3.6 实验：线性回归

3.6.1 Importing packages

我们在此顶层导入标准库。

In [1]:
```
import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
```

新的导入

在整个实验过程中，我们将介绍新的函数和库。不过，我们在此处导入它们是为了强调这些是本次实验中的新代码对象。将导入语句保持在笔记本的顶部可以使代码更易于阅读，因为只需浏览前几行就能了解到使用了哪些库。

In [2]:
```
import statsmodels.api as sm
```

我们将在需要时提供有关以下功能的相关细节。

除了导入整个模块之外，还可以从给定模块中仅导入少量项目。这将有助于保持*namespace*的清洁。

namespace

我们将使用 `statsmodels` 包中的几个特定对象，我们在此处导入。

statsmodels

In [3]:
```
from statsmodels.stats.outliers_influence \
     import variance_inflation_factor as VIF
from statsmodels.stats.anova import anova_lm
```

由于上面的某个 import 语句非常长，我们插入了一个换行符 \ 以提高可读性。

我们还将使用本书中为实验编写的一些函数，这些函数位于`ISLP`包中。

In [4]:
```
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
                         summarize,
                         poly)
```

检查对象和命名空间

函数 `dir()` 提供命名空间中的对象列表。

dir()

In [5]:
```
dir()
```

Out[5]:
```
['In',
 'MS',
 '_',
 '__',
 '___',
 '__builtin__',
 '__builtins__',
 ...
```

```
 'poly',
 'quit',
 'sm',
 'summarize']
```

这展示了Python在顶层能找到的所有内容。有些对象，如__builtins__，包含对内置函数（如print()）的引用。

每个 Python 对象都有自己的命名空间概念，也可以通过 dir() 访问。这将包括对象的属性和与之关联的任何方法。例如，我们在数组的列表中看到 'sum'。

In [6]:
```
A = np.array([3,5,11])
dir(A)
```

Out[6]:
```
  ...
 'strides',
 'sum',
 'swapaxes',
  ...
```

这表明对象 A.sum 存在。在此情况下，它是一个可用于计算数组 A 之和的方法，通过输入 A.sum? 可以看到。

In [7]:
```
A.sum()
```

Out[7]: 19

3.6.2 Simple Linear Regression

在本节中，我们将使用ISLP.models中的ModelSpec()变换来构建模型矩阵（也称为设计矩阵）。

我们将使用 Boston 住房数据集，它包含在 ISLP 包中。Boston 数据集记录了波士顿周围506个街区的 medv (房屋中位价)。我们将建立一个回归模型，使用13个预测变量来预测 medv，例如 rmvar (每栋房屋的平均房间数)、age (1940年前建造的自住单元比例)，以及 lstat (低社会经济地位家庭的百分比)。我们将使用 statsmodels 来完成这项任务，这是一个 Python 包，实现了若干常用的回归方法。

我们在 ISLP 包中包含了一个简单的加载函数 load_data()。 load_data()
年：

In [8]:
```
Boston = load_data("Boston")
Boston.columns
```

Out[8]:
```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis',
       'rad', 'tax', 'ptratio', 'black', 'lstat', 'medv'],
      dtype='object')
```

输入 Boston? 以了解更多关于这些数据的信息。

我们首先使用 sm.OLS() 函数来拟合一个简单的线性回归 sm.OLS() 模型。我们的响应变量将是 medv，而 lstat 将是唯一的预测变量。对于这个模型，我们可以手动创建模型矩阵。

```
In [9]: X = pd.DataFrame({'intercept': np.ones(Boston.shape[0]),
                          'lstat': Boston['lstat']})
        X[:4]
```

```
Out[9]:    intercept  lstat
       0        1.0   4.98
       1        1.0   9.14
       2        1.0   4.03
       3        1.0   2.94
```

我们提取响应，并拟合模型。

```
In [10]: y = Boston['medv']
         model = sm.OLS(y, X)
         results = model.fit()
```

注意，sm.OLS() 并不拟合模型；它指定模型，然后 model.fit() 执行实际的拟合。

我们的 ISLP 函数 summarize() 生成一个简单的参数表。 summarize()
ter 估计值、其标准误、t 统计量和 p 值。该函数接受一个参数，例如此处由 fit 方法返回的对象 results，并返回这样的摘要。

```
In [11]: summarize(results)
```

```
Out[11]:                coef  std err       t  P>|t|
         intercept   34.5538    0.563  61.415    0.0
         lstat       -0.9500    0.039 -24.528    0.0
```

在我们描述其他处理拟合模型的方法之前，我们概述了一个更有用且通用的框架，用于构建模型矩阵 X。

使用变换：拟合与变换

我们上面的模型只有一个预测变量，构建 X 很直接。在实践中，我们经常拟合具有多个预测变量的模型，通常是从数组或数据框中选择的。我们可能希望在拟合模型之前对变量进行变换，指定变量之间的交互作用，并将某些特定变量扩展为变量集（例如多项式）。sklearn 包有一个特定的概念

对于此类任务：一个 *transform*。变换是一个对象，它在创建时接受一些参数作为参数。该对象有两个主要方法：fit() 和 transform()。 sklearn

我们在ISLP库中通过变换ModelSpec()提供了一种指定模型并构建模型矩阵的通用方法。 .fit()
.transform()
ModelSpec() (在前言中重命名了MS())创建了一个变换对象，然后使用一对方法transform()和fit()来构造相应的模型矩阵。 ModelSpec()

我们首先使用Boston数据框中的单一预测变量lstat来描述这个简单回归模型的过程，但在本书的这个和其他实验中，我们会反复在更复杂的任务中使用它。在我们的案例中，转换是由表达式design = MS(['lstat'])创建的。

fit() 方法接收原始数组，并可根据变换对象中的规定对其进行一些初始计算。例如，它可以计算用于中心化和缩放的平均值和标准差。transform() 方法将拟合好的变换应用于数据数组，并生成模型矩阵。

In [12]:
```
design = MS(['lstat'])
design = design.fit(Boston)
X = design.transform(Boston)
X[:4]
```

Out[12]:

	intercept	lstat
0	1.0	4.98
1	1.0	9.14
2	1.0	4.03
3	1.0	2.94

在这个简单的例子中，fit() 方法作用很小；它只是检查 design 中指定的变量 'lstat' 是否存在于 Boston 中。然后 transform() 构造包含两列的模型矩阵：一个 intercept 和变量 lstat。

这两个操作可以与fit_transform()方法结合使用。

.fit_transform()

In [13]:
```
design = MS(['lstat'])
X = design.fit_transform(Boston)
X[:4]
```

Out[13]:

	intercept	lstat
0	1.0	4.98
1	1.0	9.14
2	1.0	4.03
3	1.0	2.94

请注意，正如前一个代码块中分两步完成时那样，design 对象因 fit() 操作而改变。当我们拟合涉及交互和转换的更复杂模型时，这个管道的威力将变得更加清晰。

让我们回到已拟合的回归模型。对象 results 有多种可用于推断的方法。我们已经介绍了一个用于显示拟合基本要素的函数 summarize()。对于拟合的完整且较为详尽的摘要，我们可以使用 summary() 方法（输出未显示）。

In [14]:
```
results.summary()
```

拟合系数也可以通过 results 的 params 属性来获取。

In [15]:
```
results.params
```

```
Out[15]: intercept    34.553841
         lstat        -0.950049
         dtype: float64
```

get_prediction() 方法可用于获得预测，并且为给定lstat值时，生成medv预测的置信区间和预测区间。 .get_prediction()

我们首先创建一个新的数据框，本例中仅包含变量 lstat，其中含有我们希望进行预测的该变量的值。然后，我们使用 design 的 transform() 方法来创建相应的模型矩阵。

```
In [16]: new_df = pd.DataFrame({'lstat':[5, 10, 15]})
         newX = design.transform(new_df)
         newX
```

```
Out[16]:   intercept   lstat
         0        1.0       5
         1        1.0      10
         2        1.0      15
```

接下来我们计算在newX处的预测值，并通过提取predicted_mean属性来查看它们。

```
In [17]: new_predictions = results.get_prediction(newX);
         new_predictions.predicted_mean
```

```
Out[17]: array([29.80359411, 25.05334734, 20.30310057])
```

We 可以为预测值生成置信区间 用途。

```
In [18]: new_predictions.conf_int(alpha=0.05)
```

```
Out[18]: array([[29.00741194, 30.59977628],
                [24.47413202, 25.63256267],
                [19.73158815, 20.87461299]])
```

通过设置obs=True来计算预测区间：

```
In [19]: new_predictions.conf_int(obs=True, alpha=0.05)
```

```
Out[19]: array([[17.56567478, 42.04151344],
                [12.82762635, 37.27906833],
                [ 8.0777421 , 32.52845905]])
```

例如，lstat 值为10时对应的95%置信区间为（24.47, 25.63），而95%预测区间为（12.82, 37.28）。正如预期的那样，置信区间和预测区间都以同一点为中心（当lstat等于10时，medv的预测值为25.05），但后者明显更宽。

接下来我们将使用DataFrame.plot.scatter()绘制medv和lstat，并且希望在结果图中添加回归线。 .plot.scatter()

定义函数

虽然ISLP包中有一个函数可以向现有图形添加线条，但我们借此机会定义我们的第一个函数来实现这一点。

def

In [20]:
```
def abline(ax, b, m):
    "Add a line with slope m and intercept b to ax"
    xlim = ax.get_xlim()
    ylim = [m * xlim[0] + b, m * xlim[1] + b]
    ax.plot(xlim, ylim)
```

上面说明了几个要点。首先，我们看到了定义函数的语法：`def funcname(...)`。该函数具有参数 `ax`，`b`，`m`，其中 `ax` 是现有图表的轴对象，`b` 是截距，`m` 是所需直线的斜率。可以通过包含如下附加可选参数，将其他绘图选项传递给 `ax.plot`：

In [21]:
```
def abline(ax, b, m, *args, **kwargs):
    "Add a line with slope m and intercept b to ax"
    xlim = ax.get_xlim()
    ylim = [m * xlim[0] + b, m * xlim[1] + b]
    ax.plot(xlim, ylim, *args, **kwargs)
```

`*args` 的添加允许向 `abline` 传入任意数量的非命名参数，而 `*kwargs` 则允许向 `abline` 传入任意数量的命名参数（如 `linewidth=3`）。在我们的函数中，我们将这些参数原样传递给上面的 `ax.plot`。有兴趣进一步了解函数的读者，请参阅 docs.python.org/tutorial 中关于定义函数的部分。

让我们使用新函数将这条回归线添加到 `medv` 与 `lstat` 的图中。

In [22]:
```
ax = Boston.plot.scatter('lstat', 'medv')
abline(ax,
       results.params[0],
       results.params[1],
       'r--',
       linewidth=3)
```

因此，对`ax.plot()`的最后一次调用是`ax.plot(xlim, ylim, 'r--', linewidth=3)`。我们使用了参数`'r--'`来生成一条红色虚线，并添加了一个参数使其宽度为3。有证据表明`lstat`和`medv`之间的关系存在非线性。我们将在本实验的后续部分探讨这一问题。

如上所述，已有一个向图中添加线条的函数——`ax.axline()`——但了解如何编写此类函数使我们能够创建更具表现力的显示。

接下来我们检查一些诊断图，其中一些已在第3.3.3节中讨论过。我们可以从`results`对象的属性中找到拟合值和残差。使用`get_influence()`方法计算描述回归模型的各种影响度量。正如我们

.get_influence()

将不会使用从 `subplots()` 返回作为第一个值的 `fig` 组件，我们简单地在下面的 `ax` 中捕获第二个返回值。

In [23]:
```
ax = subplots(figsize=(8,8))[1]
```

```
ax.scatter(results.fittedvalues, results.resid)
ax.set_xlabel('Fitted value')
ax.set_ylabel('Residual')
ax.axhline(0, c='k', ls='--');
```

我们使用 ax.axhline() 方法在 0 处添加一条参考水平线，.axhline() 表示它应为黑色（c='k'）并具有虚线线型（ls='--'）。

基于残差图（未显示），有一些非线性的证据。可以使用 get_influence() 方法返回值的 hat_matrix_diag 属性来计算任意数量预测变量的杠杆统计量。

In [24]:
```
infl = results.get_influence()
ax = subplots(figsize=(8,8))[1]
ax.scatter(np.arange(X.shape[0]), infl.hat_matrix_diag)
ax.set_xlabel('Index')
ax.set_ylabel('Leverage')
np.argmax(infl.hat_matrix_diag)
```

Out[24]: 374

np.argmax() 函数用于识别 np.argmax() 数组中最大元素的索引，该计算可选择沿数组的一个轴进行。在此情形下，我们对整个数组求最大值，以确定哪个观测值具有最大的杠杆统计量。

3.6.3 Multiple Linear Regression

为了使用最小二乘法拟合多元线性回归模型，我们再次使用ModelSpec()变换来构建所需的模型矩阵和响应。ModelSpec()的参数可以相当通用，但在这种情况下，列名列表就足够了。我们在此考虑使用两个变量lstat和age进行拟合。

In [25]:
```
X = MS(['lstat', 'age']).fit_transform(Boston)
model1 = sm.OLS(y, X)
results1 = model1.fit()
summarize(results1)
```

Out[25]:

	coef	std err	t	P>|t|
intercept	33.2228	0.731	45.458	0.000
lstat	-1.0321	0.048	-21.416	0.000
age	0.0345	0.012	2.826	0.005

请注意，我们如何将第一行精简为一个简洁的表达式，描述了X的构造。

Boston 数据集包含 12 个变量，因此如果要用所有预测变量进行回归，手动输入所有这些变量会很繁琐。相反，我们可以使用以下简写：

In [26]:
```
terms = Boston.columns.drop('medv')
terms
```

.columns.drop()

```
Out[26]: Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis',
                'rad', 'tax', 'ptratio', 'lstat'],
               dtype='object')
```

现在我们可以使用相同的模型矩阵构建器，对所有terms中的变量拟合模型。

```
In [27]: X = MS(terms).fit_transform(Boston)
         model = sm.OLS(y, X)
         results = model.fit()
         summarize(results)
```

```
Out[27]:                coef   std err       t   P>|t|
         intercept   41.6173     4.936   8.431   0.000
         crim        -0.1214     0.033  -3.678   0.000
         zn           0.0470     0.014   3.384   0.001
         indus        0.0135     0.062   0.217   0.829
         chas         2.8400     0.870   3.264   0.001
         nox        -18.7580     3.851  -4.870   0.000
         rm           3.6581     0.420   8.705   0.000
         age          0.0036     0.013   0.271   0.787
         dis         -1.4908     0.202  -7.394   0.000
         rad          0.2894     0.067   4.325   0.000
         tax         -0.0127     0.004  -3.337   0.001
         ptratio     -0.9375     0.132  -7.091   0.000
         lstat       -0.5520     0.051  -10.897  0.000
```

如果我们想要使用除一个变量之外的所有变量进行回归呢？例如，在上面的回归输出中，age的p值很高。因此，我们可能希望在不包括这个预测变量的情况下进行回归。以下语法将使得在使用除age (之外的所有预测变量进行回归，输出未显示)。

```
In [28]: minus_age = Boston.columns.drop(['medv', 'age'])
         Xma = MS(minus_age).fit_transform(Boston)
         model1 = sm.OLS(y, Xma)
         summarize(model1.fit())
```

3.6.4 Multivariate Goodness of Fit

我们可以通过名称访问results的各个组件（dir(results)显示了可用的组件）。因此，results.rsquared给出了R^2，而np.sqrt(results.scale)则给出了RSE。

方差膨胀因子（第3.3.3节）有时有助于评估回归模型的设计矩阵中多重共线性的影响。我们将计算多元回归拟合中的VIF，并借此机会介绍*list comprehension*的概念。

list comprehension

列表推导式

我们经常会遇到一系列对象，希望将其转换以用于其他任务。下面，我们为 X 矩阵中的每个特征计算 VIF，并生成一个数据框，其索引与 X 的列一致。列表推导式的概念通常可以使这类任务更加容易。

列表推导式是形成Python对象列表的简单而强大的方法。该语言还支持字典和*generator*推导式，尽管这些超出了我们这里的讨论范围。让我们看一个例子。我们使用函数variance_inflation_factor()计算模型矩阵X中每个变量的VIF。

variance_
inflation_
factor()

```
In[29]: vals = [VIF(X, i)
                for i in range(1, X.shape[1])]
        vif = pd.DataFrame({'vif':vals},
                           index=X.columns[1:])
        vif
```

```
Out[29]:           vif
        crim     1.767
        zn       2.298
        indus    3.987
        chas     1.071
        nox      4.369
        rm       1.913
        age      3.088
        dis      3.954
        rad      7.445
        tax      9.002
        ptratio  1.797
        lstat    2.871
```

函数VIF()接受两个参数：一个数据框或数组，以及一个变量列索引。在上面的代码中，我们对X中的所有列即时调用VIF()。我们排除了第0列（截距），因为它不是我们关注的焦点。在这个情况下，VIFs并不那么令人兴奋。

对象 {v*} 以上可以通过以下 for 循环构造：

```
In[30]: vals = []
        for i in range(1, X.values.shape[1]):
            vals.append(VIF(X.values, i))
```

列表推导式允许我们以更直接的方式执行此类重复操作。

3.6.5 Interaction Terms

使用 ModelSpec() 可以很容易地在线性模型中包含交互项。包含元组 ("lstat","age") 会告诉模型矩阵构建器在 lstat 和 age 之间包含一个交互项。

```
In[31]: X = MS(['lstat',
                'age',
                ('lstat', 'age')]).fit_transform(Boston)
        model2 = sm.OLS(y, X)
        summarize(model2.fit())
```

```
Out[31]:               coef    std err         t    P>|t|
         intercept  36.0885      1.470    24.553    0.000
             lstat  -1.3921      0.167    -8.313    0.000
```

```
          age    -0.0007      0.020    -0.036    0.971
    lstat:age     0.0042      0.002     2.244    0.025
```

3.6.6 Non-linear Transformations of the Predictors

模型矩阵构建器可以包含除列名和交互项之外的项。例如，ISLP 中提供的 poly() 函数指定了
表示其第一个参数的多项式函数的列被添加到模型矩阵中。 poly()

In [32]:
```
X = MS([poly('lstat', degree=2), 'age']).fit_transform(Boston)
model3 = sm.OLS(y, X)
results3 = model3.fit()
summarize(results3)
```

Out[32]:

	coef	std err	t	P>|t|
intercept	17.7151	0.781	22.681	0.000
poly(lstat, degree=2)[0]	-179.2279	6.733	-26.620	0.000
poly(lstat, degree=2)[1]	72.9908	5.482	13.315	0.000
age	0.0703	0.011	6.471	0.000

与二次项相关的有效零 p 值（即上述第三行）表明它导致了模型的改进。

默认情况下，poly()会创建一个基矩阵，以便包含在模型矩阵中，其列由*orthogonal polynomials*构成，这些列是为sta-而设计的。
最小二乘计算。[13] 或者，如果我们在上述调用中包含了参数 raw=True 而调用的是 poly()，基矩阵将仅由 lstat 和 lstat**2 组成。由于这些基中的任何一个都表示二次多项式，在这种情况下拟合值不会改变，只是多项式系数会变化。此外，默认情况下，由 poly() 创建的列不包含截距列，因为截距列是由 MS() 自动添加的。 orthogonal polynomial

我们使用anova_lm()函数来进一步量化
二次拟合优于线性拟合。 anova_lm()

In [33]:
```
anova_lm(results1, results3)
```

Out[33]:

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	503.0	19168.13	0.0	NaN	NaN	NaN
1	502.0	14165.61	1.0	5002.52	177.28	7.47e-35

这里 results1 表示包含预测变量 lstat 和 age 的线性子模型，而 results3 对应于上面含有 lstat 中二次项的更大模型。anova_lm() 函数执行假设检验，比较这两个模型。原假设是较大模型中的二次项不需要，备择假设是较大模型更优。这里 F 统计量为 177.28，相关的 p 值为零。在这种情况下，F 统计量是 results3 线性模型摘要中二次项的 t 统计量的平方——这是这些嵌套模型相差一个自由度的结果。

[13] Actually, poly() is a wrapper for the workhorse and standalone function Poly() that does the work in building the model matrix.

自由度。这提供了非常明确的证据，表明 lstat 中的二次多项式改进了线性模型。这并不令人意外，因为之前我们已经看到了 medv 和 lstat 之间非线性关系的证据。

函数 anova_lm() 可以接受两个以上的嵌套模型作为输入，在这种情况下，它会比较每一对连续的模型。这也解释了为什么上面的第一行中有 NaN，因为没有先前的模型可以用来与第一个进行比较。

In [34]:
```
ax = subplots(figsize=(8,8))[1]
ax.scatter(results3.fittedvalues, results3.resid)
ax.set_xlabel('Fitted value')
ax.set_ylabel('Residual')
ax.axhline(0, c='k', ls='--')
```

我们看到，当模型中包含二次项时，残差中几乎看不出任何规律。为了创建三次或更高次的多项式拟合，我们只需将degree参数改为poly()。

3.6.7 Qualitative Predictors

这里我们使用Carseats数据，该数据包含在ISLP包中。我们将尝试根据若干预测变量，预测400个地点的Sales (儿童汽车座椅销量)。

In [35]:
```
Carseats = load_data('Carseats')
Carseats.columns
```

Out[35]:
```
Index(['Sales', 'CompPrice', 'Income', 'Advertising',
       'Population', 'Price', 'ShelveLoc', 'Age', 'Education',
       'Urban', 'US'],
      dtype='object')
```

Carseats 数据包括定性预测变量，如 ShelveLoc，这是一个货架位置质量的指标——即商店中展示汽车座椅的空间。预测变量 ShelveLoc 可能有三个值，Bad、Medium 和 Good。对于像 ShelveLoc 这样的定性变量，ModelSpec() 自动生成虚拟变量。这些变量通常被称为分类变量的 *one-hot encoding*。

one-hot encoding

特征。它们的列和为 1，因此为避免与截距产生共线性，第一列被删除。下面我们看到列 ShelveLoc[Bad] 被删除，因为 Bad 是 ShelveLoc 的第一个水平。下面我们拟合一个包含一些交互项的多元回归模型。

In [36]:
```
allvars = list(Carseats.columns.drop('Sales'))
y = Carseats['Sales']
final = allvars + [('Income', 'Advertising'),
                   ('Price', 'Age')]
X = MS(final).fit_transform(Carseats)
model = sm.OLS(y, X)
summarize(model.fit())
```

Out[36]:

	coef	std err	t	P>|t|
intercept	6.5756	1.009	6.519	0.000

```
                    CompPrice      0.0929    0.004   22.567    0.000
                       Income      0.0109    0.003    4.183    0.000
                  Advertising      0.0702    0.023    3.107    0.002
                   Population      0.0002    0.000    0.433    0.665
                        Price     -0.1008    0.007  -13.549    0.000
              ShelveLoc[Good]      4.8487    0.153   31.724    0.000
            ShelveLoc[Medium]      1.9533    0.126   15.531    0.000
                          Age     -0.0579    0.016   -3.633    0.000
                    Education     -0.0209    0.020   -1.063    0.288
                   Urban[Yes]      0.1402    0.112    1.247    0.213
                     US[Yes]     -0.1576    0.149   -1.058    0.291
Income:Advertising              0.0008    0.000    2.698    0.007
                    Price:Age      0.0001    0.000    0.801    0.424
```

在上面的第一行中，我们将`allvars`设为一个列表，以便在下方两行添加交互项。我们的模型矩阵构建器创建了一个`ShelveLoc[Good]`虚拟变量，如果货架位置为良好，则取值为1，否则为0。它还创建了一个`ShelveLoc[Medium]`虚拟变量，如果货架位置为中等，则等于1，否则为0。一个差的货架位置对应于这两个虚拟变量均为零。回归输出中`ShelveLoc[Good]`的系数为正，这一事实表明良好的货架位置与高销售额相关（相对于差的位置）。而`ShelveLoc[Medium]`有一个较小的正系数，表明中等货架位置导致的销售额高于差的货架位置，但低于良好的货架位置。

3.7 练习

Conceptual

1. 描述表3.4中给出的p值所对应的零假设。根据这些p值，解释你可以得出什么结论。你的解释应该用`sales`、`TV`、`radio`和`newspaper`来表述，而不是用线性模型的系数来表述。

2. 仔细解释KNN分类器和KNN回归方法之间的差异。

3. 假设我们有一个包含五个预测变量的数据集，$X_1 = \text{GPA}$、$X_2 = \text{IQ}$、$X_3 = \text{Level}$（大学为1，高中为0）、$X_4 = \text{GPA与IQ之间的交互作用}$，以及$X_5 = \text{GPA与Level之间的交互作用}$。响应变量是毕业后的起薪（以千美元计）。假设我们使用最小二乘法来拟合模型，并得到$\hat{\beta}_0 = 50$，$\hat{\beta}_1 = 20, \hat{\beta}_2 = 0.07, \hat{\beta}_3 = 35, \hat{\beta}_4 = 0.01, \hat{\beta}_5 = -10$。

   (a) 哪个答案是正确的，为什么？

   i. 对于固定的IQ和GPA值，高中毕业生的平均收入高于大学毕业生。ii. 对于固定的IQ和GPA值，大学毕业生的平均收入高于高中毕业生。

iii. 对于固定的智商和GPA值，只要GPA足够高，高中毕业生的平均收入高于大学毕业生。 iv. 对于固定的智商和GPA值，只要GPA足够高，大学毕业生的平均收入高于高中毕业生。

(b) 预测一名IQ为110、GPA为4.0的大學畢業生的薪水。(c) 判斷正誤：由於GPA/IQ交互項的係數非常小，幾乎沒有證據表明存在交互效應。請證明你的答案。

4. 我收集了一组数据（$n = 100$个观测值），其中包含一个预测变量和一个定量响应变量。然后，我对这些数据拟合了一个线性回归模型，以及一个单独的三次回归模型，即 $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon$。

(a) 假设X与Y之间的真实关系是线性的，即$Y = \beta_0 + \beta_1 X + \epsilon$。考虑线性回归的训练残差平方和（RSS），以及三次回归的训练RSS。我们预期其中一个会比另一个低，预期它们相同，还是信息不足，无法判断？请证明你的答案。(b) 使用测试RSS而非训练RSS回答(a)。(c) 假设X与Y之间的真实关系不是线性的，但我们不知道它偏离线性有多远。考虑线性回归的训练RSS，以及三次回归的训练RSS。我们预期其中一个会比另一个低，预期它们相同，还是信息不足，无法判断？请证明你的答案。

(d) 使用测试集而非训练集RSS来回答(c)部分。

5. 考虑在没有截距的情况下执行线性回归所得到的拟合值。在此设定中，第i个拟合值的形式为

$$\hat{y}_i = x_i \hat{\beta},$$

其中

$$\hat{\beta} = \left(\sum_{i=1}^{n} x_i y_i \right) / \left(\sum_{i'=1}^{n} x_{i'}^2 \right). \tag{3.38}$$

证明我们可以写成

$$\hat{y}_i = \sum_{i'=1}^{n} a_{i'} y_{i'}.$$

$a_{i'}$ 是什么?

Note: We interpret this result by saying that the fitted values from linear regression are 线性组合 *of the response values.*

6. 利用(3.4)，论证在简单线性回归的情况下，最小二乘线总是通过点($\bar{x}, \bar{y}$)。

7. 文中声称，在将Y对X进行简单线性回归的情况下，R^2统计量(3.17)等于X与Y (3.18)之间相关系数的平方。证明这一点。为简单起见，您可以假设$\bar{x} = \bar{y} = 0$。



Applied

8. 这个问题涉及在`Auto`数据集上使用简单线性回归。

(a) 使用 `sm.OLS()` 函数进行简单线性回归，以 `mpg` 作为响应变量，`horsepower` 作为预测变量。使用 `summarize()` 函数打印结果。对输出进行评论。例如：i. 预测变量与响应变量之间是否存在关系？ii. 预测变量与响应变量之间的关系有多强？iii. 预测变量与响应变量之间的关系是正还是负？iv. 与 `horsepower` 为 98 相关联的预测 `mpg` 是多少？相关的95%置信区间和预测区间是多少？

(b) 在新的坐标轴集 `ax` 中绘制响应变量和预测变量。使用实验室中定义的 `ax.axline()` 方法或 `abline()` 函数来显示最小二乘回归线。(c) 根据实验室描述，生成一些最小二乘回归拟合的诊断图。对拟合中发现的任何问题发表评论。

9. 本问题涉及对`Auto`数据集使用多元线性回归。

(a) 生成一个包含数据集中所有变量的散点图矩阵。(b) 使用 `DataFrame.corr()`方法计算变量之间的相关矩阵。

`.corr()`

(c) 使用 `sm.OLS()` 函数进行多元线性回归，以 `mpg` 为响应变量，并以除 `name` 之外的所有其他变量为预测变量。使用 `summarize()` 函数打印结果。对输出进行评论。例如：i. 预测变量与响应变量之间是否存在关系？使用 `statsmodels` 中的 `anova_lm()` 函数来回答这个问题。

ii. 哪些预测变量与响应变量之间存在统计学上显著的关系？iii. `year` 变量的系数暗示了什么？

(d) 根据实验描述，生成线性回归拟合的一些诊断图。对拟合中出现的任何问题进行评论。残差图是否暗示存在异常大的离群点？杠杆图是否识别出具有异常高杠杆的观测？(e) 按照实验描述拟合一些包含交互项的模型。是否有任何交互项在统计上显著？(f) 尝试对变量进行几种不同的变换，例如log(X)、$\sqrt{X}$、X^2。对你的发现进行评论。

—

10. 这个问题应使用 `Carseats` 数据集来回答。

(a) 拟合一个多元回归模型，使用`Price`、`Urban`和`US`来预测`Sales`。 (b) 对模型中每个系数进行解释。注意——模型中的一些变量是定性的！ (c) 以方程形式写出模型，小心处理定性变量。 (d) 对于哪些预测变量，你可以拒绝零假设H_0：$\beta_j = 0$？ (e) 基于你对前一个问题的回答，拟合一个仅使用与结果有 association 证据的预测变量的较小模型。 (f) (a)和(e)中的模型对数据的拟合程度如何？ (g) 使用(e)中的模型，获得系数的95%置信区间。 (h) 在(e)的模型中，是否有异常值或高杠杆观测的证据?

11. 在这个问题中，我们将研究简单线性回归无截距情况下，针对零假设$H_0 : \beta = 0$ 的 t 统计量。首先，我们按以下方式生成预测变量 `x` 和响应变量 `y`。

```
rng = np.random.default_rng(1)
x = rng.normal(size=100)
y = 2 * x + rng.normal(size=100)
```

(对 `y` 关于 `x`、*without*（截距）进行简单线性回归。报告系数估计值 $\hat{\beta}$、该系数估计的标准误，以及与零假设 H_0 相关的 t 统计量和 p 值：$\beta = 0$。对这些结果进行评论。（你可以使用关键字参数 `intercept=False` 到 `ModelSpec()` 执行无截距回归。）

()现在对x关于y进行无截距的简单线性回归，并报告系数估计、其标准误，以及与零假设H_0：$\beta = 0$对应的t统计量和p值。对这些结果进行评论。

(c) (a) 和 (b) 中得到的结果之间有什么关系?

(d) 对于无截距的Y对X的回归，H_0: $\beta = 0$的t-统计量形式为$\hat{\beta}/\text{SE}(\hat{\beta})$，其中$\hat{\beta}$由(3.38)给出，且其中



$$\text{SE}(\hat{\beta}) = \sqrt{\frac{\sum_{i=1}^{n}(y_i - x_i\hat{\beta})^2}{(n-1)\sum_{i'=1}^{n} x_{i'}^2}}.$$

(这些公式与第3.1.1节和第3.1.2节中给出的略有不同，因为此处我们执行的是无截距回归。) 通过代数方法展示，并在R中数值验证，t统计量可以写作

$$\frac{(\sqrt{n-1})\sum_{i=1}^{n} x_i y_i}{\sqrt{(\sum_{i=1}^{n} x_i^2)(\sum_{i'=1}^{n} y_{i'}^2) - (\sum_{i'=1}^{n} x_{i'} y_{i'})^2}}.$$

((e)) 利用 (d) 的结果，论证将 y 对 x 进行回归所得的 t 统计量与将 x 对 y 进行回归所得的 t 统计量相同。(f) 在 R 中，证明当回归 $with$ 一个截距时，$H_0 : \beta_1 = 0$ 的 t 统计量，对于将 y 对 x 的回归与将 x 对 y 的回归是相同的。

12. 本问题涉及无截距的简单线性回归。

（a）回忆一下，Y对X进行无截距线性回归的系数估计值$\hat{\beta}$由（3.38）给出。在什么情况下，X对Y进行回归的系数估计值与Y对X进行回归的系数估计值相同?（b）在Python中生成一个例子，该例子包含$n = 100$个观测值，其中X对Y进行回归的系数估计值是$different\ from$$Y$对$X$进行回归的系数估计值。（c）在Python中生成一个例子，该例子包含$n = 100$个观测值，其中X对Y进行回归的系数估计值是$the\ same\ as$$Y$对$X$进行回归的系数估计值。

13. 在本练习中，您将创建一些模拟数据，并对其拟合简单的线性回归模型。为确保结果一致，请在开始部分(a)之前，确保使用默认随机数生成器并将种子设置为1。

(a) 使用随机数生成器的`normal()`方法，创建一个向量`x`，包含从$N(0, 1)$分布中抽取的100个观测值。这代表一个特征X。

(b) 使用 `normal()` 方法，创建一个向量 `eps`，包含从 $N(0, 0.25)$ 分布（均值为零、方差为0.25的正态分布）中抽取的100个观测值。

(c) 使用 `x` 和 `eps`，根据模型生成向量 `y`

$$Y = -1 + 0.5X + \epsilon. \tag{3.39}$$

向量 `y` 的长度是多少？在这个线性模型中，β_0 和 β_1 的值分别是多少？

(d) 创建一个散点图，展示 `x` 与 `y` 之间的关系。就你的观察发表评论。

(e) 拟合一个最小二乘线性模型，使用 `x` 来预测 `y`。评论所得到的模型。$\hat{\beta}_0$ 和 $\hat{\beta}_1$ 与 β_0 和 β_1 相比如何？(f) 在(d)中得到的散点图上显示最小二乘线。在图上以不同的颜色绘制总体回归线。使用坐标轴的 `legend()` 方法创建适当的图例。(g) 现在拟合一个多项式回归模型，使用 `x` 和 x^2 来预测 `y`。是否有证据表明二次项改善了模型拟合？解释你的答案。(h) 在修改数据生成过程以使数据中存在 *less* 噪声后，重复(a)–(f)。模型(3.39)应保持不变。你可以通过减小(b)中用于生成误差项 ϵ 的正态分布的方差来做到这一点。描述你的结果。(i) 在修改数据生成过程以使数据中存在 *more* 噪声后，重复(a)–(f)。模型(3.39)应保持不变。你可以通过增大(b)中用于生成误差项 ϵ 的正态分布的方差来做到这一点。描述你的结果。(j) 基于原始数据集、噪声较大的数据集和噪声较小的数据集，β_0 和 β_1 的置信区间是什么？评论你的结果。

14. 本题聚焦于 *collinearity* 问题。

(a) 在`Python`中执行以下命令：

```
rng = np.random.default_rng(10)
x1 = rng.uniform(0, 1, size=100)
x2 = 0.5 * x1 + rng.normal(size=100) / 10
y = 2 + 2 * x1 + 0.3 * x2 + rng.normal(size=100)
```

最后一行对应于创建一个线性模型，其中 `y` 是 `x1` 和 `x2` 的函数。写出该线性模型的形式。回归系数是什么？

(b) x1和x2之间的相关性是什么？创建一个展示变量之间关系的散点图。(c) 使用这些数据，拟合一个最小二乘回归，用x1和x2来预测y。描述所得到的结果。$\hat{\beta}_0$、$\hat{\beta}_1$和$\hat{\beta}_2$是什么？这些与真实的β_0、β_1和β_2有何关系？你能拒绝原假设H_0：$\beta_1 = 0$吗？那么原假设H_0：$\beta_2 = 0$呢？(d) 现在仅使用x1拟合一个最小二乘回归来预测y。评论你的结果。你能拒绝原假设H_0：$\beta_1 = 0$吗？(e) 现在仅使用x2拟合一个最小二乘回归来预测y。评论你的结果。你能拒绝原假设H_0：$\beta_1 = 0$吗？(f) 在(c)–(e)中得到的结果是否相互矛盾？解释你的答案。(g) 假设我们获得了一个额外的观测值，不幸的是它被错误测量了。我们使用函数np.concatenate()来

np.concatenate()

将这一附加观察结果添加到 x1、x2 和 y 中的每一个。

```
x1 = np.concatenate([x1, [0.1]])
x2 = np.concatenate([x2, [0.8]])
y = np.concatenate([y, [6]])
```

使用这些新数据重新拟合 (c) 到 (e) 中的线性模型。这个新观测值对每个模型有什么影响？在每个模型中，这个观测值是否为离群点？是否为高杠杆点？两者都是？解释你的答案。

15. 本问题涉及Boston数据集，我们在本章的实验中已经见过。现在我们将尝试使用该数据集中的其他变量来预测人均犯罪率。换句话说，人均犯罪率是响应变量，而其他变量是预测变量。

(a) 对于每个预测变量，拟合一个简单的线性回归模型来预测响应变量。描述你的结果。在哪些模型中，预测变量与响应变量之间存在统计学上显著的关联？创建一些图表来支持你的论断。(b) 使用所有预测变量拟合一个多元回归模型来预测响应变量。描述你的结果。对于哪些预测变量，我们可以拒绝原假设 $H_0 : \beta_j = 0$？(c) 你将(a)的结果与(b)的结果如何比较？创建一个图表，在x轴上显示来自(a)的单变量回归系数，在y轴上显示来自(b)的多元回归系数。也就是说，每个预测变量在图中显示为一个点。其在简单线性回归模型中的系数显示在x轴上，其在多元线性回归模型中的系数估计显示在y轴上。

(d) 是否有证据表明任何预测变量与响应变量之间存在非线性关联? 为回答此问题，对每个预测变量X，拟合如下形式的模型

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon.$$

4
分类

第3章讨论的线性回归模型假设响应变量Y是定量的。但在许多情况下，响应变量反而是*qualitative*。例如，眼睛颜色是定性的。

定性

十个定性变量称为*categorical*；我们将交替使用这些术语。在本章中，我们研究预测定性响应的方法，这一过程被称为*classification*。预测

分类

对观察结果的定性响应可以称为*classifying*该观察结果，因为它涉及将观察结果分配到某个类别或类中。另一方面，用于分类的方法通常首先预测观察结果属于定性变量各个类别的概率，作为进行分类的基础。从这个意义上说，它们的行为也类似于回归方法。

分类器

有许多可能的分类技术，或者 *classifiers*，即一种可能用于预测定性响应。我们在第2.1.5节和第2.2.3节中已经触及了一些方法。在本章中，我们将讨论一些广泛使用的分类器：*logistic regression*、*linear discriminant analysis*、*quadratic discriminant analysisnaive Bayes* 和 K*-nearest neighbors*。讨论逻辑回归被用作讨论*gen- eralized linear models*的起点，特别是*Poisson regression*。我们讨论后续章节会介绍更多计算密集型的分类方法：包括广义加性模型（第7章）；树、随机森林和提升方法（第8章）；以及支持向量机（第9章）。

逻辑回归 线性判别分析

二次判别分析 朴素贝叶斯 K-最近邻

广义线性模型 泊松回归

4.1 分类概述

分类问题经常出现，甚至可能比回归问题更为常见。一些例子包括：

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_4

1. 一个人因一系列症状到达急诊室，这些症状可能归因于三种医疗状况之一。这个人患的是这三种状况中的哪一种？2. 一个在线银行服务必须能够根据用户的IP地址、过去的交易历史等，判断在网站上执行的交易是否具有欺诈性。3. 根据一些患有和未患有某种疾病的患者的DNA序列数据，生物学家想要弄清楚哪些DNA突变是有害的（致病）和哪些不是。

正如在回归设置中一样，在分类设置中，我们有一组训练观测值 (x_1, y_1) $, \ldots, (x_n, y_n)$，我们可以用它们来构建分类器。我们希望我们的分类器不仅在训练数据上表现良好，而且在未用于训练分类器的测试观测值上也表现良好。

在本章中，我们将使用模拟的 `Default` 数据集来说明分类的概念。我们关注的是根据年收入和每月信用卡余额来预测个人是否会对其信用卡还款违约。该数据集展示在图4.1中。在图4.1的左侧面板中，我们绘制了10,000个人子集的年`income`和每月信用卡`balance`。在给定月份违约的个人以橙色显示，未违约的以蓝色显示。（总体违约率约为3%，因此我们只绘制了一部分未违约的个人。）似乎违约的个人往往比未违约的个人有更高的信用卡余额。在图4.1的中间和右侧面板中，展示了两对箱线图。第一个展示了按二元`default`变量划分的`balance`分布；第二个是`income`的类似图。在本章中，我们学习如何构建一个模型，对任意给定的`balance` (X_1)和`income` (X_2)值预测`default` (Y)。由于Y不是定量的，第3章的简单线性回归模型不是一个好的选择：我们将在4.2节进一步阐述这一点。

值得注意的是，图4.1显示了预测变量`balance`和响应变量`default`之间非常显著的关系。在大多数实际应用中，预测变量和响应变量之间的关系不会如此强烈。然而，为了说明本章讨论的分类程序，我们使用了一个预测变量和响应变量之间关系有些夸大的例子。

4.2 为什么不是线性回归?

我们已经说过，在定性响应的情况下，线性回归是不合适的。为什么不适合呢?

假设我们试图根据急诊室患者的症状来预测其医疗状况。在这个简化示例中，有三种可能的诊断：`stroke`、`drug overdose`和

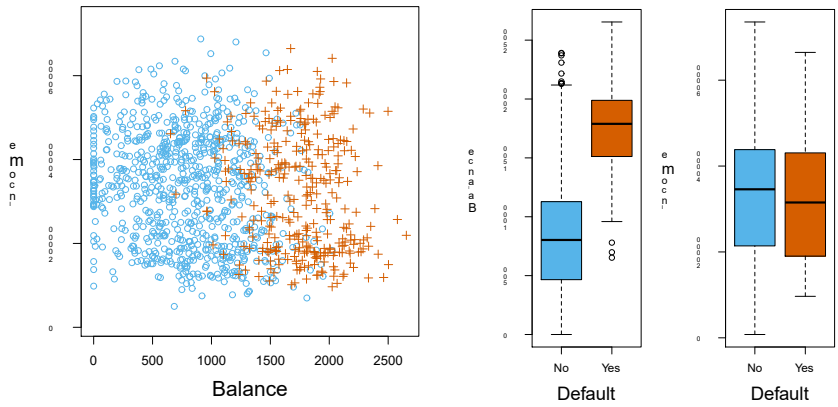


图 4.1. *The* `Default` *data set.* 左：*The annual incomes and monthly credit card balances of a number of individuals. The individuals who defaulted on their credit card payments are shown in orange, and those who did not are shown in blue.* 中：*Boxplots of* `balance` *as a function of* `default` *status.* 右：*Boxplots of* `income` *as a function of* `default` *status.*

`epileptic seizure`. 我们可以考虑将这些值编码为一个定量响应变量 Y，如下所示：

$$Y = \begin{cases} 1 & \text{if } \texttt{stroke}; \\ 2 & \text{if } \texttt{drug overdose}; \\ 3 & \text{if } \texttt{epileptic seizure}. \end{cases}$$

使用这种编码，最小二乘法可用于拟合一个线性回归模型，基于一组预测变量$X_1, \ldots, X_p$来预测Y。不幸的是，这种编码隐含了对结果的排序，将`drug overdose`置于`stroke`和`epileptic seizure`之间，并坚持认为`stroke`和`drug overdose`之间的差异与`drug overdose`和`epileptic seizure`之间的差异相同。实际上，没有特别的理由必须如此。例如，可以选择一种同样合理的编码，

$$Y = \begin{cases} 1 & \text{if } \texttt{epileptic seizure}; \\ 2 & \text{if } \texttt{stroke}; \\ 3 & \text{if } \texttt{drug overdose}, \end{cases}$$

这暗示了三个条件之间完全不同的关系。每一种编码都会产生根本不同的线性模型，最终导致对测试观测的不同预测集。

如果响应变量的值确实具有自然顺序，例如 *mild*、*moderate* 和 *severe*，并且我们认为轻度和中度之间的差距与中度和重度之间的差距相似，那么 1、2、3 编码将是合理的。不幸的是，一般来说，并没有自然的方法来

将一个具有两个以上水平的定性响应变量转换为一个适合线性回归的定量响应。

对于 *binary* (两水平)定性响应，情况要好一些。对于例如，也许病人的医疗状况只有两种可能性：`stroke` 和 `drug overdose`。然后，我们可能可以使用第3.3.1节中的 *dummy variable* 方法将响应编码如下：

binary

$$Y = \begin{cases} 0 & \text{if } \texttt{stroke}; \\ 1 & \text{if } \texttt{drug overdose}. \end{cases}$$

然后我们可以对这个二元响应拟合线性回归，如果$\hat{Y} > 0.5$，则预测`drug overdose`，否则预测`stroke`。在二元情况下，不难证明即使我们翻转上述编码，线性回归也会产生相同的最终预测。

对于如上所述采用0/1编码的二元响应变量，使用最小二乘法进行回归并非完全不合理：可以证明，在这种特殊情况下，通过线性回归得到的$X\hat{\beta}$实际上是对Pr(`drug overdose`$|X$)的估计。然而，如果使用线性回归，我们的一些估计值可能会落在[0, 1]区间之外（见图4.2），这使得它们很难被解释为概率！尽管如此，这些预测提供了一种排序，并且可以被解释为粗略的概率估计。有趣的是，事实证明，如果我们使用线性回归来预测二元响应变量，所得到的分类结果将与我们在4.4节中讨论的线性判别分析（LDA）方法相同。

总之，至少有两个原因不使用回归方法进行分类：（a）回归方法无法容纳具有两个以上类别的定性响应；（b）即使只有两个类别，回归方法也无法提供Pr($Y|X$)的有意义估计。因此，最好使用真正适用于定性响应值的分类方法。在下一节中，我们将介绍逻辑回归，它非常适合二元定性响应的情况；在后续章节中，我们将涵盖当定性响应有两个或更多类别时适用的分类方法。

4.3 逻辑回归

再次考虑`Default`数据集，其中响应`default`属于两类之一，`Yes`或`No`。逻辑回归不是直接对响应Y建模，而是对Y属于某一特定类别的*probability*进行建模。

对于`Default`数据，逻辑回归模型预测违约概率。例如，给定`balance`的违约概率可以写为

$$\Pr(\texttt{default} = \texttt{Yes}|\texttt{balance}).$$

Pr(`default` $=$ `Yes`|`balance`) 的值，我们简记为 p(`balance`)，将介于 0 和 1 之间。然后，对于任意给定的 `balance` 值，可以对 `default` 进行预测。例如，可以预测 `default` $=$ `Yes`

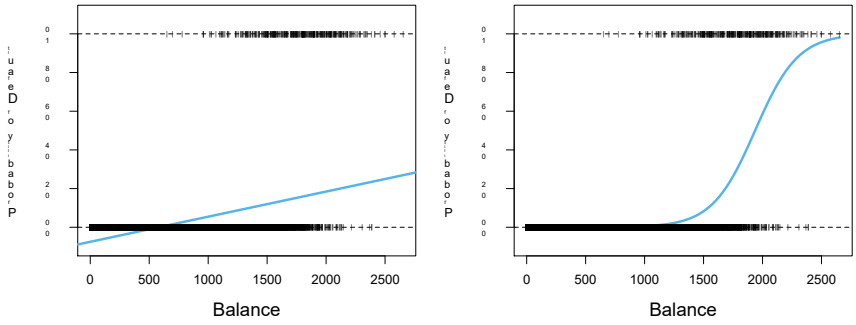


图 4.2. *Classification using the* `Default` *data.* 左： *Estimated probability of* `default` *using linear regression. Some estimated probabilities are negative! The orange ticks indicate the 0/1 values coded for* `default` *(*`No` *or* `Yes`*).* 右： *Predicted probabilities of* `default` *using logistic regression. All probabilities lie between* 0 *and* 1.

对于任何满足 $p(\texttt{balance}) > 0.5$ 的个体。或者，如果公司希望在预测有违约风险的个体时采取保守态度，那么他们可以选择使用较低的阈值，例如 $p(\texttt{balance}) > 0.1$。

4.3.1 The Logistic Model

我们应该如何建模$p(X) = \Pr(Y = 1|X)$与X之间的关系？（为方便起见，我们对响应使用通用的0/1编码。）在第4.2节中，我们考虑了使用线性回归模型来表示这些概率：

$$p(X) = \beta_0 + \beta_1 X. \tag{4.1}$$

如果我们使用这种方法来利用 `balance` 预测 `default`=`Yes`，那么我们就得到了图4.2左侧面板所示的模型。在这里，我们看到了这种方法的问题所在：当余额接近零时，我们预测出负的违约概率；如果我们对非常大的余额进行预测，我们会得到大于1的值。这些预测是不合理的，因为无论信用卡余额如何，真实的违约概率当然必须介于0和1之间。这个问题并不是信用违约数据所独有的。每当用一条直线去拟合编码为0或1的二元响应时，原则上我们总能对某些 X 值预测 $p(X) <$ 为0，而对另一些值预测 $p(X) >$ 为1(除非 X 的范围有限)。

为了避免这个问题，我们必须使用一个对于 X 的所有值都能给出0到1之间输出的函数来建模 $p(X)$。有许多函数符合这一描述。在逻辑回归中，我们使用的是 *logistic function*,

逻辑函数

$$p(X) = \frac{e^{\beta_0+\beta_1 X}}{1 + e^{\beta_0+\beta_1 X}}. \tag{4.2}$$

为了拟合模型（4.2），我们使用一种称为*maximum likelihood*的方法，该方法我们在下一节讨论。图4.2的右侧面板展示了逻辑回归模型对`Default`数据的拟合情况。请注意，对于

maximum likelihood

低余额时，我们现在预测违约概率接近于零，但绝不小于零。同样，对于高余额，我们预测违约概率接近于1，但绝不超过1。逻辑函数总会产生这种形式的*S-shaped*曲线，因此无论X的值如何，我们都能得到合理的预测。我们还看到，与左侧图中的线性回归模型相比，逻辑模型更能捕捉概率的范围。两种情况下的平均拟合概率均为0.0333（在训练数据上平均），这与数据集中违约者的总体比例相同。

经过对(4.2)式的一番推导，我们发现

$$\frac{p(X)}{1-p(X)} = e^{\beta_0+\beta_1 X}. \tag{4.3}$$

量$p(X)/[1-p(X)]$被称为*odds*，并且可以取任意奇数值在0和∞之间。赔率值接近0和∞分别表示违约概率极低和极高。例如，平均而言，每5个赔率为1/4的人中就有1人会违约，因为$p(X) = 0.2$意味着赔率为$\frac{0.2}{1-0.2} = 1/4$。同样地，平均而言，每10个赔率为9的人中就有9人会违约，因为$p(X) = 0.9$意味着赔率为$\frac{0.9}{1-0.9} = 9$。在赛马中，传统上使用赔率而不是概率，因为它们与正确的投注策略更自然地相关。

s

对(4.3)式两边取对数，我们得到

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X. \tag{4.4}$$

左侧称为 *log odds* 或 *logit*。我们看到，逻辑回归回归模型（4.2）具有一个关于X线性的logit。

g 几率 对数几率

回顾第3章，在线性回归模型中，β_1给出了与X增加一个单位相关的Y的平均变化。相比之下，在逻辑回归模型中，X增加一个单位会使对数几率变化β_1 (4.4)。等价地，它将几率乘以e^{β_1} (4.3)。然而，由于（4.2）中$p(X)$与X之间的关系不是直线，β_1并不*not*对应于与X增加一个单位相关的$p(X)$的变化。$p(X)$因X变化一个单位而变化的量取决于X的当前值。但无论X的值如何，如果β_1为正，那么增加X将与增加$p(X)$相关联，如果β_1为负，那么增加X将与减少$p(X)$相关联。$p(X)$与X之间不存在直线关系，以及$p(X)$每单位X变化的变化率取决于X的当前值，这一事实也可以通过观察图4.2右侧面板看出。

4.3.2 Estimating the Regression Coefficients

(4.2) 中的系数 β_0 和 β_1 是未知的，必须基于可用的训练数据进行估计。在第3章中，我们使用最小二乘法来估计未知的线性回归系数。尽管我们可以使用（非线性）最小二乘法来拟合模型 (4.4)，但更通用的*maximum likelihood* 方法更受青睐，因为它具有更好的统计性质。使用最大似然背后的基本直觉

拟合逻辑回归模型的方法如下：我们寻求 β_0 和 β_1 的估计值，使得使用 (4.2) 式得到的每个个体的预测违约概率 $\hat{p}(x_i)$ 尽可能接近该个体观测到的违约状态。换句话说，我们试图找到 $\hat{\beta}_0$ 和 $\hat{\beta}_1$，使得将这些估计值代入 (4.2) 中给出的 $p(X)$ 模型后，对于所有违约的个体产生接近一的数值，而对于所有未违约的个体产生接近零的数值。这种直觉可以使用一个称为 *likelihood function* 的数学方程来形式化：

likelihood function

$$\ell(\beta_0, \beta_1) = \prod_{i:y_i=1} p(x_i) \prod_{i':y_{i'}=0} (1 - p(x_{i'})). \tag{4.5}$$

估计值 $\hat{\beta}_0$ 和 $\hat{\beta}_1$ 被选择以 *maximize* 这个似然函数。

最大似然是一种非常通用的方法，用于拟合本书中研究的许多非线性模型。在线性回归设置中，最小二乘方法实际上是最大似然的一个特例。最大似然的数学细节超出了本书的范围。然而，通常逻辑回归和其他模型可以轻松地使用诸如R之类的统计软件进行拟合，因此我们无需关心最大似然拟合过程的细节。

表 4.1 展示了在 `Default` 数据上拟合逻辑回归模型以使用 `balance` 预测 `default=Yes` 的概率所得到的系数估计及相关信息。我们看到 $\hat{\beta}_1 = 0.0055$；这表明 `balance` 的增加与 `default` 概率的增加相关。准确地说，`balance` 每增加一个单位，`default` 的对数几率增加 0.0055 个单位。

表4.1中展示的逻辑回归输出的许多方面与第3章的线性回归输出相似。例如，我们可以通过计算系数估计的标准误来衡量其准确性。表4.1中的z统计量扮演的角色与线性回归输出中的t统计量相同，例如在第77页的表3.1中。比如，与β_1相关的z统计量等于$\hat{\beta}_1/\text{SE}(\hat{\beta}_1)$，因此$z$统计量的较大（绝对值）值表明有证据反对零假设$H_0$：$\beta_1 = 0$。这个零假设意味着$p(X) = \frac{e^{\beta_0}}{1+e^{\beta_0}}$：换句话说，`default`的概率不依赖于`balance`。由于表4.1中与`balance`相关的p值极小，我们可以拒绝H_0。换句话说，我们得出结论，`balance`与`default`的概率之间确实存在关联。表4.1中的估计截距通常不是关注的重点；其主要作用是将平均拟合概率调整到数据中1的比例（在这种情况下，即总体违约率）。

4.3.3 Making Predictions

一旦系数被估计出来，我们就可以计算任何给定信用卡余额的`default`概率。例如，使用表4.1中给出的系数估计值，我们预测违约概率

	Coefficient	Std. error	z-statistic	p-value
`Intercept`	−10.6513	0.3612	−29.5	<0.0001
`balance`	0.0055	0.0002	24.9	<0.0001

表4.1. *For the* `Default` *data, estimated coefficients of the logistic regression model that predicts the probability of* `default` *using* `balance`*. A one-unit increase in* `balance` *is associated with an increase in the log odds of* `default` *by* 0.0055 *units.*

	Coefficient	Std. error	z-statistic	p-value
`Intercept`	−3.5041	0.0707	−49.55	<0.0001
`student[Yes]`	0.4049	0.1150	3.52	0.0004

表 4.2. *For the* `Default` *data, estimated coefficients of the logistic regression model that predicts the probability of* `default` *using student status. Student status is encoded as a dummy variable, with a value of* 1 *for a student and a value of* 0 *for a non-student, and represented by the variable* `student[Yes]` *in the table.*

对于`balance`为$1,000的个体来说，是

$$\hat{p}(X) = \frac{e^{\hat{\beta}_0+\hat{\beta}_1 X}}{1+e^{\hat{\beta}_0+\hat{\beta}_1 X}} = \frac{e^{-10.6513+0.0055\times 1{,}000}}{1+e^{-10.6513+0.0055\times 1{,}000}} = 0.00576,$$

这低于1%。相比之下，余额为2,000的个人的预测违约概率要高得多，等于0.586或58.6%。

可以使用第3.3.1节中的虚拟变量方法，在逻辑回归模型中使用定性预测变量。例如，数据集`Default`包含定性变量`student`。为了拟合一个以学生身份作为预测变量的模型，我们只需创建一个虚拟变量，对于学生取值为1，对于非学生取值为0。根据学生身份预测违约概率所得到的逻辑回归模型可见表4.2。与虚拟变量相关的系数为正，且相关的p值在统计上显著。这表明学生往往比非学生有更高的违约概率：

$$\widehat{\Pr}(\texttt{default=Yes}|\texttt{student=Yes}) = \frac{e^{-3.5041+0.4049\times 1}}{1+e^{-3.5041+0.4049\times 1}} = 0.0431,$$
$$\widehat{\Pr}(\texttt{default=Yes}|\texttt{student=No}) = \frac{e^{-3.5041+0.4049\times 0}}{1+e^{-3.5041+0.4049\times 0}} = 0.0292.$$

4.3.4 Multiple Logistic Regression

我们现在考虑使用多个预测变量来预测二元响应的问题。通过与第3章中从简单线性回归到多元线性回归的扩展类比，我们可以将(4.4)推广如下：

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p, \tag{4.6}$$

其中 $X = (X_1, \ldots, X_p)$ 是 p 个预测变量。公式4.6可以被重写 itten as

$$p(X) = \frac{e^{\beta_0+\beta_1 X_1+\cdots+\beta_p X_p}}{1+e^{\beta_0+\beta_1 X_1+\cdots+\beta_p X_p}}. \tag{4.7}$$

	Coefficient	Std. error	z-statistic	p-value
`Intercept`	−10.8690	0.4923	−22.08	<0.0001
`balance`	0.0057	0.0002	24.74	<0.0001
`income`	0.0030	0.0082	0.37	0.7115
`student[Yes]`	−0.6468	0.2362	−2.74	0.0062

表 4.3. *For the* `Default` *data, estimated coefficients of the logistic regression model that predicts the probability of* `default` *using* `balance`*,* `income`*, and student status. Student status is encoded as a dummy variable* `student[Yes]`*, with a value of* 1 *for a student and a value of* 0 *for a non-student. In fitting this model,* `income` *was measured in thousands of dollars.*

正如第4.3.2节所述，我们使用最大似然法来估计$\beta_0, \beta_1, \ldots, \beta_p$。

表4.3展示了使用`balance`、`income`（（以千美元计））和`student`状态来预测`default`概率的逻辑回归模型的系数估计。这里有一个令人惊讶的结果。与`balance`和`student`状态的虚拟变量相关的p值非常小，表明这些变量中的每一个都与`default`的概率相关。然而，虚拟变量的系数是负的，表明学生比非学生更不可能违约。相比之下，表4.2中虚拟变量的系数是正的。学生身份在表4.2中与违约概率的*increase*相关，而在表4.3中与违约概率的*decrease*相关，这怎么可能呢？图4.3的左侧面板提供了这一明显悖论的图形说明。橙色和蓝色实线分别显示了学生和非学生的平均违约率，作为信用卡余额的函数。多元逻辑回归中`student`的负系数表明*for a fixed value of* `balance`*and* `income`，学生比非学生更不可能违约。确实，我们从图4.3的左侧面板观察到，对于每个`balance`值，学生违约率等于或低于非学生违约率。但图表底部附近的水平虚线显示了在所有`balance`和`income`值上平均的学生和非学生违约率，却暗示了相反的效应：整体学生违约率高于非学生违约率。因此，在表4.2所示的单变量逻辑回归输出中，`student`的系数为正。

图4.3右侧面板为这一差异提供了解释。变量`student`与`balance`之间存在相关性。学生往往持有较高水平的债务，而这又与较高的违约概率相关联。换言之，学生更可能拥有较高的信用卡余额，而正如我们从图4.3左侧面板所知，高信用卡余额往往与高违约率相关。因此，即便某个具有给定信用卡余额的学生个体，其违约概率往往低于具有相同信用卡余额的非学生，但学生整体上倾向于拥有更高的信用卡余额这一事实意味着，总体而言，学生的违约率往往高于非学生。对于试图确定应向谁提供信用的信用卡公司而言，这是一个重要的区别。如果没有任何关于学生信用卡的信息，学生比非学生风险更高。

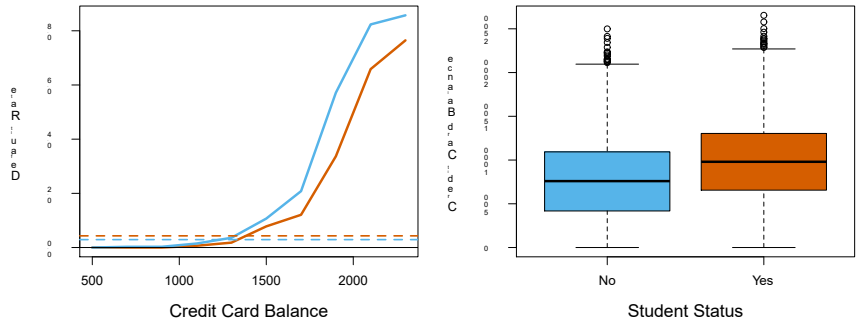


图4.3。*Confounding in the* `Default` *data.* 左：*Default rates are shown for students (orange) and non-students (blue). The solid lines display default rate as a function of* `balance`*, while the horizontal broken lines display the overall default rates.* 右：*Boxplots of* `balance` *for students (orange) and non-students (blue) are shown.*

余额可用。然而，该学生比非学生*with the same credit card balance*的风险要低！

这个简单的例子说明了当其他预测变量也可能相关时，仅涉及单个预测变量进行回归所带来的危险和微妙之处。正如在线性回归设置中一样，使用一个预测变量得到的结果可能与使用多个预测变量得到的结果有很大不同，尤其是当预测变量之间存在相关性时。一般来说，图4.3中看到的现象被称为*confounding*。

confounding

通过将表4.3中的回归系数估计值代入(4.7)式，我们可以进行预测。例如，一个信用卡余额为$1,500、收入为$40,000的学生，其估计的违约概率为

$$\hat{p}(X) = \frac{e^{-10.869+0.00574\times 1{,}500+0.003\times 40-0.6468\times 1}}{1+e^{-10.869+0.00574\times 1{,}500+0.003\times 40-0.6468\times 1}} = 0.058. \qquad (4.8)$$

一个拥有相同余额和收入的非学生预估违约概率为

$$\hat{p}(X) = \frac{e^{-10.869+0.00574\times 1{,}500+0.003\times 40-0.6468\times 0}}{1+e^{-10.869+0.00574\times 1{,}500+0.003\times 40-0.6468\times 0}} = 0.105. \qquad (4.9)$$

(这里我们将表4.3中的`income`系数估计乘以40，而不是40,000，因为在该表中，模型拟合时`income`是以$1,000)为单位测量的。

4.3.5 Multinomial Logistic Regression

有时我们希望对一个具有两个以上类别的响应变量进行分类。例如，在4.2节中，我们在急诊室有三大类医疗状况：`stroke`、`drug overdose`、`epileptic seizure`。然而，我们在本节中看到的逻辑回归方法仅允许响应变量有$K = 2$个类别。

事实证明，可以将二分类逻辑回归方法扩展到K >个类别的情形。这种扩展有时被称为*multinomial logistic regression*。为此，我们首先选择一个类作为 {v*}；不失一般性，我们选择第 K 个类来承担这一角色。然后我们将模型 (4.7) 替换为模型

multinomial logistic regression

$$\Pr(Y = k|X = x) = \frac{e^{\beta_{k0}+\beta_{k1}x_1+\cdots+\beta_{kp}x_p}}{1+\sum_{l=1}^{K-1} e^{\beta_{l0}+\beta_{l1}x_1+\cdots+\beta_{lp}x_p}} \tag{4.10}$$

对于 $k = 1, \ldots, K-1$，以及

$$\Pr(Y = K|X = x) = \frac{1}{1+\sum_{l=1}^{K-1} e^{\beta_{l0}+\beta_{l1}x_1+\cdots+\beta_{lp}x_p}}. \tag{4.11}$$

不难证明，对于 $k = 1, \ldots, K-1$,

$$\log\left(\frac{\Pr(Y = k|X = x)}{\Pr(Y = K|X = x)}\right) = \beta_{k0} + \beta_{k1}x_1 + \cdots + \beta_{kp}x_p. \tag{4.12}$$

注意，(4.12) 与 (4.6) 非常相似。公式 4.12 表明，任意一对类别之间的对数几率再次在特征中呈线性关系。

事实证明，在(4.10)–(4.12)中，将K类别视为基线的决定并不重要。例如，当将急诊室就诊分类为`stroke`、`drug overdose`和`epileptic seizure`时，假设我们拟合了两个多项逻辑回归模型：一个以`stroke`为基线，另一个以`drug overdose`为基线。由于基线的选择不同，两个拟合模型之间的系数估计会有所差异，但拟合值（预测）、任意两类之间的对数几率以及其他关键模型输出将保持不变。

然而，在多项逻辑回归模型中解释系数时必须谨慎，因为它与基准类别的选择密切相关。例如，如果我们将 `epileptic seizure` 设为基准类别，那么我们可以将 $\beta_{\texttt{stroke0}}$ 解释为：在 $x_1 = \cdots = x_p =$ 为 0 的条件下，`stroke` 相对于 `epileptic seizure` 的对数几率。此外，X_j 每增加一个单位，与 `stroke` 相对于 `epileptic seizure` 的对数几率增加 $\beta_{\texttt{stroke}j}$ 相关。换句话说，如果 X_j 增加一个单位，那么

$$\frac{\Pr(Y = \texttt{stroke}|X = x)}{\Pr(Y = \texttt{epileptic seizure}|X = x)}$$

增加 $e^{\beta_{\texttt{stroke}j}}$。

我们现在简要介绍一种多项逻辑回归的替代编码方式，称为*softmax*编码。softmax编码是等价的与刚刚描述的编码相比，拟合值、任意类别对之间的对数几率以及其他关键模型输出将保持不变，无论采用何种编码。但softmax编码在机器学习文献的某些领域中被广泛使用（并将在第10章再次出现），因此值得了解。在softmax编码中，我们不选择基线类别，而是对称地对待所有K类别，并假设对于$k = 1, \ldots, K$,

softmax

$$\Pr(Y = k|X = x) = \frac{e^{\beta_{k0}+\beta_{k1}x_1+\cdots+\beta_{kp}x_p}}{\sum_{l=1}^{K} e^{\beta_{l0}+\beta_{l1}x_1+\cdots+\beta_{lp}x_p}}. \tag{4.13}$$

因此，我们实际上不是为 $K-1$ 个类别估计系数，而是为所有 K 个类别估计系数。不难看出，由（4.13）可得，第 k 类与第 k' 类之间的对数几率比等于

$$\log\left(\frac{\Pr(Y=k|X=x)}{\Pr(Y=k'|X=x)}\right)=(\beta_{k0}-\beta_{k'0})+(\beta_{k1}-\beta_{k'1})x_1+\cdots+(\beta_{kp}-\beta_{k'p})x_p. \tag{4.14}$$

4.4 分类的生成模型

逻辑回归涉及使用逻辑函数直接对Pr($Y=k|X=x$)建模，该函数由式(4.7)给出，适用于两类响应情况。用统计术语来说，我们建模的是响应Y在预测变量X给定条件下的条件分布。现在，我们考虑一种替代且较为间接的方法来估计这些概率。在这种新方法中，我们分别在每个响应类别中（即对于Y的每个值）独立建模预测变量X的分布。随后，我们运用贝叶斯定理将其转换为对Pr($Y=k|X=x$)的估计。当假定每个类别内X的分布为正态时，所得到的模型在形式上与逻辑回归非常相似。

当我们已经有了逻辑回归时，为什么还需要另一种方法呢？原因有几个：

- 当两类之间存在显著分离时，逻辑回归模型的参数估计出奇地不稳定。本节所考虑的方法不会遇到这个问题。
- 如果预测变量 X 的分布在每个类别中近似正态分布，且样本量较小，那么本节中的方法可能比逻辑回归更准确。
- 本节中的方法可以自然地扩展到响应类别多于两个的情况。（在响应类别多于两个的情况下，我们也可以使用第4.3.5节中的多项逻辑回归。）

假设我们希望将一个观测值分类到K个类别之一，其中$K\geq 2$。换句话说，定性响应变量Y可以取K个可能的不同且无序的值。令π_k表示随机选择的观测值来自

先验

k第th类。令$f_k(X)\equiv$ Pr($X|Y=k$)[1] 表示 X 的 *density function* 对于来自第k类的观测。换句话说，如果第k类中的观测具有$X\approx x$的概率很高，则$f_k(x)$相对较大；如果第k类中的观测具有$X\approx x$的可能性非常小，则$f_k(x)$较小。然后*Bayes' theorem*表明

密度函数

Bayes' theorem

[1]Technically, this definition is only correct if X is a qualitative random variable. If X is quantitative, then $f_k(x)dx$ corresponds to the probability of X falling in a small region dx around x.

$$\Pr(Y = k|X = x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^{K} \pi_l f_l(x)}. \tag{4.15}$$

根据我们之前的符号约定，我们将使用缩写 $p_k(x) = \Pr(Y = k|X = x)$; 这是观测的 *posterior* 概率
$X = x$ 属于第 k 类。也就是说，它是观测值属于第 k 类的概率，*given* 是该观测值的预测值。

posterior

式4.15表明，无需像4.3.1节那样直接计算后验概率$p_k(x)$，我们只需将π_k和$f_k(x)$的估计值代入(4.15)即可。通常，若拥有来自总体的随机样本，估计π_k是容易的：我们只需计算属于第k类的训练观测值的比例。然而，估计密度函数$f_k(x)$则更具挑战性。正如我们将看到的，为了估计$f_k(x)$，我们通常必须做出一些简化假设。

我们从第2章了解到，贝叶斯分类器将观测值 x 归入使得 $p_k(x)$ 最大的类别，它在所有分类器中具有最低的误差率。（当然，这仅在(4.15)中所有项均被正确指定的情况下成立。）因此，如果我们能找到估计 $f_k(x)$ 的方法，那么就可以将其代入(4.15)中，以近似贝叶斯分类器。

在以下各节中，我们讨论三种分类器，它们使用(4.15)中$f_k(x)$的不同估计来近似贝叶斯分类器：*linear dis-*、
*criminant analysis, quadratic discriminant analysis,*和*naive Bayes*。

4.4.1 Linear Discriminant Analysis for $p = 1$

目前，假设p ==1——即我们仅有一个预测变量。我们希望获得$f_k(x)$的估计值，以便代入(4.15)中估计$p_k(x)$。随后，我们将观察值分类到$p_k(x)$最大的类别。为了估计$f_k(x)$，我们首先对其形式做出一些假设。

特别地，我们假设$f_k(x)$是*normal*或*Gaussian*。在单
在维度设定下，正态密度形式为

normal
Gaussian

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{1}{2\sigma_k^2}(x - \mu_k)^2\right), \tag{4.16}$$

其中 μ_k 和 σ_k^2 是第 k 类的均值和方差参数。目前，让我们进一步假设 $\sigma_1^2 = \cdots = \sigma_K^2$：即所有 K 类共享一个方差项，为简化起见，我们可以用 σ^2 表示。将 (4.16) 代入 (4.15)，我们得到

$$p_k(x) = \frac{\pi_k \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2\sigma^2}(x - \mu_k)^2\right)}{\sum_{l=1}^{K} \pi_l \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2\sigma^2}(x - \mu_l)^2\right)}. \tag{4.17}$$

(注意，在 (4.17) 中，π_k 表示观测属于第 k 类的先验概率，不要与数学常数 $\pi \approx 3.14159$ 混淆。）贝叶斯分类器[2] 涉及将观测分配给

[2]Recall that the *Bayes classifier* assigns an observation to the class for which $p_k(x)$ is largest. This is different from *Bayes' theorem* in (4.15), which allows us to manipulate conditional distributions.

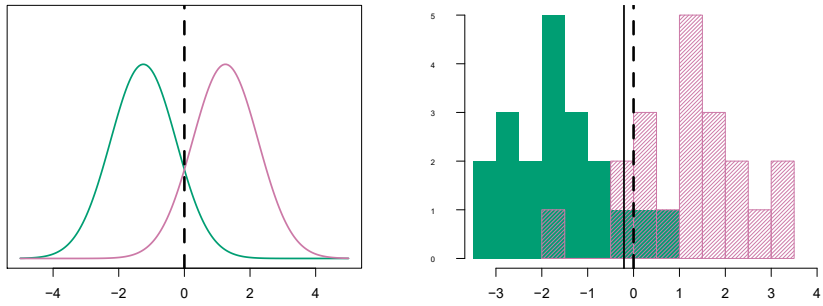


图4.4。左：*Two one-dimensional normal density functions are shown. The dashed vertical line represents the Bayes decision boundary.* 右：*20 observations were drawn from each of the two classes, and are shown as histograms. The Bayes decision boundary is again shown as a dashed vertical line. The solid vertical line represents the LDA decision boundary estimated from the training data.*

$X = x$ 将其分配给使(4.17)最大的类别。对(4.17)取对数并重新排列各项，不难证明[3]这等价于将观测值分配给满足以下条件的类别：

$$\delta_k(x) = x \cdot \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k) \tag{4.18}$$

最大。例如，如果$K = 2$且$\pi_1 = \pi_2$，那么贝叶斯分类器将观测值分配给类别1，如果$2x\,(\mu_1 - \mu_2) > \mu_1^2 - \mu_2^2$，否则分配给类别2。贝叶斯决策边界是$\delta_1(x) = \delta_2(x)$的点；可以证明这相当于

$$x = \frac{\mu_1^2 - \mu_2^2}{2(\mu_1 - \mu_2)} = \frac{\mu_1 + \mu_2}{2}. \tag{4.19}$$

一个例子如图4.4左侧面板所示。所示的两个正态密度函数，$f_1(x)$和$f_2(x)$，代表了两个不同的类别。这两个密度函数的均值和方差参数分别为$\mu_1 = -1.25$，$\mu_2 = 1.25$，以及$\sigma_1^2 = \sigma_2^2 = 1$。由于两个密度函数有重叠，因此在给定$X = x$的情况下，观测值所属的类别存在一定的不确定性。如果我们假设观测值来自任一类的可能性相等——即$\pi_1 = \pi_2 = 0.5$——那么通过观察(4.19)式，我们可以看出贝叶斯分类器将观测值分配给类别1，如果$x < 0$；否则分配给类别2。需要注意的是，在这种情况下，我们能够计算贝叶斯分类器，因为我们知道在每个类别内X是从高斯分布中抽取的，并且我们知道所有涉及的参数。在现实情况下，我们无法计算贝叶斯分类器。

在实践中，即便我们相当确信假定 X 在每个类别内服从高斯分布，应用贝叶斯分类器时仍需估计参数 $\mu_1, \ldots, \mu_K$、$\pi_1, \ldots, \pi_K$ 和 σ^2。

linear discriminant analysis (LDA) 方法近似将π_k、μ_k和σ^2的估计值代入(4.18)得到的贝叶斯分类器。在

linear discriminant analysis

[3]See Exercise 2 at the end of this chapter.

特别地，使用以下估计值：

$$\begin{aligned}\hat{\mu}_k &= \frac{1}{n_k}\sum_{i:y_i=k} x_i \\ \hat{\sigma}^2 &= \frac{1}{n-K}\sum_{k=1}^{K}\sum_{i:y_i=k}(x_i-\hat{\mu}_k)^2 \end{aligned} \tag{4.20}$$

其中 n 是训练观测总数，n_k 是第 k 类中的训练观测数。μ_k 的估计量就是第 k 类所有训练观测的平均值，而 $\hat{\sigma}^2$ 可以看作是 K 个类各自样本方差的加权平均。有时我们知道类别成员概率 $\pi_1,\ldots,\pi_K$，可以直接使用。在没有任何额外信息的情况下，LDA 使用属于第 k 类的训练观测比例来估计 π_k。换句话说，

$$\hat{\pi}_k = n_k/n. \tag{4.21}$$

LDA分类器将(4.20)和(4.21)中给出的估计值代入(4.18)，并将观测值$X=x$分配给满足以下条件的类别：

$$\hat{\delta}_k(x) = x\cdot\frac{\hat{\mu}_k}{\hat{\sigma}^2} - \frac{\hat{\mu}_k^2}{2\hat{\sigma}^2} + \log(\hat{\pi}_k) \tag{4.22}$$

最大。分类器名称中的 *linear* 一词源于 (4.22) 中的 *discriminant functions* $\hat{\delta}_k(x)$ 是关于 x (的线性函数，即与x)的更复杂函数相对。

discriminant function

图4.4右侧面板显示了从每个类别中随机抽取的20个观测值的直方图。为实现LDA，我们首先使用(4.20)和(4.21)估计π_k，μ_k和σ^2。随后，我们计算了决策边界，以黑色实线表示，该边界由将观测值分配给使得(4.22)最大的类别所确定。此线左侧的所有点将被归入绿色类别，而右侧的点则归入紫色类别。在此情形下，由于$n_1=n_2=20$，我们有$\hat{\pi}_1=\hat{\pi}_2$。因此，决策边界对应于两个类别样本均值的中点，即$(\hat{\mu}_1+\hat{\mu}_2)/2$。图中显示，LDA决策边界略微偏左于最优贝叶斯决策边界，后者等于$(\mu_1+\mu_2)/2=0$。LDA分类器在这组数据上的表现如何？由于这是模拟数据，我们可以生成大量测试观测值，以计算贝叶斯错误率和LDA测试错误率。这两个错误率分别为10.6%和11.1%。换句话说，LDA分类器的错误率仅比最小可能错误率高0.5%！这表明LDA在此数据集上的表现相当好。

重申一下，LDA分类器源于以下假设：每个类别内的观测值来自具有类别特定均值和共同方差σ^2的正态分布，并将这些参数的估计值代入贝叶斯分类器。在第4.4.3节中，我们将考虑一组不那么严格的假设，即允许第k类中的观测值具有类别特定的方差σ_k^2。

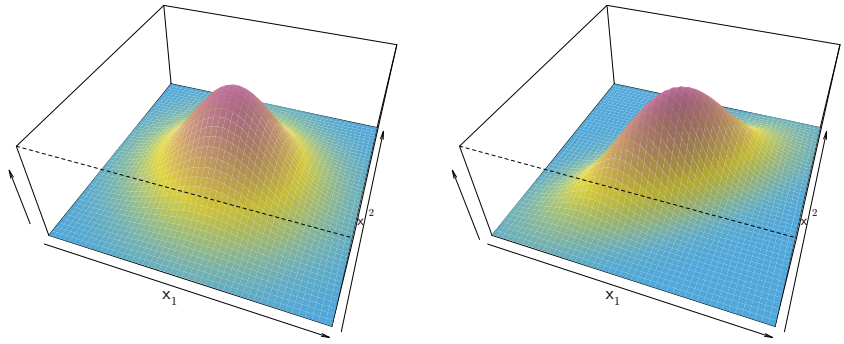


图4.5. *Two multivariate Gaussian density functions are shown, with* $p = 2$. 左：*The two predictors are uncorrelated.* 右：*The two variables have a correlation of* 0.7.

4.4.2 Linear Discriminant Analysis for $p > 1$

我们现在将LDA分类器扩展到多个预测变量的情况。为此，我们将假设$X = (X_1, X_2, \ldots, X_p)$是从*multi-variate Gaussian* (或多元正态)分布中抽取的，具有类别特定的
均值向量和共同的协方差矩阵。我们首先简要回顾这一分布。

multivariate Gaussian

多元高斯分布假设每个个体预测变量遵循一维正态分布，如式(4.16)所示，且每对预测变量之间存在一定的相关性。图4.5展示了两个具有 $p = 2$ 的多元高斯分布示例。曲面在任意特定点处的高度表示 X_1 和 X_2 同时落在该点附近小区域内的概率。在任一图中，若沿 X_1 轴或 X_2 轴切割曲面，所得的截面将呈现一维正态分布的形状。图4.5左侧面板展示了一个 $\text{Var}(X_1) = \text{Var}(X_2)$ 且 $\text{Cor}(X_1, X_2) = 0$ 的示例；该曲面具有特征性的 *bell shape*。然而，若预测变量之间存在相关性或方差不等，钟形曲线将发生畸变，如图4.5右侧面板所示。在此情况下，钟形底部将呈椭圆形而非圆形。为表明 p 维随机变量 X 具有多元高斯分布，我们记作 $X \sim N(\mu, \mathbf{\Sigma})$。此处 $\text{E}(X) = \mu$ 是 X (的均值，该向量具有 p 个分量)，而 $\text{Cov}(X) = \mathbf{\Sigma}$ 是 X 的 $p \times p$ 协方差矩阵。形式上，多元高斯密度定义为

$$f(x) = \frac{1}{(2\pi)^{p/2}|\mathbf{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^T\mathbf{\Sigma}^{-1}(x-\mu)\right). \qquad (4.23)$$

在$p > 1$个预测变量的情况下，LDA分类器假设第k类中的观测值来自多元高斯分布$N(\mu_k, \mathbf{\Sigma})$，其中$\mu_k$是类特定的均值向量，而$\mathbf{\Sigma}$是所有$K$类共有的协方差矩阵。将第$k$类的密度函数$f_k(X = x)$代入(4.15)并进行一些代数运算后，可以揭示贝叶斯分类器将观测值$X = x$

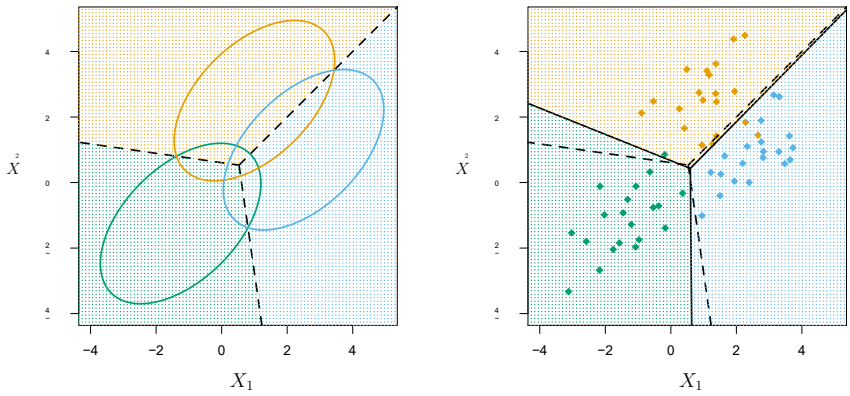


图 4.6. *An example with three classes. The observations from each class are drawn from a multivariate Gaussian distribution with* $p = 2$*, with a class-specific mean vector and a common covariance matrix.* 左：*Ellipses that contain* 95 *% of the probability for each of the three classes are shown. The dashed lines are the Bayes decision boundaries.* 右：20 *observations were generated from each class, and the corresponding LDA decision boundaries are indicated using solid black lines. The Bayes decision boundaries are once again shown as dashed lines.*

到该类对应的

$$\delta_k(x) = x^T \mathbf{\Sigma}^{-1} \mu_k - \frac{1}{2} \mu_k^T \mathbf{\Sigma}^{-1} \mu_k + \log \pi_k \tag{4.24}$$

是最大的。这是(4.18)的向量/矩阵版本。

图4.6左侧面板展示了一个示例。图中展示了三个大小相等的高斯类别，它们具有类特定均值向量和共同协方差矩阵。三个椭圆分别代表了包含三个类别各自95%概率的区域。虚线表示贝叶斯决策边界。换言之，它们代表了使得$\delta_k(x) = \delta_\ell(x)$成立的$x$值集合；即

$$x^T \mathbf{\Sigma}^{-1} \mu_k - \frac{1}{2} \mu_k^T \mathbf{\Sigma}^{-1} \mu_k = x^T \mathbf{\Sigma}^{-1} \mu_l - \frac{1}{2} \mu_l^T \mathbf{\Sigma}^{-1} \mu_l \tag{4.25}$$

对于 $k \neq l$。（来自(4.24)的 π_k 对数项已消失，因为三个类别的训练观测数量相同；即 π_k 对每个类别都相同。）注意，有三条线代表贝叶斯决策边界，因为三个类别之间存在三个 *pairs of classes*。也就是说，一条贝叶斯决策边界将类别1与类别2分开，一条将类别1与类别3分开，还有一条将类别2与类别3分开。这三条贝叶斯决策边界将预测变量空间划分为三个区域。贝叶斯分类器将根据观测所在的区域对其进行分类。

再一次，我们需要估计未知参数 $\mu_1, \ldots, \mu_K$、$\pi_1, \ldots, \pi_K$ 和 $\mathbf{\Sigma}$；公式与一维情况中使用的公式相似，见 (4.20)。为了分配一个新的观测 $X = x$，LDA 将这些估计值代入 (4.24) 以获得量 $\hat{\delta}_k(x)$，并分类到 $\hat{\delta}_k(x)$ 最大的类别。注意，在 (4.24) 中，$\delta_k(x)$ 是 x 的线性函数；也就是说，LDA 决策规则仅依赖于 x

		True default status		
		No	Yes	Total
Predicted default status	No	9644	252	9896
	Yes	23	81	104
	Total	9667	333	10000

表4.4. *A confusion matrix compares the LDA predictions to the true default statuses for the* 10,000 *training observations in the* `Default` *data set. Elements on the diagonal of the matrix represent individuals whose default statuses were correctly predicted, while off-diagonal elements represent individuals that were misclassified. LDA made incorrect predictions for* 23 *individuals who did not default and for* 252 *individuals who did default.*

通过其元素的线性组合。如前所述，这是LDA中*linear*一词的原因。

在图4.6的右侧面板中，展示了从三个类别中各抽取的20个观测值，并且由此得到的LDA决策边界以实心黑线表示。总体而言，LDA决策边界与再次以虚线显示的贝叶斯决策边界非常接近。贝叶斯分类器和LDA分类器的测试错误率分别为0.0746和0.0770。这表明LDA在这组数据上表现良好。

我们可以对 `Default` 数据进行 LDA，以根据信用卡余额和学生身份预测个人是否会违约。[4] 对 10,000 个训练样本拟合的 LDA 模型产生的错误率为 2.75%。这听起来像是一个低错误率，但必须注意两个警告。

- 首先，训练错误率通常会低于测试错误率，而后者才是真正关注的量。换句话说，如果我们用这个分类器来预测一组新的个体是否会违约，我们可能预期它的表现会更差。原因在于我们特意调整模型参数以在训练数据上表现良好。参数数量p与样本数量n的比例越高，我们越预期这种*overfitting*会发挥作用。对于

  这些数据，我们并不预期这会成为问题，因为$p = 2$ 和$n = 10,000$。 overfitting

- 其次，由于训练样本中仅有3.33%的个体违约，一个简单但无用的分类器，无论其信用卡余额和学生身份如何，总是预测个体不会违约，将导致3.33%的错误率。换言之，这个平凡的*null*分类器将达到的错误率是

  仅比LDA训练集的错误率略高一点。 null

实际上，像这样的二分类器可能会犯两种类型的错误：它可能错误地将违约的个体分配到*no default*类别，或者它可能错误地将未违约的个体分配到

[4] The careful reader will notice that student status is qualitative — thus, the normality assumption made by LDA is clearly violated in this example! However, LDA is often remarkably robust to model violations, as this example shows. Naive Bayes, discussed in Section 4.4.4, provides an alternative to LDA that does not assume normally distributed predictors.

*default*类别。确定正在犯这两类错误中的哪一类通常很有意义。一个 *confusion matrix*，展示给`Default`
表4.4中的数据，是展示这一信息的便捷方式。该表显示，LDA预测共有104人会违约。在这104人中，81人确实违约，而23人没有。因此，在9,667名未违约个体中，仅有23人被错误标记。这看起来错误率相当低！然而，在333名违约的个体中，LDA漏掉了252人（或75.7%）。所以，尽管总体错误率较低，但违约个体中的错误率非常高。从试图识别高风险个体的信用卡公司角度来看，违约个体中$252/333 = 75.7\%$的错误率很可能是不可接受的。

混淆矩阵

特定类别的性能在医学和生物学中也很重要，其中术语 *sensitivity* 和 *specificity* 描述了
分类器或筛选测试。在这种情况下，敏感性是指识别出的真正违约者的百分比，等于24.3%。特异性是指正确识别出的非违约者的百分比，等于$(1 - 23/9667) = 99.8\%$。

sensitivity
specificity

为什么LDA在对违约客户进行分类时表现如此糟糕？换句话说，为什么它的敏感度如此之低？正如我们所看到的，LDA试图逼近贝叶斯分类器，而贝叶斯分类器在所有分类器中具有最低的*total*错误率。也就是说，无论错误来自哪个类别，贝叶斯分类器都将产生尽可能少的误分类观测总数。有些误分类源于将不会违约的客户错误地分配到违约类别，另一些则源于将违约客户错误地分配到非违约类别。相比之下，信用卡公司可能尤其希望避免将将要违约的个人错误分类，而将不会违约的个人错误分类虽然仍应避免，但问题较小。我们现在将看到，可以修改LDA以开发出更好满足信用卡公司需求的分类器。

贝叶斯分类器通过将观测值分配给后验概率 $p_k(X)$ 最大的类别来工作。在二分类情况下，这相当于将观测值分配给 *default* 类，如果

$$\Pr(\texttt{default} = \texttt{Yes}|X = x) > 0.5. \tag{4.26}$$

因此，贝叶斯分类器，以及延伸开来的LDA，使用50%的后验违约概率阈值来将观测分配给*default*类别。然而，如果我们担心错误地预测那些违约个体的违约状态，那么我们可以考虑降低这个阈值。例如，我们可能将任何后验违约概率高于20%的客户标记为*default*类别。换句话说，我们不是在条件(4.26)成立时将观测归为*default*类，而是可以在以下条件成立时将其归为该类:

$$\Pr(\texttt{default} = \texttt{Yes}|X = x) > 0.2. \tag{4.27}$$

表4.5展示了采用这种方法得到的错误率。现在，LDA预测将有430人违约。在333名实际违约的人中，LDA正确预测了除138人之外的所有人，准确率为41.4%。这是一个巨大的

		True default status		
		No	Yes	Total
Predicted default status	No	9432	138	9570
	Yes	235	195	430
	Total	9667	333	10000

表 4.5. *A confusion matrix compares the LDA predictions to the true default statuses for the* 10,000 *training observations in the* `Default` *data set, using a modified threshold value that predicts default for any individuals whose posterior default probability exceeds* 20 %.

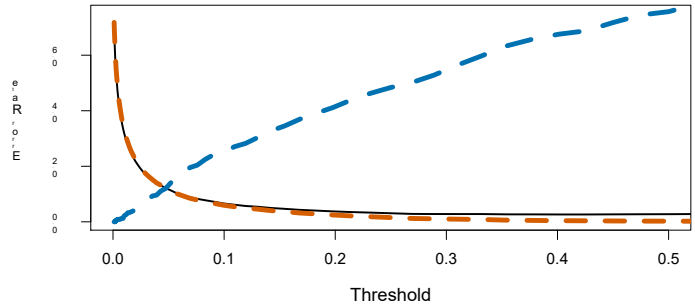


图4.7。*For the* `Default` *data set, error rates are shown as a function of the threshold value for the posterior probability that is used to perform the assignment. The black solid line displays the overall error rate. The blue dashed line represents the fraction of defaulting customers that are incorrectly classified, and the orange dotted line indicates the fraction of errors among the non-defaulting customers.*

相比使用50%阈值导致的75.7%错误率，这一改进有所提升。然而，这一改进是有代价的：现在有235名未违约的个体被错误分类。因此，总体错误率略微上升至3.73%。但信用卡公司可能认为，为了更准确地识别确实违约的个体，总体错误率的这一轻微上升是值得付出的小代价。

图4.7展示了修改违约后验概率阈值所导致的权衡。各种错误率作为阈值函数被显示出来。使用0.5的阈值，如同(4.26)中那样，可以最小化总体错误率，如图中黑色实线所示。这是预料之中的，因为贝叶斯分类器使用0.5的阈值，并已知具有最低的总体错误率。然而，当使用0.5的阈值时，违约个体中的错误率相当高（蓝色虚线）。随着阈值的降低，违约个体的错误率逐渐减少，但非违约个体的错误率则增加。我们如何决定哪个阈值最佳呢？这样的决策必须基于*domain knowledge*，比如关于违约相关成本的详细信息。

ROC curve

*ROC curve*是一种流行的图形，用于同时显示所有可能阈值下的两类错误。名称“ROC”是历史遗留的，源自通信理论。它是*receiver op-erating characteristics*的缩写。图4.8展示了LDA分类器在训练数据上的ROC曲线。分类器的整体性能，总结

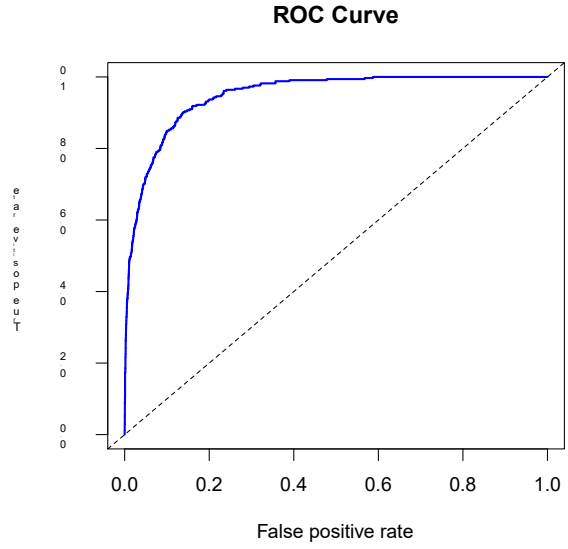


图4.8. *A ROC curve for the LDA classifier on the* `Default` *data. It traces out two types of error as we vary the threshold value for the posterior probability of default. The actual thresholds are not shown. The true positive rate is the sensitivity: the fraction of defaulters that are correctly identified, using a given threshold value. The false positive rate is 1-specificity: the fraction of non-defaulters that we classify incorrectly as defaulters, using that same threshold value. The ideal ROC curve hugs the top left corner, indicating a high true positive rate and a low false positive rate. The dotted line represents the "no information" classifier; this is what we would expect if student status and credit card balance are not associated with probability of default.*

对所有可能的阈值进行汇总，由*area under the (ROC) curve* (AUC)给出。理想的ROC曲线会紧贴左上角，因此越大
AUC越大，分类器越好。对于此数据，AUC为0.95，接近最大值1.0，因此被认为非常好。我们预期一个表现不超过随机猜测的分类器AUC为0.5（在独立测试集上评估时，该测试集未用于模型训练）。ROC曲线对于比较不同分类器很有用，因为它们考虑了所有可能的阈值。事实证明，4.3.4节中逻辑回归模型拟合这些数据的ROC曲线与LDA模型的ROC曲线几乎无法区分，因此我们在此不再展示。

(ROC) 曲线下的面积

如上所述，改变分类器阈值会改变其真阳性率和假阳性率。这些也被称为 *sensitivity* 和一
减去我们分类器的 *specificity*。由于存在一个几乎令人困惑的
此上下文中使用的术语数组，我们现在给出一个总结。表4.6展示了将分类器（或诊断测试）应用于群体时可能的结果。为了与流行病学文献建立联系，我们将“+”视为我们试图检测的“疾病”，而将“−”视为“非疾病”状态。为了与经典假设检验文献相联系，我们将“−”视为零假设，而将“+”视为

灵敏度
特异性

		True class		
		− or Null	+ or Non-null	Total
Predicted class	− or Null	True Neg. (TN)	False Neg. (FN)	N*
	+ or Non-null	False Pos. (FP)	True Pos. (TP)	P*
	Total	N	P	

表 4.6. *Possible results when applying a classifier or diagnostic test to a population.*

Name	Definition	Synonyms
False Pos. rate	FP/N	Type I error, 1−Specificity
True Pos. rate	TP/P	1−Type II error, power, sensitivity, recall
Pos. Pred. value	TP/P*	Precision, 1−false discovery proportion
Neg. Pred. value	TN/N*	

表4.7。 *Important measures for classification and diagnostic testing, derived from quantities in Table 4.6.*

替代（非零）假设。在 `Default` 数据的背景下，“+” 表示违约的个体，而 “−” 表示未违约的个体。

表4.7列出了在此背景下使用的许多流行性能指标。假阳性率和真阳性率的分母是每个类别的实际人口计数。相比之下，阳性预测值和阴性预测值的分母是每个类别的总预测计数。

4.4.3 Quadratic Discriminant Analysis

正如我们所讨论的，LDA 假设每个类别内的观测值都是从具有类别特定均值向量和所有 K 类别共有的协方差矩阵的多元高斯分布中抽取的。*Quadratic discriminant analysis* (QDA) 提供了一种替代方法。

quadratic discriminant analysis

与LDA类似，QDA分类器源于假设每个类别的观测值均来自高斯分布，并将参数估计值代入贝叶斯定理以进行预测。然而，与LDA不同的是，QDA假设每个类别拥有自己的协方差矩阵。也就是说，它假定来自第k类的观测值具有形式$X \sim N(\mu_k, \mathbf{\Sigma}_k)$，其中$\mathbf{\Sigma}_k$是第$k$类的协方差矩阵。在此假设下，贝叶斯分类器将观测值$X = x$分配给满足以下条件的类别：

$$\begin{aligned}\delta_k(x) &= -\frac{1}{2}(x-\mu_k)^T\mathbf{\Sigma}_k^{-1}(x-\mu_k) - \frac{1}{2}\log|\mathbf{\Sigma}_k| + \log\pi_k \\ &= -\frac{1}{2}x^T\mathbf{\Sigma}_k^{-1}x + x^T\mathbf{\Sigma}_k^{-1}\mu_k - \frac{1}{2}\mu_k^T\mathbf{\Sigma}_k^{-1}\mu_k - \frac{1}{2}\log|\mathbf{\Sigma}_k| + \log\pi_k\end{aligned} \tag{4.28}$$

最大。因此，QDA分类器涉及将$\mathbf{\Sigma}_k$、μ_k和π_k的估计值代入(4.28)，然后将观测$X = x$分配给该量最大的类别。与(4.24)不同，量x在(4.28)中作为*quadratic*函数出现。这就是QDA得名的原因。

为什么我们是否假设K类共享一个共同的协方差矩阵很重要？换句话说，为什么人们会更倾向于LDA而不是

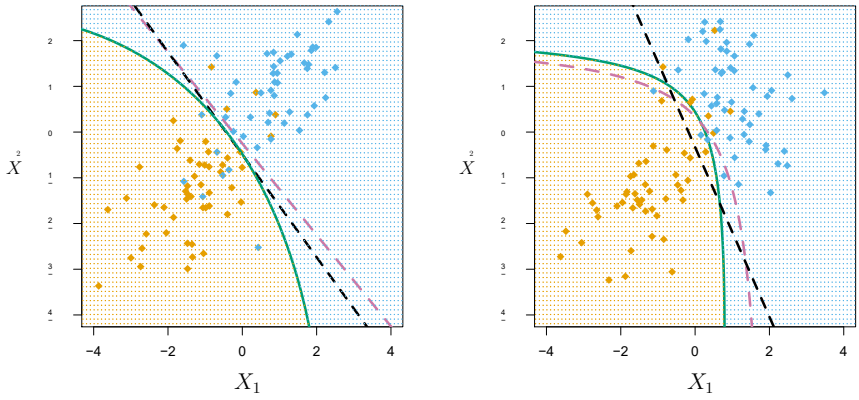


图4.9. 左图：*The Bayes (purple dashed), LDA (black dotted), and QDA (green solid) decision boundaries for a two-class problem with* $\mathbf{\Sigma}_1 = \mathbf{\Sigma}_2$. *The shading indicates the QDA decision rule. Since the Bayes decision boundary is linear, it is more accurately approximated by LDA than by QDA.* 右图：*Details are as given in the left-hand panel, except that* $\mathbf{\Sigma}_1 \neq \mathbf{\Sigma}_2$. *Since the Bayes decision boundary is non-linear, it is more accurately approximated by QDA than by LDA.*

QDA，还是反之？答案在于偏差-方差权衡。当有p个预测变量时，估计协方差矩阵需要估计$p(p+1)/2$个参数。QDA为每个类别估计一个单独的协方差矩阵，总共需要$Kp(p+1)/2$个参数。当有50个预测变量时，这大约是1,275的若干倍，参数数量很多。通过假设K个类别共享一个共同协方差矩阵，LDA模型在x中变为线性，这意味着需要估计Kp个线性系数。因此，LDA是一个远不如QDA灵活的 classifier，因此具有显著更低的方差。这可能潜在地带来更好的预测性能。但存在一个权衡：如果LDA关于K个类别共享一个共同协方差矩阵的假设严重不成立，那么LDA可能会遭受高偏差。粗略地说，如果训练观测值相对较少，因此降低方差至关重要，LDA往往比QDA更可取。相反，如果训练集非常大，从而 classifier 的方差不是主要问题，或者如果K个类别共享一个共同协方差矩阵的假设明显站不住脚，则推荐使用QDA。

图4.9展示了LDA和QDA在两种情形下的表现。左侧面板中，两个高斯类别在X_1和X_2之间具有共同的0.7相关性。因此，贝叶斯决策边界是线性的，并且被LDA决策边界准确近似。QDA决策边界较差，因为它遭受更高的方差而没有相应减少偏差。相反，右侧面板显示了一种情况，其中橙色类别在变量之间具有0.7的相关性，而蓝色类别具有-0.7的相关性。现在贝叶斯决策边界是二次的，因此QDA比LDA更准确地近似这个边界。

4.4.4 Naive Bayes

在前面的章节中，我们使用贝叶斯定理(4.15)来开发LDA和QDA分类器。在这里，我们使用贝叶斯定理来激发流行的*naive Bayes*分类器。

naive Bayes

回想一下，贝叶斯定理（4.15）通过 $\pi_1, \ldots, \pi_K$ 和 $f_1(x), \ldots, f_K(x)$ 给出了后验概率 $p_k(x) = \Pr(Y = k|X = x)$ 的表达式。为了在实践中使用（4.15），我们需要对 $\pi_1, \ldots, \pi_K$ 和 $f_1(x), \ldots, f_K(x)$ 进行估计。正如前几节所见，估计先验概率 $\pi_1, \ldots, \pi_K$ 通常很简单：例如，对于 $k = 1, \ldots, K$，我们可以将 $\hat{\pi}_k$ 估计为属于第 k 类的训练观测值的比例。

然而，估计$f_1(x), \ldots, f_K(x)$则更为微妙。回想一下，$f_k(x)$是第k类观测的p维密度函数，对于$k = 1, \ldots, K$。一般来说，估计一个p维密度函数是具有挑战性的。在LDA中，我们做了一个非常强的假设，极大地简化了任务：我们假设f_k是一个多元正态随机变量的密度函数，具有类别特定的均值μ_k和共享的协方差矩阵$\mathbf{\Sigma}$。相比之下，在QDA中，我们假设f_k是一个多元正态随机变量的密度函数，具有类别特定的均值μ_k和类别特定的协方差矩阵$\mathbf{\Sigma}_k$。通过做出这些非常强的假设，我们能够将估计Kp维密度函数这一极具挑战性的问题替换为估计K p维均值向量和一个（在LDA情况下）或K（（在QDA情况下））$(p \times p)$维协方差矩阵这一更为简单的问题。

朴素贝叶斯分类器采用不同的方法来估计$f_1(x), \ldots, f_K(x)$。我们不是假设这些函数属于特定的分布族（例如多元正态分布），而是做出一个单一的假设：

Within the kth class, the p predictors are independent.

从数学上表述，这一假设意味着对于 $k = 1, \ldots, K$，

$$f_k(x) = f_{k1}(x_1) \times f_{k2}(x_2) \times \cdots \times f_{kp}(x_p), \tag{4.29}$$

其中 f_{kj} 是第 k 类观测中第 j 个预测变量的密度函数。

为什么这个假设如此强大？本质上，估计一个p维密度函数之所以具有挑战性，是因为我们必须考虑的不仅仅是每个预测变量的*marginal distribution*——即分布每个预测变量单独来看——但也包括预测变量的*joint distribution*——也就是说，不同预测变量之间的关联。在多元正态分布的情况下，不同预测变量之间的关联由协方差矩阵的非对角元素来概括。然而，一般来说，这种关联可能非常难以刻画，并且估计起来极具挑战性。但通过假设每个类别内的p协变量相互独立，我们完全消除了考虑p个预测变量之间关联的必要性，因为我们只是假设预测变量之间存在*no*关联！

边缘分布

联合分布

我们真的相信朴素贝叶斯假设，即每个类别内的p个协变量是独立的吗？在大多数情况下，我们并不相信。但即使这种建模假设是为了方便而做出的，它往往会导致

相当不错的结果，特别是在n相对于p不够大，以至于我们无法有效估计每个类别内预测变量的联合分布的情况下。事实上，由于估计联合分布需要大量数据，朴素贝叶斯在广泛的情境中都是一个好选择。本质上，朴素贝叶斯假设引入了一些偏差，但减少了方差，从而由于偏差-方差权衡，使得分类器在实践中表现得相当好。

一旦我们做出了朴素贝叶斯假设，我们就可以将(4.29)代入(4.15)中，以得到后验概率的表达式，

$$\Pr(Y=k|X=x)=\frac{\pi_k\times f_{k1}(x_1)\times f_{k2}(x_2)\times\cdots\times f_{kp}(x_p)}{\sum_{l=1}^{K}\pi_l\times f_{l1}(x_1)\times f_{l2}(x_2)\times\cdots\times f_{lp}(x_p)} \quad (4.30)$$

对于$k=1,\ldots,K$。

为了使用训练数据$x_{1j},\ldots,x_{nj}$估计一维密度函数f_{kj}，我们有几个选项。

- 如果 X_j 是定量的，那么我们可以假设 $X_j|Y=k\sim N(\mu_{jk},\sigma_{jk}^2)$。换句话说，我们假设在每个类别内，第 j 个预测变量是从一个（单变量）正态分布中抽取的。虽然这听起来有点像 QDA，但有一个关键区别，即这里我们假设预测变量是独立的；这相当于 QDA 加上一个额外假设，即类别特定的协方差矩阵是对角矩阵。

- 如果 X_j 是定量的，那么另一个选择是对 f_{kj} 使用非参数估计。一个非常简单的方法是为每个类别中第 j 个预测变量的观测值制作直方图。然后，我们可以将 $f_{kj}(x_j)$ 估计为第 k 个类别中与 x_j 属于同一

  本质上是直方图的平滑版本。

  kernel density estimator

- 如果X_j是定性的，那么我们可以简单地计算第j个预测变量对应于每个类别的训练观测值的比例。例如，假设$X_j\in\{1,2,3\}$，并且我们在第k类中有100个观测值。假设第j个预测变量在这些观测值中分别有32、55和13个取值为1、2和3。那么我们可以估计f_{kj}为

$$\hat{f}_{kj}(x_j)=\begin{cases}0.32 & \text{if } x_j=1\\ 0.55 & \text{if } x_j=2\\ 0.13 & \text{if } x_j=3.\end{cases}$$

我们现在考虑一个朴素贝叶斯分类器的简单示例，其中包含$p=$个3预测变量和$K=$个2类别。前两个预测变量是定量的，第三个预测变量是定性的，具有三个水平。进一步假设$\hat{\pi}_1=\hat{\pi}_2=0.5$。估计的密度函数$\hat{f}_{kj}$对于$k=1,2$和$j=1,2,3$展示在图4.10中。现在假设我们希望对一个新观测进行分类，$x^*=(0.4,1.5,1)^T$。结果表明，在这个

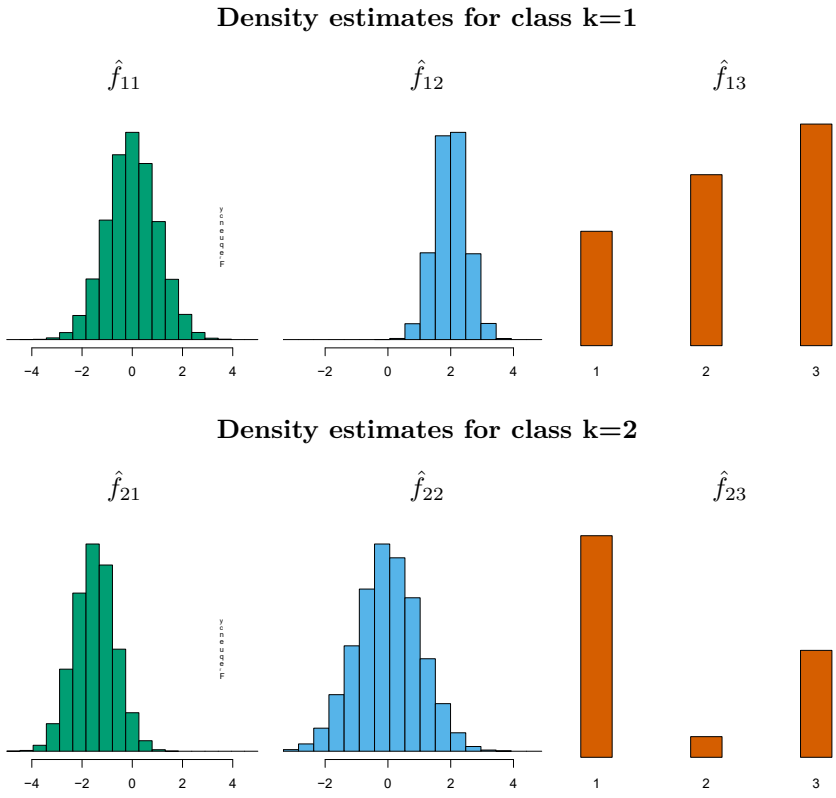


图 4.10. *In the toy example in Section 4.4.4, we generate data with* $p = 3$ *predictors and* $K = 2$ *classes. The first two predictors are quantitative, and the third predictor is qualitative with three levels. In each class, the estimated density for each of the three predictors is displayed. If the prior probabilities for the two classes are equal, then the observation* $x^* = (0.4, 1.5, 1)^T$ *has a* 94.4% *posterior probability of belonging to the first class.*

		True default status		
		No	Yes	Total
Predicted default status	No	9621	244	9865
	Yes	46	89	135
	Total	9667	333	10000

表 4.8. *Comparison of the naive Bayes predictions to the true default status for the* 10, 000 *training observations in the* `Default` *data set, when we predict default for any observation for which* $P(Y = \texttt{default}|X = x) > 0.5$.

例如，$\hat{f}_{11}(0.4) = 0.368$、$\hat{f}_{12}(1.5) = 0.484$、$\hat{f}_{13}(1) = 0.226$，以及$\hat{f}_{21}(0.4) = 0.030$、$\hat{f}_{22}(1.5) = 0.130$、$\hat{f}_{23}(1) = 0.616$。将这些估计值代入(4.30)式，得到后验概率估计值 $\Pr(Y = 1|X = x^*) = 0.944$ 和 $\Pr(Y = 2|X = x^*) = 0.056$。

表4.8展示了将朴素贝叶斯分类器应用于`Default`数据集所得到的混淆矩阵，其中如果违约的后验概率——即$P(Y = \texttt{default}|X = x)$——超过0.5，我们预测为违约。将此与表4.4中LDA的结果进行比较，我们的发现好坏参半。虽然LDA的总体错误率略低，但朴素贝叶斯

		True default status		
		No	Yes	Total
Predicted default status	No	9339	130	9469
	Yes	328	203	531
	Total	9667	333	10000

表4.9。*Comparison of the naive Bayes predictions to the true default status for the* 10, 000 *training observations in the* `Default` *data set, when we predict default for any observation for which* $P(Y = \texttt{default}|X = x) > 0.2$.

正确预测了更高比例的真实违约者。在朴素贝叶斯的这一实现中，我们假设每个定量预测变量都来自高斯分布（当然，还假设在每个类别内，每个预测变量都是独立的）。

正如LDA一样，我们可以轻松调整预测违约的概率阈值。例如，表4.9展示了当预测违约概率超过 $P(Y = \texttt{default}|X = x) >$0.2时的混淆矩阵结果。同样，与相同阈值下的LDA（表4.5）相比，结果好坏参半。朴素贝叶斯的错误率较高，但能正确预测几乎三分之二的实际违约情况。

在这个例子中，朴素贝叶斯未能令人信服地超越LDA并不令人意外：该数据集的$n =$为10,000，$p =$为2，因此由朴素贝叶斯假设带来的方差减少并不一定值得。我们预期在p较大或n较小的情况下，与LDA或QDA相比，使用朴素贝叶斯会有更大的收益，因为此时减少方差非常重要。

4.5 分类方法的比较

4.5.1 An Analytical Comparison

我们现在对LDA、QDA、朴素贝叶斯和逻辑回归进行*analytical*（或数学上的）比较。我们在具有K个类别的设定下考虑这些方法，因此我们将一个观测分配给使其Pr($Y = k|X = x$)最大化的类别。等价地，我们可以将K设为第*baseline*类，并将一个观测分配给使下式最大化的类别：

$$\log\left(\frac{\Pr(Y = k|X = x)}{\Pr(Y = K|X = x)}\right) \tag{4.31}$$

对于 $k = 1, \ldots, K$。检查每种方法的(4.31)的具体形式，可以清楚地理解它们的相似之处和差异。

首先，对于LDA，我们可以利用贝叶斯定理(4.15)以及每个类别内的预测变量都来自具有类别特定均值和共享协方差的多变量正态密度(4.23)的假设。

为了展示这一点，方差矩阵

$$\begin{aligned}
\log\left(\frac{\Pr(Y=k|X=x)}{\Pr(Y=K|X=x)}\right) &= \log\left(\frac{\pi_k f_k(x)}{\pi_K f_K(x)}\right) \\
&= \log\left(\frac{\pi_k \exp\left(-\frac{1}{2}(x-\mu_k)^T\mathbf{\Sigma}^{-1}(x-\mu_k)\right)}{\pi_K \exp\left(-\frac{1}{2}(x-\mu_K)^T\mathbf{\Sigma}^{-1}(x-\mu_K)\right)}\right) \\
&= \log\left(\frac{\pi_k}{\pi_K}\right) - \frac{1}{2}(x-\mu_k)^T\mathbf{\Sigma}^{-1}(x-\mu_k) \\
&\quad +\frac{1}{2}(x-\mu_K)^T\mathbf{\Sigma}^{-1}(x-\mu_K) \\
&= \log\left(\frac{\pi_k}{\pi_K}\right) - \frac{1}{2}(\mu_k+\mu_K)^T\mathbf{\Sigma}^{-1}(\mu_k-\mu_K) \\
&\quad + x^T\mathbf{\Sigma}^{-1}(\mu_k-\mu_K) \\
&= a_k + \sum_{j=1}^{p} b_{kj}x_j, \qquad (4.32)
\end{aligned}$$

其中 a_k = 记录 $\left(\frac{\pi_k}{\pi_K}\right) - \frac{1}{2}(\mu_k+\mu_K)^T\mathbf{\Sigma}^{-1}(\mu_k-\mu_K)$，而 b_{kj} 是 $\mathbf{\Sigma}^{-1}(\mu_k-\mu_K)$ 的第 j 个分量。因此，LDA 像逻辑回归一样，假设后验概率的对数几率在 x 中是线性的。

使用类似的计算，在QDA设置中，(4.31) 变为

$$\log\left(\frac{\Pr(Y=k|X=x)}{\Pr(Y=K|X=x)}\right) = a_k + \sum_{j=1}^{p} b_{kj}x_j + \sum_{j=1}^{p}\sum_{l=1}^{p} c_{kjl}x_j x_l, \qquad (4.33)$$

其中a_k, b_{kj}和c_{kjl}是$\pi_k, \pi_K, \mu_k, \mu_K, \mathbf{\Sigma}_k$和$\mathbf{\Sigma}_K$的函数。同样，顾名思义，QDA假设后验概率的对数几率是x的二次函数。

最后，我们考察朴素贝叶斯设置下的（4.31）。回想一下，在此设置中，$f_k(x)$被建模为p个一维函数$f_{kj}(x_j)$的乘积，其中j =∈{1, ..., , . . . , p}。因此，

$$\begin{aligned}
\log\left(\frac{\Pr(Y=k|X=x)}{\Pr(Y=K|X=x)}\right) &= \log\left(\frac{\pi_k f_k(x)}{\pi_K f_K(x)}\right) \\
&= \log\left(\frac{\pi_k \prod_{j=1}^{p} f_{kj}(x_j)}{\pi_K \prod_{j=1}^{p} f_{Kj}(x_j)}\right) \\
&= \log\left(\frac{\pi_k}{\pi_K}\right) + \sum_{j=1}^{p}\log\left(\frac{f_{kj}(x_j)}{f_{Kj}(x_j)}\right) \\
&= a_k + \sum_{j=1}^{p} g_{kj}(x_j), \qquad (4.34)
\end{aligned}$$

其中 a_k = 记录 $\left(\frac{\pi_k}{\pi_K}\right)$，而 $g_{kj}(x_j)$ = 记录 $\left(\frac{f_{kj}(x_j)}{f_{Kj}(x_j)}\right)$。因此，(4.34) 的右侧呈现为 *generalized additive model* 的形式，这一主题将在第7章进一步讨论。

检查(4.32)、(4.33)和(4.34)式，我们得出关于LDA、QDA和朴素贝叶斯的以下观察结果：

- LDA 是 QDA 的一个特例，其中对于所有 $j = 1, \ldots, p$，$l = 1, \ldots, p$ 和 $k = 1, \ldots, K$，$c_{kjl} = 0$。（当然，这并不令人惊讶，因为 LDA 仅仅是 QDA 的一个受限版本，其中 $\boldsymbol{\Sigma}_1 = \cdots = \boldsymbol{\Sigma}_K = \boldsymbol{\Sigma}$。）

- 任何具有线性决策边界的分类器都是朴素贝叶斯在$g_{kj}(x_j) = b_{kj}x_j$下的一个特例。特别地，这意味着LDA是朴素贝叶斯的一个特例！这一点从本章前面关于LDA和朴素贝叶斯的描述中完全看不出来，因为每种方法都做出了非常不同的假设：LDA假设特征呈正态分布且具有共同的类内协方差矩阵，而朴素贝叶斯则假设特征之间相互独立。

- 如果在朴素贝叶斯分类器中使用一维高斯分布$N(\mu_{kj}, \sigma_j^2)$来建模$f_{kj}(x_j)$，那么我们最终得到$g_{kj}(x_j) = b_{kj}x_j$，其中$b_{kj} = (\mu_{kj} - \mu_{Kj})/\sigma_j^2$。在这种情况下，朴素贝叶斯实际上是LDA的一个特例，其中$\boldsymbol{\Sigma}$被限制为对角矩阵，其第j个对角元素等于σ_j^2。

- QDA 和朴素贝叶斯都不是对方的特例。朴素贝叶斯可以产生更灵活的拟合，因为可以为$g_{kj}(x_j)$做出任意选择。然而，它仅限于纯粹$additive$拟合，因为在(4.34)中，x_j的函数$added$于x_l的函数，对于$j \neq l$；然而，这些项从不相乘。相比之下，QDA 包含形式为$c_{kjl}x_jx_l$的乘法项。因此，在预测变量之间的交互对于区分类别很重要的设置中，QDA 有可能更准确。

这些方法中没有一种能统一优于其他方法：在任何情况下，方法的选择将取决于每个K类别中预测变量的真实分布，以及诸如n和p的值等其他考虑因素。后者与偏差-方差权衡有关。

逻辑回归如何与此相关联？回想(4.12)中的多项式逻辑回归形式为

$$\log\left(\frac{\Pr(Y = k|X = x)}{\Pr(Y = K|X = x)}\right) = \beta_{k0} + \sum_{j=1}^{p} \beta_{kj}x_j.$$

这与LDA的线性形式（4.32）完全相同：在这两种情况下，$\log\left(\frac{\Pr(Y=k|X=x)}{\Pr(Y=K|X=x)}\right)$都是预测变量的线性函数。在LDA中，该线性函数的系数是根据假设每个类别内 $X_1, \ldots, X_p$ 遵循正态分布而得到的对 π_k、π_K、μ_k、μ_K 和 $\boldsymbol{\Sigma}$ 估计值的函数。相比之下，在逻辑回归中，系数被选择以最大化似然函数（4.5）。因此，当正态性假设（近似）成立时，我们预期LDA会比逻辑回归表现更好；而当该假设不成立时，我们预期逻辑回归表现更佳。

最后，我们简要讨论一下第2章中介绍的K-*nearest neighbors* (KNN)。回想一下，KNN采用的方法与本章中看到的分类型截然不同。为了对观测值$X = x$进行预测，会找出与x最接近的训练观测值。然后，将X分配给这些观测值中多数所属的类别。因此，KNN 是一种完全非参数化的方法：不对决策边界的形状做任何假设。关于 KNN，我们有以下观察：

- 因为 KNN 是完全非参数的，当决策边界高度非线性时，只要 n 非常大且 p 很小，我们可以预期这种方法将优于 LDA 和逻辑回归。

- 为了提供准确的分类，KNN 需要相对于预测变量数量的 $a\ lot$ 的观测值——即，n 远大于 p。这与 KNN 是非参数方法的事实有关，因此它倾向于减少偏差，同时产生大量方差。

- 在决策边界是非线性但n仅适中，或p不是很小的情形下，QDA可能优于KNN。这是因为QDA能提供非线性决策边界，同时利用参数形式，这意味着相对于KNN，它需要较少的样本量就能实现准确分类。

- 与逻辑回归不同，KNN并不会告诉我们哪些预测变量是重要的：我们无法像表4.3那样获得一个系数表。

4.5.2 An Empirical Comparison

我们现在比较逻辑回归、LDA、QDA、朴素贝叶斯和KNN的*empirical* (实际)性能。我们从六个不同的场景中生成数据，每个场景都涉及一个二分类（两类）问题。在其中三个场景中，贝叶斯决策边界是线性的，而在其余场景中是非线性的。对于每个场景，我们生成了100个随机训练数据集。在每个训练集上，我们将每种方法拟合到数据，并在一个大型测试集上计算得到的测试错误率。线性场景的结果显示在图4.11中，而非线性场景的结果在图4.12中。KNN方法需要选择K，即邻居的数量（不要与本章前面章节中的类别数量混淆）。我们使用两个K值执行KNN：$K = 1$，以及一个通过称为*cross-validation*的方法自动选择的K值，我们将在第5章进一步讨论。我们应用朴素贝叶斯，假设每个类别内特征服从单变量高斯密度（当然——由于这是朴素贝叶斯的关键特征——假设特征独立）。

在六个场景中，每个都包含25个定量预测变量。场景如下：

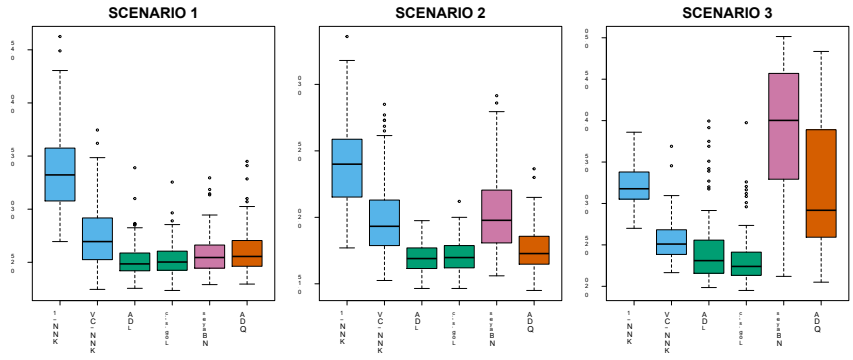


图4.11。*Boxplots of the test error rates for each of the linear scenarios described in the main text.*

Scenario 1: 两个类别中各有20个训练观测值。每个类别内的观测值是不相关的随机正态变量，且每个类别具有不同的均值。图4.11的左侧面板显示，LDA在这种情境下表现良好，正如人们所预期的那样，因为这正是LDA所假设的模型。逻辑回归也表现得相当好，因为它假设了一个线性决策边界。KNN表现不佳，因为它在方差方面付出了代价，而这种代价并未被偏差的减少所抵消。QDA的表现也逊于LDA，因为它拟合了一个比必要更灵活的分类器。朴素贝叶斯的表现略好于QDA，因为朴素贝叶斯关于预测变量独立的假设是正确的。

Scenario 2: 详情同场景 1，不同之处在于每个类内，两个预测变量的相关性为 -0.5。图 4.11 的中间面板表明，大多数方法的性能与前一场景相似。显著的例外是朴素贝叶斯，其在此表现非常差，因为朴素贝叶斯关于预测变量独立的假设被违反了。

Scenario 3: 与前一个场景一样，每个类内的预测变量之间存在显著的负相关。然而，这一次我们是从*t-distribution*生成了X_1和X_2，每个类别有50个观测值。

t-distribution

t-分布的形状与正态分布相似，但其倾向于产生更多极端点——即更多远离均值的点。在此设定下，决策边界仍然是线性的，因此适合逻辑回归框架。该设置违反了LDA的假设，因为观测值并非来自正态分布。图4.11右侧面板显示，逻辑回归优于LDA，尽管两者都优于其他方法。特别是，由于非正态性，QDA结果显著恶化。朴素贝叶斯表现极差，因为独立性假设被违反。

Scenario 4: 数据是从正态分布生成的，第一类预测变量之间的相关性为0.5，第二类预测变量之间的相关性为-0.5。这种设置符合QDA假设，并产生了二次决策边界。图4.12左侧面板显示，QDA性能优于所有

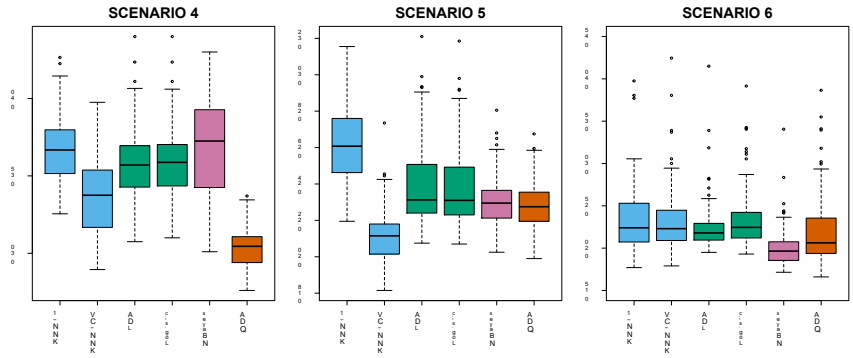


图4.12。*Boxplots of the test error rates for each of the non-linear scenarios described in the main text.*

其他方法。朴素贝叶斯关于独立预测变量的假设被违背，因此朴素贝叶斯表现不佳。

Scenario 5: 数据由正态分布生成，且预测变量间无相关性。随后，响应变量依据应用于预测变量的复杂非线性函数的逻辑斯蒂函数进行采样。图4.12的中间面板显示，QDA和朴素贝叶斯给出的结果略优于线性方法，而灵活性更高的KNN-CV方法则表现最佳。然而，当KNN的K =设为1时，其结果是所有方法中最差的。这强调了一个事实：即便数据展现出复杂的非线性关系，如果平滑程度选择不当，像KNN这样的非参数方法仍可能给出较差的结果。

Scenario 6: 这些观测数据是从正态分布中生成的，每一类都有不同的对角协方差矩阵。然而，样本量非常小：每类仅有n =个，即6个。朴素贝叶斯表现得非常好，因为其假设得到了满足。由于真实的决策边界是非线性的，这是由不相等的协方差矩阵导致的，LDA和逻辑回归表现不佳。QDA的表现略逊于朴素贝叶斯，因为在样本量极小的情况下，前者在估计每类内预测变量之间的相关性时产生了过多的方差。由于样本量非常小，KNN的性能也受到了影响。

这六个例子说明，没有一种方法在所有情况下都能优于其他方法。当真实的决策边界是线性的，那么LDA和逻辑回归方法往往表现良好。当边界适度非线性时，QDA或朴素贝叶斯可能会给出更好的结果。最后，对于复杂得多的决策边界，非参数方法如KNN可以更胜一筹。但是，非参数方法的平滑程度必须谨慎选择。在下一章中，我们将探讨一些方法，用于选择正确的平滑水平，并且总体上选择最佳的方法。

最后，回顾第3章，在回归设置中，我们可以通过使用预测变量的变换进行回归来适应预测变量与响应之间的非线性关系。在分类设置中也可以采用类似的方法。例如，我们可以

	Coefficient	Std. error	t-statistic	p-value
Intercept	73.60	5.13	14.34	0.00
workingday	1.27	1.78	0.71	0.48
temp	157.21	10.26	15.32	0.00
weathersit[cloudy/misty]	-12.89	1.96	-6.56	0.00
weathersit[light rain/snow]	-66.49	2.97	-22.43	0.00
weathersit[heavy rain/snow]	-109.75	76.67	-1.43	0.15

表 4.10. *Results for a least squares linear model fit to predict* bikers *in the* Bikeshare *data. The predictors* mnth *and* hr *are omitted from this table due to space constraints, and can be seen in Figure 4.13. For the qualitative variable* weathersit*, the baseline level corresponds to clear skies.*

通过包含X^2、X^3乃至X^4作为预测变量，创建一个更灵活的 logistic 回归版本。这是否能提升 logistic 回归的性能，取决于因增加灵活性而导致的方差增加是否能被偏差的足够大幅减少所抵消。对于 LDA，我们也可以采取同样的做法。如果我们将所有可能的二次项和交叉乘积加入 LDA，模型的形式将与 QDA 模型相同，尽管参数估计会有所不同。这一手段使我们能够在 LDA 和 QDA 模型之间找到某种中间形态。

4.6 广义线性模型

在第3章中，我们假设响应变量Y是定量的，并探讨了使用最小二乘线性回归来预测Y的方法。而在本章截至目前，我们转而假设Y是定性的。然而，我们有时会遇到Y既非定性也非定量的情况，因此，第3章中的线性回归以及本章所涵盖的分类方法均不适用。

作为一个具体的例子，我们考虑 Bikeshare 数据集。响应变量是 bikers，即华盛顿特区自行车共享计划的每小时用户数。这个响应值既不是定性的也不是定量的：相反，它取非负整数值，即 *counts*。我们将考虑使用协变量 mnth (一年中的月份)、hr (一天中的小时，从0到23)、workingday (一个指示变量，如果既不是周末也不是节假日则等于1)、temp (归一化温度，单位为摄氏度)，以及 weathersit (一个定性变量，其取以下四个可能值之一：晴朗；有雾或多云；小雨或小雪；或大雨或大雪)，来预测 bikers。

counts

在后续分析中，我们将把mnth、hr和weathersit视为定性变量。

4.6.1 Linear Regression on the Bikeshare Data

首先，我们考虑使用线性回归预测bikers。结果如表4.10所示。

例如，我们看到，天气从晴朗转变为多云平均导致每小时减少 12.89 名骑行者；然而，如果天气进一步转为雨或雪，则会进一步导致每小时减少 53.60 名骑行者。图 4.13 展示了与 mnth 相关的系数。

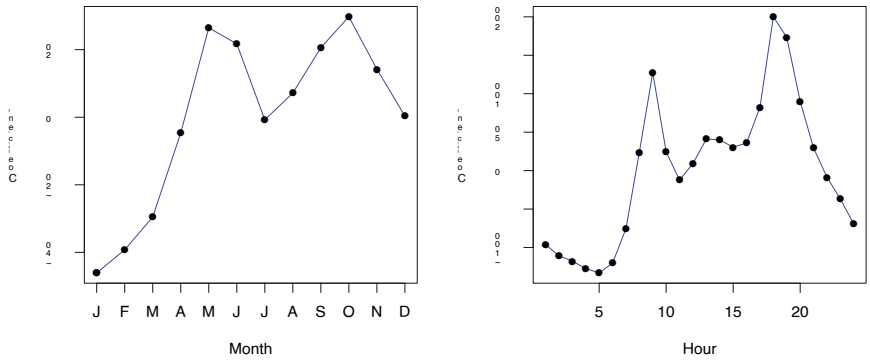


图 4.13. *A least squares linear regression model was fit to predict* `bikers` *in the* `Bikeshare` *data set.* 左图: *The coefficients associated with the month of the year. Bike usage is highest in the spring and fall, and lowest in the winter.* 右图: *The coefficients associated with the hour of the day. Bike usage is highest during peak commute times, and lowest overnight.*

以及与`hr`相关的系数。我们看到自行车使用量在春季和秋季最高，在冬季月份最低。此外，自行车使用量在高峰时段（上午9点和下午6点）最高，在夜间最低。因此，乍一看，对`Bikeshare`数据集拟合线性回归模型似乎提供了合理且直观的结果。

但经过更仔细的检查，一些问题逐渐显现。例如，在`Bikeshare`数据集中，有9.6%的拟合值为负数：即线性回归模型预测在数据集中的9.6%的小时内，用户数量为*negative*。这引发了我们对该数据进行有意义预测的能力的质疑，并且也对回归模型的系数估计、置信区间及其他输出的准确性提出了担忧。

此外，有理由怀疑，当`bikers`的期望值较小时，`bikers`的方差也应较小。例如，在12月一场大雪暴的凌晨2点，我们预期使用自行车的人数极少，而且在这些条件下，用户数量的方差也应该很小。数据证实了这一点：在12月、1月和2月的凌晨1点至4点之间，下雨时，平均有5.05名用户，标准差为3.73。相比之下，在4月、5月和6月的早上7点至10点之间，天气晴朗时，平均有243.59名用户，标准差为131.7。均值-方差关系如图4.14左侧面板所示。这是对线性模型假设的重大违背，线性模型假设$Y = \sum_{j=1}^{p} X_j \beta_j + \epsilon$，其中$\epsilon$是均值为零的误差项，方差为$\sigma^2$，即*constant*，且不是协变量的函数。因此，数据的异方差性对线性回归模型的适用性提出了质疑。

最后，响应变量 {v*} 是整数值。但在线性模型下，{v*}，其中 {v*} 是连续值误差项。这意味着在线性模型中，响应变量 {v*} 必然是连续值（定量）的。因此，响应变量 {v*} 的整数性质表明，线性回归模型对这个数据集并不完全令人满意。

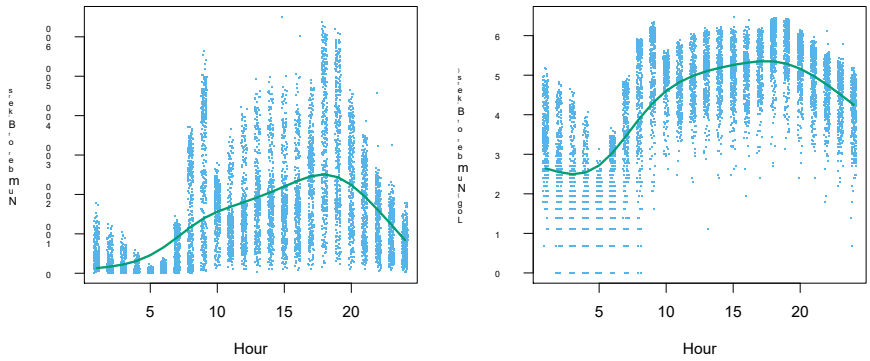


图4.14. 左：*On the* `Bikeshare` *dataset, the number of bikers is displayed on the y-axis, and the hour of the day is displayed on the x-axis. Jitter was applied for ease of visualization. For the most part, as the mean number of bikers increases, so does the variance in the number of bikers. A smoothing spline fit is shown in green.* 右：*The log of the number of bikers is now displayed on the y-axis.*

将线性回归模型拟合到`Bikeshare`数据时出现的一些问题可以通过变换响应变量来克服；例如，我们可以拟合模型

$$\log(Y) = \sum_{j=1}^{p} X_j \beta_j + \epsilon.$$

对响应变量进行变换避免了出现负预测值的可能性，并且克服了未变换数据中大部分的异方差性，如图4.14右侧面板所示。然而，这并非一个完全令人满意的解决方案，因为预测和推断是基于响应的对数而非响应本身进行的。这给解释带来了挑战，例如
"a one-unit increase in X_j *is associated with an increase in the mean of the log of* Y *by an amount* β_j*"*。此外，当响应可以取值为0时，无法对响应应用对数变换。因此，尽管对响应的变换拟合线性模型对于某些计数型数据集可能是足够的方法，但它往往不尽如人意。在下一节中我们将看到，泊松回归模型为这一任务提供了一种更加自然和优雅的方法。

4.6.2 Poisson Regression on the Bikeshare Data



为了克服线性回归在分析 `Bikeshare` 数据集时的不足，我们将采用一种替代方法，称为 *Poisson regression*。在讨论泊松回归之前，我们必须首先介引入 *Poisson distribution*。

泊松回归

泊松分布

假设一个随机变量 Y 取非负整数值，即 $Y \in \{0, 1, 2, \ldots\}$。如果 Y 服从泊松分布，那么

$$\Pr(Y = k) = \frac{e^{-\lambda}\lambda^k}{k!} \quad \text{for } k = 0, 1, 2, \ldots. \tag{4.35}$$

这里，$\lambda > 0$ 是 Y的期望值，即 E(Y)。事实证明，λ 也等于 Y的方差，即 $\lambda = \text{E}(Y) = \text{Var}(Y)$。这意味着如果 Y服从泊松分布，那么 Y的均值越大，其方差就越大。（在 (4.35) 中，符号 $k!$，读作“k 阶乘”，定义为 $k! = k \times (k-1) \times (k-2) \times \ldots \times 3 \times 2 \times 1$。）

泊松分布通常用于建模 *counts*；这是自然的选择，原因有很多，包括计数像泊松分布一样取非负整数值。为了了解如何在实际中应用泊松分布，令 Y表示在一天中的特定小时、特定天气条件下以及一年中的特定月份内，共享单车项目的用户数量。我们可以将 Y建模为均值为 $\text{E}(Y) = \lambda = 5$ 的泊松分布。这意味着在这个特定小时内没有用户的概率是 $\Pr(Y = 0) = \frac{e^{-5}5^0}{0!} = e^{-5} = 0.0067$（其中按照惯例 $0! = 1$）。恰好有一个用户的概率是 $\Pr(Y = 1) = \frac{e^{-5}5^1}{1!} = 5e^{-5} = 0.034$，两个用户的概率是 $\Pr(Y = 2) = \frac{e^{-5}5^2}{2!} = 0.084$，依此类推。

当然，在现实中，我们预期共享单车项目的用户平均数量$\lambda = \text{E}(Y)$会随着一天中的时段、一年中的月份、天气条件等因素的变化而变化。因此，与其将骑行者数量Y建模为具有固定均值如$\lambda = 5$的泊松分布，我们希望让均值作为协变量的函数而变化。特别地，我们考虑以下关于均值$\lambda = \text{E}(Y)$的模型，现在我们将其写作$\lambda(X_1, \ldots, X_p)$，以强调它是协变量$X_1, \ldots, X_p$的函数：

$$\log(\lambda(X_1, \ldots, X_p)) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p \tag{4.36}$$

或等价地

$$\lambda(X_1, \ldots, X_p) = e^{\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p}. \tag{4.37}$$

这里，$\beta_0, \beta_1, \ldots, \beta_p$ 是待估参数。(4.35) 和 (4.36) 共同定义了泊松回归模型。注意在 (4.36) 中，我们将 $\lambda(X_1, \ldots, X_p)$ 的 *log* 设为 $X_1, \ldots, X_p$ 的线性函数，而非让 $\lambda(X_1, \ldots, X_p)$ 本身是 $X_1, \ldots, X_p$ 的线性函数；这确保了 $\lambda(X_1, \ldots, X_p)$ 对于协变量的所有取值都取非负值。

为了估计系数 $\beta_0, \beta_1, \ldots, \beta_p$，我们采用与第4.3.2节中逻辑回归相同最大似然估计方法。具体而言，给定来自泊松回归模型的 n 个独立观测值，似然函数形式为

$$\ell(\beta_0, \beta_1, \ldots, \beta_p) = \prod_{i=1}^{n} \frac{e^{-\lambda(x_i)} \lambda(x_i)^{y_i}}{y_i!}, \tag{4.38}$$

其中 $\lambda(x_i) = e^{\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}}$，由于 (4.37)。我们估计使似然 $\ell(\beta_0, \beta_1, \ldots, \beta_p)$ 最大化的系数，即让观测数据尽可能可能的系数。

我们现在对 `Bikeshare` 数据集拟合一个泊松回归模型。结果如表 4.11 和图 4.15 所示。从定性上看，结果与第 4.6.1 节中线性回归的结果相似。我们再次看到，自行车使用量在春季和秋季以及高峰时段最高，

	Coefficient	Std. error	z-statistic	p-value
`Intercept`	4.12	0.01	683.96	0.00
`workingday`	0.01	0.00	7.5	0.00
`temp`	0.79	0.01	68.43	0.00
`weathersit[cloudy/misty]`	-0.08	0.00	-34.53	0.00
`weathersit[light rain/snow]`	-0.58	0.00	-141.91	0.00
`weathersit[heavy rain/snow]`	-0.93	0.17	-5.55	0.00

表4.11. *Results for a Poisson regression model fit to predict* `bikers` *in the* `Bikeshare` *data. The predictors* `mnth` *and* `hr` *are omitted from this table due to space constraints, and can be seen in Figure 4.15. For the qualitative variable* `weathersit`*, the baseline corresponds to clear skies.*

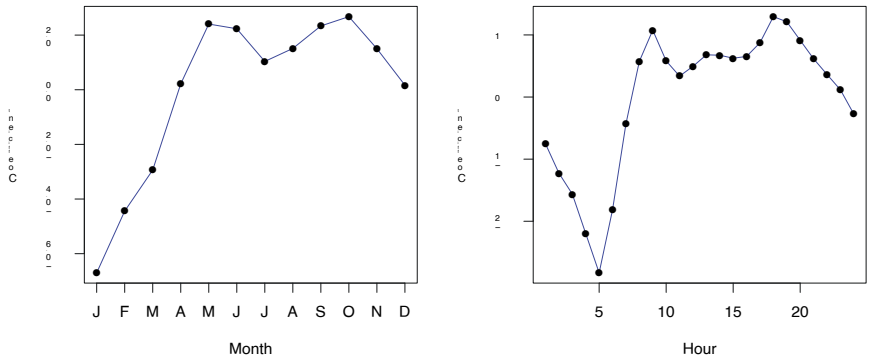


图4.15。 *A Poisson regression model was fit to predict* `bikers` *in the* `Bikeshare` *data set.* 左： *The coefficients associated with the month of the year. Bike usage is highest in the spring and fall, and lowest in the winter.* 右： *The coefficients associated with the hour of the day. Bike usage is highest during peak commute times, and lowest overnight.*

并且在冬季和清晨时段最低。此外，自行车使用量随着温度升高而增加，随着天气恶化而减少。有趣的是，与`workingday`相关的系数在泊松回归模型下具有统计学显著性，但在线性回归模型下则不显著。

泊松回归模型与线性回归模型之间的一些重要区别如下：

- *Interpretation:* 为了解释泊松回归模型中的系数，我们必须密切关注(4.37)式，该式表明X_j每增加一个单位，会伴随$\mathrm{E}(Y) = \lambda$以$\exp(\beta_j)$的因子变化。例如，天气从晴朗转为多云时，平均自行车使用量会以$\exp(-0.08) = 0.923$的因子变化，即平均而言，多云时使用自行车的人数仅为晴朗时的92.3%。如果天气进一步恶化并开始下雨，那么平均自行车使用量将再以$\exp(-0.5) = 0.607$的因子变化，即平均而言，雨天使用自行车的人数仅为多云时的60.7%。

- *Mean-variance relationship:* 如前所述，在泊松模型下，$\lambda = \mathrm{E}(Y) = \mathrm{Var}(Y)$。因此，通过使用泊松回归对自行车使用量进行建模，我们隐含地假设某一给定小时内的平均自行车使用量等于该小时内自行车使用量的方差。相比之下，在线性回归模型下，自行车使用量的方差始终取一个常数值。回想图4.14，在Bikeshare数据中，当骑行条件有利时，自行车使用量的均值*and*方差都远高于条件不利时。因此，泊松回归模型能够处理Bikeshare数据中呈现的均值-方差关系，而线性回归模型则无法做到。[5]

overdispersion

- *nonnegative fitted values:* 使用泊松回归模型不会出现负预测值。这是因为泊松模型本身只允许非负值；见(4.35)。相比之下，当我们将线性回归模型拟合到 Bikeshare 数据集时，几乎10%的预测值为负。

4.6.3 Generalized Linear Models in Greater Generality



到目前为止，我们已经讨论了三种类型的回归模型：线性、逻辑和泊松。这些方法具有一些共同特征：

1. 每种方法都使用预测因子 $X_1, \ldots, X_p$ 来预测响应 Y。我们假设，在给定 $X_1, \ldots, X_p$ 的条件下，Y 属于某一类分布。对于线性回归，我们通常假设 Y 服从高斯分布或正态分布。对于逻辑回归，我们假设 Y 服从伯努利分布。最后，对于泊松回归，我们假设 Y 服从泊松分布。

2. 每种方法都将 Y 的均值建模为预测变量的函数。在线性回归中，Y 的均值形式为

$$\mathrm{E}(Y|X_1, \ldots, X_p) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p, \tag{4.39}$$

即它是预测变量的线性函数。对于逻辑回归，均值则采用以下形式

$$\begin{aligned} \mathrm{E}(Y|X_1, \ldots, X_p) &= \Pr(Y = 1|X_1, \ldots, X_p) \\ &= \frac{e^{\beta_0+\beta_1 X_1+\cdots+\beta_p X_p}}{1 + e^{\beta_0+\beta_1 X_1+\cdots+\beta_p X_p}}, \end{aligned} \tag{4.40}$$

而对于泊松回归，其形式为

$$\mathrm{E}(Y|X_1, \ldots, X_p) = \lambda(X_1, \ldots, X_p) = e^{\beta_0+\beta_1 X_1+\cdots+\beta_p X_p}. \tag{4.41}$$

方程（4.39）至（4.41）可以用 *link function*、η 表示，其中

link function

[5] In fact, the variance in the Bikeshare data appears to be much higher than the mean, a situation referred to as *overdispersion*. This causes the Z-values to be inflated in Table 4.11. A more careful analysis should account for this overdispersion to obtain more accurate Z-values, and there are a variety of methods for doing this. But they are beyond the scope of this book.

对 $\mathrm{E}(Y|X_1,\ldots,X_p)$ 应用变换，使得变换后的均值是预测变量的线性函数。即，

$$\eta(\mathrm{E}(Y|X_1,\ldots,X_p)) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p. \tag{4.42}$$

线性回归、逻辑回归和泊松回归的链接函数分别为$\eta(\mu) = \mu$、$\eta(\mu) = \log(\mu/(1-\mu))$ 和 $\eta(\mu) = \log(\mu)$。

高斯分布、伯努利分布和泊松分布都属于更广泛的一类分布，称为*exponential family*。其他该家族中已知的成员有*exponential*分布，*Gamma*分布，以及*negative binomial*分布。一般来说，我们可以通过将响应 Y 建模为来自指数族特定成员，然后变换响应的均值，使得变换后的均值通过 (4.42) 成为预测变量的线性函数，从而形成回归。任何遵循这一非常通用方法的回归方法被称为*generalized linear model* (GLM)。因此，线性回归、逻辑回归和泊松回归是GLM的三个例子。此处未涵盖的其他例子包括*Gamma regression*和*negative binomial regression*。

指数族

指数 Gamma

负二项式

generalized linear model

4.7 实验：逻辑回归、LDA、QDA 和 KNN

4.7.1 The Stock Market Data

在本实验中，我们将研究`Smarket`数据，该数据是`ISLP`库的一部分。该数据集包含2001年初至2005年底期间，标普500股票指数在1,250天内的百分比回报率。对于每个日期，我们记录了前五个交易日中每一天的百分比回报率，即`Lag1`到`Lag5`。我们还记录了`Volume`(前一日交易的股票数量，以十亿为单位)，`Today` (所讨论日期的百分比回报率)，以及`Direction` (该日期市场是`Up`还是`Down`)。

我们首先在顶层导入我们的库；这些都是我们在之前的实验中见过的导入。

In [1]:
```
import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
                         summarize)
```

我们还汇总了本实验所需的新导入。

In [2]:
```
from ISLP import confusion_table
from ISLP.models import contrast
from sklearn.discriminant_analysis import \
     (LinearDiscriminantAnalysis as LDA,
      QuadraticDiscriminantAnalysis as QDA)
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

现在我们准备好加载 Smarket 数据。

In [3]:
```
Smarket = load_data('Smarket')
Smarket
```

这给出了数据的截断列表，我们在此不展示。我们可以看到变量名是什么。

In [4]:
```
Smarket.columns
```

Out[4]:
```
Index(['Year', 'Lag1', 'Lag2', 'Lag3', 'Lag4', 'Lag5', 'Volume',
       'Today', 'Direction'],
      dtype='object')
```

我们使用 corr() 方法计算数据 帧的相关矩阵，这会生成一个包含变量间所有成对相关性的矩阵。（我们在此省略输出。）pandas 库不报告 Direction 变量的相关性，因为它是一个定性变量。

corr()

In [5]:
```
Smarket.corr()
```

正如预期的那样，滞后收益率变量与今日收益率之间的相关性接近于零。唯一显著的相关性是 Year 与 Volume 之间。通过绘制数据图，我们看到 Volume 随时间递增。换言之，从2001年到2005年，日均交易股票数量有所增加。

In [6]:
```
Smarket.plot(y='Volume');
```

4.7.2 Logistic Regression

接下来，我们将使用Lag1至Lag5以及Volume来拟合一个逻辑回归模型以预测Direction。sm.GLM()函数拟合*generalized linear models*，一类包含逻辑回归的模型。另外，函数 sm.Logit() 直接拟合逻辑回归模型。sm.GLM() 的语法与 sm.OLS() 类似，不同之处在于我们必须传入参数 family=sm.families.Binomial()，以告知 statsmodels 运行逻辑回归，而不是其他类型的广义线性模型。

sm.GLM()
广义线性模型

In [7]:
```
allvars = Smarket.columns.drop(['Today', 'Direction', 'Year'])
design = MS(allvars)
X = design.fit_transform(Smarket)
y = Smarket.Direction == 'Up'
glm = sm.GLM(y,
             X,
             family=sm.families.Binomial())
results = glm.fit()
summarize(results)
```

```
Out[7]:              coef     std err         z     P>|z|
       intercept -0.1260       0.241    -0.523     0.601
       Lag1      -0.0731       0.050    -1.457     0.145
       Lag2      -0.0423       0.050    -0.845     0.398
       Lag3       0.0111       0.050     0.222     0.824
       Lag4       0.0094       0.050     0.187     0.851
       Lag5       0.0103       0.050     0.208     0.835
       Volume     0.1354       0.158     0.855     0.392
```

这里最小的 p 值与 Lag1 相关。该预测变量的负系数表明，如果昨天市场有正回报，那么今天上涨的可能性较小。然而，在 0.15 的值下，p 值仍然相对较大，因此没有明确证据表明 Lag1 与 Direction 之间存在真实关联。

我们使用results的params属性来仅访问此拟合模型的系数。

```
In [8]: results.params
```

```
Out[8]: intercept   -0.126000
        Lag1        -0.073074
        Lag2        -0.042301
        Lag3         0.011085
        Lag4         0.009359
        Lag5         0.010313
        Volume       0.135441
        dtype: float64
```

同样，我们可以使用pvalues属性来访问系数的p值（未显示）。

```
In [9]: results.pvalues
```

results 的 predict() 方法可用于在给定预测变量值的情况下预测市场上涨的概率。该方法返回概率尺度上的预测值。如果没有向 predict() 函数提供数据集，则针对用于拟合逻辑回归模型的训练数据计算概率。与线性回归一样，如果需要，可以传入与设计矩阵一致的可选 exog 参数。这里我们只打印了前十个概率。

```
In [10]: probs = results.predict()
         probs[:10]
```

```
Out[10]: array([0.5070841, 0.4814679, 0.4811388, 0.5152223, 0.5107812,
                0.5069565, 0.4926509, 0.5092292, 0.5176135, 0.4888378])
```

为了预测市场在某一天会上涨还是下跌，我们必须将这些预测概率转换为类别标签，Up 或 Down。以下两个命令基于市场上涨的预测概率是大于还是小于0.5来创建一个类别预测向量。

```
In [11]: labels = np.array(['Down']*1250)
         labels[probs>0.5] = "Up"
```

来自 ISLP 包的 confusion_table() 函数总结了这些预测结果，显示了有多少观测被正确或错误分类。我们的函数，改编自模块 sklearn.metrics 中的一个类似函数，转置结果矩阵并包含行和列标签。confusion_table() 函数以预测标签作为第一个参数，真实标签作为第二个参数。

confusion_table()

```
In [12]: confusion_table(labels, Smarket.Direction)
```

```
Out[12]:     Truth  Down   Up
         Predicted
              Down   145  141
                Up   457  507
```

混淆矩阵的对角线元素表示正确预测，而非对角线元素表示错误预测。因此，我们的模型正确预测了市场在507天会上涨，在145天会下跌，总共507 + 145 = 652次正确预测。np.mean() 函数可用于计算预测正确的天数比例。在这种情况下，逻辑回归正确预测了市场走势的时间比例为52.2%。

```
In [13]: (507+145)/1250, np.mean(labels == Smarket.Direction)
```

```
Out[13]: (0.5216, 0.5216)
```

乍看之下，逻辑回归模型的表现似乎略优于随机猜测。然而，这一结果具有误导性，因为我们是在同一组包含1250个观测值的数据上训练并测试了模型。换言之，100 − 52.2 = 47.8%即为*training*错误率。正如我们之前所见，训练错误率往往过于乐观——它倾向于低估测试错误率。为了更准确地评估逻辑回归模型在此情境下的准确性，我们可以利用部分数据拟合模型，然后检验其对*held out*数据的预测效果。这将得出一个更为贴近现实的错误率，因为在实践中，我们关注的是模型对未来市场走势未知日子的预测性能，而非对用于拟合模型的数据的表现。

为了实现这一策略，我们首先创建一个与2001年至2004年观测数据相对应的布尔向量。然后，利用这个向量构建一个包含2005年观测数据的保留数据集。

```
In [14]: train = (Smarket.Year < 2005)
         Smarket_train = Smarket.loc[train]
         Smarket_test = Smarket.loc[~train]
         Smarket_test.shape
```

```
Out[14]: (252, 9)
```

对象 train 是一个包含 1,250 个元素的向量，对应于我们数据集中的观测值。该向量中对应于 2005 年之前发生的观测值的元素被设置为 True，而对应于 2005 年观测值的元素被设置为 False。因此，train 是一个 *boolean* 数组，因为其元素为 True 和 False。布尔数组可用于使用以下方法获取数据框的行或列的子集：

loc 方法。例如，命令Smarket.loc[train]会选取股票市场数据集的一个子矩阵，仅对应于2005年之前的日期，因为这些日期是train的元素为True的那些日期。符号~可用于否定布尔向量中的所有元素。也就是说，~train是一个与train相似的向量，只是在train中为True的元素在~train中被交换为False，反之亦然。因此，Smarket.loc[~train]产生股票市场数据框的行的一个子集，仅包含那些train为False的观测。上面的输出表明有252个这样的观测。

我们现在仅使用对应于2005年之前日期的观测子集来拟合逻辑回归模型。然后，我们获得测试集中每一天——即2005年的每一天——股市上涨的预测概率。

In [15]:
```
X_train, X_test = X.loc[train], X.loc[~train]
y_train, y_test = y.loc[train], y.loc[~train]
glm_train = sm.GLM(y_train,
                   X_train,
                   family=sm.families.Binomial())
results = glm_train.fit()
probs = results.predict(exog=X_test)
```

请注意，我们是在两个完全独立的数据集上训练和测试模型的：训练仅使用2005年之前的日期进行，而测试仅使用2005年的日期进行。

最后，我们将2005年的预测与该时间段内市场的实际走势进行比较。我们将首先存储测试和训练标签（回想一下，y_test是二进制的）。

In [16]:
```
D = Smarket.Direction
L_train, L_test = D.loc[train], D.loc[~train]
```

现在我们将拟合概率的阈值设为50%，以形成我们的预测标签。

In [17]:
```
labels = np.array(['Down']*252)
labels[probs>0.5] = 'Up'
confusion_table(labels, L_test)
```

Out[17]:

Truth	Down	Up
Predicted		
Down	77	97
Up	34	44

Th 测试准确率约为48%，而错误率约为 t 52%

In [18]:
```
np.mean(labels == L_test), np.mean(labels != L_test)
```

Out[18]: (0.4802, 0.5198)

!= 表示 *not equal to*，因此最后一条命令计算的是 != 测试集错误率。结果相当令人失望：测试错误率为 52%，比随机猜测还要差！当然，这一结果并不十分令人惊讶，因为人们通常不会期望能够用前几天的收益率来预测未来的市场表现。（毕竟，如果这真的可行，那么本书的作者们早就去发大财了，而不是在写统计学教科书。）

我们记得，逻辑回归模型与所有预测变量相关联的{p}值都非常不理想，而且最小的{p}值虽然不算很小，却对应于Lag1。也许通过移除那些在预测Direction时似乎没有帮助的变量，我们能获得一个更有效的模型。毕竟，使用与响应无关的预测变量往往会导致测试错误率的恶化（因为这类预测变量会导致方差增加，而没有相应的偏差减少），因此移除这类预测变量可能会带来改进。下面我们仅使用Lag1和Lag2重新拟合逻辑回归，这两个变量在原始逻辑回归模型中似乎具有最高的预测能力。

In [19]:
```
model = MS(['Lag1', 'Lag2']).fit(Smarket)
X = model.transform(Smarket)
X_train, X_test = X.loc[train], X.loc[~train]
glm_train = sm.GLM(y_train,
                   X_train,
                   family=sm.families.Binomial())
results = glm_train.fit()
probs = results.predict(exog=X_test)
labels = np.array(['Down']*252)
labels[probs>0.5] = 'Up'
confusion_table(labels, L_test)
```

Out[19]:

Truth	Down	Up
Predicted		
Down	35	35
Up	76	106

让我们评估整体准确率以及逻辑回归预测增长的那些天内的准确率。

In [20]:
```
(35+106)/252,106/(106+76)
```

Out[20]: (0.5595, 0.5824)

现在结果看起来稍好一些：56%的每日波动被正确预测了。值得注意的是，在这种情况下，一个更简单的策略——即每天都预测市场会上涨——同样有56%的时间是正确的！因此，从总体错误率来看，逻辑回归方法并不比这种朴素方法更好。然而，混淆矩阵显示，在逻辑回归预测市场上涨的日子里，其准确率达到了58%。这表明可能存在一种交易策略，即在模型预测市场将上涨的日子买入，并在预测下跌的日子避免交易。当然，需要更仔细地研究这种微小改进是真实的，还是仅仅由于随机机会所致。

假设我们希望预测与 Lag1 和 Lag2 的特定值相关联的回报。特别地，我们希望在 Lag1 和 Lag2 分别等于 1.2 和 1.1 的那一天，以及在它们分别等于 1.5 和 -0.8 的那一天预测 Direction。我们使用 predict() 函数来完成这项工作。

In [21]:
```
newdata = pd.DataFrame({'Lag1':[1.2, 1.5],
                        'Lag2':[1.1, -0.8]});
```

```
newX = model.transform(newdata)
results.predict(newX)
```

```
Out[21]: 0    0.4791
         1    0.4961
         dtype: float64
```

4.7.3 Linear Discriminant Analysis

我们首先使用函数 LinearDiscriminantAnalysis() 对 Smarket 数据进行 LDA，我们将其缩写为 LDA()。我们拟合仅使用2005年之前观测数据的模型。

LinearDiscriminantAnalysis()

```
In [22]: lda = LDA(store_covariance=True)
```

由于LDA估计器自动添加截距，我们应移除X_train和X_test中对应于截距的列。我们也可以直接使用标签，而不是布尔向量y_train。

```
In [23]: X_train, X_test = [M.drop(columns=['intercept'])
                            for M in [X_train, X_test]]
         lda.fit(X_train, L_train)
```

```
Out[23]: LinearDiscriminantAnalysis(store_covariance=True)
```

这里我们使用了第3.6.4节介绍的列表推导式。观察上面的第一行，我们可以看到右侧是一个长度为二的列表。这是因为代码
for M in [X_train, X_test]遍历了一个长度为二的列表。虽然这里我们循环遍历的是一个列表，但列表推导方法适用于循环遍历任何可迭代对象。然后我们对迭代中的每个元素应用drop()方法，收集结果

.drop()

在列表中。左侧告诉Python解包这个长度为二的列表，将其元素分别赋值给变量X_train和X_test。当然，这会覆盖X_train和X_test原来的值。

拟合模型后，我们可以利用means_属性提取两个类别中的均值。这些均值是每个类别内各预测变量的平均值，被LDA用作μ_k的估计。这些结果表明，在市场上涨的日子里，前两天的回报有负向趋势；而在市场下跌的日子里，前几天的回报有正向趋势。

```
In [24]: lda.means_
```

```
Out[24]: array([[ 0.04,  0.03],
                [-0.04, -0.03]])
```

估计的先验概率存储在 priors_ 属性中。包 sklearn 通常使用尾随的 _ 来表示使用 fit() 方法估计的量。通过查看 classes_ 属性，我们可以确定哪个条目对应于哪个标签。

```
In [25]: lda.classes_
```

```
Out[25]: array(['Down', 'Up'], dtype='<U4')
```

LDA 输出表明 $\hat{\pi}_{\texttt{Down}} = 0.492$ 和 $\hat{\pi}_{\texttt{Up}} =$ 0.508.

In [26]:
```
lda.priors_
```

Out[26]:
```
array([0.492, 0.508])
```

线性判别向量可以在 scalings_ 属性中找到:

In [27]:
```
lda.scalings_
```

Out[27]:
```
array([[-0.642],
       [-0.513]])
```

这些值提供了Lag1和Lag2的线性组合，用于形成LDA决策规则。换言之，这些是(4.24)式中$X = x$元素的乘数。如果$-0.64 \times \texttt{Lag1} - 0.51 \times \texttt{Lag2}$较大，则LDA分类器将预测市场上涨；如果较小，则LDA分类器将预测市场下跌。

In [28]:
```
lda_pred = lda.predict(X_test)
```

正如我们在分类方法比较中（第4.5节）所观察到的，LDA和逻辑回归的预测几乎是相同的。

In [29]:
```
confusion_table(lda_pred, L_test)
```

Out[29]:
```
Truth      Down   Up
Predicted
Down         35   35
Up           76  106
```

我们还可以估计训练集中每个点属于每个类别的概率。对属于第一类的后验概率应用50%的阈值，即可重现lda_pred中包含的预测。

In [30]:
```
lda_prob = lda.predict_proba(X_test)
np.all(
       np.where(lda_prob[:,1] >= 0.5, 'Up','Down') == lda_pred
       )
```

Out[30]:
```
True
```

上面，我们使用了np.where()函数，该函数创建一个数组，对于lda_prob (中第二列——即'Up')的估计后验概率——大于0.5的索引，其值为'Up'。对于超过两个类别的问题，标签选择为后验概率最高的类别:

np.where()

In [31]:
```
np.all(
       [lda.classes_[i] for i in np.argmax(lda_prob, 1)] ==
           lda_pred
       )
```

Out[31]:
```
True
```

如果我们想要使用除了50%以外的后验概率阈值来进行预测，那么我们可以轻松做到。例如，假设我们仅在我们非常确定市场会下跌时才预测其下跌。

市场在那一天确实会下跌——比如，如果后验概率至少为90%。我们知道，在检查了classes_属性后，lda_prob的第一列对应于标签Down，因此我们使用列索引0而不是如上所述的1。

```
In [32]: np.sum(lda_prob[:,0] > 0.9)
```

```
Out[32]: 0
```

2005年没有一天达到这个阈值！事实上，2005年全年下降的最大后验概率为52.02%。

上面的LDA分类器是sklearn库中的第一个分类器。我们将使用该库中的其他几个对象。这些对象遵循一种通用结构，简化了诸如交叉验证之类的任务，我们将在第5章中看到这一点。具体来说，这些方法首先创建一个不引用任何数据的通用分类器。然后使用fit()方法将该分类器拟合到数据，并且始终使用predict()方法生成预测。这种先实例化分类器、随后拟合它、然后生成预测的模式是sklearn的一个明确设计选择。这种统一性使得可以干净地复制分类器，以便它可以拟合不同的数据；例如，交叉验证中出现的不同训练集。这种标准模式还允许可预测地形成工作流。

4.7.4 Quadratic Discriminant Analysis

现在我们将拟合一个QDA模型到Smarket数据。QDA在sklearn包中通过QuadraticDiscriminantAnalysis()实现，我们 abbreviate 简写为 QDA()。语法与 LDA() 非常相似。

Quadratic Discriminant Analysis()

```
In [33]: qda = QDA(store_covariance=True)
         qda.fit(X_train, L_train)
```

```
Out[33]: QuadraticDiscriminantAnalysis(store_covariance=True)
```

QDA() 函数将再次计算 means_ 和 priors_。

```
In [34]: qda.means_, qda.priors_
```

```
Out[34]: (array([[ 0.04279022,  0.03389409],
                 [-0.03954635, -0.03132544]]),
          array([0.49198397, 0.50801603]))
```

QDA() 分类器将为每个类别估计一个协方差。以下是第一个类别中估计的协方差：

```
In [35]: qda.covariance_[0]
```

```
Out[35]: array([[ 1.50662277, -0.03924806],
                [-0.03924806,  1.53559498]])
```

输出包含组均值。但它不包含线性判别式的系数，因为QDA分类器涉及预测变量的二次函数，而不是线性函数。predict()函数的工作方式与LDA完全相同。

```
In [36]: qda_pred = qda.predict(X_test)
         confusion_table(qda_pred, L_test)
```

```
Out[36]:     Truth   Down   Up
         Predicted
              Down     30   20
                Up     81  121
```

有趣的是，尽管2005年的数据并未用于拟合模型，QDA预测的准确率仍接近60%。

```
In [37]: np.mean(qda_pred == L_test)
```

```
Out[37]: 0.599
```

对于股票市场数据而言，这种准确度相当令人印象深刻，因为众所周知，股市数据很难精确建模。这表明，QDA 所假设的二次形式可能比 LDA 和逻辑回归所假设的线性形式更准确地捕捉到了真实关系。然而，我们建议在更大的测试集上评估该方法的性能，然后再押注这种方法将始终跑赢市场！

4.7.5 Naive Bayes

接下来，我们为 Smarket 数据拟合一个朴素贝叶斯模型。语法与 LDA() 和 QDA() 类似。默认情况下，这一实现 GaussianNB() 的朴素贝叶斯分类器使用高斯分布对每个定量特征建模。然而，也可以使用核密度方法来估计分布。

GaussianNB()

```
In [38]: NB = GaussianNB()
         NB.fit(X_train, L_train)
```

```
Out[38]: GaussianNB()
```

这些类别存储为classes_。

```
In [39]: NB.classes_
```

```
Out[39]: array(['Down', 'Up'], dtype='<U4')
```

类先验概率存储在class_prior_属性中。

```
In [40]: NB.class_prior_
```

```
Out[40]: array([0.49, 0.51])
```

特征参数可以在theta_和var_属性中找到。行数等于类别数，而列数等于特征数。下面我们看到，在Down类别中，特征Lag1的均值为0.043。

```
In [41]: NB.theta_
```

```
Out[41]: array([[ 0.043,  0.034],
                [-0.040, -0.031]])
```

其方差为1.503。

```
In [42]: NB.var_
```

```
Out[42]: array([[1.503, 1.532],
                [1.514, 1.487]])
```

我们如何知道这些属性的名称？我们使用 `NB?` (或 `?NB`)。我们可以轻松验证均值计算：

```
In [43]: X_train[L_train == 'Down'].mean()
```

```
Out[43]: Lag1    0.042790
         Lag2    0.033894
         dtype: float64
```

方差同理：

```
In [44]: X_train[L_train == 'Down'].var(ddof=0)
```

```
Out[44]: Lag1    1.503554
         Lag2    1.532467
         dtype: float64
```

`GaussianNB()` 函数使用 $1/n$ 公式计算方差。[6] 由于 `NB()` 是 `sklearn` 库中的分类器，因此进行预测所使用的语法与上文的 `LDA()` 和 `QDA()` 相同。

```
In [45]: nb_labels = NB.predict(X_test)
         confusion_table(nb_labels, L_test)
```

```
Out[45]:     Truth   Down   Up
         Predicted
              Down     29   20
                Up     82  121
```

朴素贝叶斯在这些数据上表现良好，准确预测率超过59%。这比QDA略差，但比LDA好得多。

关于 `LDA`，`predict_proba()` 方法估计每个观测属于特定类别的概率。

```
In [46]: NB.predict_proba(X_test)[:5]
```

```
Out[46]: array([[0.4873, 0.5127],
                [0.4762, 0.5238],
                [0.4653, 0.5347],
                [0.4748, 0.5252],
                [0.4902, 0.5098]])
```

4.7.6 K-Nearest Neighbors

我们现在将使用`KNeighborsClassifier()`函数执行KNN。这

`KNeighbors Classifier()`

[6] There are two formulas for computing the sample variance of n observations $x_1, \ldots, x_n$: $\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$ and $\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$ where $\bar{x}$ is the sample mean. In most cases the distinction is not important.

该函数的工作原理与我们到目前为止遇到的其他模型拟合函数类似。

与 LDA 和 QDA 的情况一样，我们使用 `fit` 方法拟合分类器。新预测是通过 `fit()` 返回对象的 `predict` 方法形成的。

In [47]:
```
knn1 = KNeighborsClassifier(n_neighbors=1)
knn1.fit(X_train, L_train)
knn1_pred = knn1.predict(X_test)
confusion_table(knn1_pred, L_test)
```

Out[47]:
```
Truth   Down  Up
Predicted
Down      43  58
Up        68  83
```

使用 $K = 1$ 的结果不是很好，因为只有 50% 的观测值被正确预测。当然，也可能是 $K = 1$ 导致了对数据的过度灵活拟合。

In [48]:
```
(83+43)/252, np.mean(knn1_pred == L_test)
```

Out[48]:
```
(0.5, 0.5)
```

We repeat the analysis below using $K = 3$.

In [49]:
```
knn3 = KNeighborsClassifier(n_neighbors=3)
knn3_pred = knn3.fit(X_train, L_train).predict(X_test)
np.mean(knn3_pred == L_test)
```

Out[49]:
```
0.532
```

结果略有改善。但进一步增加K并未带来更多改进。看来对于这些数据以及这种训练/测试划分，QDA在我们迄今为止所考察的方法中给出了最佳结果。

KNN 在 `Smarket` 数据上表现不佳，但它通常确实能提供令人印象深刻的结果。作为一个例子，我们将把 KNN 方法应用于 `Caravan` 数据集，该数据集是 `ISLP` 库的一部分。此数据集包含 85 个预测变量，用于衡量 5,822 名个体的人口统计特征。响应变量是 `Purchase`，它表示给定个体是否购买了房车保险。在此数据集中，只有 6% 的人购买了房车保险。

In [50]:
```
Caravan = load_data('Caravan')
Purchase = Caravan.Purchase
Purchase.value_counts()
```

Out[50]:
```
No      5474
Yes      348
Name: Purchase, dtype: int64
```

方法 `value_counts()` 接受一个 `pd.Series` 或 `pd.DataFrame`，并返回一个 `pd.Series`，其中包含每个唯一元素的对应计数。在这种情况下，`Purchase` 只有 `Yes` 和 `No` 值，并返回每个值有多少个。

```
In [51]: 348 / 5822
```

```
Out[51]: 0.0598
```

Our features will include all columns except `Purchase`.

```
In [52]: feature_df = Caravan.drop(columns=['Purchase'])
```

Because the KNN classifier predicts the class of a given test observation by identifying the observations that are nearest to it, the scale of the variables matters. Any variables that are on a large scale will have a much larger effect on the *distance* between the observations, and hence on the KNN classifier, than variables that are on a small scale. For instance, imagine a data set that contains two variables, `salary` and `age` (measured in dollars and years, respectively). As far as KNN is concerned, a difference of 1,000 USD in salary is enormous compared to a difference of 50 years in age. Consequently, `salary` will drive the KNN classification results, and `age` will have almost no effect. This is contrary to our intuition that a salary difference of 1,000 USD is quite small compared to an age difference of 50 years. Furthermore, the importance of scale to the KNN classifier leads to another issue: if we measured `salary` in Japanese yen, or if we measured `age` in minutes, then we'd get quite different classification results from what we get if these two variables are measured in dollars and years.

A good way to handle this problem is to *standardize* the data so that all variables are given a mean of zero and a standard deviation of one. Then all variables will be on a comparable scale. This is accomplished using the `StandardScaler()` transformation.

standardize

`StandardScaler()`

```
In [53]: scaler = StandardScaler(with_mean=True,
                                 with_std=True,
                                 copy=True)
```

The argument `with_mean` indicates whether or not we should subtract the mean, while `with_std` indicates whether or not we should scale the columns to have standard deviation of 1 or not. Finally, the argument `copy=True` indicates that we will always copy data, rather than trying to do calculations in place where possible.

This transformation can be fit and then applied to arbitrary data. In the first line below, the parameters for the scaling are computed and stored in `scaler`, while the second line actually constructs the standardized set of features.

```
In [54]: scaler.fit(feature_df)
         X_std = scaler.transform(feature_df)
```

Now every column of `feature_std` below has a standard deviation of one and a mean of zero.

```
In [55]: feature_std = pd.DataFrame(
                        X_std,
                        columns=feature_df.columns);
         feature_std.std()
```

```
Out[55]: MOSTYPE     1.000086
         MAANTHUI    1.000086
```

```
MGEMOMV     1.000086
MGEMLEEF    1.000086
MOSHOOFD    1.000086
              ...
AZEILPL     1.000086
APLEZIER    1.000086
AFIETS      1.000086
AINBOED     1.000086
ABYSTAND    1.000086
Length: 85, dtype: float64
```

请注意，此处标准差并不完全是1；这同样是因为某些程序对方差采用了$1/n$约定（在此情况下为`scaler()`），而其他程序则使用$1/(n-1)$ (`std()`方法)。参见脚注 .std() 在第183页。在这种情况下，只要所有变量都在同一尺度上，就无关紧要。

使用函数`train_test_split()`，我们现在将观测值拆分为一个包含1000个观测值的测试集，以及包含其余观测值的训练集。参数 train_test_split() `random_state=0`确保我们每次重新运行代码时都得到相同的划分。

In [56]:
```
(X_train,
 X_test,
 y_train,
 y_test) = train_test_split(feature_std,
                            Purchase,
                            test_size=1000,
                            random_state=0)
```

`?train_test_split` 揭示了非关键字参数可以是 `lists`、`arrays`、`pandas dataframes` 等，它们都具有相同的长度 (`shape[0]`)，因此是 *indexable*。在这种情况下，它们是数据框 `feature_std` 和响应变量 `Purchase`。我们使用 $K = 1$ 在训练数据上拟合了一个 KNN 模型，并在测试数据上评估其性能。 可索引

In [57]:
```
knn1 = KNeighborsClassifier(n_neighbors=1)
knn1_pred = knn1.fit(X_train, y_train).predict(X_test)
np.mean(y_test != knn1_pred), np.mean(y_test != "No")
```

Out[57]:
```
(0.111, 0.067)
```

在1,000个测试观察样本上，KNN的错误率约为11%。乍一看，这似乎相当不错。然而，由于仅有略高于6%的客户购买了保险，如果我们始终预测`No`，而不考虑预测变量的值，就能将错误率降至几乎6%！这被称为 *null rate*。

假设向某个特定个人推销保险存在一定的非平凡成本。例如，也许销售人员必须拜访每个潜在客户。如果公司试图向随机选择的客户推销保险，那么成功率将仅为6%，考虑到所涉及的成本，这可能太低了。相反，公司希望只向可能购买保险的客户推销。因此，整体错误率并不受关注。相反，被正确预测为购买保险的个体比例才是关注的焦点。 null rate

In [58]:
```
confusion_table(knn1_pred, y_test)
```

```
Out[58]:    Truth   No  Yes
          Predicted
                No  880   58
               Yes   53    9
```

事实证明，当K =取1时，KNN在预测会购买保险的客户中的表现远胜于随机猜测。在这62名客户中，有9名，即14.5%，实际购买了保险。这是随机猜测所得比例的两倍。

```
In [59]: 9/(53+9)
```

```
Out[59]: 0.145
```

调整参数

KNN 中邻居的数量被称为 *tuning parameter*，也称为 *hyperparameter*。我们不知道 *a priori* 该用什么值因此，观察分类器在测试数据上的表现如何随着这些参数的变化而变化是很有意义的。这可以通过第2.3.8节中描述的 `for` 循环来实现。在这里，我们使用一个 for 循环来查看当邻居数量从1变化到5时，我们的分类器在被预测为购买保险的群体中的准确率：

调优参数

超参数

```
In [60]: for K in range(1,6):
             knn = KNeighborsClassifier(n_neighbors=K)
             knn_pred = knn.fit(X_train, y_train).predict(X_test)
             C = confusion_table(knn_pred, y_test)
             templ = ('K={0:d}: # predicted to rent: {1:>2},' +
                      '  # who did rent {2:d}, accuracy {3:.1%}')
             pred = C.loc['Yes'].sum()
             did_rent = C.loc['Yes','Yes']
             print(templ.format(
                   K,
                   pred,
                   did_rent,
                   did_rent / pred))
```

```
K=1: # predicted to rent: 62,# who did rent 9, accuracy 14.5%
K=2: # predicted to rent: 6,# who did rent 1, accuracy 16.7%
K=3: # predicted to rent: 20,# who did rent 3, accuracy 15.0%
K=4: # predicted to rent: 3,# who did rent 0, accuracy 0.0%
K=5: # predicted to rent: 7,# who did rent 1, accuracy 14.3%
```

我们看到了一些变异性——`K=4` 的数值与其他数值差异很大。

与逻辑回归的比较

作为比较，我们也可以对数据拟合逻辑回归模型。这也可以通过 `sklearn` 完成，不过默认情况下，它拟合的是类似于第6章中介绍的 *ridge regression* 版本的逻辑回归。可以通过适当设置下面的参数 `C` 来修改这一点。其默认值为1，但通过将其设置为一个非常大的数，算法会收敛到与上面讨论的通常（未正则化）逻辑回归估计器相同的解。

与statsmodels包不同，sklearn更侧重于分类而非推断。因此，在statsmodels中见到的summary方法以及在summarize中看到的简化版方法，并不普遍适用于sklearn中的分类器。

```
In [61]: logit = LogisticRegression(C=1e10, solver='liblinear')
         logit.fit(X_train, y_train)
         logit_pred = logit.predict_proba(X_test)
         logit_labels = np.where(logit_pred[:,1] > 5, 'Yes', 'No')
         confusion_table(logit_labels, y_test)
```

```
Out[61]:     Truth   No  Yes
         Predicted
                No  933   67
               Yes    0    0
```

我们使用上面的参数solver='liblinear'来避免默认求解器发出警告，该警告会表明算法不收敛。

如果我们使用0.5作为分类器的预测概率截断值，那么问题就来了：没有任何测试观测被预测会购买保险。然而，我们并不一定要使用0.5作为截断值。如果我们改为在预测购买概率超过0.25时就预测购买，结果会好得多：我们预测有29人会购买保险，而对于这些人中的约31%，我们的预测是正确的。这几乎是随机猜测的五倍！

```
In [62]: logit_labels = np.where(logit_pred[:,1]>0.25, 'Yes', 'No')
         confusion_table(logit_labels, y_test)
```

```
Out[62]:     Truth   No  Yes
         Predicted
                No  913   58
               Yes   20    9
```

```
In [63]: 9/(20+9)
```

```
Out[63]: 0.310
```

4.7.7 Linear and Poisson Regression on the Bikeshare Data

这里我们将线性回归和泊松回归模型拟合到 Bikeshare 数据，如第4.6节所述。响应变量 bikers 衡量了华盛顿特区2010-2012年间每小时的自行车租赁数量。

```
In [64]: Bike = load_data('Bikeshare')
```

让我们来看看这个数据框中变量的维度和名称。

```
In [65]: Bike.shape, Bike.columns
```

```
Out[65]: ((8645, 15),
         Index(['season', 'mnth', 'day', 'hr', 'holiday', 'weekday',
                'workingday', 'weathersit', 'temp', 'atemp', 'hum',
                'windspeed', 'casual', 'registered', 'bikers'],
                dtype='object'))
```

线性回归

我们首先对数据拟合一个线性回归模型。

```
In [66]: X = MS(['mnth',
                 'hr',
                 'workingday',
                 'temp',
                 'weathersit']).fit_transform(Bike)
         Y = Bike['bikers']
         M_lm = sm.OLS(Y, X).fit()
         summarize(M_lm)
```

```
Out[66]:                  coef  std err       t  P>|t|
         intercept    -68.6317    5.307 -12.932  0.000
         mnth[Feb]      6.8452    4.287   1.597  0.110
         mnth[March]   16.5514    4.301   3.848  0.000
         mnth[April]   41.4249    4.972   8.331  0.000
         mnth[May]     72.5571    5.641  12.862  0.000
         mnth[June]    67.8187    6.544  10.364  0.000
         mnth[July]    45.3245    7.081   6.401  0.000
         mnth[Aug]     53.2430    6.640   8.019  0.000
         mnth[Sept]    66.6783    5.925  11.254  0.000
         mnth[Oct]     75.8343    4.950  15.319  0.000
         mnth[Nov]     60.3100    4.610  13.083  0.000
         mnth[Dec]     46.4577    4.271  10.878  0.000
         hr[1]        -14.5793    5.699  -2.558  0.011
         hr[2]        -21.5791    5.733  -3.764  0.000
         hr[3]        -31.1408    5.778  -5.389  0.000
         .....         .......    .....   .....  .....
```

`hr` 中有 24 个级别，总共有 40 行，因此我们对摘要进行了截断。在 `M_lm` 中，前几个级别 `hr[0]` 和 `mnth[Jan]` 被视为基线值，因此没有为它们提供系数估计值：隐含地，它们的系数估计值为零，所有其他级别都是相对于这些基线来衡量的。例如，2月系数 6.845 表示，在所有其他变量保持不变的情况下，2月的骑手数量平均比1月多约7人。同样，3月的骑手数量比1月多约16.5人。

第4.6.1节中展示的结果使用了变量`hr`和`mnth`的稍微不同的编码方式，具体如下：

```
In [67]: hr_encode = contrast('hr', 'sum')
         mnth_encode = contrast('mnth', 'sum')
```

Refitting again:

```
In [68]: X2 = MS([mnth_encode,
                  hr_encode,
                 'workingday',
                 'temp',
```

```
        'weathersit']).fit_transform(Bike)
M2_lm = sm.OLS(Y, X2).fit()
S2 = summarize(M2_lm)
S2
```

Out[68]:

	coef	std err	t	P>|t|
intercept	73.5974	5.132	14.340	0.000
mnth[Jan]	-46.0871	4.085	-11.281	0.000
mnth[Feb]	-39.2419	3.539	-11.088	0.000
mnth[March]	-29.5357	3.155	-9.361	0.000
mnth[April]	-4.6622	2.741	-1.701	0.089
mnth[May]	26.4700	2.851	9.285	0.000
mnth[June]	21.7317	3.465	6.272	0.000
mnth[July]	-0.7626	3.908	-0.195	0.845
mnth[Aug]	7.1560	3.535	2.024	0.043
mnth[Sept]	20.5912	3.046	6.761	0.000
mnth[Oct]	29.7472	2.700	11.019	0.000
mnth[Nov]	14.2229	2.860	4.972	0.000
hr[0]	-96.1420	3.955	-24.307	0.000
hr[1]	-110.7213	3.966	-27.916	0.000
hr[2]	-117.7212	4.016	-29.310	0.000
.....	.....	.....	.....	.....

两种编码方式有何区别？在M2_lm中，除了hr的23水平和mnth的Dec水平外，其他所有水平都报告了系数估计值。重要的是，在M2_lm中，mnth最后一个水平的（未报告的）系数估计值并非零：相反，它等于其他所有水平系数估计值之和的负值。类似地，在M2_lm中，hr最后一个水平的系数估计值是其他所有水平系数估计值之和的负值。这意味着M2_lm中hr和mnth的系数总和始终为零，并且可以解释为与平均水平之间的差异。例如，−中一月的系数46.087表明，在其他所有变量保持不变的情况下，一月份的骑行者数量通常比年平均水平少46人。

重要的是要认识到，编码的选择实际上并不重要，只要我们根据所使用的编码正确解释模型输出即可。例如，我们可以看到，无论采用何种编码，线性模型的预测结果都是相同的：

```
In [69]: np.sum((M_lm.fittedvalues - M2_lm.fittedvalues)**2)
```

```
Out[69]: 1.53e-20
```

平方差之和为零。我们也可以使用 np.allclose() 函数看到这一点：

np.allclose()

```
In [70]: np.allclose(M_lm.fittedvalues, M2_lm.fittedvalues)
```

```
Out[70]: True
```

为了重现图4.13的左侧，我们首先需要获取与mnth相关的系数估计。1月至11月的系数可以直接从M2_lm对象中获取。12月的系数必须显式计算为其他所有月份系数的负和。我们首先从M2_lm的系数中提取所有月份的系数。

```
In[71]: coef_month = S2[S2.index.str.contains('mnth')]['coef']
        coef_month
```

```
Out[71]: mnth[Jan]     -46.0871
         mnth[Feb]     -39.2419
         mnth[March]   -29.5357
         mnth[April]    -4.6622
         mnth[May]      26.4700
         mnth[June]     21.7317
         mnth[July]     -0.7626
         mnth[Aug]       7.1560
         mnth[Sept]     20.5912
         mnth[Oct]      29.7472
         mnth[Nov]      14.2229
         Name: coef, dtype: float64
```

Next, we append `Dec` as the negative of the sum of all other months.

```
In[72]: months = Bike['mnth'].dtype.categories
        coef_month = pd.concat([
                              coef_month,
                              pd.Series([-coef_month.sum()],
                                        index=['mnth[Dec]'
                                       ])
                              ])
        coef_month
```

```
Out[72]: mnth[Jan]     -46.0871
         mnth[Feb]     -39.2419
         mnth[March]   -29.5357
         mnth[April]    -4.6622
         mnth[May]      26.4700
         mnth[June]     21.7317
         mnth[July]     -0.7626
         mnth[Aug]       7.1560
         mnth[Sept]     20.5912
         mnth[Oct]      29.7472
         mnth[Nov]      14.2229
         mnth[Dec]       0.3705
         Name: coef, dtype: float64
```

Finally, to make the plot neater, we'll just use the first letter of each month, which is the 6th entry of each of the labels in the index.

```
In[73]: fig_month, ax_month = subplots(figsize=(8,8))
        x_month = np.arange(coef_month.shape[0])
        ax_month.plot(x_month, coef_month, marker='o', ms=10)
        ax_month.set_xticks(x_month)
        ax_month.set_xticklabels([l[5] for l in coef_month.index], fontsize
            =20)
        ax_month.set_xlabel('Month', fontsize=20)
        ax_month.set_ylabel('Coefficient', fontsize=20);
```

Reproducing the right-hand plot in Figure 4.13 follows a similar process.

```
In[74]: coef_hr = S2[S2.index.str.contains('hr')]['coef']
        coef_hr = coef_hr.reindex(['hr[{0}]'.format(h) for h in range(23)])
        coef_hr = pd.concat([coef_hr,
```

```
                    pd.Series([-coef_hr.sum()], index=['hr[23]'])
                   ])
```

We now make the hour plot.

In [75]:
```
fig_hr, ax_hr = subplots(figsize=(8,8))
x_hr = np.arange(coef_hr.shape[0])
ax_hr.plot(x_hr, coef_hr, marker='o', ms=10)
ax_hr.set_xticks(x_hr[::2])
ax_hr.set_xticklabels(range(24)[::2], fontsize=20)
ax_hr.set_xlabel('Hour', fontsize=20)
ax_hr.set_ylabel('Coefficient', fontsize=20);
```

Poisson Regression

Now we fit instead a Poisson regression model to the `Bikeshare` data. Very little changes, except that we now use the function `sm.GLM()` with the Poisson family specified:

In [76]:
```
M_pois = sm.GLM(Y, X2, family=sm.families.Poisson()).fit()
```

We can plot the coefficients associated with `mnth` and `hr`, in order to reproduce Figure 4.15. We first complete these coefficients as before.

In [77]:
```
S_pois = summarize(M_pois)
coef_month = S_pois[S_pois.index.str.contains('mnth')]['coef']
coef_month = pd.concat([coef_month,
                        pd.Series([-coef_month.sum()],
                                  index=['mnth[Dec]'])])
coef_hr = S_pois[S_pois.index.str.contains('hr')]['coef']
coef_hr = pd.concat([coef_hr,
                     pd.Series([-coef_hr.sum()],
                     index=['hr[23]'])])
```

The plotting is as before.

In [78]:
```
fig_pois, (ax_month, ax_hr) = subplots(1, 2, figsize=(16,8))
ax_month.plot(x_month, coef_month, marker='o', ms=10)
ax_month.set_xticks(x_month)
ax_month.set_xticklabels([l[5] for l in coef_month.index], fontsize
    =20)
ax_month.set_xlabel('Month', fontsize=20)
ax_month.set_ylabel('Coefficient', fontsize=20)
ax_hr.plot(x_hr, coef_hr, marker='o', ms=10)
ax_hr.set_xticklabels(range(24)[::2], fontsize=20)
ax_hr.set_xlabel('Hour', fontsize=20)
ax_hr.set_ylabel('Coefficient', fontsize=20);
```

We compare the fitted values of the two models. The fitted values are stored in the `fittedvalues` attribute returned by the `fit()` method for both the linear regression and the Poisson fits. The linear predictors are stored as the attribute `lin_pred`.

In [79]:
```
fig, ax = subplots(figsize=(8, 8))
ax.scatter(M2_lm.fittedvalues,
           M_pois.fittedvalues,
           s=20)
ax.set_xlabel('Linear Regression Fit', fontsize=20)
```

```
ax.set_ylabel('Poisson Regression Fit', fontsize=20)
ax.axline([0,0], c='black', linewidth=3,
          linestyle='--', slope=1);
```

泊松回归模型的预测值与线性模型的预测值相关；然而，前者的预测值非负。因此，对于极低或极高的客流量水平，泊松回归的预测值往往比线性模型的预测值更大。

在本节中，我们使用带有参数`family=sm.families.Poisson()`的`sm.GLM()`函数来拟合泊松回归模型。在本实验的早期，我们使用了带有`family=sm.families.Binomial()`的`sm.GLM()`函数来执行逻辑回归。`family`参数的其他选择可用于拟合其他类型的广义线性模型。例如，`family=sm.families.Gamma()`拟合伽马回归模型。

4.8 练习

Conceptual

1. 使用一点代数知识，证明(4.2)式等同于(4.3)式。换句话说，逻辑回归模型的对数几率表示与逻辑函数表示是等价的。

2. 文中指出，将观测值分类到 (4.17) 最大的类别，等同于将观测值分类到 (4.18) 最大的类别。证明这一情况成立。换句话说，在假设第 k 类的观测值来自 $N(\mu_k, \sigma^2)$ 分布下，贝叶斯分类器将观测值分配到判别函数最大的类别。

3. 这个问题涉及QDA模型，其中每个类别内的观测值都是从具有类别特定均值向量和类别特定协方差矩阵的正态分布中抽取的。我们考虑简单情况，即$p = 1$；也就是说，只有一个特征。

   假设我们有 K 个类别，并且如果一个观测属于第 k 类，那么 X 来自一维正态分布 $X \sim N(\mu_k, \sigma_k^2)$。回忆一下，一维正态分布的密度函数在(4.16)中给出。证明在这种情况下，贝叶斯分类器是 *not* 线性的。论证它实际上是二次的。

   Hint: For this problem, you should follow the arguments laid out in Section 4.4.1, but without making the assumption that $\sigma^{21} = \cdots = \sigma^{2K}$.

4. 当特征数量 p 很大时，KNN 和其他仅使用与待预测测试观测值 *near* 的观测值进行预测的 *local* 方法的性能往往会下降。这种现象被称为 *curse of dimensionality*，它与以下事实有关：



curse of dimensionality

   当p很大时，非参数方法往往表现不佳。我们现在将探究这一诅咒。

(假设我们有一组观测值，每个观测值都在 $p = 1$ 个特征 X 上有测量值。我们假设 X 在 [0, 1] 上均匀（等概率）分布。每个观测值都关联一个响应值。假设我们希望仅使用那些在 X 取值范围内最接近该测试观测值的10%范围内的观测值，来预测一个测试观测值的响应。例如，为了预测一个 $X = 0.6$ 的测试观测值的响应，我们将使用范围在 [0.55, 0.65] 内的观测值。平均而言，我们将使用可用观测值的多大比例来进行预测?

(b) 现在假设我们有一组观测数据，每个观测都在 $p = 2$ 个特征上进行了测量，X_1 和 X_2。我们假设 (X_1, X_2) 在 $[0, 1] \times [0, 1]$ 上均匀分布。我们希望仅使用那些在 X_1 *and* 的范围内 10% 以内、且在 X_2 的范围内 10% 以内且最接近该测试观测的观测来预测测试观测的响应。例如，为了预测一个具有 $X_1 = 0.6$ 和 $X_2 = 0.35$ 的测试观测的响应，我们将使用 X_1 在区间 [0.55, 0.65] 内的观测，以及 X_2 在区间 [0.3, 0.4] 内的观测。平均而言，我们将使用可用观测的多大比例来进行预测?

(c) 现在假设我们有一组关于 $p = 100$ 个特征的观测数据。同样，这些观测值在每个特征上均匀分布，并且每个特征的取值范围再次从 0 到 1。我们希望利用每个特征范围内最接近测试观测值的那 10% 范围内的观测值，来预测该测试观测值的响应。我们将使用可用观测值的多大比例来进行预测?

(d) 利用你对(a)至(c)部分的回答，论证当p较大时，KNN的一个缺点是任何给定测试观测值“附近”的训练观测值都非常少。

(e) 现在假设我们希望通过围绕测试观测创建一个p维超立方体来进行预测，该超立方体平均包含10%的训练观测。对于p =1,2和100，超立方体每边的长度是多少？请对您的答案进行评论。

Note: A hypercube is a generalization of a cube to an arbitrary number of dimensions. When $p = 1$*, a hypercube is simply a line segment, when* $p = 2$ *it is a square, and when* $p = 100$ *it is a 100-dimensional cube.*

5. 我们现在探讨LDA与QDA之间的差异。

(a) 如果贝叶斯决策边界是线性的，我们预期LDA还是QDA在训练集上表现更好？在测试集上呢？(b) 如果贝叶斯决策边界是非线性的，我们预期LDA还是QDA在训练集上表现更好？在测试集上呢?

(c) 一般来说，随着样本量 n 的增加，我们预期QDA相对于LDA的测试预测准确率会提高、下降还是保持不变？为什么？(d) 判断正误：即使给定问题的贝叶斯决策边界是线性的，我们使用QDA而不是LDA很可能会获得更优的测试错误率，因为QDA足够灵活以建模线性决策边界。证明你的答案。

6. 假设我们收集了一组统计课学生的数据，其中变量包括 $X_1 =$ 学习小时数、$X_2 =$ 本科GPA和 $Y =$ 是否获得A。我们拟合了一个逻辑回归模型，并得到估计系数 $\hat{\beta}_0 = -6, \hat{\beta}_1 = 0.05, \hat{\beta}_2 = 1$。

(a) 估计一个学习了40小时且本科GPA为3.5的学生在这门课上得到A的概率。(b) (a)部分中的学生需要学习多少小时才能有50%的机会在这门课上得到A?

7. 假设我们希望根据X，即去年的利润百分比，来预测某只股票今年是否会发放股息（“是”或“否”）。我们考察了大量公司，发现发放股息的公司中X的均值为$\bar{X} = 10$，而未发放股息的公司的均值为$\bar{X} = 0$。此外，这两组公司中X的方差均为$\hat{\sigma}^2 = 36$。最后，80%的公司发放了股息。假设X服从正态分布，预测一家公司去年利润百分比为$X = 4$时，今年发放股息的概率。

Hint: Recall that the density function for a normal random variable is $f(x) = \frac{1}{\sigma\sqrt{2}} e^{-(x-\mu)^2/2}$. *You will need to use Bayes' theorem.*

8. 假设我们取一个数据集，将其等分为训练集和测试集，然后尝试两种不同的分类方法。首先，我们使用逻辑回归，在训练数据上得到20%的错误率，在测试数据上得到30%的错误率。接着，我们使用1-最近邻（即$K = 1$），得到平均错误率（在测试和训练数据集上取平均）为18%。基于这些结果，我们应该更倾向于使用哪种方法来对新观测进行分类？为什么?

9. 这个问题与*odds*有关。

(a) 平均而言，对于那些信用卡支付违约赔率为0.37的人，实际上违约的比例是多少？(b) 假设某个人信用卡支付违约的概率为16%，她违约的赔率是多少?

10. 方程4.32在设定$p > 1$的情况下导出了$\log\left(\frac{\Pr(Y=k|X=x)}{\Pr(Y=K|X=x)}\right)$的表达式，因此第$k$类的均值$\mu_k$是一个$p$维向量，共享协方差$\mathbf{\Sigma}$是一个$p \times p$矩阵。然而，在$p = 1$的设定下，(4.32)呈现出更简洁的形式，因为均值$\mu_1, \ldots, \mu_K$和方差σ^2均为标量。在这个更简单的设定中，重复(4.32)中的计算，并给出a_k和b_{kj}关于π_k、π_K、μ_k、μ_K和σ^2的表达式。



11. 推导出(4.33)中a_k、b_{kj}和b_{kjl}的具体形式。你的答案应包含π_k、π_K、μ_k、μ_K、$\mathbf{\Sigma}_k$和$\mathbf{\Sigma}_K$。



12. 假设你希望将一个观测值 $X \in \mathbb{R}$ 分类到 `apples` 和 `oranges`。你拟合了一个逻辑回归模型，并发现

$$\widehat{\Pr}(Y = \texttt{orange}|X = x) = \frac{\exp(\hat{\beta}_0 + \hat{\beta}_1 x)}{1 + \exp(\hat{\beta}_0 + \hat{\beta}_1 x)}.$$

你的朋友使用(4.13)中的*softmax*公式对相同的数据拟合了一个逻辑回归模型，并发现

$$\widehat{\Pr}(Y = \texttt{orange}|X = x) = \frac{\exp(\hat{\alpha}_{\texttt{orange0}} + \hat{\alpha}_{\texttt{orange1}} x)}{\exp(\hat{\alpha}_{\texttt{orange0}} + \hat{\alpha}_{\texttt{orange1}} x) + \exp(\hat{\alpha}_{\texttt{apple0}} + \hat{\alpha}_{\texttt{apple1}} x)}.$$

(a) 在你的模型中，`orange` 与 `apple` 的对数几率是多少?

(b) 在你朋友的模型中，`orange` 与 `apple` 的对数几率是多少?

(c) 假设在你的模型中，$\hat{\beta}_0 = 2$ 和 $\hat{\beta}_1 = -1$。在你朋友的模型中，系数估计值是多少？请尽可能具体。

(d) 现在假设你和你的朋友在不同的数据集上拟合相同的两个模型。这次，你的朋友得到的系数估计是 $\hat{\alpha}_{\texttt{orange0}} = 1.2, \hat{\alpha}_{\texttt{orange1}} = -2$, $\hat{\alpha}_{\texttt{orange0}} = 3, \hat{\alpha}_{\texttt{orange1}} = 0.6$。你的模型中的系数估计是多少?

(e) 最后，假设你将(d)中的两个模型都应用到一个包含2,000个测试观察值的数据集上。你预计你的模型预测的类别标签与朋友的模型预测的类别标签一致的时间比例是多少？请解释你的答案。

Applied

13. 本问题应使用`Weekly`数据集回答，该数据集是`ISLP`包的一部分。此数据在性质上类似于本章实验中使用的`Smarket`数据，不同之处在于它包含从1990年初至2010年底共21年间的1, 089个周回报率。

(a) 对`Weekly`数据生成一些数值和图形摘要。是否有任何模式出现?

(b) 使用整个数据集进行逻辑回归，以Direction作为响应变量，五个滞后变量加上Volume作为预测变量。使用summary函数打印结果。是否有任何预测变量在统计上显著？如果有，是哪些？(c) 计算混淆矩阵和总体正确预测比例。解释混淆矩阵告诉您关于逻辑回归所犯错误类型的信息。(d) 现在使用1990年至2008年的训练数据期拟合逻辑回归模型，仅以Lag2作为预测变量。计算保留数据（即2009年和2010年的数据）的混淆矩阵和总体正确预测比例。

(e) 使用LDA重复(d)。

(f) 使用QDA重复(d)。 (g) 使用$K = $为1的KNN重复(d)。 (h) 使用朴素贝叶斯重复(d)。 (i) 在这些方法中，哪一种似乎在这份数据上提供了最好的结果？ (j) 对每种方法尝试不同的预测变量组合，包括可能的变换和交互作用。报告在保留数据上表现最好的变量、方法及相关的混淆矩阵。注意，你还应尝试KNN分类器中K的不同取值。

14. 在这个问题中，您将根据 Auto 数据集开发一个模型，以预测给定汽车的燃油效率是高还是低。

(a) 创建一个二值变量 mpg01，如果 mpg 包含高于其中位数的值，则该变量为1，如果 mpg 包含低于其中位数的值，则该变量为0。你可以使用数据框的 median() 方法来计算中位数。注意，你可能会发现通过赋值向数据框添加一列 mpg01 会很有帮助。假设你将数据框存储为 Auto，则可以按以下方式操作：

```
Auto['mpg01'] = mpg01
```

(b) 以图形方式探索数据，以研究 mpg01 与其他特征之间的关联。其他哪些特征似乎最有可能在预测 mpg01 时有用？散点图和箱线图可能是回答这个问题的有用工具。描述你的发现。(c) 将数据分为训练集和测试集。(d) 在训练数据上执行LDA，以便使用在(b)中似乎与 mpg01 最相关的变量来预测 mpg01。所获得模型的测试误差是多少？

(e) 对训练数据执行 QDA，以使用在 (b) 中似乎与 `mpg01` 最相关的变量来预测 `mpg01`。所得模型的测试误差是多少？(f) 对训练数据执行逻辑回归，以使用在 (b) 中似乎与 `mpg01` 最相关的变量来预测 `mpg01` 。所得模型的测试误差是多少？(g) 对训练数据执行朴素贝叶斯，以使用在 (b) 中似乎与 `mpg01` 最相关的变量来预测 `mpg01`。所得模型的测试误差是多少？(h) 对训练数据执行 KNN，使用多个 K 值，以预测 `mpg01`。仅使用在 (b) 中似乎与 `mpg01` 最相关的变量。你得到哪些测试误差？哪个 K 值在此数据集上表现最佳?

15. 这个问题涉及编写函数。

(a) 编写一个函数 `Power()`，打印出将2提升到3次方的结果。换句话说，你的函数应计算 2^3 并打印出结果。
Hint: Recall that `x**a` *raises* `x` *to the power* `a`*. Use the* `print()` *function to display the result.* (b) 创建一个新函数 `Power2()`，允许你传入两个数 *any*，`x` 和 `a`，并打印出 `x**a` 的值。你可以通过在函数开头写入以下行来实现这一点

```
def Power2(x, a):
```

您应该能够通过输入例如 `Power2(3, 8)` 来调用您的函数

在命令行中。这应输出3^8的值，即6, 561。

(c) 使用你刚刚编写的 `Power2()` 函数，计算 10^3、8^{17} 和 131^3。

(d) 现在创建一个新函数 `Power3()`，它实际上将结果 `x**a` 作为 `Python` 对象 *returns*，而不是简单地将其打印到屏幕上。也就是说，如果你将值 `x**a` 存储在一个名为 `result` 的对象中，那么你可以简单地 `return`

此结果，使用以下行：

```
    return result
```

请注意，上一行应该是函数中的最后一行，并且应缩进4个空格。

(e) 现在使用 `Power3()` 函数，创建 $f(x) = x^2$ 的图表。x 轴应显示从 1 到 10 的整数范围，y 轴应显示 x^2。适当标注坐标轴，并为图形使用合适的标题。考虑将 x 轴、y 轴或两者都显示在对数刻度上。你可以通过使用以下对象的 `ax.set_xscale()` 和 `ax.set_yscale()` 方法来实现这一点：

您正在绘制的坐标轴。

(创建一个函数 `PlotPower()`，使您能够为固定的 `a` 和一系列 `x` 值创建 `x` 与 `x**a` 的图表。例如，如果您调用

```
PlotPower(np.arange(1, 11), 3)
```

然后应创建一个图表，其中 x 轴的取值范围为 $1, 2, \ldots, 10$，y 轴的取值范围为 $1^3, 2^3, \ldots, 10^3$。

16. 使用 `Boston` 数据集，拟合分类模型以预测给定郊区犯罪率是否高于或低于中位数。探索逻辑回归、LDA、朴素贝叶斯和KNN模型，使用不同的预测变量子集。描述你的发现。

Hint: You will have to create the response variable yourself, using the variables that are contained in the `Boston` *data set.*

5
重采样方法



Resampling methods 是现代统计学中不可或缺的工具。它们涉及从训练集中反复抽取样本，并在每个样本上重新拟合感兴趣的模型，以获取关于拟合模型的额外信息。例如，为了估计线性回归拟合的变异性，我们可以从训练数据中反复抽取不同的样本，对每个新样本拟合线性回归，然后检查所得拟合结果的差异程度。这种方法可能使我们能够获得仅使用原始训练样本拟合模型一次所无法获得的信息。

重采样方法可能在计算上代价高昂，因为它们涉及使用训练数据的不同子集多次拟合相同的统计方法。然而，由于近年来计算能力的进步，重采样方法的计算需求通常并不构成障碍。在本章中，我们讨论两种最常用的重采样方法，*cross-validation*和*bootstrap*。这两种方法都是许多统计学习程序实际应用中的重要工具。例如，交叉验证可用于估计与给定统计学习方法相关的测试误差，以评估其性能，或选择适当的灵活性水平。评估模型性能的过程称为*model assessment*，而

模型评估

为模型选择合适灵活度的过程被称为*model selection*。Bootstrap在多种场景中得到应用，最常见的是
提供参数估计或给定统计学习方法的准确性度量。

模型选择

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_5

5.1 交叉验证

在第2章中，我们讨论了*test error rate*与*training error rate*之间的区别。测试误差是指使用统计学习方法对新观测（即未用于训练该方法的测量数据）进行响应预测时所产生的平均误差。给定一个数据集，如果某种特定的统计学习方法能够导致较低的测试误差，那么使用该方法是合理的。如果有指定的测试集可用，测试误差可以很容易地计算出来。遗憾的是，通常情况并非如此。相比之下，训练误差可以通过将统计学习方法应用于其训练中所使用的观测数据来轻松计算。但正如我们在第2章中所看到的，训练误差率往往与测试误差率有相当大的差异，特别是前者可能会大幅低估后者。

在缺乏一个非常大的指定测试集来直接估计测试错误率的情况下，可以使用多种技术来利用可用的训练数据估计这一量值。一些方法对训练错误率进行数学调整，以估计测试错误率。这些方法将在第6章讨论。在本节中，我们转而考虑一类方法，这些方法通过*holding out*从拟合过程中留出一部分训练观测值，然后将统计学习方法应用于这些留出的观测值来估计测试错误率。

在5.1.1至5.1.4节中，为简便起见，我们假设关注的是执行具有定量响应的回归。在5.1.5节中，我们考虑具有定性响应的分类情况。正如我们将看到的，无论响应是定量还是定性，关键概念保持不变。

5.1.1 The Validation Set Approach

假设我们想要估计将某种特定的统计学习方法拟合到一组观测数据上所产生的测试误差。图5.1中展示的 *validation set approach* 是一种非常简单的策略
对于此任务。它涉及将可用的观测集随机分成两部分，一个*training set*和一个*validation set*或*hold-out set*。
模型在训练集上进行拟合，然后使用拟合后的模型对验证集中的观测值进行响应预测。由此得到的验证集错误率——在定量响应情况下通常使用MSE进行评估——提供了测试错误率的估计。

验证集方法

验证集 留出集

我们在 `Auto` 数据集上说明验证集方法。回想第 3 章，`mpg` 与 `horsepower` 之间似乎存在非线性关系，并且使用 `horsepower` 和 $\texttt{horsepower}^2$ 预测 `mpg` 的模型比仅使用线性项的模型给出更好的结果。自然要问，三次或更高阶拟合是否会提供更好的结果。我们在第 3 章通过查看线性回归中三次项和更高阶多项式项相关的 p 值来回答这个问题。但我们也可以使用验证方法来回答这个问题。我们将 392 个观测值随机分成两部分

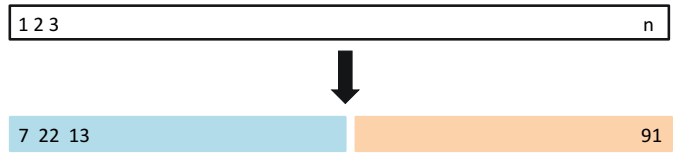


图5.1。*A schematic display of the validation set approach. A set of n observations are randomly split into a training set (shown in blue, containing observations 7, 22, and 13, among others) and a validation set (shown in beige, and containing observation 91, among others). The statistical learning method is fit on the training set, and its performance is evaluated on the validation set.*

集合，一个包含196个数据点的训练集，以及一个包含剩余196个观测值的验证集。使用MSE作为验证集误差的度量，通过在训练样本上拟合各种回归模型并在验证样本上评估其性能所得到的验证集误差率，如图5.2左侧面板所示。二次拟合的验证集MSE远小于线性拟合。然而，三次拟合的验证集MSE实际上略大于二次拟合。这意味着在回归中包含三次项并不比仅使用二次项带来更好的预测。

回顾一下，为了生成图5.2左侧面板，我们通过随机将数据集分为两部分：训练集和验证集。如果我们重复将样本集随机分成两部分的过程，我们将得到对测试MSE略有不同的估计。作为说明，图5.2右侧面板展示了来自`Auto`数据集的十条不同的验证集MSE曲线，这些曲线是通过将观测值随机分成训练集和验证集的十种不同划分方式生成的。所有十条曲线均表明，包含二次项的模型其验证集MSE显著小于仅含线性项的模型。此外，所有十条曲线都表明，在模型中包含三次或更高阶多项式项并无太大益处。但值得注意的是，对于考虑的十个回归模型中的每一个，这十条曲线各自得出的测试MSE估计值都不相同。并且，关于哪个模型能产生最小的验证集MSE，这些曲线之间并没有达成共识。基于这些曲线间的变异性，我们能有信心得出的结论仅仅是：线性拟合对于这些数据来说是不够的。

验证集方法在概念上简单且易于实现。但它有两个潜在的缺点：

1. 如图5.2右侧面板所示，测试误差率的验证估计可能变化很大，具体取决于哪些观测被包含在训练集中，哪些观测被包含在验证集中。2. 在验证方法中，只有一部分观测——那些包含在训练集中而非验证集中的观测——被用于拟合模型。由于统计方法在训练观测较少时往往表现较差，这表明

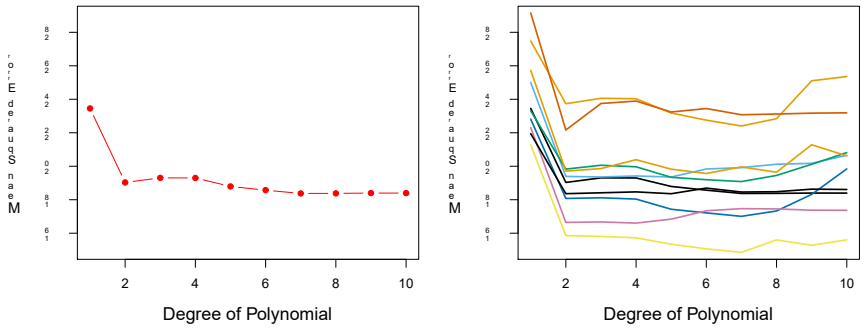


图 5.2。*The validation set approach was used on the* `Auto` *data set in order to estimate the test error that results from predicting* `mpg` *using polynomial functions of* `horsepower`*. 左：Validation error estimates for a single split into training and validation data sets. 右：The validation method was repeated ten times, each time using a different random split of the observations into a training set and a validation set. This illustrates the variability in the estimated test MSE that results from this approach.*

> 验证集错误率可能趋于 *overestimate* 整个数据集上拟合模型的测试错误率。

在接下来的小节中，我们将介绍*cross-validation*，这是对验证集方法的一种改进，解决了这两个问题。

5.1.2 Leave-One-Out Cross-Validation

Leave-one-out cross-validation (LOOCV)与验证密切相关采用第5.1.1节的方法，但试图解决该方法的缺点。

留一法交叉验证

与验证集方法类似，留一交叉验证（LOOCV）也涉及将观测集分为两部分。然而，不同于创建两个大小相当的子集，LOOCV使用单个观测（x_1, y_1）作为验证集，而剩余的观测$\{(x_2, y_2), \ldots, (x_n, y_n)\}$构成训练集。统计学习方法在$n-1$个训练观测上进行拟合，并对被排除的观测作出预测$\hat{y}_1$，利用其值$x_1$。由于$(x_1, y_1)$未参与拟合过程，$\text{MSE}_1 = (y_1 - \hat{y}_1)^2$为测试误差提供了近似无偏的估计。但即便$\text{MSE}_1$对于测试误差是无偏的，它仍是一个较差的估计，因为其高度可变，这是因为它基于单个观测（x_1, y_1）。

我们可以通过选择验证数据的 (x_2, y_2)，在 $n-1$ 个观测 $\{(x_1, y_1), (x_3, y_3), \ldots, (x_n, y_n)\}$ 上训练统计学习过程，并计算 $\text{MSE}_2 = (y_2 - \hat{y}_2)^2$，来重复这一过程。重复这种方法 n 次会产生 n 个平方误差，$\text{MSE}_1, \ldots, \text{MSE}_n$。测试 MSE 的 LOOCV 估计是这些 n 个测试误差估计的平均值：

$$\text{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^{n} \text{MSE}_i. \tag{5.1}$$

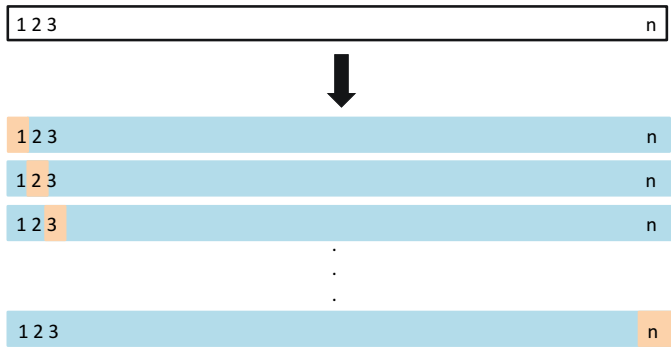


图 5.3. *A schematic display of LOOCV. A set of n data points is repeatedly split into a training set (shown in blue) containing all but one observation, and a validation set that contains only that observation (shown in beige). The test error is then estimated by averaging the n resulting MSEs. The first training set contains all but observation 1, the second training set contains all but observation 2, and so forth.*

留一交叉验证方法的示意图如图5.3所示。

与验证集方法相比，LOOCV 有几个主要优势。首先，它的偏差要小得多。在 LOOCV 中，我们反复使用包含 $n-1$ 个观测值的训练集来拟合统计学习方法，这几乎与整个数据集中的观测值数量一样多。这与验证集方法形成对比，在验证集方法中，训练集通常约为原始数据集大小的一半。因此，LOOCV 方法往往不像验证集方法那样高估测试错误率。其次，与验证方法不同——由于训练/验证集划分的随机性，重复应用时会产生不同的结果——多次执行 LOOCV 将始终产生相同的结果：在训练/验证集划分中没有随机性。

我们在 `Auto` 数据集上使用 LOOCV，以获得通过使用 `horsepower` 的多项式函数拟合线性回归模型来预测 `mpg` 所产生的测试集 MSE 的估计。结果如图 5.4 左侧面板所示。

LOOCV 的实现可能成本高昂，因为模型必须拟合 n 次。如果 n 很大，并且每个单独模型拟合缓慢，这可能会非常耗时。对于最小二乘线性或多项式回归，一个惊人的捷径使 LOOCV 的成本与单次模型拟合相同！以下公式成立：

$$\mathrm{CV}_{(n)} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{y}_i}{1-h_i}\right)^2, \tag{5.2}$$

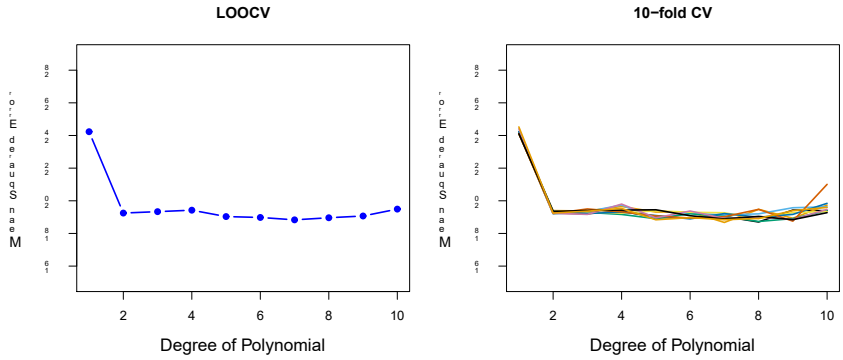


图5.4。*Cross-validation was used on the* `Auto` *data set in order to estimate the test error that results from predicting* `mpg` *using polynomial functions of* `horsepower`. 左：*The LOOCV error curve.* 右：10-*fold CV was run nine separate times, each with a different random split of the data into ten parts. The figure shows the nine slightly different CV error curves.*

其中 $\hat{y}_i$ 是原始最小二乘拟合中的第 i 个拟合值，而 h_i 是第 105 页 (3.37) 中定义的杠杆值。[1] 这类似于普通 MSE，不同之处在于第 i 个残差被除以 1 $-$ h_i。杠杆值介于 $1/n$ 和 1 之间，反映了某个观测值对其自身拟合的影响程度。因此，在这个公式中，高杠杆点的残差被恰好按使该等式成立所需的正确幅度进行了放大。

LOOCV 是一种非常通用的方法，可用于任何类型的预测建模。例如，我们可以将其用于逻辑回归或线性判别分析，或任何后续章节中讨论的方法。魔法公式 (5.2) 通常并不成立，在这种情况下，模型必须重新拟合 n 次。

5.1.3 k-Fold Cross-Validation

k-fold CV

LOOCV 的另一种替代方法是 *k-fold CV*。这种方法涉及随机将观测集划分为 k 组，或 *folds*，大小大致相等。第一折被视为验证集，方法在剩余的 $k-1$ 折上进行拟合。然后，在留出折中的观测上计算均方误差 MSE_1。此过程重复 k 次；每次将不同组的观测视为验证集。此过程产生 k 个测试误差的估计值，$\text{MSE}_1, \text{MSE}_2, \ldots, \text{MSE}_k$。$k$ 折 CV 估计值通过平均这些值来计算，

$$\text{CV}_{(k)} = \frac{1}{k}\sum_{i=1}^{k}\text{MSE}_i. \tag{5.3}$$

图5.5展示了k折交叉验证方法。

[1] In the case of multiple linear regression, the leverage takes a slightly more complicated form than (3.37), but (5.2) still holds.

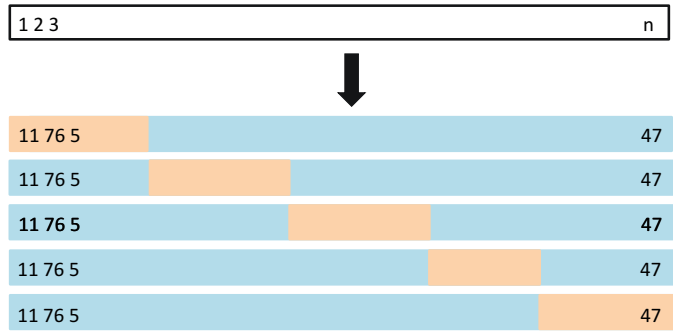


图5.5。*A schematic display of* 5*-fold CV. A set of* n *observations is randomly split into five non-overlapping groups. Each of these fifths acts as a validation set (shown in beige), and the remainder as a training set (shown in blue). The test error is estimated by averaging the five resulting MSE estimates.*

不难看出，LOOCV 是 k 折 CV 的一个特例，其中 k 被设定为等于 n。在实践中，人们通常执行 k 折 CV，使用 $k = 5$ 或 $k = 10$。使用 $k = 5$ 或 $k = 10$ 而不是 $k = n$ 的优势是什么？最明显的优势在于计算方面。LOOCV 需要拟合统计学习方法 n 次。这可能会造成高昂的计算成本（除非是通过最小二乘法拟合的线性模型，在这种情况下可以使用公式 (5.2)）。但交叉验证是一种非常通用的方法，几乎可以应用于任何统计学习方法。一些统计学习方法具有计算密集型的拟合过程，因此执行 LOOCV 可能会带来计算问题，尤其是在 n 极大的情况下。相比之下，执行 10 折 CV 仅需拟合学习过程十次，这可能更加可行。正如我们在第 5.1.4 节中所见，执行 5 折或 10 折 CV 还可能具有其他非计算方面的优势，这些优势涉及偏差-方差权衡。

图5.4的右侧面板展示了`Auto`数据集的九种不同的10折CV估计，每种估计都是通过将观测值随机划分为十折的不同方式得到的。如图中所示，由于将观测数据划分成十折的方式存在差异，CV估计呈现出一定的变动性。然而，这种变动性通常远低于由验证集方法导致的测试误差估计的变动性（图5.2右侧面板）。

当我们检查真实数据时，我们不知道*true*测试MSE，因此很难确定交叉验证估计的准确性。然而，如果我们检查模拟数据，那么我们可以计算真实的测试MSE，从而可以评估我们的交叉验证结果的准确性。在图5.6中，我们绘制了应用于第2章图2.9–2.11所示模拟数据集的平滑样条的交叉验证估计值和真实测试错误率。真实的测试MSE以蓝色显示。黑色虚线和橙色实线分别显示估计的LOOCV和10折CV估计。在所有三个图中，两个交叉验证估计非常相似。在右侧面板中

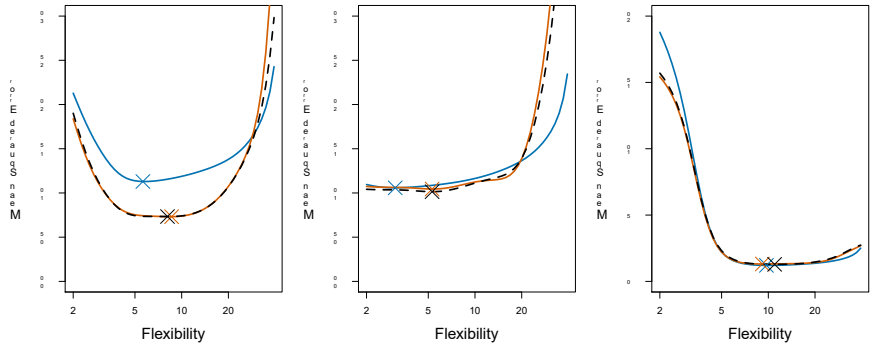


图5.6。*True and estimated test MSE for the simulated data sets in Figures 2.9 (左), 2.10 (中), and 2.11 (右). The true test MSE is shown in blue, the LOOCV estimate is shown as a black dashed line, and the* 10*-fold CV estimate is shown in orange. The crosses indicate the minimum of each of the MSE curves.*

图5.6中，真实的测试MSE与交叉验证曲线几乎完全一致。在图5.6的中间面板，两组曲线在较低灵活性程度上相似，而当灵活性程度较高时，交叉验证曲线高估了测试集MSE。在图5.6的左侧面板，交叉验证曲线具有正确的一般形状，但低估了真实的测试MSE。

当我们进行交叉验证时，我们的目标可能是确定一个给定的统计学习程序在独立数据上的预期表现如何；在这种情况下，测试MSE的实际估计值是我们感兴趣的。但在其他时候，我们只对*minimum point in the estimated test MSE curve*的位置感兴趣。这是因为我们可能对多种统计学习方法进行交叉验证，或者对使用不同灵活性水平的单一方法进行交叉验证，以确定导致最低测试误差的方法。为此，估计的测试MSE曲线中最小值点的位置很重要，但估计的测试MSE的实际值并不重要。我们在图5.6中发现，尽管它们有时低估了真实的测试MSE，但所有CV曲线都接近识别出正确的灵活性水平——即对应于最小测试MSE的灵活性水平。

5.1.4 Bias-Variance Trade-Off for k-Fold Cross-Validation

我们在第5.1.3节中提到，采用k折交叉验证（$k < n$）在计算上比LOOCV有优势。但撇开计算问题不谈，k折交叉验证一个不太明显但可能更重要的优点是，它通常比LOOCV能提供更准确的测试错误率估计。这与偏差-方差权衡有关。

在第5.1.1节中提到，验证集方法可能导致对测试错误率的高估，因为在这种方法中，用于拟合统计学习方法的训练集仅包含整个数据集一半的观测值。根据这一逻辑，不难看出

LOOCV 将给出测试误差的近似无偏估计，因为每个训练集包含 $n-1$ 个观测值，这几乎与完整数据集中的观测数量一样多。而执行 k 折 CV，比如 $k=5$ 或 $k=10$，将导致中等程度的偏差，因为每个训练集大约包含 $(k-1)n/k$ 个观测值——比 LOOCV 方法中的少，但比验证集方法中的多得多。因此，从减少偏差的角度来看，LOOCV 显然优于 k 折 CV。

然而，我们知道偏差并非估计过程中唯一需要关注的来源；我们还必须考虑该过程的方差。结果表明，当$k<n$时，LOOCV的方差比k折CV更高。为什么会这样？当我们执行LOOCV时，实际上是对n个拟合模型的输出取平均，而每个模型几乎都是在相同的观测集上训练的；因此，这些输出之间高度（正）相关。相反，当我们执行$k<n$的k折CV时，我们是对k个拟合模型的输出取平均，这些模型之间的相关性要低一些，因为每个模型中训练集之间的重叠更小。由于许多高度相关的量的均值比许多相关性不那么高的量的均值具有更高的方差，因此由LOOCV得到的测试误差估计往往比由k折CV得到的测试误差估计具有更高的方差。

总之，在选择k折交叉验证中的k时，存在偏差-方差权衡。通常，鉴于这些考虑，人们会使用$k=5$或$k=10$来进行k折交叉验证，因为经验表明，这些值产生的测试错误率估计既不会遭受过高偏差，也不会遭受非常高的方差。

5.1.5 Cross-Validation on Classification Problems

在本章中，到目前为止，我们已经展示了在结果 Y 为定量时的回归设置中使用交叉验证的方法，因此使用了 MSE 来量化测试误差。但交叉验证在 Y 为定性时的分类设置中也可以是一种非常有用的方法。在这种设置中，交叉验证的工作原理与本章前面所述完全相同，只是我们不使用 MSE 来量化测试误差，而是使用误分类观测的数量来代替。例如，在分类设置中，LOOCV 错误率的形式为

$$\mathrm{CV}_{(n)}=\frac{1}{n}\sum_{i=1}^{n}\mathrm{Err}_i, \tag{5.4}$$

其中$\mathrm{Err}_i=I(y_i\neq\hat{y}_i)$。$k$折CV错误率和验证集错误率的定义类似。

作为一个示例，我们对图2.13中展示的二维分类数据拟合了多种逻辑回归模型。在图5.7的左上角面板中，实心黑线显示了对此数据集拟合标准逻辑回归模型后估计得到的决策边界。由于这是模拟数据，我们可以计算出$true$测试错误率，其值为0.201，因此远大于贝叶斯

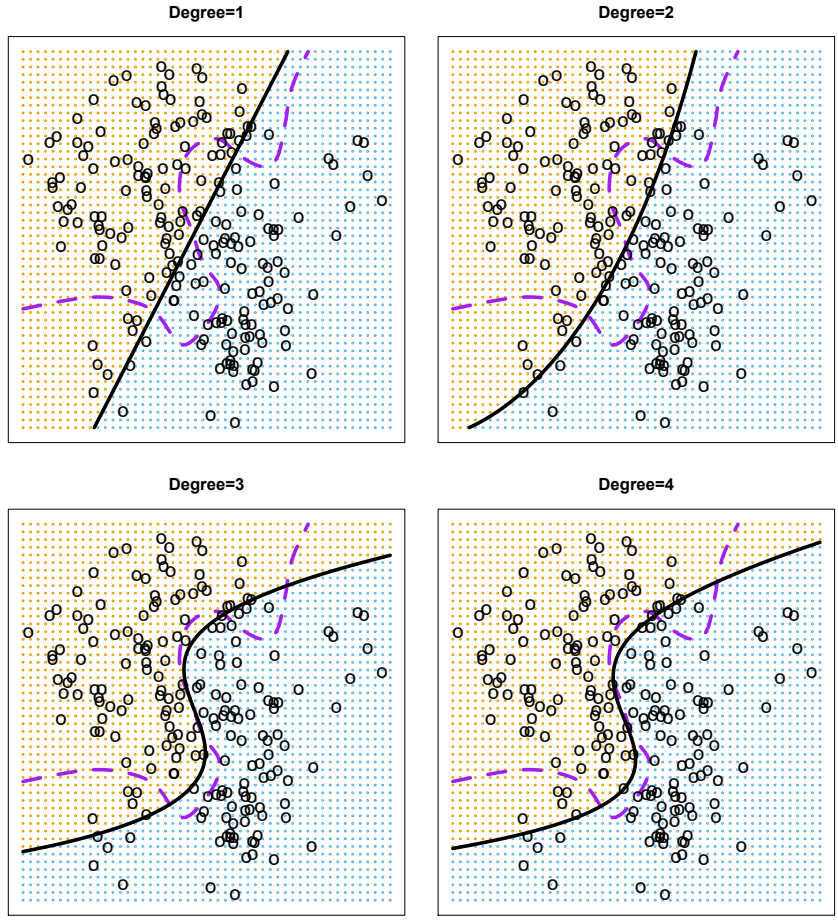


图 5.7. *Logistic regression fits on the two-dimensional classification data displayed in Figure 2.13. The Bayes decision boundary is represented using a purple dashed line. Estimated decision boundaries from linear, quadratic, cubic and quartic (degrees 1–4) logistic regressions are displayed in black. The test error rates for the four logistic regression fits are respectively* 0.201, 0.197, 0.160, *and* 0.162, *while the Bayes error rate is* 0.133.

错误率为0.133。显然，逻辑回归在此情境下缺乏足够的灵活性来建模贝叶斯决策边界。正如我们在第3.3.2节回归设置中所做，通过使用预测变量的多项式函数，我们可以轻松扩展逻辑回归以获得非线性决策边界。例如，我们可以拟合一个*quadratic*逻辑回归模型，其表达式为

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \beta_3 X_2 + \beta_4 X_2^2. \qquad (5.5)$$

图5.7的右上角面板显示了由此产生的决策边界，现在呈现出曲线形态。然而，测试错误率仅略有改善，降至0.197。左下角面板中显示出了更大的改进。

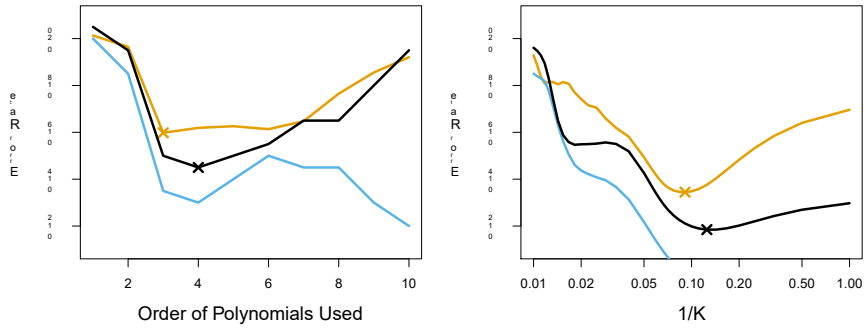


图 5.8. *Test error (brown), training error (blue), and* 10*-fold CV error (black) on the two-dimensional classification data displayed in Figure 5.7.* 左: *Logistic regression using polynomial functions of the predictors. The order of the polynomials used is displayed on the x-axis.* 右: *The KNN classifier with different values of* K*, the number of neighbors used in the KNN classifier.*

图5.7中，我们拟合了一个涉及预测变量三次多项式的逻辑回归模型。现在，测试错误率已降至0.160。而采用四次多项式（右下角）则略微增加了测试错误。

在实践中，对于真实数据，贝叶斯决策边界和测试错误率是未知的。那么，我们如何在图5.7中展示的四种逻辑回归模型之间做出选择呢？我们可以使用交叉验证来做出这一决定。图5.8左侧面板以黑色显示了通过拟合十个逻辑回归模型至数据所得到的10折交叉验证错误率，这些模型使用了预测变量的多项式函数，最高达十阶。真实的测试误差以棕色显示，训练误差以蓝色显示。如前所述，随着拟合灵活性的增加，训练误差趋于减小。（图中表明，尽管训练错误率并非严格单调下降，但随着模型复杂度的增加，总体上趋于减少。）相比之下，测试误差呈现出典型的U形。10折交叉验证错误率相当好地 approximated 了测试错误率。虽然它略微低估了错误率，但在使用四阶多项式时达到最小值，这非常接近于测试曲线的最小值，而测试曲线的最小值出现在使用三阶多项式时。事实上，使用四阶多项式可能会带来良好的测试集性能，因为对于三阶、四阶、五阶和六阶多项式，真实的测试错误率大致相同。

图5.8的右侧面板展示了使用KNN方法进行分类时的相同三条曲线，其变化取决于K (的值，在此上下文中，该值表示KNN分类器中使用的邻居数量，而非使用的交叉验证折数)。同样，随着方法变得更加灵活，训练错误率下降，因此我们看到训练错误率不能用来选择K的最优值。尽管交叉验证错误曲线略微低估了测试错误率，但其最小值非常接近K的最佳值。

5.2 自助法

*bootstrap*是一种广泛适用且极其强大的统计工具
可以用来量化与给定估计器或统计学习方法相关的不确定性。举一个简单的例子，自助法可以用来估计线性回归拟合中系数的标准误。在线性回归的具体情形中，这并不特别有用，因为我们在第3章中看到，诸如R这样的标准统计软件会自动输出这样的标准误。然而，自助法的强大之处在于，它可以很容易地应用于广泛的统计学习方法，包括一些否则难以获得变异性度量且统计软件不会自动输出的方法。

bootstrap

在本节中，我们通过一个简单的玩具示例来说明自助法，该示例旨在确定简单模型下的最佳投资分配。在第5.3节中，我们将探讨使用自助法来评估线性模型拟合中回归系数相关的变异性。

假设我们希望将一笔固定金额的资金投资于两种金融资产，其收益率分别为 X 和 Y，其中 X 和 Y 是随机变量。我们将把资金的一部分 α 投资于 X，并将剩余的 $1-\alpha$ 投资于 Y。由于这两种资产的收益率存在波动性，我们希望选择 α 来最小化投资的总风险或方差。换句话说，我们要最小化 $\text{Var}(\alpha X + (1-\alpha)Y)$。可以证明，使风险最小化的取值由下式给出

$$\alpha = \frac{\sigma_Y^2 - \sigma_{XY}}{\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}}, \tag{5.6}$$

其中 $\sigma_X^2 = \text{Var}(X), \sigma_Y^2 = \text{Var}(Y)$，并且 $\sigma_{XY} = \text{Cov}(X, Y)$。

实际上，数量 σ_X^2、σ_Y^2 和 σ_{XY} 是未知的。我们可以使用包含 X 和 Y 过去测量值的数据集，计算这些数量的估计值 $\hat{\sigma}_X^2$、$\hat{\sigma}_Y^2$ 和 $\hat{\sigma}_{XY}$。然后，我们可以估计使我们的投资方差最小化的 α 的值，使用

$$\hat{\alpha} = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}. \tag{5.7}$$

图5.9展示了在模拟数据集上估计α的这一方法。在每个面板中，我们为投资X和Y模拟了100对收益率。我们利用这些收益率来估计σ_X^2, σ_Y^2和σ_{XY}，随后将其代入(5.7)以得到α的估计值。由每个模拟数据集得到的$\hat{\alpha}$值范围从0.532到0.657。

很自然地希望量化我们对α估计的准确性。为了估计$\hat{\alpha}$的标准差，我们重复了模拟100对X和Y的观测值并使用(5.7)估计α的过程，共1,000次。由此，我们获得了1,000个关于α的估计值，可将其称为$\hat{\alpha}_1, \hat{\alpha}_2, \ldots, \hat{\alpha}_{1,000}$。图5.10左侧面板展示了所得估计值的直方图。在这些模拟中，参数设置为σ_X^2 ==1，σ_Y^2 ==1，.=25，以及σ_{XY} ==0，.=5，因此我们知道α的真实值是0，.=6。我们在直方图上用一条垂直实线标出了这个值。

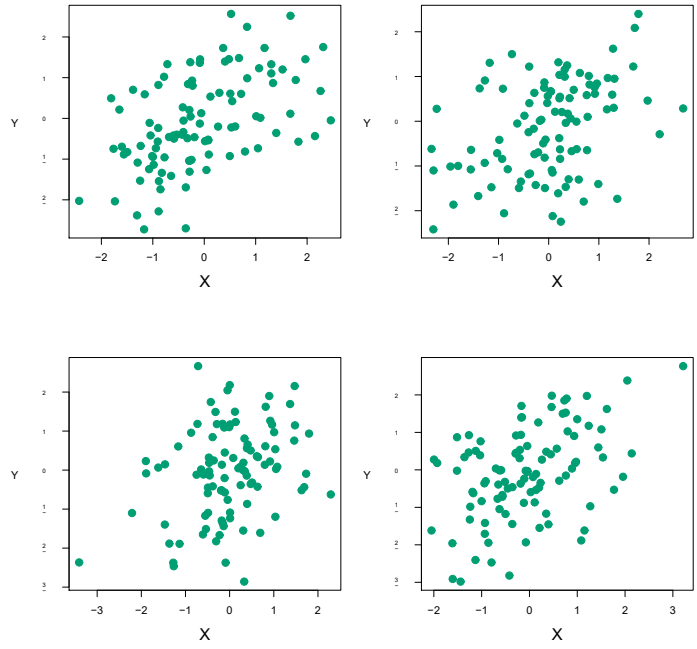


图5.9. *Each panel displays* 100 *simulated returns for investments* X *and* Y. *From left to right and top to bottom, the resulting estimates for* α *are* 0.576, 0.532, 0.657, *and* 0.651.

α 的所有 1,000 个估计值的均值为

$$\bar{\alpha} = \frac{1}{1000}\sum_{r=1}^{1000}\hat{\alpha}_r = 0.5996,$$

非常 接近$\alpha = 0.6$，并且估计的标准差 ates是

$$\sqrt{\frac{1}{1000-1}\sum_{r=1}^{1000}(\hat{\alpha}_r - \bar{\alpha})^2} = 0.083.$$

这让我们对$\hat{\alpha}$的准确性有了很好的了解：SE($\hat{\alpha}$) $\approx$ 0.083。因此，粗略地说，对于来自总体的一次随机抽样，我们预期$\hat{\alpha}$与α平均相差约0.08。

然而，实际上，上述估计SE($\hat{\alpha}$)的过程无法应用，因为对于真实数据，我们无法从原始总体中生成新的样本。但是，自助法允许我们使用计算机来模拟获取新样本集的过程，从而使我们能够在无需生成额外样本的情况下估计$\hat{\alpha}$的变异性。我们不是反复从总体中获取独立数据集，而是通过反复抽样观测值*from the original data set*来获得不同的数据集。

该方法在图5.11中通过一个简单的数据集进行了说明，我们称之为Z，它仅包含n =个3观测值。我们随机从数据集中选择n个观测值以生成一个自助数据集，

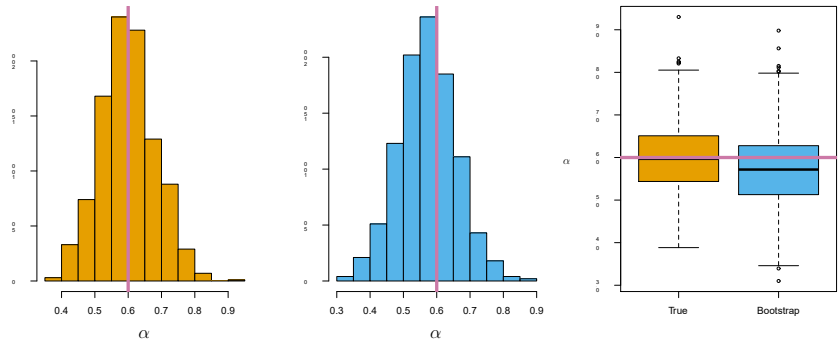


图5.10. 左：*A histogram of the estimates of* α *obtained by generating 1,000 simulated data sets from the true population.* 中：*A histogram of the estimates of* α *obtained from 1,000 bootstrap samples from a single data set.* 右：*The estimates of* α *displayed in the left and center panels are shown as boxplots. In each panel, the pink line indicates the true value of* α.

Z^{*1}. 采样是以 *with replacement* 进行的，这意味着同一个观测值可能在自助法数据集中出现多次。在这个例子中，Z^{*1}包含第三次观测两次，第一次观测一次，而第二次观测没有实例。请注意，如果一个观测值包含在Z^{*1}中，那么它的X和Y值都会被包含。我们可以使用Z^{*1}来生成对α的新自助法估计，我们称之为$\hat{\alpha}^{*1}$。对于某个较大的B值，此过程重复B次，以生成B个不同的自助法数据集$Z^{*1}, Z^{*2}, \ldots, Z^{*B}$，以及$B$个相应的$\alpha$估计值$\hat{\alpha}^{*1}, \hat{\alpha}^{*2}, \ldots, \hat{\alpha}^{*B}$。我们可以使用以下公式计算这些自助法估计的标准误：

with replacement

$$\mathrm{SE}_B(\hat{\alpha}) = \sqrt{\frac{1}{B-1}\sum_{r=1}^{B}\left(\hat{\alpha}^{*r} - \frac{1}{B}\sum_{r'=1}^{B}\hat{\alpha}^{*r'}\right)^2}. \tag{5.8}$$

这用作从原始数据集中估计的$\hat{\alpha}$标准误的估计值。

自助法在图5.10的中间面板中进行了展示，该面板显示了α的1,000个自助估计值的直方图，每个估计值都是使用不同的自助数据集计算得出的。这个面板是基于单一数据集构建的，因此可以使用真实数据来创建。请注意，该直方图与左侧面板非常相似，左侧面板显示了通过从真实总体生成1,000个模拟数据集而获得的α估计值的理想化直方图。特别地，来自(5.8)的自助估计SE($\hat{\alpha}$)为0.087，非常接近使用1,000个模拟数据集获得的估计值0.083。右侧面板以不同的方式展示了中间和左侧面板中的信息，通过箱线图展示了通过从真实总体生成1,000个模拟数据集并使用自助法获得的α估计值。同样，箱线图具有相似的分散程度，表明自助法可以有效地用于估计与$\hat{\alpha}$相关的变异性。

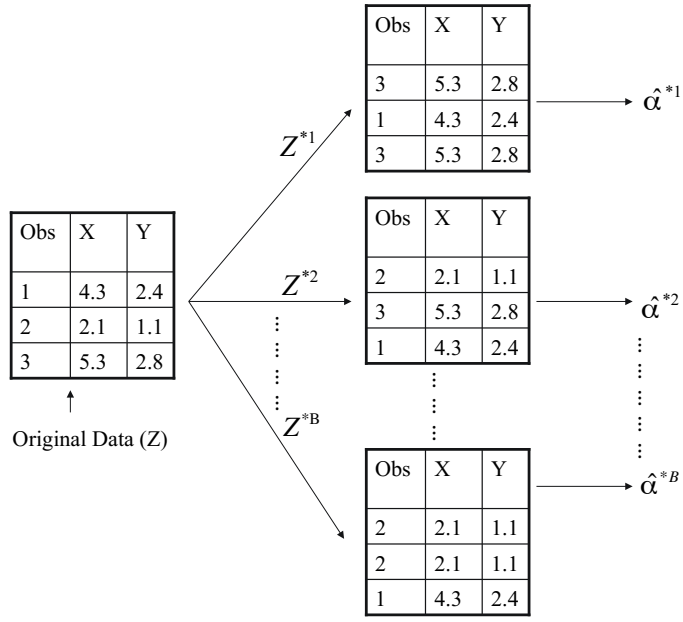


图5.11。*A graphical illustration of the bootstrap approach on a small sample containing* $n = 3$ *observations. Each bootstrap data set contains* n *observations, sampled with replacement from the original data set. Each bootstrap data set is used to obtain an estimate of* α.

5.3 实验：交叉验证与自助法

在本实验中，我们将探索本章所涵盖的重采样技术。本实验中的某些命令在您的计算机上可能需要一段时间才能运行完毕。

我们再次从将大部分导入项置于顶层开始。

In [1]:
```
import numpy as np
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
                         summarize,
                         poly)
from sklearn.model_selection import train_test_split
```

这个实验需要几个新的导入。

In [2]:
```
from functools import partial
from sklearn.model_selection import \
     (cross_validate,
      KFold,
      ShuffleSplit)
from sklearn.base import clone
from ISLP.models import sklearn_sm
```

5.3.1 The Validation Set Approach

我们探索使用验证集方法来估计在Auto数据集上拟合各种线性模型所得到的测试错误率。

我们使用函数train_test_split()将数据拆分为训练集和验证集。由于有392个观测值，我们使用参数test_size=196将其分成两个大小均为196的等量集合。在执行像这样包含随机性因素的操作时，设置随机种子通常是个好主意，这样以后可以精确复现所获得的结果。我们使用参数random_state=0设置分割器的随机种子。

train_test_split()

In [3]:
```
Auto = load_data('Auto')
Auto_train, Auto_valid = train_test_split(Auto,
                                          test_size=196,
                                          random_state=0)
```

现在我们可以仅使用对应于训练集Auto_train的观测值来拟合线性回归。

In [4]:
```
hp_mm = MS(['horsepower'])
X_train = hp_mm.fit_transform(Auto_train)
y_train = Auto_train['mpg']
model = sm.OLS(y_train, X_train)
results = model.fit()
```

我们现在使用在验证数据集上为该模型创建的模型矩阵上评估的results的predict()方法。我们还计算我们模型的验证MSE。

In [5]:
```
X_valid = hp_mm.transform(Auto_valid)
y_valid = Auto_valid['mpg']
valid_pred = results.predict(X_valid)
np.mean((y_valid - valid_pred)**2)
```

Out[5]: 23.6166

因此，线性回归拟合的验证MSE估计值为23.62。

我们也可以估计更高次多项式回归的验证误差。我们首先提供一个函数evalMSE()，它接收模型字符串以及训练集和测试集，并返回测试集上的MSE。

In [6]:
```
def evalMSE(terms,
            response,
            train,
            test):

   mm = MS(terms)
   X_train = mm.fit_transform(train)
   y_train = train[response]

   X_test = mm.transform(test)
   y_test = test[response]
```

```
    results = sm.OLS(y_train, X_train).fit()
    test_pred = results.predict(X_test)

    return np.mean((y_test - test_pred)**2)
```

让我们使用这个函数来估计线性、二次和三次拟合的验证MSE。我们在这里使用enumerate()函数，它给出
在遍历for循环时，同时获取对象的值和索引。

enumerate()

In [7]:
```
MSE = np.zeros(3)
for idx, degree in enumerate(range(1, 4)):
    MSE[idx] = evalMSE([poly('horsepower', degree)],
                       'mpg',
                       Auto_train,
                       Auto_valid)
MSE
```

Out[7]:
```
array([23.62, 18.76, 18.80])
```

这些错误率分别是23.62,18.76和18.80。如果我们选择不同的训练/验证集划分，那么验证集上的错误率可能会有所不同。

In [8]:
```
Auto_train, Auto_valid = train_test_split(Auto,
                                          test_size=196,
                                          random_state=3)
MSE = np.zeros(3)
for idx, degree in enumerate(range(1, 4)):
    MSE[idx] = evalMSE([poly('horsepower', degree)],
                       'mpg',
                       Auto_train,
                       Auto_valid)
MSE
```

Out[8]:
```
array([20.76, 16.95, 16.97])
```

使用将观测数据分为训练集和验证集的这种划分，我们发现具有线性、二次和三次项的模型的验证集错误率分别为 20.76、16.95 和 16.97。

这些结果与我们之前的发现一致：使用horsepower的二次函数预测mpg的模型比仅涉及horsepower线性函数的模型表现更好，并且没有证据表明使用horsepower的三次函数会有所改进。

5.3.2 Cross-Validation

理论上，交叉验证估计可以针对任何广义线性模型进行计算。然而，在实践中，Python中进行交叉验证最简单的方法是使用sklearn，它具有与我们一直用来拟合GLMs的statsmodels代码不同的接口或API。

这是数据科学家经常面临的一个问题：“我有一个执行任务 A 的函数，需要将它输入到执行任务 B 的某个东西中，以便我能计算 $B(A(D))$，其中 D 是我的数据。”当 A 和 B 不能自然地相互沟通时，这需要使用一个 *wrapper*。在 ISLP 中

wrapper

包中，我们提供了一个包装器 sklearn_sm()，使我们能够轻松使用 sklearn 的交叉验证工具，使用由 statsmodels 拟合的模型。

sklearn_sm()

类 sklearn_sm() 的第一个参数是来自 statsmodels 的模型。它还可以接受两个额外的可选参数：model_str 可用于指定公式，model_args 应该是一个字典，包含拟合模型时使用的附加参数。例如，要拟合逻辑回归模型，我们必须指定一个 family 参数。这通过 model_args={'family':sm.families.Binomial()} 传递。

这是我们的封装器正在运行：

In [9]:
```
hp_model = sklearn_sm(sm.OLS,
                      MS(['horsepower']))
X, Y = Auto.drop(columns=['mpg']), Auto['mpg']
cv_results = cross_validate(hp_model,
                            X,
                            Y,
                            cv=Auto.shape[0])
cv_err = np.mean(cv_results['test_score'])
cv_err
```

Out[9]:
```
24.2315
```

cross_validate() 的参数如下：一个具有适当的 fit()、predict() 和 score() 方法的对象，一个特征数组 X 和一个响应 Y。我们还向 cross_validate() 添加了一个额外的参数 cv；指定一个整数 K 会导致 K 折交叉验证。我们提供了一个对应于总观测数的值，这导致留一法交叉验证（LOOCV）。cross_validate() 函数

cross_validate()

tion 生成一个包含多个组件的字典；我们这里只需要交叉验证测试得分（MSE），估计为 24.23。

我们可以对越来越复杂的多项式拟合重复此过程。为了自动化该过程，我们再次使用一个 for 循环，它迭代地拟合 1 到 5 次的多项式回归，计算相关的交叉验证误差，并将其存储在向量 cv_error 的第 i 个元素中。for 循环中的变量 d 对应于多项式的次数。我们首先初始化该向量。此命令可能需要几秒钟才能运行。

In [10]:
```
cv_error = np.zeros(5)
H = np.array(Auto['horsepower'])
M = sklearn_sm(sm.OLS)
for i, d in enumerate(range(1,6)):
    X = np.power.outer(H, np.arange(d+1))
    M_CV = cross_validate(M,
                          X,
                          Y,
                          cv=Auto.shape[0])
    cv_error[i] = np.mean(M_CV['test_score'])
cv_error
```

Out[10]:
```
array([24.2315, 19.2482, 19.3350, 19.4244, 19.0332])
```

如图5.4所示，我们观察到线性与二次拟合之间估计的测试MSE急剧下降，但随后使用更高次多项式并没有明显改善。

上面我们介绍了 `np.power()` 函数的 `outer()` 方法。`outer()` 方法应用于具有两个参数的操作，例如 `add()`、`min()` 或 `power()`。它接受两个数组作为参数，然后形成一个更大的数组，其中该操作应用于两个数组的每对元素。

.outer()
np.power()

```
In [11]: A = np.array([3, 5, 9])
         B = np.array([2, 4])
         np.add.outer(A, B)
```

```
Out[11]: array([[ 5,  7],
                [ 7,  9],
                [11, 13]])
```

在上面的CV示例中，我们使用了$K = n$，当然我们也可以使用$K < n$。代码与上面非常相似（并且速度明显更快）。这里我们使用`KFold()`将数据划分为$K =$个随机组。我们使用`KFold()`设置随机种子，并初始化一个向量`cv_error`，其中将存储对应于一次到五次多项式拟合的CV误差。

```
In [12]: cv_error = np.zeros(5)
         cv = KFold(n_splits=10,
                    shuffle=True,
                    random_state=0) # use same splits for each degree
         for i, d in enumerate(range(1,6)):
             X = np.power.outer(H, np.arange(d+1))
             M_CV = cross_validate(M,
                                   X,
                                   Y,
                                   cv=cv)
             cv_error[i] = np.mean(M_CV['test_score'])
         cv_error
```

```
Out[12]: array([24.2077, 19.1853, 19.2763, 19.4785, 19.1372])
```

注意，计算时间比LOOCV短得多。（原则上，对于最小二乘线性模型，LOOCV的计算时间应该比K折CV更快，因为LOOCV有公式(5.2)可用；然而，通用的`cross_validate()`函数并未利用这个公式。）我们仍然几乎看不到使用三次或更高次多项式项会导致比简单使用二次拟合更低的测试误差的证据。

`cross_validate()` 函数非常灵活，可以接受不同的分割机制作为参数。例如，可以使用 `ShuffleSplit()` 用于像K折交叉验证一样轻松实现验证集方法的函数。

Shuffle
Split()

```
In [13]: validation = ShuffleSplit(n_splits=1,
                                   test_size=196,
                                   random_state=0)
         results = cross_validate(hp_model,
                                  Auto.drop(['mpg'], axis=1),
                                  Auto['mpg'],
                                  cv=validation);
         results['test_score']
```

```
Out[13]: array([23.6166])
```

One can estimate the variability in the test error by running the following:

```
In[14]: validation = ShuffleSplit(n_splits=10,
                                  test_size=196,
                                  random_state=0)
        results = cross_validate(hp_model,
                                 Auto.drop(['mpg'], axis=1),
                                 Auto['mpg'],
                                 cv=validation)
        results['test_score'].mean(), results['test_score'].std()
```

```
Out[14]: (23.8022, 1.4218)
```

请注意，这个标准差并不是对平均测试分数或个体分数的抽样变异性的有效估计，因为随机选择的训练样本存在重叠，从而引入了相关性。但它确实能让我们了解到通过选择不同随机折（folds）所产生的蒙特卡洛变异的情况。

5.3.3 The Bootstrap

我们通过在5.2节的简单示例中以及一个涉及估计`Auto`数据集上线性回归模型准确性的例子来展示自助法的使用。

估计感兴趣统计量的准确性

自助法的一大优势在于它几乎适用于所有情况。不需要复杂的数学计算。虽然Python中有几种自助法的实现，但对于估计标准误差而言，其用法足够简单，因此当我们的数据存储在数据框中时，我们在下面编写自己的函数。

为了说明自助法，我们从一个简单的例子开始。`ISLP`包中的`Portfolio`数据集在5.2节中描述。目标是估计公式(5.7)中给出的参数α的抽样方差。我们将创建一个函数`alpha_func()`，其输入为一个数据框`D`，假定其包含列`X`和`Y`，以及一个向量`idx`，指示应使用哪些观测来估计α。该函数随后基于选定的观测输出α的估计值。

```
In[15]: Portfolio = load_data('Portfolio')
        def alpha_func(D, idx):
           cov_ = np.cov(D[['X','Y']].loc[idx], rowvar=False)
           return ((cov_[1,1] - cov_[0,1]) /
                   (cov_[0,0]+cov_[1,1]-2*cov_[0,1]))
```

该函数基于对由参数`idx`索引的观测值应用最小方差公式（5.7），返回α的估计值。例如，以下命令使用全部100个观测值来估计α。

```
In[16]: alpha_func(Portfolio, range(100))
```

```
Out[16]: 0.5758
```

接下来，我们从range(100)中随机选取100个观测值，是有放回地选取。这相当于构建一个新的自助数据集，并基于新数据集重新计算$\hat{\alpha}$。

```
In [17]: rng = np.random.default_rng(0)
         alpha_func(Portfolio,
                    rng.choice(100,
                               100,
                               replace=True))
```

```
Out[17]: 0.6074
```

这个过程可以推广，以创建一个简单的函数boot_SE()，用于计算仅接受数据框作为参数的任意函数的自助法标准误。

```
In [18]: def boot_SE(func,
                     D,
                     n=None,
                     B=1000,
                     seed=0):
             rng = np.random.default_rng(seed)
             first_, second_ = 0, 0
             n = n or D.shape[0]
             for _ in range(B):
                 idx = rng.choice(D.index,
                                  n,
                                  replace=True)
                 value = func(D, idx)
                 first_ += value
                 second_ += value**2
             return np.sqrt(second_ / B - (first_ / B)**2)
```

注意在 for _ in range(B) 中使用 _ 作为循环变量。这通常用于计数器的值不重要，只是确保循环执行 B 次的情况。

让我们使用我们的函数来评估使用$B = 1{,}000$次自助重抽样对α估计的准确性。

```
In [19]: alpha_SE = boot_SE(alpha_func,
                            Portfolio,
                            B=1000,
                            seed=0)
         alpha_SE
```

```
Out[19]: 0.0912
```

最终输出显示，SE($\hat{\alpha}$) 的 bootstrap 估计值为 0.0912。

估计线性回归模型的准确性

自助法可用于评估统计学习方法的系数估计和预测的变异性。这里我们使用自助法来评估

使用 horsepower 预测 Auto 数据集中 mpg 的线性回归模型的截距和斜率项 β_0 和 β_1 的估计值。我们将把使用自助法得到的估计值与使用第 3.1.2 节中描述的 SE($\hat{\beta}_0$) 和 SE($\hat{\beta}_1$) 公式得到的估计值进行比较。

要使用我们的 boot_SE() 函数，我们必须编写一个函数（它的第一个参数），该函数仅接受数据框 D 和索引 idx 作为参数。但在这里，我们想要对由模型公式和数据指定的特定回归模型进行自助法。我们通过几个简单步骤展示如何做到这一点。

我们首先编写一个通用函数 boot_OLS()，用于对回归模型进行自助法，该函数接受一个公式来定义相应的回归。我们使用 clone() 函数来复制公式，从而可以
重新拟合到新的数据框。这意味着任何派生特征，例如由poly() (我们稍后会看到的)所定义的，都将在重采样的数据框上重新拟合。

clone()

In [20]:
```
def boot_OLS(model_matrix, response, D, idx):
    D_ = D.loc[idx]
    Y_ = D_[response]
    X_ = clone(model_matrix).fit_transform(D_)
    return sm.OLS(Y_, X_).fit().params
```

这并不是boot_SE()所需的第一个参数。指定模型的前两个参数在引导过程中不会改变，我们希望对它们进行*freeze*。函数partial()来自

functools 模块正是这样做的：它接受一个函数作为参数，并从左侧开始冻结其中一些参数。我们用它来冻结 boot_OLS() 的前两个 model-formula 参数。

partial()

In [21]:
```
hp_func = partial(boot_OLS, MS(['horsepower']), 'mpg')
```

输入 hp_func? 将显示它有两个参数 D 和 idx —— 这是一个 boot_OLS() 的版本，其前两个参数已固定 —— 因此非常适合作为 boot_SE() 的第一个参数。

hp_func() 函数现在可用于通过从观测中有放回地随机抽样，为截距和斜率项创建自助估计。我们首先在 10 个自助样本上演示其实用性。

In [22]:
```
rng = np.random.default_rng(0)
np.array([hp_func(Auto,
          rng.choice(392,
                     392,
                     replace=True)) for _ in range(10)])
```

Out[22]:
```
array([[39.8806, -0.1568],
       [38.733 , -0.147 ],
       [38.3173, -0.1444],
       [39.9145, -0.1578],
       [39.4335, -0.1507],
       [40.3663, -0.1591],
       [39.6233, -0.1545],
       [39.0581, -0.1495],
       [38.6669, -0.1452],
       [39.6428, -0.1556]])
```

接下来，我们使用boot_SE()函数来计算截距和斜率项的1,000次自助法估计的标准误差。

In [23]:
```
hp_se = boot_SE(hp_func,
                Auto,
                B=1000,
                seed=10)
hp_se
```

Out[23]:
```
intercept     0.8488
horsepower    0.0074
dtype: float64
```

这表明 SE($\hat{\beta}_0$) 的 bootstrap 估计为 0.85，而 SE($\hat{\beta}_1$) 的 bootstrap 估计为 0.0074。如第 3.1.2 节所述，标准公式可用于计算线性模型中回归系数的标准误。这些可以通过 ISLP.sm 中的 summarize() 函数获得。

In [24]:
```
hp_model.fit(Auto, Auto['mpg'])
model_se = summarize(hp_model.results_)['std err']
model_se
```

Out[24]:
```
intercept     0.717
horsepower    0.006
Name: std err, dtype: float64
```

使用第3.1.2节公式得到的$\hat{\beta}_0$和$\hat{\beta}_1$的标准误估计分别为截距的0.717和斜率的0.006。有趣的是，这些结果与使用自助法得到的估计有所不同。这是否表明自助法存在问题？事实上，它暗示了相反的情况。回想一下，第75页公式3.8中给出的标准公式依赖于某些假设。例如，它们依赖于未知参数σ^2，即噪声方差。然后我们使用RSS来估计σ^2。现在，尽管标准误的公式不依赖于线性模型的正确性，但对σ^2的估计却依赖于它。我们在第99页的图3.8中看到数据中存在非线性关系，因此线性拟合的残差会被放大，$\hat{\sigma}^2$也会如此。其次，标准公式假设（有些不现实地）x_i是固定的，所有变异性都来自误差ϵ_i的变化。自助法不依赖于任何这些假设，因此它很可能比来自sm.OLS的结果更准确地估计$\hat{\beta}_0$和$\hat{\beta}_1$的标准误。

下面我们计算自举标准误估计值和标准线性回归估计值，这些估计值是将二次模型拟合到数据后得到的。由于该模型对数据提供了良好的拟合（图3.8），现在自举估计值与SE($\hat{\beta}_0$)、SE($\hat{\beta}_1$)和SE($\hat{\beta}_2$)的标准估计值之间有了更好的对应关系。

In [25]:
```
quad_model = MS([poly('horsepower', 2, raw=True)])
quad_func = partial(boot_OLS,
                    quad_model,
                    'mpg')
boot_SE(quad_func, Auto, B=1000)
```

```
Out[25]: intercept                              2.067840
         poly(horsepower, 2, raw=True)[0]    0.033019
         poly(horsepower, 2, raw=True)[1]    0.000120
         dtype: float64
```

我们将结果与使用 {v*} 计算的标准误进行比较。

```
In [26]: M = sm.OLS(Auto['mpg'],
                    quad_model.fit_transform(Auto))
         summarize(M.fit())['std err']
```

```
Out[26]: intercept                              1.800
         poly(horsepower, 2, raw=True)[0]    0.031
         poly(horsepower, 2, raw=True)[1]    0.000
         Name: std err, dtype: float64
```

5.4 练习

Conceptual

1. 利用方差的基本统计性质以及单变量微积分，推导(5.6)。换言之，证明由(5.6)给出的α确实使Var($\alpha X + (1-\alpha)Y$)达到最小。

2. 我们现在推导出给定观测值属于自助样本的概率。假设我们从一个包含 n 个观测值的数据集中获取一个自助样本。

(a) 第一个自助法观测值是原始样本中第 j 个观测值的概率是多少？请说明理由。(b) 第二个自助法观测值是原始样本中第 j 个观测值的概率是多少？(c) 论证自助样本中第 j 个观测值是 not 的概率为 $(1-1/n)^n$。(d) 当 $n=5$ 时，第 j 个观测值在自助样本中的概率是多少？(e) 当 $n=100$ 时，第 j 个观测值在自助样本中的概率是多少？(f) 当 $n=10,000$ 时，第 j 个观测值在自助样本中的概率是多少？(g) 创建一个图，展示对于从 1 到 100, 000 的每个整数值 n，第 j 个观测值在自助样本中的概率。对你观察到的现象进行评论。(h) 我们现在将数值研究大小为 $n=100$ 的自助样本包含第 j 个观测值的概率。这里 $j=4$。我们首先创建一个数组 `store`，其中的值随后将使用函数 `np.empty()` 被覆盖。然后我们

反复创建自助法样本，并每次记录第五个观测值是否包含在自助法样本中。

```
rng = np.random.default_rng(10)
store = np.empty(10000)
for i in range(10000):
    store[i] = np.sum(rng.choice(100, replace=True) == 4)
         > 0
np.mean(store)
```

评论所得结果。

3. 我们现在回顾k折交叉验证。

(a) 解释如何实现k折交叉验证。(b) 相对于以下方法，k折交叉验证的优点和缺点是什么：i. 验证集方法？ii. LOOCV？

4. 假设我们使用某种统计学习方法对预测变量 X 的特定值对应的响应 Y 进行预测。仔细描述我们如何估计预测的标准差。

Applied

5. 在第4章中，我们使用了逻辑回归来预测在`Default`数据集上利用`income`和`balance`预测`default`的概率。现在，我们将采用验证集方法来估计这一逻辑回归模型的测试误差。在开始分析之前，别忘了设置随机种子。

(a) 拟合一个逻辑回归模型，使用 `income` 和 `balance` 来预测 `default`。(b) 使用验证集方法估计该模型的测试误差。为此，您必须执行以下步骤：i. 将样本集划分为训练集和验证集。ii. 仅使用训练观测值拟合一个多元逻辑回归模型。iii. 通过计算验证集中每个个体的违约后验概率，并对后验概率大于0.5的个体分类为 `default` 类别，获得其违约状态的预测。iv. 计算验证集误差，即验证集中被错误分类的观测比例。(c) 重复(b)中的过程三次，使用三种不同的观测划分方式将数据分为训练集和验证集。对所得结果进行评论。

(d) 现在考虑一个逻辑回归模型，该模型使用 income、balance 以及 student 的虚拟变量来预测 default 的概率。使用验证集方法估计该模型的测试误差。评论是否为 student 包含一个虚拟变量会导致测试错误率的降低。

6. 我们继续考虑使用逻辑回归模型，在Default数据集上利用income和balance预测default的概率。特别地，我们现在将以两种不同方式计算income和balance逻辑回归系数的标准误估计：(1)使用自助法，(2)使用sm.GLM()函数中计算标准误的标准公式。在开始分析前，不要忘记设置随机种子。

(a) 使用summarize()和sm.GLM()函数，确定在同时使用两个预测变量的多元逻辑回归模型中，与income和balance相关的系数的估计标准误。(b) 编写一个函数boot_fn()，该函数接受Default数据集以及观测索引作为输入，并输出多元逻辑回归模型中income和balance的系数估计。(c) 参照实验室中的自助法示例，使用您的boot_fn()函数来估计income和balance的逻辑回归系数的标准误。(d) 对使用sm.GLM()函数和自助法得到的估计标准误进行评论。

7. 在 5.1.2 和 5.1.3 节中，我们看到 cross_validate() 函数可用于计算 LOOCV 测试误差估计。或者，也可以仅使用 sm.GLM() 和拟合模型的 predict() 方法在 for 循环中计算这些量。现在，您将采用这种方法，在 Weekly 数据集上为简单的逻辑回归模型计算 LOOCV 误差。回想一下，在分类问题的背景下，LOOCV 误差由 (5.4) 给出。

(a) 拟合一个逻辑回归模型，使用 Lag1 和 Lag2 预测 Direction。(b) 拟合一个逻辑回归模型，使用 Lag1 和 Lag2 *using all but the first observation* 预测 Direction。(c) 使用(b)中的模型预测第一个观测的方向。若 $P(\texttt{Direction = "Up"}|\texttt{Lag1}, \texttt{Lag2}) > 0.5$，则通过预测第一个观测将上升来实现。该观测是否被正确分类？(d) 编写一个从 $i = 1$ 到 $i = n$ 的 for 循环，其中 n 是数据集中的观测数量，执行以下每个步骤：

i. 使用除第i个观测之外的所有数据拟合一个逻辑回归模型，以Lag1和Lag2预测Direction。ii. 计算第i个观测市场上涨的后验概率。iii. 利用第i个观测的后验概率来预测市场是否会上涨。iv. 确定在预测第i个观测的方向时是否出错。如果出错，则表示为1，否则表示为0。

(e) 取(d)iv中得到的n个数的平均值，以获得测试误差的LOOCV估计。对结果进行评论。

8. 现在我们将对模拟数据集进行交叉验证。

(a) 如下生成一个模拟数据集：

```
rng = np.random.default_rng(1)
x = rng.normal(size=100)
y = x - 2 * x**2 + rng.normal(size=100)
```

在此数据集中，n和p分别是什么？将生成数据的模型以方程形式写出。

(b) 创建X与Y的散点图。对你发现的现象进行评论。

(c) 设置一个随机种子，然后计算使用最小二乘法拟合以下四个模型所产生的LOOCV误差：

i. $Y = \beta_0 + \beta_1 X + \epsilon$ ii. $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$ iii. $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon$ iv. $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \beta_4 X^4 + \epsilon$。

请注意，你可能会发现使用 `data.frame()` 函数来创建一个同时包含 X 和 Y 的单一数据集会很有帮助。

(d) 使用另一个随机种子重复(c)，并报告你的结果。你的结果与(c)中得到的结果相同吗？为什么？(e) 在(c)中，哪个模型的LOOCV误差最小？这是你所预期的吗？解释你的答案。(f) 对使用最小二乘法拟合(c)中每个模型所得的系数估计值的统计显著性进行评论。这些结果是否与基于交叉验证结果得出的结论一致?

9. 我们现在将考虑来自 `ISLP` 库的 `Boston` 住房数据集。

(a) 基于此数据集，为 `medv` 的总体均值提供一个估计值。将此估计值称为 $\hat{\mu}$。

({b) 提供 $\hat{\mu}$ 的标准误估计值。解释这个结果。
*Hint: We can compute the standard error of the sample mean by dividing the sample standard deviation by the square root of the number of observations.*c) 现在使用自助法估计 $\hat{\mu}$ 的标准误。这与你在 (b) 中的答案相比如何？(d) 基于你在 (c) 中的自助估计，为 `medv` 的均值提供一个 95% 置信区间。将其与使用 `Boston['medv'].std()` 和两个标准误规则 (3.9) 获得的结果进行比较。*Hint: You can approximate a 95 % confidence interval using the formula* $[\hat{\mu} - 2\text{SE}(\hat{\mu}), \hat{\mu} + 2\text{SE}(\hat{\mu})]$. (e) 基于此数据集，为总体中 `medv` 的中位数提供一个估计值 $\hat{\mu}_{med}$。(f) 我们现在想估计 $\hat{\mu}_{med}$ 的标准误。不幸的是，计算中位数标准误没有简单的公式。相反，使用自助法估计中位数的标准误。对你的发现进行评论。(g) 基于此数据集，为波士顿人口普查区中 `medv` 的第十百分位数提供一个估计值。将此量称为 $\hat{\mu}_{0.1}$。（你可以使用 `np.percentile()` 函数。）(h) 使用自助法估计 $\hat{\mu}_{0.1}$ 的标准误。对你的发现进行评论。

(

`np.percentile()`

6
线性模型选择与正则化

在回归设置中，标准线性模型

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \epsilon \tag{6.1}$$

通常用于描述响应 Y 与一组变量 $X_1, X_2, \ldots, X_p$ 之间的关系。我们在第3章已经看到，通常使用最小二乘法来拟合这个模型。

在接下来的章节中，我们将探讨一些扩展线性模型框架的方法。在第7章，我们推广了(6.1)式，以适应非线性但仍然是可加的关系，而在第8章和第10章，我们考虑更为一般的非线性模型。然而，线性模型在推断方面具有明显优势，并且在现实世界的问题中，往往与非线性方法相比具有惊人的竞争力。因此，在进入非线性领域之前，本章我们将讨论一些改进简单线性模型的方法，即通过用一些替代的拟合程序替换普通的最小二乘拟合。

为什么我们可能想要使用另一种拟合程序而不是最小二乘法？正如我们将看到的，替代的拟合程序可以产生更好的 *pre- diction accuracy* 和 *model interpretability*。

- *Prediction Accuracy*如果响应变量与预测变量之间的真实关系近似线性，那么最小二乘估计将具有低偏差。如果$n \gg p$——也就是说，如果n，即观测值的数量，远大于p，即变量的数量——那么最小二乘估计往往也具有低方差，因此在测试观测值上表现良好。然而，如果n并不远大于p，那么最小二乘拟合中可能存在很大的变异性，导致过拟合，进而对未用于模型训练的未来观测值做出糟糕的预测。并且如果$p > n$，那么就不再存在唯一的最小二乘系数估计：存在无限

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_6

许多解。这些最小二乘解中的每一个在训练数据上给出零误差，但通常由于极高的方差而导致非常差的测试集性能。通过*constraining*或*shrinking*估计系数，我们通常可以在偏差几乎不增加的情况下大幅减少方差。这可以显著提高我们对未用于模型训练的观测值进行响应预测的准确性。

- *Model Interpretability*: 在多元回归模型中，常常会出现某些或许多使用的变量实际上与响应变量并无关联。纳入这些*irrelevant*变量会导致所得模型不必要的复杂性。通过剔除这些变量——即，将相应的系数估计值设为零——我们可以获得一个更易于解释的模型。然而，最小二乘法极不可能产生任何恰好为零的系数估计。在本章中，我们将探讨一些自动执行*feature selection*或*variable selection*的方法——也就是说，

  用于从多元回归模型中排除无关变量。

feature selection
variable selection

对于拟合（6.1）而言，除了最小二乘法之外，还有许多经典与现代的替代方法。本章我们将讨论三类重要的方法。

- *Subset Selection*。这种方法涉及识别我们认为与响应相关的一部分 p 预测变量。然后，我们在缩减后的变量集上使用最小二乘法拟合模型。

- *Shrinkage*. 这种方法涉及拟合一个包含所有p个预测变量的模型。然而，相对于最小二乘估计，估计的系数会向零收缩。这种收缩（也称为*regularization*）具有减少方差的效果。根据执行的收缩类型，一些系数可能被估计为恰好为零。因此，收缩方法还可以执行变量选择。

- *Dimension Reduction*. 该方法涉及将p预测变量投影到M维子空间，其中$M < p$。这是通过计算变量的M个不同的*linear combinations*或*projections*来实现的。然后，这些M投影被用作预测变量，通过最小二乘法拟合线性回归模型。

在以下各节中，我们将更详细地描述这些方法，以及它们的优缺点。尽管本章描述了对第3章中线性回归模型的扩展和修改，但同样的概念也适用于其他方法，例如第4章中看到的分类模型。

[1]When $p \gg n$, the least squares solution that has the smallest sum of squared coefficients can sometimes perform quite well. See Section 10.8 for a more detailed discussion.

6.1 子集选择

在本节中，我们考虑一些选择预测变量子集的方法。这些方法包括最优子集和逐步模型选择过程。

6.1.1 Best Subset Selection

为了执行 {v*}，我们拟合一个单独的普通最小二乘回归对于p个预测变量的每一种可能组合。也就是说，我们拟合所有恰好包含一个预测变量的p个模型，所有恰好包含两个预测变量的$\binom{p}{2} = p(p-个1)/2$模型，以此类推。然后，我们查看所有生成的模型，目标是找出那个$best$的模型。

最佳子集选择

从最佳子集选择所考虑的2^p种可能性中选择$best\ model$的问题并不简单。这通常分为两个阶段，如算法6.1所述。

算法6.1 *Best subset selection*

---

1. 令$\mathcal{M}_0$表示$null\ model$，其中不含预测变量。该模型仅对每个观测值预测样本均值。

2. 对于 $k = 1, 2, \ldots p$:

   (a) 拟合所有恰好包含 k 个预测变量的 $\binom{p}{k}$ 模型。

   (b) 在这些 $\binom{p}{k}$ 模型中选择最佳的一个，称之为 $\mathcal{M}_k$。这里的 $best$ 定义为具有最小的 RSS，或等价地具有最大的 R^2。

3. 使用验证集上的预测误差、C_p (AIC)、BIC或调整后的R^2，从$\mathcal{M}_0, \ldots, \mathcal{M}_p$中选择一个最佳模型。或者使用交叉验证方法。

在算法6.1中，步骤2针对每个子集大小确定了（基于训练数据）最佳模型，从而将问题从2^p个可能模型缩减至$p+$个1可能模型。在图6.1中，这些模型构成了红色表示的下边界。

现在，为了选择一个最优模型，我们只需在这些$p+1$个选项中进行选择。这项任务必须谨慎执行，因为这些$p+1$个模型的RSS单调递减，而R^2随着模型中包含的特征数量增加而单调递增。因此，如果我们使用这些统计量来选择最佳模型，那么最终总是会得到一个包含所有变量的模型。问题在于，低RSS或高R^2表明模型具有较低的$training$误差，而我们希望选择的是具有较低$test$误差的模型。（如第2章图2.9-2.11所示，训练误差往往比测试误差小得多，低训练误差绝不能保证低测试误差。）因此，在第3步中，我们使用验证集上的误差、C_p、BIC或调整后的R^2来在$\mathcal{M}_0, \mathcal{M}_1, \ldots, \mathcal{M}_p$中进行选择。如果使用交叉验证来选择最佳模型，则对每个训练折重复第2步，并对验证误差取平均以选择k的最佳值。

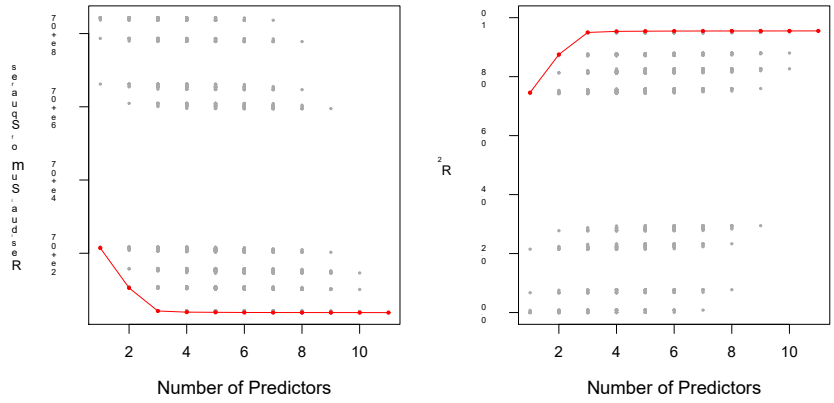


图6.1。 *For each possible model containing a subset of the ten predictors in the* `Credit` *data set, the RSS and* R^2 *are displayed. The red frontier tracks the* 最佳 *model for a given number of predictors, according to RSS and* R^2*. Though the data set contains only ten predictors, the* x*-axis ranges from* 1 *to* 11*, since one of the variables is categorical and takes on three values, leading to the creation of two dummy variables.*

然后将拟合完整训练集的模型 $\mathcal{M}_k$ 交付给所选的 k。这些方法将在第6.1.3节中讨论。

最佳子集选择的一个应用如图6.1所示。图中的每个点对应一个使用第3章讨论的`Credit`数据集中10个预测变量的不同子集拟合的最小二乘回归模型。这里，变量`region`是一个三水平定性变量，因此由两个虚拟变量表示，在这种情况下分别选择。因此，模型中可能包含的变量总共有11个。我们绘制了每个模型的RSS和R^2统计量，作为变量数量的函数。红色曲线连接了根据RSS或R^2为每个模型大小选择的最佳模型。该图显示，正如预期的那样，这些量随着变量数量的增加而改善；然而，从三变量模型开始，由于包含额外的预测变量，RSS和R^2的改善很小。

尽管我们在此针对最小二乘回归介绍了最佳子集选择，但这些理念同样适用于其他类型的模型，如逻辑回归。在逻辑回归的情况下，我们不再按照算法6.1第2步中的RSS对模型进行排序，而是使用*deviance*这一衡量标准。它在更广泛的一类模型中扮演着RSS的角色。偏差是最大化对数似然的负两倍；偏差越小，拟合越好。

deviance

尽管最优子集选择是一种简单且在概念上具有吸引力的方法，但它受到计算限制的困扰。随着p的增加，必须考虑的潜在模型数量迅速增长。一般来说，涉及p个预测变量子集的模型共有2^p个。因此，如果p =为10，那么大约有1,000个可能的模型需要考虑；而如果p =为20，则存在超过一百万种可能性！ consequently，当p大于

算法6.2 *Forward stepwise selection*

1. 令$\mathcal{M}_0$表示*null*模型，该模型不包含任何预测变量。

2. 对于 $k = 0, \ldots, p-1$:

   (a) 考虑所有$p-k$模型，这些模型在$\mathcal{M}_k$中的预测变量基础上增加一个额外的预测变量。(b) 在这些$p-k$模型中选择*best*，并称之为$\mathcal{M}_{k+1}$。这里*best*被定义为具有最小RSS或最高R^2。

3. 从$\mathcal{M}_0, \ldots, \mathcal{M}_p$中选择一个最佳模型，依据验证集上的预测误差、$C_p$ (AIC)、BIC或调整后的R^2。或者使用交叉验证方法。

大约40个，即使使用极快的现代计算机也是如此。有一些计算捷径——所谓的分支定界技术——可以消除一些选择，但当p变大时，这些技术有其局限性。它们也只适用于最小二乘线性回归。接下来我们介绍计算效率高的最佳子集选择替代方法。

6.1.2 Stepwise Selection

由于计算原因，最佳子集选择不能应用于非常大的 p。当 p 很大时，最佳子集选择也可能遭遇统计问题。搜索空间越大，找到在训练数据上表现良好的模型的机会就越高，尽管这些模型对未来数据可能没有任何预测能力。因此，庞大的搜索空间可能导致过拟合和系数估计的高方差。

由于这两个原因，探索模型集更为受限的*stepwise*方法，成为了优于最佳子集选择的有吸引力的替代方案。

前向逐步选择

Forward stepwise selection 是一种计算高效的替代方案，优于最佳方法子集选择。虽然最优子集选择过程考虑了包含p个预测变量子集的所有2^p个可能模型，但前向逐步选择考虑的模型集要小得多。前向逐步选择从一个不包含任何预测变量的模型开始，然后一次一个地向模型中添加预测变量，直到所有预测变量都在模型中。特别地，在每一步中，将能最大程度改善拟合的变量*additional*添加到模型中。更正式地说，前向逐步选择过程在算法6.2中给出。

forward stepwise selection

与最佳子集选择不同，后者涉及拟合2^p个模型，前向逐步选择涉及拟合一个零模型，以及在k次迭代中拟合$p-k$个模型，对于$k = 0, \ldots, p-1$。总计为$1 + \sum_{k=0}^{p-1}(p-k) = 1 + p(p+1)/2$个模型。这是一个显著的差异：当

# Variables	Best subset	Forward stepwise
One	`rating`	`rating`
Two	`rating`, `income`	`rating`, `income`
Three	`rating`, `income`, `student`	`rating`, `income`, `student`
Four	`cards`, `income`	`rating`, `income`,
	`student`, `limit`	`student`, `limit`

表格 6.1. *The first four selected models for best subset selection and forward stepwise selection on the* `Credit` *data set. The first three models are identical but the fourth models differ.*

$p = 20$，最佳子集选择需要拟合1,048,576个模型，而前向逐步选择仅需拟合211个模型。[2]

在算法6.2的第2(b)步中，我们必须从那些通过增加一个额外预测变量来扩充$\mathcal{M}_k$的$p-k$中识别出*best*模型。我们可以简单地选择具有最低RSS或最高R^2的模型来完成这一步。然而，在第3步中，我们必须在具有不同变量数量的一组模型中识别出最佳模型。这一步骤更具挑战性，将在第6.1.3节中进行讨论。

前向逐步选择相较于最优子集选择的计算优势是显而易见的。尽管前向逐步选择在实践中往往表现良好，但并不能保证在包含 p 个预测变量子集的所有 2^p 个模型中，找出最优可能模型。例如，假设在给定数据集中有 $p = 3$ 个预测变量，最优的单变量模型包含 X_1，而最优的双变量模型却包含 X_2 和 X_3。那么前向逐步选择将无法选出最优的双变量模型，因为 $\mathcal{M}_1$ 会包含 X_1，所以 $\mathcal{M}_2$ 也必须包含 X_1 以及另一个附加变量。

表6.1展示了在`Credit`数据集上进行最佳子集选择和前向逐步选择时选出的前四个模型，说明了这一现象。最佳子集选择和前向逐步选择都选择`rating`作为最佳单变量模型，然后分别在双变量和三变量模型中纳入`income`和`student`。然而，最佳子集选择在四变量模型中将`rating`替换为`cards`，而前向逐步选择则必须在其四变量模型中保留`rating`。在这个例子中，图6.1表明三变量模型和四变量模型在RSS方面差异不大，因此任一四变量模型可能都是足够的。

前向逐步选择甚至可以在高维设置中应用，即使$n < p$；然而，在这种情况下，只能构建子模型$\mathcal{M}_0, \ldots, \mathcal{M}_{n-1}$，因为每个子模型都是使用最小二乘法拟合的，如果$p \geq n$，则不会得到唯一解。

向后逐步选择

与前向逐步选择类似，*backward stepwise selection* 提供了一种

向后逐步选择

[2]Though forward stepwise selection considers $p(p+1)/2+1$ models, it performs a *guided* search over model space, and so the *effective* model space considered contains substantially more than $p(p+1)/2+1$ models.

高效替代最优子集选择的方法。然而，与前向逐步选择不同，它从包含所有 p 个预测变量的完整最小二乘模型开始，然后逐一迭代移除最无用的预测变量。详情见算法6.3。

算法 6.3 *Backward stepwise selection*

1. 令 $\mathcal{M}_p$ 表示 *full* 模型，该模型包含所有 p 预测变量。
2. 对于 $k = p, p-1, \ldots, 1$:
   (a) 考虑所有k模型，它们包含$\mathcal{M}_k$中除一个预测变量外的所有预测变量，总共有k –个1预测变量。(b) 在这些k模型中选择*best*，并称之为$\mathcal{M}_{k-1}$。这里*best*定义为具有最小RSS或最高R^2。
3. 从$\mathcal{M}_0, \ldots, \mathcal{M}_p$中根据验证集上的预测误差、$C_p$ (AIC)、BIC或调整后的R^2选择一个最佳模型。或者使用交叉验证方法。

与前向逐步选择类似，后向选择方法仅搜索$1+p(p+1)/2$个模型，因此适用于p太大而无法应用最佳子集选择的情况。[3]同样与前向逐步选择相似，后向逐步选择也不保证能够得到包含p个预测变量子集的*best*模型。

向后选择要求样本数量 n 大于变量数量 p (，以便能够拟合完整模型)。相比之下，即使 $n < p$，前向逐步回归也可以使用，因此当 p 非常大时，它是唯一可行的子集方法。

混合方法

最佳子集、前向逐步和后向逐步选择方法通常给出相似但不完全相同的模型。作为另一种选择，还可以使用前向和后向逐步选择的混合版本，其中变量按顺序添加到模型中，类似于前向选择。然而，在添加每个新变量之后，该方法还可能移除任何不再对模型拟合提供改进的变量。这种方法试图更接近地模拟最佳子集选择，同时保留前向和后向逐步选择的计算优势。

6.1.3 Choosing the Optimal Model

最佳子集选择、向前选择和向后选择会生成一组模型，每个模型都包含 p 的一个子集。

[3]Like forward stepwise selection, backward stepwise selection performs a *guided* search over model space, and so effectively considers substantially more than $1 + p(p + 1)/2$ models.

预测变量。为了应用这些方法，我们需要一种方式来确定这些模型中哪一个是 *best*。正如我们在第6.1.1节中讨论的，包含所有预测变量的模型总是具有最小的RSS和最大的 R^2，因为这些量与训练误差相关。相反，我们希望选择一个测试误差较低的模型。正如这里所显而易见的，并且如我们在第2章中所示，训练误差可能是测试误差的一个糟糕估计。因此，RSS和 R^2 不适合在具有不同数量预测变量的模型集合中选择最佳模型。

为了根据测试误差选择最佳模型，我们需要估计这个测试误差。有两种常见的方法：

1. 我们可以通过向训练误差添加一个 *adjustment* 来间接估计测试误差，以考虑由于过拟合带来的偏差。

2. 我们可以使用 *directly* 估计测试误差，采用验证集方法或交叉验证方法，如第5章所述。

我们在下面考虑这两种方法。

C_p、AIC、BIC 和调整后的 R^2

我们在第2章中展示，训练集MSE通常是测试MSE的一个低估。（回顾MSE $= \text{RSS}/n$。）这是因为当我们使用最小二乘法将模型拟合到训练数据时，我们特别估计回归系数，使得训练RSS（而非测试RSS）尽可能小。特别是，随着模型中包含更多变量，训练误差会减少，但测试误差可能不会。因此，训练集RSS和训练集 R^2 不能用于从具有不同变量数量的模型集合中进行选择。

然而，针对模型大小，有多种技术可用于 *adjusting* 训练误差。这些方法可用于从一组具有不同变量数量的模型中进行选择。我们现在考虑四种这样的方法：C_p、*Akaike information criterion* (AIC)、*Bayesian information criterion* (BIC)，以及 *adjusted* R^2。图6.2显示了 C_p、BIC和调整后的 R 在 `Credit` 数据集上通过最佳子集选择生成的每种尺寸的最佳模型的2。

C_p 赤池信息量准则

贝叶斯信息准则调整后的 R^2

对于包含 d 个预测变量的拟合最小二乘模型，测试 MSE 的 C_p 估计值使用以下方程计算：

$$C_p = \frac{1}{n}\left(\text{RSS} + 2d\hat{\sigma}^2\right), \tag{6.2}$$

其中 $\hat{\sigma}^2$ 是 (6.1) 中与每个响应测量相关联的误差 ϵ 的方差的估计。[4] 通常 $\hat{\sigma}^2$ 使用包含所有预测变量的完整模型来估计。本质上，C_p 统计量在训练 RSS 上添加 $2d\hat{\sigma}^2$ 的惩罚，以调整训练误差倾向于低估测试误差这一事实。显然，随着模型中预测变量数量的增加，惩罚也会增加；这是为了调整

[4] Mallow's C_p is sometimes defined as $C_p' = \text{RSS}/\hat{\sigma}^2 + 2d - n$. This is equivalent to the definition given above in the sense that $C_p = \frac{1}{n}\hat{\sigma}^2(C_p' + n)$, and so the model with smallest C_p also has smallest C_p'.

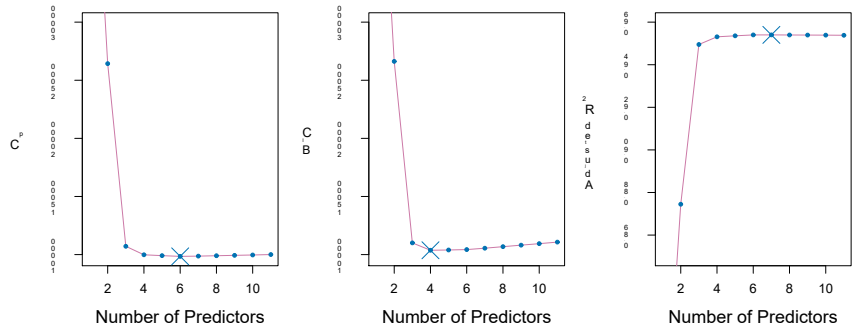


图6.2。*C_p, BIC, and adjusted R^2 are shown for the best models of each size for the* `Credit` *data set (the lower frontier in Figure 6.1). C_p and BIC are estimates of test MSE. In the middle plot we see that the BIC estimate of test error shows an increase after four variables are selected. The other two plots are rather flat after four variables are included.*

对于训练RSS的相应减少。尽管这超出了本书的范围，但可以证明，如果(6.2)中的$\hat{\sigma}^2$是σ^2的无偏估计，那么C_p是测试MSE的无偏估计。因此，对于测试误差低的模型，C_p统计量往往取值较小，所以在确定一组模型中哪个最佳时，我们选择C_p值最低的模型。在图6.2中，C_p选择了包含预测变量`income`、`limit`、`rating`、`cards`、`age`和`student`的六变量模型。

AIC准则定义于一大类通过最大似然拟合的模型中。对于(6.1)式中含有高斯误差的模型，最大似然与最小二乘法本质相同。在这种情况下，AIC由下式给出：

$$\text{AIC} = \frac{1}{n}\left(\text{RSS} + 2d\hat{\sigma}^2\right),$$

其中，为简化起见，我们省略了无关常数。[5] 因此，对于最小二乘模型，C_p 与 AIC 成比例，所以在图6.2中仅显示了 C_p。

BIC源自贝叶斯视角，但最终看起来也与C_p (和AIC)相似。对于包含d个预测变量的最小二乘模型，BIC在不考虑无关常数的情况下，由下式给出：

$$\text{BIC} = \frac{1}{n}\left(\text{RSS} + \log(n)d\hat{\sigma}^2\right). \tag{6.3}$$

与C_p类似，对于测试误差较低的模型，BIC倾向于取较小的值，因此我们通常选择BIC值最低的模型。注意，BIC将C_p所使用的$2d\hat{\sigma}^2$替换为$\log(n)$ $d\hat{\sigma}^2$项，其中n是观测数。由于对于任意$n > 7$，$\log n > 2$，

[5] There are two formulas for AIC for least squares regression. The formula that we provide here requires an expression for σ^2, which we obtain using the full model containing all predictors. The second formula is appropriate when σ^2 is unknown and we do not want to explicitly estimate it; that formula has a log(RSS) term instead of an RSS term. Detailed derivations of these two formulas are outside of the scope of this book.

BIC统计量通常对具有许多变量的模型施加更重的惩罚，因此导致选择比C_p更小的模型。在图6.2中，我们看到对于Credit数据集确实如此；BIC选择了一个仅包含四个预测变量income、limit、cards和student的模型。在这种情况下，曲线非常平坦，因此四变量模型和六变量模型在准确性上似乎没有太大差异。

调整后的 R^2 统计量是在包含不同变量数量的模型集合中进行选择的另一种流行方法。回顾第3章，通常的 R^2 定义为 $1 - \text{RSS}/\text{TSS}$，其中 $\text{TSS} = \sum(y_i - \underline{y})^2$ 是响应的 *total sum of squares*。由于随着更多变量加入模型，RSS 总是减少，因此 R^2 总是随着更多变量的加入而增加。对于一个具有 d 变量的最小二乘模型，调整后的 R^2 统计量计算为

$$\text{Adjusted } R^2 = 1 - \frac{\text{RSS}/(n-d-1)}{\text{TSS}/(n-1)}. \tag{6.4}$$

与 C_p、AIC 和 BIC 不同，*small*值表示测试误差较低的模型，而调整后的 R^2 的 *large* 值表示测试误差较小的模型。最大化调整后的 R^2 等价于最小化 $\frac{\text{RSS}}{n-d-1}$。虽然 RSS 总是随着模型中变量数量的增加而减小，但由于分母中存在 d，$\frac{\text{RSS}}{n-d-1}$ 可能增大或减小。

调整后的R^2背后的直觉是，一旦所有正确的变量都被纳入模型，添加额外的*noise*变量只会导致RSS非常小的下降。由于添加噪声变量会导致d增加，这样的变量将导致$\frac{\text{RSS}}{n-d-1}$增加，进而导致调整后的R^2减少。因此，理论上，调整后的R^2最大的模型将仅包含正确变量而不包含噪声变量。与R^2统计量不同，调整后的R^2统计量*paysa price*模型中包含不必要的变量。图6.2展示了Credit数据集的调整后的R^2。使用此统计量导致选择一个包含七个变量的模型，在由C_p和AIC选择的模型基础上增加了own。

C_p、AIC 和 BIC 都有严格的理论依据，这些依据超出了本书的范围。这些依据依赖于渐近论证（样本量 n 非常大的情形）。尽管调整后的 R^2 很受欢迎，甚至相当直观，但它在统计理论中的动机不如 AIC、BIC 和 C_p 那样充分。所有这些度量都易于使用和计算。这里我们展示了它们在使用最小二乘法拟合线性模型情况下的公式；然而，AIC 和 BIC 也可以为更一般的模型类型定义。

验证与交叉验证

作为刚刚讨论方法的替代方案，我们可以直接使用第5章中讨论的验证集和交叉验证方法来估计测试误差。我们可以计算所考虑每个模型的验证集误差或交叉验证误差，然后选择

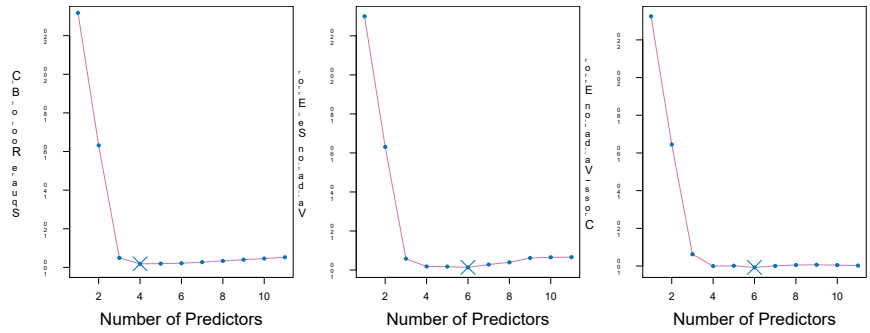


图 6.3. *For the* `Credit` *data set, three quantities are displayed for the best model containing* d *predictors, for* d *ranging from* 1 *to* 11. *The overall* best *model, based on each of these quantities, is shown as a blue cross.* 左：*Square root of BIC.* 中：*Validation set errors.* 右：*Cross-validation errors.*

交叉验证中选择使估计测试误差最小的模型。相对于AIC、BIC、C_p和调整后的R^2，该方法的优势在于它直接提供了测试误差的估计，并且对真实的底层模型做了更少的假设。它还可以用于更广泛的模型选择任务，即使在难以确定模型自由度（例如模型中预测变量的数量）或难以估计误差方差σ^2的情况下也是如此。注意，当使用交叉验证时，算法6.1–6.3中的模型序列$\mathcal{M}_k$是针对每个训练折分别确定的，并且对于每个模型大小k，验证误差在所有折上取平均。这意味着，例如在最优子集回归中，$\mathcal{M}_k$，即大小为k的最优子集，在不同折之间可能不同。一旦选定了最优大小k，我们就在完整数据集上找到该大小的最优模型。

过去，对于许多具有大的p和/或大的n的问题来说，执行交叉验证在计算上是令人望而却步的，因此AIC、BIC、C_p和调整后的R^2在从一组模型中选择模型时更具吸引力。然而，如今有了快速计算机，执行交叉验证所需的计算几乎不再是问题。因此，交叉验证是从众多考虑中的模型中进行选择的一种非常有吸引力的方法。

图6.3展示了在`Credit`数据上，最佳d变量模型的BIC、验证集误差和交叉验证误差随d变化的情况。验证误差通过随机选取四分之三的观测值作为训练集，其余作为验证集来计算。交叉验证误差采用k =的10折计算。在此情况下，验证和交叉验证方法均得出六变量模型。然而，三种方法都表明四变量、五变量和六变量模型在其测试误差方面大致相当。

事实上，图6.3中间和右侧面板显示的估计测试误差曲线相当平坦。虽然三变量模型的估计测试误差明显低于二变量模型，但3至11变量模型的估计测试误差非常相似。此外，如果我们

重复使用不同的数据划分方式将数据分为训练集和验证集来进行验证集方法，或者如果我们使用不同的交叉验证折叠集重复交叉验证，那么估计测试误差最低的精确模型肯定会改变。在这种情况下，我们可以使用*one-standard-error rule*来选择模型。我们首先计算

一标准差规则

每个模型大小的估计测试MSE的标准误差，然后选择估计测试误差在曲线最低点的一个标准误差范围内的最小模型。这里的理由是，如果一组模型看起来差不多同样好，那么我们不妨选择最简单的模型——即预测变量数量最少的模型。在这种情况下，对验证集或交叉验证方法应用一标准误差规则会导致选择三变量模型。

6.2 收缩方法

第6.1节中描述的子集选择方法涉及使用最小二乘法来拟合包含预测变量子集的线性模型。作为替代方案，我们可以使用一种技术来拟合包含所有p预测变量的模型，该技术*constrains*或*regularizes*系数估计，或者等价地，将系数估计*shrinks*向零。为什么这样的约束应该改善拟合可能并不立即显而易见，但事实证明，收缩系数估计可以显著降低其方差。将回归系数向零收缩的两种最著名技术是*ridge regression*和*lasso*。

6.2.1 Ridge Regression

回想第3章，最小二乘拟合程序使用使下式最小化的值来估计$\beta_0, \beta_1, \ldots, \beta_p$

$$\text{RSS} = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 .$$

Ridge regression 与最小二乘法非常相似，只是岭回归的系数是通过最小化一个略有不同的量来估计的。特别地，岭回归系数估计值$\hat{\beta}^R$是使下式最小化的值：

$$\sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 = \text{RSS} + \lambda \sum_{j=1}^{p} \beta_j^2, \qquad (6.5)$$

其中 $\lambda \geq 0$ 是一个 *tuning parameter*，需单独确定。方程 6.5 节权衡了两个不同的标准。与最小二乘法一样，岭回归寻求能够很好地拟合数据的系数估计，通过使 RSS 较小。然而，第二项，$\lambda \sum_j \beta_j^2$，称为 *shrinkage penalty*，是当$\beta_1, \ldots, \beta_p$接近零时，{v*}*shrinking*的值较小，因此它具有将β_j的估计值向零调整的效果。调优参数λ用于控制

调优参数

收缩惩罚

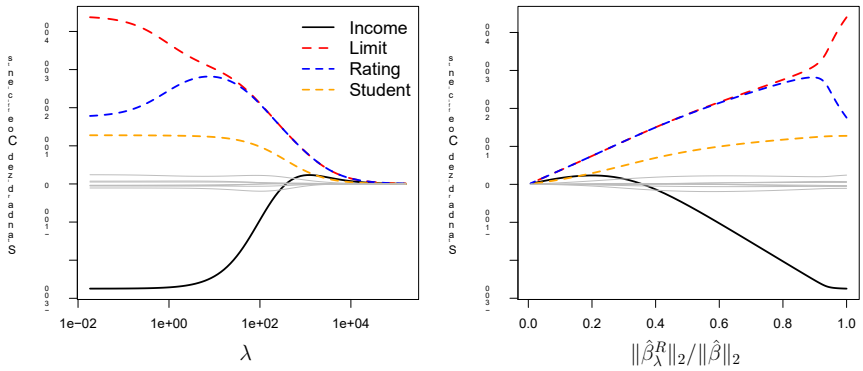


图6.4。*The standardized ridge regression coefficients are displayed for the* `Credit` *data set, as a function of* λ *and* $\|\hat{\beta}^R_\lambda\|_2/\|\hat{\beta}\|_2$.

这两个项对回归系数估计值的相对影响。当 $\lambda = 0$ 时，惩罚项不起作用，岭回归将产生最小二乘估计。然而，随着 $\lambda \to \infty$，收缩惩罚的影响增大，岭回归系数估计将趋近于零。与最小二乘法仅生成一组系数估计不同，岭回归将为每个 λ 值生成一组不同的系数估计 $\hat{\beta}^R_\lambda$。为 λ 选择一个好的值至关重要；我们将这一讨论推迟到第6.2.3节，在那里我们使用交叉验证。

注意，在(6.5)中，收缩惩罚应用于$\beta_1, \ldots, \beta_p$，但不应用于截距β_0。我们希望缩小每个变量与响应之间估计的关联；然而，我们不希望缩小截距，截距仅仅是当$x_{i1} = x_{i2} = \ldots = x_{ip}$ =为0时响应平均值的度量。如果我们假设变量——即数据矩阵$\mathbf{X}$的列——在进行岭回归之前已经中心化，使其均值为零，那么估计的截距将采取$\hat{\beta}_0 = \bar{y} = \sum_{i=1}^{n} y_i/n$的形式。

信贷数据的一个应用

在图6.4中，展示了针对`Credit`数据集的岭回归系数估计。左侧面板中，每条曲线代表十个变量之一的岭回归系数估计，作为λ的函数绘制。例如，黑色实线表示`income`系数的岭回归估计，随着λ的变化而变化。在图的极左侧，λ基本为零，因此相应的岭回归系数估计与普通最小二乘估计相同。但随着λ的增加，岭回归系数估计向零收缩。当λ极大时，所有岭回归系数估计基本上为零；这对应于不包含任何预测变量的*null model*。在此图中，`income`、`limit`、`rating`和`student`变量以不同的颜色显示，因为这些变量的系数估计往往远大于其他变量。虽然随着λ的增加，岭回归系数估计总体上趋于减小，但个别系数，如`rating`和`income`，有时可能会随着λ的增加而增大。

图6.4的右侧面板展示了与左侧面板相同的岭回归系数估计，但不是在x轴上显示λ，我们现在显示$\|\hat{\beta}_\lambda^R\|_2/\|\hat{\beta}\|_2$，其中$\hat{\beta}$表示最小二乘系数估计的向量。符号$\|\beta\|_2$表示$\ell_2$ *norm* (显著的

ℓ_2 norm

向量的“ell 2”)，定义为$\|\beta\|_2 = \sqrt{\sum_{j=1}^{p} \beta_j{}^2}$。它衡量$\beta$到零的距离。随着$\lambda$增加，$\hat{\beta}_\lambda^R$的$\ell_2$范数会*always*减小，$\|\hat{\beta}_\lambda^R\|_2/\|\hat{\beta}\|_2$也会如此。后者的值范围从1（当$\lambda$ =为0时，此时岭回归系数估计与最小二乘估计相同，因此它们的ℓ_2范数相同）到0（当$\lambda = \infty$时，此时岭回归系数估计是一个零向量，其ℓ_2范数等于零）。因此，我们可以认为图6.4右侧面板中的x轴表示岭回归系数估计向零收缩的程度；值越小，表示它们被收缩得越接近零。

第3章讨论的标准最小二乘系数估计*scale equivariant*：将X_j乘以常数c只会导致

最小二乘系数估计按因子$1/c$缩放。换言之，无论第j个预测变量如何缩放，$X_j\hat{\beta}_j$都将保持不变。相比之下，当给定预测变量乘以常数时，岭回归系数估计会*sub-stantially*变化。例如，考虑`income`变量，以美元计量。可以合理地以千美元计量收入，这将导致`income`的观测值缩小1000倍。现在，由于岭回归公式(6.5)中的系数平方和项，这种尺度变化不会简单地使`income`的岭回归系数估计变化1000倍。换言之，$X_j\hat{\beta}_{j,\lambda}^R$不仅取决于$\lambda$的值，还取决于第$j$个预测变量的缩放。事实上，$X_j\hat{\beta}_{j,\lambda}^R$的值甚至可能取决于*other*个预测变量的缩放！因此，最好在*standardizing the predictors*之后应用岭回归，使用公式

scale equivariant

$$\tilde{x}_{ij} = \frac{x_{ij}}{\sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_{ij} - \overline{x}_j)^2}}, \tag{6.6}$$

因此它们都处于相同的尺度上。在(6.6)中，分母是第j个预测变量的估计标准差。因此，所有标准化后的预测变量都将具有标准差为一。结果，最终拟合不会依赖于预测变量测量的尺度。在图6.4中，y轴显示了标准化的岭回归系数估计——即使用标准化预测变量进行岭回归所得到的系数估计。

为什么岭回归优于最小二乘法?

岭回归相对于最小二乘法的优势源于 *bias-variance trade-off*。随着 λ 的增加，岭回归拟合的灵活性降低，导致方差减小但偏差增大。这一点在图6.5的左侧面板中得到了展示，使用的是包含 $p = 45$个预测变量和 $n = 50$个观测值的模拟数据集。左侧面板中的绿色曲线

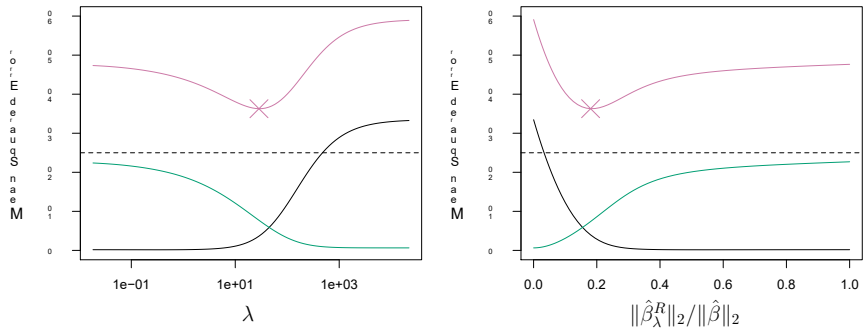


图6.5。*Squared bias (black), variance (green), and test mean squared error (purple) for the ridge regression predictions on a simulated data set, as a function of* λ *and* $\|\hat{\beta}^R_\lambda\|_2/\|\hat{\beta}\|_2$*. The horizontal dashed lines indicate the minimum possible MSE. The purple crosses indicate the ridge regression models for which the MSE is smallest.*

图6.5展示了岭回归预测的方差作为λ的函数。在最小二乘系数估计处，对应于λ =为0的岭回归，方差很高但没有偏差。然而，随着λ的增加，岭回归系数估计的收缩导致预测方差大幅减少，代价是偏差略有增加。回想一下，以紫色绘制的测试均方误差（MSE）与方差加上偏差的平方密切相关。对于λ的值直到约10，方差迅速下降，而偏差（以黑色绘制）增加非常少。因此，当λ从0增加到10时，MSE显著下降。超过这一点后，由于λ增加而导致的方差下降减缓，并且系数的收缩导致它们被显著低估，从而导致偏差大幅增加。最小MSE在大约λ =为30时实现。有趣的是，由于其高方差，当λ =为0时，与最小二乘拟合相关的MSE几乎与所有系数估计为零的零模型一样高，当$\lambda = \infty$时。然而，对于λ的中间值，MSE要低得多。

图6.5右侧面板展示了与左侧面板相同的曲线，这次是相对于岭回归系数估计的ℓ_2范数除以其最小二乘估计的ℓ_2范数绘制的。现在，当我们从左向右移动时，拟合变得更灵活，因此偏差减小，方差增大。

一般来说，在响应变量与预测变量之间的关系接近线性的情况下，最小二乘估计将具有低偏差但可能具有高方差。这意味着训练数据的微小变化可能导致最小二乘系数估计的巨大变化。特别地，当变量数量 p 几乎与观测数量 n 一样大时，如图6.5中的例子所示，最小二乘估计将极其多变。并且如果 $p > n$，那么最小二乘估计甚至没有唯一解，而岭回归仍然可以通过以偏差的小幅增加换取良好的表现。

方差大幅下降。因此，岭回归在最小二乘估计具有高方差的情况下效果最好。

岭回归相比最优子集选择也具有显著的计算优势，后者需要搜索2^p个模型。正如我们之前所讨论的，即使对于中等大小的p值，这样的搜索在计算上也可能不可行。相比之下，对于任何固定的λ值，岭回归只拟合一个单一模型，且模型拟合过程可以相当快速地完成。事实上，可以证明求解(6.5)所需的计算，*simultaneously for all values of* λ，与使用最小二乘法拟合模型的计算几乎相同。

6.2.2 The Lasso

岭回归确实有一个明显的缺点。与最佳子集、前向逐步和后向逐步选择通常会选择仅涉及变量子集的模型不同，岭回归会在最终模型中包含所有 p 个预测变量。公式 (6.5) 中的惩罚项 $\lambda \sum \beta_j^2$ 会将所有系数向零收缩，但不会将其中任何一个系数精确地设为零（除非 $\lambda = \infty$）。这对预测准确性可能不是问题，但在变量数量 p 相当大的情况下，它可能会给模型解释带来挑战。例如，在 `Credit` 数据集中，最重要的变量似乎是 `income`、`limit`、`rating` 和 `student`。因此，我们可能希望构建一个仅包含这些预测变量的模型。然而，岭回归总是会生成一个涉及全部十个预测变量的模型。增大 λ 的值往往会减小系数的大小，但不会导致排除任何变量。

lasso 是相对较新的岭回归替代方法，克服了这一缺点。套索系数 $\hat{\beta}_\lambda^L$ 最小化以下量

$$\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=1}^{p}|\beta_j| \;= \mathrm{RSS} + \lambda\sum_{j=1}^{p}|\beta_j|. \qquad (6.7)$$

比较（6.7）与（6.5），我们可以看到套索回归和岭回归有相似的公式。唯一的区别是岭回归惩罚项（6.5）中的β_j^2项被套索惩罚项（6.7）中的$|\beta_j|$所取代。用统计术语来说，套索使用的是一个ℓ_1 (读作“ell 1”)惩罚项，而不是ℓ_2惩罚项。系数向量β的ℓ_1范数由$\|\beta\|_1 = \sum|\beta_j|$给出。

与岭回归一样，lasso 将系数估计向零收缩。然而，在 lasso 的情况下，当调优参数 λ 足够大时，ℓ_1 惩罚具有迫使某些系数估计恰好等于零的效果。因此，与最佳子集选择非常相似，lasso 执行 *variable selection*。因此，由 lasso 生成的模型通常比岭回归生成的模型更容易解释。我们说 lasso 产生 *sparse* 模型——也就是说，

sparse

仅涉及变量子集的模型。如同岭回归一样，为套索选择恰当的 λ 值至关重要；我们将在第6.2.3节使用交叉验证时详细讨论这一点。

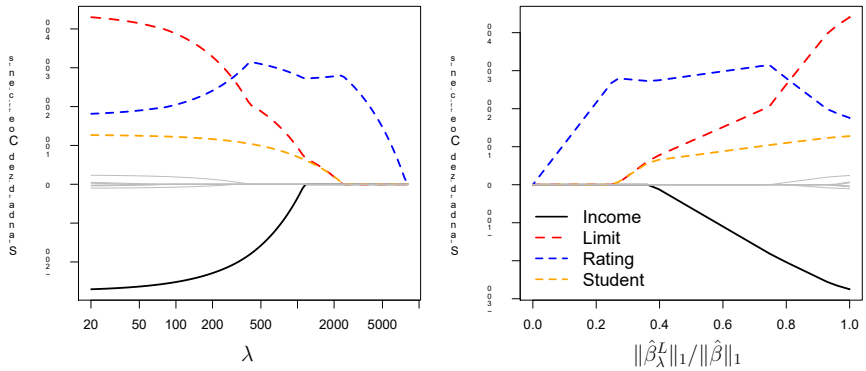


图6.6。*The standardized lasso coefficients on the* `Credit` *data set are shown as a function of* λ *and* $\|\hat{\beta}^L_\lambda\|_1/\|\hat{\beta}\|_1$.

例如，考虑图6.6中的系数图，这些图是通过将套索应用于`Credit`数据集生成的。当λ =为0时，套索简单地给出最小二乘拟合，而当λ变得足够大时，套索给出所有系数估计都等于零的零模型。然而，在这两个极端之间，岭回归和套索模型彼此相当不同。在图6.6右侧面板中从左向右移动，我们观察到起初套索产生的模型仅包含`rating`预测变量。然后`student`和`limit`几乎同时进入模型，紧随其后是`income`。最终，其余变量进入模型。因此，根据λ的值，套索可以产生涉及任意数量变量的模型。相比之下，岭回归将始终在模型中包含所有变量，尽管系数估计的大小将取决于λ。

岭回归和套索的另一种公式化方法

可以证明，套索和岭回归的系数估计解决了以下问题：

$$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^{p} |\beta_j| \leq s \tag{6.8}$$

和

$$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^{p} \beta_j^2 \leq s, \tag{6.9}$$

分别地。换句话说，对于λ的每一个值，都存在某个s，使得方程(6.7)和(6.8)将给出相同的lasso系数估计。类似地，对于λ的每一个值，都存在一个对应的s，使得方程(6.5)和(6.9)将给出相同的岭回归系数估计。

当 $p = 2$ 时，(6.8) 表明套索系数估计在由 $|\beta_1| + |\beta_2| \leq s$ 定义的菱形内所有点中具有最小的 RSS。类似地，岭回归估计在由 $\beta_1^2 + \beta_2^2 \leq s$ 定义的圆内所有点中具有最小的 RSS。

我们可以这样理解(6.8)。当我们执行套索回归时，我们试图找到一组系数估计，使得在{4}对$\sum_{j=1}^{p} |\beta_j|$大小的限制下，达到最小的RSS。当s极大时，这个预算并不十分受限，因此系数估计可以很大。事实上，如果s足够大，使得最小二乘解落在预算之内，那么(6.8)将直接给出最小二乘解。相反，如果s很小，那么$\sum_{j=1}^{p} |\beta_j|$也必须很小，以避免超出预算。同样，(6.9)表明，当我们进行岭回归时，我们寻找一组系数估计，使得在满足$\sum^{p}$的要求下，RSS尽可能小。

$_{j=1}\beta 2_j$不超过预算s。

公式 (6.8) 和 (6.9) 揭示了套索、岭回归和最佳子集选择之间的密切联系。考虑问题

$$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^{p} I(\beta_j \neq 0) \leq s. \tag{6.10}$$

这里 $I(\beta_j \neq 0)$ 是一个指示变量：如果 $\beta_j \neq 0$，则取值为1，否则等于零。那么（6.10）就等于找到一组系数估计，使得在非零系数不超过 s 个的约束下，RSS尽可能小。问题（6.10）等价于最佳子集选择。不幸的是，当 p 很大时，求解（6.10）在计算上是不可行的，因为它需要考虑所有包含 s 个预测变量的 $\binom{p}{s}$ 个模型。因此，我们可以将岭回归和套索解释为最佳子集选择的计算上可行的替代方案，它们用更容易求解的形式替换了（6.10）中难以处理的预算形式。当然，套索与最佳子集选择的关系更为密切，因为在（6.8）中当 s 足够小时，套索执行特征选择，而岭回归则不会。

套索的变量选择性质

为什么lasso与岭回归不同，其系数估计值会恰好等于零？公式(6.8)和(6.9)可以用来阐明这个问题。图6.7展示了这一情况。最小二乘解标记为$\hat{\beta}$，而蓝色菱形和圆形分别代表(6.8)和(6.9)中的lasso和岭回归约束。如果s足够大，那么约束区域将包含$\hat{\beta}$，因此岭回归和lasso估计将与最小二乘估计相同。（如此大的s值对应于(6.5)和(6.7)中的$\lambda = 0$。）然而，在图6.7中，最小二乘估计位于菱形和圆形之外，因此最小二乘估计与lasso和岭回归估计不同。

围绕$\hat{\beta}$的每个椭圆代表一个*contour*：这意味着特定椭圆上的所有点都具有相同的RSS值。如同

contour

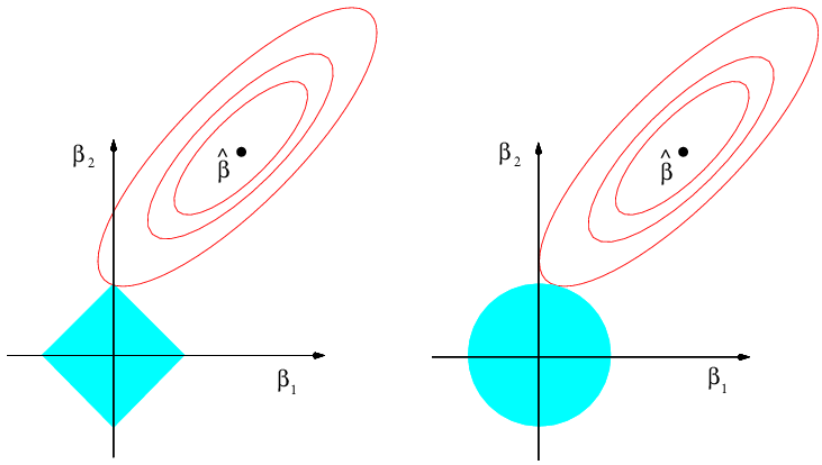


图 6.7. *Contours of the error and constraint functions for the lasso* (left) *and ridge regression* (right). *The solid blue areas are the constraint regions,* $|\beta_1|+|\beta_2| \leq s$ *and* $\beta_1^2+\beta_2^2 \leq s$, *while the red ellipses are the contours of the RSS.*

椭圆从最小二乘系数估计值向外扩展时，RSS增加。方程(6.8)和(6.9)表明套索和岭回归系数估计值由椭圆与约束区域接触的第一个点给出。由于岭回归具有没有尖点的圆形约束，这种相交通常不会发生在坐标轴上，因此岭回归系数估计值将全部非零。然而，套索约束在每个坐标轴上都有*corners*，因此椭圆通常会与约束区域在坐标轴上相交。当这种情况发生时，其中一个系数将等于零。在更高维度中，许多系数估计值可能同时等于零。在图6.7中，相交发生在$\beta_1 = 0$处，因此得到的模型将仅包含β_2。

在图6.7中，我们考虑了p ==2的简单情况。当p ==3时，岭回归的约束区域变为一个球体，而套索的约束区域变为一个多面体。当p >=3时，岭回归的约束变为超球体，套索的约束变为多胞形。然而，图6.7所描绘的关键思想仍然成立。特别地，当p >=2时，由于多面体或多胞形的尖锐角，套索导致特征选择。

比较 Lasso 和岭回归

显然，lasso相较于岭回归具有显著优势，因为它能生成更简单、更易于解释的模型，且仅涉及预测变量子集。然而，哪种方法能带来更好的预测精度呢？图6.8展示了应用于图6.5相同模拟数据的lasso的方差、平方偏差和测试MSE。显然，lasso在定性上表现出与岭回归相似的行为，即随着λ的增加，方差减小而偏差增大。在右侧

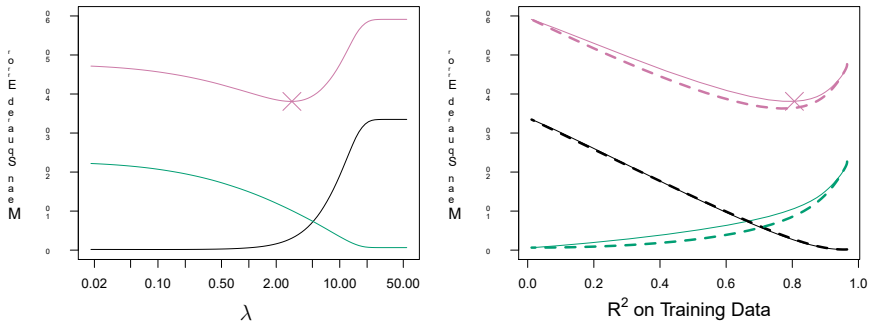


图 6.8. 左：*Plots of squared bias (black), variance (green), and test MSE (purple) for the lasso on a simulated data set.* 右：*Comparison of squared bias, variance, and test MSE between lasso (solid) and ridge (dotted). Both are plotted against their* R^2 *on the training data, as a common form of indexing. The crosses in both plots indicate the lasso model for which the MSE is smallest.*

图6.8的面板中，虚线代表岭回归拟合结果。此处，我们将两者与其在训练数据上的R^2进行对比绘制。这是另一种有用的模型索引方式，并能用于比较采用不同类型正则化的模型，如此例所示。在此示例中，套索回归与岭回归产生的偏差几乎相同。然而，岭回归的方差略低于套索回归的方差。因此，岭回归的最小MSE略小于套索回归的最小MSE。

然而，图6.8中的数据是以这样一种方式生成的：所有45个预测变量都与响应变量相关——也就是说，没有任何一个真实系数$\beta_1, \ldots, \beta_{45}$等于零。Lasso隐含地假定许多系数实际上等于零。因此，在这种情况下，岭回归在预测误差方面优于Lasso并不令人意外。图6.9展示了类似的情形，只不过现在响应变量仅是45个预测变量中2个的函数。此时，Lasso在偏差、方差和MSE方面往往优于岭回归。

这两个例子表明，岭回归和套索回归都不会普遍优于对方。一般来说，在相对较少的预测变量具有显著系数，而其余预测变量的系数非常小或等于零的情况下，人们可能会预期套索回归表现更好。当响应变量是许多预测变量的函数，且所有预测变量的系数大小大致相等时，岭回归将表现更好。然而，对于真实数据集，与响应变量相关的预测变量数量从来都是未知的*a priori*。可以使用诸如交叉验证之类的技术来确定在特定数据集上哪种方法更好。

与岭回归相似，当最小二乘估计的方差过大时，套索解能在偏差略微增加的情况下减少方差，从而能生成更准确的预测。与岭回归不同，套索方法进行了变量选择，因此得到的模型更易于解释。

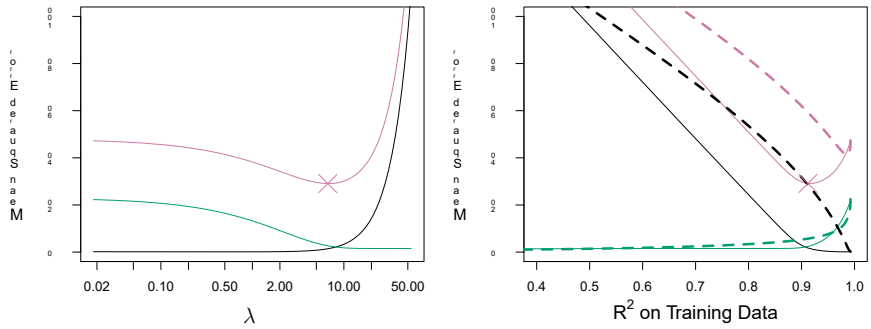


图6.9. 左：*Plots of squared bias (black), variance (green), and test MSE (purple) for the lasso. The simulated data is similar to that in Figure 6.8, except that now only two predictors are related to the response.* 右：*Comparison of squared bias, variance, and test MSE between lasso (solid) and ridge (dotted). Both are plotted against their R^2 on the training data, as a common form of indexing. The crosses in both plots indicate the lasso model for which the MSE is smallest.*

有非常高效的算法来拟合岭回归和套索模型；在这两种情况下，计算整个系数路径的工作量与一次最小二乘拟合大致相同。我们将在本章末尾的实验室中进一步探讨这一点。

岭回归和套索的一个简单特例

为了更好地直观理解岭回归和套索的行为，考虑一个简单的特例，其中 $n = p$，且 $\mathbf{X}$ 是一个对角矩阵，对角线上的元素为1，所有非对角线元素为0。为了进一步简化问题，还假设我们在没有截距的情况下进行回归。基于这些假设，通常的最小二乘问题简化为寻找 $\beta_1, \ldots, \beta_p$ 以最小化

$$\sum_{j=1}^{p}(y_j - \beta_j)^2. \tag{6.11}$$

在这种情况下，最小二乘解由下式给出：

$$\hat{\beta}_j = y_j.$$

在这种情况下，岭回归相当于找到 $\beta_1, \ldots, \beta_p$，使得

$$\sum_{j=1}^{p}(y_j - \beta_j)^2 + \lambda\sum_{j=1}^{p}\beta_j^2 \tag{6.12}$$

被最小化，而lasso相当于找到这样的系数，使得

$$\sum_{j=1}^{p}(y_j - \beta_j)^2 + \lambda\sum_{j=1}^{p}|\beta_j| \tag{6.13}$$

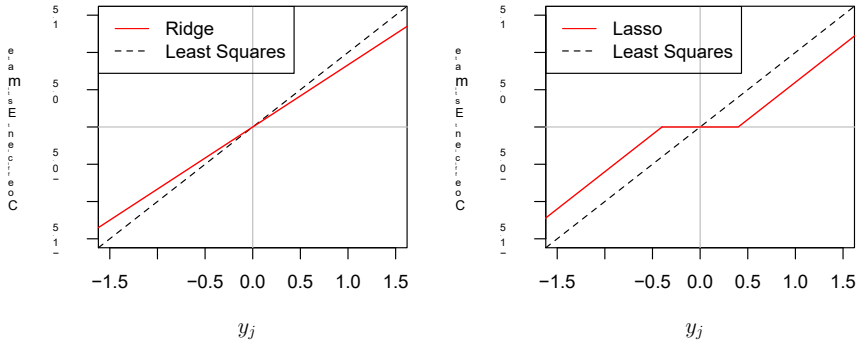


图6.10。*The ridge regression and lasso coefficient estimates for a simple setting with $n = p$ and $\mathbf{X}$ a diagonal matrix with 1's on the diagonal.* 左：*The ridge regression coefficient estimates are shrunken proportionally towards zero, relative to the least squares estimates.* 右：*The lasso coefficient estimates are soft-thresholded towards zero.*

被最小化。可以证明，在这种设定下，岭回归估计量具有如下形式：

$$\hat{\beta}_j^R = y_j/(1+\lambda), \tag{6.14}$$

而lasso估计的形式为

$$\hat{\beta}_j^L = \begin{cases} y_j - \lambda/2 & \text{if } y_j > \lambda/2; \\ y_j + \lambda/2 & \text{if } y_j < -\lambda/2; \\ 0 & \text{if } |y_j| \le \lambda/2. \end{cases} \tag{6.15}$$

图6.10展示了这种情况。我们可以看到，岭回归和套索回归执行了两种非常不同的收缩方式。在岭回归中，每个最小二乘系数估计值都被按相同比例收缩。相比之下，套索回归将每个最小二乘系数以固定量 $\lambda/2$ 向零收缩；绝对值小于 $\lambda/2$ 的最小二乘系数被完全收缩至零。在这种简单设定（6.15）下，套索回归所执行的收缩类型被称为 *soft- thresholding*。套索回归中某些系数被完全收缩至

soft-thresholding

零解释了为什么套索算法会执行特征选择。

对于更一般的数据矩阵$\mathbf{X}$，情况比图6.10所描绘的要复杂一些，但主要思想仍然大致成立：岭回归或多或少地以相同的比例缩小数据的每个维度，而套索则或多或少地将所有系数以相似的量收缩至零，并且足够小的系数会被一直收缩至零。

岭回归与套索的贝叶斯解释



我们现在展示如何通过贝叶斯视角来看待岭回归和套索回归。回归的贝叶斯观点假设系数向量 β 具有某种 *prior* 分布，比如 $p(\beta)$，其中 $\beta = (\beta_0, \beta_1, \ldots, \beta_p)^T$。数据的似然可以表示为 $f(Y|X, \beta)$，

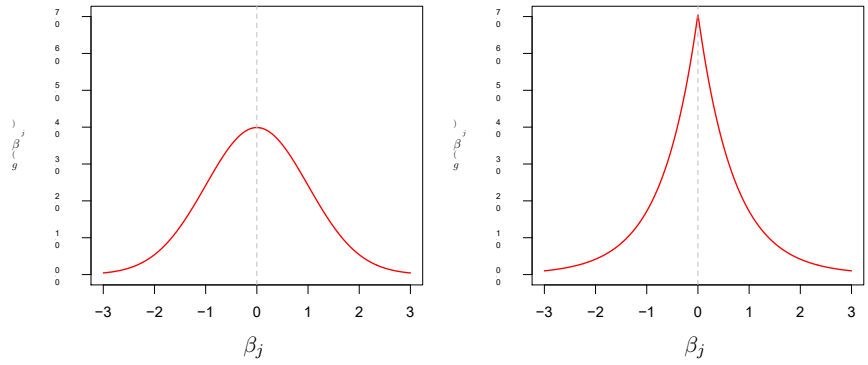


图6.11. 左：*Ridge regression is the posterior mode for* β *under a Gaus- sian prior.* 右：*The lasso is the posterior mode for* β *under a double-exponential prior.*

其中 $X = (X_1, \ldots, X_p)$。将先验分布乘以似然函数得到（在相差一个比例常数的意义下）*posterior distribution*,
其形式为

posterior distribution

$$p(\beta|X, Y) \propto f(Y|X, \beta)p(\beta|X) = f(Y|X, \beta)p(\beta),$$

其中，上述比例关系由贝叶斯定理得出，而上述等式则源自假定X固定的前提。

我们假设通常的线性模型，

$$Y = \beta_0 + X_1\beta_1 + \cdots + X_p\beta_p + \epsilon,$$

并且假设误差是独立的，且来自正态分布。此外，假设对于某个密度函数 g 有 $p(\beta) = \prod_{j=1}^{p} g(\beta_j)$。事实证明，岭回归和套索回归自然地源自 g 的两种特殊情况：

- 如果 g 是一个均值为零且标准差为 λ 的函数的高斯分布，那么可推得 β 的 *posterior mode*——即
  也就是说，给定数据下 β 最可能的值，是由岭回归解给出的。（事实上，岭回归解也是后验均值。）

  后验众数

- 如果 g 是双指数（拉普拉斯）分布，其均值为零，尺度参数是 λ 的函数，那么可以得出 β 的后验众数是套索解。（然而，套索解 *not* 后验均值，实际上，后验均值并不产生稀疏系数向量。）

高斯先验和双指数先验如图6.11所示。因此，从贝叶斯的视角来看，岭回归和套索方法直接源自于假设通常的线性模型具有正态误差，并且为β设定了一个简单的先验分布。需要注意的是，套索先验在零点处呈尖锐峰值，而高斯先验在零点附近则更为平缓和宽厚。因此，套索方法先验地预期许多系数（完全）为零，而岭回归则假设这些系数围绕零随机分布。

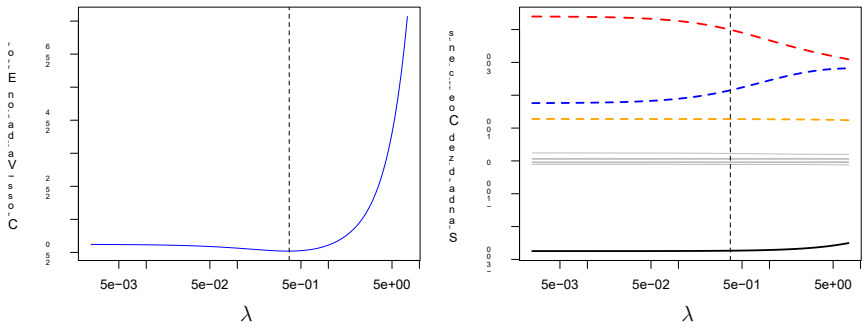


图6.12. 左: *Cross-validation errors that result from applying ridge regression to the* `Credit` *data set with various values of* λ. 右: *The coefficient estimates as a function of* λ. *The vertical dashed lines indicate the value of* λ *selected by cross-validation.*

6.2.3 Selecting the Tuning Parameter

正如6.1节中考虑的子集选择方法需要一种方法来确定所考虑的模型中哪个最佳一样，实现岭回归和套索回归也需要一种方法来选择(6.5)和(6.7)中调优参数λ的值，或者等价地，选择(6.9)和(6.8)中约束条件s的值。交叉验证提供了一种简单的方法来解决这个问题。我们选择一组λ值的网格，并如第5章所述，计算每个λ值的交叉验证误差。然后，我们选择使交叉验证误差最小的调优参数值。最后，使用所有可用的观测值和所选的调优参数值重新拟合模型。

图6.12展示了在`Credit`数据集上对岭回归拟合执行留一交叉验证后所选择的λ。虚线垂直线表示选定的λ值。在这种情况下，该值相对较小，表明最优拟合相对于最小二乘解仅涉及少量的收缩。此外，下降并不十分明显，因此有相当宽范围的值会产生非常相似的误差。在这种情况下，我们可以简单地使用最小二乘解。

图6.13展示了应用于图6.9中稀疏模拟数据套索拟合的十折交叉验证。图6.13左侧面板显示了交叉验证误差，而右侧面板显示了系数估计。垂直虚线表示交叉验证误差最小的点。图6.13右侧面板中的两条彩色线代表与响应相关的两个预测变量，而灰色线代表不相关的预测变量；这些通常被称为*signal*和*noise*变量，

signal

分别。lasso不仅正确地给两个信号预测变量赋予了更大的系数估计值，而且最小交叉验证误差对应的系数估计集中，只有信号变量非零。因此，交叉验证与lasso结合正确识别了模型中的两个信号变量，尽管这是一个具有挑战性的设置，有$p = 45$个变量且仅有$n = 50$

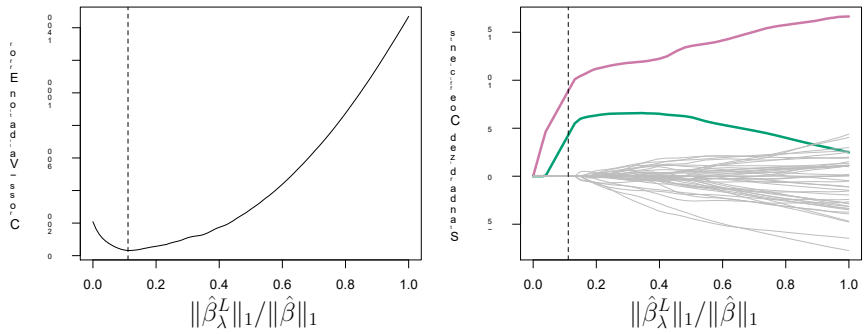


图 6.13. 左: *Ten-fold cross-validation MSE for the lasso, applied to the sparse simulated data set from Figure 6.9.* 右: *The corresponding lasso coefficient estimates are displayed. The two signal variables are shown in color, and the noise variables are in gray. The vertical dashed lines indicate the lasso fit for which the cross-validation error is smallest.*

观测值。相比之下，最小二乘解——如图6.13右侧面板最右端所示——仅对两个信号变量中的一个赋予了较大的系数估计值。

6.3 降维方法

到目前为止，本章讨论的方法以两种不同的方式控制了方差，要么通过使用原始变量的子集，要么通过将其系数向零收缩。所有这些方法都是使用原始预测变量 $X_1, X_2, \ldots, X_p$ 定义的。我们现在探索一类方法，这些方法 *transform* 预测变量，然后使用变换后的变量拟合最小二乘模型。我们将这些技术称为 *dimension reduction* 方法。

dimension reduction
linear combination

让$Z_1, Z_2, \ldots, Z_M$表示我们原始的$M < p$ *linear combinations* p 预测变量。即，

$$Z_m = \sum_{j=1}^{p} \phi_{jm} X_j \tag{6.16}$$

对于某些常数 $\phi_{1m}, \phi_{2m} \ldots, \phi_{pm}$，$m = 1, \ldots, M$。我们接着可以拟合线性回归模型

$$y_i = \theta_0 + \sum_{m=1}^{M} \theta_m z_{im} + \epsilon_i, \quad i = 1, \ldots, n, \tag{6.17}$$

使用最小二乘法。注意，在(6.17)中，回归系数由$\theta_0, \theta_1, \ldots, \theta_M$给出。如果常数$\phi_{1m}, \phi_{2m}, \ldots, \phi_{pm}$选择得当，那么这种降维方法往往能优于最小二乘回归。换句话说，使用最小二乘法拟合(6.17)可能会比使用最小二乘法拟合(6.1)得到更好的结果。

术语*dimension reduction*来源于这样一种事实：该方法将估计$p+1$个系数$\beta_0, \beta_1, \ldots, \beta_p$的问题简化为

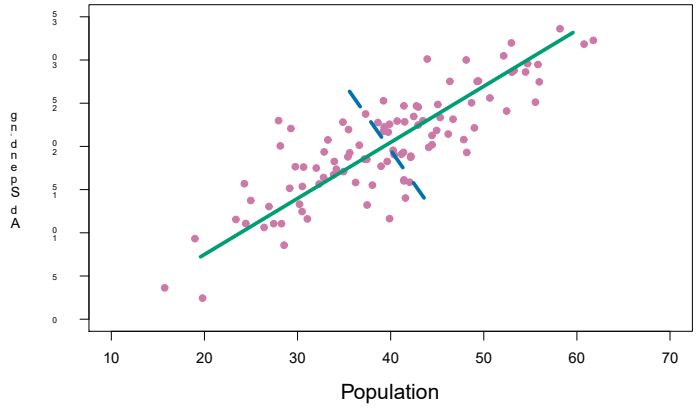


图6.14。*The population size (*pop*) and ad spending (*ad*) for* 100 *different cities are shown as purple circles. The green solid line indicates the first principal component, and the blue dashed line indicates the second principal component.*

更简单的问题是估计 $M+1$ 系数 $\theta_0, \theta_1, \ldots, \theta_M$，其中 $M < p$。换句话说，问题的维度已从 $p+1$ 降到 $M+1$。

注意从(6.16)式可知，

$$\sum_{m=1}^{M} \theta_m z_{im} = \sum_{m=1}^{M} \theta_m \sum_{j=1}^{p} \phi_{jm} x_{ij} = \sum_{j=1}^{p} \sum_{m=1}^{M} \theta_m \phi_{jm} x_{ij} = \sum_{j=1}^{p} \beta_j x_{ij},$$

其中

$$\beta_j = \sum_{m=1}^{M} \theta_m \phi_{jm}. \tag{6.18}$$

因此，(6.17) 可以看作是由 (6.1) 给出的原始线性回归模型的一个特例。降维用于约束估计的 β_j 系数，因为它们现在必须采用 (6.18) 的形式。对系数形式的这种约束有可能使系数估计产生偏差。然而，在 p 相对于 n 较大的情况下，选择 $M \ll p$ 的值可以显著降低拟合系数的方差。如果 $M = p$，并且所有 Z_m 都是线性无关的，那么 (6.18) 不构成任何约束。在这种情况下，不会发生降维，因此拟合 (6.17) 等价于对原始 p 个预测变量执行最小二乘法。

所有降维方法都分两步进行。首先，获得变换后的预测变量 $Z_1, Z_2, \ldots, Z_M$。其次，使用这些 M 预测变量拟合模型。然而，$Z_1, Z_2, \ldots, Z_M$ 的选择，或者等价地，ϕ_{jm} 的选择，可以通过不同的方式实现。在本章中，我们将考虑两种方法来完成这项任务：*principal components* 和 *partial least squares*。

6.3.1 Principal Components Regression

Principal components analysis (PCA)是一种常用的推导方法

principal components analysis

从大量变量中提取低维特征集。PCA作为*unsupervised learning*的工具将在第12章中更详细地讨论。此处我们描述其作为回归降维技术的应用。

主成分分析概述

PCA 是一种降低 $n \times p$ 数据矩阵 $\mathbf{X}$ 维度的技术。数据的 *first principal component* 方向是观测值 *vary the most* 的方向。例如，考虑图 6.14，它展示了 100 个城市的人口规模（pop），以万人为单位，以及某特定公司的广告支出（ad），以千美元为单位。[6] 绿色实线代表数据的第一个主成分方向。我们可以直观地看到，这是数据变化最大的方向。也就是说，如果我们将这 100 个观测值 *projected* 到这条线上（如图 6.15 左侧面板所示），那么得到的投影观测值将具有最大的可能方差；将观测值投影到任何其他线上都会得到方差更低的投影观测值。将一个点投影到一条线上，就是找到这条线上离该点最近的位置。

第一个主成分在图6.14中以图形方式显示，但如何用数学方式概括呢? 它由公式给出

$$Z_1 = 0.839 \times (\texttt{pop} - \overline{\texttt{pop}}) + 0.544 \times (\texttt{ad} - \overline{\texttt{ad}}). \tag{6.19}$$

这里，$\phi_{11} = 0.839$ 和 $\phi_{21} = 0.544$ 是主成分载荷，它们定义了上述方向。在(6.19)中，pop 表示该数据集中所有 pop 值的均值，而 $\overline{\texttt{ad}}$ 表示所有广告支出的均值。其思想是，在所有可能的 pop 和 ad 的 *linear combination* 中，满足 $\phi_{11}^2 + \phi_{21}^2 = 1$ 的这一特定线性组合产生最高方差：即这是使 Var($\phi_{11} \times (\texttt{pop} - \overline{\texttt{pop}}) + \phi_{21} \times (\texttt{ad} - \overline{\texttt{ad}})$) 最大化的线性组合。只需考虑形式为 $\phi_{11}^2 + \phi_{21}^2 = 1$ 的线性组合，因为否则我们可以任意增大 ϕ_{11} 和 ϕ_{21} 以放大方差。在(6.19)中，两个载荷均为正且大小相近，因此 Z_1 几乎是这两个变量的 *average*。

由于n =为100，pop和ad是长度为100的向量，(6.19)中的Z_1也是如此。例如，

$$z_{i1} = 0.839 \times (\texttt{pop}_i - \overline{\texttt{pop}}) + 0.544 \times (\texttt{ad}_i - \overline{\texttt{ad}}). \tag{6.20}$$

$z_{11}, \ldots, z_{n1}$ 的值称为 *principal component scores*，可以在图6.15的右侧面板中看到。

PCA 还有另一种解释：第一主成分向量定义了与数据 *as close as possible* 的直线。例如，在图 6.14 中，第一主成分直线最小化每个点与该直线之间的垂直距离平方和。这些距离在左侧以虚线线段绘制

[6] This dataset is distinct from the **Advertising** data discussed in Chapter 3.

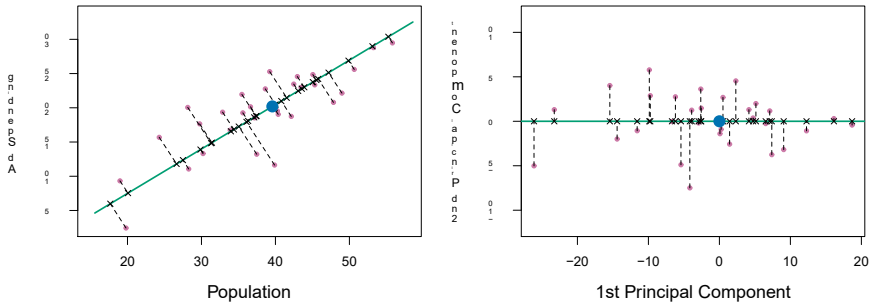


图 6.15. *A subset of the advertising data. The mean* pop *and* ad *budgets are indicated with a blue circle.* 左：*The first principal component direction is shown in green. It is the dimension along which the data vary the most, and it also defines the line that is closest to all* n *of the observations. The distances from each observation to the principal component are represented using the black dashed line segments. The blue dot represents* $(\overline{\texttt{pop}}, \overline{\texttt{ad}})$. 右：*The left-hand panel has been rotated so that the first principal component direction coincides with the* x*-axis.*

图6.15的面板中，十字代表每个点在第一条主成分线上的*projection*。第一条主成分的选择使得投影后的观测值与原观测值之间的*as close as possible*。

在图6.15的右侧面板中，左侧面板已旋转，使得第一主成分方向与x轴重合。可以证明，第i个观测值的*first principal component score*，如(6.20)所示，是第i个交叉点在x方向上与零的距离。例如，图6.15左侧面板左下角的点具有较大的负主成分得分，$z_{i1} = -26.1$，而右上角的点则具有较大的正得分，$z_{i1} = 18.7$。这些得分可以直接使用(6.20)计算得出。

我们可以将主成分Z_1的值视为每个地点联合pop和ad预算的单数值摘要。在此示例中，如果$z_{i1} = 0.839 \times (\texttt{pop}_i - \overline{\texttt{pop}}) + 0.544 \times (\texttt{ad}_i - \overline{\texttt{ad}}) < 0$，则这表明该城市的人口规模和广告支出均低于平均水平。正分数则表明相反的情况。单个数值能在多大程度上同时代表pop和ad？在这种情况下，图6.14表明pop和ad具有近似线性关系，因此我们可能预期单数值摘要将表现良好。图6.16展示了z_{i1}与pop和ad的关系。[7]这些图显示第一主成分与这两个特征之间存在强关系。换句话说，第一主成分似乎捕捉了pop和ad预测变量中包含的大部分信息。

到目前为止，我们一直专注于第一个主成分。通常，可以构建多达p个不同的主成分。第二个

[7]The principal components were calculated after first standardizing both pop and ad, a common approach. Hence, the x-axes on Figures 6.15 and 6.16 are not on the same scale.

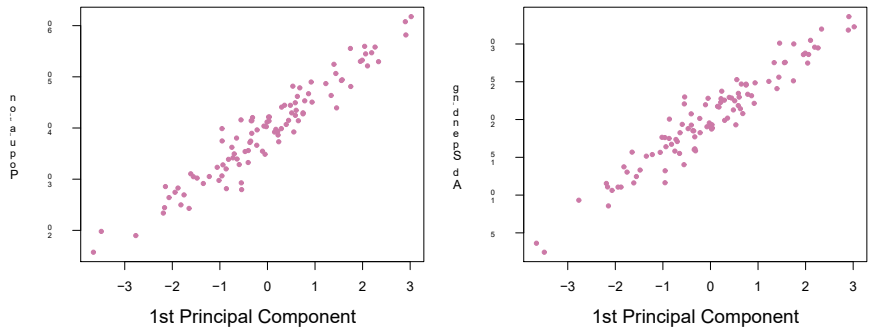


图6.16。 *Plots of the first principal component scores* z_{i1} *versus* pop *and* ad. *The relationships are strong.*

主成分Z_2是变量的线性组合，与Z_1不相关，并且在此约束下具有最大方差。第二主成分方向在图6.14中以蓝色虚线表示。结果表明，Z_1与Z_2的零相关条件等价于方向必须为*perpendicular*的条件，即

orthogonal，到第一主成分方向。第二主组件由公式给出

perpendicular
orthogonal

$$Z_2 = 0.544 \times (\texttt{pop} - \overline{\texttt{pop}}) - 0.839 \times (\texttt{ad} - \overline{\texttt{ad}}).$$

由于广告数据有两个预测变量，前两个主成分包含了pop和ad中的所有信息。然而，按照构造原理，第一主成分将包含最多的信息。例如，考虑图6.15右侧面板中z_{i1} (x轴)与z_{i2} (y轴)相比大得多的变异性。第二主成分得分更接近于零这一事实表明，该成分所捕获的信息要少得多。作为另一个示例，图6.17展示了z_{i2}与pop和ad的关系。第二主成分与这两个预测变量之间几乎没有关系，这再次表明在这种情况下，只需第一主成分即可准确代表pop和ad预算。

对于二维数据，如我们的广告示例所示，我们最多可以构建两个主成分。然而，如果我们有其他预测变量，比如人口年龄、收入水平、教育程度等，那么可以构建更多的成分。这些成分会依次最大化方差，但须满足与先前成分不相关的约束。

主成分回归方法

principal components regression (PCR)方法涉及构建-使用前 M 个主成分 $Z_1, \ldots, Z_M$，然后将这些成分作为线性回归模型中的预测变量，并使用最小二乘法进行拟合。关键思想是，通常少数几个主成分就足以解释数据中的大部分变异性以及与响应变量的关系。换句话说，我们假设 *the*

principal components regression

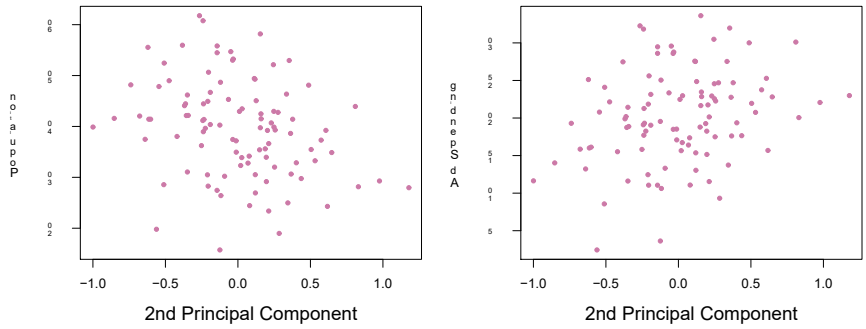


图 6.17。*Plots of the second principal component scores z_{i2} versus* pop *and* ad*. The relationships are weak.*

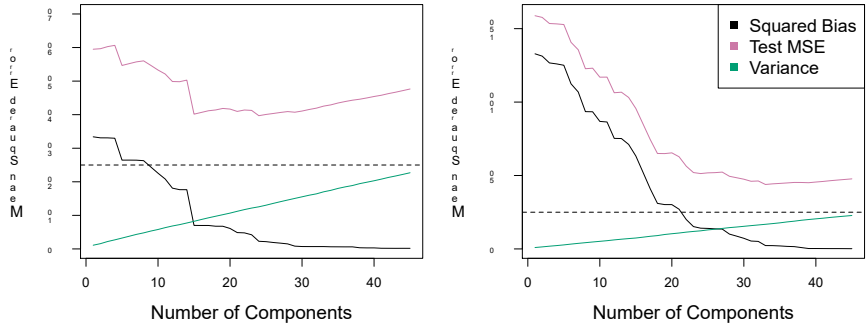


图6.18。*PCR was applied to two simulated data sets. In each panel, the horizontal dashed line represents the irreducible error.* 左：*Simulated data from Figure 6.8.* 右：*Simulated data from Figure 6.9.*

directions in which $X_1, \ldots, X_p$ *show the most variation are the directions that are associated with* Y。虽然这个假设不保证总是成立，但往往结果是足够合理的近似，能够给出良好的结果。

如果PCR背后的假设成立，那么对$Z_1, \ldots, Z_M$拟合最小二乘模型将比拟合$X_1, \ldots, X_p$得到更好的结果，因为数据中与响应相关的大部分或全部信息都包含在$Z_1, \ldots, Z_M$中，而且通过仅估计$M \ll p$个系数，我们可以减轻过拟合。在广告数据中，第一个主成分解释了pop和ad中的大部分方差，因此使用这个单一变量来预测某些感兴趣的响应（如sales）的主成分回归可能会表现得相当好。

图6.18展示了对图6.8和图6.9中模拟数据集的PCR拟合结果。回想一下，两个数据集都是使用n =50个观测值和p =45个预测变量生成的。然而，第一个数据集中的响应是所有预测变量的函数，而第二个数据集中的响应仅使用其中两个预测变量生成。这些曲线是作为M的函数绘制的，M是回归模型中用作预测变量的主成分数量。随着使用更多主成分

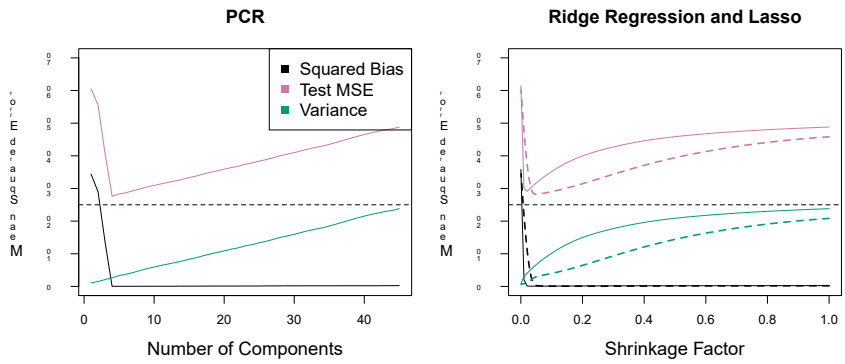


图6.19。*PCR, ridge regression, and the lasso were applied to a simulated data set in which the first five principal components of X contain all the information about the response Y. In each panel, the irreducible error* Var(ϵ) *is shown as a horizontal dashed line.* 左：*Results for PCR.* 右：*Results for lasso (solid) and ridge regression (dotted). The x-axis displays the shrinkage factor of the coefficient estimates, defined as the ℓ_2 norm of the shrunken coefficient estimates divided by the ℓ_2 norm of the least squares estimate.*

在回归模型中，偏差减小，但方差增加。这导致均方误差呈现典型的U形。当$M = p =$为45时，PCR就等同于使用所有原始预测变量的最小二乘拟合。该图表明，选用适当的M进行PCR，相比最小二乘可实现显著改进，尤其是在左侧面板中。然而，通过观察图6.5、6.8和6.9中的岭回归与套索结果，我们发现，在此示例中，PCR的表现不如这两种收缩方法。

图6.18中PCR表现相对较差，是因为数据的生成方式使得为了充分建模响应需要许多主成分。相反，当前几个主成分足以捕获预测变量中的大部分变异以及与响应变量的关系时，PCR往往会表现良好。图6.19左侧面板展示了另一个模拟数据集的结果，该数据集被设计为更有利于PCR。在这里，响应变量的生成方式使其仅依赖于前五个主成分。现在，随着M（PCR中使用的主成分数量）增加，偏差迅速降至零。均方误差在$M = 5$处显示出明显的最小值。图6.19右侧面板展示了在这些数据上使用岭回归和lasso的结果。三种方法都比最小二乘法有显著改进。然而，PCR和岭回归略微优于lasso。

我们注意到，尽管PCR提供了一种使用 $M < p$ 个预测变量进行回归的简便方法，但它 *not* 一种特征选择方法。这是因为回归中使用的 M 个主成分中的每一个都是所有 p 个 *original* 特征的线性组合。例如，在(6.19)中，Z_1 是 pop 和 ad 两者的线性组合。因此，尽管PCR在许多实际场景中通常表现相当好，但它并不会导致

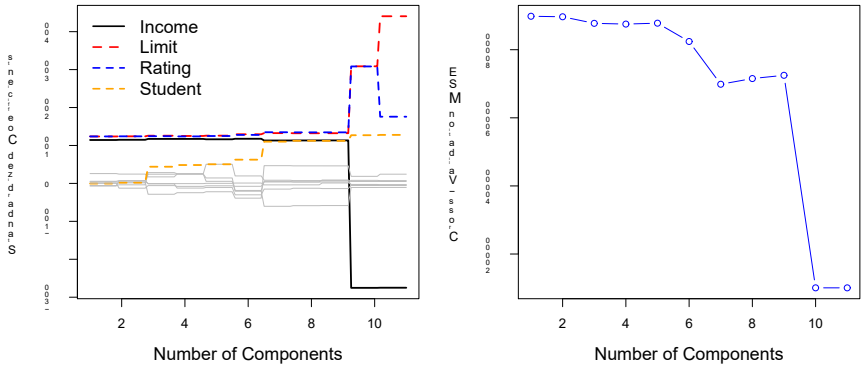


图 6.20. 左：*PCR standardized coefficient estimates on the* `Credit` *data set for different values of* M. 右：*The ten-fold cross-validation* MSE *obtained using PCR, as a function of* M.

开发一个依赖于原始特征的一小部分的模型。从这个意义上讲，PCR与岭回归的关系比与套索更密切。实际上，可以证明PCR和岭回归非常相关。甚至可以把岭回归看作是PCR的连续版本！[8]

在PCR中，主成分的数量 M 通常通过交叉验证来选择。将PCR应用于 `Credit` 数据集的结果如图6.20所示；右侧面板展示了所得到的交叉验证误差，它是 M 的函数。在这些数据上，最低的交叉验证误差出现在有 $M = 10$个主成分时；这几乎完全没有实现降维，因为当 $M = 11$时，PCR等价于直接执行最小二乘法。

在执行PCR时，我们通常建议在生成主成分之前，使用(6.6)对每个预测变量进行*standardizing*。这种标准化确保了所有变量都处于相同的尺度上。如果不进行标准化，高方差的变量将在获得的主成分中倾向于发挥更大的作用，而变量测量的尺度最终会对最终的PCR模型产生影响。然而，如果所有变量都以相同的单位测量（例如，千克或英寸），那么可以选择不对它们进行标准化。

6.3.2 Partial Least Squares

我们刚刚描述的PCR方法涉及识别线性组合或*directions*，这些组合最能代表预测变量$X_1, \ldots, X_p$。这些方向是以*unsupervised*方式确定的，因为响应变量Y并未用于帮助确定主成分方向。也就是说，响应变量不*supervise*主成分的识别。因此，PCR存在一个缺点：无法保证

[8] More details can be found in Section 3.5 of *The Elements of Statistical Learning* by Hastie, Tibshirani, and Friedman.

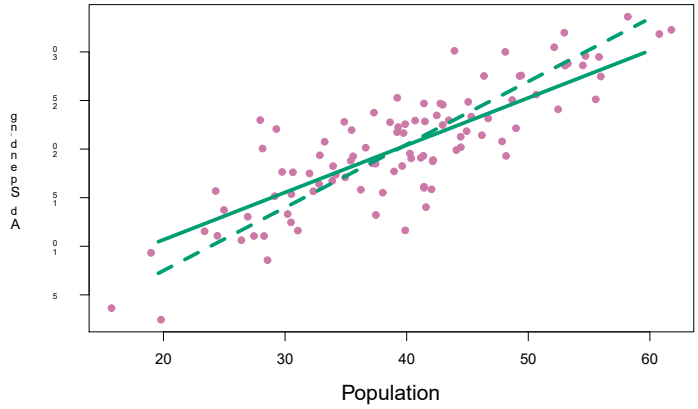


图6.21。*For the advertising data, the first PLS direction (solid line) and first PCR direction (dotted line) are shown.*

最能解释预测变量的方向也将是用于预测响应的最佳方向。无监督方法将在第12章进一步讨论。

我们现在介绍 *partial least squares* (PLS)，它是 *supervised* 的替代方案，用于PCR。与PCR类似，PLS是一种降维方法，它首先识别一组新的特征 $Z_1, \ldots, Z_M$，这些特征是原始特征的线性组合，然后使用这些M新特征通过最小二乘法拟合线性模型。但与PCR不同，PLS以有监督的方式识别这些新特征——也就是说，它利用响应Y来识别不仅能够很好地近似旧特征，而且还能够*arerelated to the response*的新特征。粗略地说，PLS方法试图找到有助于解释响应和预测变量的方向。

partial least squares

我们现在描述如何计算第一个PLS方向。在对p个预测变量进行标准化之后，PLS通过将(6.16)中的每个ϕ_{j1}设为Y对X_j的简单线性回归系数来计算第一个方向Z_1。可以证明，该系数与Y和X_j之间的相关性成正比。因此，在计算$Z_1 = \sum_{j=1}^{p} \phi_{j1} X_j$时，PLS对与响应关系最强的变量赋予最高的权重。

图6.21展示了在一个合成数据集上应用PLS的示例，该数据集包含100个区域的销售额作为响应变量，以及两个预测变量：人口规模和广告支出。实心绿线表示第一个PLS方向，而虚线则展示了第一个主成分方向。与PCA相比，PLS选择的在`pop`维度每单位变化时，`ad`维度的变化较小。这表明`pop`与响应变量的相关性比`ad`更高。PLS方向虽不如PCA那样紧密拟合预测变量，但在解释响应变量方面表现更佳。

为了确定第二个PLS方向，我们首先对每个变量针对Z_1进行*adjust*，通过将每个变量对Z_1进行回归并取*residuals*。这些残差可以解释为尚未被第一个PLS方向解释的剩余信息。然后，我们使用这个*or*-计算Z_2。

thogonalized 以与基于原始数据计算 Z_1 完全相同的方式处理数据。这种迭代方法可以重复 M 次，以识别多个 PLS 成分 $Z_1, \ldots, Z_M$。最后，在此过程结束时，我们使用最小二乘法拟合一个线性模型，以与 PCR 完全相同的方式使用 $Z_1, \ldots, Z_M$ 预测 Y。

与PCR一样，PLS中使用的偏最小二乘方向数量M是一个调优参数，通常通过交叉验证来选择。在执行PLS之前，我们通常会对预测变量和响应变量进行标准化处理。

偏最小二乘在化学计量学领域很受欢迎，因为在该领域中，许多变量来源于数字化光谱信号。在实践中，它通常并不比岭回归或主成分回归表现更好。虽然偏最小二乘的监督降维可以减少偏差，但它也有可能增加方差，因此偏最小二乘相对于主成分回归的整体优势并不明显。

6.4 高维情况下的考量

6.4.1 High-Dimensional Data

大多数用于回归和分类的传统统计技术都是针对 *low-dimensional* 情形，其中 n 表示观测值的数量。

观测值，远大于特征数量 p。这部分是因为在该领域的大部分历史中，需要使用统计学的大多数科学问题都是低维的。例如，考虑开发一个模型，根据患者的年龄、性别和体重指数（BMI）来预测其血压的任务。这里有三个预测变量，如果模型中包含截距，则为四个，而可能有数千名患者的血压以及年龄、性别和BMI数据可供使用。因此，$n \gg p$，所以问题是低维的。（这里的维度指的是 p 的大小。）

low-dimensional

在过去20年里，新技术改变了金融、市场营销和医学等各个领域数据收集的方式。如今，收集几乎无限数量的特征测量值（p非常大）已司空见惯。尽管p可能极其庞大，但由于成本、样本可用性或其他因素，观测数量n往往有限。以下是两个例子：

1. 并非仅依据年龄、性别和BMI预测血压，还可以收集五十万个*single nucleotide polymorphisms* (SNP的测量数据；这些是群体中相对常见的个体DNA突变)，可纳入预测模型。然后$n \approx 200$和$p \approx 500{,}000$。

2. 一位对理解人们在线购物模式感兴趣的市场分析师，可以将搜索引擎用户输入的所有搜索词视为特征。这有时被称为“词袋”模型。同一位研究者可能只能够访问几百或几千名同意与研究者分享信息的搜索引擎用户的搜索历史。对于给定用户，p 个搜索词中的每一个都被标记为存在（0）或

缺失 (1)，创建一个大型二值特征向量。然后 $n \approx 1{,}000$ 而 p 大得多。

特征数多于观测数的数据集通常被称为*high-dimensional*。经典方法如最小二乘线性
回归在这种情况下并不适用。本书前面已经讨论了高维数据分析中出现的许多问题，因为当$n > p$时这些问题同样适用：这些问题包括偏差-方差权衡的作用以及过拟合的危险。尽管这些问题始终相关，但当特征数量相对于观测数量非常大时，它们可能变得尤为重要。

high-dimensional

我们将*high-dimensional setting*定义为特征数量p大于观测数量n的情况。但我们接下来要讨论的考量同样适用于p略小于n的情形，并且在进行监督学习时最好始终牢记这些考量。

6.4.2 What Goes Wrong in High Dimensions?

为了说明当$p > n$时，回归和分类需要额外小心和专门技术，我们首先考察如果应用一种并非为高维设定设计的统计技术可能会出现什么问题。为此，我们研究最小二乘回归。但同样的概念也适用于逻辑回归、线性判别分析以及其他经典统计方法。

当特征数量 p 与观测数量 n 一样大或更大时，第 3 章中描述的最小二乘法无法（或者说，*should not*）执行。原因很简单：无论特征与响应之间是否真的存在关系，最小二乘法都会产生一组系数估计，使得数据完美拟合，从而使残差为零。

图6.22展示了一个示例，其中在两种情况下，$p = 1$个特征（加上截距）：当有20个观测值时，以及当只有两个观测值时。当有20个观测值时，$n > p$，最小二乘回归线并不能完美拟合数据；相反，回归线试图尽可能地近似这20个观测值。另一方面，当只有两个观测值时，那么无论这些观测值是多少，回归线都将完全拟合数据。这是有问题的，因为这种完美拟合几乎肯定会导致对数据的过拟合。换句话说，尽管在高维设置下可以完美拟合训练数据，但所得线性模型在独立测试集上的表现将极差，因此不构成一个有用的模型。事实上，我们可以看到这种情况发生在图6.22中：右图中得到的最小二乘线在由左图中的观测值组成的测试集上将表现得很差。问题很简单：当 $p > n$ 或 $p \approx n$ 时，简单的最小二乘回归线过于*flexible*，从而过拟合数据。

图6.23进一步说明了在特征数量p很大时，粗心应用最小二乘法的风险。

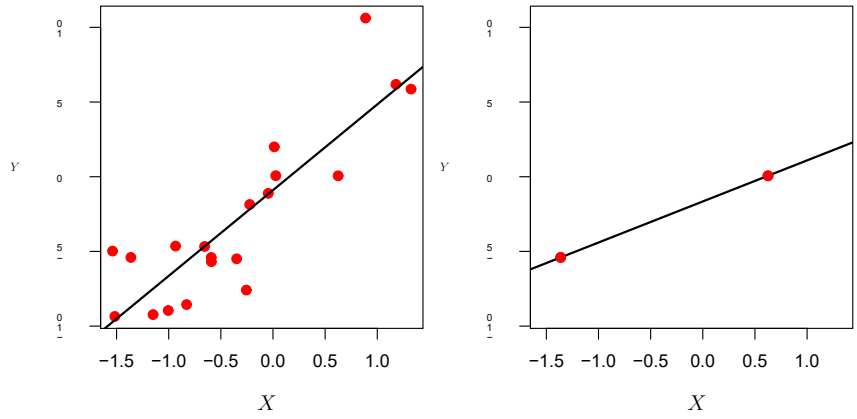


图 6.22. 左：*Least squares regression in the low-dimensional setting.* 右: *Least squares regression with* $n = 2$ *observations and two parameters to be estimated (an intercept and a coefficient).*

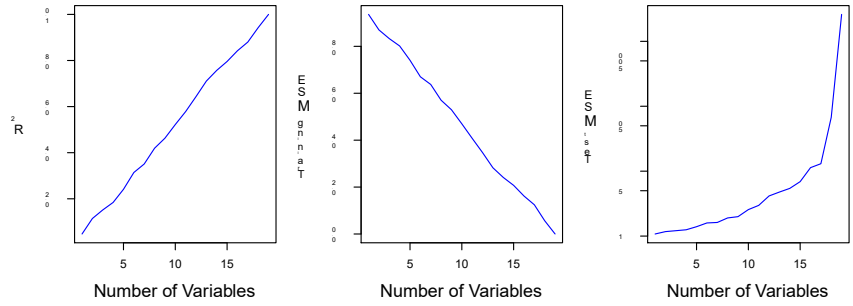


图6.23。*On a simulated example with* $n = 20$ *training observations, features that are completely unrelated to the outcome are added to the model.* 左：*The* R^2 *increases to 1 as more features are included.* 中：*The training set MSE decreases to 0 as more features are included.* 右：*The test set MSE increases as more features are included.*

其中每一个都与响应完全无关。如图所示，模型R^2随着模型中包含的特征数量增加而增加到1，相应地，训练集MSE随着特征数量的增加而减小到0，*even though the features are completely unrelated to the response*。另一方面，*independent test set*上的MSE随着模型中包含的特征数量增加而变得极大，因为包含额外的预测变量会导致系数估计的方差大幅增加。观察测试集MSE，可以清楚地看到最佳模型最多包含几个变量。然而，若有人粗心大意地只检查R^2或训练集MSE，可能会错误地得出结论，认为变量数量最多的模型是最好的。这表明在分析具有大量变量的数据集时需要格外小心，并且始终要在独立的测试集上评估模型性能。

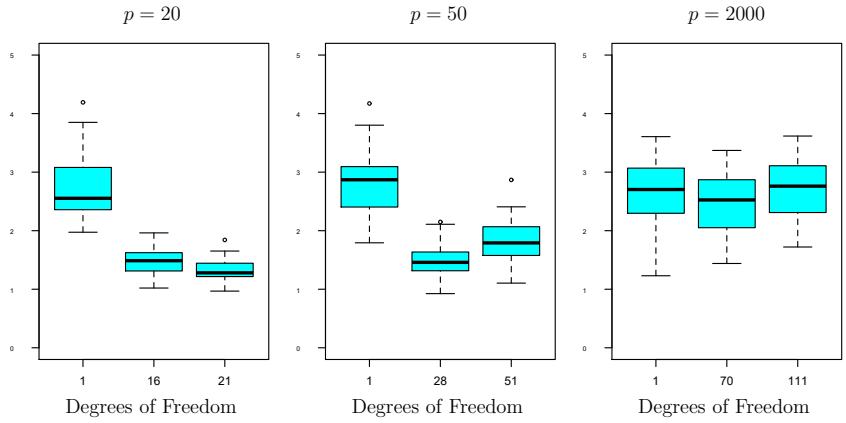


图6.24。 *The lasso was performed with* $n = 100$ *observations and three values of* p*, the number of features. Of the* p *features, 20 were associated with the response. The boxplots show the test MSEs that result using three different values of the tuning parameter* λ *in (6.7). For ease of interpretation, rather than reporting* λ*, the* 自由度 *are reported; for the lasso this turns out to be simply the number of estimated non-zero coefficients. When* $p = 20$*, the lowest test MSE was obtained with the smallest amount of regularization. When* $p = 50$*, the lowest test MSE was achieved when there is a substantial amount of regularization. When* $p = 2{,}000$ *the lasso performed poorly regardless of the amount of regularization, due to the fact that only 20 of the 2,000 features truly are associated with the outcome.*

在第6.1.3节中，我们看到了若干方法，用于调整训练集RSS或R^2，以考虑拟合最小二乘模型时使用的变量数量。遗憾的是，C_p、AIC和BIC方法在高维设定中并不适用，因为估计$\hat{\sigma}^2$存在问题。（例如，第3章中的$\hat{\sigma}^2$公式在此设定下给出的估计值$\hat{\sigma}^2 =$为0。）同样，在高维设定中应用调整后的R^2时也会出现问题，因为很容易获得一个调整后R^2值为1的模型。显然，需要更适合高维设定的替代方法。

6.4.3 Regression in High Dimensions

事实证明，本章中用于拟合*less flexible*最小二乘模型的许多方法，如向前逐步选择、岭回归、套索法和主成分回归，在高维场景下进行回归分析时尤为有用。本质上，这些方法通过采用比最小二乘法灵活性更低的拟合方式来避免过拟合。

图6.24展示了lasso在一个简单模拟示例中的性能表现。存在$p = 20$、50或2,000个特征，其中20个与结果真正相关。lasso在$n = 100$个训练观测上执行，并在独立测试集上评估均方误差。随着特征数量的增加，测试集误差也增加。当$p = 20$时，最低验证集误差在λ in时达到。

(6.7) 较小；然而，当 p 较大时，使用较大的 λ 值可获得最低的验证集误差。在每个箱线图中，不是报告所使用的 λ 值，而是显示所得 lasso 解的 *degrees of freedom*；这仅仅是 lasso 解中非零系数估计的数量，是衡量 lasso 拟合灵活性的一个指标。图 6.24 强调了三个要点：（1）正则化或收缩在高维问题中起着关键作用，（2）适当的调优参数选择对于良好的预测性能至关重要，（3）随着问题维度（即特征或预测变量的数量）的增加，测试误差往往会增加，除非额外特征真正与响应相关。

上述第三点实际上是高维数据分析中的一个关键原理，被称为 *curse of dimensionality*。人们可能会认为随着用于拟合模型的特征数量增加，拟合模型的质量也会提高。然而，比较图6.24中的左侧和右侧面板，我们发现情况并非必然如此：在此例中，当p从20增加到2,000时，测试集MSE几乎翻倍。一般来说，

curse of dimensionality

adding additional signal features that are truly associated with the response will improve the fitted model，即导致测试集误差减少。然而，添加与响应并非真正相关的噪声特征会导致拟合模型恶化，进而增加测试集误差。这是因为噪声特征增加了问题的维度，加剧了过拟合的风险（因为噪声特征可能因训练集中与响应的偶然关联而被赋予非零系数），而在改善测试集误差方面没有任何潜在好处。因此，我们看到，允许收集数千或数百万个特征测量的新技术是一把双刃剑：如果这些特征实际上与当前问题相关，它们可以带来更好的预测模型，但如果特征不相关，则会导致更差的结果。即使它们相关，拟合其系数所产生的方差也可能超过它们所带来的偏差减少。

6.4.4 Interpreting Results in High Dimensions

当我们在高维设置中执行套索、岭回归或其他回归程序时，我们必须非常谨慎地报告所获得的结果。在第3章中，我们了解了*multi- collinearity*，即回归中的变量可能彼此相关的概念。在高维设置中，多重共线性问题极为严重：模型中的任何变量都可以写成模型中所有其他变量的线性组合。本质上，这意味着我们永远无法确切知道哪些变量（如果有的话）真正对结果具有预测性，也永远无法确定用于回归的*best*系数。我们至多只能希望将大的回归系数分配给那些与真正对结果具有预测性的变量相关的变量。

例如，假设我们试图基于五十万个SNP来预测血压，并且前向逐步选择表明，在训练数据上，其中17个SNP能导出一个良好的预测模型。若据此断定这17个SNP比其他未纳入模型的SNP更能有效预测血压，则是不正确的。很可能存在许多包含17个SNP的集合，它们对血压的预测能力与所选模型相当。如果我们获取一个独立数据集，并在该数据集上执行前向逐步选择，我们很可能得到一个包含不同，甚至可能完全不重叠的SNP集合的模型。这并不减损所得模型的价值——例如，该模型在独立患者群体中预测血压可能非常有效，并对医生具有临床实用价值。但我们必须谨慎，不应夸大所得结果，而要明确我们所识别出的仅是预测血压的*one of many possible models*，并且必须在独立数据集上进一步验证。

在高维设定中报告误差和模型拟合度量时，尤其需要特别小心。我们已经看到，当$p > n$时，很容易得到一个残差为零的无用模型。因此，在高维设定中，不应将训练数据上的平方误差和、p值、R^2统计量或其他传统模型拟合度量作为模型拟合良好的证据，而应*never*使用这些度量。例如，正如我们在图6.23中看到的，当$p > n$时，很容易得到一个R^2 =为1的模型。报告这一事实可能会误导他人，使其认为已经获得了一个统计上有效且有用的模型，而实际上这完全不能证明模型具有说服力。相反，报告独立测试集上的结果或交叉验证误差非常重要。例如，独立测试集上的MSE或R^2是模型拟合的有效度量，但训练集上的MSE当然不是。

6.5 实验：线性模型与正则化方法

在本实验中，我们将实现本章讨论的许多技术。我们会在此顶层导入一些库。

In [1]:
```
import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
from statsmodels.api import OLS
import sklearn.model_selection as skm
import sklearn.linear_model as skl
from sklearn.preprocessing import StandardScaler
from ISLP import load_data
from ISLP.models import ModelSpec as MS
from functools import partial
```

我们再次收集本实验所需的新导入项。

In [2]:
```
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA
```

```
from sklearn.cross_decomposition import PLSRegression
from ISLP.models import \
     (Stepwise,
      sklearn_selected,
      sklearn_selection_path)
!pip install l0bnb
from l0bnb import fit_path
```

我们即时安装了包 l0bnb。请注意转义的 !pip install——这是作为单独的系统命令运行的。

6.5.1 Subset Selection Methods

这里我们实现了一些方法，通过将模型限制在输入变量的一个子集上，来减少模型中的参数数量。

前向选择

我们将对Hitters数据应用前向选择方法。我们希望通过与上一年表现相关的各种统计数据来预测棒球运动员的Salary。

首先，我们注意到一些球员的Salary变量缺失。可以使用np.isnan()函数来识别缺失的ob-
观测值。它返回一个与输入向量形状相同的数组，对于任何缺失的元素为True，对于非缺失元素为False。然后可以使用sum()方法来计数所有缺失的
元素。

np.isnan()

sum()

In [3]:
```
Hitters = load_data('Hitters')
np.isnan(Hitters['Salary']).sum()
```

Out[3]: 59

我们看到59名球员的Salary缺失。数据框的dropna()方法会删除任何变量中有缺失值的所有行（默认情况下——参见Hitters.dropna?）。

In [4]:
```
Hitters = Hitters.dropna();
Hitters.shape
```

Out[4]: (263, 20)

我们首先基于 C_p (6.2) 使用前向选择法选择最佳模型。这个分数并非 sklearn 内置的指标。因此，我们自定义一个函数来计算它，并将其用作评分器。默认情况下，sklearn 尝试最大化分数，因此我们的评分函数计算的是负的 C_p 统计量。

In [5]:
```
def nCp(sigma2, estimator, X, Y):
    "Negative Cp statistic"
    n, p = X.shape
    Yhat = estimator.predict(X)
    RSS = np.sum((Y - Yhat)**2)
    return -(RSS + 2 * p * sigma2) / n
```

We need to estimate the residual variance σ^2, which is the first argument in our scoring function above. We will fit the biggest model, using all the variables, and estimate σ^2 based on its MSE.

In [6]:
```
design = MS(Hitters.columns.drop('Salary')).fit(Hitters)
Y = np.array(Hitters['Salary'])
X = design.transform(Hitters)
sigma2 = OLS(Y,X).fit().scale
```

The function `sklearn_selected()` expects a scorer with just three arguments — the last three in the definition of `nCp()` above. We use the function `partial()` first seen in Section 5.3.3 to freeze the first argument with our estimate of σ^2.

In [7]:
```
neg_Cp = partial(nCp, sigma2)
```

We can now use `neg_Cp()` as a scorer for model selection.

Along with a score we need to specify the search strategy. This is done through the object `Stepwise()` in the `ISLP.models` package. The method `Stepwise.first_peak()` runs forward stepwise until any further additions to the model do not result in an improvement in the evaluation score. Similarly, the method `Stepwise.fixed_steps()` runs a fixed number of steps of stepwise search.

In [8]:
```
strategy = Stepwise.first_peak(design,
                               direction='forward',
                               max_terms=len(design.terms))
```

We now fit a linear regression model with `Salary` as outcome using forward selection. To do so, we use the function `sklearn_selected()` from the `ISLP.models` package. This takes a model from `statsmodels` along with a search strategy and selects a model with its `fit` method. Without specifying a `scoring` argument, the score defaults to MSE, and so all 19 variables will be selected (output not shown).

`sklearn_selected()`

In [9]:
```
hitters_MSE = sklearn_selected(OLS,
                               strategy)
hitters_MSE.fit(Hitters, Y)
hitters_MSE.selected_state_
```

Using `neg_Cp` results in a smaller model, as expected, with just 10 variables selected.

In [10]:
```
hitters_Cp = sklearn_selected(OLS,
                              strategy,
                              scoring=neg_Cp)
hitters_Cp.fit(Hitters, Y)
hitters_Cp.selected_state_
```

Out[10]:
```
('Assists',
 'AtBat',
 'CAtBat',
 'CRBI',
 'CRuns',
 'CWalks',
 'Division',
```

```
 'Hits',
 'PutOuts',
 'Walks')
```

使用验证集方法和交叉验证在模型之间进行选择

作为使用C_p的替代方案，我们可以尝试通过交叉验证来在向前选择中选择模型。为此，我们需要一种方法，能够存储向前选择过程中找到的完整模型路径，并允许对每个模型进行预测。这可以通过`sklearn_selection_path()`估计器来实现。

`sklearn_selection_path()`

来自`ISLP.models`的tor。来自`ISLP.models`的函数`cross_val_predict()`计算路径上每个模型的交叉验证预测值，我们可以用它来评估路径上的交叉验证均方误差（MSE）。

`cross_val_predict()`

这里我们定义一种拟合完整前向选择路径的策略。虽然`sklearn_selection_path()`有多种参数选择，我们在这里使用默认值，即在每一步基于RSS的最大缩减来选择模型。

In [11]:
```
strategy = Stepwise.fixed_steps(design,
                                len(design.terms),
                                direction='forward')
full_path = sklearn_selection_path(OLS, strategy)
```

我们现在在 `Hitters` 数据上拟合完整的前向选择路径，并计算拟合值。

In [12]:
```
full_path.fit(Hitters, Y)
Yhat_in = full_path.predict(Hitters)
Yhat_in.shape
```

Out[12]:
```
(263, 20)
```

这为我们提供了一个拟合值数组——总共20步，包括零模型的拟合均值——我们可以用它来评估样本内MSE。正如预期的那样，样本内MSE随着我们采取的每一步而改善，这表明我们必须使用验证或交叉验证方法来选择步数。我们将y轴固定为从50,000到250,000的范围，以便与下面的交叉验证和验证集MSE进行比较，以及与岭回归、套索和主成分回归等其他方法进行比较。

In [13]:
```
mse_fig, ax = subplots(figsize=(8,8))
insample_mse = ((Yhat_in - Y[:,None])**2).mean(0)
n_steps = insample_mse.shape[0]
ax.plot(np.arange(n_steps),
        insample_mse,
        'k', # color black
        label='In-sample')
ax.set_ylabel('MSE',
              fontsize=20)
ax.set_xlabel('# steps of forward stepwise',
              fontsize=20)
ax.set_xticks(np.arange(n_steps)[::2])
ax.legend()
```

```
ax.set_ylim([50000,250000]);
```

注意上面Y[:,None]中的表达式None。这为一位数组Y添加了一个轴（维度），使得在从二维数组Yhat_in中减去它时可以进行广播。

现在我们可以使用交叉验证来估计模型路径上的测试误差。我们必须使用*only the training observations*来执行模型拟合的所有方面——包括变量选择。因此，确定给定大小的最佳模型必须在每个训练折中使用*only the training observations*来进行。这一点很微妙但很重要。如果使用完整数据集在每个步骤中选择最佳子集，那么我们获得的验证集误差和交叉验证误差将不是测试误差的准确估计。

我们现在使用5折交叉验证来计算交叉验证后的预测值。

In [14]:
```
K = 5
kfold = skm.KFold(K,
                  random_state=0,
                  shuffle=True)
Yhat_cv = skm.cross_val_predict(full_path,
                                Hitters,
                                Y,
                                cv=kfold)
Yhat_cv.shape
```

Out[14]: (263, 20)

skm.KFold()
skm.cross_val_predict()

预测矩阵 Yhat_cv 与 Yhat_in 具有相同的形状；不同之处在于，每一行对应特定样本索引的预测，是由未包含该行的训练折上拟合的模型做出的。

在路径上的每个模型中，我们计算各自交叉验证折中的MSE。我们将这些值平均以得到平均MSE，并可以利用各个值来粗略估计均值标准误。[9]因此，我们必须知道每个交叉验证分割的测试索引。这可以通过使用kfold的split()方法找到。由于我们之前固定了随机状态，每当我们将任何与Y具有相同行数的数组进行分割时，我们都会恢复相同的训练和测试索引，尽管下面我们简单地忽略了训练索引。

In [15]:
```
cv_mse = []
for train_idx, test_idx in kfold.split(Y):
    errors = (Yhat_cv[test_idx] - Y[test_idx,None])**2
    cv_mse.append(errors.mean(0)) # column means
cv_mse = np.array(cv_mse).T
cv_mse.shape
```

Out[15]: (20, 5)

[9]The estimate is crude because the five error estimates are based on overlapping training sets, and hence are not independent.

我们现在将交叉验证误差估计添加到我们的MSE图中。我们包括五折的平均误差，以及均值标准误的估计。

In [16]:
```
ax.errorbar(np.arange(n_steps),
            cv_mse.mean(1),
            cv_mse.std(1) / np.sqrt(K),
            label='Cross-validated',
            c='r') # color red
ax.set_ylim([50000,250000])
ax.legend()
mse_fig
```

要重复上述过程并使用验证集方法，我们只需将cv参数改为验证集：即将数据随机划分为测试集和训练集各一次。我们选择测试集大小为20%，与5折交叉验证中每个测试集的大小相似。

skm.ShuffleSplit()

In [17]:
```
validation = skm.ShuffleSplit(n_splits=1,
                              test_size=0.2,
                              random_state=0)
for train_idx, test_idx in validation.split(Y):
    full_path.fit(Hitters.iloc[train_idx],
                  Y[train_idx])
    Yhat_val = full_path.predict(Hitters.iloc[test_idx])
    errors = (Yhat_val - Y[test_idx,None])**2
    validation_mse = errors.mean(0)
```

对于样本内MSE情况，验证集方法并不提供标准误差。

In [18]:
```
ax.plot(np.arange(n_steps),
        validation_mse,
        'b--', # color blue, broken line
        label='Validation')
ax.set_xticks(np.arange(n_steps)[::2])
ax.set_ylim([50000,250000])
ax.legend()
mse_fig
```

最佳子集选择

前向逐步是一种*greedy*选择过程；在每一步中，它通过纳入一个额外的变量来扩展当前集合。我们现在对Hitters数据应用最佳子集选择，对于每个子集大小，搜索最佳的预测变量集合。

我们将使用一个名为 l0bnb 的包来执行最佳子集选择。与将子集限制为给定大小不同，该包通过将子集大小作为惩罚项而非约束条件，生成一系列解路径。尽管这一区别很微妙，但在进行交叉验证时便会显现出来。

In [19]:
```
D = design.fit_transform(Hitters)
D = D.drop('intercept', axis=1)
X = np.asarray(D)
```

这里我们排除了对应于截距的第一列，因为l0bnb将单独拟合截距。我们可以使用fit_path()函数找到一条路径。

In [20]:
```
path = fit_path(X,
                Y,
                max_nonzeros=X.shape[1])
```

函数 fit_path() 返回一个列表，其中的值包括拟合系数 B、截距 B0 以及与所使用的特定路径算法相关的其他几个属性。这些细节超出了本书的范围。

In [21]:
```
path[3]
```

Out[21]:
```
{'B': array([0.        , 3.254844, 0.        , 0.        , 0.      ,
             0.        , 0.        , 0.        , 0.        , 0.      ,
             0.        , 0.677753 , 0.        , 0.        , 0.      ,
             0.        , 0.        , 0.        , 0.        ]),
 'B0': -38.98216739555494,
 'lambda_0': 0.011416248027450194,
 'M': 0.5829861733382011,
 'Time_exceeded': False}
```

在上面的例子中，我们看到在路径的第四步，'B'中有两个非零系数，对应于惩罚参数lambda_0的值0.114。我们可以利用这一系列拟合在验证集上做出预测，作为lambda_0的函数，或者通过交叉验证进行更多工作。

6.5.2 Ridge Regression and the Lasso

我们将使用 sklearn.linear_model 包（下文简称 skl）在 Hitters 数据上拟合岭回归和套索正则化线性模型。我们从上节最佳子集回归中计算得到的无截距模型矩阵 X（without an intercept）开始。

岭回归

我们将使用函数 skl.ElasticNet() 来同时拟合岭回归和套索回归。为了拟合岭回归模型的*path*，我们使用skl.ElasticNet.path()，其可以拟合岭回归和套索回归，以及混合模型；岭回归对应于l1_ratio=0。在这些应用中，如果变量以不同的单位测量，标准化X的列是一种良好的做法。由于skl.ElasticNet()不进行归一化，我们必须自己处理这一点。由于我们先进行了标准化，为了找到原始尺度上的系数估计，我们必须对系数估计进行*unstandardize*。在(6.5)和(6.7)中的参数λ在sklearn中被称为alphas。为了与本章其余部分保持一致，我们在后续内容中使用lambdas而不是alphas。[10]

skl.ElasticNet()
skl.ElasticNet.path()

[10] At the time of publication, ridge fits like the one in code chunk [22] issue unwarranted convergence warning messages; we expect these to disappear as this package matures.

In [22]:
```
Xs = X - X.mean(0)[None,:]
X_scale = X.std(0)
Xs = Xs / X_scale[None,:]
lambdas = 10**np.linspace(8, -2, 100) / Y.std()
soln_array = skl.ElasticNet.path(Xs,
                                 Y,
                                 l1_ratio=0.,
                                 alphas=lambdas)[1]
soln_array.shape
```

Out[22]: (19, 100)

这里我们提取与正则化路径上解相对应的系数数组。默认情况下，`skl.ElasticNet.path` 方法会沿着一系列自动选择的 λ 值拟合路径，但当 `l1_ratio=0` 时例外，此时会执行岭回归（如此处所示）。[11] 因此，这里我们选择在一组值范围上实现该函数，这些值从 $\lambda = 10^8$ 到 $\lambda = 10^{-2}$ 乘以 y 的标准差缩放，基本上覆盖了从仅包含截距的零模型到最小二乘拟合的完整场景范围。

与λ的每个值相关联的是一个岭回归系数向量，可以通过`soln_array`的一列来访问。在这种情况下，`soln_array`是一个19×100矩阵，其中有19行（每个预测变量一行）和100列（λ的每个值一列）。

我们将此矩阵转置并转换为数据框，以便于查看和绘图。

In [23]:
```
soln_path = pd.DataFrame(soln_array.T,
                         columns=D.columns,
                         index=-np.log(lambdas))
soln_path.index.name = 'negative log(lambda)'
soln_path
```

Out[23]:

negative log(lambda)	AtBat	Hits	HmRun	Runs	...
-12.310855	0.000800	0.000889	0.000695	0.000851	...
-12.078271	0.001010	0.001122	0.000878	0.001074	...
-11.845686	0.001274	0.001416	0.001107	0.001355	...
-11.613102	0.001608	0.001787	0.001397	0.001710	...
-11.380518	0.002029	0.002255	0.001763	0.002158	...
...	...	...	...	...	...

100 rows × 19 columns

我们绘制路径以了解系数如何随 λ 变化。为了控制图例的位置，我们首先在绘图方法中将 `legend` 设置为 `False`，之后使用 `ax` 的 `legend()` 方法添加它。

In [24]:
```
path_fig, ax = subplots(figsize=(8,8))
soln_path.plot(ax=ax, legend=False)
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
```

[11] The reason is rather technical; for all models except ridge, we can find the smallest value of λ for which all coefficients are zero. For ridge this value is ∞.

```
ax.set_ylabel('Standardized coefficients', fontsize=20)
ax.legend(loc='upper left');
```

(我们在水平标签中使用了 latex 格式化，以便适当地格式化希腊字母 λ。) 我们预期，当使用较大的 λ 值时，系数估计在 ℓ_2 范数方面会比使用较小的 λ 值时小得多。（回想一下，ℓ_2 范数是系数值平方和的平方根。）我们展示第40步的系数，其中 λ 为25.535。

In [25]:
```
beta_hat = soln_path.loc[soln_path.index[39]]
lambdas[39], beta_hat
```

Out[25]:
```
(25.535,
 AtBat            5.433750
 Hits             6.223582
 HmRun            4.585498
 Runs             5.880855
 RBI              6.195921
 Walks            6.277975
 Years            5.299767
 ...              ...
```

让我们计算标准化系数的 ℓ_2 范数。

In [26]:
```
np.linalg.norm(beta_hat)
```

Out[26]:
```
24.17
```

相反，当 λ 为 2.44e-01 时，这是 ℓ_2 范数。请注意，与此较小的 λ 值相关的系数的 ℓ_2 范数要大得多。

In [27]:
```
beta_hat = soln_path.loc[soln_path.index[59]]
lambdas[59], np.linalg.norm(beta_hat)
```

Out[27]:
```
(0.2437, 160.4237)
```

上面我们预先对 X 进行了归一化，并使用 Xs 拟合岭模型。sklearn 中的 Pipeline() 对象提供了一种清晰的方式，将特征归一化与岭模型本身的拟合分开。

In [28]:
```
ridge = skl.ElasticNet(alpha=lambdas[59], l1_ratio=0)
scaler = StandardScaler(with_mean=True,  with_std=True)
pipe = Pipeline(steps=[('scaler', scaler), ('ridge', ridge)])
pipe.fit(X, Y)
```

我们表明，它给出了与之前在标准化数据上拟合时相同的ℓ_2范数。

In [29]:
```
np.linalg.norm(ridge.coef_)
```

Out[29]:
```
160.4237
```

注意，上面的操作 pipe.fit(X, Y) 改变了 ridge 对象，特别是添加了之前不存在的诸如 coef_ 之类的属性。

估计岭回归的测试误差

对于岭回归，选择$a\ priori$值为λ是困难的，甚至是不可能的。我们将希望使用验证方法或交叉验证来选择调优参数。读者可能不会惊讶，`Pipeline()`方法可以用于`skm.cross_validate()`中，采用验证方法(即`validation`)或k折交叉验证。

我们固定分割器的随机状态，以便获得的结果是可重复的。

In [30]:
```
validation = skm.ShuffleSplit(n_splits=1,
                              test_size=0.5,
                              random_state=0)
ridge.alpha = 0.01
results = skm.cross_validate(ridge,
                             X,
                             Y,
                             scoring='neg_mean_squared_error',
                             cv=validation)
-results['test_score']
```

Out[30]:
```
array([134214.0])
```

测试MSE为1.342e+05。注意，如果我们仅拟合一个带有截距的模型，我们将使用训练观测值的均值来预测每个测试观测值。通过拟合一个$very$大的λ值的岭回归模型，我们可以得到相同的结果。注意`1e10`表示10^{10}。

In [31]:
```
ridge.alpha = 1e10
results = skm.cross_validate(ridge,
                             X,
                             Y,
                             scoring='neg_mean_squared_error',
                             cv=validation)
-results['test_score']
```

Out[31]:
```
array([231788.32])
```

显然，选择$\lambda = 0.01$是任意的，因此我们将使用交叉验证或验证集方法来选取调优参数λ。对象`GridSearchCV()`允许进行详尽的网格搜索以选择这样的参数。

我们首先使用验证集方法来选择λ。

`GridSearchCV()`

In [32]:
```
param_grid = {'ridge__alpha': lambdas}
grid = skm.GridSearchCV(pipe,
                        param_grid,
                        cv=validation,
                        scoring='neg_mean_squared_error')
grid.fit(X, Y)
grid.best_params_['ridge__alpha']
grid.best_estimator_
```

Out[32]:
```
Pipeline(steps=[('scaler', StandardScaler()),
                ('ridge', ElasticNet(alpha=0.005899, l1_ratio=0))])
```

Alternatively, we can use 5-fold cross-validation.

In [33]:
```
grid = skm.GridSearchCV(pipe,
                        param_grid,
                        cv=kfold,
                        scoring='neg_mean_squared_error')
grid.fit(X, Y)
grid.best_params_['ridge__alpha']
grid.best_estimator_
```

Recall we set up the `kfold` object for 5-fold cross-validation on page 271. We now plot the cross-validated MSE as a function of $-\log(\lambda)$, which has shrinkage decreasing from left to right.

In [34]:
```
ridge_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
            -grid.cv_results_['mean_test_score'],
            yerr=grid.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_ylim([50000,250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated MSE', fontsize=20);
```

One can cross-validate different metrics to choose a parameter. The default metric for `skl.ElasticNet()` is test R^2. Let's compare R^2 to MSE for cross-validation here.

In [35]:
```
grid_r2 = skm.GridSearchCV(pipe,
                           param_grid,
                           cv=kfold)
grid_r2.fit(X, Y)
```

Finally, let's plot the results for cross-validated R^2.

In [36]:
```
r2_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
            grid_r2.cv_results_['mean_test_score'],
            yerr=grid_r2.cv_results_['std_test_score'] / np.sqrt(K)
                )
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated $R^2$', fontsize=20);
```

Fast Cross-Validation for Solution Paths

The ridge, lasso, and elastic net can be efficiently fit along a sequence of λ values, creating what is known as a *solution path* or *regularization path*. Hence there is specialized code to fit such paths, and to choose a suitable value of λ using cross-validation. Even with identical splits the results will not agree *exactly* with our `grid` above because the standardization of each feature in `grid` is carried out on each fold, while in `pipeCV` below it is carried out only once. Nevertheless, the results are similar as the normalization is relatively stable across folds.

In [37]:
```
ridgeCV = skl.ElasticNetCV(alphas=lambdas,
                           l1_ratio=0,
                           cv=kfold)
pipeCV = Pipeline(steps=[('scaler', scaler),
```

```
                          ('ridge', ridgeCV)])
pipeCV.fit(X, Y)
```

让我们再次绘制交叉验证误差图，以查看其是否与使用skm.GridSearchCV时相似。

In [38]:
```
tuned_ridge = pipeCV.named_steps['ridge']
ridgeCV_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
            tuned_ridge.mse_path_.mean(1),
            yerr=tuned_ridge.mse_path_.std(1) / np.sqrt(K))
ax.axvline(-np.log(tuned_ridge.alpha_), c='k', ls='--')
ax.set_ylim([50000,250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated MSE', fontsize=20);
```

我们看到，使得交叉验证误差最小的 λ 值为 1.19e-02，可通过值tuned_ridge.alpha_ 获取。与这个 λ 值相关的测试 MSE 是多少?

In [39]:
```
np.min(tuned_ridge.mse_path_.mean(1))
```

Out[39]:
```
115526.71
```

这比我们使用 $\lambda = 4$ 得到的测试 MSE 有了进一步的改进。最后，tuned_ridge.coef_ 的系数是在 λ 的该值上对整个数据集拟合得到的。

In [40]:
```
tuned_ridge.coef_
```

Out[40]:
```
array([-222.80877051,  238.77246614,    3.21103754,   -2.93050845,
          3.64888723,  108.90953869,  -50.81896152, -105.15731984,
        122.00714801,   57.1859509 ,  210.35170348,  118.05683748,
       -150.21959435,   30.36634231,  -61.62459095,   77.73832472,
         40.07350744,  -25.02151514,  -13.68429544])
```

正如预期的那样，没有一个系数为零——岭回归并不进行变量选择!

评估交叉验证岭回归的测试误差

使用交叉验证选择 λ 提供了一个单一的回归估计器，类似于我们在第3章中看到的拟合线性回归模型。因此，估计其测试误差是合理的。我们在这里遇到了一个问题，因为交叉验证将使用其所有数据来选择 λ，因此我们没有更多的数据来估计测试误差。一个折衷方案是首先将数据分成两个不相交的集合：训练集和测试集。然后，我们在训练集上拟合一个经过交叉验证调优的岭回归，并在测试集上评估其性能。我们可以称之为嵌套在验证集方法中的交叉验证。先验地，没有理由在验证中为这两个集合各使用一半的数据。下面，我们使用75%用于训练和25%用于测试，估计器是使用5折交叉验证调优的岭回归。这可以通过以下代码实现：

In [41]:
```
outer_valid = skm.ShuffleSplit(n_splits=1,
                               test_size=0.25,
                               random_state=1)
inner_cv = skm.KFold(n_splits=5,
                     shuffle=True,
                     random_state=2)
ridgeCV = skl.ElasticNetCV(alphas=lambdas,
                           l1_ratio=0,
                           cv=inner_cv)
pipeCV = Pipeline(steps=[('scaler', scaler),
                         ('ridge', ridgeCV)]);
```

In [42]:
```
results = skm.cross_validate(pipeCV,
                             X,
                             Y,
                             cv=outer_valid,
                             scoring='neg_mean_squared_error')
-results['test_score']
```

Out[42]:
```
array([132393.84])
```

Lasso

我们看到，在Hitters数据集上，选择恰当的λ进行岭回归，其表现既能优于最小二乘法，也能超越空模型。现在我们要问，套索方法是否能比岭回归提供更准确或更易于解释的模型。为了拟合套索模型，我们再次使用ElasticNetCV()函数；不过，这次我们采用参数l1_ratio=1。除此之外，我们的操作与拟合岭回归模型时相同。

In [43]:
```
lassoCV = skl.ElasticNetCV(n_alphas=100,
                           l1_ratio=1,
                           cv=kfold)
pipeCV = Pipeline(steps=[('scaler', scaler),
                         ('lasso', lassoCV)])
pipeCV.fit(X, Y)
tuned_lasso = pipeCV.named_steps['lasso']
tuned_lasso.alpha_
```

Out[43]:
```
3.147
```

In [44]:
```
lambdas, soln_array = skl.Lasso.path(Xs,
                                     Y,
                                     l1_ratio=1,
                                     n_alphas=100)[:2]
soln_path = pd.DataFrame(soln_array.T,
                         columns=D.columns,
                         index=-np.log(lambdas))
```

从标准化系数的系数图中我们可以看出，根据调优参数的选择，某些系数将精确等于零。

In [45]:
```
path_fig, ax = subplots(figsize=(8,8))
soln_path.plot(ax=ax, legend=False)
ax.legend(loc='upper left')
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Standardized coefficiients', fontsize=20);
```

最小的交叉验证误差低于空模型和最小二乘法的测试集MSE，且与岭回归（第278页）的测试MSE 115526.71非常相似，其中λ是通过交叉验证选择的。

In [46]:
```
np.min(tuned_lasso.mse_path_.mean(1))
```

Out[46]:
```
114690.73
```

让我们再次生成交叉验证误差的图表。

In [47]:
```
lassoCV_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(tuned_lasso.alphas_),
            tuned_lasso.mse_path_.mean(1),
            yerr=tuned_lasso.mse_path_.std(1) / np.sqrt(K))
ax.axvline(-np.log(tuned_lasso.alpha_), c='k', ls='--')
ax.set_ylim([50000,250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated MSE', fontsize=20);
```

然而，套索回归相比岭回归有一个显著优势，即所得系数估计是稀疏的。在这里我们看到，19个系数估计中有6个恰好为零。因此，通过交叉验证选择λ的套索模型仅包含13个变量。

In [48]:
```
tuned_lasso.coef_
```

Out[48]:
```
array([-210.01008773,  243.4550306 ,    0.        ,    0.        ,
          0.        ,   97.69397357,  -41.52283116,   -0.        ,
          0.        ,   39.62298193,  205.75273856,  124.55456561,
       -126.29986768,   15.70262427,  -59.50157967,   75.24590036,
         21.62698014,  -12.04423675,   -0.        ])
```

与岭回归一样，我们可以通过先将数据分为测试集和训练集，并在训练集内部运行交叉验证，来评估交叉验证套索的测试误差。我们将此留作练习。

6.5.3 PCR and PLS Regression

主成分回归

主成分回归（PCR）可以使用 `PCA()` 从 `sklearn.decomposition` 模块。我们现在将 PCR 应用于 `Hitters` 数据，以预测 `Salary`。同样，确保已按第 6.5.1 节所述从数据中移除缺失值。

`PCA()`

我们在这里使用 `LinearRegression()` 来拟合回归模型。注意默认情况下它会拟合截距，这与前面在第6.5.1节看到的`OLS()`函数不同。

`Linear Regression()`

```
In[49]: pca = PCA(n_components=2)
        linreg = skl.LinearRegression()
        pipe = Pipeline([('pca', pca),
                         ('linreg', linreg)])
        pipe.fit(X, Y)
        pipe.named_steps['linreg'].coef_
```

```
Out[49]: array([0.09846131, 0.4758765 ])
```

在执行 PCA 时，结果会因数据是否已进行 *standardized* 而有所不同。正如前面的例子所示，这可以通过在管道中添加一个额外的步骤来实现。

```
In[50]: pipe = Pipeline([('scaler', scaler),
                         ('pca', pca),
                         ('linreg', linreg)])
        pipe.fit(X, Y)
        pipe.named_steps['linreg'].coef_
```

```
Out[50]: array([106.36859204, -21.60350456])
```

我们当然可以使用CV来选择组件的数量，通过使用`skm.GridSearchCV`，在这种情况下固定参数以变化`n_components`。

```
In[51]: param_grid = {'pca__n_components': range(1, 20)}
        grid = skm.GridSearchCV(pipe,
                                param_grid,
                                cv=kfold,
                                scoring='neg_mean_squared_error')
        grid.fit(X, Y)
```

让我们像对其他方法一样，将结果绘制出来。

```
In[52]: pcr_fig, ax = subplots(figsize=(8,8))
        n_comp = param_grid['pca__n_components']
        ax.errorbar(n_comp,
                    -grid.cv_results_['mean_test_score'],
                    grid.cv_results_['std_test_score'] / np.sqrt(K))
        ax.set_ylabel('Cross-validated MSE', fontsize=20)
        ax.set_xlabel('# principal components', fontsize=20)
        ax.set_xticks(n_comp[::2])
        ax.set_ylim([50000,250000]);
```

我们发现，当使用17个成分时，交叉验证误差最小。然而，从图中我们还可以看出，当模型中仅包含一个成分时，交叉验证误差大致相同。这表明，仅使用少量成分的模型可能就足够了。

为1至19（含）每个可能的成分数量提供了CV分数。如果我们仅使用`n_components=0`尝试拟合截距，`PCA()`方法会报错，因此我们也仅针对这些分割计算空模型的MSE。

```
In[53]: Xn = np.zeros((X.shape[0], 1))
        cv_null = skm.cross_validate(linreg,
```

```
                              Xn,
                              Y,
                              cv=kfold,
                              scoring='neg_mean_squared_error')
-cv_null['test_score'].mean()
```

```
Out[53]: 204139.31
```

我们的PCA对象的explained_variance_ratio_属性使用不同数量的成分提供了预测变量和响应中的*percentage of variance explained*。这一概念在第12.2节中有更详细的讨论。

```
In [54]: pipe.named_steps['pca'].explained_variance_ratio_
```

```
Out[54]: array([0.3831424 , 0.21841076])
```

简而言之，我们可以将其视为通过M个主成分所捕获的关于预测变量的信息量。例如，设置M =为1时，仅能捕获38.31%的方差；而设置M =为2时，则额外捕获了21.84%，总计达到60.15%的方差。到M =为6时，这一比例增加至88.63%。此后，增量逐渐减少，直至我们使用全部$M = p$ =个19个成分，从而捕获了100%的方差。

偏最小二乘法

Partial least squares (PLS) is implemented in the PLSRegression() function.

PLS Regression()

```
In [55]: pls = PLSRegression(n_components=2,
                             scale=True)
         pls.fit(X, Y)
```

As was the case in PCR, we will want to use CV to choose the number of components.

```
In [56]: param_grid = {'n_components':range(1, 20)}
         grid = skm.GridSearchCV(pls,
                                 param_grid,
                                 cv=kfold,
                                 scoring='neg_mean_squared_error')
         grid.fit(X, Y)
```

As for our other methods, we plot the MSE.

```
In [57]: pls_fig, ax = subplots(figsize=(8,8))
         n_comp = param_grid['n_components']
         ax.errorbar(n_comp,
                     -grid.cv_results_['mean_test_score'],
                     grid.cv_results_['std_test_score'] / np.sqrt(K))
         ax.set_ylabel('Cross-validated MSE', fontsize=20)
         ax.set_xlabel('# principal components', fontsize=20)
         ax.set_xticks(n_comp[::2])
         ax.set_ylim([50000,250000]);
```

交叉验证误差在12处达到最小，尽管此点与更低的如2或3个成分之间差异并不明显。

6.6 练习

Conceptual

1. 我们在单个数据集上执行最佳子集选择、前向逐步选择和后向逐步选择。对于每种方法，我们获得 $p + 1$ 个模型，包含 0, 1, 2, . . . , p 个预测变量。请解释你的答案:

(a) 在采用k个预测变量的三个模型中，哪一个的$training$ RSS最小? (b) 在采用k个预测变量的三个模型中，哪一个的$test$ RSS最小? (c) 判断正误:

i. 通过前向逐步选择确定的 k 变量模型中的预测变量是通过前向逐步选择确定的 (k+1) 变量模型中的预测变量的子集。 ii. 通过后向逐步选择确定的 k 变量模型中的预测变量是通过后向逐步选择确定的 ($k + 1$) 变量模型中的预测变量的子集。 iii. 通过后向逐步选择确定的 k 变量模型中的预测变量是通过前向逐步选择确定的 ($k + 1$) 变量模型中的预测变量的子集。 iv. 通过前向逐步选择确定的 k 变量模型中的预测变量是通过后向逐步选择确定的 (k+1) 变量模型中的预测变量的子集。 v. 通过最佳子集选择确定的 k 变量模型中的预测变量是通过最佳子集选择确定的 ($k + 1$) 变量模型中的预测变量的子集。

2. 对于(a)至(c)部分，指出i至iv中哪一项是正确的，并证明你的答案。

(a) 套索回归，相对于最小二乘法，是:

i. 当其偏差的增加小于方差的减少时，模型更为灵活，因此将提供更高的预测准确性。ii. 当其方差的增加小于偏差的减少时，模型更为灵活，因此将提供更高的预测准确性。iii. 当其偏差的增加小于方差的减少时，模型较不灵活，因此将提供更高的预测准确性。iv. 当其方差的增加小于偏差的减少时，模型较不灵活，因此将提供更高的预测准确性。(b) 对于岭回归相对于最小二乘法，重复(a)部分。(c) 对于非线性方法相对于最小二乘法，重复(a)部分。

3. 假设我们通过最小化来估计线性回归模型中的回归系数

$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2 \quad \text{subject to} \quad \sum_{j=1}^{p}|\beta_j|\le s$$

对于 s 的特定值。对于 (a) 到 (e) 部分，指出i.到v.中哪个是正确的。证明你的答案。

(a) 随着我们从0开始增加s，训练RSS将会： i. 起初增加，然后最终呈倒U形开始下降。 ii. 起初减少，然后最终呈U形开始增加。 iii. 稳步增加。 iv. 稳步减少。 v. 保持不变。 (b) 对测试RSS重复(a)。 (c) 对方差重复(a)。 (d) 对（平方）偏差重复(a)。 (e) 对不可约误差重复(a)。

4. 假设我们通过最小化来估计线性回归模型中的回归系数

$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}\beta_j^2$$

对于{v*}的特定值。对于(a)到(e)部分，指出i到v中哪一个是正确的。证明你的答案。

(a) 当我们从0开始增加 λ 时，训练RSS将会： i. 最初增加，然后最终开始以倒U形下降。 ii. 最初减少，然后最终开始以U形增加。 iii. 稳定增加。 iv. 稳定减少。 v. 保持不变。 (b) 对测试RSS重复(a)。 (c) 对方差重复(a)。 (d) 对（平方）偏差重复(a)。 (e) 对不可约误差重复(a)。

5. 众所周知，岭回归倾向于给相关变量相似的系数值，而套索则可能给相关变量相当不同的系数值。我们现在将在一个非常简单的设定中探索这一性质。



假设 $n = 2$，$p = 2$，$x_{11} = x_{12}$，$x_{21} = x_{22}$。此外，假设 $y_1 + y_2 = 0$ 且 $x_{11} + x_{21} = 0$ 且 $x_{12} + x_{22} = 0$，因此在最小二乘、岭回归或套索模型中的截距估计值为零：$\hat{\beta}_0 = 0$。

(a) 写出这种情况下的岭回归优化问题。(b) 论证在这种情况下，岭回归系数估计满足$\hat{\beta}_1 = \hat{\beta}_2$。(c) 写出这种情况下的套索优化问题。(d) 论证在这种情况下，套索系数$\hat{\beta}_1$和$\hat{\beta}_2$不是唯一的——换句话说，(c)中的优化问题有许多可能的解。描述这些解。

6. 现在我们将进一步探讨(6.12)和(6.13)。

(a) 考虑公式(6.12)，其中p =取1。对于y_1和λ >的某些选择，取0，将(6.12)绘制为β_1的函数。你的图应确认(6.12)可由(6.14)求解。(b) 考虑公式(6.13)，其中p =取1。对于y_1和λ >的某些选择，取0，将(6.13)绘制为β_1的函数。你的图应确认(6.13)可由(6.15)求解。

7. 现在我们将推导第6.2.2节中讨论的lasso和岭回归的贝叶斯联系。



(a) 假设$y_i = \beta_0 + \sum_{j=1}^{p} x_{ij}\beta_j + \epsilon_i$，其中$\epsilon_1, \ldots, \epsilon_n$是独立同分布的，来自$N(0, \sigma^2)$分布。写出数据的似然函数。(b) 假设$\beta$的先验如下：$\beta_1, \ldots, \beta_p$独立同分布，遵循均值为0、共同尺度参数为$b$的双指数分布：即$p(\beta) = \frac{1}{2b}\exp(-|\beta|/b)$。在此设定下写出$\beta$的后验分布。(c) 论证在此后验分布下，lasso估计是β的$mode$。(d) 现在假设β的先验如下：$\beta_1, \ldots, \beta_p$独立同分布，遵循均值为零、方差为c的正态分布。在此设定下写出β的后验分布。(e) 论证在此后验分布下，岭回归估计既是β的$mode$也是$mean$。

Applied

8. 在这个练习中，我们将生成模拟数据，然后将使用这些数据执行前向和后向逐步选择。

(a) 创建一个随机数生成器，并使用其`normal()`方法生成长度为$n = 100$的预测器X，以及长度为$n = 100$的噪声向量ϵ。

(b) 根据模型生成一个长度为 $n = 100$ 的响应向量 Y

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon,$$

其中 β_0、β_1、β_2 和 β_3 是您选择的常数。(c) 使用前向逐步选择来选择一个包含预测变量 $X, X^2, \ldots, X^{10}$ 的模型。根据 C_p 得到的模型是什么？报告所得模型的系数。(d) 重复 (c)，使用后向逐步选择。您的答案与 (c) 中的结果相比如何？(e) 现在对模拟数据拟合一个 lasso 模型，再次使用 $X, X^2, \ldots, X^{10}$ 作为预测变量。使用交叉验证来选择 λ 的最优值。创建交叉验证误差作为 λ 函数的图。报告所得的系数估计，并讨论所得结果。(f) 现在根据模型生成响应向量 Y

$$Y = \beta_0 + \beta_7 X^7 + \epsilon,$$

并执行前向逐步选择和套索回归。讨论所得结果。

9. 在本练习中，我们将利用`College`数据集中的其他变量来预测收到的申请数量。

(a) 将数据集分为训练集和测试集。(b) 在训练集上使用最小二乘法拟合线性模型，并报告得到的测试误差。(c) 在训练集上拟合岭回归模型，其中λ通过交叉验证选择。报告得到的测试误差。(d) 在训练集上拟合套索模型，其中λ通过交叉验证选择。报告得到的测试误差，以及非零系数估计的数量。(e) 在训练集上拟合PCR模型，其中M通过交叉验证选择。报告得到的测试误差，以及通过交叉验证选择的M值。(f) 在训练集上拟合PLS模型，其中M通过交叉验证选择。报告得到的测试误差，以及通过交叉验证选择的M值。

(g) 对所得结果进行评论。我们预测收到的大学申请数量的准确度如何？这五种方法产生的测试误差之间有很大差异吗?

10. 我们已经看到，随着模型中使用的特征数量增加，训练误差必然会减少，但测试误差却不一定。现在我们将在一个模拟数据集中探索这一点。

(a) 生成一个数据集，包含 $p = 20$个特征，$n = 1{,}000$个观测值，以及根据模型生成的关联定量响应向量。

$$Y = X\beta + \epsilon,$$

其中 β 有一些元素恰好等于零。

(b) 将您的数据集拆分为包含100个观测值的训练集和包含900个观测值的测试集。

(c) 对训练集进行最佳子集选择，并绘制与每个大小的最佳模型相关的训练集MSE。(d) 绘制与每个大小的最佳模型相关的测试集MSE。(e) 测试集MSE在哪一个模型大小上达到最小值？对你的结果进行评论。如果它在仅包含截距的模型或包含所有特征的模型上达到最小值，那么请调整你在(a)中生成数据的方式，直到你得到一个测试集MSE在中间模型大小上最小化的情景。(f) 测试集MSE最小化的模型与用于生成数据的真实模型相比如何？对系数值进行评论。(g) 创建一个图，显示$\sqrt{\sum_{j=1}^{p}(\beta_j - \hat{\beta}_j^r)^2}$在一系列$r$值上的情况，其中$\hat{\beta}_j^r$是包含$r$个系数的最佳模型中第$j$个系数估计值。对你观察到的情况进行评论。这与(d)中的测试MSE图相比如何?

---

11. 我们现在将尝试预测`Boston`数据集中的犯罪率。

(a) 尝试本章探讨的一些回归方法，如最佳子集选择、套索、岭回归和PCR。展示并讨论您认为值得考虑的方法的结果。

(b) 提出一个（或一组）在该数据集上表现良好的模型，并说明理由。确保你使用验证集误差、交叉验证或其他合理的替代方法来评估模型性能，而不是使用训练误差。

(c) 你选择的模型是否涉及数据集中的所有特征？为什么？

7
超越线性



到目前为止，本书主要聚焦于线性模型。线性模型在描述和实施上相对简单，并且相较于其他方法，在解释和推断方面具有优势。然而，标准线性回归在预测能力方面可能存在显著局限。这是因为线性假设几乎总是近似，有时甚至是很差的近似。在第6章中，我们看到可以通过岭回归、套索法、主成分回归以及其他技术来改进最小二乘法。在这种情况下，改进是通过降低线性模型的复杂度，从而减少估计的方差实现的。但我们仍然在使用线性模型，其改进空间有限！本章中，我们在放宽线性假设的同时，仍尽可能保持可解释性。我们通过研究线性模型的非常简单扩展，如多项式回归和阶跃函数，以及更复杂的方法，如样条、局部回归和广义可加模型，来实现这一点。

- *Polynomial regression* 通过添加额外的预测变量来扩展线性模型，这些预测变量是通过将每个原始预测变量提升至幂次得到的。例如，一个*cubic*回归使用三个变量X、X^2和X^3作为预测变量。这种方法提供了一种简单的方式来实现数据的非线性拟合。

- *Step functions* 将变量的取值范围切分为K个不同的区域，以产生一个定性变量。这样做的效果是拟合出一个分段常数函数。

- *Regression splines* 比多项式和阶跃函数更灵活，实际上是这两者的扩展。它们涉及将 X 的范围划分为 K 个不同的区域。在每个区域内，多项式函数被拟合到数据上。然而，这些多项式是

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_7

约束使得它们在区域边界处平滑连接，即{v*}。只要区间被划分成足够多的区域，这就能产生极其灵活的拟合效果。

- *Smoothing splines* 类似于回归样条，但出现在略有不同的情况下。平滑样条是通过最小化受平滑惩罚约束的残差平方和准则而产生的。

- *Local regression* 类似于样条函数，但有一个重要的区别。区域之间允许重叠，而且它们确实以一种非常平滑的方式重叠。

- *Generalized additive models* 允许我们将上述方法扩展到处理多个预测变量。

在第7.1–7.6节中，我们介绍了几种灵活建模响应变量Y与单个预测变量X之间关系的方法。在第7.7节中，我们展示了这些方法可以无缝整合，以便将响应变量Y建模为多个预测变量$X_1, \ldots, X_p$的函数。

7.1 多项式回归

历史上，将线性回归扩展到预测变量与响应之间关系为非线性的情况的标准方法是用标准线性模型替代

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

使用多项式函数

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \cdots + \beta_d x_i^d + \epsilon_i, \tag{7.1}$$

其中 ϵ_i 是误差项。这种方法被称为 *polynomial regression*，实际上，在3.3.2节中我们已经看到了这种方法的一个例子。对于足够大的度数d，多项式回归允许我们生成极其非线性的曲线。注意，(7.1)式中的系数可以通过最小二乘线性回归轻松估计，因为这只是带有预测变量$x_i, x_i^2, x_i^3, \ldots, x_i^d$的标准线性模型。一般来说，使用大于3或4的$d$并不常见，因为对于较大的$d$值，多项式曲线可能变得过于灵活，呈现出一些非常奇怪的形状。这在X变量的边界附近尤其如此。

polynomial regression

图7.1中的左侧面板是针对`Wage`数据集的`wage`与`age`的散点图，该数据集包含居住在美国中大西洋地区男性的收入与人口统计信息。我们看到了使用最小二乘法拟合一个四次多项式的结果（实心蓝色曲线）。尽管这和其他任何线性回归模型一样，但个别系数并不特别令人感兴趣。相反，我们考察整个拟合函数在从18到80的`age`的63个值组成的网格上的情况，以理解`age`与`wage`之间的关系。

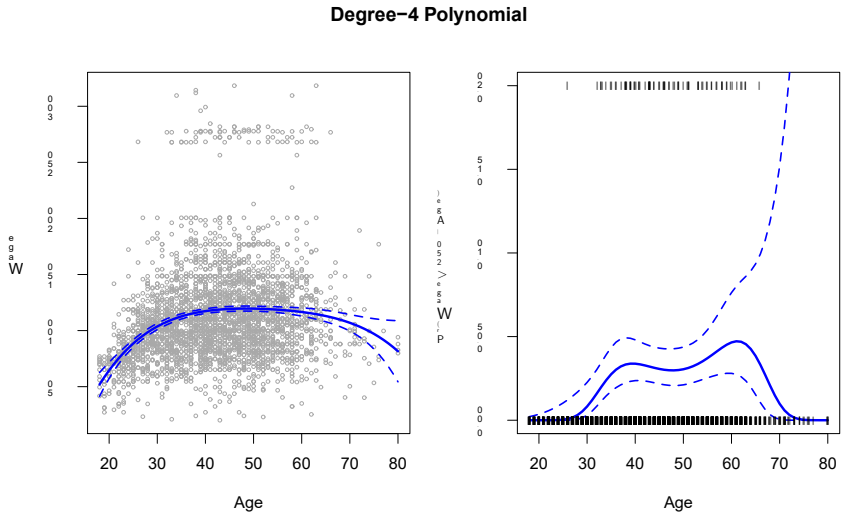


图7.1。*The* `Wage` *data.* 左：*The solid blue curve is a degree-4 polynomial of* `wage` *(in thousands of dollars) as a function of* `age`*, fit by least squares. The dashed curves indicate an estimated 95 % confidence interval.* 右：*We model the binary event* `wage>250` *using logistic regression, again with a degree-4 polynomial. The fitted posterior probability of* `wage` *exceeding* $250,000 *is shown in blue, along with an estimated 95 % confidence interval.*

在图7.1中，一对虚线伴随拟合曲线，它们表示（2×）标准误差曲线。让我们看看这些是如何产生的。假设我们已经计算了在特定`age`、x_0值下的拟合：

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \hat{\beta}_2 x_0^2 + \hat{\beta}_3 x_0^3 + \hat{\beta}_4 x_0^4. \tag{7.2}$$

拟合的方差是多少，即Var$\hat{f}(x_0)$？最小二乘法为每个拟合系数$\hat{\beta}_j$返回方差估计值，以及系数估计对之间的协方差。我们可以使用这些来计算$\hat{f}(x_0)$的估计方差。[1] $\hat{f}(x_0)$的估计*pointwise*标准误是这个方差的平方根。在每个参考点x_0重复此计算，我们绘制拟合曲线，以及拟合曲线两侧两倍标准误。我们绘制两倍标准误是因为，对于正态分布的误差项，这个量对应于大约95%的置信区间。

图7.1中的工资似乎来自两个不同的群体：似乎有一个年收入超过250000美元的*high earners*群体，以及一个*low earners*群体。我们可以通过将`wage`分成这两组来将其视为二元变量。然后可以使用逻辑回归来预测这个二元响应，使用`age`的多项式函数。

[1] If $\hat{\mathbf{C}}$ is the 5×5 covariance matrix of the $\hat{\beta}_j$, and if $\ell_0^T = (1, x_0, x_0^2, x_0^3, x_0^4)$, then $\text{Var}[\hat{f}(x_0)] = \ell_0^T \hat{\mathbf{C}} \ell_0$.

作为预测变量。换句话说，我们拟合模型

$$\Pr(y_i > 250|x_i) = \frac{\exp(\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d)}{1 + \exp(\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d)}. \quad (7.3)$$

结果如图7.1右侧面板所示。面板顶部和底部的灰色标记分别表示高收入者和低收入者的年龄。实线蓝色曲线表示作为age函数的高收入者的拟合概率。同时展示了估计的95%置信区间。我们看到，这里的置信区间相当宽，尤其是在右侧。尽管该数据集的样本量相当大（$n = 3{,}000$），但只有79名高收入者，这导致估计系数的方差很大，因此置信区间很宽。

7.2 阶跃函数

在线性模型中使用特征的多项式函数作为预测变量，会对X的非线性函数施加*global*结构。我们可以改用*step functions*，以避免施加这种全局结构。这里

我们将 X 的范围划分成 *bins*，并在每个区间内拟合一个不同的常数。这相当于将一个连续变量转换为一个 *ordered categorical variable*。

step function

ordered categorical variable

更详细地说，我们在X的范围内创建割点c_1、$c_2, \ldots, c_K$，然后构造$K+$个新变量

$$\begin{aligned} C_0(X) &= I(X < c_1), \\ C_1(X) &= I(c_1 \le X < c_2), \\ C_2(X) &= I(c_2 \le X < c_3), \\ &\vdots \\ C_{K-1}(X) &= I(c_{K-1} \le X < c_K), \\ C_K(X) &= I(c_K \le X), \end{aligned} \quad (7.4)$$

其中 $I(\cdot)$ 是一个 *indicator function*，如果条件为真，则返回 1，否则返回0。例如，若$c_K \le X$，则$I(c_K \le X)$等于1，否则等于0。这些有时被称为*dummy*变量。注意对于任何X的值，$C_0(X) + C_1(X) + \cdots + C_K(X) = 1$，因为$X$必须恰好位于$K+1$个区间之一。随后，我们使用最小二乘法，以$C_1(X), C_2(X), \ldots, C_K(X)$作为预测变量[2]来拟合线性模型：

指示函数

$$y_i = \beta_0 + \beta_1 C_1(x_i) + \beta_2 C_2(x_i) + \cdots + \beta_K C_K(x_i) + \epsilon_i. \quad (7.5)$$

对于给定的 X 值，$C_1, C_2, \ldots, C_K$ 中至多有一个可以非零。注意，当 $X < c_1$ 时，(7.5) 中的所有预测变量均为零，因此 β_0 可以

[2]We exclude $C_0(X)$ as a predictor in (7.5) because it is redundant with the intercept. This is similar to the fact that we need only two dummy variables to code a qualitative variable with three levels, provided that the model will contain an intercept. The decision to exclude $C_0(X)$ instead of some other $C_k(X)$ in (7.5) is arbitrary. Alternatively, we could include $C_0(X), C_1(X), \ldots, C_K(X)$, and exclude the intercept.

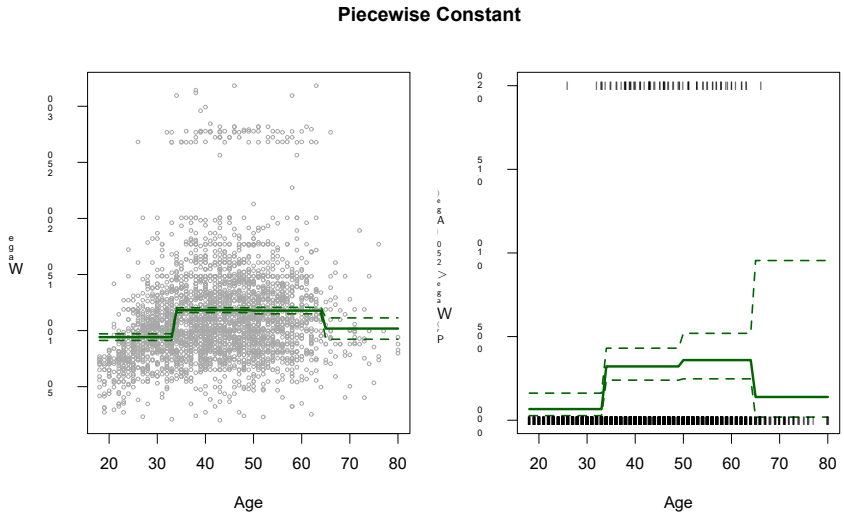


图 7.2. *The* `Wage` *data.* 左： *The solid curve displays the fitted value from a least squares regression of* `wage` *(in thousands of dollars) using step functions of* `age`*. The dashed curves indicate an estimated 95 % confidence interval.* 右: *We model the binary event* `wage>250` *using logistic regression, again using step functions of* `age`*. The fitted posterior probability of* `wage` *exceeding* $250,000 *is shown, along with an estimated 95 % confidence interval.*

应被解释为$X < c_1$的Y均值。相比之下，(7.5)预测了对$c_j \leq X < c_{j+1}$的响应为$\beta_0+\beta_j$，因此β_j表示在$c_j \leq X < c_{j+1}$中相对于$X < c_1$，X响应的平均增加量。

图7.2左侧面板展示了一个将阶跃函数拟合到图7.1中`Wage`数据的示例。我们还拟合了逻辑回归模型

$$\Pr(y_i > 250|x_i) = \frac{\exp(\beta_0 + \beta_1 C_1(x_i) + \cdots + \beta_K C_K(x_i))}{1 + \exp(\beta_0 + \beta_1 C_1(x_i) + \cdots + \beta_K C_K(x_i))} \tag{7.6}$$

为了根据`age`预测个体成为高收入者的概率。图7.2的右侧面板展示了使用这种方法得到的拟合后验概率。

遗憾的是，除非预测变量中存在自然的断点，否则分段常数函数可能会错过关键变化。例如，在图7.2左侧面板中，第一个箱体明显未能捕捉到`wage`随`age`增加的趋势。尽管如此，阶梯函数方法在生物统计学、流行病学以及其他学科中非常受欢迎。例如，常用5年年龄组来定义箱体。

7.3 基函数

多项式和分段常数回归模型实际上是*basis function*方法的特例。其思路是手头有一个家-

basis function

可以应用于变量 X 的函数或变换族：$b_1(X), b_2(X), \ldots, b_K(X)$。与其在 X 中拟合线性模型，我们拟合模型

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \beta_3 b_3(x_i) + \cdots + \beta_K b_K(x_i) + \epsilon_i. \quad (7.7)$$

请注意，基函数$b_1(\cdot), b_2(\cdot), \ldots, b_K(\cdot)$是固定且已知的。（换言之，我们预先选择这些函数。）对于多项式回归，基函数为$b_j(x_i) = x_i^j$，而对于分段常数函数，它们为$b_j(x_i) = I(c_j \leq x_i < c_{j+1})$。我们可以将(7.7)视为一个带有预测变量$b_1(x_i), b_2(x_i), \ldots, b_K(x_i)$的标准线性模型。因此，我们可以使用最小二乘法来估计(7.7)中的未知回归系数。重要的是，这意味着第3章讨论的所有线性模型推断工具，如系数估计的标准误和模型整体显著性的F统计量，在此情境下均适用。

到目前为止，我们已经考虑了使用多项式函数和分段常数函数作为基函数；然而，还有许多其他选择是可行的。例如，我们可以使用小波或傅里叶级数来构建基函数。在下一节中，我们将探讨一种非常常见的基函数选择：*regression splines*。

regression spline

7.4 回归样条

现在我们讨论一类灵活的基础函数，它扩展了我们刚刚看到的 polynomial r egression 和 piecewise constant regression 方法。

7.4.1 Piecewise Polynomials

与其在整个X范围内拟合高次多项式，*piece-wise polynomial regression*涉及拟合单独的低次多项式
在X的不同区域上。例如，分段三次多项式通过拟合形式为 的三次回归模型来工作。

分段多项式回归

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \epsilon_i, \quad (7.8)$$

其中系数 β_0、β_1、β_2 和 β_3 在 X 的范围内不同部分有所差异。系数发生变化的点称为 *knots*。

knot

例如，一个没有节点的分段三次多项式就是标准的三次多项式，如（7.1）中所示，其中 $d =$ 为3。一个在点 c 处有一个节点的分段三次多项式形式为

$$y_i = \begin{cases} \beta_{01} + \beta_{11} x_i + \beta_{21} x_i^2 + \beta_{31} x_i^3 + \epsilon_i & \text{if } x_i < c \\ \beta_{02} + \beta_{12} x_i + \beta_{22} x_i^2 + \beta_{32} x_i^3 + \epsilon_i & \text{if } x_i \geq c. \end{cases}$$

换句话说，我们对数据拟合了两个不同的多项式函数，一个是在观测子集 $x_i < c$上，另一个是在观测子集$x_i \geq c$上。第一个多项式函数的系数为

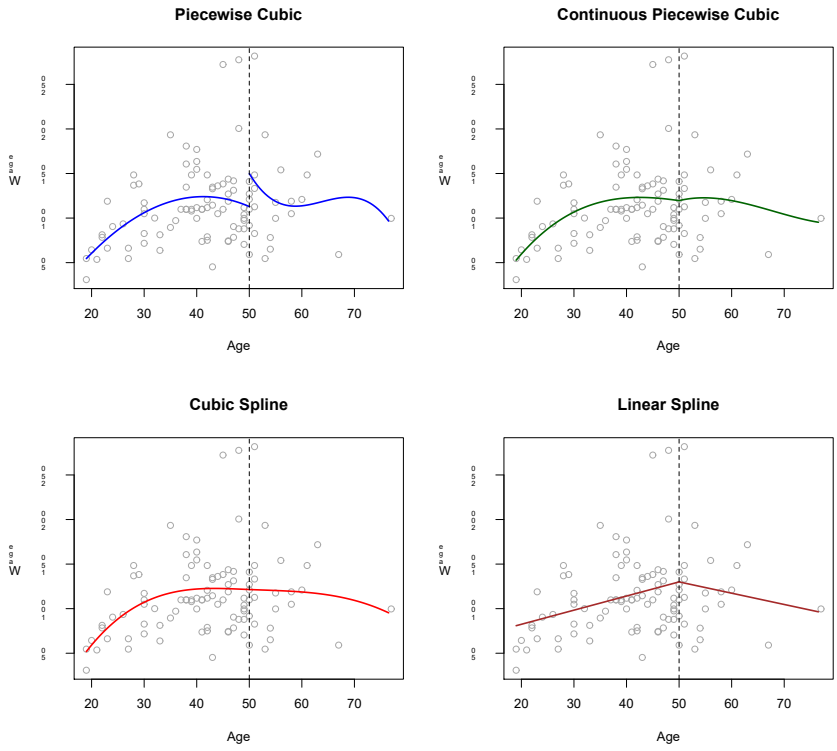


图7.3。 *Various piecewise polynomials are fit to a subset of the* `Wage` *data, with a knot at* `age=50`. 左上： *The cubic polynomials are unconstrained.* 右上： *The cubic polynomials are constrained to be continuous at* `age=50`. 左下： *The cubic polynomials are constrained to be continuous, and to have continuous first and second derivatives.* 右下： *A linear spline is shown, which is constrained to be continuous.*

$\beta_{01}, \beta_{11}, \beta_{21}$, 和 β_{31}，第二个具有系数 $\beta_{02}, \beta_{12}, \beta_{22}$, 和 β_{32}。这些多项式函数中的每一个都可以通过应用最小二乘法对原始预测变量的简单函数进行拟合。

使用更多的结点会导致更灵活的分段多项式。一般来说，如果我们在X的范围内放置K个不同的结点，那么最终我们将拟合$K+$个不同的三次多项式。注意，我们并不一定需要使用三次多项式。例如，我们可以改为拟合分段线性函数。事实上，我们在第7.2节中的分段常数函数就是零次的分段多项式！

图7.3左上角展示了针对`Wage`数据子集进行的分段三次多项式拟合，其中在`age=50`处有一个结点。我们立刻发现一个问题：该函数不连续，看起来荒谬至极！由于每个多项式有四个参数，我们在拟合这个分段多项式模型时总共使用了八个*degrees of freedom*。

degrees of freedom

7.4.2 Constraints and Splines

图7.3左上角的面板看起来不对，因为拟合曲线过于灵活。为了解决这个问题，我们可以在*constraint*下拟合一个分段多项式，要求拟合曲线必须是连续的。换句话说，当age=50时不能有跳跃。图7.3右上角的图显示了结果拟合。这看起来比左上角的图更好，但V形连接看起来不自然。

在左下角的图中，我们增加了两个额外约束：现在分段多项式的一阶和二阶*derivatives*都是连续的。
在 age=50 处。换句话说，我们要求分段多项式不仅在 age=50 时连续，而且非常 *smooth*。我们对分段三次多项式施加的每个约束实际上都会减少所得分段多项式拟合的复杂度，从而释放一个自由度。因此，在左上图中，我们使用了八个自由度，但在左下图中，我们施加了三个约束（连续性、一阶导数的连续性和二阶导数的连续性），因此剩下五个自由度。左下图中 的曲线称为*cubic spline*。[3] 一般来说，具有 K 个节点的三次样条使用

导数

cubic spline

总共 $4 + K$ 个自由度。

在图7.3中，右下角的图是一个*linear spline*，它是连续的
在 age=50 处。d 次样条的一般定义是：它是一个分段 d 次多项式，并且在每个结点处具有直到 $d-1$ 次的导数连续性。因此，线性样条是通过在由结点定义的预测变量空间的每个区域内拟合一条直线来得到的，并要求在每个结点处连续。

线性样条

在图7.3中，age=50处有一个单结节。当然，我们可以添加更多的结节，并在每个结节处施加连续性。

7.4.3 The Spline Basis Representation

我们上一节刚刚看到的回归样条可能显得有些复杂：在它（以及可能它的前 $d-$ 阶导数）连续的约束下，我们如何拟合一个分段 d 次多项式？事实证明，我们可以使用基模型（7.7）来表示回归样条。一个具有 K 个结点的三次样条可以建模为

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \cdots + \beta_{K+3} b_{K+3}(x_i) + \epsilon_i, \qquad (7.9)$$

对于适当选择的基函数$b_1, b_2, \ldots, b_{K+3}$，模型(7.9)随后可以使用最小二乘法进行拟合。

正如表示多项式有多种方式一样，使用(7.9)中不同的基函数选择来表示三次样条也有许多等价的方法。使用(7.9)表示三次样条最直接的方式是，从一个三次多项式的基开始——即x, x^2,和x^3——然后为每个节点添加一个*truncated power basis*函数。

truncated power basis

[3] Cubic splines are popular because most human eyes cannot detect the discontinuity at the knots.

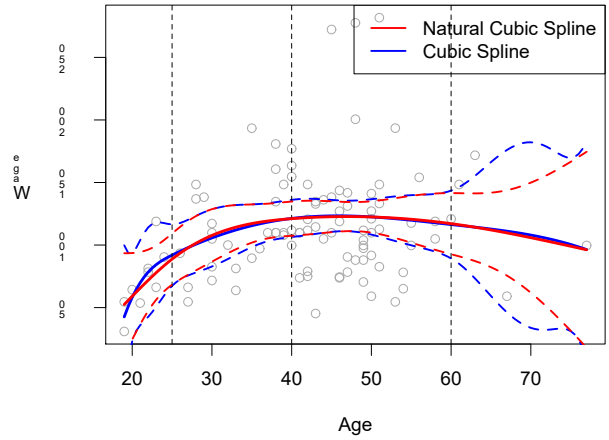


图7.4。*A cubic spline and a natural cubic spline, with three knots, fit to a subset of the* `Wage` *data. The dashed lines denote the knot locations.*

截断幂基函数定义为

$$h(x,\xi) = (x-\xi)_+^3 = \begin{cases} (x-\xi)^3 & \text{if } x > \xi \\ 0 & \text{otherwise,} \end{cases} \tag{7.10}$$

其中 ξ 是节点。可以证明，将形式为 $\beta_4 h(x,\xi)$ 的项添加到三次多项式模型 (7.8) 中，仅会导致在 ξ 处的三阶导数不连续；该函数在每个节点处仍保持连续，且其一阶和二阶导数也连续。

换句话说，为了将三次样条拟合到具有K个节点的数据集，我们执行带有截距和3个$+\ K$预测变量的最小二乘回归，形式为 $X, X^2, X^3, h(X,\xi_1), h(X,\xi_2), \ldots, h(X,\xi_K)$，其中$\xi_1, \ldots, \xi_K$是节点。这相当于估计总共$K\ +$个4回归系数；因此，拟合具有$K$个节点的三次样条使用$K+$个4自由度。

不幸的是，样条在预测变量的外围范围可能会有高方差——也就是说，当X取非常小或非常大的值时。图7.4展示了使用三个节点的`Wage`数据拟合。我们看到边界区域的置信带显得相当不稳定。一个*natu- ral spline*是带有额外*boundary constraints*的回归样条：

natural spline

要求函数在边界处（即 X 小于最小节点或大于最大节点的区域）是线性的。这一额外约束意味着自然样条通常在边界处产生更稳定的估计。在图7.4中，自然三次样条也显示为红线。注意相应的置信区间更窄。

7.4.4 Choosing the Number and Locations of the Knots

当我们拟合样条时，应该在哪里设置节点？回归样条在包含大量节点的区域最为灵活，因为在这些区域，多项式系数可以快速变化。因此，一个

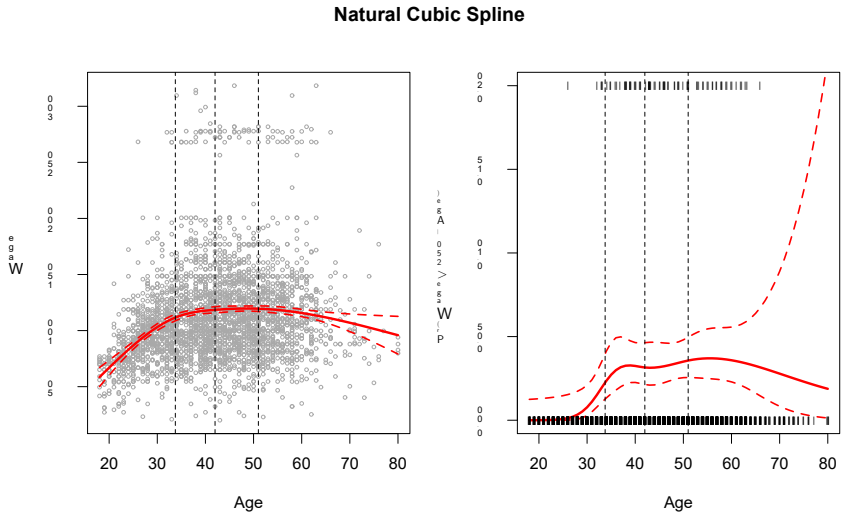


图 7.5。*A natural cubic spline function with four degrees of freedom is fit to the* Wage *data. 左: A spline is fit to* wage *(in thousands of dollars) as a function of* age*. 右: Logistic regression is used to model the binary event* wage>250 *as a function of* age*. The fitted posterior probability of* wage *exceeding* $250,000 *is shown. The dashed lines denote the knot locations.*

一种选择是在我们认为函数可能变化最快的地方放置更多的结点，而在看起来更稳定的地方放置较少的结点。尽管这种选择可能效果很好，但在实践中，通常是以统一的方式放置结点。实现这一点的一种方法是，指定所需的自由度，然后让软件自动在数据的均匀分位数处放置相应数量的结点。

图7.5展示了Wage数据的一个例子。如同图7.4，我们拟合了一个具有三个结点的自然三次样条，不同之处在于这次的结点位置自动选为了age的第25、第50和第75百分位数。这是通过请求四个自由度来指定的。四个自由度导致三个内部结点的论点有些技术性。[4]

我们应该使用多少个结点，或者等价地说，我们的样条应该包含多少个自由度？一种选择是尝试不同数量的结点，看看哪种产生的曲线最好看。一个较为客观的方法是使用交叉验证，如第5章和第6章所讨论的。使用这种方法，我们移除一部分数据（比如10%），用一定数量的结点对剩余数据进行样条拟合，然后使用该样条对留出的部分进行预测。我们多次重复这个过程，直到每个观测值都被留出一次，并且

[4]There are actually five knots, including the two boundary knots. A cubic spline with five knots has nine degrees of freedom. But natural cubic splines have two additional *natural* constraints at each boundary to enforce linearity, resulting in $9 - 4 = 5$ degrees of freedom. Since this includes a constant, which is absorbed in the intercept, we count it as four degrees of freedom.

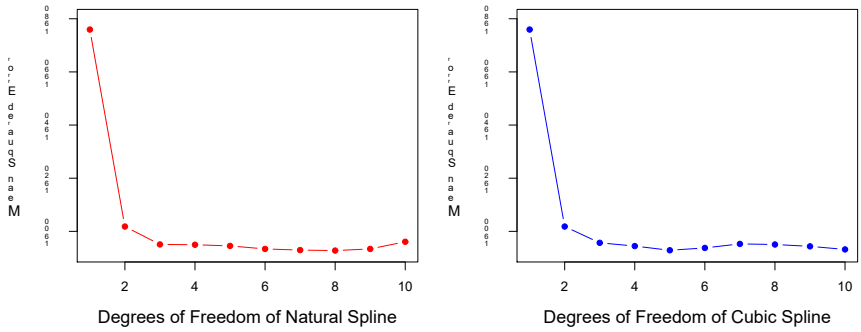


图7.6。*Ten-fold cross-validated mean squared errors for selecting the degrees of freedom when fitting splines to the* `Wage` *data. The response is* `wage` *and the predictor* `age`. 左：*A natural cubic spline.* 右：*A cubic spline.*

然后计算整体交叉验证的RSS。此过程可针对不同的结点数K重复进行。随后，选取使得RSS最小的K值。

图7.6展示了拟合`Wage`数据的、具有不同自由度的样条的十折交叉验证均方误差。左侧面板对应自然三次样条，右侧面板对应三次样条。两种方法产生的结果几乎相同，并有明确证据表明一阶拟合（线性回归）并不充分。两条曲线都迅速趋于平缓，似乎自然样条的三自由度和三次样条的四自由度已经相当足够。

在第7.7节中，我们同时在多个变量上拟合加性样条模型。这可能需要为每个变量选择自由度。在这种情况下，我们通常采取更为务实的做法，将所有项的自由度设为固定数值，比如四。

7.4.5 Comparison to Polynomial Regression

图7.7比较了在`Wage`数据集上，具有15个自由度的自然三次样条与15次多项式。多项式额外的灵活性在边界处产生了不理想的结果，而自然三次样条仍然为数据提供了合理的拟合。回归样条通常比多项式回归提供更好的结果。这是因为与必须使用高次（最高单项式中的指数，例如X^{15}）来产生灵活拟合的多项式不同，样条通过增加节点数量但保持次数不变来引入灵活性。通常，这种方法产生更稳定的估计。样条还允许我们在函数f似乎快速变化的区域放置更多的节点，从而增加灵活性，而在f看起来更稳定的区域放置较少的节点。

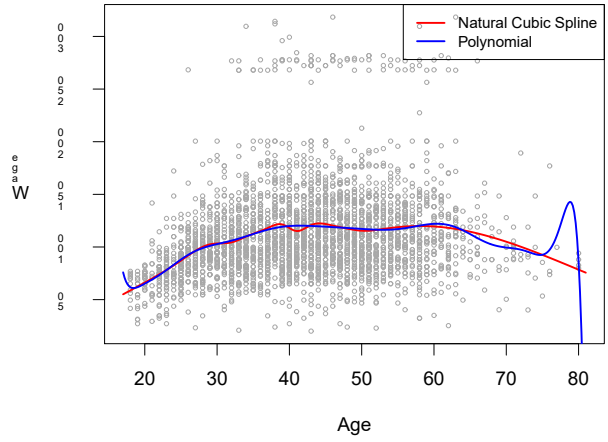


图 7.7. *On the* `Wage` *data set, a natural cubic spline with 15 degrees of freedom is compared to a degree-*15 *polynomial. Polynomials can show wild behavior, especially near the tails.*

7.5 平滑样条

在上一节中，我们讨论了回归样条，其构建过程为指定一组节点，生成一系列基函数，然后利用最小二乘法估计样条的系数。现在我们介绍一种略有不同的方法，它同样能生成样条。

7.5.1 An Overview of Smoothing Splines

在将平滑曲线拟合到一组数据时，我们真正想要做的是找到某个函数，比如$g(x)$，它能很好地拟合观测数据：也就是说，我们希望RSS $= \sum_{i=1}^{n}(y_i - g(x_i))^2$ 很小。然而，这种方法存在一个问题。如果我们对$g(x_i)$不施加任何约束，那么我们总是可以通过选择g使其*interpolates*所有y_i来使RSS为零。这样的函数将极其过度拟合数据——它将过于灵活。我们真正想要的是一个函数g，它使RSS小，但同时也*smooth*。

我们如何确保 g 是光滑的？有多种方法可以实现这一点。一种自然的方法是找到使下式最小化的函数 g

$$\sum_{i=1}^{n}(y_i - g(x_i))^2 + \lambda \int g''(t)^2 dt \tag{7.11}$$

其中 λ 是一个非负的 *tuning parameter*。使 (7.11) 最小化的函数 g 被称为 *smoothing spline*。

smoothing spline

(7.11) 是什么意思？方程 7.11 采用了我们在第 6 章岭回归和套索回归中遇到的“损失+惩罚”公式。项 $\sum_{i=1}^{n}(y_i - g(x_i))^2$ 是一个 *loss function*，它鼓励

loss function

年龄 g 与数据拟合良好，而项 $\lambda \int g''(t)^2 dt$ 是一个 *penalty term*，用于惩罚 g 中的变异性。符号 $g''(t)$ 表示函数 g 的二阶导数。一阶导数 $g'(t)$ 衡量斜率

函数的t处的值，而二阶导数对应于斜率的变化量。因此，从广义上讲，函数的二阶导数是其*roughness*的一个度量：如果$g(t)$在t附近非常波动，则其绝对值很大，否则接近于零。（直线的二阶导数为零；注意直线是完全平滑的。）$\int$符号是一个*integral*，我们可以将其视为对t范围的求和。换句话说，$\int g''(t)^2dt$只是函数$g'(t)$在其整个范围内的总变化的一个度量。如果g非常平滑，那么$g'(t)$将接近常数，$\int g''(t)^2dt$将取一个较小的值。相反，如果g跳跃且多变，那么$g'(t)$将显著变化，$\int g''(t)^2dt$将取一个较大的值。因此，在(7.11)中，$\lambda\int g''(t)^2dt$鼓励g变得平滑。λ的值越大，g将越平滑。

当 $\lambda = 0$ 时，(7.11) 中的惩罚项不起作用，因此函数 g 将非常跳跃，并且会精确地插值训练观测值。当 $\lambda \to \infty$ 时，g 将完全平滑——它将只是一条尽可能接近训练点的直线。事实上，在这种情况下，g 将是线性最小二乘线，因为 (7.11) 中的损失函数等同于最小化残差平方和。对于 λ 的中间值，g 将近似训练观测值，但会有些平滑。我们看到 λ 控制着平滑样条的偏差-方差权衡。

最小化(7.11)的函数$g(x)$可以证明具有一些特殊性质：它是一个分段三次多项式，其节点位于$x_1, \ldots, x_n$的唯一值处，并且在每个节点处具有连续的一阶和二阶导数。此外，在极端节点之外的区域，它是线性的。换句话说，*the function $g(x)$ that minimizes (7.11) is a natural cubic spline with knots at $x_1, \ldots, x_n$!*然而，它并不等同于在$x_1, \ldots, x_n$处应用第7.4.3节所述基函数方法所得到的自然三次样条——而是这种自然三次样条的一个*shrunken*版本，其中(7.11)中调优参数λ的值控制着收缩的程度。

7.5.2 Choosing the Smoothing Parameter λ

我们已经看到，平滑样条本质上是在 x_i 的每个唯一值处都有节点的自然三次样条。看起来平滑样条可能会有过多的自由度，因为每个数据点处都有一个节点赋予了极大的灵活性。但调优参数 λ 控制着平滑样条的粗糙度，从而控制 *effective degrees of freedom*。可以证明，随着 λ 从 0 增加到 ∞，有效

自由度，我们记作 df_λ，从 n 减少到 2。

effective degrees of freedom

在平滑样条的背景下，为什么我们讨论*effective*自由度而不是自由度?通常，自由度指的是自由参数的数量，比如多项式或三次样条中拟合的系数数量。尽管平滑样条有n个参数，因此名义上有n个自由度，但这n个参数受到了严格的约束或缩减。因此，df_λ是平滑样条灵活性的一种度量——其值越高，平滑样条就越灵活（偏差更低但方差更高）。有效自由度的定义

自由在某种程度上有些技术性。我们可以写

$$\hat{\mathbf{g}}_\lambda = \mathbf{S}_\lambda \mathbf{y}, \tag{7.12}$$

其中 $\hat{\mathbf{g}}_\lambda$ 是 (7.11) 在特定选择 λ 下的解——也就是说，它是一个 n 向量，包含平滑样条在训练点 $x_1, \ldots, x_n$ 处的拟合值。方程7.12表明，将平滑样条应用于数据时，拟合值向量可以写成 $n \times n$ 矩阵 $\mathbf{S}_\lambda$ (，其中存在公式) 乘以响应向量 $\mathbf{y}$。那么有效自由度定义为

$$df_\lambda = \sum_{i=1}^{n} \{\mathbf{S}_\lambda\}_{ii}, \tag{7.13}$$

矩阵 $\mathbf{S}_\lambda$ 的对角线元素之和。

在拟合平滑样条时，我们无需选择节点的数量或位置——每个训练观测点处都会有一个节点，$x_1, \ldots, x_n$。相反，我们面临另一个问题：需要选择λ的值。意料之中的是，解决此问题的一种可能方法是交叉验证。换句话说，我们可以找到使交叉验证RSS尽可能小的λ值。事实证明，对于平滑样条，*leave- one-out*交叉验证误差（LOOCV）可以非常高效地计算，其成本与计算单次拟合基本相同，使用的公式如下：

$$\mathrm{RSS}_{cv}(\lambda) = \sum_{i=1}^{n} (y_i - \hat{g}_\lambda^{(-i)}(x_i))^2 = \sum_{i=1}^{n} \left[\frac{y_i - \hat{g}_\lambda(x_i)}{1 - \{\mathbf{S}_\lambda\}_{ii}} \right]^2 .$$

符号 $\hat{g}_\lambda^{(-i)}(x_i)$ 表示在 x_i 处评估的此平滑样条的拟合值，其中拟合使用了除第 i 个观测值 (x_i, y_i) 之外的所有训练观测值。相反，$\hat{g}_\lambda(x_i)$ 表示拟合到所有训练观测值并在 x_i 处评估的平滑样条函数。这个卓越的公式表明，我们可以仅使用 $\hat{g}_\lambda$，即对 *all* 个数据的原始拟合，来计算这 *leave-* 个 *one-out* 拟合中的每一个！[5] 我们在第 5 章第 205 页有一个非常相似的公式 (5.2)，用于最小二乘线性回归。使用 (5.2)，我们可以非常快速地对本章前面讨论的回归样条执行 LOOCV，以及使用任意基函数的最小二乘回归。

图7.8展示了将平滑样条拟合到`Wage`数据的结果。红色曲线表示通过预先指定我们希望使用具有16个有效自由度的平滑样条所获得的拟合。蓝色曲线是在使用LOOCV选择λ时获得的平滑样条；在这种情况下，所选择的λ值导致6.8个有效自由度(使用(7.13)计算)。对于这些数据，两条平滑样条之间几乎没有明显差异，除了16个自由度的样条似乎略微更曲折。由于两种拟合之间差异很小，使用6.8个自由度的平滑样条拟合

[5] The exact formulas for computing $\hat{g}(x_i)$ and $\mathbf{S}_\lambda$ are very technical; however, efficient algorithms are available for computing these quantities.

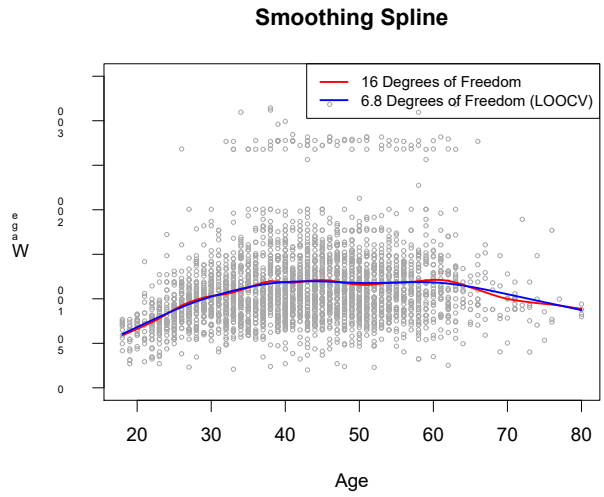


图7.8。 *Smoothing spline fits to the* `Wage` *data. The red curve results from specifying* 16 *effective degrees of freedom. For the blue curve,* λ *was found automatically by leave-one-out cross-validation, which resulted in* 6.8 *effective degrees of freedom.*

更可取，因为通常情况下，除非数据支持更复杂的模型，否则更简单的模型更好。

7.6 局部回归

Local regression 是一种用于拟合灵活非线性函数的不同方法。在目标点x_0处仅使用附近的训练观测值来计算拟合。图7.9通过一些模拟数据说明了这一思想，其中一个目标点接近0.4，另一个接近边界0.05。在这个图中，蓝线表示生成数据的函数$f(x)$，浅橙色线对应于局部回归估计$\hat{f}(x)$。局部回归在算法7.1中描述。

local regression

注意，在算法7.1的第3步中，权重{

为了进行局部回归，需要做出许多选择，例如如何定义权重函数K，以及在步骤3中是采用线性、常数还是二次回归。（公式7.14对应线性回归。）尽管所有这些选择都会产生一定影响，但最重要的选择是*span* s，即用于在x_0处计算局部回归的点的比例，如上述步骤1所定义。跨度所起的作用类似于平滑中的调优参数λ。

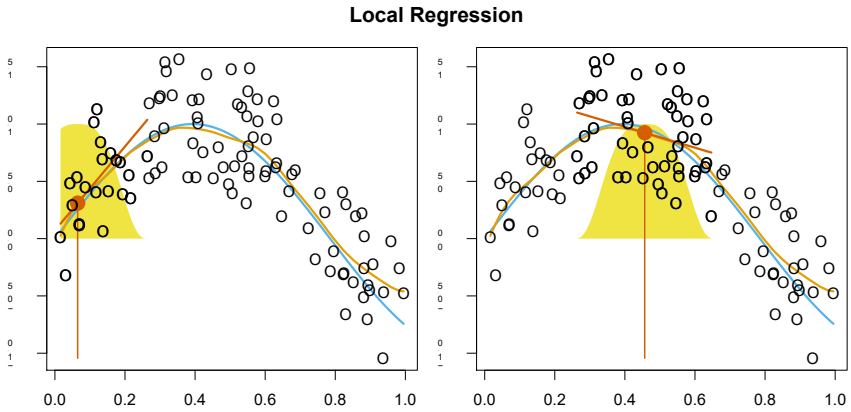


图7.9。 *Local regression illustrated on some simulated data, where the blue curve represents $f(x)$ from which the data were generated, and the light orange curve corresponds to the local regression estimate $\hat{f}(x)$. The orange colored points are local to the target point x_0, represented by the orange vertical line. The yellow bell-shape superimposed on the plot indicates weights assigned to each point, decreasing to zero with distance from the target point. The fit $\hat{f}(x_0)$ at x_0 is obtained by fitting a weighted linear regression (orange line segment), and using the fitted value at x_0 (orange solid dot) as the estimate $\hat{f}(x_0)$.*

样条：它控制非线性拟合的灵活性。s的值越小，我们的拟合将越*local*且波动越大；相反，s的值非常大时，将导致使用所有训练观测值对数据进行全局拟合。我们可以再次使用交叉验证来选择s，或者直接指定它。图7.10展示了在`Wage`数据上的局部线性回归拟合，使用了两个s值：0.7和0.2。正如预期的那样，使用$s = 0.7$获得的拟合比使用$s = 0.2$获得的拟合更平滑。

局部回归的思想可以通过多种不同的方式进行推广。在具有多个特征$X_1, X_2, \ldots, X_p$的设定中，一种非常有用的推广方式是拟合一个多元线性回归模型，该模型在某些变量上是全局的，但在另一个变量（例如时间）上是局部的。这种 *varying coefficient models* 是一种有用的方法，可以使模型适应最近收集到的数据

varying coefficient model

数据。当我们想要拟合在一对变量X_1和X_2中局部的模型，而不是一个变量时，局部回归也非常自然地推广开来。我们可以简单地使用二维邻域，并利用二维空间中接近每个目标点的观测值来拟合双变量线性回归模型。理论上，同样的方法可以在更高维度中实现，使用拟合到p维邻域的线性回归。然而，如果p远大于大约3或4，局部回归的表现可能会很差，因为通常会有非常少的训练观测值接近x_0。在第3章中讨论的最近邻回归在高维度中也面临类似的问题。

算法 7.1 *Local Regression At* $X = x_0$

1. 收集训练点中x_i最接近x_0的分数$s = k/n$。

2. 为邻域中的每个点分配一个权重 $K_{i0} = K(x_i, x_0)$，使得距离 x_0 最远的点权重为零，最近的点权重最高。除这 k 个最近邻之外，所有其他点的权重均为零。

3. 使用上述权重，在x_i上拟合y_i的*weighted least squares regression*，通过找到使以下最小化的$\hat{\beta}_0$和$\hat{\beta}_1$：

$$\sum_{i=1}^{n} K_{i0}(y_i - \beta_0 - \beta_1 x_i)^2. \tag{7.14}$$

4. x_0 处的拟合值由$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$ 给出。

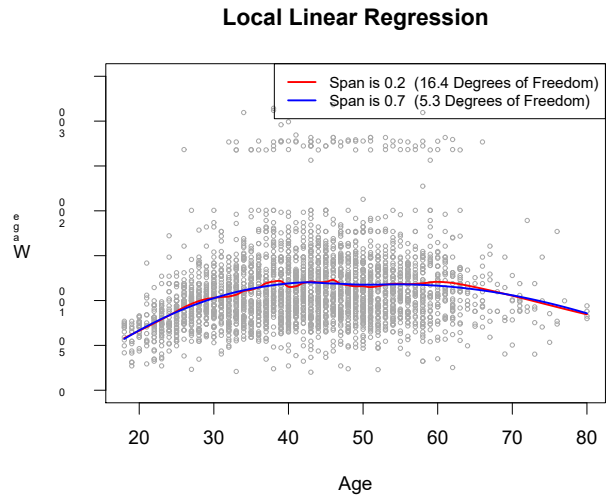


图 7.10. *Local linear fits to the* `Wage` *data. The span specifies the fraction of the data used to compute the fit at each target point.*

7.7 广义加性模型

在7.1至7.6节中，我们介绍了多种基于单一预测变量X灵活预测响应变量Y的方法。这些方法可视为简单线性回归的扩展。此处，我们探讨基于多个预测变量$X_1, \ldots, X_p$灵活预测Y的问题。这相当于多元线性回归的扩展。

Generalized additive models (GAMs) 提供了一个通用框架，用于通过允许每个变量的非线性函数来扩展标准线性模型，同时保持 *additivity*。就像线性模型一样，GAMs可以同时应用于定量和定性响应。我们首先

generalized additive model

additivity

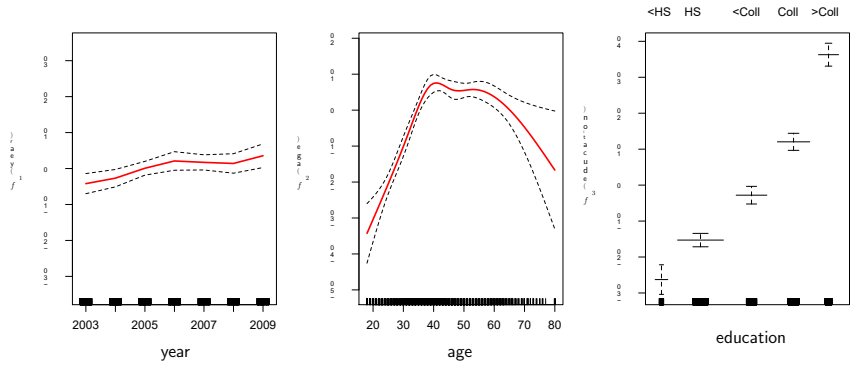


图 7.11. *For the* `Wage` *data, plots of the relationship between each feature and the response,* `wage`*, in the fitted model (7.16). Each plot displays the fitted function and pointwise standard errors. The first two functions are natural splines in* `year` *and* `age`*, with four and five degrees of freedom, respectively. The third function is a step function, fit to the qualitative variable* `education`*.*

在第7.7.1节中，我们探讨了针对定量响应的广义加性模型，随后在第7.7.2节中，转向了针对定性响应的分析。

7.7.1 GAMs for Regression Problems

一种扩展多元线性回归模型的自然方式

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \epsilon_i$$

为了允许每个特征与响应之间存在非线性关系，可以将每个线性分量 $\beta_j x_{ij}$ 替换为一个（平滑的）非线性函数 $f_j(x_{ij})$。然后，我们可以将模型写成

$$\begin{aligned} y_i &= \beta_0 + \sum_{j=1}^{p} f_j(x_{ij}) + \epsilon_i \\ &= \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \epsilon_i. \end{aligned} \tag{7.15}$$

这是一个GAM的示例。它被称为*additive*模型，因为我们为每个X_j计算一个单独的f_j，然后将它们的所有贡献相加。

在第7.1至7.6节中，我们讨论了许多单变量函数拟合方法。广义加性模型（GAM）的妙处在于，我们可以将这些方法作为构建加性模型的基石。实际上，对于本章迄今为止我们所见到的大多数方法而言，实现这一点相当直接。以自然样条为例，考虑拟合模型的任务

$$\texttt{wage} = \beta_0 + f_1(\texttt{year}) + f_2(\texttt{age}) + f_3(\texttt{education}) + \epsilon \tag{7.16}$$

在 `Wage` 数据上。这里 `year` 和 `age` 是定量变量，而变量 `education` 是具有五个水平的定性变量：`<HS`、`HS`、`<Coll`、`Coll`、`>Coll`，指的是个体已完成的高中或大学教育量。我们使用自然样条拟合前两个函数。我们

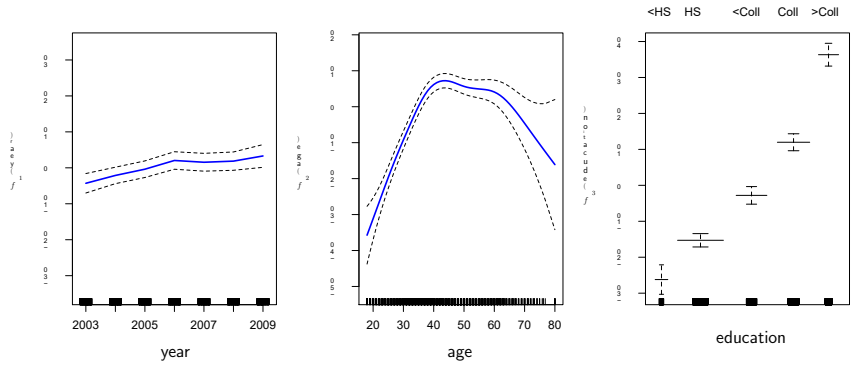


图7.12。*Details are as in Figure 7.11, but now f_1 and f_2 are smoothing splines with four and five degrees of freedom, respectively.*

用每个水平单独的常数来拟合第三个函数，通过第3.3.1节中常用的虚拟变量方法。

图7.11展示了使用最小二乘法拟合模型（7.16）的结果。这一过程易于实现，因为正如第7.4节所讨论的，自然样条可以通过适当选择的基函数集来构建。因此，整个模型实质上是对样条基变量和虚拟变量进行的一个大型回归，所有这些都整合到一个大型回归矩阵中。

图7.11可以很容易地解读。左侧面板表明，在固定`age`和`education`的情况下，`wage`随着`year`略有增加的趋势；这可能是因为通货膨胀。中间面板显示，在固定`education`和`year`的情况下，`wage`在`age`的中间值处趋于最高，在非常年轻和非常年老的人群中最低。右侧面板指出，在固定`year`和`age`的情况下，`wage`随着`education`趋于增加：平均而言，一个人受教育程度越高，其薪资就越高。所有这些都是直观的发现。

图7.12展示了类似的三组图，但这次f_1和f_2分别是具有四度和五度自由度的平滑样条。使用平滑样条拟合GAM并不像使用自然样条那样简单，因为在平滑样条的情况下，不能使用最小二乘法。然而，可以使用标准软件，如`Python`包`pygam`，通过一种方法

`pygam`

backfitting

称为 *backfitting*。该方法拟合了一个涉及多个预测
通过依次对每个预测变量重复更新拟合，同时保持其他变量固定来估计因子。这种方法的妙处在于，每次我们更新一个函数时，只需对该变量应用相应的拟合方法到一个*partial residual*。[6]

图7.11和图7.12中的拟合函数看起来相当相似。在大多数情况下，使用平滑样条与自然样条得到的GAM之间的差异很小。

---

[6] A partial residual for X_3, for example, has the form $r_i = y_i - f_1(x_{i1}) - f_2(x_{i2})$. If we know f_1 and f_2, then we can fit f_3 by treating this residual as a response in a non-linear regression on X_3.

我们不必将样条作为GAM的构建模块：我们同样可以使用局部回归、多项式回归或本章前面看到的任何方法的组合来创建GAM。GAM在本章末尾的实验中将进行更详细的研究。

GAMs 的优缺点

在我们继续之前，让我们总结一下GAM的优势和局限性。

! GAM允许我们为每个X_j拟合一个非线性f_j，这样我们就能自动建模标准线性回归会遗漏的非线性关系。这意味着我们无需手动对每个变量单独尝试多种不同的变换。! 非线性拟合有可能对响应变量Y做出更准确的预测。! 由于模型是可加的，我们可以在保持所有其他变量不变的情况下，单独考察每个X_j对Y的影响。! 变量X_j的函数f_j的平滑度可以通过自由度来概括。“ GAM的主要局限在于模型被限制为可加的。当变量很多时，重要的交互作用可能会被遗漏。然而，与线性回归一样，我们可以通过加入形式为$X_j \times X_k$的额外预测变量，手动向GAM模型添加交互项。此外，我们还可以向模型中添加形式为$f_{jk}(X_j, X_k)$的低维交互函数；这类项可以使用二维平滑器（如局部回归或二维样条（此处不涉及））来拟合。

对于完全通用的模型，我们必须寻找更为灵活的方法，如第8章所述的随机森林和提升法。广义可加模型在线性模型和完全非参数模型之间提供了一个有用的折衷方案。

7.7.2 GAMs for Classification Problems

GAM 也可用于 Y 为定性的情况。为简单起见，这里我们假设 Y 取值为 0 或 1，并令 $p(X) = \Pr(Y = 1|X)$ 为响应等于 1 的条件概率（在给定预测变量的情况下）。回忆逻辑回归模型 (4.6):

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p. \tag{7.17}$$

左侧是$P(Y = 1|X)$与$P(Y = 0|X)$发生比的对数， (7.17)将其表示为预测变量的线性函数。允许非线性关系以扩展(7.17)的一种自然方式是使用模型

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p). \tag{7.18}$$

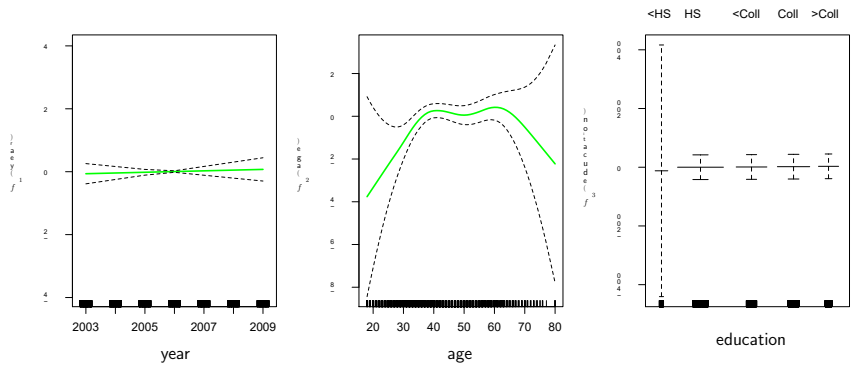


图 7.13. *For the* `Wage` *data, the logistic regression GAM given in (7.19) is fit to the binary response* `I(wage>250)`*. Each plot displays the fitted function and pointwise standard errors. The first function is linear in* `year`*, the second function a smoothing spline with five degrees of freedom in* `age`*, and the third a step function for* `education`*. There are very wide standard errors for the first level* `<HS` *of* `education`*.*

方程 7.18 是一个逻辑回归 GAM。它具有与前一节中讨论的定量响应相同的优缺点。

我们将GAM拟合到`Wage`数据，以预测个人年收入超过250,000美元的概率。我们拟合的GAM形式为

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 \times \texttt{year} + f_2(\texttt{age}) + f_3(\texttt{education}), \quad (7.19)$$

其中

$$p(X) = \Pr(\texttt{wage} > 250|\texttt{year}, \texttt{age}, \texttt{education}).$$

再次使用五自由度的平滑样条对 f_2 进行拟合，并通过为每个教育水平创建哑变量，将 f_3 拟合为阶梯函数。拟合结果如图 7.13 所示。最后一个面板看起来很可疑，水平 `<HS` 的置信区间非常宽。事实上，该类别中没有响应值等于 1 的观测：没有受教育程度低于高中的人年收入超过 250,000 美元。因此，我们重新拟合 GAM，排除受教育程度低于高中的个体。得到的模型如图 7.14 所示。与图 7.11 和图 7.12 一样，三个面板具有相似的垂直尺度。这使我们能够直观地评估每个变量的相对贡献。我们观察到，`age` 和 `education` 对成为高收入者的概率的影响远大于 `year`。

7.8 实验：非线性建模

在本实验中，我们演示了本章讨论的一些非线性模型。我们使用 `Wage` 数据作为贯穿始终的示例，并展示所讨论的许多复杂非线性拟合过程可以在 `Python` 中轻松实现。

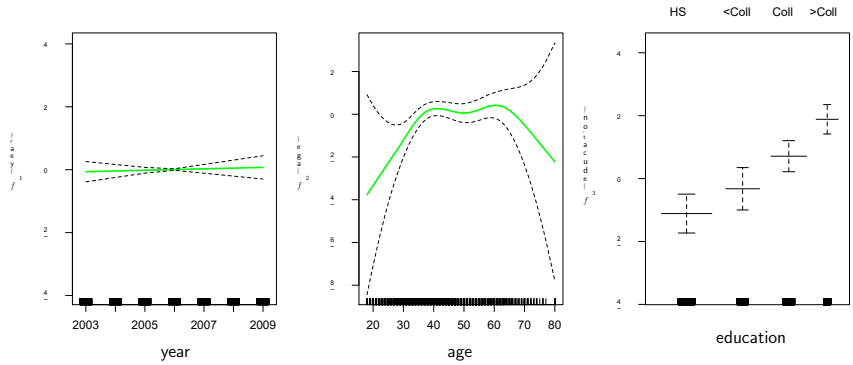


图 7.14. *The same model is fit as in Figure 7.13, this time excluding the observations for which* `education` *is* `<HS`. *Now we see that increased education tends to be associated with higher salaries.*

和往常一样，我们先从一些标准导入开始。

In [1]:
```
import numpy as np, pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (summarize,
                         poly,
                         ModelSpec as MS)
from statsmodels.stats.anova import anova_lm
```

我们再次收集本实验所需的新导入项。其中许多是专门为 `ISLP` 包开发的。

In [2]:
```
from pygam import (s as s_gam,
                   l as l_gam,
                   f as f_gam,
                   LinearGAM,
                   LogisticGAM)

from ISLP.transforms import (BSpline,
                             NaturalSpline)
from ISLP.models import bs, ns
from ISLP.pygam import (approx_lam,
                        degrees_of_freedom,
                        plot as plot_gam,
                        anova as anova_gam)
```

7.8.1 Polynomial Regression and Step Functions

我们首先演示如何复现图7.1。让我们从加载数据开始。

In [3]:
```
Wage = load_data('Wage')
y = Wage['wage']
age = Wage['age']
```

在本实验的大部分内容中，我们的响应变量是Wage['wage']，我们已将其存储为上述的y。正如第3.6.6节所述，我们将使用poly()函数来创建一个模型矩阵，该矩阵将拟合关于age的四次多项式。

In [4]:
```
poly_age = MS([poly('age', degree=4)]).fit(Wage)
M = sm.OLS(y, poly_age.transform(Wage)).fit()
summarize(M)
```

Out[4]:

	coef	std err	t	P>|t|
intercept	111.7036	0.729	153.283	0.000
poly(age, degree=4)[0]	447.0679	39.915	11.201	0.000
poly(age, degree=4)[1]	-478.3158	39.915	-11.983	0.000
poly(age, degree=4)[2]	125.5217	39.915	3.145	0.002
poly(age, degree=4)[3]	-77.9112	39.915	-1.952	0.051

该多项式是使用函数poly()构造的，它使用sklearn术语为特征创建了一个特殊的*transformer* Poly()（诸如 PCA() 在第 6.5.3 节）中看到的变换，它允许在新数据点上轻松评估多项式。这里 poly() 被称为 *helper* 函数，并建立该变换；Poly() 是 actransformer

helper

真正的主力，负责计算变换。另见第118页关于变换的讨论。

在上面的代码中，第一行使用数据框Wage执行了fit()方法。这样会重新计算并存储Poly()在训练数据上所需的任何参数为属性，这些参数将用于后续对transform()方法的所有评估中。例如，它在第二行以及下面开发的绘图函数中都有使用。

我们现在为 age 创建一个值网格，以便在其上进行预测。

In [5]:
```
age_grid = np.linspace(age.min(),
                       age.max(),
                       100)
age_df = pd.DataFrame({'age': age_grid})
```

最后，我们希望绘制数据并添加来自四次多项式的拟合。由于我们将在下面制作几个类似的图，我们首先编写一个函数来创建所有元素并生成图形。我们的函数接受一个模型规格（这里是由变换指定的基），以及一个age值的网格。该函数生成拟合曲线以及95%置信带。通过为basis使用一个参数，我们可以使用几种不同的变换生成并绘制结果，例如我们很快将看到的样条。

In [6]:
```
def plot_wage_fit(age_df,
                  basis,
                  title):

    X = basis.transform(Wage)
    Xnew = basis.transform(age_df)
    M = sm.OLS(y, X).fit()
    preds = M.get_prediction(Xnew)
    bands = preds.conf_int(alpha=0.05)
    fig, ax = subplots(figsize=(8,8))
    ax.scatter(age,
               y,
```

```
                facecolor='gray',
                alpha=0.5)
    for val, ls in zip([preds.predicted_mean,
                        bands[:,0],
                        bands[:,1]],
                       ['b','r--','r--']):
        ax.plot(age_df.values, val, ls, linewidth=3)
    ax.set_title(title, fontsize=20)
    ax.set_xlabel('Age', fontsize=20)
    ax.set_ylabel('Wage', fontsize=20);
    return ax
```

我们包含一个参数 `alpha` 到 `ax.scatter()`，以增加点的透明度。这提供了密度的视觉指示。注意上面 `for` 循环中使用的 `zip()` 函数（见第2.3.8节）。我们有三条线要绘制，每条线具有不同的颜色和线型。在这里，`zip()` 方便地将这些作为迭代器捆绑在循环中。[7]

迭代 或

我们现在使用该函数绘制四次多项式的拟合图。

In [7]:
```
plot_wage_fit(age_df,
              poly_age,
              'Degree-4 Polynomial');
```

使用多项式回归时，我们必须决定所用多项式的次数。有时我们只是随意决定使用二次或三次多项式，仅仅是为了获得非线性拟合。但我们能以更系统的方式做出这样的决定。一种方法是通过假设检验，我们在此进行演示。现在我们拟合一系列模型，从线性（一次）到五次多项式，并试图确定足以解释 `wage` 和 `age` 之间关系的最简单模型。我们使用 `anova_lm()` 函数，它执行一系列 ANOVA 检验。一个 *analysis of variance* 或 ANOVA 检验的原假设是

analysis of variance

模型 $\mathcal{M}_1$ 足以解释数据，相对于需要更复杂模型 $\mathcal{M}_2$ 的备择假设。该判断基于F检验。为了进行检验，模型 $\mathcal{M}_1$ 和 $\mathcal{M}_2$ 必须是 *nested*：由 $\mathcal{M}_1$ 中预测变量张成的空间必须是由 $\mathcal{M}_2$ 中预测变量张成的空间的子空间。在这种情况下，我们拟合五个不同的多项式模型，并依次将较简单的模型与更复杂的模型进行比较。

In [8]:
```
models = [MS([poly('age', degree=d)])
          for d in range(1, 6)]
Xs = [model.fit_transform(Wage) for model in models]
anova_lm(*[sm.OLS(y, X_).fit()
           for X_ in Xs])
```

Out[8]:

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	2998.0	5.022e+06	0.0	NaN	NaN	NaN
1	2997.0	4.793e+06	1.0	228786.010	143.593	2.364e-32
2	2996.0	4.778e+06	1.0	15755.694	9.889	1.679e-03
3	2995.0	4.772e+06	1.0	6070.152	3.810	5.105e-02

[7] In `Python` speak, an "iterator" is an object with a finite number of values, that can be iterated on, as in a loop.

```
4    2994.0  4.770e+06       1.0    1282.563    0.805  3.697e-01
```

注意上面 anova_lm() 行中的 *。这个函数接受可变数量的非关键字参数，在这种情况下是拟合模型。当这些模型以列表形式提供（如这里所做的）时，它必须以 * 为前缀。

比较线性models[0]与二次models[1]的p值基本上为零，表明线性拟合不够充分。[8] 同样，比较二次models[1]与三次models[2]的p值非常低（0.0017），因此二次拟合也不足够。比较三次和四次多项式models[2]和models[3]的p值约为5%，而五次多项式models[4]似乎没有必要，因为其p值为0.37。因此，三次或四次多项式似乎都能为数据提供合理的拟合，但更低阶或更高阶的模型则不成立。

在这种情况下，我们本可以通过利用 poly() 创建正交多项式这一事实，更简洁地获得这些 p 值，而不是使用 anova() 函数。

```
In [9]: summarize(M)
```

```
Out[9]:                             coef  std err        t  P>|t|
intercept                       111.7036    0.729  153.283  0.000
poly(age, degree=4)[0]          447.0679   39.915   11.201  0.000
poly(age, degree=4)[1]         -478.3158   39.915  -11.983  0.000
poly(age, degree=4)[2]          125.5217   39.915    3.145  0.002
poly(age, degree=4)[3]          -77.9112   39.915   -1.952  0.051
```

请注意，p值相同，实际上t统计量的平方等于来自anova_lm()函数的F统计量；例如：

```
In [10]: (-11.983)**2
```

```
Out[10]: 143.59228
```

然而，无论是否使用正交多项式，只要模型是嵌套的，方差分析（ANOVA）方法都适用。例如，我们可以使用 anova_lm() 来比较以下三个模型，这些模型在 education 中都有一个线性项，并且在 age 中有一个不同次数的多项式：

```
In [11]: models = [MS(['education', poly('age', degree=d)])
                   for d in range(1, 4)]
         XEs = [model.fit_transform(Wage)
                for model in models]
         anova_lm(*[sm.OLS(y, X_).fit() for X_ in XEs])
```

```
Out[11]:    df_resid        ssr  df_diff     ss_diff         F      Pr(>F)
         0    2997.0  3.902e+06      0.0         NaN       NaN         NaN
         1    2996.0  3.759e+06      1.0  142862.701   113.992   3.838e-26
         2    2995.0  3.754e+06      1.0    5926.207     4.729   2.974e-02
```

[8] Indexing starting at zero is confusing for the polynomial degree example, since models[1] is quadratic rather than linear!

作为假设检验和ANOVA的替代方法，我们可以使用交叉验证来选择多项式的阶数，如第5章所述。

接下来我们考虑预测个人年收入是否超过250,000美元的任务。我们的处理方式与之前大致相同，只是首先创建相应的响应向量，然后使用二项式族应用glm()函数，以拟合多项式逻辑回归模型。

In [12]:
```
X = poly_age.transform(Wage)
high_earn = Wage['high_earn'] = y > 250 # shorthand
glm = sm.GLM(y > 250,
             X,
             family=sm.families.Binomial())
B = glm.fit()
summarize(B)
```

Out[12]:

	coef	std err	z	P>|z|
intercept	-4.3012	0.345	-12.457	0.000
poly(age, degree=4)[0]	71.9642	26.133	2.754	0.006
poly(age, degree=4)[1]	-85.7729	35.929	-2.387	0.017
poly(age, degree=4)[2]	34.1626	19.697	1.734	0.083
poly(age, degree=4)[3]	-47.4008	24.105	-1.966	0.049

再次使用 get_prediction() 方法进行预测。

In [13]:
```
newX = poly_age.transform(age_df)
preds = B.get_prediction(newX)
bands = preds.conf_int(alpha=0.05)
```

我们现在绘制估计的关系。

In [14]:
```
fig, ax = subplots(figsize=(8,8))
rng = np.random.default_rng(0)
ax.scatter(age +
           0.2 * rng.uniform(size=y.shape[0]),
           np.where(high_earn, 0.198, 0.002),
           fc='gray',
           marker='|')
for val, ls in zip([preds.predicted_mean,
                    bands[:,0],
                    bands[:,1]],
                   ['b','r--','r--']):
    ax.plot(age_df.values, val, ls, linewidth=3)
ax.set_title('Degree-4 Polynomial', fontsize=20)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylim([0,0.2])
ax.set_ylabel('P(Wage > 250)', fontsize=20);
```

我们在图的上方用灰色标记绘制了与wage值高于250的观测相对应的age值，而wage值低于250的则显示为图下方的灰色标记。我们对age值添加了少量噪声以使其稍微抖动，这样具有相同age值的观测就不会相互覆盖。这种类型的图通常被称为*rug plot*。

rug plot

为了拟合阶梯函数，如第7.2节所述，我们首先使用 pd.qcut() 函数根据分位数对 age 进行离散化。然后我们使用

pd.qcut()

pd.get_dummies() 为创建该分类变量的模型矩阵的列-分类变量。请注意，对于给定的分类变量，此函数将包含*all*列，而不是通常的丢弃一个水平的方法。

pd.get_dummies( )

```
In [15]: cut_age = pd.qcut(age, 4)
         summarize(sm.OLS(y, pd.get_dummies(cut_age)).fit())
```

```
Out[15]:                       coef  std err        t  P>|t|
         (17.999, 33.75]    94.1584    1.478  63.692    0.0
           (33.75, 42.0]   116.6608    1.470  79.385    0.0
            (42.0, 51.0]   119.1887    1.416  84.147    0.0
            (51.0, 80.0]   116.5717    1.559  74.751    0.0
```

这里 pd.qcut() 根据分位数 25%、50% 和 75% 自动选择了切点，从而得到四个区域。我们也可以直接指定自己的分位数，而不是使用参数 4。对于不基于分位数的切分，我们会使用 pd.cut() 函数。函数

pd.cut()

pd.qcut() (而 pd.cut()) 返回的是一个有序分类变量。随后的回归模型会生成一组虚拟变量以用于回归分析。由于 age 是模型中唯一的变量，值 $94,158.40 表示33.75岁以下个体的平均薪资，而其他系数则代表其他年龄段的平均薪资。我们可以像处理多项式拟合时那样，生成预测值和图表。

7.8.2 Splines

为了拟合回归样条，我们使用了 ISLP 包中的变换。实际的样条评估函数位于 scipy.interpolate 包中；我们只是像 Poly() 和 PCA() 那样，将它们封装为变换。

在第7.4节中，我们看到可以通过构建适当的基函数矩阵来拟合回归样条。BSpline() 函数生成
具有指定结点集的样条的整个基函数矩阵。默认情况下，产生的B样条是三次的。要改变次数，请使用参数 degree。

BSpline()

```
In [16]: bs_ = BSpline(internal_knots=[25,40,60], intercept=True).fit(age)
         bs_age = bs_.transform(age)
         bs_age.shape
```

```
Out[16]: (3000, 7)
```

这会得到一个七列矩阵，正是3个内部结点的三次样条基所期望的。我们可以使用 bs() 对象来构建同样的矩阵，这有助于将其添加到模型矩阵构建器中（如第7.8.1节所述的 poly() 与其主力 Poly() 的对比）。

我们现在对 Wage 数据拟合一个三次样条模型。

```
In [17]: bs_age = MS([bs('age', internal_knots=[25,40,60])])
         Xbs = bs_age.fit_transform(Wage)
         M = sm.OLS(y, Xbs).fit()
         summarize(M)
```

```
Out[17]:                                          coef  std err  ...
                                     intercept  60.494    9.460  ...
bs(age, internal_knots=[25, 40, 60])[0]   3.980   12.538  ...
bs(age, internal_knots=[25, 40, 60])[1]  44.631    9.626  ...
bs(age, internal_knots=[25, 40, 60])[2]  62.839   10.755  ...
bs(age, internal_knots=[25, 40, 60])[3]  55.991   10.706  ...
bs(age, internal_knots=[25, 40, 60])[4]  50.688   14.402  ...
bs(age, internal_knots=[25, 40, 60])[5]  16.606   19.126  ...
```

列名有点繁琐，导致我们在打印摘要时不得不截断。可以在构造时使用 `name` 参数设置它们，如下所示。

```
In[18]: bs_age = MS([bs('age',
                        internal_knots=[25,40,60],
                        name='bs(age)')])
        Xbs = bs_age.fit_transform(Wage)
        M = sm.OLS(y, Xbs).fit()
        summarize(M)
```

```
Out[18]:                          coef std err      t   P>|t|
         intercept       60.494   9.460  6.394   0.000
bs(age, knots)[0]         3.981  12.538  0.317   0.751
bs(age, knots)[1]        44.631   9.626  4.636   0.000
bs(age, knots)[2]        62.839  10.755  5.843   0.000
bs(age, knots)[3]        55.991  10.706  5.230   0.000
bs(age, knots)[4]        50.688  14.402  3.520   0.000
bs(age, knots)[5]        16.606  19.126  0.868   0.385
```

注意，这里有 6 个样条系数而非 7 个。这是因为默认情况下，`bs()` 假定 `intercept=False`，因为模型中通常包含一个全局截距。因此，它会根据给定的结点生成样条基，然后为了适应截距而丢弃其中一个基函数。

我们也可以使用`df` (自由度)选项来指定样条的复杂度。从上面可以看到，当有3个节点时，样条基有6列或6个自由度。当我们指定`df=6`而不是实际节点时，`bs()`将生成一个样条，其3个节点选在训练数据的均匀分位数处。我们可以最方便地直接使用`Bspline()`来查看这些选定的节点：

```
In[19]: BSpline(df=6).fit(age).internal_knots_
```

```
Out[19]: array([33.75, 42.0, 51.0])
```

当要求六个自由度时，变换会在年龄33.75、42.0和51.0处选择结点，这些分别对应于`age`的第25、第50和第75百分位数。

使用B样条时，我们不必局限于三次多项式（即`degree=3`）。例如，使用`degree=0`会得到分段常数函数，如同我们上面`pd.qcut()`的例子所示。

```
In[20]: bs_age0 = MS([bs('age',
                         df=3,
                         degree=0)]).fit(Wage)
        Xbs0 = bs_age0.transform(Wage)
        summarize(sm.OLS(y, Xbs0).fit())
```

Out[20]:

	coef	std err	t	P>|t|
intercept	94.158	1.478	63.687	0.0
bs(age, df=3, degree=0)[0]	22.349	2.152	10.388	0.0
bs(age, df=3, degree=0)[1]	24.808	2.044	12.137	0.0
bs(age, df=3, degree=0)[2]	22.781	2.087	10.917	0.0

应将此拟合与单元格 [15] 进行比较，在那里我们使用 `qcut()` 在 `age` 的 25%、50% 和 75% 分位数处切分以创建四个分箱。由于我们在此为0阶样条指定了 `df=3`，因此在相同的三个分位数处也会有结点。虽然系数看起来不同，但我们看到这是编码不同的结果。例如，第一个系数在两种情况下相同，并且是第一个分箱中的平均响应。对于第二个系数，我们有 $94.158 + 22.349 = 116.507 \approx 116.611$，后者是单元格 [15] 中第二个分箱的均值。这里截距由一列1编码，因此第二、第三和第四个系数是那些分箱的增量。为什么总和不是完全相同？事实证明，`qcut()` 使用 $\leq$，而 `bs()` 在决定分箱归属时使用 $<$。

为了拟合自然样条，我们使用 `NaturalSpline()` 变换使用相应的辅助函数`ns()`。这里我们拟合了一个具有五个自由度（不包括截距）的自然样条，并绘制了结果。

`NaturalSpline()`

In [21]:
```
ns_age = MS([ns('age', df=5)]).fit(Wage)
M_ns = sm.OLS(y, ns_age.transform(Wage)).fit()
summarize(M_ns)
```

Out[21]:

	coef	std err	t	P>|t|
intercept	60.475	4.708	12.844	0.000
ns(age, df=5)[0]	61.527	4.709	13.065	0.000
ns(age, df=5)[1]	55.691	5.717	9.741	0.000
ns(age, df=5)[2]	46.818	4.948	9.463	0.000
ns(age, df=5)[3]	83.204	11.918	6.982	0.000
ns(age, df=5)[4]	6.877	9.484	0.725	0.468

我们现在使用绘图函数来绘制自然样条。

In [22]:
```
plot_wage_fit(age_df,
              ns_age,
              'Natural spline, df=5');
```

7.8.3 Smoothing Splines and GAMs

平滑样条是GAM的一个特例，采用平方误差损失和单一特征。为了在Python中拟合GAM，我们将使用`pygam`包。可以通过`pip install pygam`进行安装。估计器`LinearGAM()`使用平方误差损失。GAM 通过将模型矩阵的每一列与特定的平滑操作相关联来指定：`s` 用于平滑样条；`l` 用于线性变量，`f` 用于因子或分类变量。下面传递给 `s` 的参数 `0` 表明此平滑器将应用于特征矩阵的第一列。在下面，我们向其传递一个单列矩阵：`X_age`。参数 `lam` 是惩罚参数 λ，如第 7.5.2 节所述。

`pygam`

`LinearGAM()`

In [23]:
```
X_age = np.asarray(age).reshape((-1,1))
gam = LinearGAM(s_gam(0, lam=0.6))
gam.fit(X_age, y)
```

```
Out[23]: LinearGAM(callbacks=[Deviance(), Diffs()], fit_intercept=True,
           max_iter=100, scale=None, terms=s(0) + intercept, tol=0.0001,
           verbose=False)
```

pygam 库通常期望一个特征矩阵，因此我们将 age 重塑为矩阵（二维数组），而不是向量（即一维数组）。在调用 reshape() 方法时，-1 告诉 numpy 根据形状元组的其余条目来推断该维度的大小。

让我们研究拟合如何随平滑参数 lam 变化。函数 np.logspace() 与 np.linspace() 类似，但间隔点
在对数尺度上均匀分布。下面我们将lam从10^{-2}变化到10^{6}。 np.logspace()

```
In [24]: fig, ax = subplots(figsize=(8,8))
         ax.scatter(age, y, facecolor='gray', alpha=0.5)
         for lam in np.logspace(-2, 6, 5):
             gam = LinearGAM(s_gam(0, lam=lam)).fit(X_age, y)
             ax.plot(age_grid,
                     gam.predict(age_grid),
                     label='{:.1e}'.format(lam),
                     linewidth=3)
         ax.set_xlabel('Age', fontsize=20)
         ax.set_ylabel('Wage', fontsize=20);
         ax.legend(title='$\lambda$');
```

pygam 包可以执行最优平滑参数的搜索。

```
In [25]: gam_opt = gam.gridsearch(X_age, y)
         ax.plot(age_grid,
                 gam_opt.predict(age_grid),
                 label='Grid search',
                 linewidth=4)
         ax.legend()
         fig
```

或者，我们可以使用 ISLP.pygam 包中的一个函数来固定平滑样条的自由度。下面我们找到一个 λ 的值，它大约给出四个自由度。这里我们注意到，这些自由度包括平滑样条中未惩罚的截距项和线性项，因此至少有两个自由度。

```
In [26]: age_term = gam.terms[0]
         lam_4 = approx_lam(X_age, age_term, 4)
         age_term.lam = lam_4
         degrees_of_freedom(X_age, age_term)
```

```
Out[26]: 4.000000100004728
```

让我们在与上述类似的图中改变自由度。我们选择自由度作为期望的自由度加一，以考虑到这些平滑样条总是有一个截距项的事实。因此，df 的值为一就是线性拟合。

```
In [27]: fig, ax = subplots(figsize=(8,8))
         ax.scatter(X_age,
                    y,
```

```
           facecolor='gray',
           alpha=0.3)
for df in [1,3,4,8,15]:
    lam = approx_lam(X_age, age_term, df+1)
    age_term.lam = lam
    gam.fit(X_age, y)
    ax.plot(age_grid,
            gam.predict(age_grid),
            label='{:d}'.format(df),
            linewidth=4)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
ax.legend(title='Degrees of freedom');
```

包含多个项的可加模型

广义加性模型的优势在于其能够比线性模型更灵活地拟合多变量回归模型。我们展示了两种方法：第一种采用更手动的方式，利用自然样条和分段常数函数；第二种则使用`pygam`包和平滑样条。

我们现在手动拟合一个GAM，使用`year`和`age`的自然样条函数来预测`wage`，并将`education`视为定性预测变量，如(7.16)中所示。由于这只是一个使用适当选择基函数的大型线性回归模型，我们可以简单地使用`sm.OLS()`函数来实现。

我们将以更手动的方式构建模型矩阵，因为在构建部分依赖图时，我们希望分别访问各个部分。

In [28]:
```
ns_age = NaturalSpline(df=4).fit(age)
ns_year = NaturalSpline(df=5).fit(Wage['year'])
Xs = [ns_age.transform(age),
      ns_year.transform(Wage['year']),
      pd.get_dummies(Wage['education']).values]
X_bh = np.hstack(Xs)
gam_bh = sm.OLS(y, X_bh).fit()
```

这里函数`NaturalSpline()`是支持`ns()`辅助函数的主力。我们选择使用分类变量`education`指示矩阵的所有列，从而使截距变得多余。最后，我们将三个组件矩阵水平堆叠以形成模型矩阵`X_bh`。

我们现在展示如何为我们的基本GAM中的每个项构建偏依赖图。给定`age`和`year`的网格，我们可以手动完成这一过程。我们只需用新的X矩阵进行预测，每次固定除一个特征外的所有特征。

In [29]:
```
age_grid = np.linspace(age.min(),
                       age.max(),
                       100)
X_age_bh = X_bh.copy()[:100]
X_age_bh[:] = X_bh[:].mean(0)[None,:]
X_age_bh[:,:4] = ns_age.transform(age_grid)
preds = gam_bh.get_prediction(X_age_bh)
bounds_age = preds.conf_int(alpha=0.05)
```

```
partial_age = preds.predicted_mean
center = partial_age.mean()
partial_age -= center
bounds_age -= center
fig, ax = subplots(figsize=(8,8))
ax.plot(age_grid, partial_age, 'b', linewidth=3)
ax.plot(age_grid, bounds_age[:,0], 'r--', linewidth=3)
ax.plot(age_grid, bounds_age[:,1], 'r--', linewidth=3)
ax.set_xlabel('Age')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of age on wage', fontsize=20);
```

让我们详细解释一下上面所做的。其思路是创建一个新的预测矩阵，其中除了属于age的列之外，所有列都是常数（并设为其训练数据的均值）。age的四列则用自然样条基在age_grid中的100个值上进行评估后填入。

1. 我们在age中生成了一个长度为100的网格，并创建了一个矩阵X_age_bh，该矩阵有100行，列数与X_bh相同。2. 我们将此矩阵的每一行替换为原矩阵的列均值。3. 接着，我们仅将代表age的前四列替换为在age_grid中的值处计算得到的自然样条基。

剩下的步骤现在应该已经很熟悉了。

我们还研究year对wage的影响；过程是一样的。

In [30]:
```
year_grid = np.linspace(2003, 2009, 100)
year_grid = np.linspace(Wage['year'].min(),
                        Wage['year'].max(),
                        100)
X_year_bh = X_bh.copy()[:100]
X_year_bh[:] = X_bh[:].mean(0)[None,:]
X_year_bh[:,4:9] = ns_year.transform(year_grid)
preds = gam_bh.get_prediction(X_year_bh)
bounds_year = preds.conf_int(alpha=0.05)
partial_year = preds.predicted_mean
center = partial_year.mean()
partial_year -= center
bounds_year -= center
fig, ax = subplots(figsize=(8,8))
ax.plot(year_grid, partial_year, 'b', linewidth=3)
ax.plot(year_grid, bounds_year[:,0], 'r--', linewidth=3)
ax.plot(year_grid, bounds_year[:,1], 'r--', linewidth=3)
ax.set_xlabel('Year')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of year on wage', fontsize=20);
```

我们现在使用平滑样条而不是自然样条来拟合模型（7.16）。模型（7.16）中的所有项同时拟合，彼此考虑以解释响应。pygam包仅适用于矩阵，因此我们必须将分类序列education转换为其数组表示，这可以通过education的cat.codes属性找到。由于year只有7个唯一值，我们仅为其使用七个基函数。

In [31]:
```
gam_full = LinearGAM(s_gam(0) +
                     s_gam(1, n_splines=7) +
                     f_gam(2, lam=0))
Xgam = np.column_stack([age,
                        Wage['year'],
                        Wage['education'].cat.codes])
gam_full = gam_full.fit(Xgam, y)
```

两个s_gam()项导致平滑样条拟合，并使用λ (lam=0.6)的默认值，这有些武断。对于分类项education，使用f_gam()项指定，我们指定lam=0以避免任何收缩。我们在age中生成部分依赖图，以查看这些选择的效果。

该图的数值由 pygam 包生成。我们在 ISLP.pygam 中提供了一个用于偏依赖图的 plot_gam() 函数，该函数
使得这项工作比我们上一个使用自然样条的示例更容易。 plot_gam()

In [32]:
```
fig, ax = subplots(figsize=(8,8))
plot_gam(gam_full, 0, ax=ax)
ax.set_xlabel('Age')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of age on wage - default lam=0.6',
     fontsize=20);
```

我们看到该函数有些波动。为df指定值比指定lam的值更为自然。我们使用{**v16**}和{**v17**}各自四个自由度重新拟合了一个GAM。回想一下，下面加一考虑到了平滑样条的截距。

In [33]:
```
age_term = gam_full.terms[0]
age_term.lam = approx_lam(Xgam, age_term, df=4+1)
year_term = gam_full.terms[1]
year_term.lam = approx_lam(Xgam, year_term, df=4+1)
gam_full = gam_full.fit(Xgam, y)
```

请注意，更新上面的 age_term.lam 也会在 gam_full.terms[0] 中更新它！同样地，对于 year_term.lam 也是如此。

为 age 重复绘图，我们看到它要平滑得多。我们还为 year 生成图表。

In [34]:
```
fig, ax = subplots(figsize=(8,8))
plot_gam(gam_full,
         1,
         ax=ax)
ax.set_xlabel('Year')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of year on wage', fontsize=20)
```

最后我们绘制 education，这是一个分类变量。部分依赖图有所不同，更适合该变量每个水平的拟合常数集合。

In [35]:
```
fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_full, 2)
ax.set_xlabel('Education')
ax.set_ylabel('Effect on wage')
```

```
ax.set_title('Partial dependence of wage on education',
             fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories, fontsize=8);
```

加法模型的方差分析检验

在我们所有的模型中，year 的函数看起来相当线性。我们可以执行一系列ANOVA检验，以确定这三个模型中哪个最佳：排除 year ($\mathcal{M}_1$) 的GAM、使用 year ($\mathcal{M}_2$) 线性函数的GAM，或使用 year ($\mathcal{M}_3$) 样条函数的GAM。

In [36]:
```
gam_0 = LinearGAM(age_term + f_gam(2, lam=0))
gam_0.fit(Xgam, y)
gam_linear = LinearGAM(age_term +
                       l_gam(1, lam=0) +
                       f_gam(2, lam=0))
gam_linear.fit(Xgam, y)
```

Out[36]:
```
LinearGAM(callbacks=[Deviance(), Diffs()], fit_intercept=True,
   max_iter=100, scale=None, terms=s(0) + l(1) + f(2) + intercept,
   tol=0.0001, verbose=False)
```

请注意我们在上面的表达式中使用了age_term。我们这样做是因为之前我们将这一项中的lam的值设置为达到四个自由度。

为了直接评估year的效果，我们对上面拟合的三个模型进行了ANOVA分析。

In [37]:
```
anova_gam(gam_0, gam_linear, gam_full)
```

Out[37]:

	deviance	df	deviance_diff	df_diff	F	pvalue
0	3714362.366	2991.004	NaN	NaN	NaN	NaN
1	3696745.823	2990.005	17616.543	0.999	14.265	0.002
2	3693142.930	2987.007	3602.894	2.998	0.972	0.436

我们发现有力证据表明，包含year线性函数的GAM优于完全不含year的GAM（p值= 0.002）。然而，没有证据表明需要year的非线性函数（p值=0.435）。换言之，根据此方差分析的结果，$\mathcal{M}_2$更受青睐。

我们可以对 {v*} 重复相同的过程。我们看到有非常明确的证据表明 {v*} 需要非线性项。

In [38]:
```
gam_0 = LinearGAM(year_term +
                  f_gam(2, lam=0))
gam_linear = LinearGAM(l_gam(0, lam=0) +
                       year_term +
                       f_gam(2, lam=0))
gam_0.fit(Xgam, y)
gam_linear.fit(Xgam, y)
anova_gam(gam_0, gam_linear, gam_full)
```

```
Out[38]:        deviance        df deviance_diff  df_diff        F  pvalue
        0   3975443.045  2991.001            NaN      NaN      NaN     NaN
        1   3850246.908  2990.001      125196.137    1.000  101.270   0.000
        2   3693142.930  2987.007      157103.978    2.993   42.448   0.000
```

GAM拟合有一个（详细的）summary()方法。（我们在此不重现它。）

```
In [39]: gam_full.summary()
```

我们可以像从lm对象一样，从gam对象进行预测，使用类gam的predict()方法。在这里，我们对训练集进行预测。

```
In [40]: Yhat = gam_full.predict(Xgam)
```

为了拟合逻辑回归GAM，我们使用来自LogisticGAM的LogisticGAM()() pygam。

```
In [41]: gam_logit = LogisticGAM(age_term +
                                 l_gam(1, lam=0) +
                                 f_gam(2, lam=0))
         gam_logit.fit(Xgam, high_earn)
```

```
Out[41]: LogisticGAM(callbacks=[Deviance(), Diffs(), Accuracy()],
             fit_intercept=True, max_iter=100,
             terms=s(0) + l(1) + f(2) + intercept, tol=0.0001, verbose=False)
```

```
In [42]: fig, ax = subplots(figsize=(8, 8))
         ax = plot_gam(gam_logit, 2)
         ax.set_xlabel('Education')
         ax.set_ylabel('Effect on wage')
         ax.set_title('Partial dependence of wage on education',
                      fontsize=20);
         ax.set_xticklabels(Wage['education'].cat.categories, fontsize=8);
```

该模型看起来非常平缓，尤其是第一个类别的误差线非常高。让我们更仔细地查看一下数据。

```
In [43]: pd.crosstab(Wage['high_earn'], Wage['education'])
```

我们看到，在第一类教育类别中没有高收入者，这意味着模型将难以拟合。我们将拟合一个逻辑回归GAM，排除所有落入这一类别的观测值。这样能提供更合理的结果。

为此，我们可以对模型矩阵进行子集化，尽管这不会从 Xgam 中移除该列。虽然我们可以推断出对应于该特征的列，但为了可复现性，我们在这一较小的子集上重新构建模型矩阵。

```
In [44]: only_hs = Wage['education'] == '1. < HS Grad'
         Wage_ = Wage.loc[~only_hs]
         Xgam_ = np.column_stack([Wage_['age'],
                                  Wage_['year'],
                                  Wage_['education'].cat.codes-1])
         high_earn_ = Wage_['high_earn']
```

在上面的倒数第二行中，我们从类别的代码中减去1，这是由于pygam中的一个bug。它只是重新标记了教育值，因此对拟合没有影响。

我们现在拟合这个模型。

In [45]:
```
gam_logit_ = LogisticGAM(age_term +
                         year_term +
                         f_gam(2, lam=0))
gam_logit_.fit(Xgam_, high_earn_)
```

Out[45]:
```
LogisticGAM(callbacks=[Deviance(), Diffs(), Accuracy()],
  fit_intercept=True, max_iter=100,
  terms=s(0) + s(1) + f(2) + intercept, tol=0.0001, verbose=False)
```

让我们看看在移除那些观测值后，education、year和age对高收入者状态的影响。

In [46]:
```
fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 2)
ax.set_xlabel('Education')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of high earner status on education
    ', fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories[1:],
                   fontsize=8);
```

In [47]:
```
fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 1)
ax.set_xlabel('Year')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of high earner status on year',
             fontsize=20);
```

In [48]:
```
fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 0)
ax.set_xlabel('Age')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of high earner status on age',
    fontsize=20);
```

7.8.4 Local Regression

我们使用 `lowess()` 中的 `lowess()` 函数来说明局部回归的使用。sm.nonparametric一些GAM的实现允许使用局部回归算子作为项；在pygam中并非如此。

这里我们使用跨度0.2和0.5来拟合局部线性回归模型；也就是说，每个邻域包含20%或50%的观测值。如预期的那样，使用0.5的跨度比0.2更平滑。

In [49]:
```
lowess = sm.nonparametric.lowess
fig, ax = subplots(figsize=(8,8))
ax.scatter(age, y, facecolor='gray', alpha=0.5)
for span in [0.2, 0.5]:
    fitted = lowess(y,
```

```
                    age,
                    frac=span,
                    xvals=age_grid)
    ax.plot(age_grid,
            fitted,
            label='{:.1f}'.format(span),
            linewidth=4)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
ax.legend(title='span', fontsize=15);
```

7.9 练习

Conceptual



1. 本章提到，通过形式为 x、x^2、x^3、$(x-\xi)^3_+$ 的基，可以得到一个在 ξ 处有一个结点的三次回归样条，其中如果 $x > \xi$，则 $(x-\xi)^3_+ = (x-\xi)^3$，否则等于 0。我们现在将展示形式为

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x-\xi)^3_+$$

确实是三次回归样条，无论$\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$的值如何。

(a) 求一个三次多项式

$$f_1(x) = a_1 + b_1 x + c_1 x^2 + d_1 x^3$$

使得对于所有 $x \leq \xi$，都有 $f(x) = f_1(x)$。用 $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$ 表示 a_1, b_1, c_1, d_1。

(b) 求一个三次多项式

$$f_2(x) = a_2 + b_2 x + c_2 x^2 + d_2 x^3$$

使得对于所有 $x > \xi$，都有 $f(x) = f_2(x)$。将 a_2, b_2, c_2, d_2 用 $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$ 表示。我们现在已经确定 $f(x)$ 是分段多项式。

(c) 证明$f_1(\xi) = f_2(\xi)$。即，$f(x)$在ξ处连续。(d) 证明 $f_1'(\xi) = f_2'(\xi)$。即，$f'(x)$在ξ处连续。(e) 证明$f_1''(\xi) = f_2''(\xi)$。即，$f''(x)$在ξ处连续。

因此，$f(x)$ 确实是一个三次样条。

Hint: Parts (d) and (e) of this problem require knowledge of single-variable calculus. As a reminder, given a cubic polynomial

$$f_1(x) = a_1 + b_1 x + c_1 x^2 + d_1 x^3,$$

the first derivative takes the form

$$f_1'(x) = b_1 + 2c_1 x + 3d_1 x^2$$

and the second derivative takes the form

$$f_1''(x) = 2c_1 + 6d_1x.$$

2. 假设使用以下公式计算曲线 $\hat{g}$，以平滑地拟合一组 n 点:

$$\hat{g} = \arg\min_g \left(\sum_{i=1}^{n} (y_i - g(x_i))^2 + \lambda \int \left[g^{(m)}(x) \right]^2 dx \right),$$

其中 $g^{(m)}$ 表示 g (和 $g^{(0)} = g$) 的第 m 阶导数。在以下每种情形中，给出 $\hat{g}$ 的示例草图。

(a) $\lambda = \infty, m = 0$ 。 (b) $\lambda = \infty, m = 1$。 (c) $\lambda = \infty, m = 2$。 (d) $\lambda = \infty, m = 3$。 (e) $\lambda = 0, m = 3$。

3. 假设我们用基函数 $b_1(X) = X, b_2(X) = (X-1)^2 I(X \geq 1)$ 来拟合一条曲线。（注意，对于 $X \geq 1$，$I(X \geq 1)$ 等于1，否则为0。）我们拟合线性回归模型

$$Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X) + \epsilon,$$

并得到系数估计 $\hat{\beta}_0 = 1, \hat{\beta}_1 = 1, \hat{\beta}_2 = -2$。在 $X = -2$ 和 $X = 2$ 之间绘制估计曲线。注意截距、斜率及其他相关信息。

4. 假设我们用基函数 $b_1(X) = I(0 \leq X \leq 2) - (X-1)I(1 \leq X \leq 2)$, $b_2(X) = (X-3)I(3 \leq X \leq 4) + I(4 < X \leq 5)$ 拟合一条曲线。我们拟合线性回归模型

$$Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X) + \epsilon,$$

并得到系数估计值 $\hat{\beta}_0 = 1, \hat{\beta}_1 = 1, \hat{\beta}_2 = 3$。绘制 $X = -2$ 到 $X = 6$ 之间的估计曲线。注意截距、斜率和其他相关信息。

5. 考虑由以下定义的 $\hat{g}_1$ 和 $\hat{g}_2$ 两条曲线,

$$\hat{g}_1 = \arg\min_g \left(\sum_{i=1}^{n} (y_i - g(x_i))^2 + \lambda \int \left[g^{(3)}(x) \right]^2 dx \right),$$

$$\hat{g}_2 = \arg\min_g \left(\sum_{i=1}^{n} (y_i - g(x_i))^2 + \lambda \int \left[g^{(4)}(x) \right]^2 dx \right),$$

其中$g^{(m)}$表示g的第m阶导数。

(a) 当 $\lambda \to \infty$ 时，$\hat{g}_1$ 或 $\hat{g}_2$ 会有较小的训练 RSS 吗? (b) 当 $\lambda \to \infty$ 时，$\hat{g}_1$ 或 $\hat{g}_2$ 会有较小的测试 RSS 吗? (c) 对于 $\lambda = 0$，$\hat{g}_1$ 或 $\hat{g}_2$ 会有较小的训练和测试 RSS 吗?

Applied

6. 在本练习中，您将进一步分析贯穿本章所考虑的 Wage 数据集。

(a) 执行多项式回归，使用age预测wage。采用交叉验证选择多项式的最优度数d。被选中的度数是多少，这与使用ANOVA进行假设检验的结果相比如何？绘制出最终多项式拟合数据的图表。

(b) 使用age拟合一个阶梯函数来预测wage，并进行交叉验证以选择最佳切割数。绘制所得到拟合的图形。

7. Wage 数据集包含本章未探讨的许多其他特征，例如婚姻状况（maritl）、职业类别（jobclass）等。探索这些其他预测变量与 wage 之间的关系，并使用非线性拟合技术来拟合灵活的模型。创建所得结果的图表，并撰写你的发现总结。

8. 将本章中研究的一些非线性模型拟合到Auto数据集。该数据集中是否存在非线性关系的证据？创建一些信息丰富的图表来证明你的答案。

9. 本题使用变量 dis (到五个波士顿就业中心的加权平均距离) 和 nox (每千万分之一氧化氮浓度)，数据来自 Boston。我们将 dis 视为预测变量，nox 视为响应变量。

(a) 使用 ISLP.models 模块中的 poly() 函数拟合一个三次多项式回归，以 dis 预测 nox。报告回归输出，并绘制结果数据及多项式拟合曲线。(b) 绘制一系列不同多项式次数（例如，从1到10）的多项式拟合曲线，并报告相应的残差平方和。(c) 执行交叉验证或其他方法以选择多项式的最优次数，并解释你的结果。(d) 使用 ISLP.models 模块中的 bs() 函数拟合一个回归样条，以 dis 预测 nox。报告使用四个自由度拟合的输出。你是如何选择节点的？绘制结果拟合曲线。(e) 现在对一系列自由度拟合回归样条，并绘制结果拟合曲线及报告相应的RSS。描述所获得的结果。(f) 在此数据上，执行交叉验证或其他方法以选择回归样条的最佳自由度。描述你的结果。

10. 此问题涉及`College`数据集。

(a) 将数据分为训练集和测试集。以州外学费作为响应变量，其他变量作为预测变量，在训练集上进行前向逐步选择，以识别出一个仅使用部分预测变量的满意模型。(b) 在训练数据上拟合一个GAM，以州外学费作为响应变量，并使用上一步中选择的特征作为预测变量。绘制结果，并解释你的发现。(c) 在测试集上评估所得到的模型，并解释所得到的结果。(d) 对于哪些变量（如果有的话），有证据表明与响应变量存在非线性关系?

11. 在第7.7节中提到，广义可加模型通常采用*backfitting*方法进行拟合。回退拟合背后的理念其实相当简单。接下来，我们将在线性回归多元化的背景下探讨回退拟合。

假设我们希望执行多元线性回归，但没有相应的软件来实施。相反，我们仅拥有执行简单线性回归的软件。因此，我们采取以下迭代方法：反复固定除一个系数估计外的所有其他系数估计于当前值，仅使用简单线性回归更新那个系数估计。这一过程持续到*convergence*——即直到系数估计不再变化为止。

我们现在用一个玩具示例来尝试一下。

(a) 生成一个响应变量 Y和两个预测变量 X_1 与 X_2，其中 $n =$ 为 100。(b) 编写一个函数 `simple_reg()`，它接受两个参数 `outcome` 和 `feature`，用该结果变量和特征变量拟合一个简单线性回归模型，并返回估计的截距和斜率。(c) 将 `beta1` 初始化为你选择的某个值。你选择什么值无关紧要。(d) 保持 `beta1` 固定，使用你的函数 `simple_reg()` 来拟合模型:

$$Y - \texttt{beta1} \cdot X_1 = \beta_0 + \beta_2 X_2 + \epsilon.$$

将结果值存储为 `beta0` 和 `beta2`。

(e) 保持 `beta2` 不变，拟合模型

$$Y - \texttt{beta2} \cdot X_2 = \beta_0 + \beta_1 X_1 + \epsilon.$$

将结果存储为`beta0`和`beta1` (，覆盖它们先前的值)。

(f) 编写一个 for 循环，重复 (c) 和 (d) 1000 次。报告在 for 循环的每次迭代中 `beta0`、`beta1` 和 `beta2` 的估计值。创建一个图，其中显示这些值中的每一个，以及 `beta0`、`beta1` 和 `beta2`。

(g) 将你在(e)中的答案与直接使用X_1和X_2预测Y的多元线性回归结果进行比较。使用`axline()`方法将这些多元线性回归系数估计叠加到(e)中获得的图上。(h) 在这个数据集上，为了获得对多元回归系数估计的“良好”近似，需要进行多少次反向拟合迭代?

12. 这个问题是上一个练习的延续。在一个$p =$为100的玩具示例中，展示通过反复执行简单线性回归的后向拟合过程，可以近似多元线性回归系数估计。为了获得对多元回归系数估计的“良好”近似，需要进行多少次后向拟合迭代？创建一个图表来证明你的答案。

8
基于树的方法



在本章中，我们介绍用于回归和分类的*tree-based*方法。这些方法涉及将预测空间划分或分割成若干简单区域。为了对给定的观测进行预测，我们通常使用该观测所属区域内训练观测的响应值的均值或众数。由于用于划分预测空间的分割规则集可以用树来概括，这类方法被称为*decision tree*方法。

decision tree

基于树的方法简单且有助于解释。然而，在预测准确性方面，它们通常无法与最佳的有监督学习方法相竞争，如第6章和第7章中所见。因此，在本章中我们还将介绍*bagging*、*random forests*、*boosting*和*Bayesian additive regression trees*。这些方法中的每一种都涉及生成多棵树，然后将它们结合起来以得出一个共识预测。我们将看到，结合大量的树往往能在预测准确性上带来显著提升，代价是在解释性上有所损失。

8.1 决策树基础

决策树可以应用于回归和分类问题。我们首先考虑回归问题，然后再转向分类。

8.1.1 Regression Trees

为了激发 *regression trees*，我们从一个简单的例子开始。

regression tree

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_8

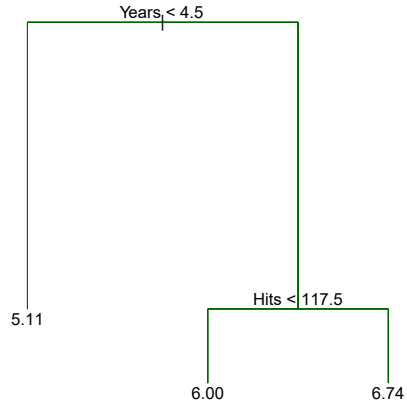


图 8.1. *For the* `Hitters` *data, a regression tree for predicting the log salary of a baseball player, based on the number of years that he has played in the major leagues and the number of hits that he made in the previous year. At a given internal node, the label (of the form* $X_j < t_k$*) indicates the left-hand branch emanating from that split, and the right-hand branch corresponds to* $X_j \geq t_k$*. For instance, the split at the top of the tree results in two large branches. The left-hand branch corresponds to* `Years<4.5`*, and the right-hand branch corresponds to* `Years>=4.5`*. The tree has two internal nodes and three terminal nodes, or leaves. The number in each leaf is the mean of the response for the observations that fall there.*

P 使用回归树预测棒球运动员的薪水 ees

我们使用 `Hitters` 数据集，基于 `Years` (该球员在大联盟中的打球年数) 和 `Hits` (他前一年所击出的安打数)，来预测一名棒球运动员的 `Salary`。我们首先剔除 `Salary` 值缺失的观测，并对 `Salary` 进行对数变换，使其分布更接近典型的钟形。（注意 `Salary` 以千美元为单位。）

图8.1展示了一棵拟合这些数据的回归树。它由一系列分割规则组成，从树的顶部开始。顶部的分割将具有`Years<4.5`的观测值分配到左侧分支。[1] 这些球员的预测薪资由数据集中具有`Years<4.5`的球员的平均响应值给出。对于这样的球员，平均对数薪资为5.107，因此我们对这些球员的预测为 $e^{5.107}$千美元，即165,174美元。具有`Years>=4.5`的球员被分配到右侧分支，然后该组由`Hits`进一步细分。总体而言，该树将球员分层或分割成预测空间的三个区域：打球四年或更少的球员，打球五年或更多且去年少于118次安打的球员，以及打球五年或更多且去年至少118次安打的球员。这三个区域可以写作$R_1 = \{\mathrm{X} \mid$ `Years<4.5`$\}$，$R_2 = \{\mathrm{X} \mid$ `Years>=4.5`，`Hits<117.5`$\}$，和 $R_3 = \{\mathrm{X} \mid$ `Years>=4.5`，`Hits>=117.5`$\}$。图8.2展示了

[1] Both `Years` and `Hits` are integers in these data; the function used to fit this tree labels the splits at the midpoint between two adjacent values.

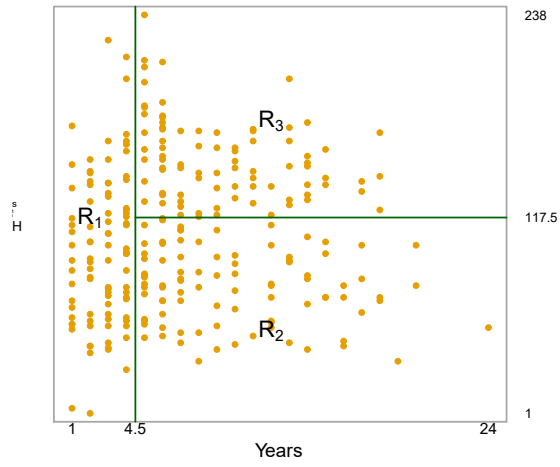


图8.2. *The three-region partition for the* `Hitters` *data set from the regression tree illustrated in Figure 8.1.*

作为`Years`和`Hits`的函数的区域。这三组的预测薪资分别为\$1,000$\times e^{5.107}$ = \$165,174、\$1,000$\times e^{5.999}$ =\$402,834和\$1,000$\times e^{6.740}$ =\$845,346。

为了与 *tree* 的类比保持一致，区域 R_1、R_2 和 R_3 被称为树的 *terminal nodes* 或 *leaves*。正如 Figure 8.1 的情况一样，决策树通常是*upside down*绘制的，即叶子位于树的底部。树中划分预测空间的点被称为*internal nodes*。在图8.1中，两个内部节点由文本`Years<4.5`和`Hits<117.5`表示。我们将连接节点的树段称为 *branches*。

终端节点叶子

内部节点

分支

我们可以将图8.1中展示的回归树解释如下：`Years`是决定`Salary`的最重要因素，经验较少的球员比经验丰富的球员挣得更低的薪水。鉴于球员经验较少，他前一年击球次数似乎对他的薪水影响不大。但在大联盟打了五年或以上的球员中，前一年击球次数确实影响薪水，前一年击球次数较多的球员往往薪水更高。图8.1中显示的回归树可能是对`Hits`、`Years`和`Salary`之间真实关系的一种过度简化。然而，它相较于其他类型的回归模型（如第3章和第6章中所见）具有优势：更易于解释，并且有良好的图形表示。

通过特征空间分层进行预测

现在我们讨论构建回归树的过程。大致来说，有两个步骤。

1. 我们将预测变量空间——即$X_1, X_2, \ldots, X_p$可能取值的集合——划分为J个互不重叠的不同区域，$R_1, R_2, \ldots, R_J$。

2. 对于落入区域 R_j 的每一个观测，我们做出相同的预测，这个预测就是 R_j 中训练观测响应值的均值。

例如，假设在第1步中我们得到两个区域，R_1和R_2，并且第一个区域中训练观测的响应均值为10，而第二个区域中训练观测的响应均值为20。那么对于给定的观测$X = x$，如果$x \in R_1$，我们将预测值为10，如果$x \in R_2$，我们将预测值为20。

我们现在详细阐述上述步骤1。我们如何构建区域 $R_1, \ldots, R_J$? 理论上，这些区域可以具有任何形状。然而，为了简单起见以及便于解释所得的预测模型，我们选择将预测变量空间划分为高维矩形，或 *boxes*。目标是找到使RSS最小化的箱体 $R_1, \ldots, R_J$，其定义为

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2, \tag{8.1}$$

其中 $\hat{y}_{R_j}$ 是第 j 个箱内训练观测的平均响应。遗憾的是，考虑将特征空间划分为 J 个箱的所有可能划分在计算上是不可行的。因此，我们采用一种称为 *recursive binary splitting* 的 *top-down*、*greedy* 方法。该

recursive binary splitting

该方法是*top-down*，因为它从树的顶部开始（此时所有观测都属于一个单一区域），然后逐次分割预测变量空间；每次分割都通过树下方两个新的分支来表示。它是*greedy*，因为在树构建过程的每一步，都是在那个特定步骤进行*best*分割，而不是前瞻性地选择一个会在未来某个步骤产生更好树的分割。

为了执行递归二叉分裂，我们首先选择预测变量 X_j 和切分点 s，使得将预测变量空间划分为区域 $\{X|X_j < s\}$ 和 $\{X|X_j \geq s\}$ 能够带来 RSS 的最大可能减少。（记号 $\{X|X_j < s\}$ 表示 *the region of predictor space in which* X_j *takes on a value less than* s。）也就是说，我们考虑所有预测变量 $X_1, \ldots, X_p$，以及每个预测变量的切分点 s 的所有可能取值，然后选择使得所得树具有最低 RSS 的预测变量和切分点。更详细地说，对于任意 j 和 s，我们定义这对半平面

$$R_1(j, s) = \{X|X_j < s\} \text{ and } R_2(j, s) = \{X|X_j \geq s\}, \tag{8.2}$$

我们寻求使该方程最小化的 j 和 s 的值。

$$\sum_{i:\, x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:\, x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2, \tag{8.3}$$

其中 $\hat{y}_{R_1}$ 是 $R_1(j, s)$ 中训练观测值的平均响应，而 $\hat{y}_{R_2}$ 是 $R_2(j, s)$ 中训练观测值的平均响应。找到使 (8.3) 最小化的 j 和 s 的值可以相当快地完成，特别是当特征数量 p 不太大时。

接下来，我们重复该过程，寻找最佳预测变量和最佳切割点，以进一步划分数据，从而最小化内部的残差平方和

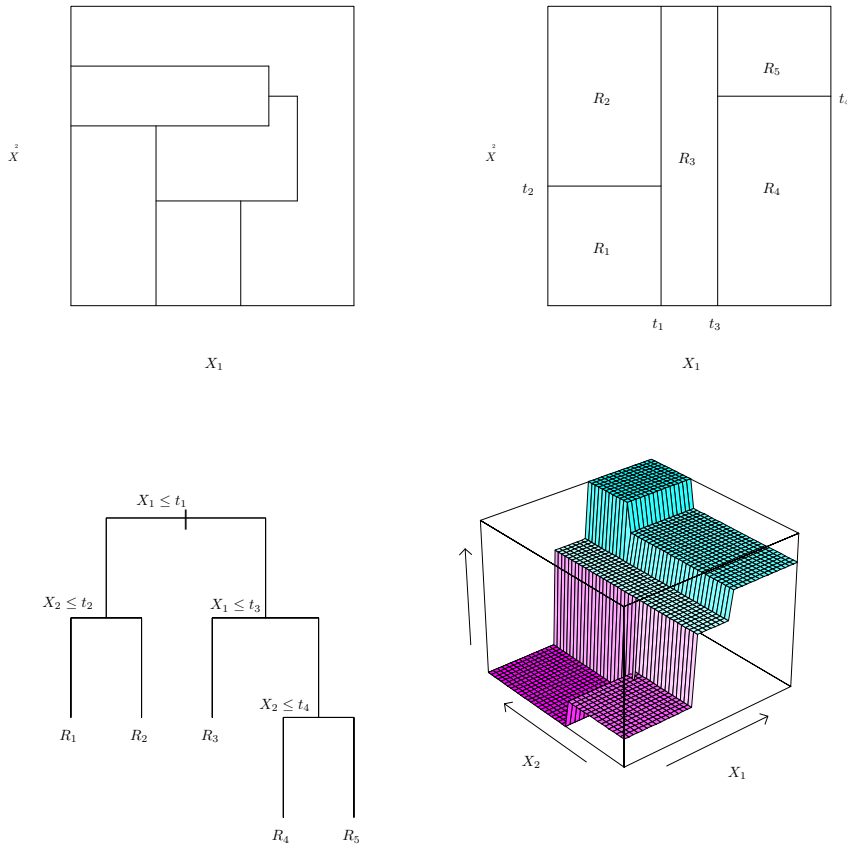


图 8.3. 左上：*A partition of two-dimensional feature space that could not result from recursive binary splitting.* 右上：*The output of recursive binary splitting on a two-dimensional example.* 左下：*A tree corresponding to the partition in the top right panel.* 右下：*A perspective plot of the prediction surface corresponding to that tree.*

对每个得到的区域。然而，这次我们不是分割整个预测器空间，而是分割先前确定的两个区域之一。我们现在有三个区域。再次，我们考虑进一步分割这三个区域中的一个，以最小化RSS。这个过程持续到达到停止准则；例如，我们可以持续到没有区域包含超过五个观测值。

一旦区域 $R_1, \ldots, R_J$ 被创建，我们使用该测试观测所属区域内训练观测的均值来预测给定测试观测的响应。

图8.3展示了这种方法的五区域示例。

树剪枝

上述过程可能在训练集上产生良好的预测，但很可能过拟合数据，导致测试集性能不佳。这是因为生成的树可能过于复杂。一棵较小的树

如果分裂次数较少（即区域 $R_1, \ldots, R_J$ 数量较少），可能会降低方差并提升解释性，但会以少许偏差为代价。针对上述过程的一种可能替代方案是，只要每次分裂导致的 RSS 减少量超过某个（较高的）阈值，就继续构建树。这种策略会生成较小的树，但过于短视，因为树早期一个看似无用的分裂之后，可能紧接着一个非常好的分裂——也就是说，这个分裂会在后续导致 RSS 的大幅减少。

因此，一个更好的策略是生长一棵非常大的树 T_0，然后通过 *prune* 来获得一个 *subtree*。我们如何确定最佳修剪树的方法？直观上，我们的目标是选择一个能带来最低测试错误率的子树。给定一个子树，我们可以通过交叉验证或验证集方法来估计其测试错误。然而，为每一个可能的子树估计交叉验证误差将过于繁琐，因为可能的子树数量极其庞大。相反，我们需要一种方法来选取一小部分子树进行考虑。

剪枝子树

Cost complexity pruning—也被称为*weakest link pruning*—给了我们一种实现此目的的方法。我们不考察所有可能的子树，而是考虑由非负调优参数 α 索引的一系列树。对于 α 的每个值，都存在对应的子树 $T \subset T_0$，使得

成本复杂度剪枝 最弱环节剪枝

$$\sum_{m=1}^{|T|} \sum_{i:\, x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T| \tag{8.4}$$

尽可能小。这里$|T|$表示树T的终端节点数，R_m是对应于第m个终端节点的矩形（即预测空间的子集），而$\hat{y}_{R_m}$是与R_m相关的预测响应——即R_m中训练观测值的均值。调优参数α控制子树复杂度与其对训练数据的拟合之间的权衡。当α =为0时，子树T将简单地等于T_0，因为此时(8.4)仅衡量训练误差。然而，随着α的增加，拥有许多终端节点的树需要付出代价，因此量(8.4)倾向于在较小的子树上达到最小化。公式8.4让人联想到第6章的套索(6.7)，其中使用了类似的表述来控制线性模型的复杂度。

事实证明，当我们在(8.4)中从零开始增加α时，树枝会以嵌套且可预测的方式从树中被剪除，因此获取作为α函数的整个子树序列是很容易的。我们可以使用验证集或交叉验证来选择α的值。然后我们回到完整数据集并获取对应于α的子树。这一过程在算法8.1中进行了总结。

图8.4和图8.5展示了在`Hitters`数据上使用九个特征拟合和修剪回归树的结果。首先，我们将数据集随机分为两半，训练集中有132个观测值，测试集中有131个观测值。然后，我们在训练数据上构建了一棵大型回归树，并在(8.4)中变化α，以创建具有不同终端节点数量的子树。最后，我们进行了六折交叉验证，以估计树的交叉验证MSE，如下所示

算法8.1 *Building a Regression Tree*

1. Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum number of observations.
2. Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of α.
3. Use K-fold cross-validation to choose α. That is, divide the training observations into K folds. For each $k = 1, \ldots, K$:
   (a) Repeat Steps 1 and 2 on all but the kth fold of the training data.
   (b) Evaluate the mean squared prediction error on the data in the left-out kth fold, as a function of α.
   Average the results for each value of α, and pick α to minimize the average error.
4. Return the subtree from Step 2 that corresponds to the chosen value of α.

α 的函数。（我们选择进行六折交叉验证，因为132是六的精确倍数。）未修剪的回归树如图8.4所示。图8.5中的绿色曲线显示了作为叶节点数量函数的交叉验证误差，[2]，而橙色曲线表示测试误差。同时，还展示了估计误差周围的标准误差条。作为参考，训练误差曲线以黑色显示。交叉验证误差是测试误差的一个合理近似：交叉验证误差在三节点树时达到最小值，而测试误差也在三节点树时下降（尽管它在十节点树时达到最低值）。包含三个终端节点的修剪树如图8.1所示。

8.1.2 Classification Trees

一个 *classification tree* 与回归树非常相似，不同之处在于它是用于预测定性响应而非定量响应。回顾一下，对于回归树，一个观测的预测响应由属于同一终端节点的训练观测的均值响应给出。相比之下，对于分类树，我们预测每个观测属于其所属区域中训练观测的 *most commonly occurring class*。在解释分类树的结果时，我们通常不仅对特定终端节点区域对应的类别预测感兴趣，而且对该区域内的训练观测中的 *class proportions* 也感兴趣。

分类树

构建分类树的任务与构建回归树的任务非常相似。正如在回归设置中一样，我们使用递归的

[2]Although CV error is computed as a function of α, it is convenient to display the result as a function of $|T|$, the number of leaves; this is based on the relationship between α and $|T|$ in the original tree grown to all the training data.

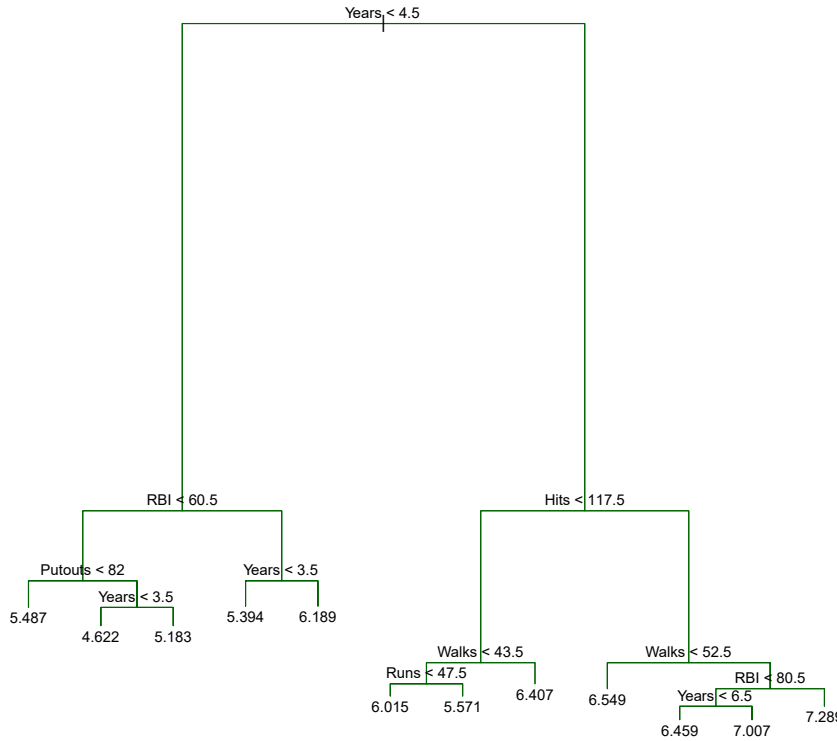


图 8.4. *Regression tree analysis for the* `Hitters` *data. The unpruned tree that results from top-down greedy splitting on the training data is shown.*

二分类分割以生长分类树。然而，在分类设置中，RSS不能用作进行二分类分割的标准。RSS的一个自然替代是 *classification error rate*。由于我们计划为了将给定区域中的观测分配给该区域中训练观测的 *most commonly occurring class*，分类错误率就是该区域中不属于最常见类别的训练观测的比例：

分类错误率

$$E = 1 - \max_k(\hat{p}_{mk}). \tag{8.5}$$

这里，$\hat{p}_{mk}$ 表示第 m 个区域中来自第 k 类的训练观测值的比例。然而，事实证明，分类误差对于树的生长不够敏感，在实践中，另外两种度量更为可取。

Gini index 定义为

基尼指数

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}), \tag{8.6}$$

一种衡量 K 类总方差的度量。不难看出，如果所有 $\hat{p}_{mk}$ 都接近于零或一，基尼指数就会取较小的值。因此，基尼指数被称为一种衡量

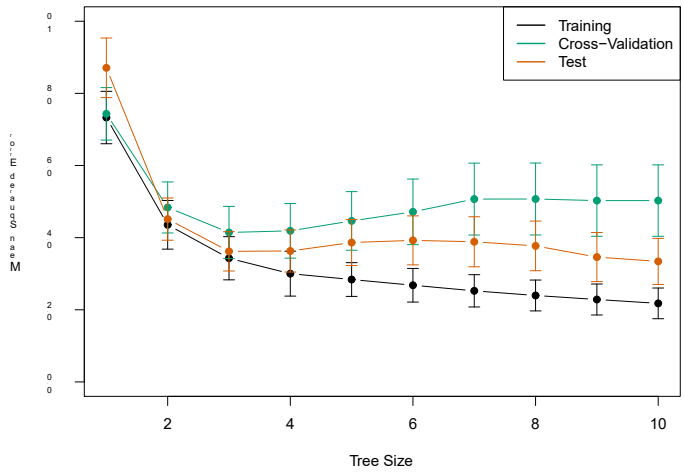


图 8.5. *Regression tree analysis for the* `Hitters` *data. The training, cross-validation, and test MSE are shown as a function of the number of terminal nodes in the pruned tree. Standard error bands are displayed. The minimum cross-validation error occurs at a tree size of three.*

节点 *purity*——较小的值表明节点主要包含来自单一类别的观测值。

基尼指数的一个替代方案是*entropy*，由下式给出

entropy

$$D = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}. \tag{8.7}$$

由于 $0 \leq \hat{p}_{mk} \leq 1$，因此 $0 \leq -\hat{p}_{mk} \log \hat{p}_{mk}$。可以证明，如果所有 $\hat{p}_{mk}$ 都接近零或接近一，熵将取接近零的值。因此，与基尼指数一样，如果第 m 个节点是纯的，熵将取一个很小的值。事实上，结果表明基尼指数和熵在数值上非常相似。

在构建分类树时，通常使用基尼指数或熵来评估特定分裂的质量，因为这两种方法比分类错误率对节点纯度更为敏感。在修剪树时*pruning*，这三种方法都可能被使用，但如果目标是最终修剪树的预测准确性，分类错误率则更为可取。

图8.6展示了`Heart`数据集的一个示例。这些数据包含了303名因胸痛就诊患者的二元结局`HD`。结局值`Yes`表示通过血管造影测试存在心脏病，而`No`则表示无心脏病。共有13个预测因子，包括`Age`、`Sex`、`Chol` (胆固醇测量值)，以及其他心肺功能测量指标。交叉验证结果生成了一棵含有六个终端节点的树。

到目前为止的讨论中，我们假设预测变量取连续值。然而，即使存在定性预测变量，也可以构建决策树。例如，在`Heart`数据中，一些预测变量，如`Sex`、`Thal` (铊应激测试)，

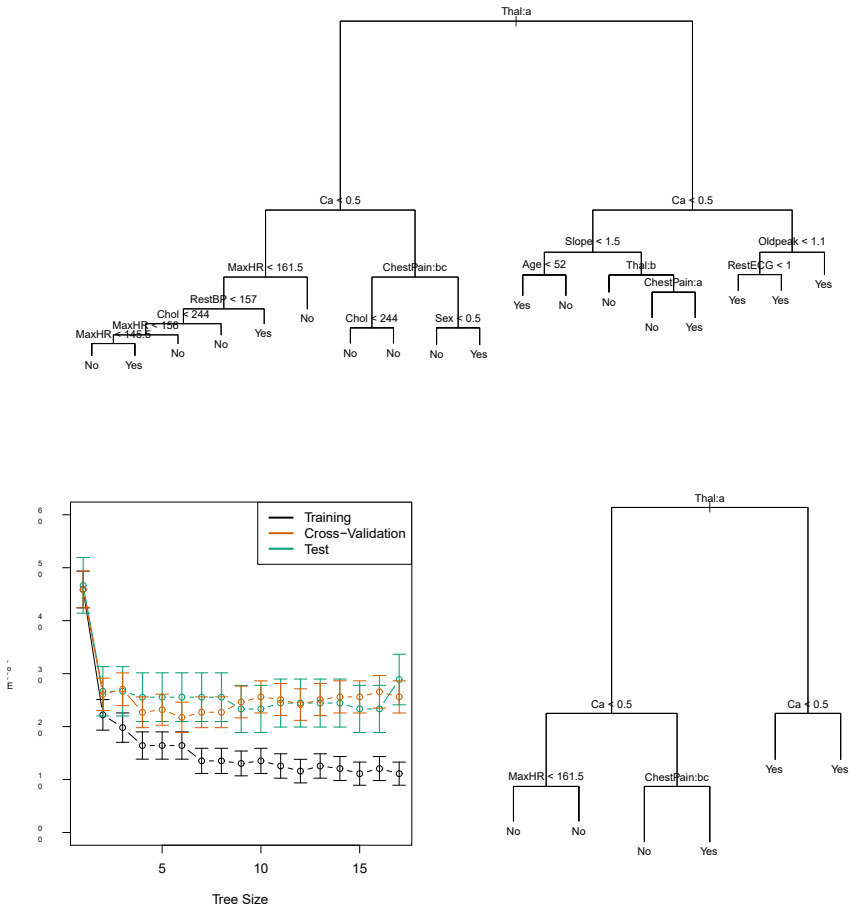


图8.6。 `Heart` *data.* 上： *The unpruned tree.* 左下： *Cross-validation error, training, and test error, for different sizes of the pruned tree.* 右下： *The pruned tree corresponding to the minimal cross-validation error.*

{v*}和ChestPain是定性的。因此，对这些变量之一进行分割相当于将一些定性值分配给一个分支，将剩余的值分配给另一个分支。在图8.6中，一些内部节点对应于分割定性变量。例如，顶部内部节点对应于分割Thal。文本Thal:a表明从该节点出来的左侧分支由具有Thal变量第一个值（正常）的观察组成，右侧节点由剩余的观察组成（固定或可逆缺陷）。左侧树向下的两次分割处的文本ChestPain:bc表明从该节点出来的左侧分支由具有ChestPain变量第二个和第三个值的观察组成，其中可能的值是典型心绞痛、非典型心绞痛、非心绞痛性疼痛和无症状。

图8.6有一个令人惊讶的特点：某些分割会产生两个终端节点，这些节点具有*same predicted value*。例如，考虑未修剪树右下方靠近的{split} RestECG<1。无论RestECG的值是多少，对于那些观测，预测的响应值为Yes。

的观测。那么，为何还要进行划分呢？划分之所以进行，是因为它能提升 *node purity*。也就是说，对应右侧叶节点的所有9个观测，其响应值为Yes，而对应左侧叶节点的观测中有7/11的响应值为Yes。节点纯度为何重要？假设我们有一个测试观测属于右侧叶节点所给定的区域，那么我们可以相当确信其响应值为Yes。相反，如果测试观测属于左侧叶节点所给定的区域，其响应值可能是Yes，但我们对此远没有那么确定。即使划分RestECG<1没有减少分类错误，它仍能改善基尼指数和熵，这两个指标对节点纯度更为敏感。

8.1.3 Trees Versus Linear Models

回归树和分类树与第3章和第4章中介绍的更经典的回归和分类方法有着非常不同的风格。特别地，线性回归假设模型形式为

$$f(X) = \beta_0 + \sum_{j=1}^{p} X_j \beta_j, \tag{8.8}$$

而回归树假设模型形式为

$$f(X) = \sum_{m=1}^{M} c_m \cdot 1_{(X \in R_m)} \tag{8.9}$$

其中 $R_1, \ldots, R_M$ 表示特征空间的一个划分，如图8.3所示。

哪个模型更好？这取决于当前的问题。如果特征与响应之间的关系如(8.8)所示，能被线性模型很好地近似，那么诸如线性回归这样的方法很可能会表现良好，并且会优于不利用这种线性结构的回归树等方法。反之，如果特征与响应之间存在如模型(8.9)所示的高度非线性和复杂关系，那么决策树可能优于经典方法。图8.7展示了一个说明性的例子。树基方法和经典方法的相对性能可以通过估计测试误差来评估，使用交叉验证或验证集方法（第5章）。

当然，在选择统计学习方法时，除了简单的测试误差外，还可能考虑其他因素；例如，在某些情况下，为了可解释性和可视化，可能更倾向于使用树进行预测。

8.1.4 Advantages and Disadvantages of Trees

用于回归和分类的决策树相比第3章和第4章中看到的更经典方法有许多优势：

- 树很容易向人们解释。事实上，它们比线性回归更容易解释！

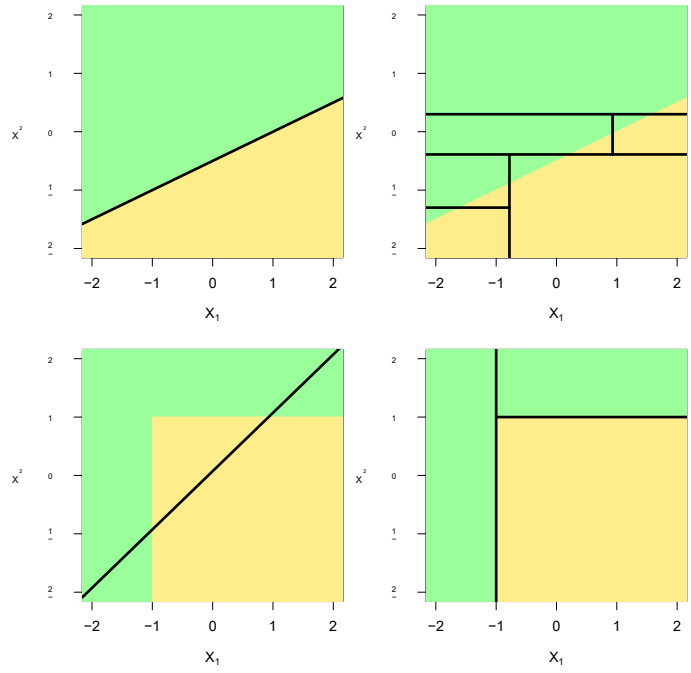


图8.7. 顶行：*A two-dimensional classification example in which the true decision boundary is linear, and is indicated by the shaded regions. A classical approach that assumes a linear boundary (left) will outperform a decision tree that performs splits parallel to the axes (right).* 底行：*Here the true decision boundary is non-linear. Here a linear model is unable to capture the true decision boundary (left), whereas a decision tree is successful (right).*

! 有些人认为，与前面章节中看到的回归和分类方法相比，决策树更接近人类的决策过程。

! 树可以以图形方式显示，并且即使是非专家也很容易理解（特别是如果它们很小）。！

树可以轻松处理定性预测变量，而无需创建虚拟变量。

遗憾的是，决策树通常不具备本书中其他一些回归和分类方法所达到的预测准确性水平。# 此外，决策树非常不稳定。换言之，数据中的微小变化可能导致最终估计树发生巨大变化。

然而，通过使用诸如 *bagging random forests* 和 *boosting* 等方法汇总众多决策树，树木的预测性能可以得到显著提升。我们将在下一节介绍这些概念。

8.2 Bagging、随机森林、提升法和贝叶斯加性回归树

ensemble 方法是一种结合了许多简单“构建集成
为了获得一个单一且可能非常强大的模型，人们对“块”模型进行了平均
。这些简单的构建块模型有时被称为*weaklearners*，因为它们单独可能会导致平庸的预测。

徽章

弱学

习器 我们现在讨论装袋法、随机森林、提升法和贝叶斯加法回归树。这些是集成方法，其简单的构建模块是回归树或分类树。

8.2.1 Bagging

第 5 章中介绍的自助法是一个非常强大的思想。它被用于许多难以甚至不可能直接计算感兴趣量的标准差的情况。我们在这里看到，自助法可以用于完全不同的情境，以改进诸如决策树之类的统计学习方法。

第8.1节中讨论的决策树存在*high variance*的问题。这意味着，如果我们随机将训练数据分为两部分，并对两部分分别拟合决策树，得到的结果可能会有很大差异。相比之下，具有*low variance*的方法在重复应用于不同数据集时将产生相似的结果；如果n与p的比例适中，线性回归往往具有较低的方差。*Bootstrap*、*aggregation*或*bagging*，是一种减少

bagging

一种统计学习方法的方差；我们在此介绍它，是因为它在决策树的背景下特别有用且经常被使用。

回忆一下，给定一组n个独立观测值$Z_1, \ldots, Z_n$，每个观测值的方差为σ^2，观测值均值$\bar{Z}$的方差由σ^2/n给出。换句话说，
averaging a set of observations reduces variance。因此，减少方差并提高统计学习方法测试集准确性的一种自然方法是从总体中抽取多个训练集，使用每个训练集构建单独的预测模型，并对得到的预测结果取平均。换句话说，我们可以使用B个单独的训练集来计算$\hat{f}^1(x), \hat{f}^2(x), \ldots, \hat{f}^B(x)$，并对它们取平均，以获得一个单一的低方差统计学习模型，其表达式为

$$\hat{f}_{\text{avg}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^b(x).$$

当然，这是不切实际的，因为我们通常无法获得多个训练集。相反，我们可以通过从（单一）训练数据集中重复采样来进行自助法。在这种方法中，我们生成 B 个不同的自助训练数据集。然后，我们在第 b 个自助训练集上训练我们的方法，以得到$\hat{f}^{*b}(x)$，最后对所有预测求平均，得到

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x).$$

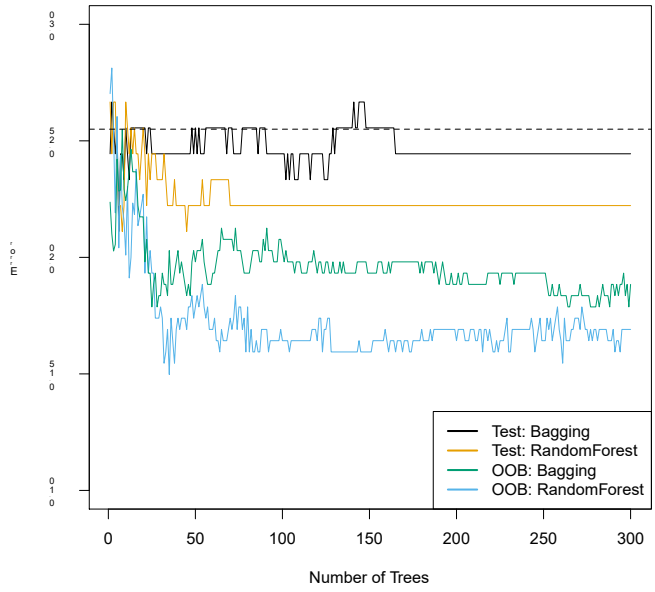


图 8.8. *Bagging and random forest results for the* `Heart` *data. The test error (black and orange) is shown as a function of* B*, the number of bootstrapped training sets used. Random forests were applied with* $m = \sqrt{p}$*. The dashed line indicates the test error resulting from a single classification tree. The green and blue traces show the OOB error, which in this case is — by chance — considerably lower.*

这被称为袋装法。

虽然bagging可以提高许多回归方法的预测效果，但它对决策树特别有用。为了将bagging应用于回归树，我们简单地使用B个自助训练集构建B棵回归树，并对得到的预测结果取平均。这些树生长得很深，并且没有进行剪枝。因此，每棵单独的树都有高方差，但低偏差。对这B棵树取平均可以减少方差。bagging已经证明，通过将数百甚至数千棵树组合成一个单一过程，可以显著提高准确性。

到目前为止，我们已在回归背景下描述了装袋过程，用于预测定量结果Y。如何将装袋扩展到Y为定性的分类问题？在这种情况下，有几种可能的方法，但最简单的方法如下。对于给定的测试观测，我们可以记录 B 棵树中每棵预测的类别，并采取 *majority vote*：总体预测是最常出现的

majority vote

在B个预测中的类别。

图8.8展示了在`Heart`数据上对树进行装袋的结果。测试错误率显示为B的函数，即使用自助训练数据集构建的树的数量。我们看到，在这种情况下，装袋测试错误率略低于从单个树获得的测试错误率。树的数量B不是装袋的关键参数；使用非常大的B值不会导致过拟合。在实践中，我们

使用一个足够大的 B 值，使误差趋于稳定。在此示例中，使用 $B = 100$ 足以达到良好的性能。

Out-of-Bag 误差估计

事实证明，有一种非常直接的方法可以估计袋装模型的测试误差，而无需进行交叉验证或验证集方法。回想一下，袋装法的关键在于树被反复拟合到观测的自助子集上。可以证明，平均而言，每棵袋装树利用了大约三分之二的观测值。[3] 未用于拟合给定袋装树的剩余三分之一观测值被称为 *out-of-bag* (OOB) 观测值。我们

out-of-bag

可以预测第i个观测的响应，使用该观测为OOB的每一棵树。这将为第i个观测产生大约$B/3$个预测。为了获得第i个观测的单一预测，我们可以对这些预测响应取平均（如果目标是回归）或进行多数投票（如果目标是分类）。这导致第i个观测的单一OOB预测。通过这种方式，可以为n个观测中的每一个获得OOB预测，从而可以计算总体OOB MSE（对于回归问题）或分类误差（对于分类问题）。得到的OOB误差是对袋装模型测试误差的有效估计，因为每个观测的响应仅使用未使用该观测拟合的树来预测。图8.8显示了`Heart`数据上的OOB误差。可以证明，当B足够大时，OOB误差实际上等同于留一交叉验证误差。在对大型数据集进行袋装时，OOB方法用于估计测试误差特别方便，因为交叉验证在计算上会很繁重。

变量重要性度量

如前所述，装袋法通常能比使用单棵树的预测提高准确性。然而，不幸的是，解释由此产生的模型可能很困难。回想一下，决策树的优点之一是产生吸引人且易于解释的图表，如图8.1所示。然而，当我们对大量树进行装袋时，不再可能用单棵树来表示所得的统计学习过程，也不再清楚哪些变量对过程最重要。因此，装袋法以可解释性为代价提高了预测准确性。

尽管装袋树的集合比单棵树更难解释，但我们可以利用RSS（对于装袋回归树）或基尼指数（对于装袋分类树）来获得每个预测变量重要性的总体概览。在装袋回归树的情况下，我们可以记录由给定预测变量上的分割导致的RSS（8.1）减少的总量，并在所有B棵树上取平均。数值较大表示该预测变量较为重要。同样地，在装袋分类的背景下，

[3] This relates to Exercise 2 of Chapter 5.

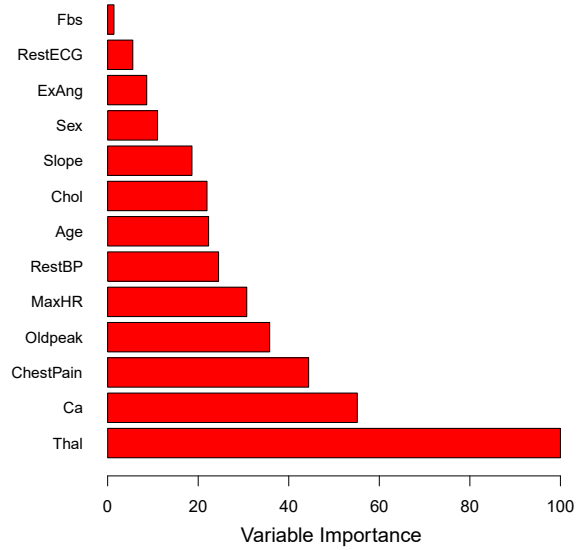


图8.9。*A variable importance plot for the* `Heart` *data. Variable importance is computed using the mean decrease in Gini index, and expressed relative to the maximum.*

对于树，我们可以将由给定预测变量上的分裂导致的基尼指数（8.6）减少的总量相加，并在所有B棵树上取平均。

`Heart`数据中*variable importances*的图形表示如图8.9所示。我们看到了每个变量相对于最大值的基尼指数平均减少量。基尼指数平均减少量最大的变量是`Thal`，`Ca`和`ChestPain`。

变量重要性

8.2.2 Random Forests

Random forests 通过以下方式提供对袋装树的改进：微小的调整，*decorrelates*了树。如同装袋法，我们在自助训练样本上构建多棵决策树。但在构建这些决策树时，每次考虑树中的一个分裂，都会从全部p个预测变量中选取*a random sample of* m *predictors*作为分裂候选。分裂仅允许使用这些m个预测变量中的一个。每次分裂都会抽取一个新的预测变量样本m，通常我们选择$m \approx \sqrt{p}$——即，每次分裂考虑的预测变量数量大约等于总预测变量数的平方根（对于`Heart`数据，即13个中的4个）。

随机森林

换句话说，在构建随机森林时，对于树中的每一次分裂，算法都会*not even allowed to consider*可用预测变量中的大多数。这听起来可能有些疯狂，但它有一个巧妙的理由。假设数据集中有一个非常强的预测变量，以及许多其他中等强度的预测变量。那么，在装袋树的集合中，大多数或所有树都会在顶层分裂中使用这个强预测变量。因此，所有的装袋树都会看起来彼此非常相似。

因此，来自装袋树的预测将会高度相关。不幸的是，对许多高度相关的量进行平均，并不会像对许多不相关的量进行平均那样大幅度减少方差。特别是，这意味着在这种情况下，装袋不会比单一树导致方差的显著减少。

随机森林通过强制每个分裂仅考虑预测变量的一个子集来克服这个问题。因此，平均而言（$p - m$）$/p$的分裂甚至不会考虑强预测变量，因此其他预测变量将有更多机会。我们可以将此过程视为对树进行*decorrelating*，从而使所得树的平均值变化更小，因此更可靠。

Bagging 与随机森林的主要区别在于预测变量子集大小 m 的选择。例如，如果使用 $m = p$ 构建随机森林，那么这实质上就等同于 bagging。在 `Heart` 数据上，使用 $m = \sqrt{p}$ 的随机森林相较于 bagging 在测试误差和 OOB 误差上都有所降低（图 8.8）。

在构建随机森林时，使用较小的m值通常会在我们拥有大量相关预测变量时有所帮助。我们将随机森林应用于一个高维生物数据集，该数据集包含从349名患者的组织样本中测得的4，718个基因的表达测量值。人类大约有20，000个基因，单个基因在特定细胞、组织和生物条件下具有不同的活性或表达水平。在这个数据集中，每个患者样本都有一个具有15个不同水平的定性标签：要么是正常，要么是14种不同类型癌症中的1种。我们的目标是使用随机森林，基于训练集中方差最大的500个基因来预测癌症类型。我们将观测数据随机分为训练集和测试集，并对训练集应用随机森林，考察分裂变量数目的三个不同取值m。结果如图8.10所示。单棵树的错误率为45.7%，而零模型率为75.4%。[4]我们看到，使用400棵树就足以给出良好性能，并且在这个例子中，选择$m = \sqrt{p}$相比bagging（$m = p$）在测试误差上带来了小幅改进。与bagging一样，如果我们增大B，随机森林不会过拟合，因此在实际应用中，我们使用一个足够大的B值，使错误率已经稳定下来。

8.2.3 Boosting

我们现在讨论 {v*}，这是另一种改进预测的方法。

boosting

由决策树产生的预测。与bagging一样，boosting是一种通用方法，可应用于许多统计学习方法中，用于回归或分类。在此，我们将boosting的讨论限制在决策树的上下文中。

回想一下，袋装法涉及使用自助法创建原始训练数据集的多个副本，为每个副本拟合一个单独的决策树，然后结合所有树以创建一个单一的预测。

[4]The null rate results from simply classifying each observation to the dominant class overall, which is in this case the normal class.

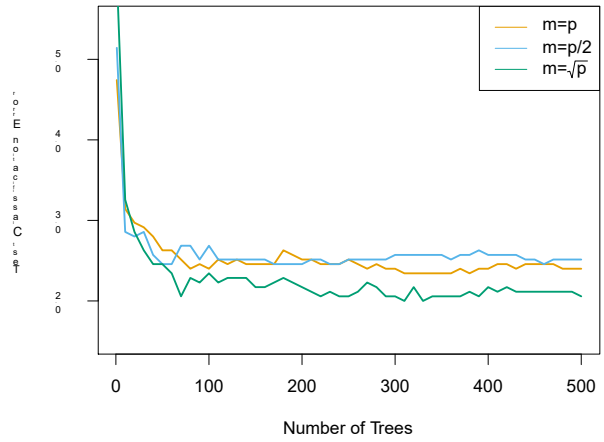


图8.10。 *Results from random forests for the 15-class gene expression data set with* $p = 500$ *predictors. The test error is displayed as a function of the number of trees. Each colored line corresponds to a different value of* m*, the number of predictors available for splitting at each interior tree node. Random forests* ($m < p$) *lead to a slight improvement over bagging* ($m = p$)*. A single classification tree has an error rate of 45.7%.*

自适应模型。值得注意的是，每棵树都是基于一个自助数据集构建的，相互独立。提升法的工作原理类似，但树的生长方式不同*sequentially*：每棵树都是利用先前生长树的信息来生长。提升法不涉及自助采样；相反，每棵树都是拟合原始数据集的一个修改版本。

首先考虑回归设置。与bagging类似，boosting涉及结合大量的决策树，$\hat{f}^1, \ldots, \hat{f}^B$。Boosting在算法8.2中描述。

这个过程的理念是什么？与拟合一个大型决策树到数据上，这相当于*fitting the data hard*并可能过拟合，而提升方法则是*learns slowly*。给定当前模型，我们针对模型的残差拟合一个决策树。也就是说，我们使用当前残差而非结果Y作为响应来拟合树。然后，我们将这棵新的决策树加入到已拟合的函数中，以更新残差。这些树中的每一棵都可以相当小，仅拥有少数几个终端节点，这由算法中的参数d决定。通过对残差拟合小型树，我们在模型表现不佳的区域逐步改进$\hat{f}$。收缩参数λ进一步减缓这一过程，允许更多形状各异的树去处理残差。通常，那些*learn slowly*的统计学习方法往往表现良好。需要注意的是，在提升方法中，与装袋法不同，每棵树的构建强烈依赖于已经生长出的树。

我们刚刚描述了提升回归树的过程。提升分类树的过程类似但稍微复杂一些，这里省略细节。

算法8.2 *Boosting for Regression Trees*

1. 将训练集中的所有 i 的 $\hat{f}(x) =$ 设为 0，$r_i = y_i$ 设为 i。

2. 对于 $b = 1, 2, \ldots, B$，重复：

   (a) 用 d 次分裂（$d+$ 个终端节点）将一棵树 $\hat{f}^b$ 拟合到训练数据（X, r）中。

   (b) 通过加入新树的收缩版本更新 $\hat{f}$ ：

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \tag{8.10}$$

   (c) 更新残差，

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \tag{8.11}$$

输出增强后的模型，

$$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x). \tag{8.12}$$

Boosting有三个调参参数：

1. 树的数量 B。与装袋法和随机森林不同，如果 B 过大，提升法可能会过拟合，尽管这种过拟合往往发生得较慢，甚至可能不会发生。我们使用交叉验证来选择 B。

2. 收缩参数 λ，一个小的正数。这控制着提升学习的速度。典型值为 0.01 或 0.001，正确的选择可能取决于具体问题。非常小的 λ 可能需要使用非常大的 B 值才能达到良好的性能。

3. 每棵树中的分裂次数 d，它控制着提升集成的复杂度。通常 $d = 1$ 就很有效，此时每棵树是一个 *stump*，仅由一次分裂构成。在这种情况下，提升
   集成是在拟合一个加法模型，因为每一项仅涉及单个变量。更一般地，d是*interaction depth*，并且控制
   提升模型的交互阶数，因为 d 分裂最多涉及 d 个变量。

树桩

交互深度

在图8.11中，我们对15类癌症基因表达数据集应用了提升方法，以开发一个能够将正常类别与14个癌症类别区分开的分类器。我们展示了测试误差作为树的总数和交互深度d的函数。我们看到，如果包含足够多的简单树桩，其交互深度为1时表现良好。这个模型优于深度为2的模型，而两者都优于随机森林。这突显了提升与随机森林之间的一个差异：在提升中，由于特定树的生长考虑了已经生长的其他树，较小的

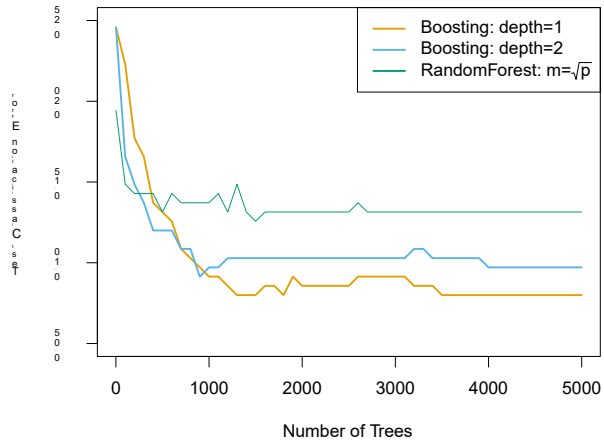


图8.11. *Results from performing boosting and random forests on the 15-class gene expression data set in order to predict* 癌症 *versus* 正常. *The test error is displayed as a function of the number of trees. For the two boosted models,* $\lambda = 0.01$. *Depth-1 trees slightly outperform depth-2 trees, and both outperform the random forest, although the standard errors are around 0.02, making none of these differences significant. The test error rate for a single tree is 24 %.*

树通常是足够的。使用较小的树也有助于提高可解释性；例如，使用树桩会导致一个可加模型。

8.2.4 Bayesian Additive Regression Trees

贝叶斯加法回归树

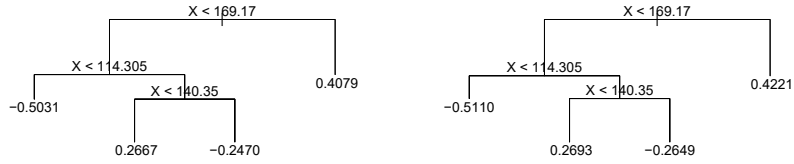
最后，我们讨论 *Bayesian additive regression trees* (BART)，另一个 en- 使用决策树作为构建模块的集成方法。为简便起见，我们展示用于回归的 BART（而非分类）。

回忆一下，bagging 和随机森林通过对回归树的平均值进行预测，每棵树都是基于数据和/或预测变量的随机样本构建的。每棵树都是独立于其他树构建的。相比之下，boosting 使用树的加权和，每棵树都是通过将树拟合到当前拟合的残差来构建的。因此，每棵新树都试图捕捉当前树集尚未解释的信号。BART 与这两种方法都有关联：每棵树都像 bagging 和随机森林中那样以随机方式构建，并且每棵树都试图捕捉当前模型尚未解释的信号，就像 boosting 中那样。BART 的主要创新在于新树的生成方式。

在我们介绍 BART 算法之前，先定义一些符号。我们让 K 表示回归树的数量，B 表示 BART 算法将运行的迭代次数。符号 $\hat{f}_k^b(x)$ 表示在第 b 次迭代中使用的第 k 棵回归树在 x 处的预测。每次迭代结束时，该次迭代中的 K 棵树将被求和，即对于 $b = 1, \ldots, B$，有 $\hat{f}^b(x) = \sum_{k=1}^{K} \hat{f}_k^b(x)$。

在BART算法的首次迭代中，所有树均被初始化为仅含一个根节点，该节点的值为 $\hat{f}_k^1(x) = \frac{1}{nK} \sum_{i=1}^{n} y_i$，即响应的均值。

(a): $\hat{f}_k^{b-1}(X)$ (b): Possibility #1 for $\hat{f}_k^b(X)$



(c): Possibility #2 for $\hat{f}_k^b(X)$ (d): Possibility #3 for $\hat{f}_k^b(X)$

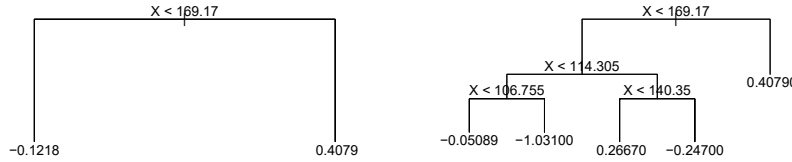


图 8.12. *A schematic of perturbed trees from the BART algorithm.* (a): *The kth tree at the* $(b-1)$*st iteration,* $\hat{f}_k^{b-1}(X)$*, is displayed. Panels (b)–(d) display three of many possibilities for* $\hat{f}_k^b(X)$*, given the form of* $\hat{f}_k^{b-1}(X)$*.* (b): *One possibility is that* $\hat{f}_k^b(X)$ *has the same structure as* $\hat{f}_k^{b-1}(X)$*, but with different predictions at the terminal nodes.* (c): *Another possibility is that* $\hat{f}_k^b(X)$ *results from pruning* $\hat{f}_k^{b-1}(X)$*.* (d): *Alternatively,* $\hat{f}_k^b(X)$ *may have more terminal nodes than* $\hat{f}_k^{b-1}(X)$*.*

值除以树的总数。因此，$\hat{f}^1(x) = \sum_{k=1}^K \hat{f}_k^1(x) = \frac{1}{n}\sum_{i=1}^n y_i$。

–

在后续迭代中，BART会逐个更新每一棵 K 树。在第 b 次迭代中，为了更新第 k 棵树，我们从每个响应值中减去除第 k 棵树之外的所有树的预测值，以获得一个 *partial residual*。

$$r_i = y_i - \sum_{k'<k} \hat{f}_{k'}^b(x_i) - \sum_{k'>k} \hat{f}_{k'}^{b-1}(x_i)$$

对于第 i 个观测值，$i = 1, \ldots, n$。而不是对这个部分残差拟合一棵新树，BART 从一组可能的扰动中随机选择对上一轮迭代的树（$\hat{f}_k^{b-1}$）进行扰动，倾向于那些能改善对部分残差拟合的扰动。这个扰动包含两个部分：

1. 我们可以通过添加或修剪分支来改变树的结构。

2. 我们可以更改树中每个终端节点的预测。

图8.12展示了可能对树进行的扰动示例。BART的输出是一系列预测模型，

$$\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x), \text{ for } b = 1, 2, \ldots, B.$$

算法8.3 *Bayesian Additive Regression Trees*

1. 设$\hat{f}_1^1(x) = \hat{f}_2^1(x) = \cdots = \hat{f}_K^1(x) = \frac{1}{nK}\sum_{i=1}^n y_i$。

2. 计算$\hat{f}^1(x) = \sum_{k=1}^K \hat{f}_k^1(x) = \frac{1}{n}\sum_{i=1}^n y_i$。

3. 对于 $b = 2, \ldots, B$:

   (a) 对于 $k = 1, 2, \ldots, K$:

      i. 对于 $i = 1, \ldots, n$，计算当前部分残差

$$r_i = y_i - \sum_{k'<k} \hat{f}_{k'}^b(x_i) - \sum_{k'>k} \hat{f}_{k'}^{b-1}(x_i).$$

      ii. 通过随机扰动上一次迭代中的第k棵树，$\hat{f}_k^{b-1}(x)$，来拟合一棵新树，$\hat{f}_k^b(x)$，以适应r_i。能够改善拟合效果的扰动更受青睐。

   (b) Compute $\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x)$.

4. 计算 L 个预热样本后的均值，

$$\hat{f}(x) = \frac{1}{B-L}\sum_{b=L+1}^{B} \hat{f}^b(x).$$

我们通常会丢弃这些预测模型中的前几个，因为在较早迭代中得到的模型——称为*burn-in*阶段
— 往往不能提供很好的结果。我们可以让 L 表示预热迭代的次数；例如，我们可以取 $L = 200$。然后，为了获得单一预测，我们只需在预热迭代后取平均值，$\hat{f}(x) = \frac{1}{B-L}\sum_{b=L+1}^{B} \hat{f}^b(x)$。然而，也可以计算除平均值之外的其他量：例如，$\hat{f}^{L+1}(x), \ldots, \hat{f}^B(x)$ 的百分位数提供了最终预测中不确定性的度量。整体 BART 过程总结在算法 8.3 中。

burn-in

BART 方法的一个关键要素在于，在步骤 3(a)ii. 中，我们并非通过 *not* 对当前部分残差拟合一棵全新的树：相反，我们试图通过略微修改前一次迭代中得到的树来改善对当前部分残差的拟合（参见图 8.12）。粗略地说，这可以防止过拟合，因为它限制了我们在每次迭代中拟合数据的“强度”。此外，单棵树通常相当小。我们限制树的大小，以避免对数据过拟合，而如果生长非常大的树，则更有可能发生过拟合。

图8.13展示了将BART应用于`Heart`数据，使用K =200棵树，并当迭代次数增加到10,000时的结果。在最初的迭代过程中，测试误差和训练误差有些波动。经过这段初始预热期后，错误率趋于稳定。我们注意到，训练误差与测试误差之间的差异很小，这表明树扰动过程在很大程度上避免了过拟合。

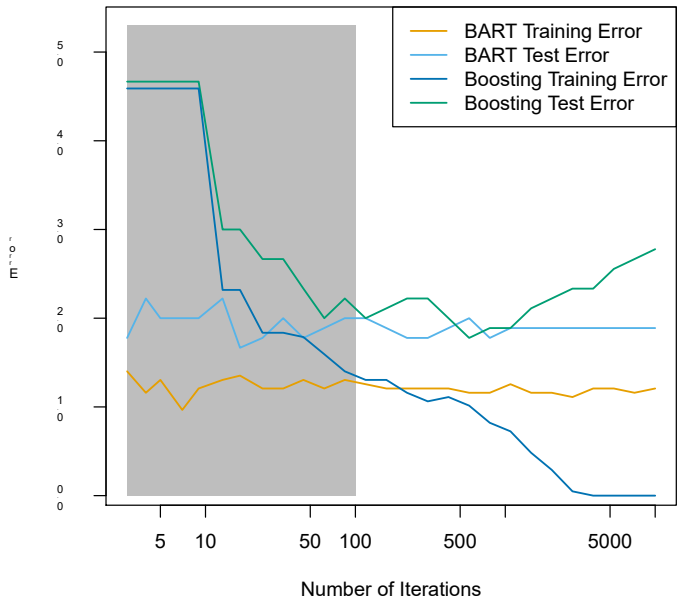


图 8.13. *BART and boosting results for the* `Heart` *data. Both training and test errors are displayed. After a burn-in period of* 100 *iterations (shown in gray), the error rates for BART settle down. Boosting begins to overfit after a few hundred iterations.*

提升方法的训练误差和测试误差也展示在图8.13中。我们可以看到，提升方法的测试误差接近于BART，但随着迭代次数的增加，测试误差开始增大。此外，提升方法的训练误差随着迭代次数的增加而减小，这表明提升方法对数据出现了过拟合。

尽管细节超出了本书的范围，但事实证明，BART 方法可以被视为一种 *Bayesian* 方法来拟合树的集成：每次我们为了拟合残差而随机扰动一棵树时，实际上是从 *posterior* 分布中抽取一棵新树。（当然，这种贝叶斯联系正是 BART 名称的动机。）此外，算法 8.3 可以被视为一种 *Markov chain Monte Carlo* 用于拟合BART模型的算法。

Markov chain Monte Carlo

当我们应用BART时，必须选择树的数量K、迭代次数B以及预烧迭代次数L。我们通常为B和K选择较大的值，而为L选择一个适中的值：例如，K =取200，B =取1,000，L =取100是一个合理的选择。BART已展现出非常出色的开箱即用性能——也就是说，它在最小调参下也能表现良好。

8.2.5 Summary of Tree Ensemble Methods

树是集成方法中弱学习器的一个有吸引力的选择，原因有很多，包括它们的灵活性以及处理能力。

混合类型的预测变量（即定性和定量）。我们现在已经看到了四种拟合树集成的方法：装袋法、随机森林、提升法和BART。

- 在*bagging*中，树木独立地在观察的随机样本上生长。因此，这些树往往彼此相当相似。因此，套袋法可能会陷入局部最优，且可能无法彻底探索模型空间。

- 在*random forests*中，树依然独立地在观察的随机样本上生长。然而，每棵树上的每次分裂都是通过特征的随机子集来执行的，从而使树之间去相关，并相对于装袋法导致对模型空间更为彻底的探索。

- 在*boosting*中，我们仅使用原始数据，不抽取任何随机样本。树是逐步生长的，采用了一种“慢”学习方式：每棵新树都适应于先前树木留下的信号，并在使用前进行缩减。

- 在*BART*中，我们再次仅利用原始数据，并逐次生成树。然而，为了避开局部最小值并更全面地探索模型空间，每棵树都经过了扰动处理。

8.3 实验：基于树的方法

我们在此顶层导入一些常用的库。

In [1]:
```
import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
from statsmodels.datasets import get_rdataset
import sklearn.model_selection as skm
from ISLP import load_data, confusion_table
from ISLP.models import ModelSpec as MS
```

我们还收集了本实验所需的新导入内容。

In [2]:
```
from sklearn.tree import (DecisionTreeClassifier as DTC,
                          DecisionTreeRegressor as DTR,
                          plot_tree,
                          export_text)
from sklearn.metrics import (accuracy_score,
                             log_loss)
from sklearn.ensemble import \
     (RandomForestRegressor as RF,
      GradientBoostingRegressor as GBR)
from ISLP.bart import BART
```

8.3.1 Fitting Classification Trees

我们首先使用分类树来分析 Carseats 数据集。在这些数据中，Sales 是一个连续变量，因此我们首先将其重新编码为二值变量。我们使用 where() 函数创建一个变量，称为 High，如果Sales变量超过8，则取值为Yes，否则取值为No。

where()

In [3]:
```
Carseats = load_data('Carseats')
High = np.where(Carseats.Sales > 8,
                "Yes",
                "No")
```

我们现在使用 DecisionTreeClassifier() 来按顺序拟合一个分类树为了使用除Sales以外的所有变量预测High，我们必须像拟合回归模型时那样构建模型矩阵。

DecisionTree Classifier()

In [4]:
```
model = MS(Carseats.columns.drop('Sales'), intercept=False)
D = model.fit_transform(Carseats)
feature_names = list(D.columns)
X = np.asarray(D)
```

我们已将 D 从数据框转换为数组 X，这在下面的某些分析中是必需的。我们还需要 feature_names 以便稍后注释我们的图表。

需要若干选项来指定分类器，例如 max_depth (树生长的深度)、min_samples_split (节点中可参与分裂的最小观测数)以及 criterion (使用基尼系数还是交叉熵作为分裂准则)。我们还设置了 random_state 以确保可重复性；分裂准则中的并列情况随机打破。

In [5]:
```
clf = DTC(criterion='entropy',
          max_depth=3,
          random_state=0)
clf.fit(X, High)
```

Out[5]:
```
DecisionTreeClassifier(criterion='entropy', max_depth=3)
```

在第3.3节关于定性特征的讨论中，我们提到对于线性回归模型，这样的特征可以通过在模型矩阵中包含一个哑变量矩阵（独热编码）来表示，使用statsmodels的公式表示法。正如第8.1节所述，在构建决策树时，有一种更自然的方法来处理定性特征，不需要这样的哑变量；每次分裂相当于将水平分成两组。然而，决策树的sklearn实现并未利用这种方法；相反，它只是将独热编码后的水平视为单独的变量。

In [6]:
```
accuracy_score(High, clf.predict(X))
```

Out[6]:
```
0.7275
```

仅使用默认参数时，训练错误率为21%。对于分类树，我们可以使用 log_loss()来获取偏差值，

log_loss()

$$-2\sum_{m}\sum_{k} n_{mk}\log\hat{p}_{mk},$$

其中n_{mk}是属于第k类的第m个终端节点中的观测数量。

In [7]:
```
resid_dev = np.sum(log_loss(High, clf.predict_proba(X)))
resid_dev
```

Out[7]: 0.4711

这与公式 (8.7) 中定义的 *entropy* 密切相关。小的偏差表明该树对（训练）数据拟合良好。

树最吸引人的特性之一是它们可以以图形方式显示。这里我们使用 `plot()`函数来显示树结构（此处未展示）。

In [8]:
```
ax = subplots(figsize=(12,12))[1]
plot_tree(clf,
          feature_names=feature_names,
          ax=ax);
```

Sales最重要的指标似乎是ShelveLoc。

我们可以看到使用 `export_text()` 的树的文本表示，它显示每个分支的分裂标准（例如 `Price <= 92.5`）。对于叶节点，它显示整体预测（`Yes` 或 `No`）。通过指定 `show_weights=True`，我们还可以看到该叶中取值为 `Yes` 和 `No` 的观测数量。

export_text()

In [9]:
```
print(export_text(clf,
                  feature_names=feature_names,
                  show_weights=True))
```

Out[9]:
```
|--- ShelveLoc[Good] <= 0.50
|   |--- Price <= 92.50
|   |   |--- Income <= 57.00
|   |   |   |--- weights: [7.00, 3.00] class: No
|   |   |--- Income >  57.00
|   |   |   |--- weights: [7.00, 29.00] class: Yes
|   |--- Price >  92.50
|   |   |--- Advertising <= 13.50
|   |   |   |--- weights: [183.00, 41.00] class: No
|   |   |--- Advertising >  13.50
|   |   |   |--- weights: [20.00, 25.00] class: Yes
|--- ShelveLoc[Good] >  0.50
|   |--- Price <= 135.00
|   |   |--- US[Yes] <= 0.50
|   |   |   |--- weights: [6.00, 11.00] class: Yes
|   |   |--- US[Yes] >  0.50
|   |   |   |--- weights: [2.00, 49.00] class: Yes
|   |--- Price >  135.00
|   |   |--- Income <= 46.00
|   |   |   |--- weights: [6.00, 0.00] class: No
|   |   |--- Income >  46.00
|   |   |   |--- weights: [5.00, 6.00] class: Yes
```

为了正确评估分类树在这些数据上的性能，我们必须估计测试误差，而不是简单地计算训练误差。我们将观测数据分为训练集和测试集，使用训练集构建树，并在测试数据上评估其性能。这一模式与第6章相似，只是这里的线性模型被决策树所取代——验证的代码几乎相同。这种方法对测试数据集中的位置做出了68.5%的正确预测。

In [10]:
```
validation = skm.ShuffleSplit(n_splits=1,
                              test_size=200,
                              random_state=0)
results = skm.cross_validate(clf,
                             D,
                             High,
                             cv=validation)
results['test_score']
```

Out[10]:
```
array([0.685])
```

接下来，我们考虑修剪树是否能提升分类性能。首先，我们将数据分为训练集和测试集。我们将利用交叉验证在训练集上对树进行修剪，然后在测试集上评估修剪后树的性能。

In [11]:
```
(X_train,
 X_test,
 High_train,
 High_test) = skm.train_test_split(X,
                                   High,
                                   test_size=0.5,
                                   random_state=0)
```

我们首先在训练集上重新拟合完整树；这里我们不设置 `max_depth` 参数，因为我们将通过交叉验证来学习它。

In [12]:
```
clf = DTC(criterion='entropy', random_state=0)
clf.fit(X_train, High_train)
accuracy_score(High_test, clf.predict(X_test))
```

Out[12]:
```
0.735
```

Next we use the `cost_complexity_pruning_path()` method of `clf` to extract cost-complexity values.

`cost_complexity_pruning_path()`

In [13]:
```
ccp_path = clf.cost_complexity_pruning_path(X_train, High_train)
kfold = skm.KFold(10,
                  random_state=1,
                  shuffle=True)
```

This yields a set of impurities and α values from which we can extract an optimal one by cross-validation.

In [14]:
```
grid = skm.GridSearchCV(clf,
                        {'ccp_alpha': ccp_path.ccp_alphas},
                        refit=True,
```

```
                         cv=kfold,
                         scoring='accuracy')
grid.fit(X_train, High_train)
grid.best_score_
```

```
Out[14]: 0.685
```

让我们看一下修剪后的真值。

```
In [15]: ax = subplots(figsize=(12, 12))[1]
         best_ = grid.best_estimator_
         plot_tree(best_,
                   feature_names=feature_names,
                   ax=ax);
```

This is quite a bushy tree. We could count the leaves, or query `best_` instead.

```
In [16]: best_.tree_.n_leaves
```

```
Out[16]: 30
```

具有30个终端节点的树产生了最低的交叉验证错误率，准确率为68.5%。这个经过修剪的树在测试数据集上的表现如何？我们再次应用`predict()`函数。

```
In [17]: print(accuracy_score(High_test,
                              best_.predict(X_test)))
         confusion = confusion_table(best_.predict(X_test),
                                     High_test)
         confusion
```

```
Out[17]: 0.72
```

Truth	No	Yes
Predicted		
No	108	61
Yes	10	21

现在，测试观测中有72.0%被正确分类，这比完整树（有35个叶节点）的错误率略差。因此，交叉验证在这里并没有帮到我们太多；它仅修剪掉了5个叶节点，代价是错误率略有上升。如果改变上述随机数种子，这些结果会有所变化；尽管交叉验证提供了一种无偏的模型选择方法，但它确实存在方差。

8.3.2 Fitting Regression Trees

这里我们对`Boston`数据集拟合一个回归树。步骤与分类树类似。

```
In [18]: Boston = load_data("Boston")
         model = MS(Boston.columns.drop('medv'), intercept=False)
         D = model.fit_transform(Boston)
         feature_names = list(D.columns)
         X = np.asarray(D)
```

首先，我们将数据分为训练集和测试集，并用训练数据拟合树。这里我们使用30%的数据作为测试集。

In [19]:
```
(X_train,
 X_test,
 y_train,
 y_test) = skm.train_test_split(X,
                                Boston['medv'],
                                test_size=0.3,
                                random_state=0)
```

在形成我们的训练和测试数据集后，我们拟合了回归树。

In [20]:
```
reg = DTR(max_depth=3)
reg.fit(X_train, y_train)
ax = subplots(figsize=(12,12))[1]
plot_tree(reg,
          feature_names=feature_names,
          ax=ax);
```

变量lstat衡量社会经济地位较低个体的百分比。树表明lstat值越低对应于房价越高。树预测小型房屋（rm < 6.8）的中位房价为12042美元，这些房屋位于居民社会经济地位低（lstat > 14.4）且犯罪率中等（crim > 5.8）的郊区。

现在我们使用交叉验证函数来查看修剪树是否能提高性能。

In [21]:
```
ccp_path = reg.cost_complexity_pruning_path(X_train, y_train)
kfold = skm.KFold(5,
                  shuffle=True,
                  random_state=10)
grid = skm.GridSearchCV(reg,
                        {'ccp_alpha': ccp_path.ccp_alphas},
                        refit=True,
                        cv=kfold,
                        scoring='neg_mean_squared_error')
G = grid.fit(X_train, y_train)
```

与交叉验证结果一致，我们使用修剪后的树在测试集上进行预测。

In [22]:
```
best_ = grid.best_estimator_
np.mean((y_test - best_.predict(X_test))**2)
```

Out[22]: 28.07

换句话说，与回归树相关的测试集MSE为28.07。因此，MSE的平方根约为5.30，表明该模型导致的测试预测值与郊区真实房价中位数的偏差大约在$5300以内。

让我们绘制最佳树，看看它的可解释性如何。

In [23]:
```
ax = subplots(figsize=(12,12))[1]
plot_tree(G.best_estimator_,
          feature_names=feature_names,
          ax=ax);
```

8.3.3 Bagging and Random Forests

这里我们将bagging和随机森林应用于Boston数据，使用来自sklearn.ensemble包的RandomForestRegressor()。回忆一下bagging只是具有 $m = p$ 的随机森林的一个特例。因此，RandomForestRegressor() 函数可用于执行 bagging 和随机森林。我们从 bagging 开始。

RandomForestRegressor()
sklearn.ensemble

```
In [24]: bag_boston = RF(max_features=X_train.shape[1], random_state=0)
         bag_boston.fit(X_train, y_train)
```

```
Out[24]: RandomForestRegressor(max_features=12, random_state=0)
```

参数 max_features 表明在树的每次分裂中应考虑所有 12 个预测变量——换言之，应进行袋装法。这个袋装模型在测试集上的表现如何?

```
In [25]: ax = subplots(figsize=(8,8))[1]
         y_hat_bag = bag_boston.predict(X_test)
         ax.scatter(y_hat_bag, y_test)
         np.mean((y_test - y_hat_bag)**2)
```

```
Out[25]: 14.63
```

与装袋回归树相关的测试集MSE为14.63，大约是使用最优修剪单棵树所得结果的一半。我们可以通过使用 n_estimators 参数来改变默认种植的树的数量（默认值为100）：

```
In [26]: bag_boston = RF(max_features=X_train.shape[1],
                         n_estimators=500,
                         random_state=0).fit(X_train, y_train)
         y_hat_bag = bag_boston.predict(X_test)
         np.mean((y_test - y_hat_bag)**2)
```

```
Out[26]: 14.61
```

变化不大。Bagging和随机森林不会因为增加树的数量而过拟合，但如果数量太少则可能欠拟合。

生长随机森林的过程完全相同，只是我们使用了较小的 max_features 参数值。默认情况下，RandomForestRegressor() 在构建回归树的随机森林时使用 p 个变量（即默认为袋装法），而 RandomForestClassifier() 在构建分类树的随机森林时使用 $\sqrt{p}$ 个变量。这里我们使用 max_features=6。

```
In [27]: RF_boston = RF(max_features=6,
                        random_state=0).fit(X_train, y_train)
         y_hat_RF = RF_boston.predict(X_test)
         np.mean((y_test - y_hat_RF)**2)
```

```
Out[27]: 20.04
```

测试集MSE为20.04；这表明在这种情况下，随机森林的表现比bagging稍差。从拟合模型中提取feature_importances_值，我们可以查看每个变量的重要性。

In [28]:
```
feature_imp = pd.DataFrame(
    {'importance':RF_boston.feature_importances_},
    index=feature_names)
feature_imp.sort_values(by='importance', ascending=False)
```

Out[28]:

	importance
lstat	0.368683
rm	0.333842
ptratio	0.057306
indus	0.053303
crim	0.052426
dis	0.042493
nox	0.034410
age	0.024327
tax	0.022368
rad	0.005048
zn	0.003238
chas	0.002557

这是一个相对度量，表示通过对该变量进行分割而导致的节点不纯度的总下降量，在所有树上取平均（在图8.9中绘制了拟合到{v*}数据的模型）。

结果表明，在随机森林中考虑的所有树中，社区的财富水平（lstat）和房屋大小（rm）是迄今为止最重要的两个变量。

8.3.4 Boosting

这里我们使用来自 sklearn.ensemble 的 GradientBoostingRegressor() 进行拟合提升回归树应用于 Boston 数据集。对于分类问题，我们将使用 GradientBoostingClassifier()。参数 n_estimators=5000 表示表明我们希望有5000棵树，而选项max_depth=3限制了每棵树的深度。参数 learning_rate是之前 boosting 描述中提到的λ。

GradientBoostingRegressor()

GradientBoostingClassifier()

In [29]:
```
boost_boston = GBR(n_estimators=5000,
                   learning_rate=0.001,
                   max_depth=3,
                   random_state=0)
boost_boston.fit(X_train, y_train)
```

我们可以看到训练误差如何随着 train_score_ 属性而减少。为了了解测试误差如何减少，我们可以使用 staged_predict() 方法来获取沿路径的预测值。

In [30]:
```
test_error = np.zeros_like(boost_boston.train_score_)
for idx, y_ in enumerate(boost_boston.staged_predict(X_test)):
   test_error[idx] = np.mean((y_test - y_)**2)

plot_idx = np.arange(boost_boston.train_score_.shape[0])
ax = subplots(figsize=(8,8))[1]
ax.plot(plot_idx,
        boost_boston.train_score_,
        'b',
        label='Training')
```

```
ax.plot(plot_idx,
        test_error,
        'r',
        label='Test')
ax.legend();
```

我们现在使用提升模型在测试集上预测 medv：

In [31]:
```
y_hat_boost = boost_boston.predict(X_test);
np.mean((y_test - y_hat_boost)**2)
```

Out[31]: 14.48

获得的测试MSE为14.48，与bagging的测试MSE相似。如有需要，我们可以在(8.10)中使用不同的收缩参数值λ来进行提升。默认值为0.001，但这很容易修改。此处我们取$\lambda = 0.2$。

In [32]:
```
boost_boston = GBR(n_estimators=5000,
                   learning_rate=0.2,
                   max_depth=3,
                   random_state=0)
boost_boston.fit(X_train,
                 y_train)
y_hat_boost = boost_boston.predict(X_test);
np.mean((y_test - y_hat_boost)**2)
```

Out[32]: 14.50

在这种情况下，使用$\lambda = 0.2$会导致与使用$\lambda = 0.001$时几乎相同的测试MSE。

8.3.5 Bayesian Additive Regression Trees

在本节中，我们将展示在 ISLP.bart 包中发现的 BART 的 Python 实现。我们针对 Boston 房屋数据集拟合了一个模型。尽管这个 BART() 估计器是为定量结果变量设计的，但是
其他实现可用于拟合分类结果的logistic和probit模型。

BART()

In [33]:
```
bart_boston = BART(random_state=0, burnin=5, ndraw=15)
bart_boston.fit(X_train, y_train)
```

Out[33]: BART(burnin=5, ndraw=15, random_state=0)

在这个数据集上，通过这样的测试和训练划分，我们看到BART的测试误差与随机森林的测试误差相似。

In [34]:
```
yhat_test = bart_boston.predict(X_test.astype(np.float32))
np.mean((y_test - yhat_test)**2)
```

Out[34]: 20.92

我们可以检查每个变量在树集合中出现的次数。这给出了类似于提升和随机森林的变量重要性图的摘要。

```
In [35]: var_inclusion = pd.Series(bart_boston.variable_inclusion_.mean(0),
                                   index=D.columns)
         var_inclusion
```

```
Out[35]:    crim     25.333333
              zn     27.000000
           indus     21.266667
            chas     20.466667
             nox     25.400000
              rm     32.400000
             age     26.133333
             dis     25.666667
             rad     24.666667
             tax     23.933333
         ptratio     25.000000
           lstat     31.866667
         dtype: float64
```

8.4 练习

Conceptual

1. 绘制一个（你自己发明的）可能由递归二叉分裂产生的二维特征空间划分示例。你的示例应至少包含六个区域。绘制与这个划分相对应的决策树。确保标注图形的所有方面，包括区域 $R_1, R_2, \ldots$、切点 $t_1, t_2, \ldots$ 等。

Hint: Your result should look something like Figures 8.1 and 8.2.

2. 在第8.2.3节中提到，使用深度为一的树（或*stumps*）进行提升会导致一个*additive*模型：即形式为

$$f(X) = \sum_{j=1}^{p} f_j(X_j).$$

解释为什么会这样。你可以从算法8.2中的(8.12)开始。

3. 考虑在一个简单的二分类问题中的基尼指数、分类误差和熵。创建一个单独的图，将这些量每个都表示为 $\hat{p}_{m1}$ 的函数。x 轴应显示 $\hat{p}_{m1}$，范围从 0 到 1，而 y 轴应显示基尼指数、分类误差和熵的值。

Hint: In a setting with two classes, $\hat{p}_{m}1 = 1 - \hat{p}_{m2}$. You could make this plot by hand, but it will be much easier to make in `R`.

4. 此问题与图8.14中的图表有关。

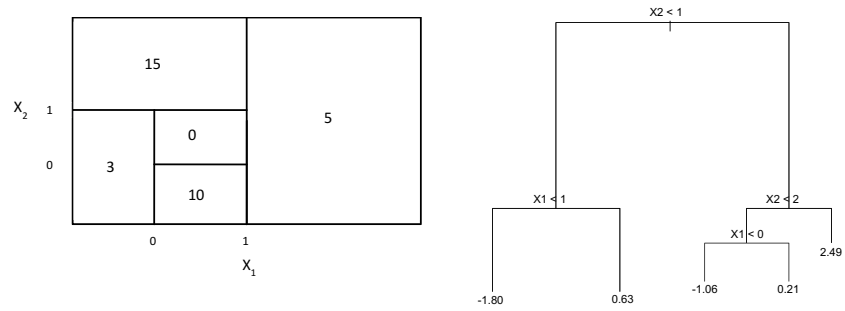


图8.14。*左: A partition of the predictor space corresponding to Exer- cise 4a.* 右 *: A tree corresponding to Exercise 4b.*

((a)) 绘制与图8.14左侧面板所示的预测变量空间划分相对应的树。框内的数字表示每个区域内Y的均值。(b) 使用同一图右侧面板所示的树，创建一个类似于图8.14左侧面板的图。你应该将预测变量空间划分为正确的区域，并指出每个区域的均值。

5. 假设我们从一个包含红色和绿色类别的数据集中生成十个自助采样样本。然后，我们对每个自助采样样本应用分类树，并针对X的一个特定值，产生10个P(类别为红色$|X$)的估计：

$$0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7, \text{ and } 0.75.$$

有两种常见的方法将这些结果合并为一个类别预测。一种是本章讨论的多数投票法。第二种方法是根据平均概率进行分类。在这个例子中，这两种方法下的最终分类分别是什么?

6. 详细解释用于拟合回归树的算法。

Applied

7. 在8.3.3节中，我们应用随机森林于`Boston`数据集，分别使用了`max_features = 6`以及`n_estimators = 100`和`n_estimators =500`。请创建一个图表，展示在该数据集上，对于更广泛的`max_features`和`n_estimators`值范围，随机森林所产生的测试误差。您可以参照图8.10来设计您的图表。描述所得到的结果。

8. 在实验室中，将`Carseats`数据集应用于分类树，在此之前已将`Sales`转换为定性响应变量。现在，我们将使用回归树及相关方法，将响应视为定量变量，来预测`Sales`。

((a)) 将数据集分为训练集和测试集。(b) 对训练集拟合一个回归树。绘制该树，并解释结果。你得到的测试MSE是多少？(c) 使用交叉验证来确定树复杂度的最优水平。剪枝树是否改善了测试MSE？(d) 使用袋装法来分析这些数据。你得到的测试MSE是多少？使用`feature_importance_`值来确定哪些变量最重要。(e) 使用随机森林来分析这些数据。你得到的测试MSE是多少？使用`feature_importance_`值来确定哪些变量最重要。描述m（每次分割时考虑的变量数量）对所得错误率的影响。(f) 现在使用BART分析数据，并报告你的结果。

9. 此问题涉及 `OJ` 数据集，该数据集是 `ISLP` 包的一部分。

(a) 创建一个包含800个随机观测样本的训练集，以及一个包含剩余观测的测试集。(b) 使用`Purchase`作为响应变量，其他变量作为预测变量，对训练数据拟合一棵树。训练误差率是多少？(c) 绘制树的图形，并解释结果。树有多少个终端节点？(d) 使用`export_tree()`函数生成拟合树的文本摘要。选择一个终端节点，并解释所显示的信息。(e) 对测试数据进行响应预测，并生成一个混淆矩阵，比较测试标签与预测的测试标签。测试误差率是多少？(f) 在训练集上使用交叉验证，以确定最优树大小。(g) 生成一个图，x轴为树大小，y轴为交叉验证分类误差率。(h) 哪个树大小对应最低的交叉验证分类误差率？(i) 生成一棵与通过交叉验证获得的最优树大小对应的剪枝树。如果交叉验证未导致选择剪枝树，则创建一棵有五个终端节点的剪枝树。(j) 比较剪枝树和未剪枝树的训练误差率。哪个更高？(k) 比较剪枝树和未剪枝树的测试误差率。哪个更高？

10. W 我们现在使用提升来预测Hitters数据中的Salary ta 集合。

(a) 删除薪资信息未知的观测，然后对薪资进行对数变换。(b) 创建一个由前200个观测组成的训练集，以及一个由剩余观测组成的测试集。(c) 对训练集使用1,000棵树进行提升，针对一系列收缩参数λ的值。绘制一张图，x轴显示不同的收缩值，y轴显示相应的训练集MSE。(d) 绘制一张图，x轴显示不同的收缩值，y轴显示相应的测试集MSE。(e) 将提升的测试MSE与第3章和第6章中看到的两种回归方法的测试MSE进行比较。(f) 在提升模型中，哪些变量似乎是最重要的预测因子？(g) 现在对训练集应用装袋法。这种方法的测试集MSE是多少?

11. 本题使用 Caravan 数据集。

(a) 创建一个由前1,000个观测值组成的训练集，以及由剩余观测值组成的测试集。 (b) 对训练集拟合一个提升模型，以Purchase作为响应变量，其他变量作为预测变量。使用1,000棵树，收缩值为0.01。哪些预测变量显得最重要？ (c) 使用提升模型对测试数据进行响应预测。如果购买估计概率大于20%，则预测该人会购买。形成一个混淆矩阵。预测会购买的人中，实际购买的比例是多少？这与应用KNN或逻辑回归到该数据集得到的结果相比如何?

12. 将提升法、装袋法、随机森林和BART应用于您选择的数据集。确保在训练集上拟合模型，并在测试集上评估其性能。与线性回归或逻辑回归等简单方法相比，这些结果的准确性如何？这些方法中哪一种表现最佳?

9
支持向量机



在本章中，我们讨论*support vector machine* (支持向量机)，这是一种分类方法，于20世纪90年代在计算机科学界被开发出来，并自此之后日益流行。支持向量机已被证明在多种场景下表现良好，且常被视为最佳的“开箱即用”分类器之一。

支持向量机是一种简单直观的分类器的泛化，该分类器称为*maximal margin classifier*，我们在第9.1节中介绍。尽管它优雅且简单，但我们会看到，这个分类器不幸的是不能应用于大多数数据集，因为它要求类别可以通过线性边界分离。在第9.2节中，我们介绍*support vector classifier*，它是最大间隔分类器的扩展，可以应用于更广泛的情况。第9.3节介绍*support vector machine*，它是支持向量分类器的进一步扩展，以适应非线性类别边界。支持向量机适用于二分类设置，其中有两个类别；在第9.4节中，我们讨论支持向量机扩展到多于两个类别的情况。在第9.5节中，我们讨论支持向量机与其他统计方法（如逻辑回归）之间的密切联系。

人们常常笼统地将最大间隔分类器、支持向量分类器以及支持向量机统称为“支持向量机”。为了避免混淆，在本章中我们将仔细区分这三个概念。

9.1 最大间隔分类器

在本节中，我们将定义超平面，并介绍最优 separating 超平面的概念。

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_9

9.1.1 What Is a Hyperplane?

在p维空间中，*hyperplane*是
维度 $p-1$.[1] 例如，在二维空间中，超平面是一维的平坦子空间——换句话说，是一条直线。在三维空间中，超平面是二维的平坦子空间——即一个平面。在 $p>3$ 维空间中，超平面可能难以可视化，但 $(p-1)$ 维平坦子空间的概念仍然适用。

超平面

超平面的数学定义相当简单。在二维空间中，超平面由方程定义

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0 \tag{9.1}$$

对于参数β_0, β_1和β_2。当我们说(9.1)“定义”了超平面时，我们的意思是任何满足(9.1)的$X = (X_1, X_2)^T$都是超平面上的一个点。注意，(9.1)实际上是直线的方程，因为在二维中，超平面确实就是一条直线。

方程 9.1 可以轻松扩展到 p 维设定：

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = 0 \tag{9.2}$$

定义了一个 p 维超平面，同样地，如果 p 维空间中的一个点 $X = (X_1, X_2, \ldots, X_p)^T$（即长度为 p 的向量）满足 (9.2)，那么 X 就位于该超平面上。

现在，假设 X 不满足(9.2)；而是，

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p > 0. \tag{9.3}$$

然后这告诉我们 X 位于超平面的一侧。另一方面，如果

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p < 0, \tag{9.4}$$

那么 X 位于超平面的另一侧。因此，我们可以将超平面视为将 p 维空间分为两半。通过简单地计算（9.2）式左侧的符号，即可轻松确定一个点位于超平面的哪一侧。二维空间中的超平面如图9.1所示。

9.1.2 Classification Using a Separating Hyperplane

现在假设我们有一个$n \times p$数据矩阵$\mathbf{X}$，它由p维空间中的n个训练观测组成，

$$x_1 = \begin{pmatrix} x_{11} \\ \vdots \\ x_{1p} \end{pmatrix}, \ldots, x_n = \begin{pmatrix} x_{n1} \\ \vdots \\ x_{np} \end{pmatrix}, \tag{9.5}$$

并且这些观测值分为两类——即，$y_1, \ldots, y_n \in \{-1, 1\}$，其中$-1$代表一类，1代表另一类。我们还有一个

[1] The word *affine* indicates that the subspace need not pass through the origin.

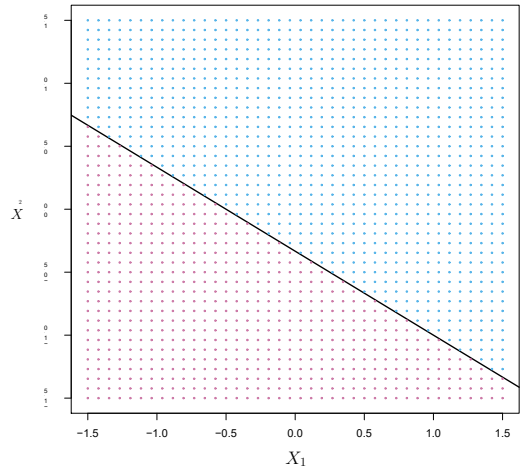


图 9.1. *The hyperplane* $1 + 2X_1 + 3X_2 = 0$ *is shown. The blue region is the set of points for which* $1 + 2X_1 + 3X_2 > 0$*, and the purple region is the set of points for which* $1 + 2X_1 + 3X_2 < 0$.

测试观测，一个观测特征$x^* = \begin{pmatrix} x_1^* & \dots & x_p^* \end{pmatrix}^T$的$p$向量。我们的目标是基于训练数据开发一个分类器，该分类器将利用测试观测的特征测量值对其进行正确分类。我们已经看到了多种针对此任务的方法，例如第4章中的线性判别分析和逻辑回归，以及第8章中的分类树、装袋法和提升法。我们现在将看到一种基于*separating hyperplane*概念的新方法。

separating hyperplane

假设可以构造一个超平面，根据训练观测的类别标签将其完美分离。图9.2左侧面板展示了三个这样的*separating hyperplanes*示例。我们可以将蓝色类别的观测标记为$y_i = 1$，紫色类别的观测标记为$y_i = -1$。那么，一个分离超平面具有这样的性质：

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} > 0 \text{ if } y_i = 1, \tag{9.6}$$

和

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} < 0 \text{ if } y_i = -1. \tag{9.7}$$

等价地，分离超平面具有以下性质：

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) > 0 \tag{9.8}$$

对于所有 $i = 1, \dots, n$。

如果存在分离超平面，我们可以利用它构建一个非常自然的分类器：根据测试观测点位于超平面的哪一侧，将其分配到相应的类别。图9.2右侧面板展示了这种分类器的一个例子。也就是说，我们根据$f(x^*) = \beta_0 + \beta_1 x_1^* + \beta_2 x_2^* + \cdots + \beta_p x_p^*$的符号来对测试观测点$x^*$进行分类。如果$f(x^*)$为正，则将测试观测点归为类别1；如果$f(x^*)$为负，则将其归为类别$-1$。我们还可以利用$f(x^*)$的*magnitude*。如果

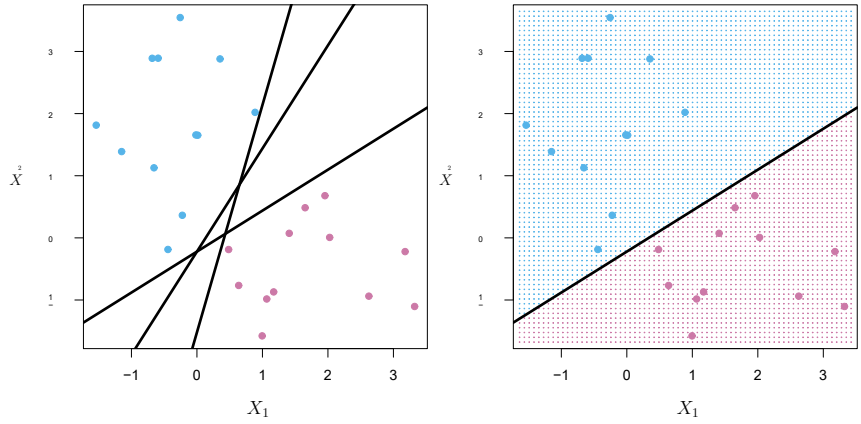


图9.2。左：*There are two classes of observations, shown in blue and in purple, each of which has measurements on two variables. Three separating hyperplanes, out of many possible, are shown in black.* 右：*A separating hyperplane is shown in black. The blue and purple grid indicates the decision rule made by a classifier based on this separating hyperplane: a test observation that falls in the blue portion of the grid will be assigned to the blue class, and a test observation that falls into the purple portion of the grid will be assigned to the purple class.*

$f(x^*)$ 远离零，则意味着x^*远离超平面，因此我们对x^*的类别分配有信心。相反，若$f(x^*)$接近零，则x^*位于超平面附近，故对x^*的类别分配信心较低。不出所料，如图9.2所示，基于分离超平面的分类器导致线性决策边界。

9.1.3 The Maximal Margin Classifier

一般来说，如果我们的数据能够通过一个超平面完美分离，那么实际上会存在无数个这样的超平面。这是因为给定的分离超平面通常可以上下微移或旋转，而不会与任何观测点接触。图9.2左侧面板展示了三个可能的分离超平面。为了基于分离超平面构建分类器，我们必须有一种合理的方式来决定使用无数可能分离超平面中的哪一个。

一个自然的选择是*maximal margin hyperplane* (，也被称为*optimal separating hyperplane*)，即分离超平面，该超平面是距离训练观测值最远的。也就是说，我们可以计算每个训练观测值到给定分离超平面的（垂直）距离；这些距离中最小的一个就是观测值到超平面的最小距离，被称为*margin*。最大间隔超平面是使间隔最大的分离超平面——也就是说，它是距离训练观测值的最小距离最远的超平面。然后，我们可以根据测试观测值位于最大间隔超平面的哪一侧来对其进行分类。这被称为

最大间隔超平面

最优分离超平面

边距

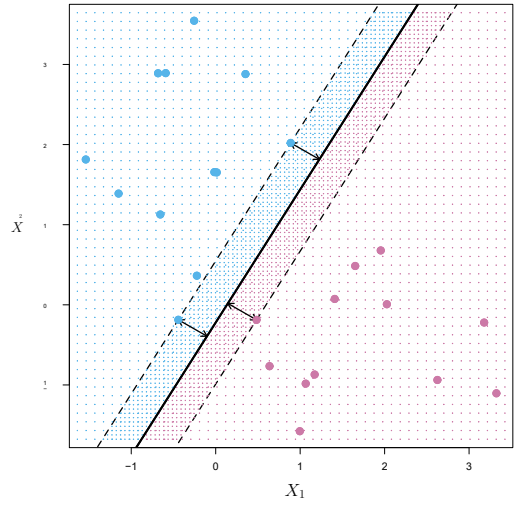


图9.3. *There are two classes of observations, shown in blue and in purple. The maximal margin hyperplane is shown as a solid line. The margin is the distance from the solid line to either of the dashed lines. The two blue points and the purple point that lie on the dashed lines are the support vectors, and the distance from those points to the hyperplane is indicated by arrows. The purple and blue grid indicates the decision rule made by a classifier based on this separating hyperplane.*

作为 *maximal margin classifier*。我们希望一个具有大
训练数据上的间隔在测试数据上也会有很大间隔，因此能够正确分类测试观测值。尽管最大间隔分类器通常很成功，但当 p 很大时，它也可能导致过拟合。

最大间隔分类器

如果 $\beta_0, \beta_1, \ldots, \beta_p$ 是最大间隔超平面的系数，那么最大间隔分类器根据 $f(x^*) = \beta_0 + \beta_1 x_1^* + \beta_2 x_2^* + \cdots + \beta_p x_p^*$ 的符号对测试观测 x^* 进行分类。

图9.3展示了图9.2数据集上的最大间隔超平面。对比图9.2右侧面板与图9.3，我们可以看到图9.3中展示的最大间隔超平面确实使得观测值与分离超平面之间的最小距离增大——即拥有更大的间隔。从某种意义上说，最大间隔超平面代表了我们能在两个类别间插入的最宽的“平板”的中线。

观察图9.3，我们看到有三个训练观测值与最大间隔超平面等距，并位于表示间隔宽度的虚线上。这三个观测值被称为*support vectors*，因为它们是p维空间中的向量（在图9.3中，

$p = 2$) 并且它们“支持”最大间隔超平面，意思 是如果这些点稍微移动，那么最大间隔超平面也会随之移动。有趣的是，最大间隔超平面直接依赖于支持向量，而不依赖于其他观测值：只要其他观测值的移动不导致它

support vector

穿过由间隔所设定的边界。最大间隔超平面仅直接依赖于一小部分观测值这一事实，是一个重要的属性，在讨论支持向量分类器和支持向量机时，这一属性将在本章后续部分显现。

9.1.4 Construction of the Maximal Margin Classifier

我们现在考虑基于一组n训练观测值$x_1, \ldots, x_n \in \mathbb{R}^p$及其关联的类别标签$y_1, \ldots, y_n \in \{-1, 1\}$来构造最大间隔超平面的任务。简而言之，最大间隔超平面是优化问题的解

$$\underset{\beta_0, \beta_1, \ldots, \beta_p, M}{\text{maximize}} \; M \tag{9.9}$$

$$\text{subject to } \sum_{j=1}^{p} \beta_j^2 = 1, \tag{9.10}$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) \geq M \;\; \forall \, i = 1, \ldots, n. \tag{9.11}$$

这个优化问题(9.9)–(9.11)实际上比它看起来要简单。首先，(9.11)中的约束条件

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) \geq M \;\; \forall \, i = 1, \ldots, n$$

保证每个观测点都位于超平面的正确一侧，前提是 M 为正。（实际上，为了使每个观测点都位于超平面的正确一侧，我们只需 $y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) > 0$，因此（9.11）式中的约束实际上要求每个观测点都位于超平面的正确一侧，并且有一定的缓冲，前提是 M 为正。）

其次，请注意(9.10)实际上并不是对超平面的约束，因为如果$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} = 0$定义了一个超平面，那么对于任何$k \neq 0$，$k(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) = 0$也定义了一个超平面。然而，(9.10)为(9.11)增添了意义；可以证明，在这个约束下，第i个观测值到超平面的垂直距离由下式给出：

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}).$$

因此，约束条件(9.10)和(9.11)确保每个观测点都位于超平面的正确一侧，且距离超平面至少为M。因此，M代表了我们的超平面的边距，而优化问题选择$\beta_0, \beta_1, \ldots, \beta_p$以最大化$M$。这正是最大边距超平面的定义！问题(9.9)–(9.11)可以高效求解，但优化的细节超出了本书的范围。

9.1.5 The Non-separable Case

最大间隔分类器是一种非常自然的分类方法，*if a separating hyperplane exists*。然而，正如我们所暗示的，在许多情况下不存在分离超平面，因此没有最大

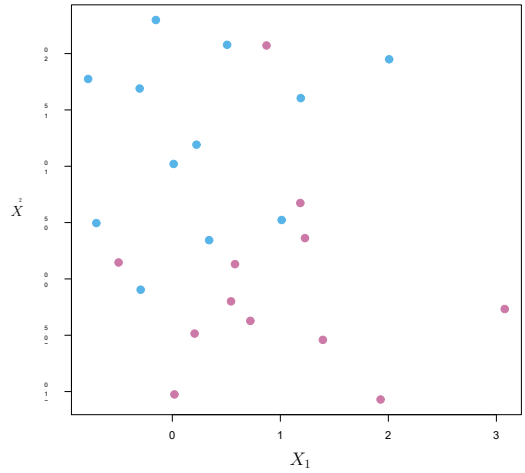


图9.4. *There are two classes of observations, shown in blue and in purple. In this case, the two classes are not separable by a hyperplane, and so the maximal margin classifier cannot be used.*

边缘分类器。在这种情况下，优化问题(9.9)–(9.11)在$M > 0$时无解。图9.4展示了一个例子。此时，我们无法*exactly*将两类分开。然而，正如我们将在下一节看到的，我们可以通过所谓的*almost*来扩展分离超平面的概念，以开发出一个能够*soft margin*分离这些类的超平面。将最大间隔分类器推广到不可分情形，被称为*support vector classifier*。

9.2 支持向量分类器

9.2.1 Overview of the Support Vector Classifier

在图9.4中，我们可以看到属于两个类别的观测值不一定能被一个超平面分开。事实上，即使确实存在一个分离超平面，也有一些情况下，基于分离超平面的分类器可能并不理想。基于分离超平面的分类器必然会完美地分类所有训练观测值；这可能导致对个别观测值的敏感性。图9.5展示了一个例子。在图9.5右侧面板中加入一个单独的观测值会导致最大间隔超平面发生剧烈变化。由此得到的最大间隔超平面并不令人满意——一方面，它的间隔非常小。这是有问题的，因为如前所述，观测值到超平面的距离可以被视为我们对该观测值被正确分类的信心的度量。此外，最大间隔超平面对单个观测值的变化极其敏感，这一事实表明它可能对训练数据过拟合。

在这种情况下，我们可能愿意考虑基于一个超平面的分类器，该超平面能够*not*完美地将两个类别分开，以期

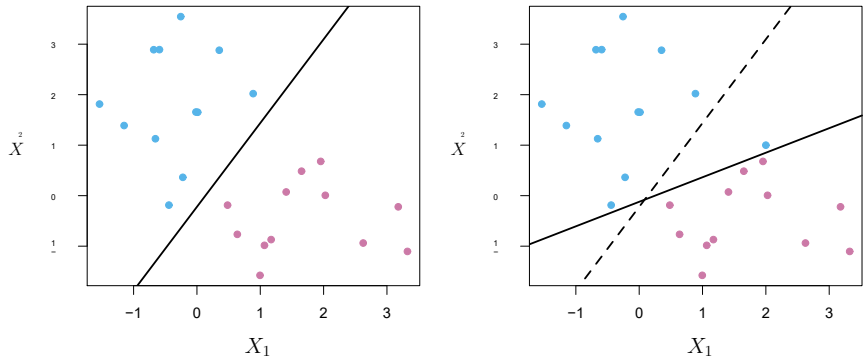


图 9.5. 左：*Two classes of observations are shown in blue and in purple, along with the maximal margin hyperplane.* 右：*An additional blue observation has been added, leading to a dramatic shift in the maximal margin hyperplane shown as a solid line. The dashed line indicates the maximal margin hyperplane that was obtained in the absence of this additional point.*

- 对个体观测的更强稳健性，以及
- 更好地分类训练观测值的*most*。

也就是说，为了对剩余的观察结果进行更好的分类，可能值得将一些训练观察结果错误分类。

支持向量分类器 软间隔分类器

The *support vector classifier*，有时称为 *soft margin classifier*，正是这样做的。我们不是寻求最大可能的间隔，以便每个观察点不仅位于超平面的正确一侧，而且位于间隔的正确一侧，而是允许一些观察点位于间隔的错误一侧，甚至位于超平面的错误一侧。（间隔为*soft*，因为它可能被一些训练观察点违反。）图9.6左侧面板显示了一个例子。大多数观察点位于间隔的正确一侧。然而，一小部分观察点位于间隔的错误一侧。

一个观测值不仅可能位于间隔的错误一侧，还可能位于超平面的错误一侧。事实上，当不存在分离超平面时，这种情况是不可避免的。位于超平面错误一侧的观测值对应于被支持向量分类器错误分类的训练观测值。图9.6右侧的面板展示了这样一种情形。

9.2.2 Details of the Support Vector Classifier

支持向量分类器根据测试观测点位于超平面的哪一侧对其进行分类。超平面的选择旨在将大多数训练观测点正确分为两类，但可能

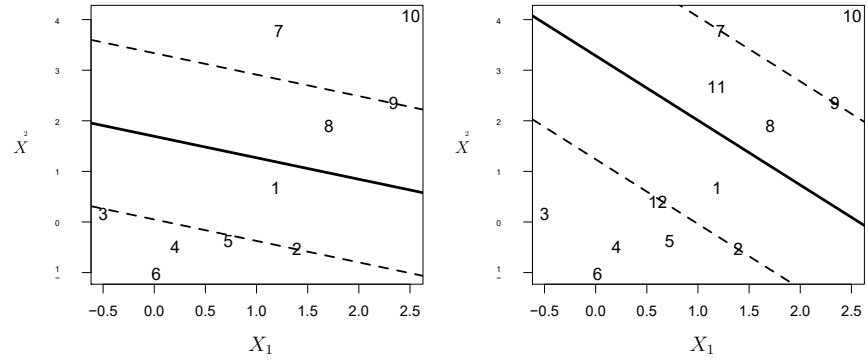


图9.6。左：*A support vector classifier was fit to a small data set. The hyperplane is shown as a solid line and the margins are shown as dashed lines.* 紫色观测值：*Observations* 3, 4, 5, *and* 6 *are on the correct side of the margin, observation* 2 *is on the margin, and observation 1 is on the wrong side of the margin.* 蓝色观测值：*Observations* 7 *and* 10 *are on the correct side of the margin, observation* 9 *is on the margin, and observation* 8 *is on the wrong side of the margin. No observations are on the wrong side of the hyperplane.* 右：*Same as left panel with two additional points,* 11 *and* 12*. These two observations are on the wrong side of the hyperplane and the wrong side of the margin.*

错误分类 一些观察。它是优化问题p的解 问题

$$\underset{\beta_0,\beta_1,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n,M}{\text{maximize}} \quad M \tag{9.12}$$

$$\text{subject to} \quad \sum_{j=1}^{p}\beta_j^2 = 1, \tag{9.13}$$

$$y_i(\beta_0+\beta_1x_{i1}+\beta_2x_{i2}+\cdots+\beta_px_{ip}) \geq M(1-\epsilon_i), \tag{9.14}$$

$$\epsilon_i \geq 0, \ \sum_{i=1}^{n}\epsilon_i \leq C, \tag{9.15}$$

其中 C 是一个非负调优参数。如同(9.11)中那样，M 是边距的宽度；我们力求使这一量尽可能大。在(9.14)中，$\epsilon_1,\ldots,\epsilon_n$ 是 *slack variables*，允许个别观测点位于
边距或超平面的错误一侧；我们稍后将更详细地解释它们。一旦我们解决了(9.12)–(9.15)，我们就像之前一样，通过简单判断测试观察值x^*位于超平面的哪一侧来对其进行分类。也就是说，我们根据
$f(x^*)=\beta_0+\beta_1x_1^*+\cdots+\beta_px_p^*$的符号对测试观察值进行分类。

slack variable

问题 (9.12)–(9.15) 看似复杂，但通过下面一系列简单的观察可以洞察其行为。首先，松弛变量 ϵ_i 告诉我们第 i 个观测值相对于超平面和间隔的位置。如果 $\epsilon_i=0$，那么如我们在第 9.1.4 节所见，第 i 个观测值位于间隔的正确一侧。如果 $\epsilon_i>0$，那么第 i 个观测值位于间隔的错误一侧，我们说第 i 个观测值 *violated* 了间隔。如果 $\epsilon_i>1$，则它位于超平面的错误一侧。

我们现在考虑调优参数 C 的作用。在 (9.15) 中，C 限制了 ϵ_i 的总和，因此它决定了我们将容忍的边界（以及超平面）违反的次数和严重程度。我们可以将 C 视为 n 个观测值可以违反边界的量的 $budget$。如果 $C = 0$，则没有违反边界的预算，并且必须有 $\epsilon_1 = \cdots = \epsilon_n = 0$，在这种情况下，(9.12)–(9.15) 就简化为最大边界超平面优化问题 (9.9)–(9.11)。（当然，最大边界超平面仅在两个类别可分时才存在。）对于 $C > 0$，最多有 C 个观测值可以位于超平面的错误一侧，因为如果一个观测值位于超平面的错误一侧，则 $\epsilon_i > 1$，并且 (9.15) 要求 $\sum_{i=1}^{n} \epsilon_i \leq C$。随着预算 C 的增加，我们对边界违反变得更加容忍，因此边界将变宽。相反，随着 C 的减小，我们对边界违反变得不那么容忍，因此边界变窄。图 9.7 中展示了一个例子。

在实践中，C被视为一种调优参数，通常通过交叉验证来选择。正如我们在本书中见到的调优参数一样，C控制着统计学习技术的偏差-方差权衡。当C较小时，我们寻求很少被违反的窄间隔；这相当于一个高度拟合数据的分类器，可能具有低偏差但高方差。另一方面，当C较大时，间隔更宽，我们允许更多违反；这相当于对数据拟合得不那么严格，得到一个可能偏差更大但方差可能更低的分类器。

优化问题(9.12)–(9.15)具有一个非常有趣的性质：事实证明，只有位于边缘上或违反边缘的观测值才会影响超平面，从而影响所获得的分类器。换句话说，严格位于边缘正确一侧的观测值不会影响支持向量分类器！只要该观测值的位置保持在边缘的正确一侧，改变其位置根本不会改变分类器。直接位于边缘上或位于其类别边缘错误一侧的观测值被称为 $support\ vectors$。这些观测值确实会影响支持向量分类器。

支持向量分类器仅受支持向量影响这一事实，与我们先前关于C控制支持向量分类器偏差-方差权衡的论断一致。当调优参数C较大时，边距较宽，许多观测值违反边距，因此存在许多支持向量。在这种情况下，许多观测值参与确定超平面。图9.7左上角的面板展示了这一设定：该分类器具有低方差（因为许多观测值是支持向量），但可能具有高偏差。相反，若C较小，则支持向量较少，因此所得分类器将具有低偏差但高方差。图9.7右下角的面板展示了这一设定，仅有八个支持向量。

支持向量分类器的决策规则仅基于训练观测中一个可能很小的子集（即支持向量），这意味着它对远离超平面的观测行为相当稳健。这一特性与一些

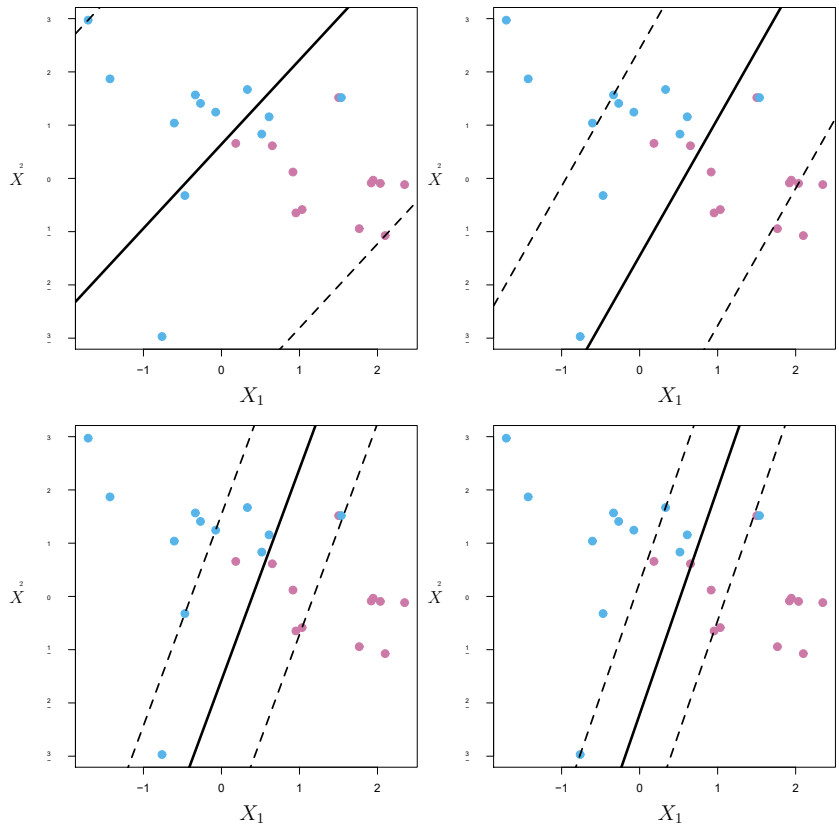


图9.7. *A support vector classifier was fit using four different values of the tuning parameter C in (9.12)–(9.15). The largest value of C was used in the top left panel, and smaller values were used in the top right, bottom left, and bottom right panels. When C is large, then there is a high tolerance for observations being on the wrong side of the margin, and so the margin will be large. As C decreases, the tolerance for observations being on the wrong side of the margin decreases, and the margin narrows.*

我们在前面章节中见过的其他分类方法，如线性判别分析。回想一下，LDA分类规则依赖于每个类内观测值的均值*all*，以及使用观测值的*all*计算的类内协方差矩阵。相比之下，逻辑回归与LDA不同，对远离决策边界的观测值具有非常低的敏感性。事实上，我们将在9.5节看到支持向量分类器和逻辑回归密切相关。

9.3 支持向量机

我们首先讨论将线性分类器转换为产生非线性决策边界的一般机制。然后我们介绍支持向量机，它以自动方式实现这一点。

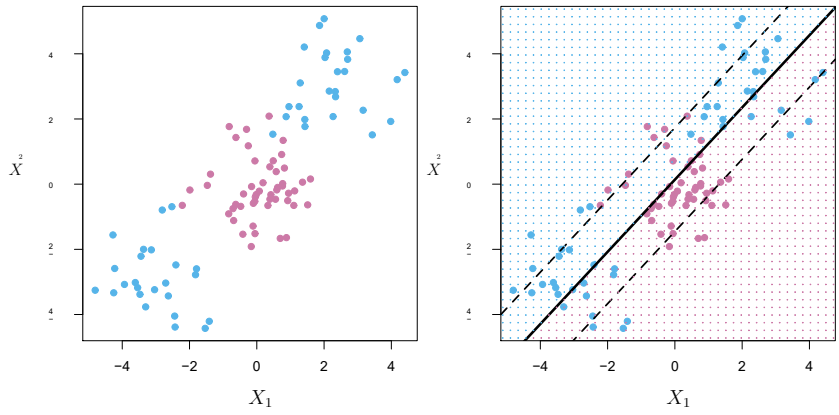


图9.8。左：*The observations fall into two classes, with a non-linear boundary between them.* 右：*The support vector classifier seeks a linear boundary, and consequently performs very poorly.*

9.3.1 Classification with Non-Linear Decision Boundaries

支持向量分类器是在二分类设置中一种自然的分类方法，前提是两类之间的边界是线性的。然而，在实践中，我们有时会遇到非线性的类别边界。例如，考虑图9.8左侧面板中的数据。显然，支持向量分类器或任何线性分类器在此处的表现都会很差。实际上，图9.8右侧面板中显示的支持向量分类器在这里毫无用处。

在第7章中，我们面临一个类似的情况。我们观察到，当预测变量与结果之间存在非线性关系时，线性回归的性能可能会受到影响。在这种情况下，我们考虑通过预测变量的函数，如二次项和三次项，来扩大特征空间，以解决这种非线性问题。对于支持向量分类器，我们可以采用类似的方法来解决类别之间可能存在的非线性边界问题，即通过使用预测变量的二次、三次乃至更高阶多项式函数来扩大特征空间。例如，不是使用p个特征来拟合支持向量分类器，

$$X_1, X_2, \ldots, X_p,$$

我们可以改为使用$2p$个特征来拟合支持向量分类器

$$X_1, X_1^2, X_2, X_2^2, \ldots, X_p, X_p^2.$$

然后(9.12)–(9.15)将变为

$$\underset{\beta_0,\beta_{11},\beta_{12},\ldots,\beta_{p1},\beta_{p2},\epsilon_1,\ldots,\epsilon_n,M}{\text{maximize}} \quad M \tag{9.16}$$

$$\text{subject to } y_i\left(\beta_0+\sum_{j=1}^{p}\beta_{j1}x_{ij}+\sum_{j=1}^{p}\beta_{j2}x_{ij}^2\right)\geq M(1-\epsilon_i),$$

$$\sum_{i=1}^{n}\epsilon_i\leq C,\ \ \epsilon_i\geq 0,\ \ \sum_{j=1}^{p}\sum_{k=1}^{2}\beta_{jk}^2=1.$$

为什么这会导致非线性的决策边界？在扩大的特征空间中，由(9.16)得出的决策边界实际上是线性的。但在原始特征空间中，决策边界的形式为$q(x)=0$，其中q是二次多项式，其解通常是非线性的。人们可能还希望用更高阶的多项式项，或者形如$X_jX_{j'}$对于$j\neq j'$的交互项来扩大特征空间。或者，可以考虑预测变量的其他函数而非多项式。不难看出，扩大特征空间有许多可能的方式，而且除非我们小心谨慎，否则可能会得到数量庞大的特征，进而导致计算变得难以处理。接下来介绍的支持向量机，允许我们以一种能够高效计算的方式，扩大支持向量分类器所使用的特征空间。

9.3.2 The Support Vector Machine

support vector machine (支持向量机)是支持向量的扩展通过以特定方式扩大特征空间而得到的分类器，使用了*kernels*。我们现在将讨论这个扩展，其细节是这有点复杂，超出了本书的范围。然而，主要思想在第9.3.1节中描述：我们可能希望扩大特征空间，以便容纳类别之间的非线性边界。我们这里描述的核心方法只是实现这一想法的一种有效计算方法。

support vector machine kernel

我们尚未确切讨论支持向量分类器是如何计算的，因为细节变得有些技术性。然而，事实证明，支持向量分类器问题（9.12）–（9.15）的解仅涉及观测值的内积（而非观测值本身）。两个{13}维向量{14}和{15}的内积定义为{16}。因此，两个观测值{17}、{18}的内积由下式给出：

$$\langle x_i,x_{i'}\rangle=\sum_{j=1}^{p}x_{ij}x_{i'j}. \tag{9.17}$$

可以证明

- 线性支持向量分类器可以表示为

$$f(x)=\beta_0+\sum_{i=1}^{n}\alpha_i\langle x,x_i\rangle, \tag{9.18}$$

其中有 n 个参数 α_i，$i = 1, \ldots, n$，每个训练观测对应一个。

- 为了估计参数$\alpha_1, \ldots, \alpha_n$和$\beta_0$，我们只需所有训练观测对之间的$\binom{n}{2}$内积$\langle x_i, x_{i'} \rangle$。（记号$\binom{n}{2}$表示$n(n-1)/2$，给出了一组$n$个项目中成对的数量。）

请注意，在(9.18)中，为了评估函数$f(x)$，我们需要计算新点x与每个训练点x_i之间的内积。然而，事实证明，α_i仅在解中的支持向量处非零——也就是说，如果一个训练观测不是支持向量，那么它的α_i等于零。因此，如果$\mathcal{S}$是这些支持点索引的集合，我们可以将形式为(9.18)的任何解函数重写为

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle, \tag{9.19}$$

这通常涉及的项数远少于(9.18)中的项数。[2]

总之，在表示线性分类器 $f(x)$ 以及计算其系数时，我们需要的仅是内积。

现在假设每次内积(9.17)在表示式(9.18)中出现，或在求解支持向量分类器解的计算中，我们将其替换为形式为内积的*generalization*

$$K(x_i, x_{i'}), \tag{9.20}$$

其中 K 是某个函数，我们将其称为 *kernel*。核就是核
量化两个观测值相似度的函数。例如，我们可以简单地采用 l

$$K(x_i, x_{i'}) = \sum_{j=1}^{p} x_{ij} x_{i'j}, \tag{9.21}$$

这只会让我们回到支持向量分类器。方程9.21被称为*linear*核，因为支持向量分类器在特征上是线性的；线性核本质上使用皮尔逊（标准）相关系数来量化一对观测值的相似性。但也可以为(9.20)选择另一种形式。例如，可以将每个$\sum_{j=1}^{p} x_{ij} x_{i'j}$实例替换为量

$$K(x_i, x_{i'}) = (1 + \sum_{j=1}^{p} x_{ij} x_{i'j})^d. \tag{9.22}$$

这被称为度数为d的*polynomial kernel*，其中d为正数
整数。在支持向量分类器算法中使用这样一个核，其中$d > 1$，而不是标准线性核（9.21），会导致一个更加灵活的决策边界。这实质上相当于拟合一个支持向量 polynomial kernel

[2]By expanding each of the inner products in (9.19), it is easy to see that $f(x)$ is a linear function of the coordinates of x. Doing so also establishes the correspondence between the α_i and the original parameters β_j.

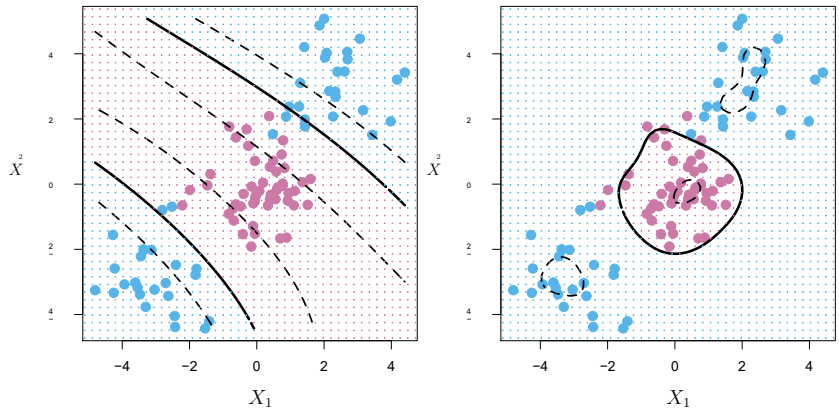


图 9.9. 左：*An SVM with a polynomial kernel of degree 3 is applied to the non-linear data from Figure 9.8, resulting in a far more appropriate decision rule.* 右：*An SVM with a radial kernel is applied. In this example, either kernel is capable of capturing the decision boundary.*

涉及次数为 d 的多项式的更高维空间中的分类器，而非原始特征空间。当支持向量分类器与诸如 (9.22) 的非线性核结合时，所得到的分类器被称为支持向量机。注意在这种情况下，(非线性) 函数的形式为

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i). \tag{9.23}$$

图 9.9 左侧面板展示了一个应用多项式核的 SVM 对图 9.8 中非线性数据的示例。该拟合相较于线性支持向量分类器有显著改进。当 $d = 1$ 时，SVM 退化为本章前面介绍的支持向量分类器。

(9.22) 中展示的多项式核是一种可能的非线性核的示例，但替代方案众多。另一个流行的选择是 *radial kernel*，其形式为

radial kernel

$$K(x_i, x_{i'}) = \exp(-\gamma \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2). \tag{9.24}$$

在(9.24)中，γ是一个正常数。图9.9右侧面板展示了在该非线性数据上使用径向核的SVM示例；它同样出色地完成了两类数据的分离。

径向核（9.24）实际上是如何工作的？如果给定的测试观测 $x^* = (x_1^*, \ldots, x_p^*)^T$ 与训练观测 x_i 在欧几里得距离上相距很远，那么 $\sum_{j=1}^{p} (x_j^* - x_{ij})^2$ 将会很大，因此 $K(x^*, x_i) = \exp(-\gamma \sum_{j=1}^{p} (x_j^* - x_{ij})^2)$ 将会很小。这意味着在（9.23）中，x_i 在 $f(x^*)$ 中几乎不会起作用。回想一下，测试观测 x^* 的预测类别标签是基于 $f(x^*)$ 的符号。换句话说，远离 x^* 的训练观测在 x^* 的预测类别标签中基本上不会发挥作用。这意味着径向核具有非常 *local*

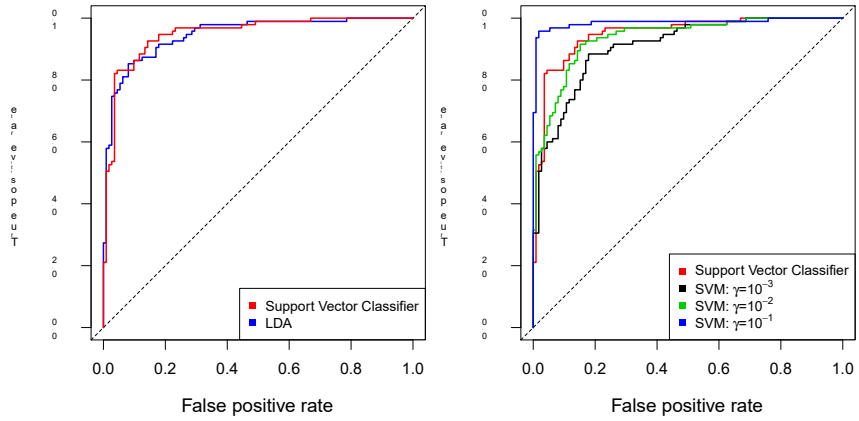


图 9.10。*ROC curves for the* `Heart` *data training set.* 左图：*The support vector classifier and LDA are compared.* 右图：*The support vector classifier is compared to an SVM using a radial basis kernel with* $\gamma = 10^{-3}, 10^{-2}, \text{ and } 10^{-1}$.

行为，即只有附近的训练观测值对测试观测值的类别标签有影响。

使用核函数而不是简单地使用原始特征的函数来扩大特征空间（如(9.16)中那样）有什么优势？一个优势是计算上的，它归结为这样一个事实：使用核函数，我们只需对所有$\binom{n}{2}$个不同的对$i,\ i'$计算$K(x_i, x_{i'})$。这可以在不显式地在扩大的特征空间中工作的情况下完成。这一点很重要，因为在SVM的许多应用中，扩大的特征空间太大，以至于计算难以处理。对于某些核函数，如径向核(9.24)，特征空间是*implicit*且无限维的，所以我们无论如何都无法在那里进行计算！

9.3.3 An Application to the Heart Disease Data

在第8章中，我们将决策树及相关方法应用于`Heart`数据。目标是利用如`Age`、`Sex`和`Chol`等13个预测变量来预测个体是否患有心脏病。我们现在探究支持向量机(SVM)与线性判别分析(LDA)在此数据上的对比效果。剔除6个缺失观测后，数据集包含297名受试者，我们随机将其分为207个训练样本和90个测试样本。

我们首先将LDA和支持向量分类器拟合到训练数据上。注意，支持向量分类器等价于使用次数为d =1的多项式核的SVM。图9.10左侧面板显示了LDA和支持向量分类器在训练集预测上的ROC曲线（在第4.4.2节中描述）。两个分类器都为每个观测计算形式为$\hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2 + \cdots + \hat{\beta}_p X_p$的得分。对于任意给定的截断值$t$，我们根据$\hat{f}(X) < t$或$\hat{f}(X) \geq t$将观测分类为*heart disease*或*no heart disease*类别。通过形成这些预测并计算一系列t值下的假阳性率和真阳性率，得到ROC曲线。最优分类器将紧贴ROC图的左上角。在此情况下

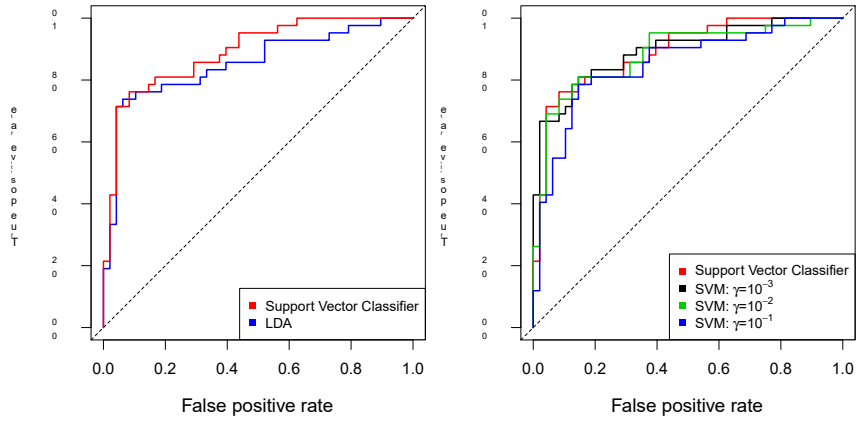


图 9.11. *ROC curves for the test set of the* `Heart` *data.* 左：*The support vector classifier and LDA are compared.* 右：*The support vector classifier is compared to an SVM using a radial basis kernel with* $\gamma = 10^{-3}, 10^{-2}, \text{ and } 10^{-1}$.

LDA和支持向量分类器都表现良好，尽管有迹象表明支持向量分类器可能略胜一筹。

图9.10的右侧面板展示了使用径向核的支持向量机在不同γ值下的ROC曲线。随着γ的增加，拟合变得更加非线性，ROC曲线有所改善。使用$\gamma = 10^{-1}$似乎给出了近乎完美的ROC曲线。然而，这些曲线代表的是训练错误率，就新测试数据上的表现而言，这可能具有误导性。图9.11显示了在90个测试观察值上计算的ROC曲线。我们观察到与训练ROC曲线的一些差异。在图9.11的左侧面板中，支持向量分类器似乎比LDA略有优势（尽管这些差异在统计上并不显著）。在右侧面板中，使用$\gamma = 10^{-1}$的支持向量机，虽然在训练数据上表现最佳，但在测试数据上却产生了最差的估计结果。这再次证明了，虽然更灵活的方法往往能产生更低的训练错误率，但这并不必然导致在测试数据上的性能提升。使用$\gamma = 10^{-2}$和$\gamma = 10^{-3}$的支持向量机与支持向量分类器表现相当，而这三者都优于使用$\gamma = 10^{-1}$的支持向量机。

9.4 多类别支持向量机

到目前为止，我们的讨论仅限于二分类的情况：即两类别设置下的分类。我们如何将SVM扩展到更一般的情况，即存在任意数量类别的情形？事实证明，SVM所基于的分离超平面的概念并不自然地适用于超过两个类别。尽管已提出了多种将SVM扩展到K类情况的方法，其中最流行的两种是*one-versus-one*和*one-versus-all*方法。我们在此简要讨论这两种方法。

9.4.1 One-Versus-One Classification

假设我们想使用支持向量机进行分类，并且有K >个2类别。*one-versus-one*或*all-pairs*方法构建$\binom{K}{2}$个一对一支持向量机，每个支持向量机比较一对类别。例如，一个这样的支持向量机可能比较第k类（编码为+1）与第k'类（编码为−1）。我们使用$\binom{K}{2}$个分类器对测试观测进行分类，并记录测试观测被分配到K个类别中每个类别的次数。最终分类是通过将测试观测分配到在这$\binom{K}{2}$个成对分类中最频繁分配到的类别来完成的。

9.4.2 One-Versus-All Classification

one-versus-all 方法（也称为 *one-versus-rest*）是一种用于在 $K > 2$ 类情况下应用 SVM 的替代性一对一对其余方法。我们拟合 K 个 SVM，每次将 K 个类别中的一个与剩余的 $K-1$ 个类别进行比较。令 $\beta_{0k}, \beta_{1k}, \ldots, \beta_{pk}$ 表示拟合一个将第 k 个类别（编码为 +1）与其他类别（编码为 −1）进行比较的 SVM 所得到的参数。令 x^* 表示一个测试观测。我们将该观测分配给 $\beta_{0k} + \beta_{1k}x_1^* + \beta_{2k}x_2^* + \cdots + \beta_{pk}x_p^*$ 最大的类别，因为这相当于对测试观测属于第 k 个类别而非任何其他类别具有高水平的置信度。

9.5 与逻辑回归的关系



支持向量机（SVMs）在20世纪90年代中期首次被引入时，在统计学和机器学习领域引起了相当大的轰动。这部分归功于它们良好的性能、有效的宣传，以及其基础方法既新颖又神秘的特点。寻找一个能够尽可能好地分离数据，同时允许一些违背这种分离的超平面的想法，明显不同于经典的分类方法，如逻辑回归和线性判别分析。此外，使用核函数来扩展特征空间以适应非线性类别边界的想法，似乎是一个独特且有价值的特性。

然而，自那时起，SVM与其他更经典的统计方法之间出现了深刻的联系。事实证明，可以将拟合支持向量分类器$f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$的准则(9.12)–(9.15)重写为

$$\underset{\beta_0,\beta_1,\ldots,\beta_p}{\text{minimize}} \left\{ \sum_{i=1}^{n} \max\left[0, 1 - y_i f(x_i)\right] + \lambda \sum_{j=1}^{p} \beta_j^2 \right\}, \tag{9.25}$$

其中 λ 是一个非负的调优参数。当 λ 很大时，$\beta_1, \ldots, \beta_p$ 很小，对间隔的更多违反会被容忍，从而产生一个低方差但高偏差的分类器。当 λ 很小时，对间隔的违反将会很少；这相当于一个高方差但低偏差

分类器。因此，(9.25) 中 λ 的小值相当于 (9.15) 中 C 的小值。注意，(9.25) 中的 $\lambda \sum_{j=1}^{p} \beta_j^2$ 项是来自第 6.2.1 节的岭惩罚项，并且在控制支持向量分类器的偏差-方差权衡中起着类似的作用。

现在（9.25）呈现出我们在本书中反复见到的“损失 + 惩罚”形式：

$$\underset{\beta_0,\beta_1,\ldots,\beta_p}{\text{minimize}} \left\{ L(\mathbf{X}, \mathbf{y}, \beta) + \lambda P(\beta) \right\}. \tag{9.26}$$

在 (9.26) 中，$L(\mathbf{X}, \mathbf{y}, \beta)$ 是某种损失函数，用于量化由 β 参数化的模型对数据 $(\mathbf{X}, \mathbf{y})$ 的拟合程度，而 $P(\beta)$ 是参数向量 β 上的惩罚函数，其效果由非负调优参数 λ 控制。例如，岭回归和套索回归都采用这种形式，其中

$$L(\mathbf{X}, \mathbf{y}, \beta) = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij} \beta_j \right)^2$$

并用 $P(\beta) = \sum_{j=1}^{p} \beta_j^2$ 表示岭回归，用 $P(\beta) = \sum_{j=1}^{p} |\beta_j|$ 表示套索。在 (9.25) 的情况下，损失函数改为如下形式

$$L(\mathbf{X}, \mathbf{y}, \beta) = \sum_{i=1}^{n} \max \left[0, 1 - y_i (\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}) \right].$$

这被称为*hinge loss*，如图9.12所示。然而，它
事实证明，hinge损失函数与逻辑回归中使用的损失函数密切相关，同样如图9.12所示。 hinge loss

支持向量分类器的一个有趣特点是，仅支持向量在所得分类器中发挥作用；位于间隔正确一侧的观测值不影响分类器。这是因为图9.12所示的损失函数对于满足 $y_i(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}) \geq 1$的观测值恰好为零；这些观测值对应于位于间隔正确一侧的观测值。[3] 相比之下，图9.12所示的逻辑回归损失函数在任何地方都不恰好为零。但对于远离决策边界的观测值，它非常小。由于它们的损失函数相似，逻辑回归和支持向量分类器通常给出非常相似的结果。当类别分离良好时，支持向量机往往比逻辑回归表现更好；在重叠较多的情况下，逻辑回归通常更受青睐。

当支持向量分类器和支持向量机首次被引入时，人们认为(9.15)中的调优参数C是一个不重要的“冗余”参数，可以设置为某个默认值，比如1。然而，支持向量分类器的“损失+惩罚”公式(9.25)表明情况并非如此。调优参数的选择非常重要，它决定了模型对数据欠拟合或过拟合的程度，例如图9.7所示。

[3] With this hinge-loss + penalty representation, the margin corresponds to the value one, and the width of the margin is determined by $\sum \beta_j^2$.

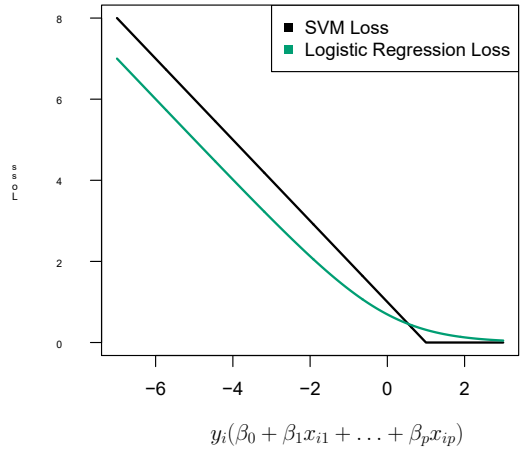


图 9.12. *The SVM and logistic regression loss functions are compared, as a function of* $y_i(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip})$. *When* $y_i(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip})$ *is greater than 1, then the SVM loss is zero, since this corresponds to an observation that is on the correct side of the margin. Overall, the two loss functions have quite similar behavior.*

我们已经确定，支持向量分类器与逻辑回归及其他已有的统计方法密切相关。支持向量机在利用核函数扩展特征空间以适应非线性类别边界方面是否独一无二？这个问题的答案是“否”。我们同样可以使用非线性核函数来执行逻辑回归或本书中见到的许多其他分类方法；这与第7章中见到的一些非线性方法密切相关。然而，由于历史原因，在支持向量机的背景下，非线性核函数的使用远比在逻辑回归或其他方法的背景下更为普遍。

尽管我们在此没有讨论，但实际上存在一种用于回归（即针对定量而非定性响应）的SVM扩展，称为*support vector regression*。在第3章中，我们看到
最小二乘回归寻求系数$\beta_0, \beta_1, \ldots, \beta_p$，使得残差平方和尽可能小。（回顾第3章，残差定义为$y_i - \beta_0 - \beta_1 x_{i1} - \cdots - \beta_p x_{ip}$。）支持向量回归则寻求使不同类型的损失最小化的系数，其中只有绝对值大于某个正常数的残差才会对损失函数有贡献。这是支持向量分类器中使用的间隔在回归设置中的扩展。

support vector regression

9.6 实验：支持向量机

在本实验中，我们使用 sklearn.svm 库来演示支持向量分类器和支持向量机。

我们导入一些常用的库。

In [1]:
```
import numpy as np
from matplotlib.pyplot import subplots, cm
import sklearn.model_selection as skm
from ISLP import load_data, confusion_table
```

We also collect the new imports needed for this lab.

In [2]:
```
from sklearn.svm import SVC
from ISLP.svm import plot as plot_svm
from sklearn.metrics import RocCurveDisplay
```

We will use the function RocCurveDisplay.from_estimator() to produce several ROC plots, using a shorthand roc_curve.

RocCurve Display.from_ estimator()

In [3]:
```
roc_curve = RocCurveDisplay.from_estimator # shorthand
```

9.6.1 Support Vector Classifier

我们现在使用 SupportVectorClassifier() 函数（缩写为 SVC()）从 sklearn 到针对给定参数值 C 拟合支持向量分类器。C 参数允许我们指定违反边距的代价。当 cost 参数较小时，边距会较宽，许多支持向量将位于边距上或违反边距。当 C 参数较大时，边距会较窄，位于边距上或违反边距的支持向量会很少。

SupportVector Classifier()

这里我们演示在二维示例中使用SVC()，以便我们可以绘制得到的决策边界。我们首先生成属于两个类别的观测值，并检查这些类别是否线性可分。

In [4]:
```
rng = np.random.default_rng(1)
X = rng.standard_normal((50, 2))
y = np.array([-1]*25+[1]*25)
X[y==1] += 1
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0],
           X[:,1],
           c=y,
           cmap=cm.coolwarm);
```

它们并非如此。我们现在拟合分类器。

In [5]:
```
svm_linear = SVC(C=10, kernel='linear')
svm_linear.fit(X, y)
```

Out[5]:
```
SVC(C=10, kernel='linear')
```

具有两个特征的支持向量分类器可以通过绘制其 *decision function* 的值来可视化。我们为此包含了一个函数，在ISLP包（受sklearn文档中类似示例的启发）。

decision function

In [6]:
```
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_linear,
         ax=ax)
```

两类之间的决策边界是线性的（因为我们使用了参数 `kernel='linear'`）。支持向量用 + 标记，其余观测值以圆点绘制。

如果我们改用较小的成本参数值呢?

In [7]:
```
svm_linear_small = SVC(C=0.1, kernel='linear')
svm_linear_small.fit(X, y)
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_linear_small,
         ax=ax)
```

当成本参数的值较小时，由于边距变得更宽，我们得到的支持向量数量更多。对于线性核，我们可以提取线性决策边界的系数如下:

In [8]:
```
svm_linear.coef_
```

Out[8]:
```
array([[1.173  , 0.7734]])
```

由于支持向量机是 `sklearn` 中的估计器，我们可以使用常规机制来调优它。

In [9]:
```
kfold = skm.KFold(5,
                  random_state=0,
                  shuffle=True)
grid = skm.GridSearchCV(svm_linear,
                        {'C':[0.001,0.01,0.1,1,5,10,100]},
                        refit=True,
                        cv=kfold,
                        scoring='accuracy')
grid.fit(X, y)
grid.best_params_
```

Out[9]:
```
{'C': 1}
```

我们可以轻松地在 `grid.cv_results_` 中访问每个模型的交叉验证误差。这会打印出大量细节，因此我们仅提取准确率结果。

In [10]:
```
grid.cv_results_[('mean_test_score')]
```

Out[10]:
```
array([0.46, 0.46, 0.72, 0.74, 0.74, 0.74, 0.74])
```

我们发现 `C=1` 产生了最高的交叉验证准确率 0.74，尽管对于若干 `C` 的值，准确率相同。分类器 `grid.best_estimator_` 可用于预测一组测试观测数据的类别标签。让我们生成一个测试数据集。

In [11]:
```
X_test = rng.standard_normal((20, 2))
y_test = np.array([-1]*10+[1]*10)
X_test[y_test==1] += 1
```

Now we predict the class labels of these test observations. Here we use the best model selected by cross-validation in order to make the predictions.

In [12]:
```
best_ = grid.best_estimator_
y_test_hat = best_.predict(X_test)
confusion_table(y_test_hat, y_test)
```

Out[12]:

Truth	-1	1
Predicted		
-1	8	4
1	2	6

Thus, with this value of `C`, 70% of the test observations are correctly classified. What if we had instead used `C=0.001`?

In [13]:
```
svm_ = SVC(C=0.001,
           kernel='linear').fit(X, y)
y_test_hat = svm_.predict(X_test)
confusion_table(y_test_hat, y_test)
```

Out[13]:

Truth	-1	1
Predicted		
-1	2	0
1	8	10

In this case 60% of test observations are correctly classified.

We now consider a situation in which the two classes are linearly separable. Then we can find an optimal separating hyperplane using the `SVC()` estimator. We first further separate the two classes in our simulated data so that they are linearly separable:

In [14]:
```
X[y==1] += 1.9;
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=y, cmap=cm.coolwarm);
```

Now the observations are just barely linearly separable.

In [15]:
```
svm_ = SVC(C=1e5, kernel='linear').fit(X, y)
y_hat = svm_.predict(X)
confusion_table(y_hat, y)
```

Out[15]:

Truth	-1	1
Predicted		
-1	25	0
1	0	25

We fit the support vector classifier and plot the resulting hyperplane, using a very large value of `C` so that no observations are misclassified.

In [16]:
```
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_,
         ax=ax)
```

确实没有产生训练误差，并且仅使用了三个支持向量。实际上，C 的大值也意味着这三个支持点是 *on the margin*，并定义了它。人们可能会好奇，仅依赖于三个数据点的分类器在测试数据上能有多好！我们现在尝试一个更小的 C 值。

In [17]:
```
svm_ = SVC(C=0.1, kernel='linear').fit(X, y)
y_hat = svm_.predict(X)
confusion_table(y_hat, y)
```

Out[17]:

Truth	-1	1
Predicted		
-1	25	0
1	0	25

使用C=0.1，我们再次没有错误分类任何训练观测，但我们也获得了更宽的间隔，并利用了十二个支持向量。这些向量共同决定了决策边界的方向，并且由于它们更多，因此更加稳定。似乎这个模型在测试数据上的表现可能比使用C=1e5 (的模型更好，事实上，一个使用大型测试集的简单实验将会证实这一点)。

In [18]:
```
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_,
         ax=ax)
```

9.6.2 Support Vector Machine

为了使用非线性核拟合SVM，我们再次使用SVC()估计器。然而，现在我们使用参数kernel的一个不同值。为了拟合具有多项式核的SVM，我们使用kernel="poly"，而为了拟合具有径向核的SVM，我们使用kernel="rbf"。在前一种情况下，我们还使用degree参数来指定多项式核的次数（这在(9.22)中是d），在后一种情况下，我们使用gamma来指定径向基核(9.24)中γ的值。

我们首先生成一些具有非线性类别边界的数据，如下所示:

In [19]:
```
X = rng.standard_normal((200, 2))
X[:100] += 2
X[100:150] -= 2
y = np.array([1]*150+[2]*50)
```

绘制数据可以清楚地表明类别边界确实是非线性的。

In [20]:
```
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0],
           X[:,1],
           c=y,
           cmap=cm.coolwarm)
```

Out[20]: <matplotlib.collections.PathCollection at 0x7faa9ba52eb0>

The data is randomly split into training and testing groups. We then fit the training data using the `SVC()` estimator with a radial kernel and $\gamma = 1$:

In [21]:
```
(X_train,
 X_test,
 y_train,
 y_test) = skm.train_test_split(X,
                                y,
                                test_size=0.5,
                                random_state=0)
svm_rbf = SVC(kernel="rbf", gamma=1, C=1)
svm_rbf.fit(X_train, y_train)
```

The plot shows that the resulting SVM has a decidedly non-linear boundary.

In [22]:
```
fig, ax = subplots(figsize=(8,8))
plot_svm(X_train,
         y_train,
         svm_rbf,
         ax=ax)
```

We can see from the figure that there are a fair number of training errors in this SVM fit. If we increase the value of `C`, we can reduce the number of training errors. However, this comes at the price of a more irregular decision boundary that seems to be at risk of overfitting the data.

In [23]:
```
svm_rbf = SVC(kernel="rbf", gamma=1, C=1e5)
svm_rbf.fit(X_train, y_train)
fig, ax = subplots(figsize=(8,8))
plot_svm(X_train,
         y_train,
         svm_rbf,
         ax=ax)
```

We can perform cross-validation using `skm.GridSearchCV()` to select the best choice of γ and `C` for an SVM with a radial kernel:

In [24]:
```
kfold = skm.KFold(5,
                  random_state=0,
                  shuffle=True)
grid = skm.GridSearchCV(svm_rbf,
                        {'C':[0.1,1,10,100,1000],
                         'gamma':[0.5,1,2,3,4]},
                        refit=True,
                        cv=kfold,
                        scoring='accuracy');
grid.fit(X_train, y_train)
grid.best_params_
```

Out[24]:
```
{'C': 100, 'gamma': 1}
```

在五折交叉验证下，参数的最佳选择在 `C=1` 和 `gamma=0.5` 处实现，尽管其他几个值也能达到相同的值。

In [25]:
```
best_svm = grid.best_estimator_
fig, ax = subplots(figsize=(8,8))
plot_svm(X_train,
```

```
         y_train,
         best_svm,
         ax=ax)

y_hat_test = best_svm.predict(X_test)
confusion_table(y_hat_test, y_test)
```

Out[25]:

Truth	1	2
Predicted		
1	69	6
2	6	19

使用这些参数，12% 的测试观测值被此 SVM 错误分类。

9.6.3 ROC Curves

支持向量机和支撑向量分类器为每个观测输出类别标签。然而，也可以获得每个观测的*fitted values*，这些是用于获取类别标签的数值分数。例如，在支撑向量分类器的情况下，观测$X = (X_1, X_2, \ldots, X_p)^T$的拟合值形式为$\hat{\beta}_0 + \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2 + \ldots + \hat{\beta}_p X_p$。对于具有非线性核的SVM，产生拟合值的方程在(9.23)中给出。拟合值的符号决定了观测位于决策边界的哪一侧。因此，对于给定的观测，拟合值与类别预测之间的关系很简单：如果拟合值大于零，则观测被分配到一个类别，如果小于零，则被分配到另一个类别。通过将这个阈值从零改变为某个正值，我们使分类偏向于一个类别而非另一个。通过考虑这些阈值的范围，包括正值和负值，我们为ROC图提供了要素。我们可以通过调用`decision_function()`来访问这些值。

拟合的SVM估计器的方法。

.function_
decision()

函数`ROCCurveDisplay.from_estimator()`（（我们将其缩写为`roc_curve()`））将生成一条ROC曲线的图。它接受一个拟合以估计器作为其第一个参数，后接模型矩阵 X 和标签 y。参数`name`用于图例，而`color`用于线条的颜色。结果绘制在我们的坐标轴对象`ax`上。

roc_curve()

In[26]:
```
fig, ax = subplots(figsize=(8,8))
roc_curve(best_svm,
          X_train,
          y_train,
          name='Training',
          color='r',
          ax=ax);
```

在这个例子中，SVM似乎提供了准确的预测。通过增加γ，我们可以产生更灵活的拟合，并进一步提高准确性。

In[27]:
```
svm_flex = SVC(kernel="rbf",
               gamma=50,
```

```
              C=1)
svm_flex.fit(X_train, y_train)
fig, ax = subplots(figsize=(8,8))
roc_curve(svm_flex,
          X_train,
          y_train,
          name='Training $\gamma=50$',
          color='r',
          ax=ax);
```

然而，这些ROC曲线都是基于训练数据的。我们实际上更关心的是测试数据上的预测准确性水平。当我们在测试数据上计算ROC曲线时，$\gamma = 0.5$的模型似乎提供了最准确的结果。

In [28]:
```
roc_curve(svm_flex,
          X_test,
          y_test,
          name='Test $\gamma=50$',
          color='b',
          ax=ax)
fig;
```

让我们看看调优后的SVM。

In [29]:
```
fig, ax = subplots(figsize=(8,8))
for (X_, y_, c, name) in zip(
     (X_train, X_test),
     (y_train, y_test),
     ('r', 'b'),
     ('CV tuned on training',
      'CV tuned on test')):
    roc_curve(best_svm,
              X_,
              y_,
              name=name,
              ax=ax,
              color=c)
```

9.6.4 SVM with Multiple Classes

如果响应是一个包含两个以上水平的因子，那么 `SVC()` 函数将使用一对一方法（当 `decision_function_shape=='ovo'` 时）或一对其余方法[4]（当 `decision_function_shape=='ovr'`）时）执行多类分类。我们通过生成第三类观测值来在此简要探讨该设置。

In [30]:
```
rng = np.random.default_rng(123)
X = np.vstack([X, rng.standard_normal((50, 2))])
y = np.hstack([y, [0]*50])
X[y==0,1] += 2
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=y, cmap=cm.coolwarm);
```

[4]One-versus-rest is also known as one-versus-all.

我们现在将SVM拟合到数据上：

In [31]:
```
svm_rbf_3 = SVC(kernel="rbf",
                C=10,
                gamma=1,
                decision_function_shape='ovo');
svm_rbf_3.fit(X, y)
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_rbf_3,
         scatter_cmap=cm.tab10,
         ax=ax)
```

sklearn.svm 库也可用于执行支持向量回归，其中使用估计器 SupportVector- Regression() 对数值响应进行预测。

SupportVector Regression()

9.6.5 *Application to Gene Expression Data*

我们现在检查 Khan 数据集，该数据集包含对应于四种不同的小圆蓝细胞肿瘤类型的多个组织样本。对于每个组织样本，都有基因表达测量值可用。该数据集包括训练数据 xtrain 和 ytrain，以及测试数据 xtest 和 ytest。

我们检查数据的维度：

In [32]:
```
Khan = load_data('Khan')
Khan['xtrain'].shape, Khan['xtest'].shape
```

Out[32]: ((63, 2308), (20, 2308))

该数据集包含2,308个基因的表达测量值。训练集和测试集分别包含63个和20个观测值。

我们将使用支持向量方法来利用基因表达测量预测癌症亚型。在这个数据集中，特征的数量相对于观测数量非常多。这表明我们应该使用线性核，因为使用多项式核或径向核所带来的额外灵活性是不必要的。

In [33]:
```
khan_linear = SVC(kernel='linear', C=10)
khan_linear.fit(Khan['xtrain'], Khan['ytrain'])
confusion_table(khan_linear.predict(Khan['xtrain']),
                Khan['ytrain'])
```

Out[33]:

Truth	1	2	3	4
Predicted				
1	8	0	0	0
2	0	23	0	0
3	0	0	12	0
4	0	0	0	20

我们看到有 *no* 个训练误差。事实上，这并不奇怪，因为变量数量相对于观测数量而言较多，意味着很容易找到完全分离各类的超平面。

我们更关注支持向量分类器在测试观测上的表现。

```
In[34]: confusion_table(khan_linear.predict(Khan['xtest']),
                        Khan['ytest'])
```

```
Out[34]:     Truth       1  2  3  4
         Predicted
                 1       3  0  0  0
                 2       0  6  2  0
                 3       0  0  4  0
                 4       0  0  0  5
```

W我们看到使用 C=10 在这些上产生了两个测试集错误 数据。

9.7 练习

Conceptual

1. 此问题涉及二维空间中的超平面。

(a) 画出超平面$1 + 3X_1 - X_2 = 0$的草图。标出满足$1 + 3X_1 - X_2 > 0$的点集，以及满足$1 + 3X_1 - X_2 < 0$的点集。(b) 在同一图上，画出超平面$-2 + X_1 + 2X_2 = 0$的草图。标出满足$-2 + X_1 + 2X_2 > 0$的点集，以及满足$-2 + X_1 + 2X_2 < 0$的点集。

2. 我们已经看到，在$p = 2$维空间中，线性决策边界的形式为$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$。现在我们来研究非线性决策边界。

(a) 绘制该曲线

$$(1 + X_1)^2 + (2 - X_2)^2 = 4.$$

(b) 在你的草图上，标出满足以下条件的点集

$$(1 + X_1)^2 + (2 - X_2)^2 > 4,$$

以及满足以下条件的点集

$$(1 + X_1)^2 + (2 - X_2)^2 \leq 4.$$

(c) 假设分类器将观测值分配给蓝色类别，如果

$$(1 + X_1)^2 + (2 - X_2)^2 > 4,$$

否则属于红色类。观测值 (0, 0) 被分类到哪个类？$(-1, 1)$？(2, 2)？(3, 8)？

(d) 论证虽然(c)中的决策边界在X_1和X_2方面不是线性的，但在X_1、X_1^2、X_2和X_2^2方面却是线性的。

3. H 我们在一个玩具数据集上探索最大间隔分类器。 数据集。

(a) 我们在p =2维中给出了n =7个观测值。对于每个观测值，都有一个关联的类别标签。

Obs.	X_1	X_2	Y
1	3	4	Red
2	2	2	Red
3	4	4	Red
4	1	4	Red
5	2	1	Blue
6	4	3	Blue
7	4	1	Blue

概述观察结果。

(b) 画出最优分离超平面的草图，并给出该超平面的方程（形式如(9.1)所示）。(c) 描述最大间隔分类器的分类规则。其表述应类似于“若 $\beta_0 + \beta_1 X_1 + \beta_2 X_2 > 0$，则分类为红色，否则分类为蓝色。”请提供 β_0、β_1 和 β_2 的值。(d) 在你的草图中，标出最大间隔超平面的间隔。(e) 指出最大间隔分类器的支持向量。(f) 论证第七个观测值的轻微变动不会影响最大间隔超平面。(g) 画出一个非最优分离超平面的超平面草图，并给出该超平面的方程。(h) 在图上添加一个额外的观测值，使得这两类无法再由一个超平面分离。

Applied

4. 生成一个包含100个观测值和两个特征的模拟二分类数据集，其中两个类别之间存在可见但非线性的分离。展示在这种情况下，使用多项式核（次数大于1）或径向核的支持向量机在训练数据上将优于支持向量分类器。哪种技术在测试数据上表现最佳？绘制图表并报告训练和测试错误率，以支持你的断言。

5. 我们已经了解到，可以通过使用非线性核函数来拟合SVM，从而实现基于非线性决策边界的分类。接下来我们将看到，通过对特征进行非线性变换，使用逻辑回归同样可以获得非线性决策边界。

(一个)生成数据集，其中n =500，p =2，使得观测值属于两个类别，且它们之间有一个二次决策边界。例如，你可以这样做：

```
rng = np.random.default_rng(5)
x1 = rng.uniform(size=500) - 0.5
x2 = rng.uniform(size=500) - 0.5
y = x1**2 - x2**2 > 0
```

() 根据类别标签绘制观察结果的颜色。你的图表应该在 x 轴上显示 X_1，并在 y 轴上显示 X_2。

(c) 使用 X_1 和 X_2 作为预测变量，对数据拟合一个逻辑回归模型。(d) 将此模型应用于 *training data*，以获得每个训练观测的预测类别标签。绘制观测点，并根据 *predicted* 类别标签着色。决策边界应为线性的。(e) 现在使用 X_1 和 X_2 的非线性函数（例如 X_1^2、$X_1 \times X_2$、log(X_2) 等）作为预测变量，对数据拟合一个逻辑回归模型。(f) 将此模型应用于 *training data*，以获得每个训练观测的预测类别标签。绘制观测点，并根据 *predicted* 类别标签着色。决策边界应明显是非线性的。如果不是，则重复 (a)–(e)，直到得出一个预测类别标签明显是非线性的例子。(g) 使用 X_1 和 X_2 作为预测变量，对数据拟合一个支持向量分类器。为每个训练观测获得类别预测。绘制观测点，并根据 *predicted class labels* 着色。(h) 使用非线性核对数据拟合一个 SVM。为每个训练观测获得类别预测。绘制观测点，并根据 *predicted class labels* 着色。(i) 评论你的结果。

6. 在9.6.1节末尾，有观点提出，对于刚好线性可分的数据，一个具有小{28}值且误分类了几对训练观察的支持向量分类器，在测试数据上的表现可能优于一个具有巨大{29}值且未误分类任何训练观察的分类器。现在，你将探究这一观点。

(a) 生成一个两类数据集，使得两类数据几乎刚好线性可分，其中 $p = 2$。 (b) 计算支持向量分类器在不同 c 值下的交叉验证错误率。对于考虑到的每个 c 值，有多少训练观测被错误分类，这与得到的交叉验证错误有何关系?

(c) 生成一个合适的测试数据集，并计算与所考虑的每个c值对应的测试误差。哪个c值导致最少的测试误差，这与产生最少训练误差和最少交叉验证误差的c值相比如何？（d）讨论你的结果。

7. 在这个问题中，你将使用支持向量机方法，基于Auto数据集来预测给定汽车的油耗是高还是低。

(a) 创建一个二元变量，对于油耗高于中位数的汽车取值为1，对于油耗低于中位数的汽车取值为0。(b) 使用不同c值的数据拟合一个支持向量分类器，以预测汽车的油耗高低。报告不同参数值对应的交叉验证误差，并对结果进行评论。请注意，为了得到合理的结果，你需要在拟合分类器时排除油耗变量。(c) 现在重复(b)步骤，这次使用径向基和多项式核的支持向量机，并尝试不同的gamma、degree和c值。对结果进行评论。(d) 绘制一些图表来支持你在(b)和(c)中的论断。

Hint: In the lab, we used the `plot_svm()` *function for fitted SVMs. When* $p > 2$*, you can use the keyword argument* `features` *to create plots displaying pairs of variables at a time.*

8. 此问题涉及OJ数据集，该数据集是ISLP包的一部分。

(a) 创建一个包含800个观测值的随机样本作为训练集，剩余的观测值作为测试集。(b) 使用`C = 0.01`对训练数据拟合一个支持向量分类器，其中Purchase作为响应变量，其他变量作为预测变量。有多少个支持点？(c) 训练和测试错误率分别是多少？(d) 使用交叉验证选择最优的C。考虑范围在0.01到10之间的值。(e) 使用c的新值计算训练和测试错误率。(f) 使用径向核的支持向量机重复(b)至(e)部分。使用gamma的默认值。(g) 使用多项式核的支持向量机重复(b)至(e)部分。设置`degree = 2`。(h) 总体而言，哪种方法在这份数据上似乎给出了最佳结果?

10
深度学习

本章涵盖 *deep learning* 的重要主题。在写作（2020），深度学习是机器学习和人工智能社区中非常活跃的研究领域。深度学习的基石是*neural network*。

深度学习

神经网络

神经网络在20世纪80年代末声名鹊起。这种方法引起了极大的兴奋和一定程度的炒作，并且它们推动了每年通常在滑雪胜地等异国情调的地方举行的热门*Neural Information Processing Systems*会议（NeurIPS，前身为NIPS）的兴起。随后是一个综合阶段，机器学习专家、数学家和统计学家对神经网络的特性进行了分析；算法得到了改进，方法论也趋于稳定。接着出现了SVM、boosting和随机森林，神经网络在一定程度上失宠了。部分原因是神经网络需要大量的调试，而新方法更加自动化。此外，在许多问题上，新方法的表现优于训练不佳的神经网络。这是新千年第一个十年的*status quo*。

一直以来，尽管这样，一个核心的神经网络爱好者群体仍在不断推动他们的技术，在越来越大的计算架构和数据集上加大力度。2010年后，神经网络以新的名称*deeplearning*重新出现，伴随着新的架构、更多附加功能，以及在图像和视频分类、语音和文本建模等一些小众问题上取得的一系列成功案例。该领域的许多人认为，这些成功的主要原因在于可用的训练数据集越来越大，而这得益于科学和工业领域数字化的大规模应用。

在本章中，我们将讨论神经网络和深度学习的基础知识，然后深入探讨针对特定问题的一些专门化方法，例如用于图像分类的卷积神经网络（CNNs），以及用于时间序列和其他序列的循环神经网络（RNNs）。我们

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_10

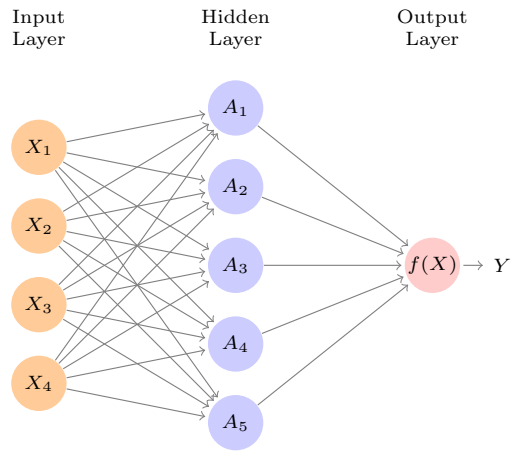


图10.1。 *Neural network with a single hidden layer. The hidden layer computes activations* $A_k = h_k(X)$ *that are nonlinear transformations of linear combinations of the inputs* $X_1, X_2, \ldots, X_p$*. Hence these* A_k *are not directly observed. The functions* $h_k(\cdot)$ *are not fixed in advance, but are learned during the training of the network. The output layer is a linear model that uses these activations* A_k *as inputs, resulting in a function* $f(X)$*.*

还将使用`Python torch`包以及许多辅助包来演示这些模型。

本章内容比本书其他部分稍微更具挑战性。

10.1 单层神经网络

神经网络接收一个由 p 个变量 $X = (X_1, X_2, \ldots, X_p)$ 组成的输入向量，并构建一个非线性函数 $f(X)$ 来预测响应 Y。在前面的章节中，我们已经使用树、提升法和广义加性模型构建了非线性预测模型。神经网络与这些方法的区别在于模型的特定 *structure*。图 10.1 展示了一个用于建模定量响应的简单 *feed-forward neural network*。

前馈神经网络

使用 $p = 4$ 个预测变量。在神经网络的术语中，这四个特征 $X_1, \ldots, X_4$ 构成了 *input layer* 中的单元。箭头表示来自输入层的每个输入都馈入每个K *hidden units* (，我们可以选择K；这里我们选择了5)。神经网络模型有形式

输入层

隐藏单元

$$\begin{aligned} f(X) &= \beta_0 + \textstyle\sum_{k=1}^{K} \beta_k h_k(X) \\ &= \beta_0 + \textstyle\sum_{k=1}^{K} \beta_k g(w_{k0} + \textstyle\sum_{j=1}^{p} w_{kj} X_j). \end{aligned} \tag{10.1}$$

它在这里分两步构建。首先是 K *activations* A_k, $k = 1, \ldots, K$, 在隐藏层是作为输入特征$X_1, \ldots, X_p$的函数计算得出的，

激活值

$$A_k = h_k(X) = g(w_{k0} + \textstyle\sum_{j=1}^{p} w_{kj} X_j), \tag{10.2}$$

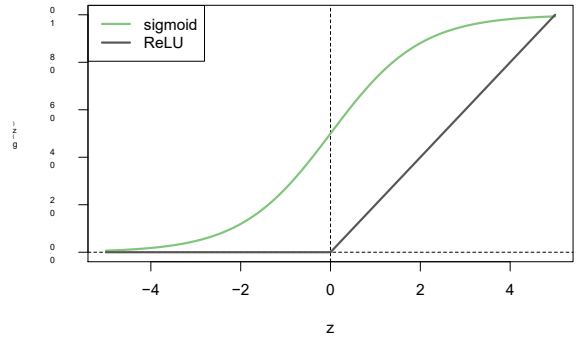


图 10.2. *Activation functions. The piecewise-linear* `ReLU` *function is popular for its efficiency and computability. We have scaled it down by a factor of five for ease of comparison.*

其中 $g(z)$ 是预先指定的非线性 *activation function*。
我们可以将每个 A_k 视为原始特征的不同变换 $h_k(X)$，很像第7章的基函数。这些来自隐藏层的 K 激活值接着输入到输出层，得到

激活函数

$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k, \tag{10.3}$$

$K = 5$ 个激活函数中的线性回归模型。所有参数 $\beta_0, \ldots, \beta_K$ 和 $w_{10}, \ldots, w_{Kp}$ 都需要从数据中估计。在神经网络的早期实例中，*sigmoid* 激活函数更受青睐，

sigmoid

$$g(z) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}, \tag{10.4}$$

这与逻辑回归中用于将线性函数转换为0到1之间概率的函数相同（见图10.2）。现代神经网络中首选的是*ReLU* (*rectified linear unit*) 激活函数，其形式为

ReLU
修正线性单元

$$g(z) = (z)_+ = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{otherwise.} \end{cases} \tag{10.5}$$

ReLU 激活函数的计算和存储比 sigmoid 激活函数更高效。尽管它以零为阈值，但由于我们将其应用于线性函数 (10.2)，常数项 w_{k0} 将移动这个拐点。

因此，用文字描述，图10.1所示的模型通过计算X的五种不同线性组合，得出五个新特征，然后通过激活函数$g(\cdot)$对每个特征进行压缩变换。最终模型在这些衍生变量上是线性的。

名称 *neural network* 最初源于将这些隐藏单元类比作大脑中的神经元——激活值 $A_k = h_k(X)$ 接近于一表示 *firing*，而接近于零则表示 *silent* (使用 sigmoid 激活函数)。

激活函数 $g(\cdot)$ 中的非线性是必不可少的，因为如果没有它，(10.1) 中的模型 $f(X)$ 将坍缩为一个简单的线性模型。

$X_1, \ldots, X_p$。此外，拥有非线性激活函数使模型能够捕捉复杂的非线性和交互效应。考虑一个非常简单的例子，有$p = 2$个输入变量$X = (X_1, X_2)$，以及$K = 2$个隐藏单元$h_1(X)$和$h_2(X)$，其中$g(z) = z^2$。我们将其他参数指定为

$$\begin{array}{lll} \beta_0 = 0, & \beta_1 = \frac{1}{4}, & \beta_2 = -\frac{1}{4}, \\ w_{10} = 0, & w_{11} = 1, & w_{12} = 1, \\ w_{20} = 0, & w_{21} = 1, & w_{22} = -1. \end{array} \tag{10.6}$$

由（10.2），这意味着

$$\begin{array}{rcl} h_1(X) & = & (0 + X_1 + X_2)^2, \\ h_2(X) & = & (0 + X_1 - X_2)^2. \end{array} \tag{10.7}$$

然后将（10.7）代入（10.1），我们得到

$$\begin{array}{rcl} f(X) & = & 0 + \frac{1}{4} \cdot (0 + X_1 + X_2)^2 - \frac{1}{4} \cdot (0 + X_1 - X_2)^2 \\ & = & \frac{1}{4}\left[(X_1 + X_2)^2 - (X_1 - X_2)^2\right] \\ & = & X_1 X_2. \end{array} \tag{10.8}$$

因此，两个线性函数的非线性变换之和可以给我们一个交互项！在实践中，我们不会对 $g(z)$ 使用二次函数，因为这样在原坐标系 $X_1, \ldots, X_p$ 中我们总是得到一个二次多项式。Sigmoid 或 ReLU 激活函数没有这样的限制。

拟合神经网络需要估计(10.1)中的未知参数。对于定量响应，通常使用平方误差损失，因此参数的选择是为了最小化

$$\sum_{i=1}^{n} \left(y_i - f(x_i)\right)^2. \tag{10.9}$$

有关如何执行此最小化的详细信息，请参见第10.7节。

10.2 多层神经网络

现代神经网络通常具有多个隐藏层，且每层往往包含许多单元。理论上，一个拥有大量单元的单隐藏层就有能力逼近大多数函数。然而，通过多个适中大小的层，发现良好解的学习任务会变得容易得多。

我们将在一个著名且公开可用的 `MNIST` 手写数字数据集上展示一个大型密集网络。[1] 图10.3展示了这些数字的示例。其思路是构建一个模型，将图像分类到正确的数字类别0–9。每张图像有 $p = 28 \times 28 = 784$ 个像素，每个像素是介于0到255之间的八位灰度值，表示

[1] See LeCun, Cortes, and Burges (2010) "The MNIST database of handwritten digits", available at `http://yann.lecun.com/exdb/mnist`.

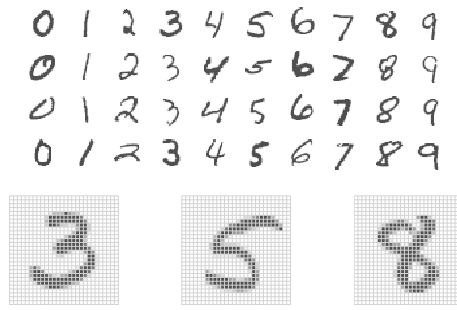


图10.3。*Examples of handwritten digits from the* MNIST *corpus. Each grayscale image has* 28×28 *pixels, each of which is an eight-bit number (0–255) which represents how dark that pixel is. The first 3, 5, and 8 are enlarged to show their 784 individual pixel values.*

那个小方格中手写数字的相对数量。[2]这些像素按列顺序存储在输入向量 X (中。输出是类别标签，由一个包含10个虚拟变量的向量 $Y = (Y_0, Y_1, \ldots, Y_9)$表示，在对应于标签的位置为1，其他位置为0。在机器学习领域，这被称为*one-hot encoding*。有60,000张训练图像和10,000张测试图像。

one-hot encoding

从历史角度看，数字识别问题是20世纪80年代末在AT&T贝尔实验室及其他地方加速神经网络技术发展的催化剂。这类模式识别任务对人类来说相对简单。我们的视觉系统占据了大脑很大一部分，良好的识别能力是生存的进化动力。这些任务对机器来说并不那么简单，花了30多年时间才完善神经网络架构以匹配人类的表现。

图10.4展示了一个多层网络架构，该架构在解决数字分类任务上表现良好。它与图10.1在几个方面有所不同：

- 它有两个隐藏层，分别为L_1 (256 个单元)和L_2 (128 个单元)，而不是一个。稍后我们将看到一个具有七个隐藏层的网络。
- 它有十个输出变量，而非一个。在这种情况下，这十个变量实际上代表了一个定性变量，因此高度相关。（为了清晰起见，我们按数字类别0-9而非1-10进行索引。）更一般地，在*multi-task learning*中可以预

  字典同时用单个网络对不同响应进行建模；它们都对隐藏层的形成有发言权。

  多任务学习
- 用于训练网络的损失函数是针对多类分类任务定制的。

[2]In the analog-to-digital conversion process, only part of the written numeral may fall in the square representing a particular pixel.

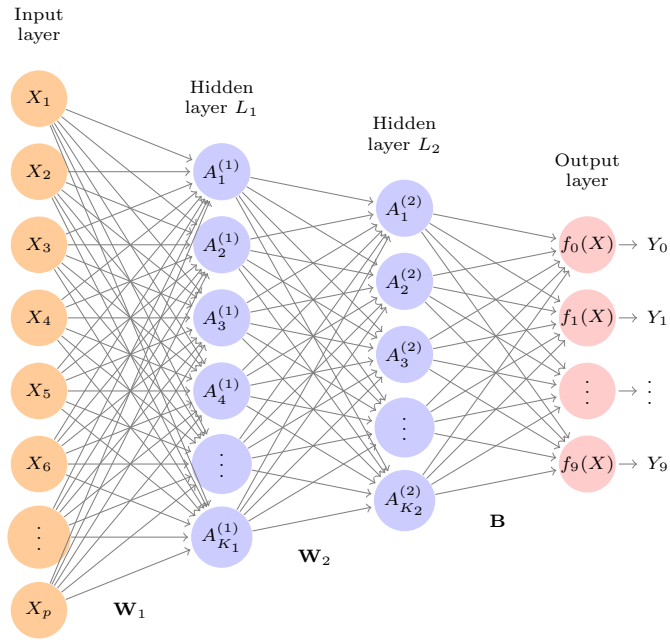


图 10.4. *Neural network diagram with two hidden layers and multiple outputs, suitable for the* MNIST *handwritten-digit problem. The input layer has* $p = 784$ *units, the two hidden layers* $K_1 = 256$ *and* $K_2 = 128$ *units respectively, and the output layer* 10 *units. Along with intercepts (referred to as* 偏置 *in the deep-learning community) this network has 235,146 parameters (referred to as weights).*

第一隐藏层如式(10.2)所示，其中

$$\begin{aligned} A_k^{(1)} &= h_k^{(1)}(X) \\ &= g(w_{k0}^{(1)} + \textstyle\sum_{j=1}^{p} w_{kj}^{(1)} X_j) \end{aligned} \tag{10.10}$$

对于 $k = 1, \ldots, K_1$。第二隐藏层将第一隐藏层的激活 $A_k^{(1)}$ 作为输入，并计算新的激活

$$\begin{aligned} A_\ell^{(2)} &= h_\ell^{(2)}(X) \\ &= g(w_{\ell 0}^{(2)} + \textstyle\sum_{k=1}^{K_1} w_{\ell k}^{(2)} A_k^{(1)}) \end{aligned} \tag{10.11}$$

对于 $\ell = 1, \ldots, K_2$. 注意，第二层中的每个激活 $A_\ell^{(2)} = h_\ell^{(2)}(X)$ 都是输入向量 X 的函数。之所以如此，是因为虽然它们明确是来自层 L_1 的激活 $A_k^{(1)}$ 的函数，但这些激活反过来又是 X 的函数。对于更多的隐藏层也是如此。因此，通过一系列变换，网络能够构建出 X 的相当复杂的变换，这些变换最终作为特征输入到输出层。

我们在(10.10)和(10.11)中引入了额外的上标表示法，如$h_\ell^{(2)}(X)$和$w_{\ell j}^{(2)}$，以指示激活值和*weights* (系数)所属的层，本例中为第2层。图$\mathbf{W}_1$中的表示法

weights

图10.4展示了从输入层馈送到第一个隐藏层L_1的整个权重矩阵。这个矩阵将有785×256 = 200,960个元素；之所以是785而不是784，是因为我们必须考虑截距或*bias*项。[3]

偏差

每个元素 $A_k^{(1)}$ 通过维度为 257 × 128 = 32,896 的权重矩阵 $\mathbf{W}_2$ 馈送到第二隐藏层 L_2。

现在我们来到输出层，这里我们有十个响应而非一个。第一步是计算十个不同的线性模型，类似于我们的单一模型（10.1），

$$\begin{aligned} Z_m &= \beta_{m0} + \textstyle\sum_{\ell=1}^{K_2} \beta_{m\ell} h_\ell^{(2)}(X) \\ &= \beta_{m0} + \textstyle\sum_{\ell=1}^{K_2} \beta_{m\ell} A_\ell^{(2)}, \end{aligned} \tag{10.12}$$

对于 $m = 0, 1, \ldots, 9$。矩阵 $\mathbf{B}$ 存储了所有这些权重中的129个 × 10 = 1,290。

如果这些都是独立的定量响应，我们只需设置每个 $f_m(X) = Z_m$ 即可完成。然而，我们希望我们的估计代表类别概率 $f_m(X) = \Pr(Y = m|X)$，就像第4.3.5节中的多项逻辑回归一样。因此，我们使用特殊的 *softmax* 激活函数（参见第145页的(4.13)），

softmax

$$f_m(X) = \Pr(Y = m|X) = \frac{e^{Z_m}}{\sum_{\ell=0}^{9} e^{Z_\ell}}, \tag{10.13}$$

对于 $m = 0, 1, \ldots, 9$，这确保了这 10 个数字表现得像概率（非负且总和为 1）。尽管目标是构建一个分类器，我们的模型实际上为这 10 个类别中的每一个估计了一个概率。分类器随后将图像分配给概率最高的类别。

为了训练这个网络，由于响应是定性的，我们寻找使负多项对数似然最小化的系数估计。

$$-\sum_{i=1}^{n}\sum_{m=0}^{9} y_{im} \log(f_m(x_i)), \tag{10.14}$$

也称为 *cross-entropy*。这是 crite- 的推广。

交叉熵

用于二分类逻辑回归的误差函数（4.5）。关于如何最小化该目标的详细信息见第10.7节。如果响应变量是定量的，我们将改为最小化如（10.9）中的平方误差损失。

表10.1比较了神经网络与第4章中提出的两种利用线性决策边界的简单模型：多项逻辑回归和线性判别分析的测试性能。神经网络相较于这两种线性方法的提升是显著的：采用dropout正则化的网络在10,000张测试图像上的测试错误率低于2%。（我们在第10.7.3节描述了dropout正则化。）在实验室的第10.9.2节中，我们展示了拟合此模型的代码，该代码在笔记本电脑上运行仅需两分多钟。

[3]The use of "weights" for coefficients and "bias" for the intercepts w_{k0} in (10.2) is popular in the machine learning community; this use of bias is not to be confused with the "bias-variance" usage elsewhere in this book.

Method	Test Error
Neural Network + Ridge Regularization	2.3%
Neural Network + Dropout Regularization	1.8%
Multinomial Logistic Regression	7.2%
Linear Discriminant Analysis	12.7%

表 10.1. *Test error rate on the* MNIST *data, for neural networks with two forms of regularization, as well as multinomial logistic regression and linear discriminant analysis. In this example, the extra complexity of the neural network leads to a marked improvement in test error.*

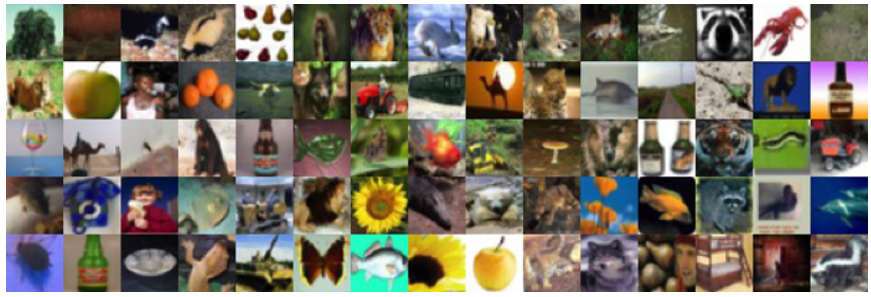


图 10.5. *A sample of images from the* CIFAR100 *database: a collection of natural images from everyday life, with 100 different classes represented.*

将 $\mathbf{W}_1$、$\mathbf{W}_2$ 和 $\mathbf{B}$ 中系数的数量相加，我们总共得到 235,146 个，是多项式逻辑回归所需的 $785 \times 9 = 7{,}065$ 这个数字的 33 倍多。回想一下，训练集中有 60,000 张图像。虽然这看起来像是一个很大的训练集，但神经网络模型中的系数数量几乎是训练集中观测值数量的四倍！为了避免过拟合，需要一些正则化。在这个例子中，我们使用了两种形式的正则化：岭正则化，类似于第6章中的岭回归，以及 *dropout* 正则化。

dropout

我们在第10.7节中讨论这两种形式的正则化。

10.3 卷积神经网络

神经网络在2010年左右随着图像分类的巨大成功而复兴。大约在那个时候，标注图像的大型数据库正在积累，类别数量不断增加。图10.5展示了从CIFAR100数据库中抽取的75张图像。[4] 该数据库包含60,000张图像，按照20个超类（例如水生哺乳动物）进行标注，每个超类下有五个类别（海狸、海豚、水獭、海豹、鲸鱼）。每张图像的分辨率为32 × 32像素，每个像素由三个八位数字表示红、绿和蓝。每张图像的数字被组织在一个称为 *feature map*的三维数组中。前两个

feature map

[4]See Chapter 3 of Krizhevsky (2009) "Learning multiple layers of features from tiny images", available at https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

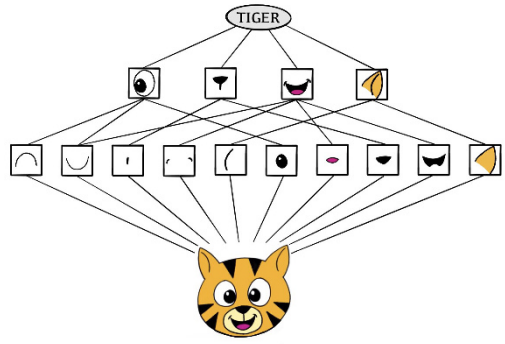


图10.6。 *Schematic showing how a convolutional neural network classifies an image of a tiger. The network takes in the image and identifies local features. It then combines the local features in order to create compound features, which in this example include eyes and ears. These compound features are used to output the label "tiger".*

坐标轴是空间的（两者都是32维的），第三个是*channel*轴，[5] 代表三种颜色。有一个指定的训练集包含 50,000 张图像，以及一个测试集包含 10,000 张。

渠道

一个为*convolutional neural networks* (CNN)演化出的特殊家族classifying images such as these, and has shown spectacular success on a wide ra nge of problems. CNNs mimic to some degree how humans classify images, by r ecognizing specific features or patterns anywhere in the image that distinguish e ach particular object class. In this section we give a brief overview of how they work.

卷积神经网络

图10.6展示了卷积神经网络在老虎卡通图像上的工作原理。[6]

网络首先识别输入图像中的低级特征，如小边缘、色块等。然后，这些低级特征被组合成更高级别的特征，如耳朵、眼睛的部分等。最终，这些高级特征的存在与否影响了任何给定输出类别的概率。

卷积神经网络如何构建这种层次结构？它结合了两种专门的隐藏层类型，称为*convolution*层和*pooling*层。卷积层在图像中搜索小模式的实例，而池化层则对这些进行下采样，以选择突出的子集。为了达到最先进的结果，当代神经网络架构利用了许多卷积层和池化层。接下来我们描述卷积层和池化层。

10.3.1 Convolution Layers

一个 *convolutional layer* 由大量 *convolution filters* 组成，每个卷积层

convolution filter

[5] The term *channel* is taken from the signal-processing literature. Each channel is a distinct source of information.

[6] Thanks to Elena Tuzhilina for producing the diagram and `https://www.cartooning4kids.com/` for permission to use the cartoon tiger.

其中是一个模板，用于确定图像中是否存在特定的局部特征。卷积滤波器依赖于一个非常简单的操作，称为 *convolution*，其基本含义是反复将矩阵元素相乘，然后相加结果。

要理解卷积滤波器的工作原理，可以考虑一个非常简单的4 × 3图像示例：

$$\text{Original Image} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \\ j & k & l \end{bmatrix}.$$

现在考虑形式为 2 × 2 的滤波器

$$\text{Convolution Filter} = \begin{bmatrix} \alpha & \beta \\ \gamma & \delta \end{bmatrix}.$$

当我们用滤波器*convolve*图像时，得到结果[7]

$$\text{Convolved Image} = \begin{bmatrix} a\alpha + b\beta + d\gamma + e\delta & b\alpha + c\beta + e\gamma + f\delta \\ d\alpha + e\beta + g\gamma + h\delta & e\alpha + f\beta + h\gamma + i\delta \\ g\alpha + h\beta + j\gamma + k\delta & h\alpha + i\beta + k\gamma + l\delta \end{bmatrix}.$$

例如，左上角的元素是由2 × 2滤波器中的每个元素与图像左上角2 × 2区域内的对应元素相乘，并将结果相加得到的。其他元素也以类似方式获得：卷积滤波器应用于原始图像的每个2 × 2子矩阵，以获得卷积后的图像。如果原始图像的一个2 × 2子矩阵与卷积滤波器相似，则它在卷积图像中会有*large*值；否则，它将有*small*值。因此，
the convolved image highlights regions of the original image that resemble the convolution filter. 我们以2 × 2为例；一般来说，卷积滤波器是小的$\ell_1 \times \ell_2$数组，其中ℓ_1和ℓ_2是小的正整数，且不一定相等。

图10.7展示了将两个卷积滤波器应用于一张尺寸为192× 179的老虎图像的过程，该图像位于左侧。[8] 每个卷积滤波器是一个15 × 15的图像，其中大部分为零（黑色），并在图像内沿垂直或水平方向有一条狭窄的一（白色）带。当每个滤波器与老虎的图像进行卷积时，老虎中与滤波器相似的区域（即具有水平或垂直条纹或边缘的区域）被赋予较大的值，而与特征不相似的老虎区域则被赋予较小的值。卷积后的图像显示在右侧。我们可以看到，水平条纹滤波器提取了原始图像中的水平条纹和边缘，而垂直条纹滤波器则提取了原始图像中的垂直条纹和边缘。

[7]The convolved image is smaller than the original image because its dimension is given by the number of 2 × 2 submatrices in the original image. Note that 2 × 2 is the dimension of the convolution filter. If we want the convolved image to have the same dimension as the original image, then padding can be applied.

[8]The tiger image used in Figures 10.7–10.9 was obtained from the public domain image resource `https://www.needpix.com/`.

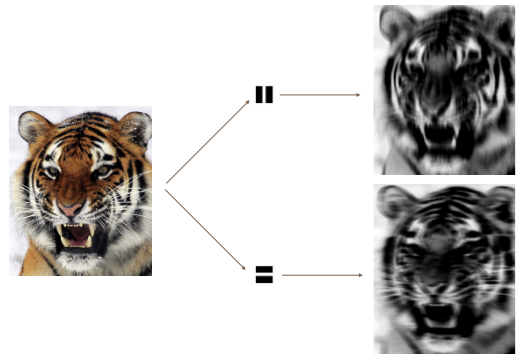


图10.7。 *Convolution filters find local features in an image, such as edges and small shapes. We begin with the image of the tiger shown on the left, and apply the two small convolution filters in the middle. The convolved images highlight areas in the original image where details similar to the filters are found. Specifically, the top convolved image highlights the tiger's vertical stripes, whereas the bottom convolved image highlights the tiger's horizontal stripes. We can think of the original image as the input layer in a convolutional neural network, and the convolved images as the units in the first hidden layer.*

我们在图10.7中使用了一张大图像和两个大型滤波器进行说明。对于CIFAR100数据库，每张图像有32 × 32个彩色像素，我们使用3 × 3卷积滤波器。

在卷积层中，我们使用一整组滤波器来提取图像中各种不同方向的边缘和形状。以这种方式使用预定义滤波器是图像处理中的标准做法。相比之下，在CNN中，滤波器是为特定分类任务*learned*的。我们可以将滤波器权重视为从输入层到隐藏层的参数，卷积图像中的每个像素对应一个隐藏单元。事实上正是如此，尽管这些参数具有高度结构化和受限的特性（详见习题4）。它们作用于输入图像中的局部区域（因此存在大量结构性零），并且给定滤波器中的相同权重会在图像中的所有可能区域中重复使用（因此权重受到约束）。[9]

我们现在给出一些额外的细节。

- 由于输入图像是彩色的，因此它有三个通道，由一个三维特征图（数组）表示。每个通道是一个二维（32 × 32）特征图——分别对应红色、绿色和蓝色。单个卷积滤波器也将有三个通道，每种颜色一个，每个通道的维度为3 × 3，且可能具有不同的滤波权重。三次卷积的结果相加，形成一个二维输出特征图。请注意，此时颜色信息已被使用，并且除了在卷积中的作用外，不会传递到后续层。

[9]This used to be called *weight sharing* in the early years of neural networks.

- 如果我们在第一个隐藏层使用K个不同的卷积滤波器，我们会得到K个二维输出特征图，这些特征图共同被视为一个三维特征图。我们将K个输出特征图中的每一个视为单独的信息通道，因此现在我们拥有K个通道，与原始输入特征图的三个颜色通道形成对比。三维特征图就像简单神经网络隐藏层中的激活一样，只是以空间结构化的方式组织和生成。

- 我们通常对卷积后的图像应用ReLU激活函数（10.5）。这一步骤有时被视为卷积神经网络中的一个独立层，在这种情况下，它被称为*detector layer*。

detector layer

10.3.2 Pooling Layers

一个 *pooling* 层提供了一种将大图像压缩为较小图像的方法。汇总图像。虽然执行池化有多种可能的方法，但 *max pooling* 操作通过使用图像中每个不重叠的 2×2 像素块中的最大值来汇总该块。这会在每个方向上将图像的大小缩小一半，并且它还提供了一些 *location invariance*：也就是说，只要该块中四个像素之一中存在较大值，整个块在缩小后的图像中就会记录为较大值。

pooling

这是一个最大池化的简单示例：

$$\text{Max pool} \begin{bmatrix} 1 & 2 & 5 & 3 \\ 3 & 0 & 1 & 2 \\ 2 & 1 & 3 & 4 \\ 1 & 1 & 2 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix}.$$

10.3.3 Architecture of a Convolutional Neural Network

到目前为止，我们已定义了一个单一的卷积层——每个滤波器生成一个新的二维特征图。卷积层中的卷积滤波器数量类似于我们在第10.2节中见到的那种全连接神经网络中特定隐藏层的单元数量。这个数量也定义了所得到的三维特征图中的通道数。我们还描述了一个池化层，它减少了每个三维特征图的前两个维度。深度CNN有许多这样的层。图10.8展示了用于`CIFAR100`图像分类任务的CNN的典型架构。

在输入层，我们看到彩色图像的三维特征图，其中通道轴通过一个32×32的二维像素特征图来表示每种颜色。每个卷积滤波器在第一个隐藏层产生一个新的通道，每个通道都是一个32×32的特征图（在边缘进行一些填充之后）。经过第一轮卷积后，我们现在有了一个新的“图像”；一个通道数远多于三个颜色输入通道的特征图（图中为六个，因为我们使用了六个卷积滤波器）。

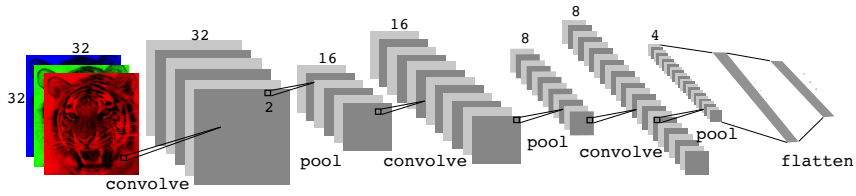


图10.8。*Architecture of a deep CNN for the* CIFAR100 *classification task. Convolution layers are interspersed with* 2×2 *max-pool layers, which reduce the size by a factor of 2 in both dimensions.*

接下来是一个最大池化层，它在每个通道上将特征图的大小减小四倍：每个维度上减少两倍。

这个先卷积后池化的序列现在在接下来的两层中重复。一些细节如下：

- 每个后续的卷积层与第一层类似。它接收来自前一层的三维特征图作为输入，并将其视为单张多通道图像。学到的每个卷积滤波器具有与该特征图相同的通道数。

- 由于每个池化层后通道特征图的大小都会减小，我们通常会增加下一个卷积层中的滤波器数量以进行补偿。

- 有时我们在池化层之前重复几个卷积层。这有效地增加了滤波器的维度。

重复这些操作，直到池化将每个通道的特征图在每个维度上缩减到仅几个像素。此时，三维特征图为*flattened*——像素被视为独立单元——并在到达输出层之前被送入一个或多个全连接层，输出层是一个针对100个类别的*softmax activation*（如(10.13)中所示）。

在构建这样的网络时，除了每层的数量、性质和大小之外，还有许多调优参数需要选择。每一层都可以使用dropout学习，以及lasso或ridge正则化（见第10.7节）。构建卷积神经网络的细节可能看起来令人生畏。幸运的是，有出色的软件可用，其中包含大量示例和小插图，为参数的合理选择提供了指导。对于CIFAR100官方测试集，截至本文撰写时，最佳准确率略高于75%，但毫无疑问，这一性能将继续提高。

10.3.4 Data Augmentation

在图像建模中使用的一个额外重要技巧是 *data augmentation*本质上，每张训练图像会被复制多次，每个副本以自然的方式随机扭曲，使得人类识别不受影响。图10.9展示了一些示例。典型的扭曲包括

data augmentation



图10.9. *Data augmentation. The original image (leftmost) is distorted in natural ways to produce different images with the same class label. These distortions do not fool humans, and act as a form of regularization when fitting the CNN.*

缩放、水平和垂直平移、剪切、小角度旋转，以及在这种情况下水平翻转。表面上看，这是一种通过略有不同的示例大幅增加训练集的方法，从而防止过拟合。实际上，我们可以将其视为一种正则化形式：我们围绕每个原始图像构建一个图像云，所有图像都具有相同的标签。这种数据增厚在精神上类似于岭正则化。

我们将在第10.7.2节中看到，用于拟合深度学习模型的随机梯度下降算法会反复处理随机选取的批次，比如一次128张训练图像。这与数据增强配合得相得益彰，因为我们可以在批次中即时对每张图像进行变形，因此不必存储所有新图像。

10.3.5 Results Using a Pretrained Classifier

这里我们使用行业级预训练分类器来预测一些新图像的类别。`resnet50`分类器是一个卷积神经网络，使用`imagenet`数据集进行训练，该数据集包含数百万张属于不断增长类别的图像。[10]图10.10展示了`resnet50`在六张照片上的表现（其中一位作者的私人收藏）。[11]该CNN在分类第二张图像中的鹰时表现合理。如果像第三张图像那样缩小，它会感到困惑，并选择喷泉而不是鹰。在最后一张图像中，“jacamar”是一种来自南美和中美洲的热带鸟类，其颜色与南非开普织布鸟相似。我们将在第10.9.4节中给出此示例的更多细节。

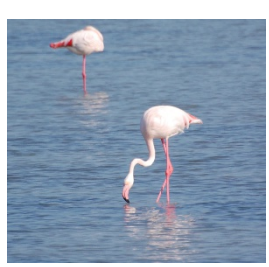
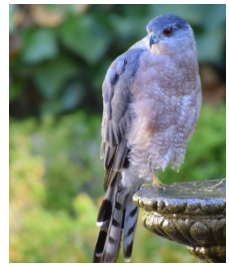
训练卷积神经网络（CNN）的大部分工作在于学习隐藏层的卷积滤波器；这些是CNN的系数。对于在如`imagenet`这样具有许多类别的大型语料库上训练的模型，这些滤波器的输出可以作为通用自然图像分类问题的特征。人们可以将这些预训练的隐藏层用于训练集小得多的新问题（这一过程称为*weight freezing*），并且

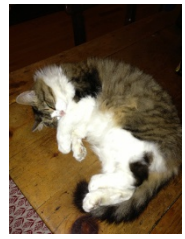
只需训练网络的最后几层，这需要的数据量要少得多。

weight freezing

[10]For more information about `resnet50`, see He, Zhang, Ren, and Sun (2015) “Deep residual learning for image recognition”, `https://arxiv.org/abs/1512.03385`. For details about `imagenet`, see Russakovsky, Deng, et al. (2015) “ImageNet Large Scale Visual Recognition Challenge”, in *International Journal of Computer Vision*.

[11]These `resnet` results can change with time, since the publicly-trained model gets updated periodically.


flamingo		Cooper's hawk		Cooper's hawk	
flamingo	0.83	kite	0.60	fountain	0.35
spoonbill	0.17	great grey owl	0.09	nail	0.12
white stork	0.00	robin	0.06	hook	0.07
Lhasa Apso		cat		Cape weaver	
Tibetan terrier	0.56	Old English sheepdog	0.82	jacamar	0.28
Lhasa	0.32	Shih-Tzu	0.04	macaw	0.12
cocker spaniel	0.03	Persian cat	0.04	robin	0.12

图 10.10. *Classification of six photographs using the* `resnet50` *CNN trained on the* `imagenet` *corpus. The table below the images displays the true (intended) label at the top of each panel, and the top three choices of the classifier (out of 100). The numbers are the estimated probabilities for each choice. (A kite is a raptor, but not a hawk.)*

随附于 `keras` 包的案例研究和书籍[12]提供了更多关于此类应用的细节。

10.4 文档分类

在本节中，我们介绍一种在工业界和科学界具有重要应用的新型示例：文档属性预测。文档的例子包括医学期刊中的文章、路透社新闻推送、电子邮件、推文等。我们的示例将是`IMDb` (互联网电影数据库)评分——观众撰写电影评论的短文档。[13]在这种情况下，响应是评论的`sentiment`，其将是*positive*或*negative*。

[12] *Deep Learning with R* by F. Chollet and J.J. Allaire, 2018, Manning Publications.

[13] For details, see Maas et al. (2011) "Learning word vectors for sentiment analysis", in *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150.

这是一篇相当有趣的负面评论的开头 :

> *This has to be one of the worst films of the 1990s. When my friends & I were watching this film (being the target audience it was aimed at) we just sat & watched the first half an hour with our jaws touching the floor at how bad it really was. The rest of the time, everyone else in the theater just started talking to each other, leaving or generally crying into their popcorn ...*

每条评论的长度可能各不相同，可能包含俚语或非标准词汇，可能存在拼写错误等。我们需要找到一种方法来 *featurize* 这样的文档。

特征化

这是定义一组预测因子的现代说法。

最简单且最常见的特征化是 *bag-of-words* 模型。

词袋

我们为每个文档打分，以判断语言字典中每个单词是否出现——此处使用的是英语字典。如果字典包含M个单词，这意味着对于每个文档，我们创建一个长度为M的二进制特征向量，对于出现的每个单词赋值为1，否则赋值为0。这可能会是一个非常宽泛的特征向量，因此我们限制字典的大小——在此情况下，限制为训练语料库中25,000条评论中最常出现的10,000个单词。幸运的是，有很好的工具可以自动完成这项工作。以下是一篇正面评论的开头，已按此方式进行了删减处理:

> $\langle START \rangle$ *this film was just brilliant casting location scenery story direction everyone's really suited the part they played and you could just imagine being there robert* $\langle UNK \rangle$ *is an amazing actor and now the same being director* $\langle UNK \rangle$ *father came from the same scottish island as myself so i loved ...*

这里我们可以看到许多单词已被省略，一些未知单词（UNK）已被标记为如此。经过这种缩减，二值特征向量的长度为10,000，并且大部分由0组成，只有少量1散布在对应于文档中出现的单词的位置上。我们有一个训练集和一个测试集，每个都有25,000个示例，并且每个都相对于`sentiment`保持平衡。得到的训练特征矩阵$\mathbf{X}$的维度为25,000 $\times$ 10,000，但只有1.3%的二值条目是非零的。我们称这样的矩阵为稀疏矩阵，因为大多数值都是相同的（在这种情况下为零）；它可以高效地存储在*sparse matrix format*中。[14]

稀疏矩阵格式

有多种方法可以考虑文档长度；这里我们仅将单词标记为出现在文档中或不在文档中，但例如也可以记录单词的相对频率。我们从25,000个训练观测中分出一个大小为2,000的验证集（用于模型调优），并拟合两个模型序列:

- 使用 `glmnet` 包的套索逻辑回归;
- 一个二分类神经网络，包含两个隐藏层，每层有16个ReLU单元。

[14] Rather than store the whole matrix, we can store instead the location and values for the nonzero entries. In this case, since the nonzero entries are all 1, just the locations are stored.

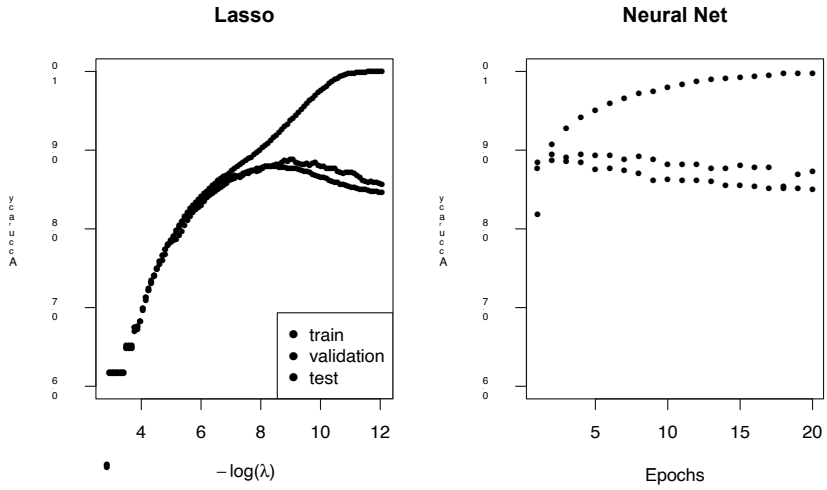


图 10.11. *Accuracy of the lasso and a two-hidden-layer neural network on the* `IMDb` *data. For the lasso, the x-axis displays* $-\log(\lambda)$*, while for the neural network it displays epochs (number of times the fitting algorithm passes through the training set). Both show a tendency to overfit, and achieve approximately the same test accuracy.*

两种方法都会产生一系列解。套索序列由正则化参数λ索引。神经网络序列由拟合过程中使用的梯度下降迭代次数索引，其衡量方式为训练轮数或遍历训练集的次数（第10.7节）。注意，图10.11中的训练准确率（黑点）在两种情况下均单调增加。我们可以利用验证误差从每个序列中挑选出一个好的解（图中的蓝点），然后用它来对测试数据集进行预测。

注意，两类神经网络相当于一个非线性逻辑回归模型。从(10.12)和(10.13)我们可以看出

$$\begin{aligned}\log\left(\frac{\Pr(Y=1|X)}{\Pr(Y=0|X)}\right) &= Z_1 - Z_0 \qquad (10.15)\\ &= (\beta_{10}-\beta_{00}) + \sum_{\ell=1}^{K_2}(\beta_{1\ell}-\beta_{0\ell})A_\ell^{(2)}.\end{aligned}$$

(这显示了softmax函数中的冗余性；对于 K 个类别，我们实际上只需要估计 $K-$ 组系数中的1组。参见第4.3.5节。) 在图10.11中，我们展示的是 *accuracy* (正确率) 而不是分类错误（分数不正确），前者在机器学习社区中更受欢迎。两个模型在测试集上的准确率均约为88%。

accuracy

词袋模型通过文档中出现的词来概括文档，并忽略它们的上下文。至少有兩種流行的方法可以考虑上下文：

- *bag-of-n-grams* 模型。例如，一个 2-gram 的词袋记录

bag-of-n-grams

每对不同单词的连续共现。在电影评论中，“异常长”可视为正面短语，而“异常短”则为负面。

- 将文档视为一个序列，考虑上下文中所有前后出现的词语。

在下一节中，我们将探讨用于数据序列的模型，这些模型在天气预报、语音识别、语言翻译和时间序列预测等领域有着广泛的应用。我们将继续以这个IMDb例子进行说明。

10.5 循环神经网络

许多数据源本质上是序列数据，在构建预测模型时需要特殊处理。例如：

- 书籍和电影评论、报纸文章、推文等文档。文档中单词的顺序和相对位置捕捉了叙述、主题和语气，可以在主题分类、情感分析和语言翻译等任务中加以利用。

- 温度、降雨量、风速、空气质量等的时间序列。我们可能希望预报未来几天的天气，或者未来几十年的气候。

- 金融时间序列，我们追踪市场指数、交易量、股票和债券价格以及汇率。这里的预测通常很困难，但正如我们将看到的，某些指数可以以合理的精度预测。

- 录制的语音、音乐录音和其他声音录音。我们可能希望给出演讲的文本转录，或者语言翻译。我们可能希望评估一段音乐的质量，或者赋予某些属性。

- 手写内容，例如医生笔记，以及手写数字，如邮政编码。在这里，我们想把手写内容转换为数字文本，或读取数字（光学字符识别）。

循环神经网络

在一个*recurrent neural network* (RNN)中，输入对象X是一个*sequence*。考虑一个文档语料库，例如IMDb电影评论的集合。每个文档可以表示为一个L单词的序列，因此$X = \{X_1, X_2, \ldots, X_L\}$，其中每个$X_\ell$代表一个单词。单词的顺序以及句子中某些单词的接近程度传达了语义。RNN旨在适应并利用此类输入对象的序列性质，就像卷积神经网络适应图像输入的空间结构一样。输出Y也可以是一个序列（例如在语言翻译中），但通常是一个标量，比如电影评论文档的二元情感标签。

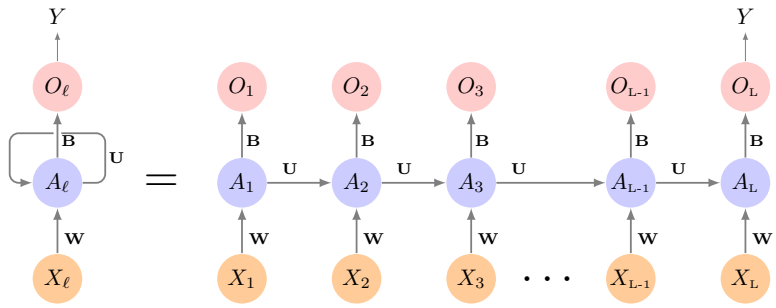


图10.12. *Schematic of a simple recurrent neural network. The input is a sequence of vectors* $\{X_\ell\}_1^L$*, and here the target is a single response. The network processes the input sequence* X *sequentially; each* X_ℓ *feeds into the hidden layer, which also has as input the activation vector* $A_{\ell-1}$ *from the previous element in the sequence, and produces the current activation vector* A_ℓ*. The same collections of weights* $\mathbf{W}$*,* $\mathbf{U}$ *and* $\mathbf{B}$ *are used as each element of the sequence is processed. The output layer produces a sequence of predictions* O_ℓ *from the current activation* A_ℓ*, but typically only the last of these,* O_L*, is of relevance. To the left of the equal sign is a concise representation of the network, which is* 展开 *into a more explicit version on the right.*

图10.12展示了一个非常基本的RNN结构，其输入为序列 $X = \{X_1, X_2, \ldots, X_L\}$，简单输出为$Y$，隐藏层序列为 $\{A_\ell\}_1^L = \{A_1, A_2, \ldots, A_L\}$。每个$X_\ell$都是一个向量；在文档示例中，$X_\ell$可以代表基于语料库语言词典的第$\ell$个词的单热编码（参见图10.13顶部的简单示例）。当序列一次处理一个向量X_ℓ时，网络更新隐藏层中的激活A_ℓ，其输入为向量X_ℓ和序列中上一步的激活向量$A_{\ell-1}$。每个A_ℓ输入到输出层并产生对Y的预测O_ℓ。O_L，其中最后一个是最相关的。

具体而言，假设输入序列的每个向量 X_ℓ 有 p 个分量 $X_\ell^T = (X_{\ell 1}, X_{\ell 2}, \ldots, X_{\ell p})$，而隐藏层由 K 个单元 $A_\ell^T = (A_{\ell 1}, A_{\ell 2}, \ldots, A_{\ell K})$ 组成。如图 10.4 所示，我们将输入层的 $K \times (p+1)$ 个共享权重 w_{kj} 的集合表示为矩阵 $\mathbf{W}$，类似地，$\mathbf{U}$ 是隐藏层到隐藏层权重的 $K \times K$ 矩阵 u_{ks}，而 $\mathbf{B}$ 是输出层权重的 $K+1$ 向量 β_k。然后

$$A_{\ell k} = g\Big(w_{k0} + \sum_{j=1}^{p} w_{kj} X_{\ell j} + \sum_{s=1}^{K} u_{ks} A_{\ell-1,s}\Big), \tag{10.16}$$

并且输出 O_ℓ 计算为

$$O_\ell = \beta_0 + \sum_{k=1}^{K} \beta_k A_{\ell k} \tag{10.17}$$

对于定量响应，或者对于二分类响应则附加一个sigmoid激活函数，例如。这里$g(\cdot)$是一个激活函数，如ReLU。注意，相同的权重$\mathbf{W}$、$\mathbf{U}$和$\mathbf{B}$被使用了，正如我们

按顺序处理序列中的每个元素，即它们不是 ℓ 的函数。这是 RNN 使用的一种 *weight sharing* 形式，类似于滤波器的使用在卷积神经网络中（第10.3.1节），随着我们从开始到结束的推进，激活值A_ℓ累积了先前所见内容的历史，从而使学习到的上下文能够用于预测。

weight sharing

对于回归问题，一个观测值 (X, Y) 的损失函数是

$$(Y - O_L)^2, \tag{10.18}$$

它仅引用最终输出 $O_L = \beta_0 + \sum_{k=1}^{K} \beta_k A_{Lk}$。因此，$O_1, O_2, \ldots, O_{L-1}$ 未被使用。当我们拟合模型时，输入序列 X 的每个元素 X_ℓ 通过链 (10.16) 对 O_L 做出贡献，因此通过损失 (10.18) 间接地对学习共享参数 $\mathbf{W}$、$\mathbf{U}$ 和 $\mathbf{B}$ 做出贡献。对于 n 个输入序列/响应对（x_i, y_i）,，参数通过最小化平方和来求得

$$\sum_{i=1}^{n}(y_i - o_{iL})^2 = \sum_{i=1}^{n}\Big(y_i - \big(\beta_0 + \sum_{k=1}^{K}\beta_k g\big(w_{k0} + \sum_{j=1}^{p} w_{kj}x_{iLj} + \sum_{s=1}^{K} u_{ks}a_{i,L-1,s}\big)\big)\Big)^2. \tag{10.19}$$

这里我们使用小写字母来表示观测到的 y_i 和向量序列 $x_i = \{x_{i1}, x_{i2}, \ldots, x_{iL}\}$、[15] 以及导出的激活值。

由于中间输出O_ℓ未被使用，人们自然会问它们为何存在。首先，它们是无代价的，因为它们使用了生成O_L所需的相同输出权重$\mathbf{B}$，并为输出提供了一种逐步演进的预测。此外，对于某些学习任务，响应也是一个序列，因此输出序列$\{O_1, O_2, \ldots, O_L\}$是明确需要的。

当完全使用时，递归神经网络可以相当复杂。我们通过两个简单的应用来说明它们的用途。在第一个应用中，我们继续上一节的IMDb情感分析，其中我们按顺序处理评论中的单词。在第二个应用中，我们展示了它们在金融时间序列预测问题中的应用。

10.5.1 Sequential Models for Document Classification

这里我们回到带有IMDb条评论的分类任务。第10.4节中的方法是使用词袋模型。这里的计划是改用文档中出现的单词序列来对整个文档的标签进行预测。

然而，我们面临一个维度问题：文档中的每个单词都由一个独热编码向量（哑变量）表示，该向量包含10,000个元素（字典中每个单词对应一个元素）！一种流行的方法是每个单词用一个更低维的*embedding*来表示。

embedding

空间。这意味着，与其用包含9999个零和某个位置上一个1的二进制向量来表示每个词，我们将转而用一组m个实数来表示它，这些数通常都不为零。这里，m是嵌入维度，可能低至几百，甚至更少。这意味着（在我们的案例中）我们需要一个维度为$m \times$×10,000的矩阵$\mathbf{E}$,

[15] This is a sequence of vectors; each element $x_{i\ell}$ is a p-vector.

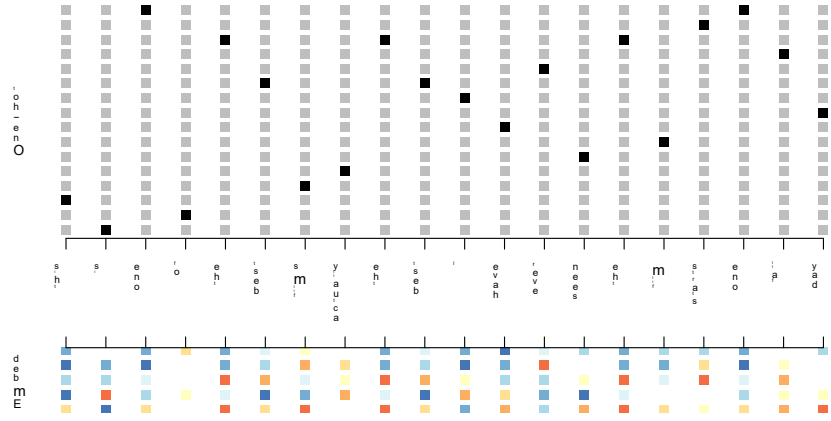


图 10.13. *Depiction of a sequence of* 20 *words representing a single document: one-hot encoded using a dictionary of* 16 *words (top panel) and embedded in an* m*-dimensional space with* $m = 5$ *(bottom panel).*

其中每一列由我们词典中的10,000个词之一索引，该列中的值给出了该词在嵌入空间中的m坐标。

图10.13展示了这一思想（使用包含16个而非10,000个词的词典，且$m = 5$）。**E**从何而来？如果我们拥有大量标注文档的语料库，我们可以将神经网络*learn* **E**作为优化的一部分。在这种情况下，**E**被称为*embedding layer*，并且针对当前任务学习一个专门的**E**。否则，我们可以在嵌入层插入一个预计算的矩阵**E**，这一过程称为*weight freezing*。两个预训练的嵌入，`word2vec`和`GloVe`，是被广泛使用。[16] 这些是通过主成分分析的一种变体（第12.2节）从非常大型的文档语料库中构建的。其理念是，词在嵌入空间中的位置保留了语义含义；例如，同义词应该彼此靠近。

embedding layer

weight freezing

`word2vec`

`GloVe`

到目前为止，一切顺利。每份文档现在被表示为一个由m向量组成的序列，这些向量代表了单词的顺序。下一步是将每份文档限制为最后的L个单词。短于L的文档会在前面用零填充。因此，现在每份文档由一个包含L个向量$X = \{X_1, X_2, \ldots, X_L\}$的序列表示，序列中的每个$X_\ell$都有$m$个分量。

我们现在使用图10.12中的RNN结构。训练语料库由n个独立的序列（文档）组成，每个序列长度为L，它们均按从左至右的顺序依次处理。在此过程中，针对每个文档，会如同(10.16)所示，生成一系列并行的隐藏激活向量A_ℓ，$\ell = 1, \ldots, L$。A_ℓ输入到输出层，产生动态预测O_ℓ。我们采用最终值O_L来预测响应：评论的情感倾向。

[16] `word2vec` is described in Mikolov, Chen, Corrado, and Dean (2013), available at `https://code.google.com/archive/p/word2vec`. `GloVe` is described in Pennington, Socher, and Manning (2014), available at `https://nlp.stanford.edu/projects/glove`.

这是一个简单的RNN，参数相对较少。如果有K个隐藏单元，常见的权重矩阵$\mathbf{W}$有$K \times (m+1)$个参数，矩阵$\mathbf{U}$有$K \times K$个参数，而$\mathbf{B}$有2($K+1$)个参数，用于如(10.15)中的二分类逻辑回归。当我们从左到右处理序列$X = \{X_\ell\}_1^L$时，这些参数会被重复使用，很像我们在图像中处理每个补丁时使用单个卷积滤波器（第10.3.1节）。如果嵌入层$\mathbf{E}$是学习得到的，那将额外增加$m \times D$个参数（此处为$D = 10{,}000$），并且是迄今为止最大的开销。

我们将按照图10.12及配套文本所述，对IMDb数据拟合RNN。模型包含一个嵌入矩阵$\mathbf{E}$，其m =为32（该矩阵在训练中学习得到，而非预先计算），随后是一个具有K =32个隐藏单元的单一循环层。该模型在指定训练集的25,000条评论上采用dropout正则化进行训练，在IMDb测试数据上仅达到了令人失望的76%准确率。而使用GloVe预训练嵌入矩阵$\mathbf{E}$的网络表现略差。

为便于阐述，我们展示了一个非常简单的RNN。更精细的版本使用了*long term*和*short term*记忆（LSTM）。维护了两条隐藏层激活轨迹，这样当计算激活A_ℓ时，它既从时间上更靠后的隐藏单元获取输入，也从时间上更接近的隐藏单元获取输入——即所谓的*LSTM RNN*。对于长序列，这克服了早期信号在通过链传播到最终的激活向量 A_L 时被冲淡的问题。

LSTM RNN

当我们使用 LSTM 架构对隐藏层重新拟合模型时，在 IMDb 测试数据上的性能提升到了 87%。这与第 10.4 节中词袋模型达到的 88% 相当。我们在第 10.9.6 节中给出了拟合这些模型的细节。

尽管增加了LSTM的复杂性，我们的RNN仍处于较为“入门级”的水平。我们或许可以通过调整模型大小、改变正则化方法以及增加更多隐藏层来获得稍好一些的结果。然而，LSTM模型的训练耗时较长，这使得探索多种架构和参数优化变得繁琐。

RNN为数据序列建模提供了丰富的框架，并且它们持续演进。RNN的发展取得了许多进展——在架构、数据增强以及学习算法方面。在撰写本文时（2020年初），领先的RNN配置在IMDb数据上报告的准确率超过95%。详细内容超出了本书的范围。[17]

10.5.2 Time Series Forecasting

图10.14展示了纽约证券交易所的历史交易统计数据。图中显示了三组日度时间序列，覆盖了从1962年12月3日到1986年12月31日的时间段：[18]

[17] An IMDb leaderboard can be found at https://paperswithcode.com/sota/sentiment-analysis-on-imdb.

[18] These data were assembled by LeBaron and Weigend (1998) *IEEE Transactions on Neural Networks*, 9(1): 213–220.

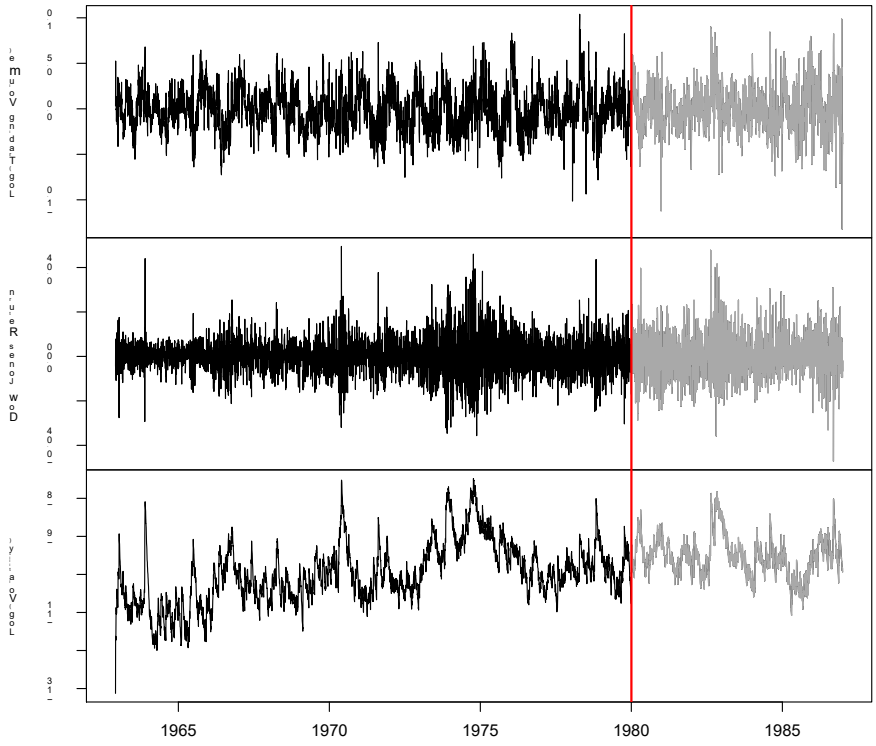


图 10.14. *Historical trading statistics from the New York Stock Exchange. Daily values of the normalized log trading volume, DJIA return, and log volatility are shown for a 24-year period from 1962–1986. We wish to predict trading volume on any day, given the history on all earlier days. To the left of the red bar (January 2, 1980) is training data, and to the right test data.*

- `Log trading volume`这是当日交易的所有流通股占过去100天平均换手率的比例，取对数。
- `Dow Jones return`. 这是道琼斯工业指数在连续交易日对数之间的差值。
- `Log volatility`。这是基于每日价格变动的绝对值。

预测股票价格是一个众所周知很难的问题，但事实证明，根据最近的过去历史来预测交易量则更为可行（且对于规划交易策略很有用）。

这里的观测包括第t天的测量值（v_t, r_t, z_t），在本例中为`log_volume`、`DJ_return`和`log_volatility`的值。总共有T =个6,051这样的三元组，每个三元组都在图10.14中绘制为时间序列。我们立即注意到的一个特征是，逐日的观测值彼此并不独立。这些序列表现出*auto-correlation*——在这种情况下，时间上相近的值往往相似

auto-correlation

彼此之间。这将时间序列与我们所遇到的其他数据集区分开来，在其他数据集中，观测值可以被假定为相互独立的。

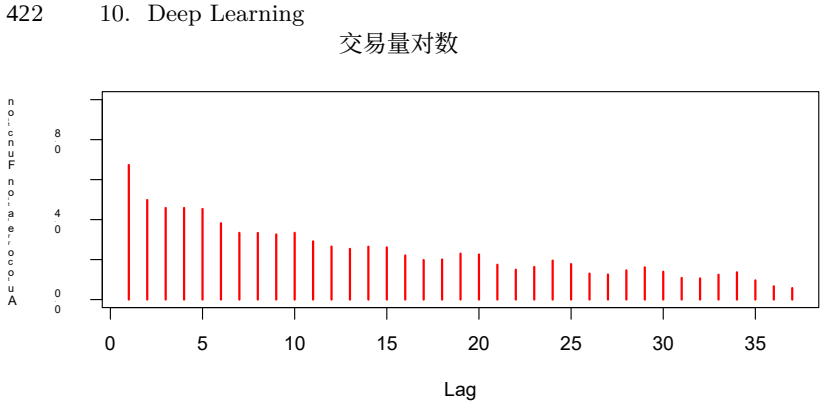


图10.15。 *The autocorrelation function for* `log_volume`. *We see that nearby values are fairly strongly correlated, with correlations above* 0.2 *as far as 20 days apart.*

彼此之间。为明确起见，考虑成对的观测值（$v_t, v_{t-\ell}$），即相隔 ℓ 滞后天数的 *lag*。如果我们取 v_t 序列中所有这样的对，并计算它们的相关系数，就得到了滞后 ℓ 处的自相关。图10.15展示了滞后高达37的所有滞后处的自相关函数，我们可以看到存在显著的相关性。

这个预测问题的另一个有趣特点是，响应变量v_t——`log_volume`——也是一个预测因子！特别是，我们将使用`log_volume`的过去值来预测未来的值。

RNN预测器

我们希望根据过去的值 $v_{t-1}, v_{t-2}, \ldots$ 预测一个值 v_t，并利用其他序列 $r_{t-1}, r_{t-2}, \ldots$ 和 $z_{t-1}, z_{t-2}, \ldots$ 的过去值。尽管我们的合并数据是一个相当长的序列，包含 6,051 个交易日，但问题的结构与之前的文档分类示例不同。

- 我们只有一系列数据，而不是25，000个。
- 我们有一整个目标 v_t 的 *series*，并且输入包含该序列的过去值。

我们如何根据图10.12所示的结构来表示这个问题？其思路是提取许多短的输入序列迷你系列$X = \{X_1, X_2, \ldots, X_L\}$，具有预定义长度$L$（，称为*lag* la g 在此上下文中），以及相应的目标Y。它们的形式为

$$X_1 = \begin{pmatrix} v_{t-L} \\ r_{t-L} \\ z_{t-L} \end{pmatrix}, \; X_2 = \begin{pmatrix} v_{t-L+1} \\ r_{t-L+1} \\ z_{t-L+1} \end{pmatrix}, \cdots, X_L = \begin{pmatrix} v_{t-1} \\ r_{t-1} \\ z_{t-1} \end{pmatrix}, \quad \text{and } Y = v_t. \tag{10.20}$$

因此，这里目标 Y 是 `log_volume` v_t 在单个时间点 t 的值，而输入序列 X 是 3 维向量 $\{X_\ell\}_1^L$ 的序列，每个向量由来自第 $t-L$、$t-L+1$ 天，至 $t-1$ 天的三个测量值 `log_volume`、`DJ_return` 和 `log_volatility` 组成。t 的每个值都构成了一个单独的（X, Y）对，其中 t 从 $L+1$ 取到 T。对于 `NYSE` 数据，我们将使用过去

测试期：观测值与预测值

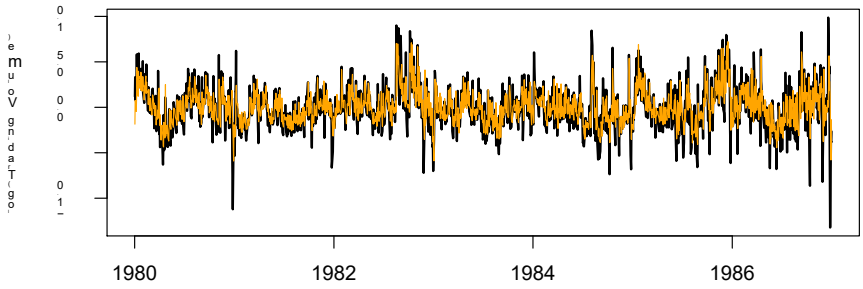


图 10.16. *RNN forecast of* `log_volume` *on the* `NYSE` *test data. The black lines are the true volumes, and the superimposed orange the forecasts. The forecasted series accounts for 42% of the variance of* `log_volume`.

五个交易日来预测次日的交易量。因此，我们使用$L = 5$。由于$T = 6{,}051$，我们可以创建6,046个这样的(X, Y)对。显然，L是一个应谨慎选择的参数，或许可以使用验证数据。

我们用$K = 12$个隐藏单元拟合该模型，使用的是1980年1月2日之前数据衍生出的4,281个训练序列（见图10.14），随后用此模型预测了该日期之后的1,770个`log_volume`值。在测试数据上，我们达到了$R^2 = 0.42$。详情见第10.9.6节。作为对比，*straw man*，使用昨日`log_volume`值作为今日预测的[19]，其R^2 =为0.18。图10.16展示了预测结果。我们用黑色标出了1980-1986年测试期间每日`log_volume`的观测值，并在此之上叠加了橙色预测序列。两者的吻合度看起来相当好。

在预测测试期`log_volume`的值时，我们必须利用测试数据本身来构成输入序列X。这或许让人感觉像是在作弊，但实际上并非如此；我们始终是在利用过去的数据来预测未来。

自回归

我们刚刚拟合的RNN与传统*autoregression*自回归（AR）线性模型有许多共同之处，我们现在提出该模型以作比较。我们首先单独考虑响应序列v_t，并构建一个响应向量$\mathbf{y}$和一个预测变量矩阵$\mathbf{M}$，以进行最小二乘回归，具体如下：

$$\mathbf{y} = \begin{bmatrix} v_{L+1} \\ v_{L+2} \\ v_{L+3} \\ \vdots \\ v_T \end{bmatrix} \qquad \mathbf{M} = \begin{bmatrix} 1 & v_L & v_{L-1} & \cdots & v_1 \\ 1 & v_{L+1} & v_L & \cdots & v_2 \\ 1 & v_{L+2} & v_{L+1} & \cdots & v_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & v_{T-1} & v_{T-2} & \cdots & v_{T-L} \end{bmatrix}. \tag{10.21}$$

$\mathbf{M}$ 并且$\mathbf{y}$各自有$T - L$行，每行对应一个观测值。我们看到，对于第t天的任意给定响应v_t，其预测变量是前L个值

[19] A straw man here refers to a simple and sensible prediction that can be used as a baseline for comparison.

的同一系列。对$\mathbf{y}$关于$\mathbf{M}$进行回归拟合相当于拟合模型

$$\hat{v}_t = \hat{\beta}_0 + \hat{\beta}_1 v_{t-1} + \hat{\beta}_2 v_{t-2} + \cdots + \hat{\beta}_L v_{t-L}, \tag{10.22}$$

被称为L阶自回归模型，或简称为AR(L)。对于NYSE数据，我们可以在预测矩阵$\mathbf{M}$中包含DJ_return和log_volatility、r_t和z_t的滞后版本，从而得到3$L+1$列。一个$L=$为5的AR模型的测试R^2为0.41，略逊于RNN所达到的0.42。

当然，RNN 和 AR 模型非常相似。它们都使用相同的响应 Y 和长度为 $L = 5$、维度为 $p = 3$ 的输入序列 X。RNN 使用与输入层) 相同的权重 $\mathbf{W}$ (从左到右处理该序列，而 AR 模型则简单地将序列中的所有 L 个元素平等地视为一个包含 $L \times p$ 个预测变量的向量——这一过程在神经网络文献中被称为 *flattening*。

flattening

当然，RNN 还包括隐藏层激活 A_ℓ，这些激活沿序列传递信息，并引入了额外的非线性。由(10.19)式，在 $K = 12$ 个隐藏单元的情况下，我们看到 RNN 有 $13 + 12 \times (1 + 3 + 12) = 205$ 个参数，相比之下 AR(5) 模型有 16 个参数。

AR 模型的一个明显扩展是将滞后预测变量集作为普通前馈神经网络（10.1）的输入向量，从而增加更多灵活性。这达到了测试 $R^2 = 0.42$，略优于线性 AR，并且与 RNN 相同。

所有模型都可以通过引入变量 {v*} 来改进，该变量对应于目标 v_t (的第 t 天，可以从数据中提供的日历日期) 学习得到；交易量通常在周一和周五较高。由于有五个交易日，这会独热编码为五个二元变量。AR 模型的性能提升到 $R^2 = 0.46$，RNN 也是如此，非线性 AR 模型提升到 $R^2 = 0.47$。

我们在此处的示例中使用了最简单版本的RNN。使用RNN的LSTM扩展进行的额外实验带来了小幅改进，在这些示例中，R^2通常最多提升1%。

我们在第10.9.6节中详细介绍了我们如何拟合这三个模型。

10.5.3 Summary of RNNs

我们通过两个简单的用例说明了RNN，但这只是冰山一角。

我们用于序列建模的简单RNN有许多变体和增强。我们没有讨论的一种方法使用一维卷积神经网络，将向量序列（例如在嵌入空间中表示的单词）视为图像。卷积滤波器以一维方式沿序列滑动，有可能学习与学习任务相关的特定短语或短子序列。

在RNN中，还可以拥有额外的隐藏层。例如，具有两个隐藏层时，序列A_ℓ以显而易见的方式被视为下一隐藏层的输入序列。

我们使用的RNN从头到尾扫描文档；替代方案*bidirectional*RNN则双向扫描序列。

bidirectional

在语言翻译中，目标也是单词序列，其语言与输入序列不同。输入序列和目标序列都由图10.12类似的结构表示，并且它们共享隐藏单元。在这个所谓的*Seq2Seq*中

Seq2Seq

在学习过程中，隐藏单元被认为捕捉了句子的语义含义。语言建模和翻译领域的一些重大突破，正是源于此类RNN相对近期的改进。

用于拟合RNN的算法可能复杂且计算成本高昂。幸运的是，优秀的软件在一定程度上保护用户免受这些复杂性的困扰，并使指定和拟合这些模型变得相对轻松。我们在日常生活中享用的许多模型（如*Google Translate*）使用了由高技能工程师团队开发的最先进架构，并且已经使用大量的计算和数据资源进行了训练。

10.6 何时使用深度学习

本章中深度学习的表现相当令人印象深刻。它解决了数字分类问题，而深度卷积神经网络确实彻底改变了图像分类。我们每天都能看到关于深度学习新成功案例的报道。其中许多与图像分类任务相关，例如对乳房X光片或数字X射线图像的机器诊断、眼科眼部扫描、MRI扫描的标注等等。同样，循环神经网络在语音和语言翻译、预测以及文档建模方面也取得了众多成功。那么，随之而来的一个亟待回答的问题是：
should we discard all our older tools, and use deep learning on every problem with data? 为了回答这个问题，我们重新审视第6章中的`Hitters`数据集。

这是一个回归问题，目标是根据棒球运动员1986年的表现统计数据，预测其在1987年的`Salary`。移除响应缺失的运动员后，我们剩下263名运动员和19个变量。我们将数据随机分为包含176名运动员（三分之二）的训练集和包含87名运动员（三分之一）的测试集。我们使用了三种方法来拟合这些数据的回归模型。

- 使用线性模型对训练数据进行拟合，并对测试数据进行预测。该模型有20个参数。
- 使用lasso正则化拟合了相同的线性模型。通过训练数据上的10折交叉验证选择了调优参数。它选择了一个具有12个非零系数变量的模型。
- 一个具有一个隐藏层的神经网络，该隐藏层由64个`ReLU`单元组成，被拟合到数据上。这个模型有1,345个参数。[20]

[20]The model was fit by stochastic gradient descent with a batch size of 32 for 1,000 epochs, and 10% dropout regularization. The test error performance flattened out and started to slowly increase after 1,000 epochs. These fitting details are discussed in Section 10.7.

Model	# Parameters	Mean Abs. Error	Test Set R^2
Linear Regression	20	254.7	0.56
Lasso	12	252.3	0.51
Neural Network	1345	257.4	0.54

表 10.2. *Prediction results on the* `Hitters` *test data for linear models fit by ordinary least squares and lasso, compared to a neural network fit by stochastic gradient descent with dropout regularization.*

	Coefficient	Std. error	t-statistic	p-value
`Intercept`	-226.67	86.26	-2.63	0.0103
`Hits`	3.06	1.02	3.00	0.0036
`Walks`	0.181	2.04	0.09	0.9294
`CRuns`	0.859	0.12	7.09	< 0.0001
`PutOuts`	0.465	0.13	3.60	0.0005

表10.3. *Least squares coefficient estimates associated with the regression of* `Salary` *on four variables chosen by lasso on the* `Hitters` *data set. This model achieved the best performance on the test data, with a mean absolute error of 224.8. The results reported here were obtained from a regression on the test data, which was not used in fitting the lasso model.*

表10.2比较了结果。我们看到三种模型的性能相似。我们报告了测试数据上的平均绝对误差，以及每种方法的测试R^2，这些都是可接受的（见练习5）。为了达到这些结果，我们花费了不少时间调整神经网络的配置参数。有可能如果我们投入更多时间，并且恰当地调整正则化的形式和量，我们或许能够匹配甚至超越线性回归和套索方法。但非常轻松地，我们获得了表现良好的线性模型。线性模型比神经网络更容易呈现和理解，后者本质上是一个黑盒。套索方法在进行预测时选择了19个变量中的12个。因此，在这种情况下，我们遵循*Occam's razor*原则会好得多：当面对多种方法时

Occam's razor

在性能大致相当的情况下，选择最简单的。

在对lasso模型进行更多探索后，我们发现了一个更简单的包含四个变量的模型。随后，我们使用这四个变量重新拟合了线性模型到训练数据（即所谓的*relaxed lasso*），并取得了224.8的测试平均绝对误差，成为总体最优模型！展示这个拟合的汇总表很诱人，这样我们可以看到系数和p值；然而，由于模型是基于训练数据选择的，这将存在*selection bias*。相反，我们在未用于选择过程的测试数据上重新拟合了模型。表10.3展示了结果。

我们手头有许多非常强大的工具可供使用，包括神经网络、随机森林和提升方法、支持向量机以及广义加性模型等，仅举几例。此外，我们还有线性模型及其简单变体。面对新的数据建模和预测问题时，人们往往倾向于总是选择那些时髦的新方法。通常，它们能给出极其令人印象深刻的结果，尤其是在数据集非常大，足以支持高维非线性模型拟合的情况下。然而，*if*我们可以使用更简单的工具来构建模型，这些模型

同样表现良好时，它们往往更易于拟合和理解，且相比更复杂的方法可能更为稳健。在可能的情况下，尝试简单模型也是有道理的，然后根据性能与复杂度的权衡做出选择。

通常，当训练集的样本量极大，且模型的可解释性不是首要考虑因素时，我们期望深度学习成为一个有吸引力的选择。

10.7 拟合神经网络



拟合神经网络有些复杂，我们在此简要概述。这些理念可推广到更为复杂的网络。若读者觉得这部分内容有难度，完全可以跳过。幸运的是，正如我们在本章末尾的实验中所见，已有优秀的软件能以相对自动化的方式拟合神经网络模型，无需担心模型拟合过程的技术细节。

我们从第10.1节图10.1中描绘的简单网络开始。在模型(10.1)中，参数为$\beta = (\beta_0, \beta_1, \ldots, \beta_K)$，以及每个$w_k = (w_{k0}, w_{k1}, \ldots, w_{kp})$, $k = 1, \ldots, K$. 给定观测值(x_i, y_i), $i = 1, \ldots, n$,，我们可以通过求解非线性最小二乘问题来拟合该模型。

$$\underset{\{w_k\}_1^K,\, \beta}{\text{minimize}} \frac{1}{2} \sum_{i=1}^{n} (y_i - f(x_i))^2, \tag{10.23}$$

其中

$$f(x_i) = \beta_0 + \sum_{k=1}^{K} \beta_k g\Big(w_{k0} + \sum_{j=1}^{p} w_{kj} x_{ij}\Big). \tag{10.24}$$

(10.23) 中的目标看起来足够简单，但由于参数的嵌套排列和隐藏单元的对称性，直接最小化并不简单。该问题在参数上是非凸的，因此存在多个解。例如，图 10.17 展示了一个关于单变量 θ 的简单非凸函数；存在两个解：一个是 *local minimum*，另一个是 *global minimum*。进

局部最小值 全局最小值

此外，（10.1）是最简单的神经网络；在本章中，我们介绍了更为复杂的网络，其中这些问题变得更加错综复杂。为了克服其中一些难题并防止过拟合，在拟合神经网络时采用了两种通用策略。

- *Slow Learning:* 该模型以某种较慢的迭代方式拟合，使用了 *gradient descent*。随后，在以下情况时停止拟合过程：检测到过拟合。

梯度下降

- *Regularization:* 对参数施加惩罚，通常是套索或岭回归，如第6.2节所述。

假设我们将所有参数表示为一个长向量θ。那么我们可以将(10.23)中的目标重写为

$$R(\theta) = \frac{1}{2} \sum_{i=1}^{n} (y_i - f_\theta(x_i))^2, \tag{10.25}$$

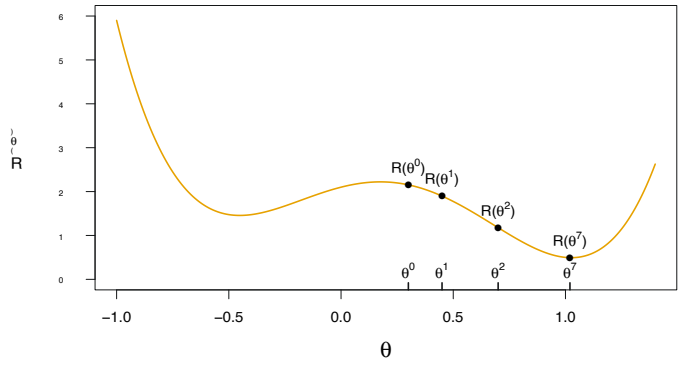


图 10.17. *Illustration of gradient descent for one-dimensional* θ*. The objective function* $R(\theta)$ *is not convex, and has two minima, one at* $\theta = -0.46$ *(local), the other at* $\theta = 1.02$ *(global). Starting at some value* θ^0 *(typically randomly chosen), each step in* θ *moves downhill — against the gradient — until it cannot go down any further. Here gradient descent reached the global minimum in 7 steps.*

在这里，我们明确表示 f 对参数的依赖关系。梯度下降的思想非常简单。

1. 从对θ中所有参数的猜测θ^0开始，并设置t =为0。
2. 迭代直到目标函数(10.25)不再下降：

   (a) 找到一个向量 δ，它反映了 θ 的小变化，使得 $\theta^{t+1} = \theta^t + \delta$ *reduces* 目标函数；即满足 $R(\theta^{t+1}) < R(\theta^t)$。(b) 设 $t \leftarrow t +$ 为 1。

可以想象（图10.17）站在山地地形中，目标是通过一系列步骤到达底部。只要每一步都是下坡，我们最终必定会到达底部。在这种情况下，我们是幸运的，因为从初始猜测 {v*} 出发，我们最终到达了全局最小值。一般来说，我们只能希望最终到达一个（好的）局部最小值。

10.7.1 Backpropagation

我们如何找到移动 θ 的方向，以便减少 (10.25) 中的目标函数 $R(\theta)$？ $R(\theta)$ 的 *gradient*，在当前值 $\theta = \theta^m$ 处评估，
是该点处的偏导数向量：

gradient

$$\nabla R(\theta^m) = \frac{\partial R(\theta)}{\partial \theta}\Big|_{\theta=\theta^m}. \tag{10.26}$$

下标 $\theta = \theta^m$ 表示在计算导数向量后，我们在当前猜测值 θ^m 处求值。这给出了在 θ 空间中 $R(\theta)$ *increases* 最快的方向。梯度下降的思想是将 θ 沿 *opposite* 方向移动一小步（因为我们希望下坡）：

$$\theta^{m+1} \leftarrow \theta^m - \rho \nabla R(\theta^m). \tag{10.27}$$

对于足够小的 *learning rate* ρ 值，此步骤将减小目标 $R(\theta)$；即 $R(\theta^{m+1}) \leq R(\theta^m)$。如果梯度向量为零，那么我们可能已经到达目标的最小值。

学习率

计算(10.26)有多复杂？事实证明，这里相当简单，即使对于更复杂的网络也依然简单，这得益于微分的*chain rule*。

链式法则

由于 {v*} 是一个和，其梯度也是对所有 n 观测值的和，因此我们只需考察其中一项，

$$R_i(\theta) = \frac{1}{2}\Big(y_i - \beta_0 - \sum_{k=1}^{K} \beta_k g\big(w_{k0} + \sum_{j=1}^{p} w_{kj} x_{ij}\big)\Big)^2. \tag{10.28}$$

为了简化后续表达式，我们记 $z_{ik} = w_{k0} + \sum_{j=1}^{p} w_{kj} x_{ij}$。首先我们对 β_k 求导：

$$\begin{aligned} \frac{\partial R_i(\theta)}{\partial \beta_k} &= \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial \beta_k} \\ &= -(y_i - f_\theta(x_i)) \cdot g(z_{ik}). \end{aligned} \tag{10.29}$$

现在我们对 w_{kj} 求导：

$$\begin{aligned} \frac{\partial R_i(\theta)}{\partial w_{kj}} &= \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial g(z_{ik})} \cdot \frac{\partial g(z_{ik})}{\partial z_{ik}} \cdot \frac{\partial z_{ik}}{\partial w_{kj}} \\ &= -(y_i - f_\theta(x_i)) \cdot \beta_k \cdot g'(z_{ik}) \cdot x_{ij}. \end{aligned} \tag{10.30}$$

请注意，这两个表达式都包含残差 $y_i - f_\theta(x_i)$。在（10.29）中，我们看到该残差的一部分根据 $g(z_{ik})$ 的值被归因于每个隐藏单元。然后在（10.30）中，我们看到通过隐藏单元 k 类似地归因于输入 j。因此，微分行为通过链式法则将残差的一部分分配给每个参数——这一过程在神经网络中被称为 *backpropagation*。

backpropagation

文献。尽管这些计算很直接，但需要仔细记录才能跟踪所有部分。

10.7.2 Regularization and Stochastic Gradient Descent

梯度下降通常需要许多步骤才能达到局部最小值。在实践中，有许多方法可以加速这一过程。此外，当n很大时，我们不必对所有n个观测值求和（10.29)–(10.30），而是每次可以抽取一小部分或*minibatch*个观测值

小批量

我们计算一个梯度步骤。这个过程被称为*stochastic gradient descent* (SGD)，是学习深度神经网络的最先进方法。

随机梯度下降

幸运的是，有非常优秀的软件可用于搭建深度学习模型并将其拟合到数据上，因此大多数技术细节对用户是隐藏的。

现在我们转向用于数字识别问题的多层网络（图10.4）。该网络拥有超过235,000个权重，大约是训练样例数量的四倍。在这里，正则化至关重要。

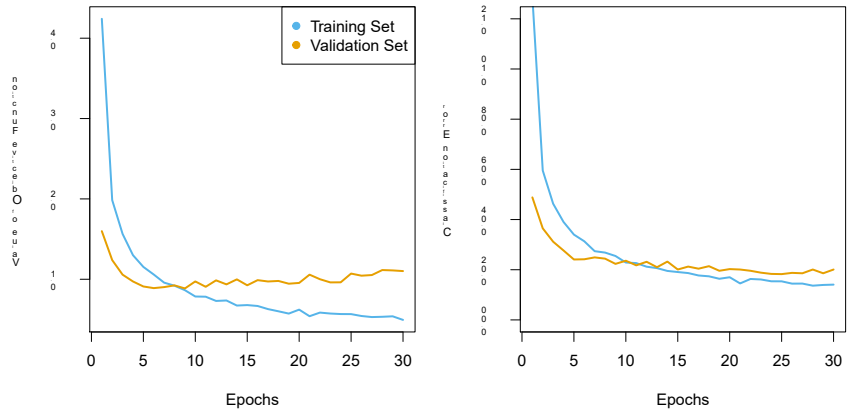


图 10.18. *Evolution of training and validation errors for the* `MNIST` *neural network depicted in Figure 10.4, as a function of training epochs. The objective refers to the log-likelihood (10.14).*

为了避免过拟合，表10.1中的第一行对权重采用了岭正则化。这是通过向目标函数(10.14)增加一个惩罚项来实现的：

$$R(\theta;\lambda) = -\sum_{i=1}^{n}\sum_{m=0}^{9} y_{im}\log(f_m(x_i)) + \lambda\sum_{j}\theta_j^2. \tag{10.31}$$

参数λ通常预设为一个较小的值，或者按照第5.3.1节的验证集方法来确定。我们还可以对不同层的权重组使用不同的λ值；在这种情况下，$\mathbf{W}_1$和$\mathbf{W}_2$受到惩罚，而输出层相对较少的权重$\mathbf{B}$则完全不受惩罚。套索正则化也常作为额外形式的正则化，或作为岭回归的替代方案而广受欢迎。

图10.18展示了在`MNIST`数据上训练网络过程中一些指标的变化情况。结果表明，SGD自然而然地实施了一种自身的近似二次正则化形式。[21]此处，每次梯度更新的小批量大小为128个观测值。标记为*epochs*的项

epochs

图 10.18 中的横轴计算了等价于完整训练集被处理的次数。对于这个网络，60,000 个训练观测值中的 20% 被用作验证集，以确定训练何时应停止。因此，实际上有 48,000 个观测值用于训练，因此每个 epoch 有 48,000/128 ≈ 375 次小批量梯度更新。我们看到，验证目标的值实际上在 30 个 epoch 时开始增加，因此 *early stopping* 也可以用作额外的

early stopping

正则化形式。

[21]This and other properties of SGD for deep learning are the subject of much research in the machine learning literature at the time of writing.

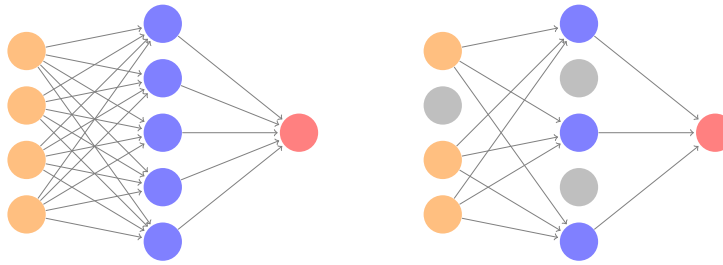


图 10.19. *Dropout Learning. Left: a fully connected network. Right: network with dropout in the input and hidden layer. The nodes in grey are selected at random, and ignored in an instance of training.*

10.7.3 Dropout Learning

表10.1中的第二行标记为*dropout*。这是一个相对较新的dropo
and efficient form of regularization, similar in some respects to ridge reg- ularization. Inspired by random forests (Section 8.2), the idea is to ran- domly remove a fraction ϕ of the units in a layer when fitting the model. Figure 10.19 illustrates this. This is done separately each time a training observation is processed. The surviving units stand in for those missing, and their weights are scaled up by a factor of $1/(1-\phi)$ to compensate. This prevents nodes from becoming over-specialized, and can be seen as a form of regularization. In practice dropout is achieved by randomly set- ting the activations for the "dropped out" units to zero, while keeping the architecture intact.

输出

10.7.4 Network Tuning

图10.4中的网络被认为是相对直接的；尽管如此，它仍需要做出一些对性能都有影响的选择：

- *The number of hidden layers, and the number of units per layer.* 现代观点认为，每个隐藏层的单元数量可以很大，并且可以通过各种形式的正则化来控制过拟合。

- *Regularization tuning parameters.* 这些包括dropout率 ϕ、lasso和岭正则化的强度 λ，并且通常在每一层分别设置。

- *Details of stochastic gradient descent.* 这些包括批量大小、迭代次数，以及如果使用的话，数据增强的细节（第10.3.4节）。

像这样的选择可能会产生影响。在准备这个 `MNIST` 示例时，经过一些试错，我们取得了相当不错的 1.8% 误分类误差。对类似网络进行更精细的调整和训练可以在这些数据上达到低于 1% 的误差，但反复调整的过程可能很繁琐，而且如果粗心大意，可能会导致过拟合。

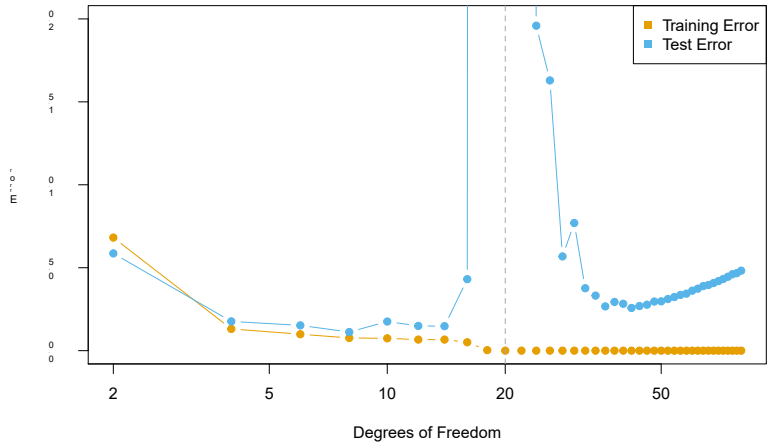


图 10.20. *Double descent phenomenon, illustrated using error plots for a one-dimensional natural spline example. The horizontal axis refers to the number of spline basis functions on the log scale. The training error hits zero when the degrees of freedom coincides with the sample size* $n = 20$*, the "interpolation threshold", and remains zero thereafter. The test error increases dramatically at this threshold, but then descends again to a reasonable value before finally increasing again.*

10.8 插值与双重下降

在本书中，我们多次讨论了偏差-方差权衡，首次提出于第2.2.2节。这一权衡表明，就测试集误差而言，统计学习方法在模型复杂度处于中等水平时往往表现最佳。特别地，如果我们以灵活性为x轴，误差为y轴作图，通常预期测试误差呈U形，而训练误差则单调递减。这种行为有两个“典型”例子，可见于第29页图2.9的右侧面板和第39页图2.17。偏差-方差权衡的一个含义是，通常不建议*interpolate*训练数据——即达到零训练误差——因为

interpolate

这通常会导致非常高的测试误差。

然而，事实证明，在某些特定情况下，一种对训练数据进行插值的统计学习方法可能表现良好——或者至少比一个稍微不那么复杂、不完全插值数据的模型表现更好。这种现象被称为*double descent*，并在图10.20中展示。“双下降”得名于测试误差在达到插值阈值之前呈U形，然后随着拟合越来越灵活的模型，它再次下降（至少在一段时间内）。

我们现在描述导致图10.20的设置。我们模拟了来自该模型的$n = 20$个观测值

$$Y = \sin(X) + \epsilon,$$

其中 $X \sim U[-5, 5]$（均匀分布），并且 $\epsilon \sim N(0, \sigma^2)$ 带有 $\sigma = 0.3$。然后我们按照第 7.4 节所述，使用 d 对数据拟合自然样条

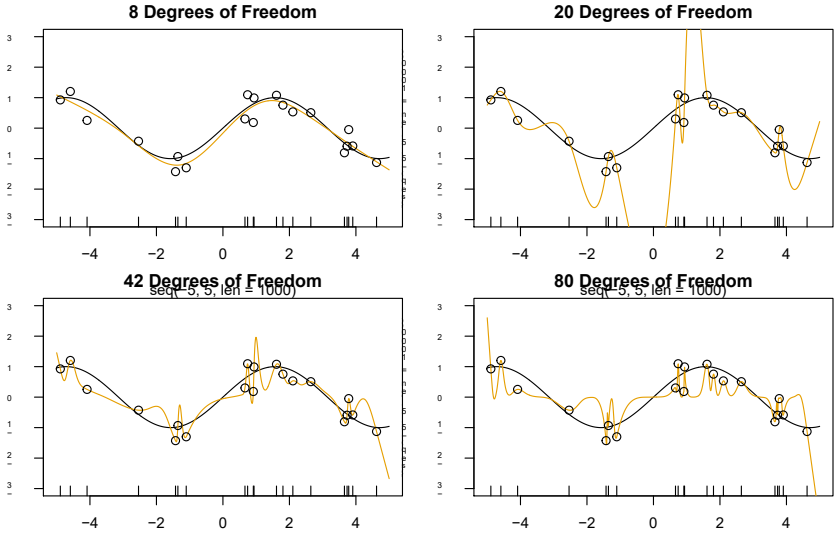


图 10.21. *Fitted functions* $\hat{f}_d(X)$ *(orange), true function* $f(X)$ *(black) and the observed* 20 *training data points. A different value of* d *(degrees of freedom) is used in each panel. For* $d \geq 20$ *the orange curves all interpolate the training points, and hence the training error is zero.*

自由度。[22] 回顾第7.4节，用d个自由度拟合自然样条等价于将响应变量对一组d个基函数进行最小二乘回归。图10.21的左上方面板显示了数据、真实函数$f(X)$以及$\hat{f}_8(X)$，即用d =8个自由度拟合的自然样条。

接下来，我们使用d =个自由度拟合一个自然样条。由于n =为20，这意味着$n = d$，并且我们的训练误差为零；换句话说，我们已经对训练数据进行了插值！从图10.21右上方的面板可以看出，$\hat{f}_{20}(X)$导致了剧烈的波动，因此测试误差将会很大。

我们现在继续用递增的d值对数据进行自然样条拟合。对于$d > 20$，Y对d基函数的最小二乘回归不唯一：存在无限多个最小二乘系数估计可实现零误差。为了从中选择，我们选取系数平方和最小的一个，$\sum_{j=1}^{d} \hat{\beta}_j^2$。这被称为*minimum-norm*解。

图10.21的两个下方子图展示了自由度分别为d =42和d =80的最小范数自然样条拟合。令人难以置信的是，$\hat{f}_{42}(X)$比$\hat{f}_{20}(X)$要狂野得多，*even though it makes use of moredegrees of freedom*。而$\hat{f}_{80}(X)$则差别不大。这怎么可能呢？本质上，$\hat{f}_{20}(X)$之所以非常狂野，是因为用d =20个基函数对n =20个观测值进行插值只有一种方式，而这种方式会导致拟合函数有些极端。相比之下，有

[22] This implies the choice of d knots, here chosen at d equi-probability quantiles of the training data. When $d > n$, the quantiles are found by interpolation.

使用$d = 42$个或$d = 80$个基函数对$n = 20$个观测值进行插值有无限多种方法，而其中最平滑的——即最小范数解——远比$\hat{f}_{20}(X)$更为规矩！

在图10.20中，我们展示了与$\hat{f}_d(X)$相关的训练误差和测试误差，针对一系列自由度d的值。我们看到，一旦d =达到20及以上，即达到插值阈值后，训练误差降至零。相比之下，测试误差在$d \leq$为20时呈现U形，在d =约为20时急剧增大，然后在$d >$为20时出现第二个下降区域。对于此示例，信噪比——$\text{Var}(f(X))/\sigma^2$——为5.9，相当高（数据点接近真实曲线）。因此，一个能够插值数据且在观测数据点之间不会偏离太远的估计很可能会表现良好。

在图10.20和图10.21中，我们使用自然样条在简单的一维环境中展示了双下降现象。然而，事实证明，同样的现象也会在深度学习中发生。基本上，当我们用大量参数拟合神经网络时，有时能够在训练误差为零的情况下取得良好结果。在高信噪比的问题中，例如自然图像识别和语言翻译，这一点尤为明显。这是因为用于拟合神经网络的技术，包括随机梯度下降，自然而然地倾向于选择一种“平滑”的插值模型，这类模型在这类问题上具有良好的测试集性能。

有几点值得强调：

- *The double-descent phenomenon does not contradict the bias-variance trade-off, as presented in Section 2.2.2.* 相反，图10.20右侧所展示的双下降曲线是由于x轴表示所使用的样条基函数的数量，而这并未恰当反映插值训练数据模型的真实“灵活性”。换句话说，在此例中，最小范数自然样条在d =取42时的方差低于自然样条在d =取20时的方差。

- *Most of the statistical learning methods seen in this book do not exhibit double descent.* 例如，正则化方法通常不会对训练数据进行插值，因此不会出现双重下降现象。这并不是正则化方法的缺点：它们能够取得出色的结果*without interpolating the data*！

  特别地，在此处的例子中，如果我们使用适当选择惩罚项的岭回归而非最小二乘法来拟合自然样条，那么我们就不会观察到双下降现象，实际上还会获得更好的测试误差结果。

- *In Chapter 9, we saw that maximal margin classifiers and SVMs that have zero training error nonetheless often achieve very good test error.* 这部分是因为那些方法寻求平滑的最小范数解。这类似于最小范数自然样条可以在零训练误差下给出良好结果的事实。

- *The double-descent phenomenon has been used by the machine learning community to explain the successful practice of using an over-*

parametrized neural network (many layers, and many hidden units), and then fitting all the way to zero training error. 然而，拟合到零误差并不总是最优的，其可取性取决于信噪比。例如，我们可以使用岭正则化来避免神经网络的过拟合，如(10.31)所示。在这种情况下，只要我们对调优参数λ做出适当的选择，我们就永远不会插值训练数据，因此不会看到双下降现象。尽管如此，我们仍能获得非常好的测试集性能，很可能远优于插值训练数据所能达到的效果。随机梯度下降过程中的早停也可以作为一种正则化形式，防止我们插值训练数据，同时在测试数据上仍能取得很好的结果。

总结：尽管双下降有时会出现在神经网络中，但我们通常不希望依赖这种行为。此外，重要的是要记住偏差-方差权衡始终成立（尽管测试误差作为灵活性的函数可能不会呈现U形，这取决于我们如何在x轴上参数化“灵活性”的概念）。

10.9 实验：深度学习

在本节中，我们展示如何拟合文中讨论的示例。我们使用Python `torch`包，以及`pytorch_lightning`提供用于简化模型拟合和评估的实用工具的软件包。在某些特殊处理器上，例如苹果的新M1芯片，这段代码可以快得令人印象深刻。该软件包结构良好、灵活，并且会让Python用户感到舒适。一个很好的伴侣是网站py-torch.org/tutorials。我们的许多代码都是从那里改编的，还有`pytorch_lightning`文档。[23]

`torch`
`pytorch_lightning`

我们首先从之前见过的几个标准导入开始。

In [1]:
```
import numpy as np, pandas as pd
from matplotlib.pyplot import subplots
from sklearn.linear_model import \
     (LinearRegression,
      LogisticRegression,
      Lasso)
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline
from ISLP import load_data
from ISLP.models import ModelSpec as MS
from sklearn.model_selection import \
     (train_test_split,
      GridSearchCV)
```

[23] The precise URLs at the time of writing are `https://pytorch.org/tutorials/beginner/basics/intro.html` and `https://pytorch-lightning.readthedocs.io/en/latest/`.

Torch 专用导入

对 torch 有许多导入。（这些不包含在 ISLP 中，因此必须单独安装。）首先，我们导入用于指定顺序结构网络的主库和基本工具。

In [2]:
```
import torch
from torch import nn
from torch.optim import RMSprop
from torch.utils.data import TensorDataset
```

对于 torch，还有几个其他辅助包。例如，torchmetrics 包具有计算各种指标以评估拟合模型时的性能。 torchinfo 包提供了一个有用的模型层摘要。我们使用read_image()函数在10.9.4节中加载测试图像时。

torchmetrics
torchinfo
read_image()

In [3]:
```
from torchmetrics import (MeanAbsoluteError,
                          R2Score)
from torchinfo import summary
from torchvision.io import read_image
```

包 pytorch_lightning 是 torch 的一个稍高级的接口，它通过减少所需的样板代码量（与单独使用 torch 相比）简化了模型的指定与拟合。

In [4]:
```
from pytorch_lightning import Trainer
from pytorch_lightning.loggers import CSVLogger
```

为了复现结果，我们使用 seed_everything()。我们还将指示 torch 在可能的情况下使用确定性算法。

seed_
everything()

In [5]:
```
from pytorch_lightning.utilities.seed import seed_everything
seed_everything(0, workers=True)
torch.use_deterministic_algorithms(True, warn_only=True)
```

我们将使用 torchvision 附带的几个数据集作为示例：一个用于图像分类的预训练网络，以及一些用于预处理的数据变换。

torchvision

In [6]:
```
from torchvision.datasets import MNIST, CIFAR100
from torchvision.models import (resnet50,
                                ResNet50_Weights)
from torchvision.transforms import (Resize,
                                    Normalize,
                                    CenterCrop,
                                    ToTensor)
```

我们在ISLP中提供了一些专门用于本实验的实用工具。SimpleDataModule和SimpleModule是pytorch_lightning中使用的对象的简化版本，pytorch_lightning是用于拟合torch模型的高级模块。尽管该模块可以实现更高级的用途，例如在图形处理单元（GPU）上进行计算和并行数据处理，但本实验不会过多关注这些方面。ErrorTracker负责处理验证或测试阶段每个小批量中的目标集合和预测集合，从而可以计算整个验证或测试数据集上的指标。

In [7]:
```
from ISLP.torch import (SimpleDataModule,
                        SimpleModule,
                        ErrorTracker,
                        rec_num_workers)
```

此外，我们还包含了一些辅助函数，用于加载IMDb数据库，以及一个将整数映射到数据库中特定键的查找表。我们收录了来自keras的预处理IMDb数据的轻微修改版，这是一个用于拟合深度学习模型的独立包。此

为我们节省了大量预处理工作，使我们能够专注于模型本身的指定和拟合 keras 。

In [8]:
```
from ISLP.torch.imdb import (load_lookup,
                             load_tensor,
                             load_sparse,
                             load_sequential)
```

最后，我们介绍一些与torch不直接相关的实用导入。来自glob模块的glob()函数用于查找所有匹配的文件
通配符，我们将在示例中使用ResNet50模型应用于我们自己的一些图像。json glob()
模块将用于加载一个 js on
用于查找类的JSON文件，以识别ResNet50示例中图片的标签。

In [9]:
```
from glob import glob
import json
```

10.9.1 Single Layer Network on Hitters Data

W我们从在 Hitte 上拟合第10.6节中的模型开始 rs 数据。

In [10]:
```
Hitters = load_data('Hitters').dropna()
n = Hitters.shape[0]
```

我们将拟合两个线性模型（最小二乘和套索），并将它们的性能与神经网络的性能进行比较。对于这种比较，我们将在验证数据集上使用平均绝对误差。

$$\text{MAE}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|.$$

我们设置了模型矩阵和响应。

In [11]:
```
model = MS(Hitters.columns.drop('Salary'), intercept=False)
X = model.fit_transform(Hitters).to_numpy()
Y = Hitters['Salary'].to_numpy()
```

上面的to_numpy()方法将pandas数据帧或序列转换为
numpy 数组。我们这样做是因为我们需要使用sklearn来拟合套索模型，而它要求这种转换。我们还使用了来自sklearn的线性回归方法，而不是来自statsmodels的第3章中的方法，以便于比较。 to_numpy()

现在我们将数据分为测试集和训练集，固定使用sklearn进行分割的随机状态。

In [12]:
```
(X_train,
 X_test,
 Y_train,
 Y_test) = train_test_split(X,
                            Y,
                            test_size=1/3,
                            random_state=1)
```

线性模型

我们拟合线性模型并直接评估测试误差。

In [13]:
```
hit_lm = LinearRegression().fit(X_train, Y_train)
Yhat_test = hit_lm.predict(X_test)
np.abs(Yhat_test - Y_test).mean()
```

Out[13]: 259.7153

接下来我们使用 sklearn 拟合套索。我们使用平均绝对误差来选择和评估模型，而不是均方误差。在6.5.2节中使用的专门求解器仅使用均方误差。因此，在这里，通过更多一点的工作，我们创建了一个交叉验证网格并直接执行交叉验证。

我们用两个步骤编码一个管道：首先使用 StandardScaler() 变换对特征进行归一化，然后在不进行进一步归一化的情况下拟合 lasso。

In [14]:
```
scaler = StandardScaler(with_mean=True, with_std=True)
lasso = Lasso(warm_start=True, max_iter=30000)
standard_lasso = Pipeline(steps=[('scaler', scaler),
                                 ('lasso', lasso)])
```

我们需要为λ创建一个值网格。按照通常做法，我们选择包含100个λ值的网格，在对数尺度上从lam_max均匀下降到0.01*lam_max。这里lam_max是λ的最小值，具有全零解。该值等于任何预测变量与（中心化的）响应之间最大的绝对内积。[24]

In [15]:
```
X_s = scaler.fit_transform(X_train)
n = X_s.shape[0]
lam_max = np.fabs(X_s.T.dot(Y_train - Y_train.mean())).max() / n
param_grid = {'alpha': np.exp(np.linspace(0, np.log(0.01), 100))
              * lam_max}
```

请注意，我们必须首先转换数据，因为变量的尺度影响了λ的选择。现在我们使用这个λ值的序列进行交叉验证。

In [16]:
```
cv = KFold(10,
           shuffle=True,
           random_state=1)
grid = GridSearchCV(lasso,
```

[24] The derivation of this result is beyond the scope of this book.

```
                    param_grid,
                    cv=cv,
                    scoring='neg_mean_absolute_error')
grid.fit(X_train, Y_train);
```

我们提取具有最佳交叉验证平均绝对误差的lasso模型，并在未用于交叉验证的X_test和Y_test上评估其性能。

In [17]:
```
trained_lasso = grid.best_estimator_
Yhat_test = trained_lasso.predict(X_test)
np.fabs(Yhat_test - Y_test).mean()
```

Out[17]: 257.2382

这与我们通过最小二乘法拟合线性模型得到的结果相似。然而，这些结果在不同的训练/测试分割下可能会有很大变化；我们鼓励读者在代码块12中尝试不同的随机种子，并重新运行后续代码至此。

指定网络：类和继承

为了拟合神经网络，我们首先建立一个描述网络的模型结构。这样做需要我们定义特定于我们希望拟合的模型的新类。通常，这在pytorch中是通过子类化网络的泛型表示来完成的，这也是我们在这里采用的方法。虽然这个例子很简单，但我们将详细地介绍这些步骤，因为它将对我们后续更复杂的例子大有裨益。

In [18]:
```
class HittersModel(nn.Module):

    def __init__(self, input_size):
        super(HittersModel, self).__init__()
        self.flatten = nn.Flatten()
        self.sequential = nn.Sequential(
            nn.Linear(input_size, 50),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(50, 1))

    def forward(self, x):
        x = self.flatten(x)
        return torch.flatten(self.sequential(x))
```

`class` 语句将代码块标识为类 `HittersModel` 的声明，该类继承自基类 `nn.Module`。此基类在 `torch` 中无处不在，并表示神经网络中的映射。

在 `class` 语句下方缩进的是此类的方法：在此情况下为 `__init__` 和 `forward`。当如下方单元格所示创建类的实例时，将调用 `__init__` 方法。在这些方法中，`self` 始终指代类的一个实例。在 `__init__` 方法中，我们将两个对象作为属性附加到 `self`：`flatten` 和 `sequential`。这些在 `forward` 方法中用于描述此模块实现的地图。

在 __init__ 方法中有一行额外的代码，即对 super() 的调用。此函数允许子类（即 HittersModel）访问 meth-
继承自的类的方法。例如，类nn.Module有自己的__init__方法，这与我们上面写的HittersModel.__init__()方法不同。使用super()允许我们调用基类的方法。对于torch模型，我们将始终进行这个super()调用，因为这对于模型被torch正确解释是必要的。

super()

对象 nn.Module 具有比仅有的 __init__ 和 forward 更多的方法。由于这种继承关系，这些方法对 HittersModel 实例而言是直接可访问的。我们很快就会见到一个这样的方法，即 eval() 方法，用于在我们希望在测试数据上评估模型时禁用 dropout。

In [19]:
```
hit_model = HittersModel(X.shape[1])
```

对象 self.sequential 是四个映射的复合。首先将 Hitters 的 19 个特征映射到 50 维，引入 $50 \times 19 + 50$ 个参数用于权重和映射的 *intercept*（通常称为 *bias*）。该层随后被映射到一个 ReLU 层，接着是一个 40% 的 drop out 层，最后是一个线性映射降至 1 维，同样带有偏置。因此，可训练参数的总数为 $50 \times 19 + 50 + 50 + 1 = 1051$。

包 torchinfo 提供了一个 summary() 函数，能简洁地汇总这些信息。我们指定输入的大小，并观察每个张量在网络各层传递时的大小。

In [20]:
```
summary(hit_model,
        input_size=X_train.shape,
        col_names=['input_size',
                   'output_size',
                   'num_params'])
```

Out[20]:
```
==========================================================================
Layer (type:depth-idx)       Input Shape    Output Shape   Param #
==========================================================================
HittersModel                 [175, 19]      [175]          --
  Flatten: 1-1               [175, 19]      [175, 19]      --
  Sequential: 1-2            [175, 19]      [175, 1]       --
       Linear: 2-1           [175, 19]      [175, 50]      1,000
       ReLU: 2-2             [175, 50]      [175, 50]      --
       Dropout: 2-3          [175, 50]      [175, 50]      --
       Linear: 2-4           [175, 50]      [175, 1]       51
==========================================================================
Total params: 1,051
Trainable params: 1,051
```

我们在此处及后续使用中略微截断了输出的末尾。

我们现在需要将训练数据转换为 torch 可访问的形式。torch 中的基本数据类型是 tensor，它与前几章的 ndarray 非常相似。我们在此还注意到，torch 通常使用 32 位 (*single precision*) 而非 64 位 (*double precision*) 浮点数。因此，在构建张量之前，我们将数据转换为 np.float32。然后，X 和 Y 张量被安排成一个 Dataset

Dataset

由torch使用TensorDataset()识别。

TensorDataset()

In [21]:
```
X_train_t = torch.tensor(X_train.astype(np.float32))
Y_train_t = torch.tensor(Y_train.astype(np.float32))
hit_train = TensorDataset(X_train_t, Y_train_t)
```

我们对测试数据也进行同样的处理。

In [22]:
```
X_test_t = torch.tensor(X_test.astype(np.float32))
Y_test_t = torch.tensor(Y_test.astype(np.float32))
hit_test = TensorDataset(X_test_t, Y_test_t)
```

最后，该数据集被传递到 DataLoader()，然后该数据集最终将数据传入我们的网络。虽然这看起来可能开销很大，但这种结构对于更复杂的任务很有帮助，在这些任务中，数据可能位于不同的机器上，或者必须将数据传递到 GPU。我们在 ISLP 中提供了一个辅助函数 SimpleDataModule()，以便为标准

SimpleDataModule()

用途。它的一个参数是 num_workers，用于指示我们将使用多少个进程来加载数据。对于像 Hitters 这样的小型数据，这几乎不会有什么影响，但对于下面的 MNIST 和 CIFAR100 示例，它确实提供了优势。torch 包将检查正在运行的进程并确定工作进程的最大数量。[25] 我们包含了一个函数 rec_num_workers() 来计算这个值，以便我们知道多少工作进程可能是合理的（这里的最大值是 16）。

In [23]:
```
max_num_workers = rec_num_workers()
```

pytorch_lightning中的一般训练设置涉及训练、验证和测试数据。这些数据分别由不同的数据加载器表示。在每个epoch中，我们运行一个训练步骤来学习模型，以及一个验证步骤来跟踪误差。测试数据通常在训练结束时用于评估模型。

在这种情况下，由于我们只分为了测试集和训练集，我们将使用测试数据作为验证数据，参数为validation=hit_test。validation参数可以是介于0和1之间的浮点数、整数，或是Dataset。如果是浮点数（相应地，整数），它会被解释为用于验证的*training*观测值的百分比（相应地，数量）。如果是Dataset，则直接传递给数据加载器。

In [24]:
```
hit_dm = SimpleDataModule(hit_train,
                          hit_test,
                          batch_size=32,
                          num_workers=min(4, max_num_workers),
                          validation=hit_test)
```

接下来，我们需要提供一个pytorch_lightning模块，用于控制训练过程中执行的步骤。我们为SimpleModule()提供了方法，这些方法仅记录每个周期结束时损失函数的值及任何额外指标。这些操作由方法 SimpleModule.[training/test/validation]_step()控制，尽管在我们的示例中不会修改这些内容。

[25] This depends on the computing hardware and the number of cores available.

```
In [25]: hit_module = SimpleModule.regression(hit_model,
                               metrics={'mae':MeanAbsoluteError()})
```

通过使用SimpleModule.regression()方法，我们表明我们将使用（10.23）中的平方误差损失。我们还要求在记录的指标中跟踪平均绝对误差。

SimpleModule.regression()

我们通过CSVLogger()记录结果，本例中将结果存储在目录logs/hitters内的一个CSV文件中。拟合完成后，这允许我们将结果加载为pd.DataFrame()并在下方进行可视化。在pytorch_lightning中有多种记录结果的方法，但我们在此不会详细讨论这些方法。

```
In [26]: hit_logger = CSVLogger('logs', name='hitters')
```

最后，我们准备训练模型并记录结果。我们使用来自pytorch_lightning的Trainer()对象来完成这项工作。参数datamodule=hit_dm告诉训练器如何生成训练/验证/测试日志，而第一个参数hit_module指定了网络架构以及训练/验证/测试步骤。callbacks参数允许在训练模型的不同阶段执行多项任务。在这里，我们的ErrorTracker()回调将使我们能够在训练过程中计算验证误差，并最终计算测试误差。我们现在对模型进行50个周期的拟合。

```
In [27]: hit_trainer = Trainer(deterministic=True,
                               max_epochs=50,
                               log_every_n_steps=5,
                               logger=hit_logger,
                               callbacks=[ErrorTracker()])
         hit_trainer.fit(hit_module, datamodule=hit_dm)
```

在 SGD 的每一步中，算法随机选择 32 个训练观测值来计算梯度。回想第 10.7 节，一个 epoch 相当于处理 n 个观测值所需的 SGD 步数。由于训练集有 $n = 175$，并且我们在构建 hit_dm 时指定了 batch_size 为 32，因此一个 epoch 是 $175/32 = 5.5$ 个 SGD 步。

在拟合模型后，我们可以使用训练器的test()方法评估测试数据上的表现。

```
In [28]: hit_trainer.test(hit_module, datamodule=hit_dm)
```

```
Out[28]: [{'test_loss': 104098.5469, 'test_mae': 229.5012}]
```

拟合结果已记录到CSV文件中。我们可以在记录器的experiment.metrics_file_path属性中找到特定于此次运行的结果。请注意，每次拟合模型时，记录器都会将结果输出到我们的logs/hitters目录下的新子目录中。

我们现在创建一个MAE（平均绝对误差）随纪元数变化的图。首先，我们获取记录的摘要。

```
hit_results = pd.read_csv(hit_logger.experiment.metrics_file_path)
```

由于我们将在后续示例中生成类似的图表，我们编写一个简单的通用函数来生成此图表。

In [29]:
```
def summary_plot(results,
                 ax,
                 col='loss',
                 valid_legend='Validation',
                 training_legend='Training',
                 ylabel='Loss',
                 fontsize=20):
    for (column,
         color,
         label) in zip([f'train_{col}_epoch',
                        f'valid_{col}'],
                       ['black',
                        'red'],
                       [training_legend,
                        valid_legend]):
        results.plot(x='epoch',
                     y=column,
                     label=label,
                     marker='o',
                     color=color,
                     ax=ax)
    ax.set_xlabel('Epoch')
    ax.set_ylabel(ylabel)
    return ax
```

现在我们来设置坐标轴，并使用我们的函数来生成MAE图。

In [30]:
```
fig, ax = subplots(1, 1, figsize=(6, 6))
ax = summary_plot(hit_results,
                  ax,
                  col='mae',
                  ylabel='MAE',
                  valid_legend='Validation (=Test)')
ax.set_ylim([0, 400])
ax.set_xticks(np.linspace(0, 50, 11).astype(int));
```

我们可以直接从最终模型进行预测，并在测试数据上评估其性能。在拟合之前，我们调用`hit_model`的`eval()`方法。这告诉`torch`有效地认为此模型已经拟合，以便我们可以使用它对新数据进行预测。对于这里的模型，最大的变化是dropout层将被关闭，即在预测新数据时不会随机丢弃权重。

In [31]:
```
hit_model.eval()
preds = hit_module(X_test_t)
torch.abs(Y_test_t - preds).mean()
```

Out[31]:
```
tensor(229.5012, grad_fn=<MeanBackward0>)
```

清理

在设置数据模块时，我们启动了几个工作进程，这些进程将持续运行。我们删除了对 torch 对象的所有引用，以确保这些进程会被终止。

In [32]:
```
del(Hitters,
    hit_model, hit_dm,
    hit_logger,
    hit_test, hit_train,
    X, Y,
    X_test, X_train,
    Y_test, Y_train,
    X_test_t, Y_test_t,
    hit_trainer, hit_module)
```

10.9.2 Multilayer Network on the MNIST Digit Data

`torchvision` 包自带许多示例数据集，包括 `MNIST` 数字数据。我们的第一步是获取训练和测试数据集；`torchvision.datasets` 中的 `MNIST()` 函数用于此目的。数据将在首次执行此函数时下载，并存储在目录 `data/MNIST` 中。

`MNIST()`

In [33]:
```
(mnist_train,
 mnist_test) = [MNIST(root='data',
                      train=train,
                      download=True,
                      transform=ToTensor())
                for train in [True, False]]
mnist_train
```

Out[33]:
```
Dataset MNIST
    Number of datapoints: 60000
    Root location: data
    Split: Train
    StandardTransform
Transform: ToTensor()
```

训练数据中有60,000张图像，测试数据中有10,000张。这些图像尺寸为28 × 28，并以像素矩阵的形式存储。我们需要将每一张转换为向量。

神经网络对输入的尺度有些敏感，就像岭回归和套索正则化受到尺度影响一样。此处的输入是介于0到255之间的八位灰度值，因此我们将其重新缩放至单位区间。[26] 这种变换，连同对轴的一些重新排序，是由 `torchvision.transforms`包中的`ToTensor()`变换执行的。

正如我们的 `Hitters` 示例中那样，我们从训练和测试数据集中构建一个数据模块，并预留 20% 的训练图像用于验证。

In [34]:
```
mnist_dm = SimpleDataModule(mnist_train,
                            mnist_test,
                            validation=0.2,
                            num_workers=max_num_workers,
                            batch_size=256)
```

[26] Note: eight bits means 2^8, which equals 256. Since the convention is to start at 0, the possible values range from 0 to 255.

让我们看看将要输入到网络中的数据。我们遍历测试数据集的前几个批次，在2个批次后中断：

In [35]:
```
for idx, (X_ ,Y_) in enumerate(mnist_dm.train_dataloader()):
    print('X: ', X_.shape)
    print('Y: ', Y_.shape)
    if idx >= 1:
        break
```

```
X:  torch.Size([256, 1, 28, 28])
Y:  torch.Size([256])
X:  torch.Size([256, 1, 28, 28])
Y:  torch.Size([256])
```

我们看到，每个批次的 X 包含 256 张大小为 1x28x28 的图像。这里的 1 表示单通道（灰度）。对于下面的 RGB 图像，例如 CIFAR100，我们将看到尺寸中的 1 将被替换为 3，对应三个 RGB 通道。

现在我们准备好指定我们的神经网络了。

In [36]:
```
class MNISTModel(nn.Module):
    def __init__(self):
        super(MNISTModel, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 256),
            nn.ReLU(),
            nn.Dropout(0.4))
        self.layer2 = nn.Sequential(
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.3))
        self._forward = nn.Sequential(
            self.layer1,
            self.layer2,
            nn.Linear(128, 10))
    def forward(self, x):
        return self._forward(x)
```

我们看到，在第一层中，每个1x28x28图像被展平，然后映射到256维，在此我们应用ReLU激活函数并设置40%的dropout。第二层将第一层的输出映射到128维，应用ReLU激活函数并设置30%的dropout。最后，这128维被映射到10维，即MNIST数据中的类别数量。

In [37]:
```
mnist_model = MNISTModel()
```

我们可以根据上面现有的批次 X_ 来检查模型产生的输出大小是否符合预期。

In [38]:
```
mnist_model(X_).size()
```

Out[38]:
```
torch.Size([256, 10])
```

让我们看一下模型的摘要。我们可以传递一个形状正确的张量，而不是 input_size。在这种情况下，我们传递上面最终的批处理X_。

In [39]:
```
summary(mnist_model,
        input_data=X_,
        col_names=['input_size',
                   'output_size',
                   'num_params'])
```

Out[39]:
```
==========================================================================
Layer (type:depth-idx)       Input Shape        Output Shape  Param #
==========================================================================
MNISTModel                   [256, 1, 28, 28]   [256, 10]     --
  Sequential: 1-1            [256, 1, 28, 28]   [256, 10]     --
       Sequential: 2-1       [256, 1, 28, 28]   [256, 256]    --
            Flatten: 3-1     [256, 1, 28, 28]   [256, 784]    --
            Linear: 3-2      [256, 784]         [256, 256]    200,960
            ReLU: 3-3        [256, 256]         [256, 256]    --
            Dropout: 3-4     [256, 256]         [256, 256]    --
       Sequential: 2-2       [256, 256]         [256, 128]    --
            Linear: 3-5      [256, 256]         [256, 128]    32,896
            ReLU: 3-6        [256, 128]         [256, 128]    --
            Dropout: 3-7     [256, 128]         [256, 128]    --
       Linear: 2-3           [256, 128]         [256, 10]     1,290
==========================================================================
Total params: 235,146
Trainable params: 235,146
```

在同时设置好模型和数据模块之后，拟合这个模型现在几乎与Hitters示例相同。与我们的回归模型相比，这里我们将使用SimpleModule.classification()方法，该方法使用交叉熵损失函数而不是均方误差。

SimpleModule.classification()

In [40]:
```
mnist_module = SimpleModule.classification(mnist_model)
mnist_logger = CSVLogger('logs', name='MNIST')
```

现在我们可以开始了。最后一步是提供训练数据，并拟合模型。

In [41]:
```
mnist_trainer = Trainer(deterministic=True,
                        max_epochs=30,
                        logger=mnist_logger,
                        callbacks=[ErrorTracker()])
mnist_trainer.fit(mnist_module,
                  datamodule=mnist_dm)
```

我们在这里抑制了输出，这是一个关于模型拟合进度的报告，按轮次分组。这非常有用，因为在大型数据集上拟合可能需要时间。在配备10核Apple M1 Pro芯片和16 GB RAM的MacBook Pro上，拟合这个模型耗时245秒。在这里，我们指定了20%的验证集划分，因此训练实际上是在训练集中的60,000个观测值的80%上进行的。这是实际提供验证数据的一种替代方法，就像我们对Hitters数据所做的那样。SGD在计算梯度时使用256个观测值的批次，通过计算，我们看到一个轮次对应于188个梯度步骤。

SimpleModule.classification() 默认包含一个准确率指标。可以从 torchmetrics 中添加其他分类指标。我们将使用我们的 summary_plot() 函数来显示各个 epoch 的准确率。

In [42]:
```
mnist_results = pd.read_csv(mnist_logger.experiment.
    metrics_file_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(mnist_results,
             ax,
             col='accuracy',
             ylabel='Accuracy')
ax.set_ylim([0.5, 1])
ax.set_ylabel('Accuracy')
ax.set_xticks(np.linspace(0, 30, 7).astype(int));
```

我们再次使用训练器的 `test()` 方法评估准确率。该模型在测试数据上达到了97%的准确率。

In [43]:
```
mnist_trainer.test(mnist_module,
                   datamodule=mnist_dm)
```

Out[43]:
```
[{'test_loss': 0.1471, 'test_accuracy': 0.9681}]
```

表10.1还报告了由LDA（第4章）和多类逻辑回归导致的错误率。关于LDA，我们请读者参阅第4.7.3节。尽管我们可以使用`sklearn`函数`LogisticRegression()`来拟合多类逻辑回归，但我们在此处设置的是使用`torch`来拟合这样的模型。我们仅有一个输入层和一个输出层，并省略了隐藏层！

In [44]:
```
class MNIST_MLR(nn.Module):
    def __init__(self):
        super(MNIST_MLR, self).__init__()
        self.linear = nn.Sequential(nn.Flatten(),
                                    nn.Linear(784, 10))
    def forward(self, x):
        return self.linear(x)

mlr_model = MNIST_MLR()
mlr_module = SimpleModule.classification(mlr_model)
mlr_logger = CSVLogger('logs', name='MNIST_MLR')
```

In [45]:
```
mlr_trainer = Trainer(deterministic=True,
                      max_epochs=30,
                      callbacks=[ErrorTracker()])
mlr_trainer.fit(mlr_module, datamodule=mnist_dm)
```

我们像之前一样拟合模型并计算测试结果。

In [46]:
```
mlr_trainer.test(mlr_module,
                 datamodule=mnist_dm)
```

Out[46]:
```
[{'test_loss': 0.3187, 'test_accuracy': 0.9241}]
```

即使这个相当简单的模型，准确率也超过90%。正如在`Hitters`示例中那样，我们删除上面创建的一些对象。

In [47]:
```
del(mnist_test,
    mnist_train,
```

```
    mnist_model,
    mnist_dm,
    mnist_trainer,
    mnist_module,
    mnist_results,
    mlr_model,
    mlr_module,
    mlr_trainer)
```

10.9.3 Convolutional Neural Networks

在本节中，我们将一个CNN模型适配到CIFAR100数据上，该数据可在torchvision包中获取。其组织方式与MNIST数据类似。

In [48]:
```
(cifar_train,
 cifar_test) = [CIFAR100(root="data",
                         train=train,
                         download=True)
            for train in [True, False]]
```

In [49]:
```
transform = ToTensor()
cifar_train_X = torch.stack([transform(x) for x in
                             cifar_train.data])
cifar_test_X = torch.stack([transform(x) for x in
                             cifar_test.data])
cifar_train = TensorDataset(cifar_train_X,
                             torch.tensor(cifar_train.targets))
cifar_test = TensorDataset(cifar_test_X,
                             torch.tensor(cifar_test.targets))
```

CIFAR100 数据集由 50,000 张训练图像组成，每张图像用一个三维张量表示：每张三色图像表示为三个通道的集合，每个通道由 32×32 个八位像素构成。我们像处理数字时那样进行标准化，但保留数组结构。这是通过 ToTensor() 变换实现的。

创建数据模块与 MNIST 示例类似。

In [50]:
```
cifar_dm = SimpleDataModule(cifar_train,
                             cifar_test,
                             validation=0.2,
                             num_workers=max_num_workers,
                             batch_size=128)
```

我们再次查看数据加载器中典型批次的数据形状。

In [51]:
```
for idx, (X_ ,Y_) in enumerate(cifar_dm.train_dataloader()):
    print('X: ', X_.shape)
    print('Y: ', Y_.shape)
    if idx >= 1:
        break
```

```
X:  torch.Size([128, 3, 32, 32])
Y:  torch.Size([128])
X:  torch.Size([128, 3, 32, 32])
Y:  torch.Size([128])
```

开始之前，我们先查看一些训练图像；类似的代码生成了第406页的图10.5。下面的示例还说明了TensorDataset对象可以用整数索引——我们通过索引cifar_train从训练数据中随机选择图像。为了正确显示，我们必须通过调用np.transpose()来重新排列维度。

In [52]:
```
fig, axes = subplots(5, 5, figsize=(10,10))
rng = np.random.default_rng(4)
indices = rng.choice(np.arange(len(cifar_train)), 25,
                     replace=False).reshape((5,5))
for i in range(5):
    for j in range(5):
        idx = indices[i,j]
        axes[i,j].imshow(np.transpose(cifar_train[idx][0],
                                      [1,2,0]),
                                      interpolation=None)
        axes[i,j].set_xticks([])
        axes[i,j].set_yticks([])
```

这里，imshow() 方法从其参数的形状中识别出
它是一个三维数组，最后一个维度索引三个RGB颜色通道。 .imshow()

我们指定一个中等规模的CNN用于演示目的，其结构与图10.8类似。我们使用若干层，每层由卷积、ReLU和最大池化步骤组成。我们首先定义一个模块，用于定义其中一层。如同之前的例子，我们重写nn.Module的__init__()和forward()方法。这个用户自定义模块现在可以像nn.Linear()或nn.Dropout()一样使用。

In [53]:
```
class BuildingBlock(nn.Module):

    def __init__(self,
                 in_channels,
                 out_channels):

        super(BuildingBlock, self).__init__()
        self.conv = nn.Conv2d(in_channels=in_channels,
                              out_channels=out_channels,
                              kernel_size=(3,3),
                              padding='same')
        self.activation = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=(2,2))

    def forward(self, x):
        return self.pool(self.activation(self.conv(x)))
```

请注意，我们使用了padding = "same"参数来传递给nn.Conv2d()，这确保了输出通道与输入通道具有相同的维度。第一隐藏层有32个通道，而输入层则有三个通道。我们在所有层中为每个通道使用3×3卷积滤波器。每个卷积之后都跟随一个在2×2块上的最大池化层。

在为 CIFAR100 数据构建深度学习模型时，我们顺序使用了几个 BuildingBlock() 模块。这个简单的例子展示了 torch 的一些强大之处。用户可以定义自己的模块，

可以在其他模块中组合。最终，一切都由通用训练器进行拟合。

In [54]:
```
class CIFARModel(nn.Module):

    def __init__(self):
        super(CIFARModel, self).__init__()
        sizes = [(3,32),
                 (32,64),
                 (64,128),
                 (128,256)]
        self.conv = nn.Sequential(*[BuildingBlock(in_, out_)
                                    for in_, out_ in sizes])

        self.output = nn.Sequential(nn.Dropout(0.5),
                                    nn.Linear(2*2*256, 512),
                                    nn.ReLU(),
                                    nn.Linear(512, 100))
    def forward(self, x):
        val = self.conv(x)
        val = torch.flatten(val, start_dim=1)
        return self.output(val)
```

我们构建模型并查看摘要。（我们之前已经创建了 {v*} 的示例。）

In [55]:
```
cifar_model = CIFARModel()
summary(cifar_model,
        input_data=X_,
        col_names=['input_size',
                   'output_size',
                   'num_params'])
```

Out[55]:
```
==========================================================================
Layer (type:depth-idx)  Input Shape        Output Shape       Param #
==========================================================================
CIFARModel              [128, 3, 32, 32]   [128, 100]         --
  Sequential: 1-1       [128, 3, 32, 32]   [128, 256, 2, 2]   --
    BuildingBlock: 2-1  [128, 3, 32, 32]   [128, 32, 16, 16]  --
        Conv2d: 3-1     [128, 3, 32, 32]   [128, 32, 32, 32]  896
        ReLU: 3-2       [128, 32, 32, 32]  [128, 32, 32, 32]  --
        MaxPool2d: 3-3  [128, 32, 32, 32]  [128, 32, 16, 16]  --
    BuildingBlock: 2-2  [128, 32, 16, 16]  [128, 64, 8, 8]    --
        Conv2d: 3-4     [128, 32, 16, 16]  [128, 64, 16, 16]  18,496
        ReLU: 3-5       [128, 64, 16, 16]  [128, 64, 16, 16]  --
        MaxPool2d: 3-6  [128, 64, 16, 16]  [128, 64, 8, 8]    --
    BuildingBlock: 2-3  [128, 64, 8, 8]    [128, 128, 4, 4]   --
        Conv2d: 3-7     [128, 64, 8, 8]    [128, 128, 8, 8]   73,856
        ReLU: 3-8       [128, 128, 8, 8]   [128, 128, 8, 8]   --
        MaxPool2d: 3-9  [128, 128, 8, 8]   [128, 128, 4, 4]   --
    BuildingBlock: 2-4  [128, 128, 4, 4]   [128, 256, 2, 2]   --
        Conv2d: 3-10    [128, 128, 4, 4]   [128, 256, 4, 4]   295,168
        ReLU: 3-11      [128, 256, 4, 4]   [128, 256, 4, 4]   --
        MaxPool2d: 3-12 [128, 256, 4, 4]   [128, 256, 2, 2]   --
  Sequential: 1-2       [128, 1024]        [128, 100]         --
       Dropout: 2-5     [128, 1024]        [128, 1024]        --
       Linear: 2-6      [128, 1024]        [128, 512]         524,800
```

```
       ReLU: 2-7        [128, 512]          [128, 512]         --
       Linear: 2-8      [128, 512]          [128, 100]         51,300
==========================================================================
Total params: 964,516
Trainable params: 964,516
```

可训练参数的总数为964,516。通过研究参数的大小，我们可以看出，在每次最大池化操作之后，两个维度上的通道数都会减半。在最后一次这样的操作之后，我们有一个具有256个通道、维度为2×2的层。这些随后被展平为一个大小为1,024的全连接层；换句话说，每个2×2矩阵被转换为一个4维向量，并在一个层中并排放置。接着是一个dropout正则化层，然后是一个大小为512的全连接层，最后是输出层。

到目前为止，我们在`SimpleModule()`中一直使用默认优化器。对这些数据而言，实验表明较小的学习率比默认的0.01表现更好。我们在此采用自定义优化器，学习率设为0.001。除此之外，日志记录和训练过程与之前的示例相似。优化器接受一个参数`params`，该参数告知优化器哪些参数参与SGD（随机梯度下降）。

我们之前看到，模块参数的条目是张量。在将参数传递给优化器时，我们做的不仅仅是传递数组；图的结构部分被编码在张量本身中。

In [56]:
```
cifar_optimizer = RMSprop(cifar_model.parameters(), lr=0.001)
cifar_module = SimpleModule.classification(cifar_model,
                                           optimizer=cifar_optimizer)
cifar_logger = CSVLogger('logs', name='CIFAR100')
```

In [57]:
```
cifar_trainer = Trainer(deterministic=True,
                        max_epochs=30,
                        logger=cifar_logger,
                        callbacks=[ErrorTracker()])
cifar_trainer.fit(cifar_module,
                  datamodule=cifar_dm)
```

该模型运行需要10分钟或更长时间，在测试数据上达到约42%的准确率。尽管对于100类数据而言这并不算太差（随机分类器的准确率为1%），但我们在网上搜索到的结果约为75%。通常，要达到这样的结果，需要大量的架构调整、对正则化的反复调试以及时间。

让我们看看不同周期下的验证和训练准确率。

In [58]:
```
log_path = cifar_logger.experiment.metrics_file_path
cifar_results = pd.read_csv(log_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(cifar_results,
             ax,
             col='accuracy',
             ylabel='Accuracy')
ax.set_xticks(np.linspace(0, 10, 6).astype(int))
ax.set_ylabel('Accuracy')
ax.set_ylim([0, 1]);
```

最后，我们在测试数据上评估我们的模型。

```
In [59]: cifar_trainer.test(cifar_module,
                            datamodule=cifar_dm)
```

```
Out[59]: [{'test_loss': 2.4238 'test_accuracy': 0.4206}]
```

硬件加速

随着深度学习在机器学习中变得无处不在，硬件制造商已经生产出专门的库，这些库通常能够加速梯度下降步骤。

例如，配备M1芯片的Mac OS设备可能启用了*Metal*编程框架，这可以加速torch计算。我们提供一个如何使用这种加速的示例。

主要更改为对 `Trainer()` 的调用以及在数据上评估的指标。这些指标需要被告知评估时数据的位置。这是通过调用指标的 `to()` 方法实现的。

```
In [60]: try:
             for name, metric in cifar_module.metrics.items():
                 cifar_module.metrics[name] = metric.to('mps')
             cifar_trainer_mps = Trainer(accelerator='mps',
                                         deterministic=True,
                                         max_epochs=30)
             cifar_trainer_mps.fit(cifar_module,
                                   datamodule=cifar_dm)
             cifar_trainer_mps.test(cifar_module,
                                   datamodule=cifar_dm)
         except:
             pass
```

这为每个epoch带来了大约两到三倍的加速。我们通过`try:`和`except:`条件保护了这个代码块；如果有效，我们就能获得加速，如果失败，则不会发生任何变化。

10.9.4 Using Pretrained CNN Models

我们现在展示如何使用在 {v*} 数据库上预训练的 CNN 对自然图像进行分类，并演示我们如何生成图 10.10。我们将数码相册中的六张 JPEG 图像复制到目录 `book_images` 中。这些图像可从 `www.statlearning.com`（ISLP 书籍网站）的数据部分获取。下载 `book_images.zip`；点击后会创建 `book_images` 目录。

我们使用的预训练网络称为 `resnet50`；详细规格可在网上找到。我们将读取图像，并将它们转换为 `torch` 软件所期望的数组格式，以符合 `resnet50` 中的规格。转换涉及调整大小、裁剪，然后对三个通道分别进行预定义的标准化。我们现在读取图像并对其进行预处理。

In [61]:
```
resize = Resize((232,232))
crop = CenterCrop(224)
normalize = Normalize([0.485,0.456,0.406],
                      [0.229,0.224,0.225])
imgfiles = sorted([f for f in glob('book_images/*')])
imgs = torch.stack([torch.div(crop(resize(read_image(f))), 255)
                    for f in imgfiles])
imgs = normalize(imgs)
imgs.size()
```

Out[61]:
```
torch.Size([6, 3, 224, 224])
```

我们现在用代码块 6 中读取的权重来设置训练好的网络。该模型有 50 层，相当复杂。

In [62]:
```
resnet_model = resnet50(weights=ResNet50_Weights.DEFAULT)
summary(resnet_model,
        input_data=imgs,
        col_names=['input_size',
                   'output_size',
                   'num_params'])
```

我们将模式设置为`eval()`，以确保模型准备好对新数据进行预测。

In [63]:
```
resnet_model.eval()
```

检查上面的输出，我们可以看到，在设置`resnet_model`时，作者定义了一个`Bottleneck`，很像我们的`BuildingBlock`模块。我们现在将六张图像输入到拟合好的网络中。

In [64]:
```
img_preds = resnet_model(imgs)
```

让我们来看一下前3个选择中每一个的预测概率。首先，我们通过对`img_preds`中的logits应用softmax来计算概率。请注意，我们必须对张量`img_preds`调用`detach()`方法，以便将其转换为我们更熟悉的`ndarray`。

In [65]:
```
img_probs = np.exp(np.asarray(img_preds.detach()))
img_probs /= img_probs.sum(1)[:,None]
```

为了查看类别标签，我们必须下载与`imagenet`.[27]相关的索引文件。

In [66]:
```
labs = json.load(open('imagenet_class_index.json'))
class_labels = pd.DataFrame([(int(k), v[1]) for k, v in
                             labs.items()],
                             columns=['idx', 'label'])
class_labels = class_labels.set_index('idx')
class_labels = class_labels.sort_index()
```

现在，我们将为每个图像文件构建一个数据框，其中包含上述模型估计的三个最高概率标签。

[27] This is avalable from the book website and s3.amazonaws.com/deep-learning-models/image-models/imagenet_class_index.json.

In [67]:
```
for i, imgfile in enumerate(imgfiles):
    img_df = class_labels.copy()
    img_df['prob'] = img_probs[i]
    img_df = img_df.sort_values(by='prob', ascending=False)[:3]
    print(f'Image: {imgfile}')
    print(img_df.reset_index().drop(columns=['idx']))
```

```
Image: book_images/Cape_Weaver.jpg
       label      prob
0    jacamar  0.287283
1  bee_eater  0.046768
2     bulbul  0.037507
Image: book_images/Flamingo.jpg
            label      prob
0        flamingo  0.591761
1       spoonbill  0.012386
2  American_egret  0.002105
Image: book_images/Hawk_Fountain.jpg
            label      prob
0  great_grey_owl  0.287959
1            kite  0.039478
2        fountain  0.029384
Image: book_images/Hawk_cropped.jpg
    label      prob
0    kite  0.301830
1     jay  0.121674
2  magpie  0.015513
Image: book_images/Lhasa_Apso.jpg
             label      prob
0            Lhasa  0.151143
1         Shih-Tzu  0.129850
2  Tibetan_terrier  0.102358
Image: book_images/Sleeping_Cat.jpg
       label      prob
0      tabby  0.173627
1  tiger_cat  0.110414
2    doormat  0.093447
```

我们看到模型对`Flamingo.jpg`相当有信心，但对其他图像则略逊一筹。

我们以通常的清理结束本节。

In [68]:
```
del(cifar_test,
    cifar_train,
    cifar_dm,
    cifar_module,
    cifar_logger,
    cifar_optimizer,
    cifar_trainer)
```

10.9.5 IMDB Document Classification

我们现在在`IMDB`数据集上实现情感分类模型（第10.4节）。如上面的代码块8所述，我们使用的是在`keras`包中找到的`IMDB`数据集的预处理版本。由于`keras`使用

tensorflow，一个不同的张量和深度学习库，我们已将数据转换为适合torch的格式。用于从keras转换的代码可在模块ISLP.torch._make_imdb中找到。它需要一些keras包才能运行。这些数据使用大小为10,000的字典。

我们为本次实验存储了评论数据的三种不同表示形式：

- load_tensor()，一种可由torch使用的稀疏张量版本；
- load_sparse()一个可供sklearn使用的稀疏矩阵版本，因为我们将与lasso拟合进行比较；
- load_sequential()，原始序列表示的填充版本，仅限于每条评论的最后500个词。

In [69]:
```
(imdb_seq_train,
 imdb_seq_test) = load_sequential(root='data/IMDB')
padded_sample = np.asarray(imdb_seq_train.tensors[0][0])
sample_review = padded_sample[padded_sample > 0][:12]
sample_review[:12]
```

Out[69]:
```
array([   1,   14,   22,   16,   43,  530,  973, 1622, 1385,
         65,  458, 4468], dtype=int32)
```

数据集imdb_seq_train和imdb_seq_test都是类TensorDataset的实例。用于构建它们的数据张量可以在tensors属性中找到，第一个张量是特征X，第二个是结果Y。我们已将特征的第一行存储为padded_sample。在构建这些数据的预处理过程中，若序列长度不足，则在起始处用0进行填充，因此我们通过限制到满足padded_sample > 0的条目来去除这种填充。随后，我们提供样本评论的前12个单词。

我们可以在ISLP.torch.imdb模块的lookup词典中找到这些词。

In [70]:
```
lookup = load_lookup(root='data/IMDB')
' '.join(lookup[i] for i in sample_review)
```

Out[70]:
```
"<START> this film was just brilliant casting location scenery
    story direction everyone's"
```

对于我们的第一个模型，我们为数据集中的10,000个可能的单词创建了二元特征，如果在评论i中出现了单词j，则在i,j项中填入1。由于大多数评论相当短，这样的特征矩阵有超过98%的零。这些数据是通过ISLP库中的load_tensor()来访问的。

In [71]:
```
max_num_workers=10
(imdb_train,
 imdb_test) = load_tensor(root='data/IMDB')
imdb_dm = SimpleDataModule(imdb_train,
                           imdb_test,
                           validation=2000,
                           num_workers=min(6, max_num_workers),
                           batch_size=512)
```

We'll use a two-layer model for our first model.

In [72]:
```
class IMDBModel(nn.Module):

    def __init__(self, input_size):
        super(IMDBModel, self).__init__()
        self.dense1 = nn.Linear(input_size, 16)
        self.activation = nn.ReLU()
        self.dense2 = nn.Linear(16, 16)
        self.output = nn.Linear(16, 1)

    def forward(self, x):
        val = x
        for _map in [self.dense1,
                     self.activation,
                     self.dense2,
                     self.activation,
                     self.output]:
            val = _map(val)
        return torch.flatten(val)
```

We now instantiate our model and look at a summary (not shown).

In [73]:
```
imdb_model = IMDBModel(imdb_test.tensors[0].size()[1])
summary(imdb_model,
        input_size=imdb_test.tensors[0].size(),
        col_names=['input_size',
                   'output_size',
                   'num_params'])
```

We'll again use a smaller learning rate for these data, hence we pass an `optimizer` to the `SimpleModule`. Since the reviews are classified into positive or negative sentiment, we use `SimpleModule.binary_classification()`.[28]

In [74]:
```
imdb_optimizer = RMSprop(imdb_model.parameters(), lr=0.001)
imdb_module = SimpleModule.binary_classification(
                         imdb_model,
                         optimizer=imdb_optimizer)
```

Having loaded the datasets into a data module and created a `SimpleModule`, the remaining steps are familiar.

In [75]:
```
imdb_logger = CSVLogger('logs', name='IMDB')
imdb_trainer = Trainer(deterministic=True,
                       max_epochs=30,
                       logger=imdb_logger,
                       callbacks=[ErrorTracker()])
imdb_trainer.fit(imdb_module,
                 datamodule=imdb_dm)
```

Evaluating the test error yields roughly 86% accuracy.

In [76]:
```
test_results = imdb_trainer.test(imdb_module, datamodule=imdb_dm)
test_results
```

[28] Our use of `binary_classification()` instead of `classification()` is due to some subtlety in how `torchmetrics.Accuracy()` works, as well as the data type of the targets.

```
Out[76]: [{'test_loss': 1.0863, 'test_accuracy': 0.8550}]
```

Comparison to Lasso

We now fit a lasso logistic regression model using `LogisticRegression()` from `sklearn`. Since `sklearn` does not recognize the sparse tensors of `torch`, we use a sparse matrix that is recognized by `sklearn`.

```
In [77]: ((X_train, Y_train),
          (X_valid, Y_valid),
          (X_test, Y_test)) = load_sparse(validation=2000,
                                          random_state=0,
                                          root='data/IMDB')
```

Similar to what we did in Section 10.9.1, we construct a series of 50 values for the lasso reguralization parameter λ.

```
In [78]: lam_max = np.abs(X_train.T * (Y_train - Y_train.mean())).max()
         lam_val = lam_max * np.exp(np.linspace(np.log(1),
                                                np.log(1e-4), 50))
```

With `LogisticRegression()` the regularization parameter C is specified as the inverse of λ. There are several solvers for logistic regression; here we use `liblinear` which works well with the sparse input format.

```
In [79]: logit = LogisticRegression(penalty='l1',
                                    C=1/lam_max,
                                    solver='liblinear',
                                    warm_start=True,
                                    fit_intercept=True)
```

The path of 50 values takes approximately 40 seconds to run.

```
In [80]: coefs = []
         intercepts = []

         for l in lam_val:
             logit.C = 1/l
             logit.fit(X_train, Y_train)
             coefs.append(logit.coef_.copy())
             intercepts.append(logit.intercept_)
```

The coefficient and intercepts have an extraneous dimension which can be removed by the `np.squeeze()` function.

```
In [81]: coefs = np.squeeze(coefs)
         intercepts = np.squeeze(intercepts)
```

We'll now make a plot to compare our neural network results with the lasso.

```
In [82]: %%capture
         fig, axes = subplots(1, 2, figsize=(16, 8), sharey=True)
         for ((X_, Y_),
              data_,
              color) in zip([(X_train, Y_train),
                             (X_valid, Y_valid),
                             (X_test, Y_test)],
```

```
                    ['Training', 'Validation', 'Test'],
                    ['black', 'red', 'blue']):
    linpred_ = X_ * coefs.T + intercepts[None,:]
    label_ = np.array(linpred_ > 0)
    accuracy_ = np.array([np.mean(Y_ == l) for l in label_.T])
    axes[0].plot(-np.log(lam_val / X_train.shape[0]),
                 accuracy_,
                 '.--',
                 color=color,
                 markersize=13,
                 linewidth=2,
                 label=data_)
axes[0].legend()
axes[0].set_xlabel(r'$-\log(\lambda)$', fontsize=20)
axes[0].set_ylabel('Accuracy', fontsize=20)
```

注意使用了 %%capture，这会抑制部分显示
完成的图形。这在制作复杂图形时很有用，因为步骤可以分散在两个或多个单元格中。我们现在添加套索准确率的图表，并只需在单元格末尾输入其名称即可显示组合后的图形。

%%capture

In [83]:
```
imdb_results = pd.read_csv(imdb_logger.experiment.metrics_file_path)
summary_plot(imdb_results,
             axes[1],
             col='accuracy',
             ylabel='Accuracy')
axes[1].set_xticks(np.linspace(0, 30, 7).astype(int))
axes[1].set_ylabel('Accuracy', fontsize=20)
axes[1].set_xlabel('Epoch', fontsize=20)
axes[1].set_ylim([0.5, 1]);
axes[1].axhline(test_results[0]['test_accuracy'],
                color='blue',
                linestyle='--',
                linewidth=3)
fig
```

从图中我们可以看到，套索逻辑回归的准确率在大约0.88处达到峰值，与神经网络的表现相同。

再次以清理结束。

In [84]:
```
del(imdb_model,
    imdb_trainer,
    imdb_logger,
    imdb_dm,
    imdb_train,
    imdb_test)
```

10.9.6 Recurrent Neural Networks

在本实验中，我们拟合了第10.5节所示的模型。

用于文档分类的序列模型

这里我们将一个简单的LSTM RNN用于情感预测，拟合到IMDb电影评论数据，如第10.5.1节所讨论。对于RNN，我们使用

文档中单词的序列，考虑其顺序。我们在第10.9.5节开始时加载了预处理后的数据。详细说明预处理过程的脚本可在ISLP库中找到。值得注意的是，由于超过90%的文档少于500个单词，我们将文档长度设为500。对于较长的文档，我们使用最后500个单词，而对于较短的文档，我们在前面用空白填充。

In [85]:
```
imdb_seq_dm = SimpleDataModule(imdb_seq_train,
                               imdb_seq_test,
                               validation=2000,
                               batch_size=300,
                               num_workers=min(6, max_num_workers)
                               )
```

RNN 的第一层是大小为 32 的嵌入层，它将在训练过程中学习。这一层将每个文档进行 one-hot 编码，得到一个维度为 $500 \times 10,003$ 的矩阵，然后将这 $10,003$ 个维度降维到 32 维。[29] 由于每个词都由一个整数表示，这实际上是通过创建一个大小为 $10,003 \times 32$ 的嵌入矩阵来实现的；然后，文档中的 500 个整数分别通过索引该矩阵的相应行，被映射为合适的 32 个实数。

第二层是一个具有32个单元的LSTM，输出层是二分类任务的单个逻辑值。在下面forward()方法的最后一行，我们取LSTM的最后一个32维输出并将其映射到我们的响应。

In [86]:
```
class LSTMModel(nn.Module):
    def __init__(self, input_size):
        super(LSTMModel, self).__init__()
        self.embedding = nn.Embedding(input_size, 32)
        self.lstm = nn.LSTM(input_size=32,
                            hidden_size=32,
                            batch_first=True)
        self.dense = nn.Linear(32, 1)
    def forward(self, x):
        val, (h_n, c_n) = self.lstm(self.embedding(x))
        return torch.flatten(self.dense(val[:,-1]))
```

我们实例化并查看模型的摘要，使用语料库中的前10个文档。

In [87]:
```
lstm_model = LSTMModel(X_test.shape[-1])
summary(lstm_model,
        input_data=imdb_seq_train.tensors[0][:10],
        col_names=['input_size',
                   'output_size',
                   'num_params'])
```

Out[87]:
```
==========================================================================
Layer (type:depth-idx)    Input Shape     Output Shape     Param #
==========================================================================
LSTMModel                 [10, 500]       [10]             --
```

[29] The extra 3 dimensions correspond to commonly occurring non-word entries in the reviews.

```
  Embedding: 1-1          [10, 500]       [10, 500, 32]     320,096
  LSTM: 1-2               [10, 500, 32]   [10, 500, 32]     8,448
  Linear: 1-3             [10, 32]        [10, 1]           33
==========================================================================
Total params: 328,577
Trainable params: 328,577
```

The 10,003 is suppressed in the summary, but we see it in the parameter count, since $10,003 \times 32 = 320,096$.

In [88]:
```
lstm_module = SimpleModule.binary_classification(lstm_model)
lstm_logger = CSVLogger('logs', name='IMDB_LSTM')
```

In [89]:
```
lstm_trainer = Trainer(deterministic=True,
                       max_epochs=20,
                       logger=lstm_logger,
                       callbacks=[ErrorTracker()])
lstm_trainer.fit(lstm_module,
                 datamodule=imdb_seq_dm)
```

The rest is now similar to other networks we have fit. We track the test performance as the network is fit, and see that it attains 85% accuracy.

In [90]:
```
lstm_trainer.test(lstm_module, datamodule=imdb_seq_dm)
```

Out[90]:
```
[{'test_loss': 0.8178, 'test_accuracy': 0.8476}]
```

We once again show the learning progress, followed by cleanup.

In [91]:
```
lstm_results = pd.read_csv(lstm_logger.experiment.metrics_file_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(lstm_results,
             ax,
             col='accuracy',
             ylabel='Accuracy')
ax.set_xticks(np.linspace(0, 20, 5).astype(int))
ax.set_ylabel('Accuracy')
ax.set_ylim([0.5, 1])
```

In [92]:
```
del(lstm_model,
    lstm_trainer,
    lstm_logger,
    imdb_seq_dm,
    imdb_seq_train,
    imdb_seq_test)
```

Time Series Prediction

We now show how to fit the models in Section 10.5.2 for time series prediction. We first load and standardize the data.

In [93]:
```
NYSE = load_data('NYSE')
cols = ['DJ_return', 'log_volume', 'log_volatility']
X = pd.DataFrame(StandardScaler(
                     with_mean=True,
                     with_std=True).fit_transform(NYSE[cols]),
                 columns=NYSE[cols].columns,
                 index=NYSE.index)
```

接下来，我们设置数据的滞后版本，使用dropna()方法删除任何包含缺失值的行。

In [94]:
```
for lag in range(1, 6):
    for col in cols:
        newcol = np.zeros(X.shape[0]) * np.nan
        newcol[lag:] = X[col].values[:-lag]
        X.insert(len(X.columns), "{0}_{1}".format(col, lag), newcol)
X.insert(len(X.columns), 'train', NYSE['train'])
X = X.dropna()
```

最后，我们提取响应、训练指示符，并丢弃当天的 DJ_return 和 log_volatility，仅根据前一天的数据进行预测。

In [95]:
```
Y, train = X['log_volume'], X['train']
X = X.drop(columns=['train'] + cols)
X.columns
```

Out[95]:
```
Index(['DJ_return_1', 'log_volume_1', 'log_volatility_1',
       'DJ_return_2', 'log_volume_2', 'log_volatility_2',
       'DJ_return_3', 'log_volume_3', 'log_volatility_3',
       'DJ_return_4', 'log_volume_4', 'log_volatility_4',
       'DJ_return_5', 'log_volume_5', 'log_volatility_5'],
      dtype='object')
```

我们首先拟合一个简单的线性模型，并使用score()方法在测试数据上计算R^2。

In [96]:
```
M = LinearRegression()
M.fit(X[train], Y[train])
M.score(X[~train], Y[~train])
```

Out[96]:
```
0.4129
```

我们重新拟合了这个模型，其中包括因子变量 day_of_week。对于 pandas 中的分类序列，我们可以使用 get_dummies() 方法形成指示符。

In [97]:
```
X_day = pd.merge(X,
                 pd.get_dummies(NYSE['day_of_week']),
                 on='date')
```

请注意，我们不必重新实例化线性回归模型，因为其 fit() 方法直接接受设计矩阵和响应。

In [98]:
```
M.fit(X_day[train], Y[train])
M.score(X_day[~train], Y[~train])
```

Out[98]:
```
0.4595
```

该模型达到了约46%的R^2。

为了拟合RNN，我们必须重塑数据，因为如下方层 nn.RNN() 的参数 input_shape 所示，它将期望每个特征有5个滞后版本。我们首先确保数据框的列使得重塑后的矩阵能够正确地滞后变量。我们使用 reindex() 方法来实现这一点。

对于输入形状 (5,3)，每一行代表三个变量的一个滞后版本。nn.RNN() 层还期望每个观测的第一行是时间上最早的，因此我们必须颠倒当前的顺序。因此，我们在下面循环遍历 range(5,0,-1)，这是一个使用 slice() 来索引可迭代对象的例子。一般记法是 start:end:step。

In [99]:
```
ordered_cols = []
for lag in range(5,0,-1):
    for col in cols:
        ordered_cols.append('{0}_{1}'.format(col, lag))
X = X.reindex(columns=ordered_cols)
X.columns
```

Out[99]:
```
Index(['DJ_return_5', 'log_volume_5', 'log_volatility_5',
       'DJ_return_4', 'log_volume_4', 'log_volatility_4',
       'DJ_return_3', 'log_volume_3', 'log_volatility_3',
       'DJ_return_2', 'log_volume_2', 'log_volatility_2',
       'DJ_return_1', 'log_volume_1', 'log_volatility_1'],
      dtype='object')
```

我们现在重塑数据。

In [100]:
```
X_rnn = X.to_numpy().reshape((-1,5,3))
X_rnn.shape
```

Out[100]:
```
(6046, 5, 3)
```

通过将第一个尺寸指定为-1，numpy.reshape()根据其余参数推断其尺寸。

现在我们准备继续使用RNN，它包含12个隐藏单元和10%的dropout。经过RNN处理后，我们提取最终时间点，如下面forward()中的val[:,-1]所示。这一结果经过10%的dropout，然后通过线性层展平。

In [101]:
```
class NYSEModel(nn.Module):
    def __init__(self):
        super(NYSEModel, self).__init__()
        self.rnn = nn.RNN(3,
                          12,
                          batch_first=True)
        self.dense = nn.Linear(12, 1)
        self.dropout = nn.Dropout(0.1)
    def forward(self, x):
        val, h_n = self.rnn(x)
        val = self.dense(self.dropout(val[:,-1]))
        return torch.flatten(val)
nyse_model = NYSEModel()
```

我们以与先前网络相似的方式拟合模型。我们将测试数据作为验证数据提供给fit函数，这样当我们监控其进度并绘制历史函数时，就能看到在测试数据上的进展。当然，我们不应将此作为提前停止的依据，因为那样测试性能会有偏差。

我们按照 Hitters 示例类似的方式构建训练数据集。

In [102]:
```
datasets = []
for mask in [train, ~train]:
    X_rnn_t = torch.tensor(X_rnn[mask].astype(np.float32))
    Y_t = torch.tensor(Y[mask].astype(np.float32))
    datasets.append(TensorDataset(X_rnn_t, Y_t))
nyse_train, nyse_test = datasets
```

Following our usual pattern, we inspect the summary.

In [103]:
```
summary(nyse_model,
        input_data=X_rnn_t,
        col_names=['input_size',
                   'output_size',
                   'num_params'])
```

Out[103]:
```
==========================================================================
Layer (type:depth-idx)    Input Shape      Output Shape      Param #
==========================================================================
NYSEModel                 [1770, 5, 3]     [1770]            --
  RNN: 1-1                [1770, 5, 3]     [1770, 5, 12]     204
  Dropout: 1-2            [1770, 12]       [1770, 12]        --
  Linear: 1-3             [1770, 12]       [1770, 1]         13
==========================================================================
Total params: 217
Trainable params: 217
```

We again put the two datasets into a data module, with a batch size of 64.

In [104]:
```
nyse_dm = SimpleDataModule(nyse_train,
                           nyse_test,
                           num_workers=min(4, max_num_workers),
                           validation=nyse_test,
                           batch_size=64)
```

We run some data through our model to be sure the sizes match up correctly.

In [105]:
```
for idx, (x, y) in enumerate(nyse_dm.train_dataloader()):
    out = nyse_model(x)
    print(y.size(), out.size())
    if idx >= 2:
        break
```

```
torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])
```

We follow our previous example for setting up a trainer for a regression problem, requesting the R^2 metric to be be computed at each epoch.

In [106]:
```
nyse_optimizer = RMSprop(nyse_model.parameters(),
                         lr=0.001)
nyse_module = SimpleModule.regression(nyse_model,
                                      optimizer=nyse_optimizer,
                                      metrics={'r2':R2Score()})
```

Fitting the model should by now be familiar. The results on the test data
与线性AR模型非常相似。

In [107]:
```
nyse_trainer = Trainer(deterministic=True,
                       max_epochs=200,
                       callbacks=[ErrorTracker()])
nyse_trainer.fit(nyse_module,
                 datamodule=nyse_dm)
nyse_trainer.test(nyse_module,
                  datamodule=nyse_dm)
```

Out[107]:
```
[{'test_loss': 0.6141, 'test_r2': 0.4172}]
```

我们也可以不使用nn.RNN()层，而是仅使用nn.Flatten()层来拟合模型。这将是一个非线性AR模型。如果额外排除隐藏层，这将等同于我们之前的线性AR模型。

相反，我们将使用包含day_of_week指标的特征集X_day来拟合非线性AR模型。为此，我们必须首先创建测试和训练数据集以及相应的数据模块。这看起来可能有些繁琐，但这是torch通用流程的一部分。

In [108]:
```
datasets = []
for mask in [train, ~train]:
    X_day_t = torch.tensor(
                  np.asarray(X_day[mask]).astype(np.float32))
    Y_t = torch.tensor(np.asarray(Y[mask]).astype(np.float32))
    datasets.append(TensorDataset(X_day_t, Y_t))
day_train, day_test = datasets
```

创建数据模块遵循一个熟悉的模式。

In [109]:
```
day_dm = SimpleDataModule(day_train,
                          day_test,
                          num_workers=min(4, max_num_workers),
                          validation=day_test,
                          batch_size=64)
```

我们构建一个 NonLinearARModel()，它以20个特征作为输入，并包含一个具有32个单元的隐藏层。剩下的步骤大家都很熟悉。

In [110]:
```
class NonLinearARModel(nn.Module):
    def __init__(self):
        super(NonLinearARModel, self).__init__()
        self._forward = nn.Sequential(nn.Flatten(),
                                      nn.Linear(20, 32),
                                      nn.ReLU(),
                                      nn.Dropout(0.5),
                                      nn.Linear(32, 1))
    def forward(self, x):
        return torch.flatten(self._forward(x))
```

In [111]:
```
nl_model = NonLinearARModel()
nl_optimizer = RMSprop(nl_model.parameters(),
                       lr=0.001)
nl_module = SimpleModule.regression(nl_model,
                                    optimizer=nl_optimizer,
                                    metrics={'r2':R2Score()})
```

我们继续进行常规训练步骤，拟合模型，并评估测试误差。我们看到测试 R^2 相较于同样包含 `day_of_week` 的线性 AR 模型略有改进。

In [112]:
```
nl_trainer = Trainer(deterministic=True,
                     max_epochs=20,
                     callbacks=[ErrorTracker()])
nl_trainer.fit(nl_module, datamodule=day_dm)
nl_trainer.test(nl_module, datamodule=day_dm)
```

Out[112]:
```
[{'test_loss': 0.5625, 'test_r2': 0.4662}]
```

10.10 练习

Conceptual

1. 考虑一个具有两个隐藏层的神经网络：$p = 4$个输入单元，第一个隐藏层有2个单元，第二个隐藏层有3个单元，以及一个输出。

   (a) 绘制一张网络图，类似于图10.1或10.4。(b) 假设采用ReLU激活函数，写出$f(X)$的表达式。尽可能明确！(c) 现在为系数代入一些值，并写出$f(X)$的值。(d) 有多少个参数?

2. 考虑(10.13)中的*softmax*函数（另见第145页的(4.13)），用于建模多项概率。

   (a) 在(10.13)中，证明如果给每个z_ℓ加上常数c，则概率不变。(b) 在(4.13)中，证明如果给每个类别的相应系数加上常数$c_j,\ j = 0, 1, \ldots, p$，则在任何新点$x$处的预测不变。

   这表明softmax函数是*over-parametrized*。然而，过度参数化正则化和 SGD 通常会约束解，使其不成问题。 ed

3. 证明当有$M = 2$个类别时，负多项对数似然函数（10.14）等价于似然表达式（4.5）的负对数。

4. 考虑一个接收32 × 32灰度图像的CNN，并具有一个包含三个5 × 5卷积滤波器（无边界填充）的单一卷积层。

   (a) 绘制一个类似于图10.8的输入层和第一隐藏层的草图。

(b) 这个模型中有多少参数？(c) 解释如何将此模型视为一个普通的的前馈神经网络，其中各个像素作为输入，并对隐藏单元的权重施加约束。这些约束是什么？(d) 如果没有约束，那么(c)中普通前馈神经网络会有多少权重?

5. 在第426页的表10.2中，我们看到三种方法在平均绝对误差方面的排序与测试集{v*}下的排序不同。这该如何解释呢?

Applied

6. 考虑简单函数 $R(\beta) = \sin(\beta) + \beta/10$。

(a) 在区间 $\beta \in [-6, 6]$ 上绘制该函数的图像。(b) 该函数的导数是什么？(c) 给定 $\beta^0 = 2.3$，使用学习率 $\rho = 0.1$ 运行梯度下降以找到 $R(\beta)$ 的局部最小值。在图中显示每个 $\beta^0, \beta^1, \ldots$，以及最终答案。(d) 用 $\beta^0 = 1.4$ 重复。

7. 对`Default`数据拟合一个神经网络。使用具有10个单元的单隐藏层，并采用dropout正则化。参考实验10.9.1-10.9.2以获取指导。将你的模型分类性能与线性逻辑回归的分类性能进行比较。

8. 从你的个人照片集中，挑选10张动物的图像（如狗、猫、鸟、农场动物等）。如果主体没有占据图像的合理部分，则裁剪图像。现在，使用如Lab 10.9.4中的预训练图像分类CNN来预测每张图像的类别，并报告每张图像前五个预测类别的概率。

9. 对`NYSE`数据拟合一个滞后5阶的自回归模型，如文中和实验10.9.6所述。用一个表示月份的12级因子重新拟合模型。这个因子是否提高了模型的性能?

10. 在第10.9.6节中，我们展示了如何使用`LinearRegression()`函数将线性AR模型拟合到`NYSE`数据。然而，我们也提到了可以“展平”为RNN模型生成的短序列，以便拟合线性AR模型。使用后一种方法将线性AR模型拟合到`NYSE`数据。比较该线性AR模型的测试R^2与我们在实验中拟合的线性AR模型的测试R^2。每种方法的优缺点是什么?

11. 重复上一练习，但这次通过“展平”为RNN模型生成的短序列来拟合非线性AR模型。

12. 考虑在第10.9.6节中对NYSE数据拟合的RNN。修改代码以允许包含变量day_of_week，并拟合RNN。计算测试R^2。

13. 使用结构相似的神经网络对IMDb数据进行第10.9.5 lab的重复分析。我们在两个隐藏层中各使用了16个隐藏单元。探究将此增加到每层32和64个单元，以及有无30% dropout正则化的影响。

11
生存分析与删失数据



在本章中，我们将讨论*survival analysis*和*censored data*这些出现在对一种独特类型的结果变量的分析中：即*time until an event occurs*。

生存分析

删失数据

例如，假设我们进行了一项为期五年的医学研究，其中患者接受了癌症治疗。我们希望拟合一个模型来预测患者的生存时间，使用的特征包括基线健康测量值或治疗类型等。乍一看，这听起来像是第3章中讨论的那种回归问题。但有一个重要的复杂因素：希望有一部分或许多患者存活到了研究结束。这样的患者的生存时间被称为*censored*：我们知道它至少为五年，但我们不知道其真实值。我们不想丢弃这部分存活患者，因为他们至少存活了五年这一事实本身就是有价值的信息。然而，如何使用本教科书迄今为止所涵盖的技术来利用这些信息，尚不清楚。

尽管“生存分析”这一术语让人联想到医学研究，但生存分析的应用远不止于医学。例如，考虑一家公司希望建模*churn*，即客户取消服务订阅的过程。该公司可能在一段时间内收集客户数据，以便将每位客户取消订阅的时间建模为人口统计学特征或其他预测变量的函数。然而，推测并非所有客户都会在这段时间结束时取消订阅；对于这些客户，取消订阅的时间是删失的。

事实上，生存分析甚至在与时间无关的应用领域中也是相关的。例如，假设我们希望将一个人的体重建模为某些协变量的函数，使用包含大量人群测量值的数据集。不幸的是，用于称量这些人的秤无法报告超过某个数值的体重。那么，任何体重

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_11

超过该数字的数据被删失。本章介绍的生存分析方法可用于分析该数据集。

生存分析在统计学中是一个研究得非常深入的课题，因为它在医学内外的多种应用中都具有极其重要的意义。然而，它在机器学习领域受到的关注相对较少。

11.1 生存时间和删失时间

对于每个个体，我们假设存在一个真实的*survival time*、T，以及作为真正的 *censoring time*，C。（生存时间也称为 *failure time* 或*event time*。）生存时间表示感兴趣的事件发生：例如，患者死亡的时间，或客户取消订阅的时间。相比之下，删失时间是删失发生的时间：例如，患者退出研究或研究结束的时间。

生存时间
删失时间
失效时间 事件时间

我们观察到生存时间T或删失时间C。具体来说，我们观察随机变量

$$Y = \min(T, C). \tag{11.1}$$

换句话说，如果事件在删失之前发生（即$T < C$），那么我们观察到真实生存时间T；然而，如果删失在事件之前发生（$T > C$），那么我们观察到删失时间。我们还观察到一个状态指示器，

$$\delta = \begin{cases} 1 & \text{if } T \leq C \\ 0 & \text{if } T > C. \end{cases}$$

因此，如果我们观测到真实生存时间，则 $\delta = 1$；如果我们观测到的是删失时间，则 $\delta = 0$。

现在，假设我们观察到 n (Y, δ) 对，我们将其表示为 $(y_1, \delta_1), \ldots, (y_n, \delta_n)$。图11.1展示了一个（虚构的）医学研究示例，其中我们观察了 $n = 4$名患者在365天随访期内的情况。对于患者1和3，我们观察到了事件发生时间（如死亡或疾病复发）$T = t_i$。患者2在研究结束时仍存活，患者4退出了研究，即“失访”；对于这些患者，我们观察到了 $C = c_i$。因此，$y_1 = t_1$, $y_3 = t_3$, $y_2 = c_2$, $y_4 = c_4$, $\delta_1 = \delta_3 = 1$，且 $\delta_2 = \delta_4 = 0$。

11.2 更深入地了解截断

为了分析生存数据，我们需要对*why*删失发生的情况做出一些假设。例如，假设有一些患者因为病情非常严重而提前退出了一项癌症研究。如果不考虑患者退出的原因，分析很可能会高估真实的平均生存时间。同样地，假设病情非常严重的男性比

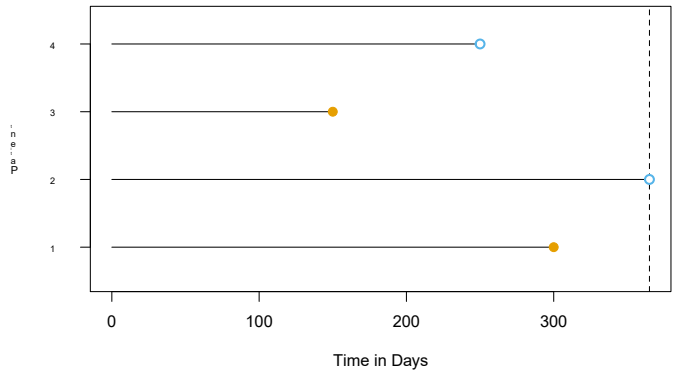


图11.1。*Illustration of censored survival data. For patients 1 and 3, the event was observed. Patient 2 was alive when the study ended. Patient 4 dropped out of the study.*

患病严重的女性。然后对男性和女性生存时间的比较可能会错误地表明男性比女性存活得更久。

一般来说，我们需要假设删失机制是*indepen- dent*：在给定特征的条件下，事件时间T与删失时间C独立。上述两个例子违反了独立删失的假设。通常，无法仅从数据本身判断删失机制是否独立。相反，必须仔细考虑数据收集过程，以确定独立删失是否为一个合理的假设。在本章的剩余部分，我们将假设删失机制是独立的。[1]

在本章中，我们聚焦于*right censoring*，它发生于$T \geq Y$时，即真实事件时间T至少与观测时间Y一样大。（注意，$T \geq Y$是（11.1）的结果。右删失之名源于时间通常从左至右显示，如图11.1所示。）然而，其他类型的删失也是可能的。例如，在*left censoring*中，真实事件时间T小于或等于观测时间Y。以妊娠期研究为例，假设我们在受孕后250天对患者进行调查，此时部分患者已经分娩。那么，我们知道对于这些患者，妊娠期少于250天。更一般地，*interval censoring*指的是我们不知道确切事件时间，但知道它落在某个区间内的情况。例如，当我们每周对患者进行一次调查以确定事件是否发生时，就会出现这种情况。虽然左删失和区间删失可以通过本章所介绍思想的变体来处理，但在下文中，我们特别关注右删失。

[1]The assumption of independent censoring can be relaxed somewhat using the notion of *non-informative censoring*; however, the definition of non-informative censoring is too technical for this book.

11.3 Kaplan-Meier 生存曲线

survival curve 或 *survival function* 定义为

生存曲线生存函数

$$S(t) = \Pr(T > t). \tag{11.2}$$

这个递减函数量化了存活超过时间 t 的概率。例如，假设一家公司有兴趣对客户流失进行建模。令 T 表示客户取消公司服务订阅的时间。那么 $S(t)$ 表示客户取消时间晚于 t 的概率。$S(t)$ 的值越大，客户在时间 t 之前取消的可能性就越小。

在本节中，我们将考虑估计生存曲线的任务。我们的研究动机来源于 `BrainCancer` 数据集，该数据集包含了接受立体定向放射方法治疗的原发性脑肿瘤患者的生存时间。[2] 预测变量包括 `gtv` (肿瘤总体积，单位为立方厘米)；`sex` (男性或女性)；`diagnosis` (脑膜瘤、低级别胶质瘤、高级别胶质瘤或其他)；`loc` (肿瘤位置：幕下或幕上)；`ki` (卡氏指数)；以及 `stereo` (立体定向方法：立体定向放射外科或分次立体定向放疗，分别缩写为SRS和SRT)。在88名患者中，研究结束时仅有53人仍然存活。

现在，我们考虑根据这些数据估计生存曲线（11.2）的任务。为了估计 $S(20) = \Pr(T > 20)$，即患者至少存活t =20个月的概率，人们很容易简单地计算已知存活超过20个月的患者比例，即满足Y >20的患者比例。结果约为48/88，即大约55%。然而，这似乎并不完全正确，因为Y和T代表不同的量。特别地，在未存活到20个月的40名患者中，有17名实际上被删失，而这种分析隐含地假设所有这些被删失患者都满足T <20；当然，我们并不知道这是否成立。

另外，为了估计$S(20)$，我们可以考虑计算在截至时间t =20被not删失的71名患者中，Y >20的患者比例；这算出来是48/71，约等于68%。然而，这也不完全正确，因为这相当于完全忽略了在时间t =20之前被删失的患者，即便他们被删失的$time$可能具有潜在信息价值。例如，一个在时间t =19.9被删失的患者，如果没有被删失，很可能存活超过t =20。

我们已经看到，由于存在删失，估计$S(t)$变得复杂。我们现在提出一种克服这些挑战的方法。我们让$d_1 < d_2 < \cdots < d_K$表示非删失患者中的K个唯一死亡时间，并让q_k表示在时间d_k死亡的患者数。对于$k = 1, \ldots, K$，我们让r_k表示存活的患者数。

[2]This dataset is described in the following paper: Selingerová et al. (2016) Survival of patients with primary brain tumors: Comparison of two statistical approaches. PLoS One, 11(2):e0148733.

而在紧邻 d_k 之前的研究中；这些是 *at risk* 患者。在给定时间处于风险中的患者集合被称为 *risk set*。

risk set

根据全概率公式，[3]

$$\begin{aligned}\Pr(T > d_k) = &\Pr(T > d_k | T > d_{k-1}) \Pr(T > d_{k-1}) \\ &+ \Pr(T > d_k | T \leq d_{k-1}) \Pr(T \leq d_{k-1}).\end{aligned}$$

$d_{k-1} < d_k$ 这一事实意味着 $\Pr(T > d_k | T \leq d_{k-1}) = 0$（如果患者未能存活至较早的时间 d_{k-1}，则不可能存活超过时间 d_k）。因此，

$$S(d_k) = \Pr(T > d_k) = \Pr(T > d_k | T > d_{k-1}) \Pr(T > d_{k-1}).$$

再次代入（11.2），我们看到

$$S(d_k) = \Pr(T > d_k | T > d_{k-1}) S(d_{k-1}).$$

这意味着

$$S(d_k) = \Pr(T > d_k | T > d_{k-1}) \times \cdots \times \Pr(T > d_2 | T > d_1) \Pr(T > d_1).$$

现在，我们只需将前一个方程右侧各项的估计值代入即可。自然可以使用估计量

$$\widehat{\Pr}(T > d_j | T > d_{j-1}) = (r_j - q_j)/r_j,$$

这是在时间d_j的风险集中，活过时间d_j的比例。这导致了生存曲线的 *Kaplan–Meier estimator*：

Kaplan-Meier估计量

$$\widehat{S}(d_k) = \prod_{j=1}^{k} \left(\frac{r_j - q_j}{r_j} \right). \tag{11.3}$$

对于d_k与d_{k+1}之间的t时刻，我们设定$\widehat{S}(t) = \widehat{S}(d_k)$。因此，Kaplan-Meier生存曲线呈阶梯状。

`BrainCancer` 数据的 Kaplan–Meier 生存曲线如图 11.2 所示。实线阶梯状曲线中的每个点显示了存活超过横轴所指示时间的估计概率。存活超过20个月的估计概率为71%，这比之前给出的55%和68%的朴素估计要高得多。

Kaplan–Meier 估计量的序贯构建——从时间零点开始，并按时间顺序描绘观察到的事件——是生存分析中许多关键技术的基础。这些技术包括第 11.4 节的对数秩检验，以及第 11.5.2 节的 Cox 比例风险模型。

[3]The law of total probability states that for any two events A and B, $\Pr(A) = \Pr(A|B)\Pr(B) + \Pr(A|B^c)\Pr(B^c)$, where B^c is the complement of the event B, i.e. it is the event that B does not hold.

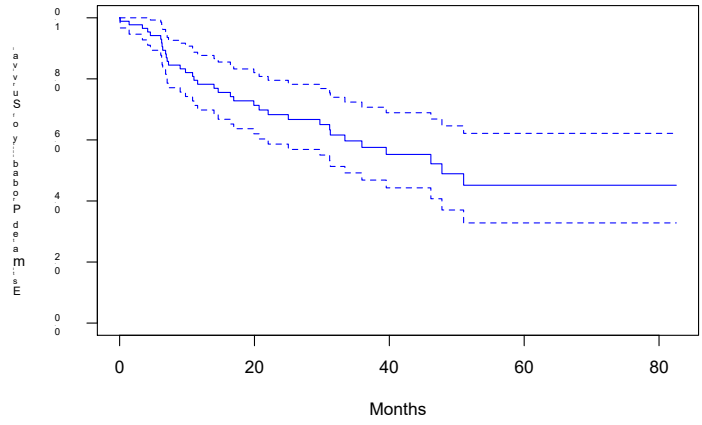


图 11.2. *For the* `BrainCancer` *data, we display the Kaplan–Meier survival curve (solid curve), along with standard error bands (dashed curves).*

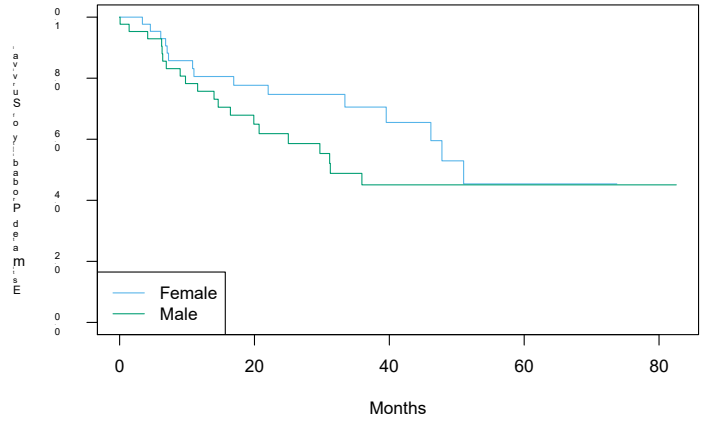


图 11.3. *For the* `BrainCancer` *data, Kaplan–Meier survival curves for males and females are displayed.*

11.4 对数秩检验

我们现在继续对第11.3节中引入的`BrainCancer`数据进行分析。我们希望比较男性和女性的生存情况。图11.3显示了这两组的Kaplan–Meier生存曲线。在约50个月之前，女性的情况似乎稍好一些，但随后两条曲线都趋于平稳，大约在50%左右。我们如何进行两生存曲线相等性的正式检验呢?

乍一看，两样本t检验似乎是一个显而易见的选择：我们可以检验雌性的平均生存时间是否等于雄性的平均生存时间。但删失的存在再次带来了复杂性。为了克服这一挑战，我们将进行*log-rank test*、[4]

log-rank test

	Group 1	Group 2	Total
Died	q_{1k}	q_{2k}	q_k
Survived	$r_{1k} - q_{1k}$	$r_{2k} - q_{2k}$	$r_k - q_k$
Total	r_{1k}	r_{2k}	r_k

表 11.1. *Among the set of patients at risk at time d_k, the number of patients who died and survived in each of two groups is reported.*

它考察了每个组中的事件如何在时间上依次展开。

回忆第11.3节，$d_1 < d_2 < \cdots < d_K$是非删失患者中的唯一死亡时间，r_k是在时间d_k处于风险中的患者数，q_k是在时间d_k死亡的患者数。我们进一步定义r_{1k}和r_{2k}分别为在时间d_k处于风险中的第1组和第2组患者数。类似地，我们定义q_{1k}和q_{2k}分别为在时间d_k死亡的第1组和第2组患者数。注意$r_{1k} + r_{2k} = r_k$和$q_{1k} + q_{2k} = q_k$。

在每个死亡时间d_k，我们构建一个2×2的计数表，形式如表11.1所示。注意，如果死亡时间是唯一的（即没有两个人同时死亡），那么q_{1k}和q_{2k}中一个等于一，另一个等于零。

对数秩检验统计量背后的主要思想如下。为了检验 H_0： $\mathrm{E}(X) = \mu$ 对于某个随机变量 X，一种方法是构造如下形式的检验统计量

$$W = \frac{X - \mu}{\sqrt{\mathrm{Var}(X)}}. \tag{11.4}$$

为了构建对数秩检验统计量，我们计算一个量，其形式恰好为(11.4)，其中$X = \sum_{k=1}^{K} q_{1k}$，而$q_{1k}$的值如表11.1左上角所示。

更详细地说，如果两组之间的生存率没有差异，并且在表11.1中固定行和列的总数，则q_{1k}的期望值为

$$\mu_k = \frac{r_{1k}}{r_k} q_k. \tag{11.5}$$

因此，$X = \sum_{k=1}^{K} q_{1k}$ 的期望值是 $\mu = \sum_{k=1}^{K} \frac{r_{1k}}{r_k} q_k$。此外，可以证明[5]，$q_{1k}$ 的方差是

$$\mathrm{Var}\left(q_{1k}\right) = \frac{q_k(r_{1k}/r_k)(1 - r_{1k}/r_k)(r_k - q_k)}{r_k - 1}. \tag{11.6}$$

尽管 $q_{11}, \ldots, q_{1K}$ 可能是相关的，我们仍然估计

$$\mathrm{Var}\left(\sum_{k=1}^{K} q_{1k}\right) \approx \sum_{k=1}^{K} \mathrm{Var}\left(q_{1k}\right) = \sum_{k=1}^{K} \frac{q_k(r_{1k}/r_k)(1 - r_{1k}/r_k)(r_k - q_k)}{r_k - 1}. \tag{11.7}$$

[4]The log-rank test is also known as the *Mantel–Haenszel test* or *Cochran–Mantel–Haenszel test.*

[5]For details, see Exercise 7 at the end of this chapter.

因此，为了计算对数秩检验统计量，我们只需按照(11.4)中的步骤进行，使用$X = \sum_{k=1}^{K} q_{1k}$，并利用(11.5)和(11.7)。也就是说，我们计算

$$W = \frac{\sum_{k=1}^{K}(q_{1k} - \mu_k)}{\sqrt{\sum_{k=1}^{K}\mathrm{Var}(q_{1k})}} = \frac{\sum_{k=1}^{K}\left(q_{1k} - \frac{q_k}{r_k}r_{1k}\right)}{\sqrt{\sum_{k=1}^{K}\frac{q_k(r_{1k}/r_k)(1-r_{1k}/r_k)(r_k-q_k)}{r_k-1}}}. \quad (11.8)$$

当样本量较大时，对数秩检验统计量W近似服从标准正态分布；这可用于计算零假设下两生存曲线无差异的p值。[6]

在`BrainCancer`数据上比较女性和男性的生存时间，得到对数秩检验统计量为$W = 1.2$，这对应于使用理论零分布的双侧p值为0.2，以及使用1,000次排列的排列零分布的p值为0.25。因此，我们不能拒绝女性和男性之间生存曲线无差异的零假设。

对数秩检验与Cox比例风险模型密切相关，我们将在第11.5.2节中讨论此模型。

11.5 含生存响应的回归模型

我们现在考虑将回归模型拟合到生存数据的任务。如第11.1节所述，观测数据的形式为 (Y, δ)，其中 $Y = \min(T, C)$ 是（可能删失的）生存时间，而 δ 是一个指示变量，如果 $T \leq C$ 则等于1。此外，$X \in \mathbb{R}^p$ 是一个包含 p 个特征的向量。我们希望预测真实的生存时间 T。

由于观测量 Y 为正且可能具有长右尾，我们可能会倾向于对 $\log(Y)$ 关于 X 进行线性回归拟合。但读者想必会猜到，删失再次带来了问题，因为我们实际感兴趣的是预测 T 而非 Y。为克服这一困难，我们转而采用一种序贯构造，类似于第11.3节中Kaplan–Meier生存曲线的构造以及第11.4节中对数秩检验的构造。

11.5.1 The Hazard Function

hazard function 或 *hazard rate* —— 也称为 *force of mortality* — 正式定义为

hazard function

$$h(t) = \lim_{\Delta t \to 0} \frac{\Pr(t < T \leq t + \Delta t | T > t)}{\Delta t}, \quad (11.9)$$

[6] Alternatively, we can estimate the p-value via permutations, using ideas that will be presented in Section 13.5. The permutation distribution is obtained by randomly swapping the labels for the observations in the two groups.

其中 T 是（未观测到的）生存时间。它是在时间 t 之后瞬间的死亡率，前提是存活超过该时间。[7] 在 (11.9) 中，我们取 Δt 趋近于零时的极限，因此我们可以将 Δt 视为一个极其微小的数。因此，更非正式地说，(11.9) 意味着

$$h(t) \approx \frac{\Pr(t < T \leq t + \Delta t | T > t)}{\Delta t}$$

对于某个任意小的 Δt。

我们为什么应该关注风险函数？首先，它与生存曲线（11.2节）密切相关，这一点我们将在接下来看到。其次，事实证明，将生存数据建模为协变量函数的关键方法在很大程度上依赖于风险函数；我们将在11.5.2节介绍这种方法——考克斯比例风险模型。

我们现在更详细地考虑风险函数 $h(t)$。回想对于两个事件 A 和 B，在 B 给定下 A 的概率可以表示为 $\Pr(A \mid B) = \Pr(A \cap B)/\Pr(B)$，即 A 和 B 同时发生的概率除以 B 发生的概率。此外，回想 (11.2) 中的 $S(t) = \Pr(T > t)$。因此，

$$\begin{aligned} h(t) &= \lim_{\Delta t \to 0} \frac{\Pr\left((t < T \leq t + \Delta t) \cap (T > t)\right)/\Delta t}{\Pr(T > t)} \\ &= \lim_{\Delta t \to 0} \frac{\Pr(t < T \leq t + \Delta t)/\Delta t}{\Pr(T > t)} \\ &= \frac{f(t)}{S(t)}, \end{aligned} \tag{11.10}$$

其中

$$f(t) = \lim_{\Delta t \to 0} \frac{\Pr(t < T \leq t + \Delta t)}{\Delta t} \tag{11.11}$$

是与 T 关联的 *probability density function*，即它是瞬时的时间 t 的瞬时死亡率。(11.10) 中的第二个等式利用了这样一个事实：如果 $t < T \leq t + \Delta t$，那么必定有 $T > t$。

probability density function

方程11.10暗示了危险函数$h(t)$、生存函数$S(t)$和概率密度函数$f(t)$之间的关系。实际上，它们是描述T分布的三种等价方式[8]。

与第i个观测值相关的可能性为

$$\begin{aligned} L_i &= \begin{cases} f(y_i) & \text{if the } i\text{th observation is not censored} \\ S(y_i) & \text{if the } i\text{th observation is censored} \end{cases} \\ &= f(y_i)^{\delta_i} S(y_i)^{1-\delta_i}. \end{aligned} \tag{11.12}$$

(11.12) 背后的直觉如下：如果 $Y = y_i$ 且第 i 个观测未被删失，则似然是在时间 y_i 附近的一个小时间段内死亡的概率。如果第 i 个观测被删失，则似然

[7] Due to the Δt in the denominator of (11.9), the hazard function is a rate of death, rather than a probability of death. However, higher values of $h(t)$ directly correspond to a higher probability of death, just as higher values of a probability density function correspond to more likely outcomes for a random variable. In fact, $h(t)$ is the probability density function for T conditional on $T > t$.

[8] See Exercise 8.

至少存活到时间 y_i 的概率。假设 n 个观测是独立的，数据的似然函数形式为

$$L = \prod_{i=1}^{n} f(y_i)^{\delta_i} S(y_i)^{1-\delta_i} = \prod_{i=1}^{n} h(y_i)^{\delta_i} S(y_i), \tag{11.13}$$

其中第二个等式由 (11.10) 得出。

我们现在考虑对生存时间进行建模的任务。如果我们假设生存时间服从指数分布，即生存时间T的概率密度函数取形式$f(t) = \lambda \exp(-\lambda t)$，那么通过最大化(11.13)式中的似然函数来估计参数λ是直接的。[9] 另一种方法是，我们可以假设生存时间是从更灵活的分布族中抽取的，例如Gamma分布或Weibull分布族。还有一种可能性是对生存时间进行非参数建模，如第11.3节中使用Kaplan–Meier估计器所做的那样。

然而，我们真正想要做的是对生存时间 *as a function of the covariates* 进行建模。为此，直接处理风险函数而非概率密度函数会更加方便。[10] 一种可能的方法是假设风险函数 $h(t|x_i)$ 具有某种函数形式，例如 $h(t|x_i) = \exp\left(\beta_0 + \sum_{j=1}^{p} \beta_j x_{ij}\right)$，其中指数函数保证了风险函数的非负性。请注意，指数风险函数是特殊的，因为它不随时间变化。[11] 给定 $h(t|x_i)$，我们可以计算 $S(t|x_i)$。将这些方程代入 (11.13)，我们就可以最大化似然函数以估计参数 $\beta = (\beta_0, \beta_1, \ldots, \beta_p)^T$。然而，这种方法相当受限，因为它要求我们对风险函数 $h(t|x_i)$ 的形式做出非常严格的假设。在下一节中，我们将考虑一种更加灵活的方法。

11.5.2 Proportional Hazards

比例风险假设

proportional hazards assumption 指出

比例风险假设

$$h(t|x_i) = h_0(t) \exp\left(\sum_{j=1}^{p} x_{ij}\beta_j\right), \tag{11.14}$$

其中 $h_0(t) \geq 0$ 是一个未指定的函数，称为 *baseline hazard*。

基线风险

这是具有特征 $x_{i1} = \cdots = x_{ip} = 0$ 的个体的危险函数。“比例危险”这一名称源于这样一个事实：具有特征向量 x_i 的个体的危险函数是某个未知函数

[9] See Exercise 9.

[10] Given the close relationship between the hazard function $h(t)$ and the density function $f(t)$ explored in Exercise 8, posing an assumption about the form of the hazard function is closely related to posing an assumption about the form of the density function, as was done in the previous paragraph.

[11] The notation $h(t|x_i)$ indicates that we are now considering the hazard function for the ith observation conditional on the values of the covariates, x_i.

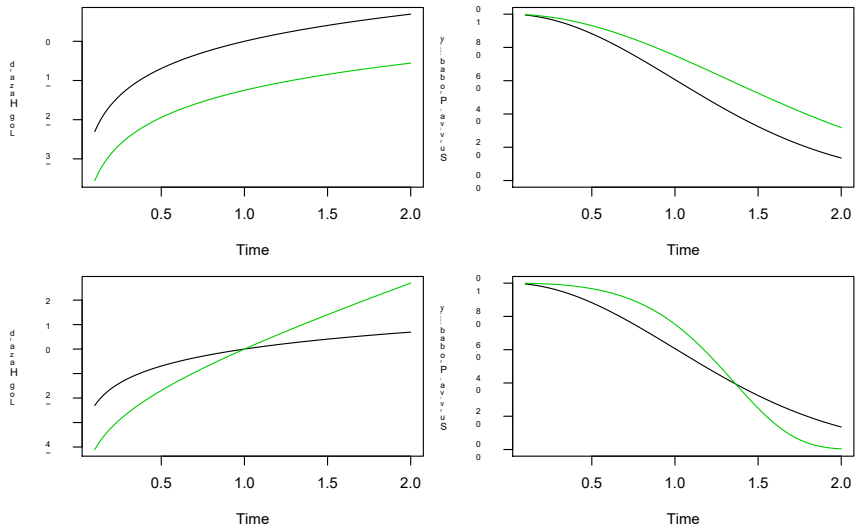


图11.4. 上：*In a simple example with* $p = 1$ *and a binary covariate* $x_i \in \{0, 1\}$*, the log hazard and the survival function under the model* (11.14) *are shown (green for* $x_i = 0$ *and black for* $x_i = 1$*). Because of the proportional hazards assumption* (11.14)*, the log hazard functions differ by a constant, and the survival functions do not cross.* 下：*Again we have a single binary covariate* $x_i \in \{0, 1\}$*. However, the proportional hazards assumption* (11.14) *does not hold. The log hazard functions cross, as do the survival functions.*

$h_0(t)$ 乘以因子 $\exp\left(\sum_{j=1}^{p} x_{ij}\beta_j\right)$。量 $\exp\left(\sum_{j=1}^{p} x_{ij}\beta_j\right)$ 称为特征向量 $x_i = (x_{i1}, \ldots, x_{ip})^T$ 相对于特征向量 $x_i = (0, \ldots, 0)^T$ 的 *relative risk*。

在（11.14）中，基线风险函数$h_0(t)$未指定意味着什么？基本上，我们对其函数形式不作任何假设。我们允许在时间t瞬间死亡的概率，在至少存活至时间t的条件下，采取任何形式。这意味着风险函数非常灵活，能够模拟协变量与生存时间之间广泛的关系。我们唯一的假设是，x_{ij}增加一个单位对应着$h(t|x_i)$以exp(β_j)的因子增加。

比例风险假设（11.14）的图示见图11.4，在一个简单的设定中，仅有一个二值协变量$x_i \in \{0, 1\}$ (，因此$p = 1$)。在顶行，比例风险假设（11.14）成立。因此，两组的风险函数彼此之间是常数倍关系，所以在对数尺度上，它们之间的差距是恒定的。此外，生存曲线从不交叉，事实上，生存曲线之间的差距（最初）倾向于随时间增加。相比之下，在底行，（11.14）不成立。我们看到两组的对数风险函数交叉，生存曲线也是如此。

Cox比例风险模型

由于比例风险假设（11.14）中$h_0(t)$的形式未知，我们不能简单地将$h(t|x_i)$代入似然函数（11.13），然后通过最大似然估计$\beta = (\beta_1, \ldots, \beta_p)^T$。*Cox's proportional hazards model*的神奇之处在于，实际上有可能估计β *without having to specify the form of* $h_0(t)$。

Cox比例风险模型

为此，我们采用了与推导Kaplan–Meier生存曲线和对数秩检验时相同的“时间序列”逻辑。为简化起见，假设失败或死亡时间之间没有并列：即每次失败发生在不同的时间。假设$\delta_i = 1$，即第i个观测值未删失，因此y_i是其失败时间。那么，第i个观测值在时间y_i的风险函数为$h(y_i|x_i) = h_0(y_i)\exp\left(\sum_{j=1}^p x_{ij}\beta_j\right)$，而在时间$y_i$对于处于风险中的观测值[12]的总风险为

$$\sum_{i':y_{i'}\geq y_i} h_0(y_i)\exp\left(\sum_{j=1}^p x_{i'j}\beta_j\right).$$

因此，第i个观测在时间y_i (失败而非风险集中)的其他观测之一的概率是

$$\frac{h_0(y_i)\exp\left(\sum_{j=1}^p x_{ij}\beta_j\right)}{\sum_{i':y_{i'}\geq y_i} h_0(y_i)\exp\left(\sum_{j=1}^p x_{i'j}\beta_j\right)} = \frac{\exp\left(\sum_{j=1}^p x_{ij}\beta_j\right)}{\sum_{i':y_{i'}\geq y_i}\exp\left(\sum_{j=1}^p x_{i'j}\beta_j\right)}. \tag{11.15}$$

注意，未指定的基线风险函数$h_0(y_i)$在分子和分母中相互抵消了！

*partial likelihood*简单地就是这些概率在所有未删失观测的，

partial likelihood

$$PL(\beta) = \prod_{i:\delta_i=1}\frac{\exp\left(\sum_{j=1}^p x_{ij}\beta_j\right)}{\sum_{i':y_{i'}\geq y_i}\exp\left(\sum_{j=1}^p x_{i'j}\beta_j\right)}. \tag{11.16}$$

至关重要的是，部分似然函数无论$h_0(t)$的真实值如何都是有效的，这使得模型非常灵活且稳健。[13]

为了估计 β，我们只需最大化关于 β 的部分似然函数（11.16）。正如第4章中逻辑回归的情况，不存在闭式解，因此需要使用迭代算法。

除了估计 β 之外，我们还可以获得在第3章的最小二乘回归和第4章的逻辑回归中看到的其他模型输出。例如，我们可以获得对应于...的 p 值

[12]Recall that the “at risk” observations at time y_i are those that are still at risk of failure, i.e. those that have not yet failed or been censored before time y_i.

[13]In general, the partial likelihood is used in settings where it is difficult to compute the full likelihood for all of the parameters. Instead, we compute a likelihood for just the parameters of primary interest: in this case, $\beta_1, \ldots, \beta_p$. It can be shown that maximizing (11.16) provides good estimates for these parameters.

针对特定的零假设（例如 $H_0 : \beta_j = 0$），以及与系数相关的置信区间。

与对数秩检验的关联

假设我们只有一个预测变量（$p = 1$），我们假定它是二元的，即 $x_i \in \{0, 1\}$。为了确定组 $\{i : x_i = 0\}$ 中的观测值与组 $\{i : x_i = 1\}$ 中的观测值在生存时间上是否存在差异，我们可以考虑采取两种可能的方法：

> *Approach #1:* 拟合Cox比例风险模型，并检验原假设 $H_0 : \beta = 0$。（由于 $p = 1$，β 是标量。）
>
> *Approach #2:* 进行对数秩检验以比较两组，如第11.4节所述。

我们应该更倾向于哪一个?

事实上，这两种方法之间存在密切关系。特别地，当采用方法 #1 时，有多种可能的方式来检验 H_0。其中一种方式被称为得分检验。结果表明，在单个二值协变量的情况下，Cox 比例风险模型中 H_0： $\beta = 0$ 的得分检验恰好等于对数秩检验。换句话说，无论我们采用方法 #1 还是方法 #2，都没有区别!

额外细节

对Cox比例风险模型的讨论略过了一些微妙之处：

- 在(11.14)及随后的方程中均无截距项，因为截距可被吸收到基线风险函数$h_0(t)$中。
- 我们假设没有相同的失效时间。在存在相同时间的情况下，部分似然函数（11.16）的确切形式会稍微复杂一些，必须采用多种计算近似方法。
- (11.16)被称为*partial*似然，因为它并非严格意义上的似然。也就是说，它并不完全对应于在假设(11.14)下数据的概率。然而，这是一个非常好的近似。
- 我们仅关注系数 $\beta = (\beta_1, \ldots, \beta_p)^T$ 的估计。然而，有时我们也可能希望估计基线风险 $h_0(t)$，例如以便我们可以估计具有特征向量 x 的个体的生存曲线 $S(t|x)$。详细内容超出了本书的范围。$h_0(t)$ 的估计在 `Python` 中的 `lifelines` 包中实现，我们将在第11.8节中看到。

11.5.3 Example: Brain Cancer Data

表11.2展示了将比例风险模型拟合到BrainCancer数据的结果，这一数据最初在第11.3节中描述。系数列显示了$\hat{\beta}_j$。结果表明，例如，男性患者的估计风险是女性患者的$e^{0.18}$ =1.2倍：换句话说，在其他所有特征保持不变的情况下，男性在任何时间点的死亡几率都比女性高1.2倍。然而，p值为0.61，表明男性和女性之间的这一差异并不显著。

再举一例，我们还发现，Karnofsky指数每增加一个单位，对应于瞬时死亡概率乘以exp(−0.05) = 0.95。换言之，Karnofsky指数越高，在任何给定时间点的死亡概率越低。这一效应极为显著，其p值为0.0027。

	Coefficient	Std. error	z-statistic	p-value
sex[Male]	0.18	0.36	0.51	0.61
diagnosis[LG Glioma]	0.92	0.64	1.43	0.15
diagnosis[HG Glioma]	2.15	0.45	4.78	0.00
diagnosis[Other]	0.89	0.66	1.35	0.18
loc[Supratentorial]	0.44	0.70	0.63	0.53
ki	-0.05	0.02	-3.00	<0.01
gtv	0.03	0.02	1.54	0.12
stereo[SRT]	0.18	0.60	0.30	0.77

表 11.2. *Results for Cox's proportional hazards model fit to the* BrainCancer *data, which was first described in Section 11.3. The variable* diagnosis *is qualitative with four levels: meningioma, LG glioma, HG glioma, or other. The variables* sex, loc, *and* stereo *are binary.*

11.5.4 Example: Publication Data

接下来，我们考虑数据集Publication，涉及报告由美国国家心肺血液研究所资助的临床试验结果的期刊论文发表时间。[14]对于244项试验，记录了直至发表的月份数。在这244项试验中，仅有156项在研究期间发表；其余研究被删失。协变量包括试验是否聚焦于临床终点（clinend）、试验是否涉及多个中心（multi）、美国国立卫生研究院内的资助机制（mech）、试验样本量（sampsize）、预算（budget）、影响力（impact，与引用次数相关），以及试验是否产生了阳性（显著）结果（posres）。最后一个协变量尤为引人关注，因为多项研究表明，阳性试验的发表率更高。

[14]This dataset is described in the following paper: Gordon et al. (2013) Publication of trials funded by the National Heart, Lung, and Blood Institute. New England Journal of Medicine, 369(20):1926–1934.

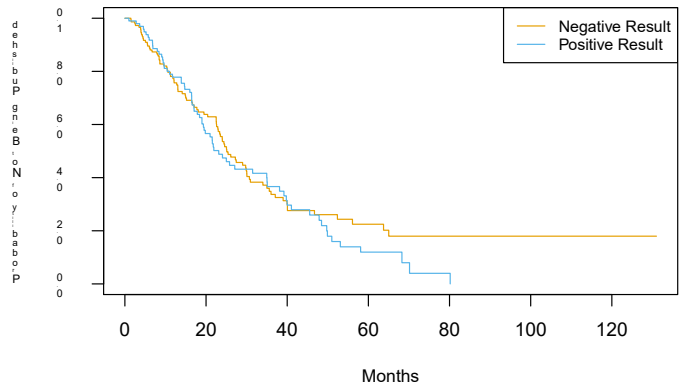


图 11.5. *Survival curves for time until publication for the* `Publication` *data described in Section 11.5.4, stratified by whether or not the study produced a positive result.*

图11.5展示了从研究完成到发表的时间的Kaplan-Meier曲线，按研究是否产生阳性结果进行分层。我们看到有轻微证据表明，阳性结果的研究发表时间较短。然而，对数秩检验得出的p值为0.36，并不显著。

我们现在考虑一个更细致的分析，该分析利用了所有可用的预测变量。使用所有可用特征拟合Cox比例风险模型的结果见表11.3。我们发现，在所有其他协变量保持固定的情况下，一项具有阳性结果的研究在任何时间点发表的机会是阴性结果研究发表机会的$e^{0.55} = 1.74$倍。表11.3中与`posres`相关的极小的p值表明这一结果高度显著。这一点尤其引人注目，特别是考虑到我们之前的发现，即比较阳性与阴性结果研究的发表时间的对数秩检验得出的p值为0.36。我们如何解释这种差异？答案源于这样一个事实：对数秩检验没有考虑任何其他协变量，而表11.3中的结果基于使用所有可用协变量的Cox模型。换句话说，在我们调整了所有其他协变量之后，研究是否产生阳性结果对发表时间具有高度预测性。

为了更深入地理解这一结果，我们在图11.6中展示了与阳性及阴性结果相关的生存曲线估计，并调整了其他预测因素的影响。为了生成这些生存曲线，我们估计了基础风险函数$h_0(t)$。此外，我们还需为其他预测因子选取代表性值；我们采用了每个预测因子的平均值，但分类预测因子`mech`除外，对其我们使用的是最普遍类别（`RO1`）。在调整了其他预测因素后，我们现在清楚地看到，阳性结果与阴性结果的研究之间，在生存曲线上存在显著差异。

从表11.3中还可以获得其他有趣的见解。例如，与采用非临床终点的研究相比，采用临床终点的研究在任何给定时间点都更有可能被发表。资金

	Coefficient	Std. error	z-statistic	p-value
posres[Yes]	0.55	0.18	3.02	0.00
multi[Yes]	0.15	0.31	0.47	0.64
clinend[Yes]	0.51	0.27	1.89	0.06
mech[K01]	1.05	1.06	1.00	0.32
mech[K23]	-0.48	1.05	-0.45	0.65
mech[P01]	-0.31	0.78	-0.40	0.69
mech[P50]	0.60	1.06	0.57	0.57
mech[R01]	0.10	0.32	0.30	0.76
mech[R18]	1.05	1.05	0.99	0.32
mech[R21]	-0.05	1.06	-0.04	0.97
mech[R24,K24]	0.81	1.05	0.77	0.44
mech[R42]	-14.78	3414.38	-0.00	1.00
mech[R44]	-0.57	0.77	-0.73	0.46
mech[RC2]	-14.92	2243.60	-0.01	0.99
mech[U01]	-0.22	0.32	-0.70	0.48
mech[U54]	0.47	1.07	0.44	0.66
sampsize	0.00	0.00	0.19	0.85
budget	0.00	0.00	1.67	0.09
impact	0.06	0.01	8.23	0.00

表 11.3. *Results for Cox's proportional hazards model fit to the* Publication *data, using all of the available features. The features* posres, multi, *and* clinend *are binary. The feature* mech *is qualitative with 14 levels; it is coded so that the baseline level is* Contract.

发表前的机制似乎与时间没有显著关联。

11.6 Cox模型的收缩

在本节中，我们展示6.2节的收缩方法可应用于生存数据设置。特别是，受6.2节“损失+惩罚”表述的启发，我们考虑最小化(11.16)中负对数部分似然的惩罚版本，

$$-\log\left(\prod_{i:\delta_i=1}\frac{\exp\left(\sum_{j=1}^{p}x_{ij}\beta_j\right)}{\sum_{i':y_{i'}\geq y_i}\exp\left(\sum_{j=1}^{p}x_{i'j}\beta_j\right)}\right)+\lambda P(\beta), \tag{11.17}$$

关于 $\beta=(\beta_1,\ldots,\beta_p)^T$。我们可以采用 $P(\beta)=\sum_{j=1}^{p}\beta_j^2$，它对应于岭惩罚，或者 $P(\beta)=\sum_{j=1}^{p}|\beta_j|$，它对应于套索惩罚。

在（11.17）中，λ是一个非负调优参数；通常我们会在λ的一系列值上对其进行最小化。当$\lambda=$为0时，那么最小化（11.17）就等价于简单地最大化通常的Cox偏似然（11.16）。然而，当$\lambda>$为0时，那么最小化（11.17）会得到系数估计的收缩版本。当λ很大时，那么使用岭惩罚将得到不完全等于零的小系数。相比之下，对于一个

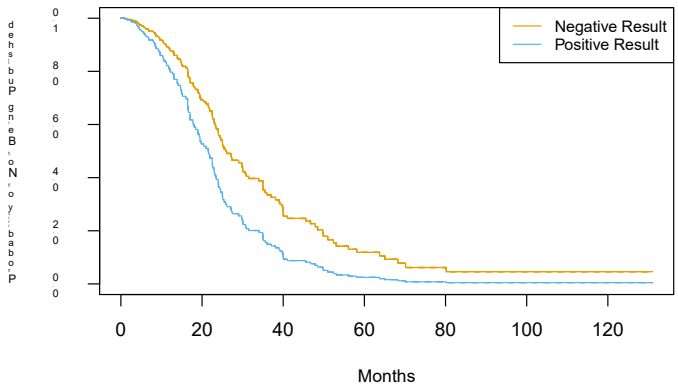


图11.6。*For the* `Publication` *data, we display survival curves for time until publication, stratified by whether or not the study produced a positive result, after adjusting for all other covariates.*

足够大的λ值，使用套索惩罚会使一些系数恰好等于零。

我们现在将套索惩罚Cox模型应用于第11.5.4节中描述的`Publication`数据。我们首先将244个试验随机分成大小相等的训练集和测试集。训练集的交叉验证结果如图11.7所示。y轴上显示的“偏似然偏差”是交叉验证的负对数偏似然的两倍；它扮演交叉验证误差的角色。[15]注意偏似然偏差的“U形”：正如我们在前几章中所看到的，交叉验证误差在中等模型复杂度水平下最小化。具体来说，这发生在只有两个预测变量`budget`和`impact`具有非零估计系数时。

现在，我们如何将这个模型应用于测试集？这引出了一个重要的概念性问题：本质上，没有简单的方法来比较测试集上的预测生存时间和真实生存时间。第一个问题是某些观测数据是删失的，因此这些观测的真实生存时间未被观测到。第二个问题源于这样一个事实：在Cox模型中，我们不是给定协变量向量x来预测单个生存时间，而是估计整个生存曲线$S(t|x)$，作为t的函数。

因此，为了评估模型拟合优度，我们必须采用另一种方法，即利用系数估计对观测值进行分层。具体而言，对于每个测试观测值，我们计算“风险”得分

$$\texttt{budget}_i \cdot \hat{\beta}_{\texttt{budget}} + \texttt{impact}_i \cdot \hat{\beta}_{\texttt{impact}},$$

其中$\hat{\beta}_{\texttt{budget}}$和$\hat{\beta}_{\texttt{impact}}$是训练集中这两个特征的系数估计。然后，我们使用这些风险评分根据观察对象的“风险”对其进行分类。例如，高风险组由$\texttt{budget}_i \cdot \hat{\beta}_{\texttt{budget}} + \texttt{impact}_i \cdot \hat{\beta}_{\texttt{impact}}$最大的观察对象组成；通过

[15]Cross-validation for the Cox model is more involved than for linear or logistic regression, because the objective function is not a sum over the observations.

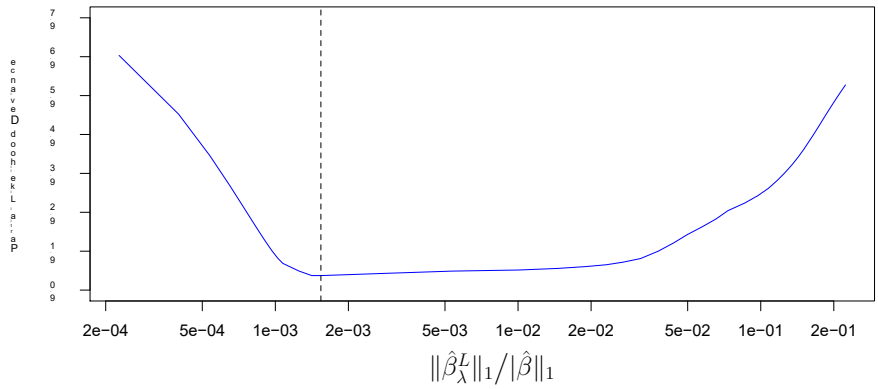


图11.7。 *For the* `Publication` *data described in Section 11.5.4, cross-validation results for the lasso-penalized Cox model are shown. The y-axis displays the partial likelihood deviance, which plays the role of the cross-validation error. The x-axis displays the* ℓ_1 *norm (that is, the sum of the absolute values) of the coefficients of the lasso-penalized Cox model with tuning parameter* λ*, divided by the* ℓ_1 *norm of the coefficients of the unpenalized Cox model. The dashed line indicates the minimum cross-validation error.*

(11.14)，我们看到这些是任意时刻发表瞬时概率最大的观测。换句话说，高风险组由可能更快发表的试验组成。在`Publication`数据上，我们将观测分层为低、中、高风险三分位数。三个分层各自的生存曲线如图11.8所示。我们看到三个分层之间存在明显分离，并且这些分层按照发表的低、中、高风险正确排序。

11.7 附加主题

11.7.1 Area Under the Curve for Survival Analysis

在第4章中，我们介绍了ROC曲线下的面积——通常被称为“AUC”——作为量化二分类器性能的一种方法。将第i个观测的*score*定义为分类器对Pr($Y = 1|X = x_i$)的估计。事实证明，如果我们考虑由类别1中的一个观测和类别2中的一个观测组成的所有配对，那么AUC就是类别1中观测的分数超过类别2中观测的分数的配对所占的比例。

这提示了一种将AUC概念推广到生存分析的方法。我们利用Cox模型系数，为$i = 1, \ldots, n$计算一个估计的风险评分$\hat{\eta}_i = \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{ip}$。如果$\hat{\eta}_{i'} > \hat{\eta}_i$，则模型预测第$i'$个观测的风险大于第$i$个观测，因此生存时间$t_i$将比$t_{i'}$更*greater*。于是，人们很自然地会尝试通过计算满足$t_i > t_{i'}$和$\hat{\eta}_{i'} > \hat{\eta}_i$的观测比例来推广AUC。然而，事情并非如此简单，因为回想一下，我们并未观测到$t_1, \ldots, t_n$；相反，我们观测到

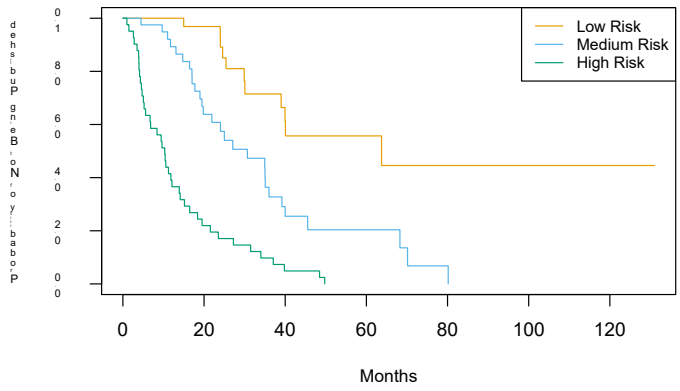


图 11.8. *For the* `Publication` *data introduced in Section 11.5.4, we compute tertiles of "risk" in the test set using coefficients estimated on the training set. There is clear separation between the resulting survival curves.*

（可能删失的）时间 $y_1, \ldots, y_n$，以及删失指示符 $\delta_1, \ldots, \delta_n$。

因此，*Harrell's concordance index* (或*C-index*)计算了该 $\hat{\eta}_{i'} > \hat{\eta}_i$ 和 $y_i > y_{i'}$ 对应的观测对比例：

Harrell's concordance index

$$C = \frac{\sum_{i,i':y_i>y_{i'}} I(\hat{\eta}_{i'} > \hat{\eta}_i)\delta_{i'}}{\sum_{i,i':y_i>y_{i'}} \delta_{i'}},$$

其中指标变量 $I(\hat{\eta}_{i'} > \hat{\eta}_i)$ 在 $\hat{\eta}_{i'} > \hat{\eta}_i$ 时等于1，否则等于零。分子和分母都乘以状态指标 $\delta_{i'}$，因为如果第 i' 个观测值未删失（即如果 $\delta_{i'} = 1$），那么 $y_i > y_{i'}$ 意味着 $t_i > t_{i'}$。相反，如果 $\delta_{i'} = 0$，那么 $y_i > y_{i'}$ 并不意味着 $t_i > t_{i'}$。

我们在`Publication`数据的训练集上拟合了Cox比例风险模型，并在测试集上计算了C指数。这得到了$C = 0.733$。粗略地说，给定测试集中的两篇随机论文，该模型能以73.3%的准确率预测哪一篇将首先发表。

11.7.2 Choice of Time Scale

在本章迄今所考虑的示例中，定义*time*的方式相当明确。例如，在`Publication`示例中，每篇论文的*time zero*被定义为研究结束时的日历时间，而失败时间则被定义为从研究结束到论文发表所经过的月数。

然而，在其他情境中，时间零点和失效时间的定义可能更为微妙。例如，在流行病学研究中考察风险因素与疾病发生之间的关联时，可以使用患者的年龄来定义时间，这样时间零点就是患者的出生日期。采用这种选择，年龄与生存之间的关联无法被测量；然而，在分析中无需对年龄进行调整。当考察与无病生存相关的协变量时（即

治疗与疾病复发之间经过的时间量），可以将治疗日期作为时间零点。

11.7.3 Time-Dependent Covariates

比例风险模型的一个强大特性是它能够处理*time-dependent covariates*，即那些值可能随时间变化的预测变量。例如，假设在一项医学研究过程中，我们每周测量患者的血压。在这种情况下，我们可以将第i个观测的血压不视为x_i，而是视为时间t时的$x_i(t)$。

由于(11.16)中的部分似然是按时间顺序构建的，处理时间依赖协变量就变得直接了当。具体而言，我们只需将(11.16)中的x_{ij}和$x_{i'j}$分别替换为$x_{ij}(y_i)$和$x_{i'j}(y_i)$；这些是时间y_i时预测变量的当前值。相比之下，在传统的参数方法框架内，如(11.13)所示，时间依赖协变量会带来更大的挑战。

一个时间依赖协变量的例子出现在斯坦福心脏移植计划数据的分析中。需要心脏移植的患者被列入等待名单。有些患者接受了移植，但其他患者则在仍处于等待名单时去世。分析的主要目的是确定移植是否与患者更长的生存期相关。

一种朴素的方法会使用固定协变量来表示移植状态：即，若第 i 位患者曾接受过移植，则 $x_i =$ 为 1，否则为 $x_i = 0$。但这种方法忽略了患者必须存活足够长的时间才能接受移植，因此平均而言，接受移植的患者更健康。这个问题可以通过使用移植的时间依赖协变量来解决：若患者在时间 t 之前接受了移植，则 $x_i(t) =$ 为 1，否则为 $x_i(t) = 0$。

11.7.4 Checking the Proportional Hazards Assumption

我们已经看到，Cox的比例风险模型依赖于比例风险假设(11.14)。虽然Cox模型的结果对此假设的违反往往相当稳健，但检查其是否成立仍然是个好主意。对于定性特征，我们可以绘制该特征每个水平的对数风险函数。如果(11.14)成立，那么对数风险函数应仅相差一个常数，如图11.4左上角面板所示。对于定量特征，我们可以通过分层该特征采取类似的方法。

11.7.5 Survival Trees

在第8章中，我们讨论了灵活且自适应的学习程序，如树、随机森林和提升方法，这些方法我们既应用于回归设置也应用于分类设置。这些方法中的大多数都可以推广到生存分析设置中。例如，*survival trees*是一种修改用于分类和回归的树，其使用的分裂准则会最大化

survival trees

在产生的子节点中生存曲线之间的差异。生存树随后可用于创建随机生存森林。

11.8 实验：生存分析

在本实验中，我们对三个独立的数据集进行生存分析。在11.8.1节中，我们分析了首次在11.3节描述的BrainCancer数据。在11.8.2节中，我们检查了来自11.5.4节的Publication数据。最后，11.8.3节探讨了一个模拟呼叫中心数据集。

我们首先在顶层导入一些库。这使代码更具可读性，因为浏览笔记本的前几行就能告诉我们这个笔记本中使用了哪些库。

In [1]:
```
from matplotlib.pyplot import subplots
import numpy as np
import pandas as pd
from ISLP.models import ModelSpec as MS
from ISLP import load_data
```

我们还收集了本实验所需的新导入内容。

In [2]:
```
from lifelines import \
     (KaplanMeierFitter,
      CoxPHFitter)
from lifelines.statistics import \
     (logrank_test,
      multivariate_logrank_test)
from ISLP.survival import sim_time
```

11.8.1 Brain Cancer Data

我们首先从包含在 ISLP 包中的 BrainCancer 数据集开始。

In [3]:
```
BrainCancer = load_data('BrainCancer')
BrainCancer.columns
```

Out[3]:
```
Index(['sex', 'diagnosis', 'loc', 'ki', 'gtv', 'stereo',
       'status', 'time'],
       dtype='object')
```

行索引88名患者，而8列包含预测变量和结果变量。我们首先简要查看数据。

In [4]:
```
BrainCancer['sex'].value_counts()
```

Out[4]:
```
Female    45
Male      43
Name: sex, dtype: int64
```

In [5]:
```
BrainCancer['diagnosis'].value_counts()
```

```
Out[5]: Meningioma    42
        HG glioma     22
        Other         14
        LG glioma      9
        Name: diagnosis, dtype: int64
```

```
In [6]: BrainCancer['status'].value_counts()
```

```
Out[6]: 0    53
        1    35
        Name: status, dtype: int64
```

在开始分析之前，了解status变量的编码方式非常重要。大多数软件采用的约定是，status为1表示未删失的观测（通常是死亡），status为0表示删失观测。但有些科学家可能采用相反的编码。对于BrainCancer数据集，35名患者在研究结束前死亡，因此我们使用的是传统编码。

为了开始分析，我们重新创建了图11.2所示的Kaplan-Meier生存曲线。我们将用于生存分析的主要包是lifelines。变量time对应于y_i，即到第i个事件（删失或死亡）。km.fit 的第一个参数是事件时间，第二个参数是删失变量，其中 1 表示观测到的失败时间。plot() 方法生成一条生存曲线

lifelines

.plot()

with pointwise confidence intervals. By default, these are 90% confidence intervals, but this can be changed by setting the alpha argument to one minus the desired confidence level.

```
In [7]: fig, ax = subplots(figsize=(8,8))
        km = KaplanMeierFitter()
        km_brain = km.fit(BrainCancer['time'], BrainCancer['status'])
        km_brain.plot(label='Kaplan Meier estimate', ax=ax)
```

接下来，我们根据sex分层创建Kaplan-Meier生存曲线，以重现图11.3。我们使用groupby()方法进行此操作。

.groupby()

一个数据框。此方法返回一个生成器，可以在 for 循环中迭代。在这种情况下，for 循环中的项是表示组的二元组：第一项是分组列 sex 的值，而第二个值是由数据框中匹配 sex 该值的所有行组成的数据框。我们希望在下面的对数秩检验中使用这些数据，因此我们将此信息存储在字典 by_sex 中。最后，我们还使用了 *string interpolation* 的概念来自动标记

字符串插值

图表中的不同线条。字符串插值是格式化字符串的一种强大技术——Python有许多方法可以促进此类操作。

```
In [8]: fig, ax = subplots(figsize=(8,8))
        by_sex = {}
        for sex, df in BrainCancer.groupby('sex'):
            by_sex[sex] = df
            km_sex = km.fit(df['time'], df['status'])
            km_sex.plot(label='Sex=%s' % sex, ax=ax)
```

如第11.4节所述，我们可以进行对数秩检验来比较男性和女性的生存情况。我们使用来自

logrank_test()

lifelines.statistics 模块。前两个参数是事件时间，第二个参数表示相应的（可选）删失指示符。

In [9]:
```
logrank_test(by_sex['Male']['time'],
             by_sex['Female']['time'],
             by_sex['Male']['status'],
             by_sex['Female']['status'])
```

Out[9]:
```
                t_0            -1
 null_distribution   chi squared
degrees_of_freedom             1
         test_name  logrank_test


 test_statistic    p  -log2(p)
           1.44 0.23      2.12
```

得到的 p 值为0.23，表明没有证据显示两种性别在生存率上存在差异。

接下来，我们使用来自 lifelines 的 CoxPHFitter() 估计器来拟合 Cox 比例风险模型。首先，我们考虑一个仅使用 sex 作为预测变量的模型。 CoxP HFitter()

In [10]:
```
coxph = CoxPHFitter # shorthand
sex_df = BrainCancer[['time', 'status', 'sex']]
model_df = MS(['time', 'status', 'sex'],
              intercept=False).fit_transform(sex_df)
cox_fit = coxph().fit(model_df,
                      'time',
                      'status')
cox_fit.summary[['coef', 'se(coef)', 'p']]
```

Out[10]:
```
                coef  se(coef)         p
covariate
sex[Male]   0.407667  0.342004  0.233263
```

fit 的第一个参数应该是一个数据框，至少包含事件时间（此处为第二个参数 time），以及一个可选的删失变量（此处为参数 status）。另请注意，Cox 模型不包含截距，这就是我们在上面使用 ModelSpec 的 intercept=False 参数的原因。summary() 方法提供了许多列；我们在此选择缩写其输出。可以按如下方式获得将此模型与无特征模型进行比较的似然比检验：

In [11]:
```
cox_fit.log_likelihood_ratio_test()
```

Out[11]:
```
null_distribution                 chi squared
  degrees_freedom                           1
        test_name   log-likelihood ratio test

 test_statistic    p  -log2(p)
           1.44 0.23      2.12
```

无论我们使用哪种检验，我们都没有发现明确证据表明男性和女性的生存率存在差异。正如本章所学，Cox模型中的评分检验与对数秩检验统计量完全相等！

现在我们拟合一个利用额外预测变量的模型。我们首先注意到，其中一个diagnosis值缺失，因此在继续之前我们删除该观测。

In [12]:
```
cleaned = BrainCancer.dropna()
all_MS = MS(cleaned.columns, intercept=False)
all_df = all_MS.fit_transform(cleaned)
fit_all = coxph().fit(all_df,
                      'time',
                      'status')
fit_all.summary[['coef', 'se(coef)', 'p']]
```

Out[12]:

covariate	coef	se(coef)	p
sex[Male]	0.183748	0.360358	0.610119
diagnosis[LG glioma]	-1.239541	0.579557	0.032454
diagnosis[Meningioma]	-2.154566	0.450524	0.000002
diagnosis[Other]	-1.268870	0.617672	0.039949
loc[Supratentorial]	0.441195	0.703669	0.530664
ki	-0.054955	0.018314	0.002693
gtv	0.034293	0.022333	0.124660
stereo[SRT]	0.177778	0.601578	0.767597

diagnosis变量已编码，使得基线对应于HG胶质瘤。结果表明，与HG胶质瘤相关的风险是脑膜瘤相关风险的八倍以上（即$e^{2.15} = 8.62$）。换句话说，在调整其他预测因素后，与脑膜瘤患者相比，HG胶质瘤患者的生存率要差得多。此外，Karnofsky指数ki值越大，风险越低，即生存期越长。

最后，我们绘制了每个诊断类别的估计生存曲线，同时调整了其他预测因子。为了制作这些图，我们将其他预测因子的值设为定量变量的均值，分类变量的众数。为此，我们使用apply()方法跨行操作（即axis=0），并配合一个函数representative来检查列是否为分类变量。

In [13]:
```
levels = cleaned['diagnosis'].unique()
def representative(series):
    if hasattr(series.dtype, 'categories'):
        return pd.Series.mode(series)
    else:
        return series.mean()
modal_data = cleaned.apply(representative, axis=0)
```

我们制作四份列均值，并将diagnosis列分配为四种不同的诊断。

In [14]:
```
modal_df = pd.DataFrame(
             [modal_data.iloc[0] for _ in range(len(levels))])
modal_df['diagnosis'] = levels
modal_df
```

```
Out[14]:    sex   diagnosis             loc       ki    gtv  stereo   ...
          Female  Meningioma  Supratentorial  80.920  8.687     SRT   ...
          Female   HG glioma  Supratentorial  80.920  8.687     SRT   ...
          Female   LG glioma  Supratentorial  80.920  8.687     SRT   ...
          Female       Other  Supratentorial  80.920  8.687     SRT   ...
```

然后，我们根据用于拟合模型的模型规格 `all_MS` 构建模型矩阵，并按照 `diagnosis` 的水平命名行。

```
In [15]: modal_X = all_MS.transform(modal_df)
         modal_X.index = levels
         modal_X
```

我们可以使用 {v*} 方法来获得估计的交配后的生存函数。

`.predict_survival_function()`

```
In [16]: predicted_survival = fit_all.predict_survival_function(modal_X)
         predicted_survival
```

```
Out[16]:       Meningioma   HG glioma   LG glioma   Other
          0.070      0.998       0.982       0.995   0.995
          1.180      0.998       0.982       0.995   0.995
          1.410      0.996       0.963       0.989   0.990
          1.540      0.996       0.963       0.989   0.990
            ...        ...         ...         ...     ...
         67.380      0.689       0.040       0.394   0.405
         73.740      0.689       0.040       0.394   0.405
         78.750      0.689       0.040       0.394   0.405
         82.560      0.689       0.040       0.394   0.405
         85 rows × 4 columns
```

这返回一个数据框，其绘图方法展示了不同的生存曲线。为了避免图表过于拥挤，我们不显示置信区间。

```
In [17]: fig, ax = subplots(figsize=(8, 8))
         predicted_survival.plot(ax=ax);
```

11.8.2 Publication Data

第11.5.4节中展示的`Publication`数据可以在`ISLP`包中找到。我们首先通过绘制按`posres`变量分层的Kaplan-Meier曲线来重现图11.5，该变量记录了研究结果是阳性还是阴性。

```
In [18]: fig, ax = subplots(figsize=(8,8))
         Publication = load_data('Publication')
         by_result = {}
         for result, df in Publication.groupby('posres'):
             by_result[result] = df
             km_result = km.fit(df['time'], df['status'])
             km_result.plot(label='Result=%d' % result, ax=ax)
```

如前所述，将Cox比例风险模型拟合到`posres`变量得到的p值相当大，没有提供证据表明阳性结果与阴性结果的研究在发表时间上存在差异。

In [19]:
```
posres_df = MS(['posres',
                'time',
                'status'],
                intercept=False).fit_transform(Publication)
posres_fit = coxph().fit(posres_df,
                         'time',
                         'status')
posres_fit.summary[['coef', 'se(coef)', 'p']]
```

Out[19]:

covariate	coef	se(coef)	p
posres	0.148076	0.161625	0.359578

然而，当我们在模型中包含其他预测变量时，结果会发生显著变化。此处我们排除了资助机制变量。

In [20]:
```
model = MS(Publication.columns.drop('mech'),
           intercept=False)
coxph().fit(model.fit_transform(Publication),
            'time',
            'status').summary[['coef', 'se(coef)', 'p']]
```

Out[20]:

covariate	coef	se(coef)	p
posres	0.570774	0.175960	1.179606e-03
multi	-0.040863	0.251194	8.707727e-01
clinend	0.546180	0.262001	3.710099e-02
sampsize	0.000005	0.000015	7.506978e-01
budget	0.004386	0.002464	7.511276e-02
impact	0.058318	0.006676	2.426779e-18

我们看到有许多统计上显著的变量，包括试验是否聚焦于临床终点、研究的影响力，以及研究结果是正面还是负面。

11.8.3 Call Center Data

在本节中，我们将利用练习8中探讨的累积风险与生存函数之间的关系来模拟生存数据。我们模拟的数据将表示2,000名致电呼叫中心的顾客所观察到的等待时间（以秒为单位）。在此情境下，如果顾客在电话被接听前挂断，则会发生删失。

有三个协变量：`Operators` (呼叫时可用的呼叫中心操作员数量，范围从5到15)，`Center`(要么是A、B，要么是C)，以及`Time`一天中的时段（早上、下午或晚上）。我们为这些协变量生成数据，使得所有可能性等概率出现：例如，早上、下午和晚上的呼叫等概率，而5到15之间的任何操作员数量也是等概率的。

In [21]:
```
rng = np.random.default_rng(10)
N = 2000
Operators = rng.choice(np.arange(5, 16),
                       N,
                       replace=True)
```

```
Center = rng.choice(['A', 'B', 'C'],
                    N,
                    replace=True)
Time = rng.choice(['Morn.', 'After.', 'Even.'],
                  N,
                  replace=True)
D = pd.DataFrame({'Operators': Operators,
                  'Center': pd.Categorical(Center),
                  'Time': pd.Categorical(Time)})
```

然后构建模型矩阵（省略截距）

In [22]:
```
model = MS(['Operators',
            'Center',
            'Time'],
           intercept=False)
X = model.fit_transform(D)
```

值得一看模型矩阵 X，以确保我们理解变量是如何编码的。默认情况下，分类变量的水平会被排序，并且像往常一样，变量的独热编码的第一列会被丢弃。

In [23]:
```
X[:5]
```

Out[23]:

	Operators	Center[B]	Center[C]	Time[Even.]	Time[Morn.]
0	13	0.0	1.0	0.0	0.0
1	15	0.0	0.0	1.0	0.0
2	7	1.0	0.0	0.0	1.0
3	7	0.0	1.0	0.0	1.0
4	13	0.0	1.0	1.0	0.0

接下来，我们指定系数和危险函数。

In [24]:
```
true_beta = np.array([0.04, -0.3, 0, 0.2, -0.2])
true_linpred = X.dot(true_beta)
hazard = lambda t: 1e-5 * t
```

在这里，我们将与Operators相关的系数设为等于0.04；换句话说，在给定Center和Time协变量的情况下，每增加一个操作员，来电被接听的“风险”就会增加$e^{0.04}$ =1.041倍。这是合理的：手头的操作员越多，等待时间越短！与Center ==B相关的系数是−0.3，而Center == A被视为基线。这意味着在B中心接听电话的风险是A中心的0.74倍；换句话说，B中心的等待时间稍长一些。

回顾第2.3.7节中使用lambda动态创建短函数的方法。我们使用ISLP.survival包中的sim_time()函数——年龄。此函数利用生存函数与累积风险之间的关系 $S(t) = \exp(-H(t))$ 以及Cox模型中累积风险函数的具体形式，根据线性预测变量 true_linpred 的值和累积风险来模拟数据。我们需要提供累积风险函数，此处即如此做。

sim_time()

In [25]:
```
cum_hazard = lambda t: 1e-5 * t**2 / 2
```

我们现在准备在Cox比例风险模型下生成数据。我们将最大时间截断为1000秒，以保持模拟等待时间合理。函数`sim_time()`接受线性预测器、累积风险函数和随机数生成器。

In [26]:
```
W = np.array([sim_time(l, cum_hazard, rng)
              for l in true_linpred])
D['Wait time'] = np.clip(W, 0, 1000)
```

我们现在模拟删失变量，假设90%的电话在客户挂断（`Failed==0`）之前得到了接听（`Failed==1`）。

In [27]:
```
D['Failed'] = rng.choice([1, 0],
                         N,
                         p=[0.9, 0.1])
D[:5]
```

Out[27]:

	Operators	Center	Time	Wait time	Failed
0	13	C	After.	525.064979	1
1	15	A	Even.	254.677835	1
2	7	B	Morn.	487.739224	1
3	7	C	Morn.	308.580292	1
4	13	C	Even.	154.174608	1

In [28]:
```
D['Failed'].mean()
```

Out[28]: 0.8985

We now plot Kaplan-Meier survival curves. First, we stratify by `Center`.

In [29]:
```
fig, ax = subplots(figsize=(8,8))
by_center = {}
for center, df in D.groupby('Center'):
    by_center[center] = df
    km_center = km.fit(df['Wait time'], df['Failed'])
    km_center.plot(label='Center=%s' % center, ax=ax)
ax.set_title("Probability of Still Being on Hold")
```

Next, we stratify by `Time`.

In [30]:
```
fig, ax = subplots(figsize=(8,8))
by_time = {}
for time, df in D.groupby('Time'):
    by_time[time] = df
    km_time = km.fit(df['Wait time'], df['Failed'])
    km_time.plot(label='Time=%s' % time, ax=ax)
ax.set_title("Probability of Still Being on Hold")
```

It seems that calls at Call Center B take longer to be answered than calls at Centers A and C. Similarly, it appears that wait times are longest in the morning and shortest in the evening hours. We can use a log-rank test to determine whether these differences are statistically significant using the function `multivariate_logrank_test()`.

In [31]:
```
multivariate_logrank_test(D['Wait time'],
                          D['Center'],
                          D['Failed'])
```

```
Out[31]:                t_0                          -1
          null_distribution                  chi squared
         degrees_of_freedom                            2
                  test_name   multivariate_logrank_test

          test_statistic      p  -log2(p)
                   20.30 <0.005     14.65
```

Next, we consider the effect of `Time`.

```
In [32]: multivariate_logrank_test(D['Wait time'],
                                   D['Time'],
                                   D['Failed'])
```

```
Out[32]:                t_0                          -1
          null_distribution                  chi squared
         degrees_of_freedom                            2
                  test_name   multivariate_logrank_test

          test_statistic      p  -log2(p)
                   49.90 <0.005     35.99
```

As in the case of a categorical variable with 2 levels, these results are similar to the likelihood ratio test from the Cox proportional hazards model. First, we look at the results for `Center`.

```
In [33]: X = MS(['Wait time',
                 'Failed',
                 'Center'],
                intercept=False).fit_transform(D)
         F = coxph().fit(X, 'Wait time', 'Failed')
         F.log_likelihood_ratio_test()
```

```
Out[33]: null_distribution                 chi squared
           degrees_freedom                           2
                 test_name   log-likelihood ratio test

          test_statistic      p  -log2(p)
                   20.58 <0.005     14.85
```

Next, we look at the results for `Time`.

```
In [34]: X = MS(['Wait time',
                 'Failed',
                 'Time'],
                intercept=False).fit_transform(D)
         F = coxph().fit(X, 'Wait time', 'Failed')
         F.log_likelihood_ratio_test()
```

```
Out[34]: null_distribution                 chi squared
           degrees_freedom                           2
                 test_name   log-likelihood ratio test

          test_statistic      p  -log2(p)
                   48.12 <0.005     34.71
```

We find that differences between centers are highly significant, as are differences between times of day.

Finally, we fit Cox's proportional hazards model to the data.

In [35]:
```
X = MS(D.columns,
       intercept=False).fit_transform(D)
fit_queuing = coxph().fit(
                  X,
                 'Wait time',
                 'Failed')
fit_queuing.summary[['coef', 'se(coef)', 'p']]
```

Out[35]:

covariate	coef	se(coef)	p
Operators	0.043934	0.007520	5.143677e-09
Center[B]	-0.236059	0.058113	4.864734e-05
Center[C]	0.012231	0.057518	8.316083e-01
Time[Even.]	0.268845	0.057797	3.294914e-06
Time[Morn.]	-0.148215	0.057334	9.734378e-03

中心B和傍晚时间的p值非常小。同样明显的是，风险——即电话被接听的瞬时风险——随着操作员数量的增加而增加。由于数据是我们自己生成的，我们知道`Operators`、`Center = B`、`Center = C`、`Time = Even.`和`Time = Morn.`的真实系数分别为0.04、-0.3、0、0.2和-0.2。根据拟合的Cox模型得到的系数估计相当准确。

11.9 练习

Conceptual

1. 对于每个例子，说明删失机制是否独立，并证明你的答案。

   (a) 在一项疾病复发研究中，由于一位粗心的研究科学家，所有电话号码以数字“2”开头的患者都失访了。(b) 在一项长寿研究中，一个格式错误导致所有超过99岁的患者年龄丢失（即我们知道那些患者超过99岁，但不知道他们的确切年龄）。(c) 医院A进行了一项长寿研究。然而，病情非常严重的患者往往被转移到医院B，并且失访。(d) 在一项失业持续时间的研究中，较早找到工作的人不太有动力与研究调查者保持联系，因此更有可能失访。(e) 在一项妊娠持续时间的研究中，早产分娩的妇女更有可能在通常医院之外分娩，因此相对于足月分娩的妇女，更有可能被删失。

(f) 一位研究者希望模拟小镇居民受教育的年数。与在镇上上大学的人相比，那些去外地大学就读的居民更可能失访，也更可能就读研究生院。

(g) 研究人员进行了一项无病生存期（即治疗后至疾病复发的时间）的研究。五年内未复发的患者被认为已治愈，因此其生存时间在五年处被截尾。

(h) 我们希望为某些电气组件的故障时间建立模型。这些组件可以在爱荷华州或匹兹堡制造，质量上没有差别。爱荷华州的工厂五年前开业，因此在爱荷华州制造的组件在五年时被审查。匹兹堡的工厂两年前开业，因此那些组件在两年时被审查。

(i) 我们希望对在两个不同工厂生产的电气组件的故障时间进行建模，其中一个工厂比另一个更早开业。我们有理由相信，较早开业的工厂生产的组件质量更高。

2. 我们进行了一项研究，涉及n =4名刚购买手机的参与者，旨在建立手机更换时间的模型。第一位参与者在1.2年后更换了手机。第二位参与者在为期两年的研究结束时仍未更换手机。第三位参与者在研究进行到1.5年时更改了电话号码并失访（但尚未更换手机）。第四位参与者在0.2年后更换了手机。

对于四位参与者中的每一位（$i = 1, \ldots, 4$），使用第11.1节中引入的符号回答以下问题：

（a）参与者更换手机的时间是否被删失？（b）c_i 的值是否已知？如果已知，那么它是多少？（c）t_i 的值是否已知？如果已知，那么它是多少？（d）y_i 的值是否已知？如果已知，那么它是多少？（e）δ_i 的值是否已知？如果已知，那么它是多少？

3. 对于练习2中的例子，报告K、$d_1, \ldots, d_K$、$r_1, \ldots, r_K$和$q_1, \ldots, q_K$的值，该记号在第11.3节中定义。

4. 本题利用图11.9所示的Kaplan-Meier生存曲线。绘制这条生存曲线所用的原始数据见表11.4。该表的协变量列在本题中不需要。

(a) 超过50天的估计生存概率是多少？

Observation (Y)	Censoring Indicator (δ)	Covariate (X)
26.5	1	0.1
37.2	1	11
57.3	1	-0.3
90.8	0	2.8
20.2	0	1.8
89.8	0	0.4

表11.4。*Data used in Exercise 4.*

(b) 写出估计生存函数的解析表达式。例如，你的答案可能类似于

$$\widehat{S}(t) = \begin{cases} 0.8 & \text{if } t < 31 \\ 0.5 & \text{if } 31 \leq t < 77 \\ 0.22 & \text{if } 77 \leq t. \end{cases}$$

(前面的等式仅为示意：它并非正确答案！)

5. 给出方程所表示的生存函数的草图

$$\widehat{S}(t) = \begin{cases} 0.8 & \text{if } t < 31 \\ 0.5 & \text{if } 31 \leq t < 77 \\ 0.22 & \text{if } 77 \leq t. \end{cases}$$

您的答案应该看起来像图11.9。

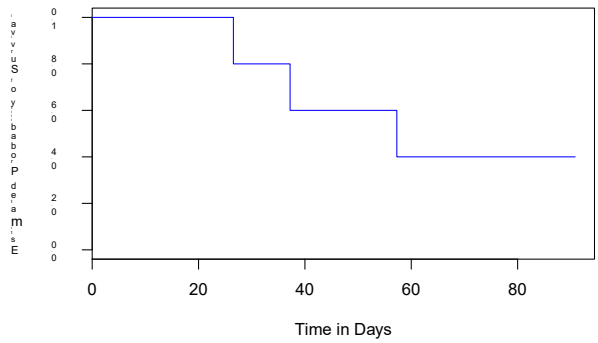


图11.9。*A Kaplan-Meier survival curve used in Exercise 4.*

6. 本题利用了图11.1中展示的数据。在解答此题时，你可以将这些观察时间称为$y_1, \ldots, y_4$。这些观察时间的顺序可以从图11.1中看出；它们的具体数值并不重要。

(a) 报告$\delta_1, \ldots, \delta_4$、$K$、$d_1, \ldots, d_K$、$r_1, \ldots, r_K$和$q_1, \ldots, q_K$的值。相关符号定义见第11.1节和第11.3节。

(b) 绘制与给定数据集对应的Kaplan-Meier生存曲线。（无需使用任何软件——您可以根据(a)中得到的结果手工绘制。）(c) 根据(b)中估计的生存曲线，事件在200天内发生的概率是多少？事件在310天内未发生的概率是多少？(d) 写出(b)中估计生存曲线的表达式。

7. 在本问题中，我们将推导出(11.5)和(11.6)，这两个公式是构建对数秩检验统计量(11.8)所必需的。回忆表11.1中的符号表示。



(a) 假设两组的生存函数之间没有差异。那么，如果从包含总共 q_k 个失效的 r_k 个观察值的风险集中，无放回地抽取 r_{1k} 个观察值，我们可以将 q_{1k} 视为失效数。论证 q_{1k} 服从 *hypergeometric distribution*。写出

该分布的参数用r_{1k}、r_k和q_k表示。

hyper-geometric distribution

(b) 基于您之前的回答以及超几何分布的性质，q_{1k}的均值和方差分别是多少？将您的答案与(11.5)和(11.6)进行比较。

8. 回顾生存函数 $S(t)$、危险函数 $h(t)$ 和密度函数 $f(t)$ 分别在 (11.2)、(11.9) 和 (11.11) 中定义。此外，定义 $F(t) = 1 - S(t)$。证明以下关系成立：



$$\begin{aligned} f(t) &= dF(t)/dt \\ S(t) &= \exp\left(-\int_0^t h(u)du\right). \end{aligned}$$

9. 在本练习中，我们将探讨假设生存时间服从指数分布所带来的后果。

(a) 假设生存时间服从参数为λ的指数分布，因此其密度函数为 $f(t) = \lambda\exp(-\lambda t)$。利用练习8中提供的关系，证明$S(t) = \exp(-\lambda t)$。

(b) 现在假设n个独立的生存时间各自服从Exp(λ)分布。写出似然函数(11.13)的表达式。

(c) 证明λ的最大似然估计量为

$$\hat{\lambda} = \sum_{i=1}^{n} \delta_i / \sum_{i=1}^{n} y_i.$$

(d) 使用你对(c)的答案来推导平均生存时间的估计量。

Hint: For (d), recall that the mean of an Exp(λ) *random variable is* $1/\lambda$.

Applied

10. 本练习聚焦于脑肿瘤数据，该数据包含在ISLP库中。

(a) 使用 `lifelines` 包中的 `KaplanMeierFitter()` 估计器，绘制带 ± 1 标准误差带的 Kaplan-Meier 生存曲线。(b) 从配对 (y_i, δ_i) 中抽取大小为 $n = 88$ 的 bootstrap 样本，并计算得到的 Kaplan-Meier 生存曲线。将该过程重复 $B = 200$ 次。利用结果获得每个时间点 Kaplan-Meier 生存曲线标准误的估计。将其与 (a) 中获得的标准误进行比较。(c) 拟合一个使用所有预测变量来预测生存的 Cox 比例风险模型。总结主要发现。(d) 按 `ki` 的值对数据进行分层。(由于只有一个观测具有 `ki==40`，你可以将该观测与具有 `ki==60` 的观测归为一组。) 绘制五个分层各自的 Kaplan-Meier 生存曲线，并针对其他预测变量进行调整。

11. 本练习使用表11.4中的数据。

(a) 创建两组观测值。在组1中，$X < 2$，而在组2中，$X \geq 2$。绘制对应于两组的Kaplan-Meier生存曲线。确保标记曲线，以便清楚哪条曲线对应哪个组。凭肉眼观察，两组生存曲线之间是否存在差异？

(b) 拟合Cox比例风险模型，使用组别指示符作为协变量。估计的系数是多少？写一句话，从风险或事件瞬时概率的角度解释这一系数。是否有证据表明真实系数值非零?

(c) 回忆第11.5.2节，在单个二分类协变量的情况下，对数秩检验统计量应与Cox模型中的得分统计量完全相同。进行对数秩检验，以判断两组的生存曲线是否存在差异。对数秩检验统计量的p值与来自(b)的Cox模型得分统计量的p值相比如何?

12
无监督学习



本书的大部分内容涉及诸如回归和分类的*supervised learning*方法。在有监督学习设置中，我们通常可以访问在n次观测上测量的一组p特征$X_1, X_2, \ldots, X_p$，以及同样在这些n次观测上测量的响应Y。目标则是利用$X_1, X_2, \ldots, X_p$来预测Y。

相反，本章将聚焦于*unsupervised learning*，这是一组统计工具，旨在应对我们仅拥有一组特征$X_1, X_2, \ldots, X_p$，这些特征在n个观测值上测量得出，而并不涉及预测，因为我们没有相应的响应变量Y。相反，目标是发现关于$X_1, X_2, \ldots, X_p$测量值的有趣信息。是否有直观的方法来可视化数据？我们能否在变量之间或观测值之间发现子群？无监督学习是指一系列多样化的技术，用于回答诸如此类的问题。本章将着重介绍两种特定类型的无监督学习：*principal components analysis*，一种用于数据可视化或在应用监督技术前进行数据预处理的工具，以及*clustering*，一类广泛的方法，用于发现数据中的未知子群。

12.1 无监督学习的挑战

监督学习是一个被充分理解的领域。事实上，如果你已经阅读了本书前面的章节，那么你现在应该已经很好地掌握了监督学习。例如，如果要求你从数据集中预测一个二元结果，你有一整套非常完善的工具可供使用（如逻辑回归、线性判别分析、分类树、支持向量机等），以及一个清晰的

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_12

理解如何评估获得的结果的质量（使用交叉验证、在独立测试集上验证等）。

相比之下，无监督学习通常更具挑战性。这项任务往往更加主观，且分析没有简单的目标，比如预测响应。无监督学习通常作为*exploratory data analysis*的一部分来进行。此外，它可能

exploratory data analysis

很难评估无监督学习方法得到的结果，因为不存在普遍接受的机制来进行交叉验证或在独立数据集上验证结果。这种差异的原因很简单。如果我们使用监督学习技术拟合预测模型，那么可以通过观察我们的模型对未用于拟合模型的观测数据预测响应Y的效果来进行*check our work*。然而，在无监督学习中，无法检查我们的工作，因为我们不知道真正的答案——这个问题是无监督的。

无监督学习技术在多个领域中日益重要。癌症研究人员可能会检测100名乳腺癌患者的基因表达水平。然后，他或她可能会在乳腺癌样本或基因中寻找亚组，以便更好地理解这种疾病。在线购物网站可能会尝试识别具有相似浏览和购买历史的购物者群体，以及每个群体内购物者特别感兴趣的物品。然后，可以根据相似购物者的购买历史，优先向个别购物者展示他或她特别可能感兴趣的物品。搜索引擎可能会根据具有相似搜索模式的其他个体的点击历史，选择向特定个人显示哪些搜索结果。这些统计学习任务以及更多任务，都可以通过无监督学习技术来完成。

12.2 主成分分析

Principal components 在第6.3.1节中，我们在主成分回归的背景下进行了讨论。当面临大量相关变量时，主成分允许我们用较少数量的代表性变量来概括这一集合，这些变量共同解释了原始集合中的大部分变异性。主成分方向在第6.3.1节中被呈现为特征空间中的方向，沿着这些方向，原始数据*highly variable*。这些方向还定义了与数据云*as close as possible*的线和子空间。为了进行主成分回归，我们只需在回归模型中使用主成分作为预测变量，以替代原始更大的变量集。

Principal components analysis (PCA) 指的是主成分计算主成分，以及随后使用这些成分来理解数据。PCA 是一种无监督方法，因为它仅涉及一组特征 $X_1, X_2, \ldots, X_p$，而没有相关联的响应 Y。除了生成用于监督学习问题的派生变量外，PCA 还可作为数据可视化的工具（可视化

principal components analysis

变量的观测或可视化）。它也可以用作数据插补的工具——即为数据矩阵中的缺失值填补。

我们现在更详细地讨论PCA，重点在于将PCA作为一种无监督数据探索工具的应用，这与本章的主题保持一致。

12.2.1 What Are Principal Components?

假设我们希望将n个观测值在一组p个特征$X_1, X_2, \ldots, X_p$上的测量结果可视化，作为探索性数据分析的一部分。我们可以通过检查数据的二维散点图来实现这一点，每个散点图包含n个观测值在两个特征上的测量结果。然而，存在$\binom{p}{2} = p(p-1)/2$个这样的散点图；例如，当p =为10时，有45个图！如果p很大，那么肯定不可能查看所有图；此外，很可能它们中没有一个会有信息量，因为每个图只包含数据集中总信息的一小部分。显然，当p很大时，需要一种更好的方法来可视化n个观测值。特别是，我们希望找到数据的一种低维表示，尽可能多地捕捉信息。例如，如果我们能够获得数据的二维表示并捕捉大部分信息，那么我们就可以在这个低维空间中绘制观测值。

PCA 正好提供了一种实现这一点的工具。它找到数据集的低维表示，其中包含尽可能多的变异。其思想是每个 n 观测值都生活在 p 维空间中，但并非所有这些维度都同样有趣。PCA 寻求尽可能有趣的少数几个维度，其中 *in- teresting* 的概念由观测值沿每个维度的变异量来衡量。PCA 找到的每个维度都是 p 个特征的线性组合。我们现在解释这些维度，即 *principal components*，是如何被找到的。

一组特征 $X_1, X_2, \ldots, X_p$ 的 *first principal component* 是这些特征的归一化线性组合

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \cdots + \phi_{p1}X_p \tag{12.1}$$

具有最大方差。根据*normalized*，我们指的是$\sum_{j=1}^{p} \phi_{j1}^2 = 1$。我们将元素$\phi_{11}, \ldots, \phi_{p1}$称为第一主成分的*loadings*
成分；载荷共同构成主成分载荷向量，$\phi_1 = (\phi_{11}\ \phi_{21}\ \ldots\ \phi_{p1})^T$。我们约束载荷，使其平方和等于一，因为如果不这样约束，将这些元素设为任意大的绝对值可能会导致任意大的方差。 loading

给定一个 $n \times p$ 数据集 $\mathbf{X}$，我们如何计算第一主成分？由于我们只关心方差，我们假设 $\mathbf{X}$ 中的每个变量都已中心化，均值为零（即 $\mathbf{X}$ 的列均值为零）。然后我们寻找样本特征值的线性组合，形式为

$$z_{i1} = \phi_{11}x_{i1} + \phi_{21}x_{i2} + \cdots + \phi_{p1}x_{ip} \tag{12.2}$$

即在约束$\sum_{j=1}^{p}\phi_{j1}^2$=1的条件下，具有最大样本方差的向量。换句话说，第一个主成分载荷向量解决了优化问题

$$\underset{\phi_{11},\ldots,\phi_{p1}}{\text{maximize}}\left\{\frac{1}{n}\sum_{i=1}^{n}\left(\sum_{j=1}^{p}\phi_{j1}x_{ij}\right)^2\right\}\text{ subject to }\sum_{j=1}^{p}\phi_{j1}^2=1. \tag{12.3}$$

由 (12.2) 我们可以将 (12.3) 中的目标写为$\frac{1}{n}\sum_{i=1}^{n}z_{i1}^2$。因为$\frac{1}{n}\sum_{i=1}^{n}x_{ij}=0$，$z_{11},\ldots,z_{n1}$的平均值也将为零。因此，我们在 (12.3) 中最大化的目标只是 z_{i1} 的 n 值的样本方差。我们将 $z_{11},\ldots,z_{n1}$ 称为第一主成分得分，伙伴组件。问题（12.3）可以通过*eigen decomposition*解决，线性代数中的一项标准技术，但其细节超出了本书的讨论范围。[1]

eigen decomposition

第一主成分有一个很好的几何解释。载荷向量 ϕ_1 及其元素 $\phi_{11},\phi_{21},\ldots,\phi_{p1}$ 定义了特征空间中的一个方向，数据在该方向上的变化最大。如果我们将 n 个数据点 $x_1,\ldots,x_n$ 投影到这个方向上，投影值就是主成分得分 $z_{11},\ldots,z_{n1}$ 本身。例如，第254页的图6.14在一个广告数据集上显示了第一主成分载荷向量（绿色实线）。在这些数据中，只有两个特征，因此观测值以及第一主成分载荷向量都可以很容易地显示出来。从(6.19)可以看出，在该数据集中，$\phi_{11}=0.839$ 且 $\phi_{21}=0.544$。

在确定特征的第一主成分 Z_1 后，我们可以找到第二主成分 Z_2。第二主成分是 $X_1,\ldots,X_p$ 的线性组合，在所有与 Z_1 为 *uncorrelated* 的线性组合中具有最大方差。第二主成分得分 $z_{12},z_{22},\ldots,z_{n2}$ 的形式为

$$z_{i2}=\phi_{12}x_{i1}+\phi_{22}x_{i2}+\cdots+\phi_{p2}x_{ip}, \tag{12.4}$$

其中 ϕ_2 是第二主成分载荷向量，元素为 $\phi_{12},\phi_{22},\ldots,\phi_{p2}$。结果表明，将 Z_2 约束为与 Z_1 不相关，等价于将方向 ϕ_2 约束为与方向 ϕ_1 正交（垂直）。在图6.14的示例中，观测值位于二维空间（因为 $p=2$），所以一旦找到 ϕ_1，ϕ_2 就只有一个可能，如图中蓝色虚线所示。（根据第6.3.1节，我们知道 $\phi_{12}=0.544$ 以及 $\phi_{22}=-0.839$。）但在包含 $p>2$个变量的更大数据集中，存在多个不同的主成分，它们的定义方式类似。为了找到 ϕ_2，我们求解一个类似于(12.3)的问题，用 ϕ_2 替换 ϕ_1，并附加约束条件：ϕ_2 与 ϕ_1 正交。[2]

---

[1] As an alternative to the eigen decomposition, a related technique called the singular value decomposition can be used. This will be explored in the lab at the end of this chapter.

[2] On a technical note, the principal component directions ϕ_1, ϕ_2, ϕ_3, ... are given by the ordered sequence of eigenvectors of the matrix $\mathbf{X}^T\mathbf{X}$, and the variances of the components are the eigenvalues. There are at most $\min(n-1,p)$ principal components.

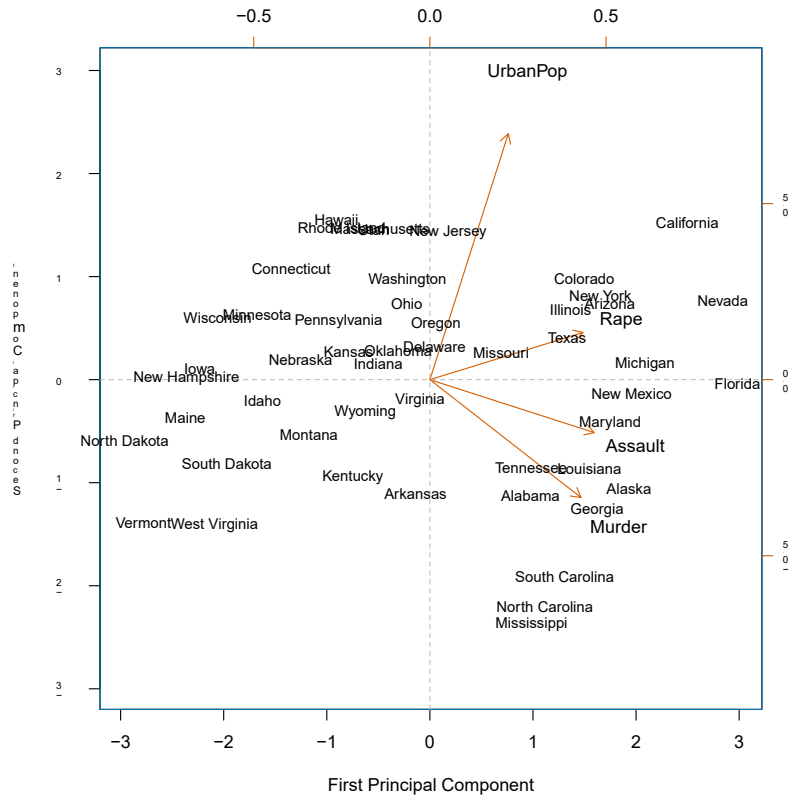


图12.1。*The first two principal components for the* `USArrests` *data. The blue state names represent the scores for the first two principal components. The orange arrows indicate the first two principal component loading vectors (with axes on the top and right). For example, the loading for* `Rape` *on the first component is*0.54*, and its loading on the second principal component*0.17 *(the word* `Rape` *is centered at the point* (0.54,0.17)*). This figure is known as a biplot, because it displays both the principal component scores and the principal component loadings.*

一旦我们计算出了主成分，就可以将它们相互绘制出来，以便生成数据的低维视图。例如，我们可以将得分向量Z_1对Z_2作图，Z_1对Z_3作图，Z_2对Z_3作图，以此类推。从几何角度来看，这相当于将原始数据投影到由ϕ_1、ϕ_2和ϕ_3张成的子空间上，并绘制投影点。

我们在`USArrests`数据集上展示了PCA的应用。对于美国50个州中的每一个，数据集包含了每100,000名居民中三种犯罪的逮捕人数：`Assault`、`Murder`和`Rape`。我们还记录了`UrbanPop` (每个州居住在城市地区的人口百分比)。主成分得分向量的长度为$n = 50$，主成分载荷向量的长度为$p = 4$。在对每个变量进行标准化处理，使其均值为零且标准差为

	PC1	PC2
Murder	0.5358995	−0.4181809
Assault	0.5831836	−0.1879856
UrbanPop	0.2781909	0.8728062
Rape	0.5434321	0.1673186

表12.1。*The principal component loading vectors, ϕ_1 and ϕ_2, for the* USArrests *data. These are also displayed in Figure 12.1.*

偏差一。图12.1展示了这些数据的前两个主成分。该图在一个*biplot*显示中同时呈现了主成分得分和载荷向量。载荷也给出在

biplot

表 12.2.1。

在图12.1中，我们可以看到第一个加载向量对Assault、Murder和Rape赋予了大致相等的权重，而对UrbanPop的权重则小得多。因此，这个成分大致对应于严重犯罪总体率的衡量。第二个加载向量将大部分权重放在UrbanPop上，而对其他三个特征的权重则小得多。因此，这个成分大致对应于州的城镇化水平。总体来看，我们看到与犯罪相关的变量（Murder、Assault和Rape）彼此位置接近，而UrbanPop变量则与其他三个相距较远。这表明与犯罪相关的变量彼此相关——谋杀率高的州往往袭击率和强奸率也高——而UrbanPop变量与其他三个的相关性较低。

我们可以通过图12.1所示的两个主成分得分向量来考察各州之间的差异。我们对载荷向量的讨论表明，在第一个主成分上得分较高的州，如加利福尼亚州、内华达州和佛罗里达州，犯罪率较高，而像北达科他州这样在第一个主成分上得分为负的州，犯罪率较低。加利福尼亚州在第二个主成分上也有较高得分，表明其城市化水平较高，而像密西西比州这样的州则相反。在两个主成分上都接近零的州，如印第安纳州，其犯罪率和城市化水平都大约处于平均水平。

12.2.2 Another Interpretation of Principal Components

在模拟的三维数据集中，前两个主成分载荷向量展示在图12.2的左侧面板；这两个载荷向量张成一个平面，沿着这个平面的观测值具有最高的方差。

在上一节中，我们将主成分载荷向量描述为特征空间中数据变化最大的方向，而主成分得分则是沿这些方向的投影。然而，主成分的另一种解释也可以是

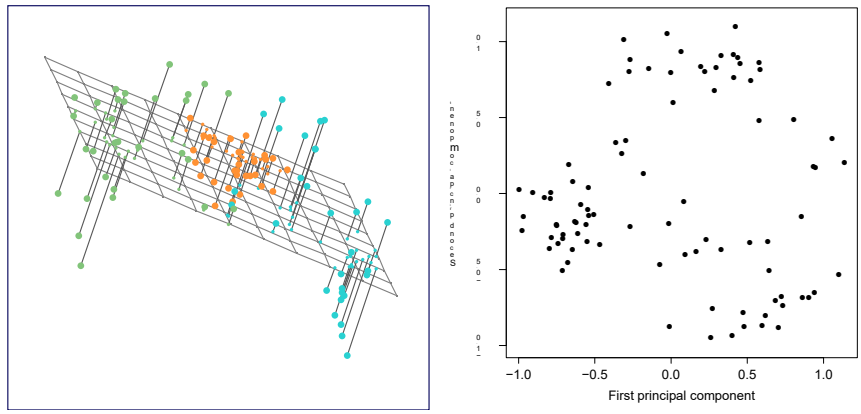


图 12.2. *Ninety observations simulated in three dimensions. The observations are displayed in color for ease of visualization.* 左：*the first two principal component directions span the plane that best fits the data. The plane is positioned to minimize the sum of squared distances to each point.* 右：*the first two principal component score vectors give the coordinates of the projection of the 90 observations onto the plane.*

有用：主成分提供了低维线性表面，这些表面对于观测值是*closest*。我们在此进一步扩展这一解释。[3]

第一个主成分载荷向量具有一个非常特殊的性质：它是在p维空间中与n观测值*closest*的那条直线（使用平均平方欧氏距离作为接近度的度量）。这种解释可以在图6.15的左侧面板中看到；虚线表示每个观测值与由第一个主成分载荷向量定义的直线之间的距离。这种解释的吸引力很明显：我们寻找数据的一个单一维度，使其尽可能接近所有数据点，因为这样一条线很可能提供对数据的良好概括。

主成分作为最接近n观测的维度这一概念不仅限于第一主成分。例如，一个数据集的前两个主成分张成的平面，在平均平方欧几里得距离意义下，是最接近n观测的平面。图12.2左侧面板展示了一个示例。一个数据集的前三个主成分张成的三维超平面是最接近n观测的超平面，依此类推。

使用这种解释，前M个主成分得分向量和前M个主成分载荷向量共同提供了对以下内容的最佳M维近似（就欧几里得距离而言）

[3]In this section, we continue to assume that each column of the data matrix $\mathbf{X}$ has been centered to have mean zero—that is, the column mean has been subtracted from each column.

t第i个观测值x_{ij}。这种表示可以写成 n 为

$$x_{ij} \approx \sum_{m=1}^{M} z_{im}\phi_{jm}. \tag{12.5}$$

我们可以通过写出一个优化问题来更正式地陈述这一点。假设数据矩阵 $\mathbf{X}$ 是列中心化的。在所有形式为 $x_{ij} \approx \sum_{m=1}^{M} a_{im}b_{jm}$ 的近似中，我们可以要求残差平方和最小的那个:

$$\underset{\mathbf{A}\in\mathbb{R}^{n\times M},\mathbf{B}\in\mathbb{R}^{p\times M}}{\text{minimize}} \left\{ \sum_{j=1}^{p}\sum_{i=1}^{n} \left(x_{ij} - \sum_{m=1}^{M} a_{im}b_{jm} \right)^2 \right\}. \tag{12.6}$$

这里，$\mathbf{A}$ 是一个 $n \times M$ 矩阵，其 (i, m) 元素为 a_{im}，而 $\mathbf{B}$ 是一个 $p \times M$ 元素，其 (j, m) 元素为 b_{jm}。

可以证明，对于任意 M 的值，解(12.6)的矩阵 $\hat{\mathbf{A}}$ 和 $\hat{\mathbf{B}}$ 的列实际上就是前 M 个主成分得分和载荷向量。换言之，如果 $\hat{\mathbf{A}}$ 和 $\hat{\mathbf{B}}$ 解(12.6)，那么 $\hat{a}_{im} = z_{im}$ 和 $\hat{b}_{jm} = \phi_{jm}$。[4] 这意味着(12.6)中目标函数的最小可能值是

$$\sum_{j=1}^{p}\sum_{i=1}^{n} \left(x_{ij} - \sum_{m=1}^{M} z_{im}\phi_{jm} \right)^2. \tag{12.7}$$

总之，当M足够大时，M主成分得分向量和M主成分载荷向量一起可以很好地近似数据。当$M = \min(n-1, p)$时，表示是精确的: $x_{ij} = \sum_{m=1}^{M} z_{im}\phi_{jm}$。

12.2.3 The Proportion of Variance Explained

在图12.2中，我们对一个三维数据集（左侧面板）执行了主成分分析，并将数据投影到前两个主成分载荷向量上，以获得数据的二维视图（即主成分得分向量；右侧面板）。我们可以看到，三维数据的这种二维表示确实成功捕捉到了数据中的主要模式：在三维空间中彼此接近的橙色、绿色和青色观测值在二维表示中仍然保持接近。同样，在`USArrests`数据集上，我们已经看到可以仅使用前两个主成分得分向量和前两个主成分载荷向量来概括50个观测值和4个变量。

现在我们可以提出一个自然的问题：将观测数据投影到前几个主成分上，给定数据集中的信息会损失多少？也就是说，数据中的方差有多少包含在前几个主成分中？更一般地说，我们感兴趣的是了解每个主成分的 *proportion of variance explained* (PVE)。

proportion of variance explained

[4] Technically, the solution to (12.6) is not unique. Thus, it is more precise to state that any solution to (12.6) can be easily transformed to yield the principal components.

主成分。数据集中的 *total variance*（假设变量已经中心化，均值为零）被定义为

$$\sum_{j=1}^{p} \mathrm{Var}(X_j) = \sum_{j=1}^{p} \frac{1}{n} \sum_{i=1}^{n} x_{ij}^2, \tag{12.8}$$

而由第 m 个主成分解释的方差为

$$\frac{1}{n} \sum_{i=1}^{n} z_{im}^2 = \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{p} \phi_{jm} x_{ij} \right)^2. \tag{12.9}$$

因此，第m个主成分的PVE为

$$\frac{\sum_{i=1}^{n} z_{im}^2}{\sum_{j=1}^{p} \sum_{i=1}^{n} x_{ij}^2} = \frac{\sum_{i=1}^{n} \left(\sum_{j=1}^{p} \phi_{jm} x_{ij} \right)^2}{\sum_{j=1}^{p} \sum_{i=1}^{n} x_{ij}^2}. \tag{12.10}$$

每个主成分的PVE是一个正值。为了计算前M个主成分的累积PVE，我们只需对前M个PVE求和 (12.10)。总共有$\min(n-1, p)$个主成分，它们的PVE之和为1。

在第12.2.2节中，我们展示了第一M主成分载荷和得分向量可以被解释为对数据的最佳M维近似，就残差平方和而言。事实证明，数据的方差可以分解为第一M主成分的方差加上这个M维近似的均方误差，如下所示：

$$\underbrace{\sum_{j=1}^{p} \frac{1}{n} \sum_{i=1}^{n} x_{ij}^2}_{\text{Var. of data}} = \underbrace{\sum_{m=1}^{M} \frac{1}{n} \sum_{i=1}^{n} z_{im}^2}_{\text{Var. of first } M \text{ PCs}} + \underbrace{\frac{1}{n} \sum_{j=1}^{p} \sum_{i=1}^{n} \left(x_{ij} - \sum_{m=1}^{M} z_{im} \phi_{jm} \right)^2}_{\text{MSE of } M\text{-dimensional approximation}} \tag{12.11}$$

这个分解中的三项分别在(12.8)、(12.9)和(12.7)中讨论。由于第一项是固定的，我们看到，通过最大化前M个主成分的方差，我们最小化了M维近似的均方误差，反之亦然。这解释了为什么主成分可以等价地视为最小化近似误差（如第12.2.2节所述）或最大化方差（如第12.2.1节所述）。

此外，我们可以利用(12.11)看出，(12.10)中定义的PVE等于

$$1 - \frac{\sum_{j=1}^{p} \sum_{i=1}^{n} \left(x_{ij} - \sum_{m=1}^{M} z_{im} \phi_{jm} \right)^2}{\sum_{j=1}^{p} \sum_{i=1}^{n} x_{ij}^2} = 1 - \frac{\mathrm{RSS}}{\mathrm{TSS}},$$

其中 TSS 表示 $\mathbf{X}$ 的元素平方总和，而 RSS 表示由主成分给出的 M 维近似的残差平方和。回顾 (3.17) 中 R^2 的定义，这意味着我们可以将由前 M 个主成分给出的 $\mathbf{X}$ 近似的 PVE 解释为 R^2。

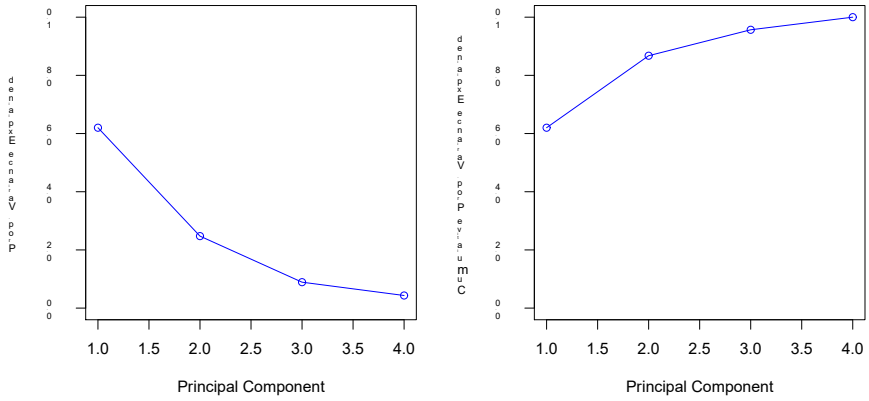


图12.3. 左：*a scree plot depicting the proportion of variance explained by each of the four principal components in the* `USArrests` *data.* 右：*the cumulative proportion of variance explained by the four principal components in the* `USArrests` *data.*

在`USArrests`数据中，第一个主成分解释了数据中62.0%的方差，而下一个主成分解释了24.7%的方差。前两个主成分合起来几乎解释了数据中87%的方差，而最后两个主成分仅解释了13%的方差。这意味着图12.1仅用两个维度就提供了对数据的相当准确的总结。每个主成分的PVE以及累积PVE如图12.3所示。左侧面板被称为*scree plot*，并将是

scree plot

本章稍后讨论。

12.2.4 More on PCA

缩放变量

我们已经提到，在执行PCA之前，应将变量中心化，使其均值为零。此外，*the results obtained when we perform PCA will also depend on whether the variables have been individually scaled* (各自乘以不同的常数)。这与一些其他监督学习和无监督学习技术，如线性回归，形成对比，在线性回归中，缩放变量没有影响。（在线性回归中，将变量乘以因子c只会导致相应系数估计值乘以因子$1/c$，因此对所得模型没有实质性影响。）

例如，图12.1是在将每个变量缩放至标准差为一后得到的。图12.4左侧的图表重现了这一结果。为什么缩放变量如此重要？在这些数据中，变量是以不同的单位测量的；`Murder`、`Rape`和`Assault`报告的是每100,000人中的发生次数，而`UrbanPop`则是居住在城市地区的州人口百分比。这四个变量的方差分别为18.97、87.73、6945.16和209.5。因此，如果我们对未缩放的变量执行PCA，那么

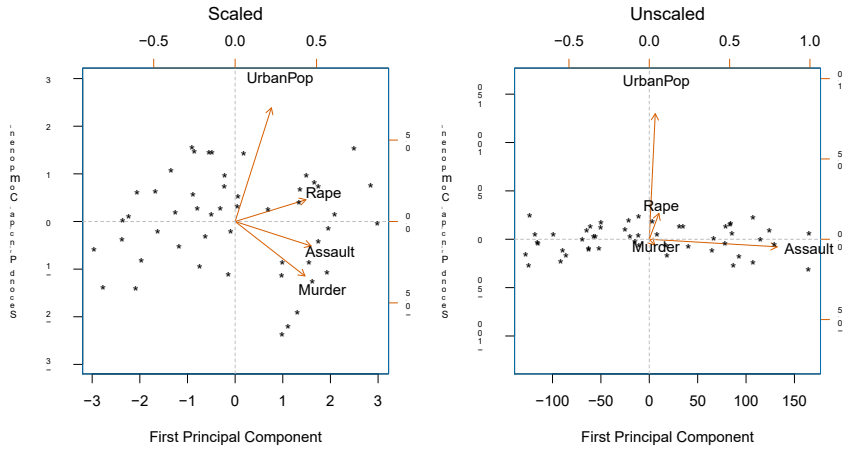


图 12.4. *Two principal component biplots for the* `USArrests` *data.* 左: *the same as Figure 12.1, with the variables scaled to have unit standard deviations.* 右: *principal components using unscaled data.* `Assault` *has by far the largest loading on the first principal component because it has the highest variance among the four variables. In general, scaling the variables to have standard deviation one is recommended.*

第一主成分载荷向量将在`Assault`上具有非常大的载荷，因为该变量的方差远高于其他变量。图12.4右侧的图表展示了`USArrests`数据集的前两个主成分，未将变量标准化至标准差为一。正如预测所示，第一主成分载荷向量几乎将所有权重放在`Assault`上，而第二主成分载荷向量几乎将所有权重放在`UrbanPop`上。与左侧图表对比，我们可以看出，标准化确实对所得结果产生了显著影响。

然而，这个结果仅仅是变量测量尺度不同的结果。例如，如果`Assault`是以每100人中发生次数为单位测量的（而不是每100,000人中发生次数），那么这就相当于将该变量的所有元素除以1,000。这样一来，该变量的方差就会变得非常小，因此第一主成分载荷向量在该变量上的值也会非常小。由于所得到的主成分依赖于任意的缩放选择是不可取的，所以在执行PCA之前，我们通常会将每个变量标准化，使其标准差为1。

然而，在某些情况下，变量可能以相同的单位测量。此时，我们在进行主成分分析之前，可能不希望将变量缩放到标准差为1。例如，假设给定数据集中的变量对应于p个基因的表达水平。那么，由于每个基因的表达是以相同的“单位”测量的，我们可能选择不将基因缩放到各自标准差为1。

主成分的唯一性

虽然在理论上主成分不必唯一，但在几乎所有实际情境中，它们都是唯一的（除符号翻转外）。这意味着两个不同的软件包将得出相同的主成分载荷向量，尽管这些载荷向量的符号可能不同。符号可能不同，因为每个主成分载荷向量都在p维空间中指定了一个方向：翻转符号没有影响，因为方向不变。（考虑图6.14——主成分载荷向量是一条向任一方向延伸的直线，翻转其符号不会有任何影响。）同样，得分向量在符号翻转的意义下是唯一的，因为Z的方差与$-Z$的方差相同。值得注意的是，当我们使用（12.5）来近似x_{ij}时，我们将z_{im}乘以ϕ_{jm}。因此，如果载荷向量和得分向量的符号都被翻转，这两个量的最终乘积不变。

决定使用多少个主成分

一般来说，一个 $n \times p$ 数据矩阵 $\mathbf{X}$ 具有 $\min(n-1, p)$ 个不同的主成分。然而，我们通常并不对所有这些主成分都感兴趣；相反，我们只想使用前几个主成分来可视化或解释数据。事实上，我们希望使用最少的主成分数量来获得对数据的 $good$ 理解。需要多少个主成分？不幸的是，这个问题没有单一（或简单！）的答案。

我们通常通过检查$scree\ \ plot$来决定可视化数据所需的主成分数量，例如图12.3左侧面板所示的那种。我们选择能够解释数据中相当大一部分变异所需的最少主成分数量。这是通过肉眼观察碎石图，并寻找每个后续主成分所解释的方差比例下降的点来完成的。这种下降在碎石图中常被称为$elbow$。例如，通过观察图12.3，人们可能会得出结论，前两个主成分解释了相当大一部分方差，并且在第二个主成分之后有一个肘部。毕竟，第三个主成分解释了数据中不到百分之十的方差，而第四个主成分解释的方差不到前者的一半，因此基本上毫无价值。

然而，这种可视化分析本质上是$ad\ hoc$。遗憾的是，对于决定需要多少个主成分才$enough$，尚无公认的客观方法。事实上，需要多少个主成分才算足够，这个问题本质上是不明确的，并且将取决于具体的应用领域和具体的数据集。在实践中，我们倾向于查看前几个主成分，以寻找数据中有趣的模式。如果在前几个主成分中没有发现任何有趣的模式，那么后续的主成分不太可能引起兴趣。相反，如果前几个主成分很有趣，那么我们通常会继续查看后续的主成分，直到不再发现任何有趣的模式。诚然，这是一个主观的

方法，并且反映了PCA通常被用作探索性数据分析工具的事实。

另一方面，如果我们计算主成分以用于监督分析，例如在第6.3.1节中介绍的主成分回归，那么有一个简单而客观的方法来确定使用多少个主成分：我们可以将回归中要使用的主成分得分向量的数量视为一个调优参数，通过交叉验证或相关方法来选择。监督分析中选择主成分数量的相对简单性，是监督分析往往比无监督分析更明确界定且更客观评估这一事实的一个体现。

12.2.5 Other Uses for Principal Components

我们在第6.3.1节中看到，可以使用主成分得分向量作为特征来进行回归。事实上，许多统计技术，如回归、分类和聚类，都可以轻松地适应使用列为前$M \ll p$个主成分得分向量的$n \times M$矩阵，而不是使用完整的$n \times p$数据矩阵。这可能会带来*less noisy*的结果，因为数据集中（与噪声相对的）信号往往集中在其前几个主成分中。

12.3 缺失值与矩阵补全

数据集常常存在缺失值，这可能很麻烦。例如，假设我们希望分析`USArrests`数据，并发现200个值中有20个被随机破坏并标记为缺失。不幸的是，我们在本书中见过的统计学习方法无法处理缺失值。我们该如何进行?

我们可以删除包含缺失观测值的行，并在完整行上进行数据分析。但这似乎很浪费，而且取决于缺失的比例，可能并不现实。或者，如果x_{ij}缺失，我们可以用第j列的均值来替代它（使用非缺失项计算均值）。尽管这是一种常见且方便的策略，但通常我们可以通过利用变量之间的相关性做得更好。

在本节中，我们展示如何使用主成分来*impute*缺失值，通过称为*matrix completion*的过程。该然后，完成的矩阵可以用于统计学习方法，如线性回归或LDA。

插补 插补

矩阵补全

这种缺失数据插补方法在缺失是随机的情况下是合适的。例如，如果患者的体重缺失是因为考试时电子秤的电池没电了。相比之下，如果体重缺失是因为患者太重无法上秤，那么这并非随机缺失；缺失性

missing at random

提供信息，且此处描述的缺失数据处理方法并不合适。

有时数据是必然缺失的。例如，如果我们构建一个矩阵，表示 n 位客户对 p 部电影的整个 Netflix 目录的评分（按 1 到 5 的等级），那么矩阵中的大部分元素都会缺失，因为没有客户会看过并评价过整个目录中超过极小一部分的内容。如果我们能很好地填补缺失值，那么我们就能了解每位客户对他们尚未看过的电影会有什么看法。因此，矩阵补全可以用来驱动 *recommender systems*。

recommender systems

含缺失值的主成分分析

在第12.2.2节中，我们展示了第一个M主成分得分和载荷向量在(12.6)的意义上提供了对数据矩阵$\mathbf{X}$的“最佳”近似。假设有些观测值x_{ij}缺失。我们现在展示如何同时填补缺失值并解决主成分问题。我们回到优化问题(12.6)的修改形式，

$$\underset{\mathbf{A}\in\mathbb{R}^{n\times M},\mathbf{B}\in\mathbb{R}^{p\times M}}{\text{minimize}} \left\{ \sum_{(i,j)\in\mathcal{O}} \left(x_{ij} - \sum_{m=1}^{M} a_{im} b_{jm} \right)^2 \right\}, \tag{12.12}$$

其中 $\mathcal{O}$ 是所有 *observed* 对索引对 (i, j) 的集合，即可能的 $n \times p$ 对的一个子集。

一旦我们解决这个问题：

- 我们可以使用$\hat{x}_{ij} = \sum_{m=1}^{M} \hat{a}_{im}\hat{b}_{jm}$来估计缺失的观测值$x_{ij}$，其中$\hat{a}_{im}$和$\hat{b}_{jm}$分别是求解（12.12）的矩阵$\hat{\mathbf{A}}$和$\hat{\mathbf{B}}$中的（$i, m$）和（$j, m$）元素；并且
- 我们可以（近似地）恢复 M 主成分得分和载荷，就像数据完整时那样。

事实证明，精确求解(12.12)是困难的，与完整数据的情况不同：特征分解不再适用。但算法12.1中的简单迭代方法，在第12.5.2节中得到了演示，通常能提供良好的解决方案。[5][6]

我们在 `USArrests` 数据上说明算法12.1。有 $p = 4$ 个变量和 $n = 50$ 个观测（州）。我们首先对数据进行了标准化，使每个变量的均值为零、标准差为一。然后，我们随机选择了50个州中的20个，接着对其中的每个州随机将四个变量之一设为缺失。因此，数据矩阵中有10%的元素缺失。我们应用算法12.1，使用 $M = 1$ 个主成分。图12.5显示缺失元素的恢复

[5] This algorithm is referred to as “Hard-Impute” in Mazumder, Hastie, and Tibshirani (2010) “Spectral regularization algorithms for learning large incomplete matrices”, published in *Journal of Machine Learning Research*, pages 2287–2322.

[6] Each iteration of Step 2 of this algorithm decreases the objective (12.14). However, the algorithm is not guaranteed to achieve the global optimum of (12.12).

A算法12.1 *Iterative Algorithm for Matrix Complet ion*

1. 创建一个维度为$n \times p$的完整数据矩阵$\tilde{\mathbf{X}}$，其中(i,j)元素等于

$$\tilde{x}_{ij} = \begin{cases} x_{ij} & \text{if } (i,j) \in \mathcal{O} \\ \bar{x}_j & \text{if } (i,j) \notin \mathcal{O}, \end{cases}$$

其中$\bar{x}_j$是不完整数据矩阵$\mathbf{X}$中第j个变量的观测值的平均值。这里，$\mathcal{O}$索引在$\mathbf{X}$中观测到的观测值。

2. R 重复步骤(a)–(c)，直到目标函数(12.14)不再下降 折痕:

(a) 求解

$$\underset{\mathbf{A}\in\mathbb{R}^{n\times M},\mathbf{B}\in\mathbb{R}^{p\times M}}{\text{minimize}} \left\{ \sum_{j=1}^{p}\sum_{i=1}^{n} \left(\tilde{x}_{ij} - \sum_{m=1}^{M} a_{im} b_{jm} \right)^2 \right\} \tag{12.13}$$

通过计算$\tilde{\mathbf{X}}$的主成分。

(b) 对于每个元素$(i,j) \notin \mathcal{O}$，设置$\tilde{x}_{ij} \leftarrow \sum_{m=1}^{M} \hat{a}_{im}\hat{b}_{jm}$。

(c) 计算目标

$$\sum_{(i,j)\in\mathcal{O}} \left(x_{ij} - \sum_{m=1}^{M} \hat{a}_{im}\hat{b}_{jm} \right)^2. \tag{12.14}$$

3. 返回估计的缺失条目$\tilde{x}_{ij}$，$(i,j) \notin \mathcal{O}$。

相当准确。在这个实验的100多次随机运行中，缺失元素的真实值与插补值之间的平均相关系数为0.63，标准差为0.11。这是否算表现良好？为了回答这个问题，我们可以将这一相关系数与使用*complete*数据估计这20个值时得到的结果进行比较——也就是说，如果我们只是计算$\hat{x}_{ij} = z_{i1}\phi_{j1}$，其中$z_{i1}$和$\phi_{j1}$是完整数据第一主成分得分和载荷向量的元素。[7]以这种方式使用完整数据，这20个元素的真实值与估计值之间的平均相关系数为0.79，标准差为0.08。因此，我们的插补方法不如使用全部数据的方法（0.63 ± 0.11 对比 0.79 ± 0.08），但其表现仍然相当不错。（当然，使用全部数据的方法无法应用于存在缺失数据的现实场景。）

图12.6进一步表明，算法12.1在此数据集上表现相当好。

[7]This is an unattainable gold standard, in the sense that with missing data, we of course cannot compute the principal components of the complete data.

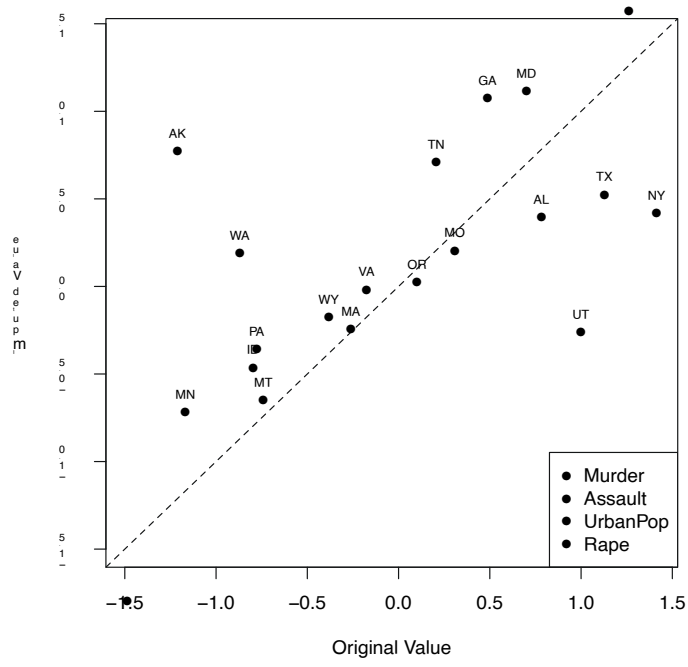


图12.5。*Missing value imputation on the* `USArrests` *data. Twenty values (10% of the total number of matrix elements) were artificially set to be missing, and then imputed via Algorithm 12.1 with* $M = 1$. *The figure displays the true value* x_{ij} *and the imputed value* $\hat{x}_{ij}$ *for all twenty missing values. For each of the twenty missing values, the color indicates the variable, and the label indicates the state. The correlation between the true and imputed values is around* 0.63.

最后，我们提出几点观察：

- `USArrests` 数据只有四个变量，对于像算法12.1这样的方法来说，这属于较低水平，难以良好运作。因此，为了本演示，我们随机设定每个状态最多一个变量缺失，并且仅使用了 $M = 1$ 个主成分。

- 一般来说，为了应用算法12.1，我们必须选择M，即用于填补的主成分数量。一种方法是从矩阵中随机额外留出一些元素，并根据这些已知值的恢复情况来选择M。这与第5章中看到的验证集方法密切相关。

推荐系统

像Netflix和Amazon这样的数字流媒体服务会利用客户过去观看过的内容数据以及其他客户的数据，来为客户推荐其他内容。举一个具体的例子，几年前，Netflix让客户对他们看过的每部电影按1–5分进行评分。这产生了一个非常大的$n \times p$矩阵，其中(i, j)元素是第i位客户对

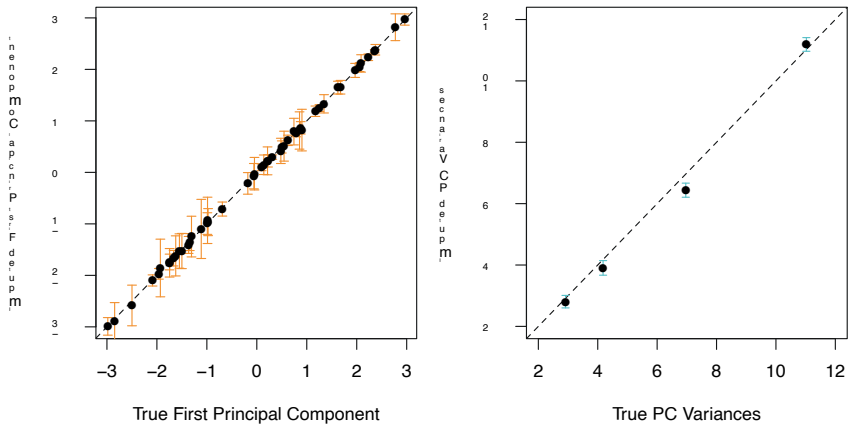


图12.6. *As described in the text, in each of 100 trials, we left out 20 elements of the* `USArrests` *dataset. In each trial, we applied Algorithm 12.1 with* $M = 1$ *to impute the missing elements and compute the principal components.* 左: *For each of the 50 states, the imputed first principal component scores (averaged over 100 trials, and displayed with a standard deviation bar) are plotted against the first principal component scores computed using all the data.* 右: *The imputed principal component loadings (averaged over 100 trials, and displayed with a standard deviation bar) are plotted against the true principal component loadings.*

jth 电影。这个矩阵的一个具体早期示例有 $n = 480{,}189$ 个客户和 $p = 17{,}770$ 部电影。然而，平均每个客户看过大约 200 部电影，因此矩阵中有 99% 的元素缺失。表 12.2 说明了这一设置。

为了向特定客户推荐其可能喜欢的电影，Netflix需要一种方法来估算此数据矩阵中的缺失值。关键思路如下：第i位客户看过的电影集合将与其他客户看过的电影有重叠。此外，其中一些其他客户将拥有与第i位客户相似的电影偏好。因此，应该能够使用相似客户对第i位客户尚未看过的电影的评价，来预测第i位客户是否会喜欢这些电影。

更具体地说，通过应用算法12.1，我们可以利用$\hat{x}_{ij} = \sum_{m=1}^{M} \hat{a}_{im}\hat{b}_{jm}$预测第$i$位顾客对第$j$部电影的评分。此外，我们还可以从“派系”和“类型”的角度来解释M中的成分：

- $\hat{a}_{im}$ 表示第i个用户属于第m个派系的强度，其中$clique$是一群喜欢第m种类型电影的顾客；

- $\hat{b}_{jm}$ 表示第j部电影属于第m个$genre$的强度。

E类型示例包括浪漫、西部和动作 n.
类似于算法12.1的主成分模型是许多推荐系统的核心。尽管所涉及的数据矩阵

	Jurassic Park	Braveheart	Terminator 2	Amadeus	Christmas Vacation	Dumbo	Top Gun	Terminator	Casablanca	Titanic	⋱
Customer 1	•	•	•	•	4	•	•	•	•	•	⋯
Customer 2	•	•	3	•	•	•	3	•	•	3	⋯
Customer 3	•	2	•	4	•	•	•	•	2	•	⋯
Customer 4	3	•	•	•	•	•	•	•	•	•	⋯
Customer 5	5	1	•	•	4	•	•	•	•	•	⋯
Customer 6	•	•	•	•	•	2	4	•	•	•	⋯
Customer 7	•	•	5	•	•	•	•	3	•	•	⋯
Customer 8	•	•	•	•	•	•	•	•	•	•	⋯
Customer 9	3	•	•	•	5	•	•	1	•	•	⋯
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋱

表12.2。*Excerpt of the Netflix movie rating data. The movies are rated from 1 (worst) to 5 (best). The symbol • represents a missing value: a movie that was not rated by the corresponding customer.*

通常是海量的，已开发出能够利用高水平缺失性来执行高效计算的算法。

12.4 聚类方法

Clustering 指的是一组非常广泛的用于寻找 *subgroups* 的技术，或者 *clusters*，在一个数据集中。当我们对数据集的观测进行聚类时，我们试图将它们划分为不同的组，使得每组内的观测彼此非常相似，而不同组中的观测彼此差异很大。当然，为了使其具体化，我们必须定义两个或多个观测为*similar*或*different*意味着什么。实际上，这通常是一个特定领域的考量，必须基于对所研究数据的了解来做出。

clustering

例如，假设我们有一组 n 个观测值，每个观测值有 p 个特征。这些 n 个观测值可能对应于乳腺癌患者的组织样本，而 p 个特征可能对应于每个组织样本收集的测量数据；这些数据可能是临床测量结果，如肿瘤分期或分级，也可能是基因表达测量值。我们可能有理由相信这 n 个组织样本之间存在某种异质性；例如，也许存在几种不同的 *un- known* 乳腺癌亚型。聚类可以用来发现这些亚组。这是一个无监督问题，因为我们试图根据一个数据集发现结构——在这种情况下，即不同的聚类。另一方面，监督问题的目标是尝试预测某个结果向量，例如生存时间或对药物治疗的反应。

聚类和PCA都试图通过少量摘要来简化数据，但它们的机制不同：

- PCA旨在找到观测数据的低维表示，以解释大部分方差；
- 聚类旨在在观测值中寻找同质的子组。

聚类的另一个应用出现在市场营销中。我们可能拥有针对大量人员的大量测量数据（例如家庭收入中位数、职业、距最近城区的距离等等）。我们的目标是通过识别可能更容易接受某种特定广告形式，或更可能购买某种特定产品的人群子组来执行*market segmentation*。执行市场细分的任务相当于对数据集中的这些人进行聚类。

由于聚类在许多领域都很流行，因此存在大量的聚类方法。在本节中，我们重点介绍也许是最著名的两种聚类方法：*K-means clustering*和*hierarchical clustering*在*K*-均值聚类中，我们试图划分观测值转换成预先指定数量的聚类。另一方面，在层次聚类中，我们事先并不知道想要多少个聚类；事实上，我们最终得到的是观测值的树状可视化表示，称为*dendrogram*，它使我们能够一次性查看在每个可能的聚类数量，从1到n。这些聚类方法各有优缺点，我们将在本章中予以强调。

K-means 聚类

层次聚类

dendrogram

一般来说，我们可以根据特征对观测进行聚类，以识别观测中的子群，或者我们可以根据观测对特征进行聚类，以发现特征中的子群。在下文中，为简便起见，我们将讨论基于特征对观测的聚类，尽管通过简单地转置数据矩阵，也可以进行相反的操作。

12.4.1 K-Means Clustering

*K*K-均值聚类是一种简单而优雅的方法，用于将数据集划分为 K 个不同的、不重叠的簇。为了执行 *K*-均值聚类，我们必须首先指定期望的簇数量 K；然后 *K*-均值算法将把每个观测值分配到 K 个簇中的恰好一个。图 12.7 展示了在一个由二维数据中 150 个观测值组成的模拟示例上执行 *K*-均值聚类所得到的结果，其中使用了三个不同的 K 值。

K 均值聚类过程源于一个简单直观的数学问题。我们首先定义一些符号。令 $C_1, \ldots, C_K$ 表示包含每个聚类中观测值索引的集合。这些集合满足两个性质：

1. $C_1 \cup C_2 \cup \cdots \cup C_K = \{1, \ldots, n\}$。换言之，每个观测值至少属于$K$个聚类之一。2. 对于所有$k \neq k'$，有$C_k \cap C_{k'} = \emptyset$。换言之，聚类之间互不重叠：没有观测值属于一个以上的聚类。

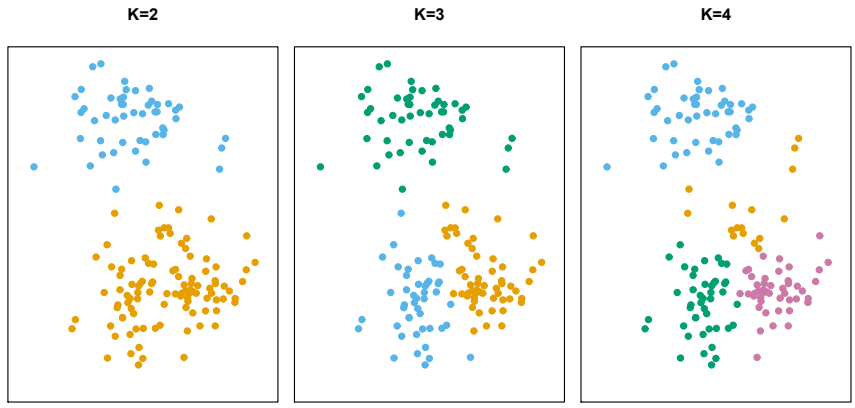


图12.7. *A simulated data set with 150 observations in two-dimensional space. Panels show the results of applying K-means clustering with different values of K, the number of clusters. The color of each observation indicates the cluster to which it was assigned using the K-means clustering algorithm. Note that there is no ordering of the clusters, so the cluster coloring is arbitrary. These cluster labels were not used in clustering; instead, they are the outputs of the clustering procedure.*

例如，如果第 i 个观测值属于第 k 个簇，则 $i \in C_k$。K 均值聚类背后的思想是，*good* 聚类是使 *within-cluster variation* 尽可能小的聚类。簇 C_k 的簇内变异是簇内观测值彼此差异程度的度量 $W(C_k)$。因此，我们希望解决这个问题

$$\underset{C_1,\ldots,C_K}{\text{minimize}} \left\{ \sum_{k=1}^{K} W(C_k) \right\}. \tag{12.15}$$

简而言之，此公式表达的是我们希望将观测值划分为K个聚类，使得所有K个聚类内部变异的总和尽可能小。

求解(12.15)看似合理，但为了使其可操作，我们需要定义簇内变异。定义这一概念有许多可能的方法，但迄今为止最常见的选择涉及 *squared Euclidean distance*。即，我们定义

$$W(C_k) = \frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2, \tag{12.16}$$

其中 $|C_k|$ 表示第 k 个聚类中的观测数。换句话说，第 k 个聚类的簇内变异是第 k 个聚类中观测之间的所有成对平方欧几里得距离之和，除以第 k 个聚类中的总观测数。结合（12.15）和（12.16）给出了定义

K-means 聚类,

$$\underset{C_1,\ldots,C_K}{\text{minimize}} \left\{ \sum_{k=1}^{K} \frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2 \right\}. \tag{12.17}$$

现在，我们想要找到一种算法来解决（12.17）——即一种将观测值划分为K个聚类，使得（12.17）的目标函数最小化的方法。事实上，要精确解决这个问题是非常困难的，因为将n个观测值划分为K个聚类几乎有K^n种方式。除非K和n非常小，否则这是一个巨大的数字！幸运的是，可以证明一种非常简单的算法能够为K-均值优化问题（12.17）提供一个局部最优解——一个*pretty goodsolution*。这种方法在算法12.2中给出。

算法 12.2 *K-Means Clustering*

1. 随机为每个观测值分配一个从1到K的数字。这些作为观测值的初始聚类分配。

2. 迭代直到聚类分配不再变化：

   (a) 对于每个K聚类，计算聚类*centroid*。第k个聚类中心是第k个聚类中观测值的p特征均值的向量。

   (b) 将每个观测值分配给质心最近的聚类（其中 *closest* 使用欧几里得距离定义）。

算法12.2保证在每一步都会降低目标值（12.17）。为了理解这一点，以下恒等式具有启发意义：

$$\frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2 = 2 \sum_{i \in C_k} \sum_{j=1}^{p} (x_{ij} - \bar{x}_{kj})^2, \tag{12.18}$$

其中 $\bar{x}_{kj} = \frac{1}{|C_k|} \sum_{i \in C_k} x_{ij}$ 是聚类 C_k 中特征 j 的均值。在步骤2(a)中，每个特征的聚类均值是使偏差平方和最小化的常数，而在步骤2(b)中，重新分配观测值只能改进(12.18)。这意味着随着算法的运行，所得到的聚类会不断改进，直到结果不再变化；(12.17)的目标函数值永远不会增加。当结果不再变化时，就达到了一个*local optimum*。图12.8展示了算法在图12.7的玩具示例上的进展。K-均值聚类得名于在步骤2(a)中，聚类质心被计算为分配给每个聚类的观测值的均值。

因为K-均值算法找到的是局部最优而非全局最优，所获得的结果将取决于算法12.2第1步中每个观测值的初始（随机）簇分配。因此，重要的是从不同的随机

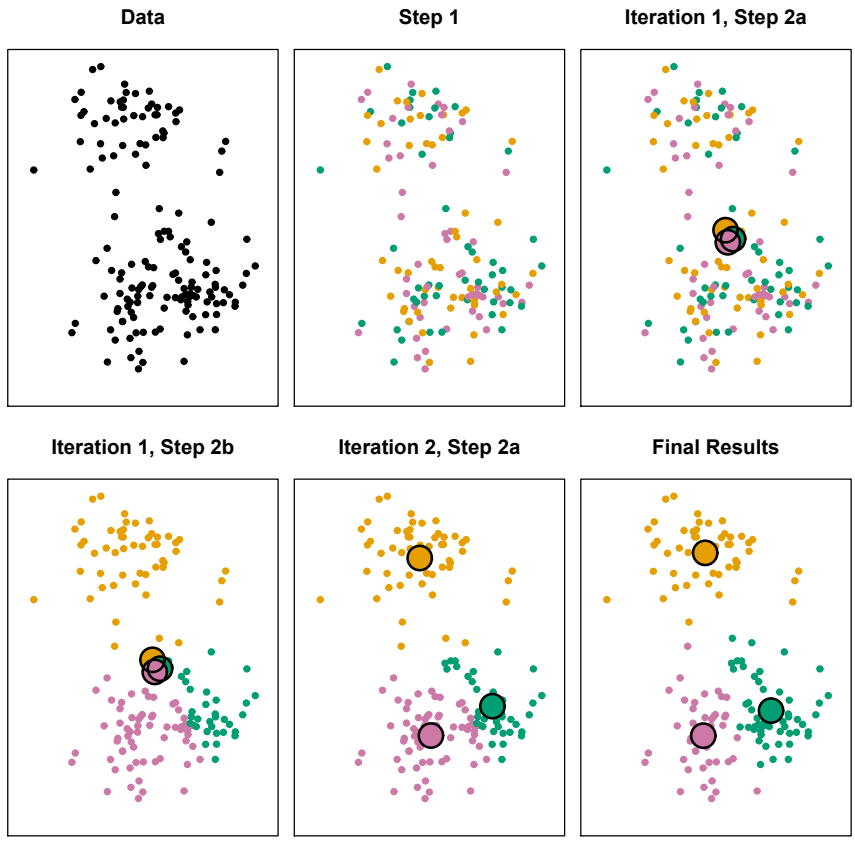


图12.8。 *The progress of the K-means algorithm on the example of Figure 12.7 with K=3.* 左上： *the observations are shown.* 中上： *in Step 1 of the algorithm, each observation is randomly assigned to a cluster.* 右上： *in Step 2(a), the cluster centroids are computed. These are shown as large colored disks. Initially the centroids are almost completely overlapping because the initial cluster assignments were chosen at random.* 左下： *in Step 2(b), each observation is assigned to the nearest centroid.* 中下： *Step 2(a) is once again performed, leading to new cluster centroids.* 右下： *the results obtained after ten iterations.*

初始配置。然后选择 *best* 解，即使目标函数 (12.17) 最小的那个。图12.9展示了使用图12.7中的玩具数据，通过运行 K-均值聚类六次，每次使用不同的初始聚类分配所得到的局部最优解。在这个情况下，最佳聚类是目标函数值为235.8的那个。

正如我们所见，执行K-均值聚类时，我们必须决定数据中预期的聚类数量。选择K的问题远非简单。这一问题连同执行K-均值聚类时出现的其他实际考虑因素，将在第12.4.3节中讨论。

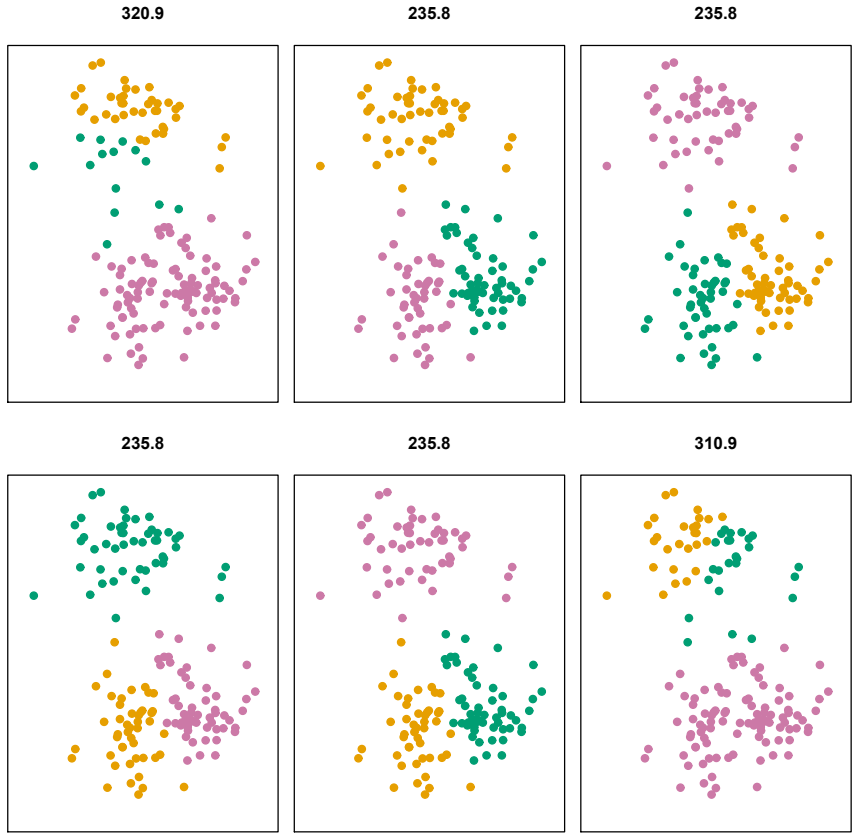


图 12.9. *K-means clustering performed six times on the data from Figure 12.7 with $K = 3$, each time with a different random assignment of the observations in Step 1 of the K-means algorithm. Above each plot is the value of the objective (12.17). Three different local optima were obtained, one of which resulted in a smaller value of the objective and provides better separation between the clusters. Those labeled in red all achieved the same best solution, with an objective value of 235.8.*

12.4.2 Hierarchical Clustering

K-均值聚类的一个潜在缺点是它要求我们预先指定聚类数量*K*。*Hierarchical clustering*是一种替代方法，它不要求我们承诺特定的*K*选择。层次聚类相较于*K*-均值聚类的一个额外优势在于，它能以吸引人的树状结构表示观测数据，称为*dendrogram*。

在本节中，我们描述*bottom-up*或*agglomerative*聚类。这是最常见类型的层次聚类，指的是从叶节点开始构建树状图（通常被描绘成倒置的树；见图12.11），并逐渐合并聚类直至树干。我们将从讨论如何解读树状图开始。

bottom-up
agglomerative

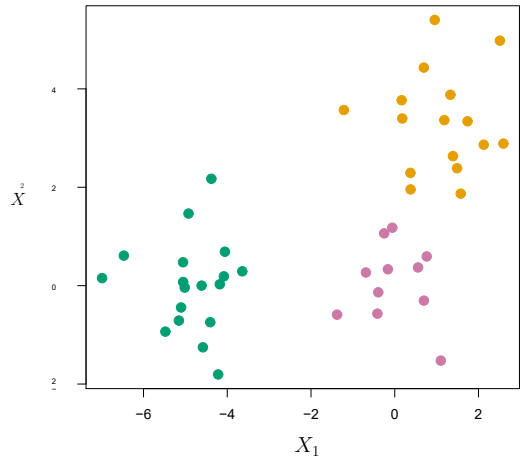


图 12.10. *Forty-five observations generated in two-dimensional space. In reality there are three distinct classes, shown in separate colors. However, we will treat these class labels as unknown and will seek to cluster the observations in order to discover the classes from the data.*

然后讨论层次聚类实际上是如何执行的——也就是说，树状图是如何构建的。

解读树状图

我们从图12.10所示的模拟数据集开始，该数据集包含二维空间中的45个观测值。这些数据由一个三类模型生成；每个观测值的真实类别标签以不同的颜色显示。然而，假设我们观察到的数据没有类别标签，并且希望对数据进行层次聚类。层次聚类（采用完全连接，稍后讨论）得到的结果如图12.11左侧面板所示。我们如何解释这个树状图?

在图12.11的左侧面板中，树状图的每个*leaf*代表图12.10中的45个观测值之一。然而，当我们沿着树向上移动时，一些叶子开始*fuse*成树枝。这些对应于彼此相似的观测值。随着我们在树上继续向上移动，树枝本身也会融合，要么与叶子融合，要么与其他树枝融合。融合发生得越早（在树中位置越低），观测值组之间就越相似。另一方面，融合发生得较晚（接近树顶）的观测值可能相当不同。事实上，这一说法可以精确表述：对于任意两个观测值，我们可以找到树中包含这两个观测值的树枝首次融合的位置。这个融合的高度，按垂直轴测量，表明这两个观测值有多么不同。因此，在树的最底部融合的观测值彼此相当相似，而在接近树顶处融合的观测值则往往相当不同。

这突出了在解读树状图时一个非常重要且常被误解的点。考虑图12.12左侧面板，它展示了一个通过对九个对象进行层次聚类得到的简单树状图。

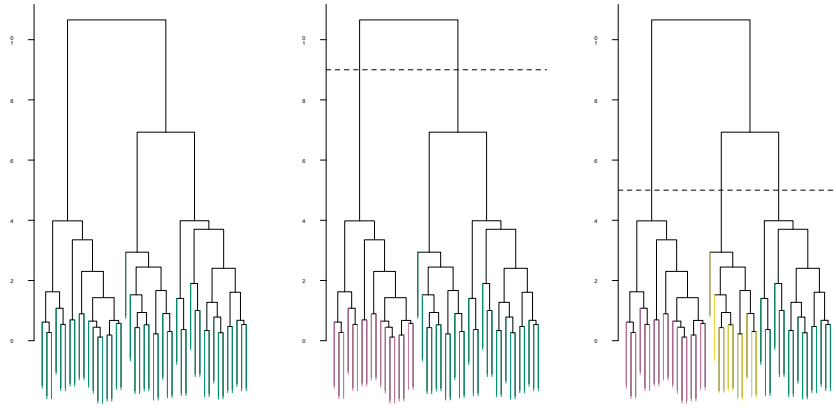


图12.11. 左：*dendrogram obtained from hierarchically clustering the data from Figure 12.10 with complete linkage and Euclidean distance.* 中：*the dendrogram from the left-hand panel, cut at a height of nine (indicated by the dashed line). This cut results in two distinct clusters, shown in different colors.* 右：*the dendrogram from the left-hand panel, now cut at a height of five. This cut results in three distinct clusters, shown in different colors. Note that the colors were not used in clustering, but are simply used for display purposes in this figure.*

观察结果。可以看到，观测5和7彼此相当相似，因为它们在树状图的最低点融合。观测1和6也彼此相当相似。然而，从图中得出观测9和2彼此相当相似这一结论是诱人但不正确的，因为它们在树状图上彼此靠近。事实上，根据树状图所包含的信息，观测9与观测2的相似程度并不超过它与观测8,、5,和7的相似程度。（这可以从图12.12的右侧面板中看出，其中显示了原始数据。）用数学语言来说，树状图有2^{n-1}种可能的重新排序，其中n是叶子的数量。这是因为在发生融合的$n-1$个点中的每一个点处，两个融合分支的位置都可以互换而不影响树状图的含义。因此，我们不能根据两个观测在*horizontal axis*上的接近程度来得出关于它们相似性的结论。相反，我们根据包含这两个观测的分支首次融合所在的*vertical axis*上的位置来得出关于两个观测相似性的结论。

既然我们已经理解了如何解释图12.11左侧面板的含义，接下来就可以探讨基于树状图识别聚类的问题。为此，我们在树状图上进行水平切割，如图12.11的中间和右侧面板所示。切割下方所呈现的不同观测集可被视为聚类。在图12.11的中间面板中，将树状图在高度为9的位置切割，得到两个聚类，用不同的颜色表示。在右侧面板中，将树状图在高度为5的位置切割，则得到三个聚类。随着我们沿树状图向下移动，可以进一步切割以获得任意数量的聚类，从1个（对应无切割）到n

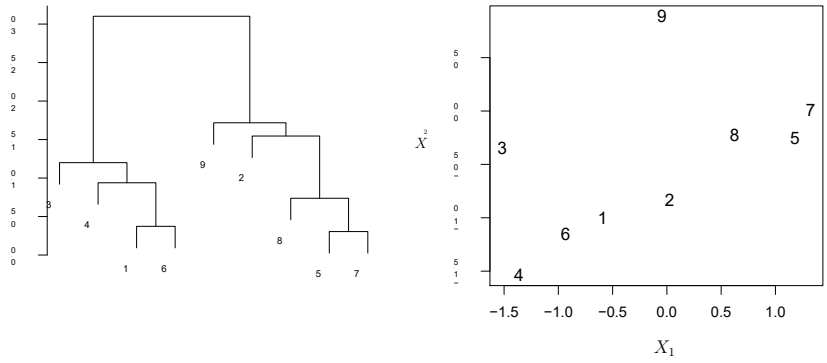


图 12.12。*An illustration of how to properly interpret a dendrogram with nine observations in two-dimensional space.* 左：*a dendrogram generated using Euclidean distance and complete linkage. Observations* 5 *and* 7 *are quite similar to each other, as are observations* 1 *and* 6*. However, observation* 9 *is* 与 *observation* 不再相似 2 *than it is to observations* 8, 5, *and* 7*, even though observations* 9 *and* 2 *are close together in terms of horizontal distance. This is because observations* 2, 8, 5, *and* 7 *all fuse with observation* 9 *at the same height, approximately* 1.8. 右：*the raw data used to generate the dendrogram can be used to confirm that indeed, observation* 9 *is no more similar to observation* 2 *than it is to observations* 8, 5, *and* 7.

(对应于高度 0 处的切割，因此每个观测值都位于其自己的聚类中)。换句话说，树状图切割的高度在K均值聚类中起着与K相同的作用：它控制获得的聚类数量。

图12.11因此突显了层次聚类一个极具吸引力的特点：一个单一的树状图可以用来获得任意数量的聚类。实际上，人们经常查看树状图，并依据融合高度及期望的聚类数量，用肉眼选择一个合理的聚类数。在图12.11的情况下，可以选择两个或三个聚类。然而，很多时候决定在哪里切割树状图并不那么明确。

术语*hierarchical*指的是这样一个事实：通过在某一高度切割树状图得到的聚类必然嵌套在通过在任何更高高度切割树状图得到的聚类之内。然而，在任意数据集上，这种层次结构的假设可能不切实际。例如，假设我们的观测对象对应于一组男性和女性，均匀分布在美国人、日本人和法国人中。我们可以设想这样一种情形：最佳的二组划分可能按性别将这些人群分开，而最佳的三组划分可能按国籍将他们分开。在这种情况下，真实聚类并非嵌套的，也就是说，最佳的三组划分并非来自取最佳的二组划分并拆分其中一个组。因此，这种情况无法由层次聚类很好地表示。由于诸如此类的情况，对于给定的聚类数，层次聚类有时会产生*worse* (即不太准确)的结果，不如K-均值聚类。

算法 12.3 *Hierarchical Clustering*

1. 从n个观测值及所有$\binom{n}{2} = n(n-$个1)/2配对不相似度的度量（如欧几里得距离）开始。将每个观测值视为一个独立的聚类。

2. 对于 $i = n, n-1, \ldots, 2$:

   (a) 检查 i 个聚类之间的所有成对聚类间不相似性，并找出最不相似（即最相似）的一对聚类。融合这两个聚类。这两个聚类之间的不相似性表示在树状图中融合应放置的高度。

   (b) 计算 $i-1$ 剩余聚类之间的新成对聚类间不相似度。

层次聚类算法

层次聚类树状图是通过一个极其简单的算法获得的。我们首先定义每对观测值之间的某种 *dissimilarity* 度量。最常用的是欧几里得距离；我们将在本章后面讨论相异度度量的选择。该算法迭代进行。从树状图的底部开始，每个 n 观测值都被视为其自身的聚类。然后将彼此最相似的两个聚类 *fused*，使得现在有 $n-1$ 个聚类。接下来，再次融合彼此最相似的两个聚类，使得现在有 $n-2$ 个聚类。该算法以这种方式进行，直到所有观测值都属于一个单一聚类，并且树状图完成。图12.13描绘了该算法针对图12.12中数据的前几个步骤。总而言之，层次聚类算法在算法12.3中给出。

这个算法看起来很简单，但有一个问题尚未解决。考虑图12.13中的右下角面板。我们是如何确定簇{5, 7}应该与簇{8}融合的？我们有一个观测对之间差异度的概念，但如果两个簇中的一个或两个包含多个观测，我们如何定义两个簇之间的差异度？一对观测之间的差异度概念需要扩展到一对 *groups of observations*。这种扩展是通过发展*linkage*的概念来实现的，它定义了两组观测之间的差异度。

linkage

最常见的四种连接类型——*complete*、*average*、*single*和*centroid*——在表12.3中简要描述。平均连接、完全连接和单连接在统计学家中最受欢迎。平均连接和完全连接通常比单连接更受青睐，因为它们往往产生更平衡的树状图。质心连接常用于基因组学，但存在一个重大缺陷，即可能出现 *inversion*，从而

inversion

两个聚类在树状图中以高度 *below* 融合， 或是单个聚类。这可能导致在可视化以及树状图的解释上遇到困难。在步骤2(b)中计算的不相似度

Linkage	*Description*
Complete	Maximal intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B, and record the *largest* of these dissimilarities.
Single	Minimal intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B, and record the *smallest* of these dissimilarities. Single linkage can result in extended, trailing clusters in which single observations are fused one-at-a-time.
Average	Mean intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B, and record the *average* of these dissimilarities.
Centroid	Dissimilarity between the centroid for cluster A (a mean vector of length p) and the centroid for cluster B. Centroid linkage can result in undesirable *inversions*.

表 12.3. {v*} {v*}

层次聚类算法的结果将取决于所使用的链接类型，以及所选的不相似度度量。因此，如图12.14所示，最终生成的树状图通常在很大程度上依赖于所使用的链接类型。

相异性度量选择

到目前为止，本章中的示例均使用欧几里得距离作为相异度度量。但有时其他相异度度量可能更为可取。例如，*correlation-based distance* 认为，如果两个观测的特征高度相关，即使观测值在欧几里得距离上可能相距甚远，它们也是相似的。这是相关性的一种不寻常用法，相关性通常在变量之间计算；而这里是在每对观测的观测轮廓之间计算。图 12.15 展示了欧几里得距离与基于相关性的距离之间的差异。基于相关性的距离侧重于观测轮廓的形状，而非其大小。

相异度量的选择非常重要，因为它对生成的树状图有很大影响。一般来说，应仔细关注被聚类的数据类型以及当前的科研问题。这些考虑因素应决定层次聚类中使用哪种类型的相异度量。

例如，假设有一家在线零售商希望根据顾客过去的购物历史对他们进行聚类。目标是识别出*similar*顾客的子群体，以便可以向每个子群体内的顾客展示他们可能特别感兴趣的商品和广告。假设数据以矩阵形式呈现，其中行代表顾客，列代表可供购买的商品；数据矩阵中的元素表示某个给定顾客购买某件商品的次数。

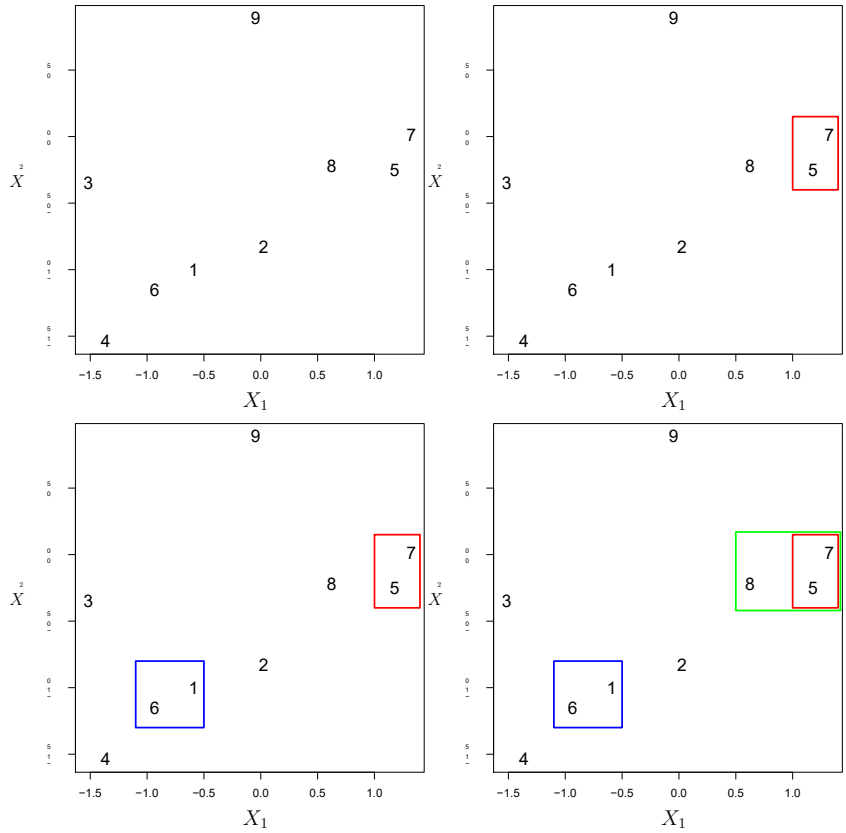


图 12.13. *An illustration of the first few steps of the hierarchical clustering algorithm, using the data from Figure 12.12, with complete linkage and Euclidean distance.* 左上： *initially, there are nine distinct clusters,* $\{1\}, \{2\}, \ldots, \{9\}$. 右上： *the two clusters that are closest together,* $\{5\}$ *and* $\{7\}$, *are fused into a single cluster.* 左下： *the two clusters that are closest together,* $\{6\}$ *and* $\{1\}$, *are fused into a single cluster.* 右下： *the two clusters that are closest together using* 完全连接, $\{8\}$ *and the cluster* $\{5, 7\}$, *are fused into a single cluster.*

给定项（即如果购物者从未购买过该商品，则为0；如果购物者购买过一次，则为1，依此类推）应使用哪种相异性度量来对购物者进行聚类？如果使用欧几里得距离，那么总体上购买很少商品的购物者（即在线购物网站的不频繁用户）会被聚类在一起。这可能并不理想。另一方面，如果使用基于相关性的距离，那么具有相似偏好的购物者（例如购买过商品A和B但从未购买过商品C或D的购物者）会被聚类在一起，即使其中一些具有这些偏好的购物者比其他人的购买量更大。因此，对于此应用，基于相关性的距离可能是更好的选择。

除了仔细选择所用的不相似性度量外，还必须考虑在计算观测值之间的不相似性之前，是否应该将变量缩放至标准差为1。为了说明这一点，我们继续使用在线购物示

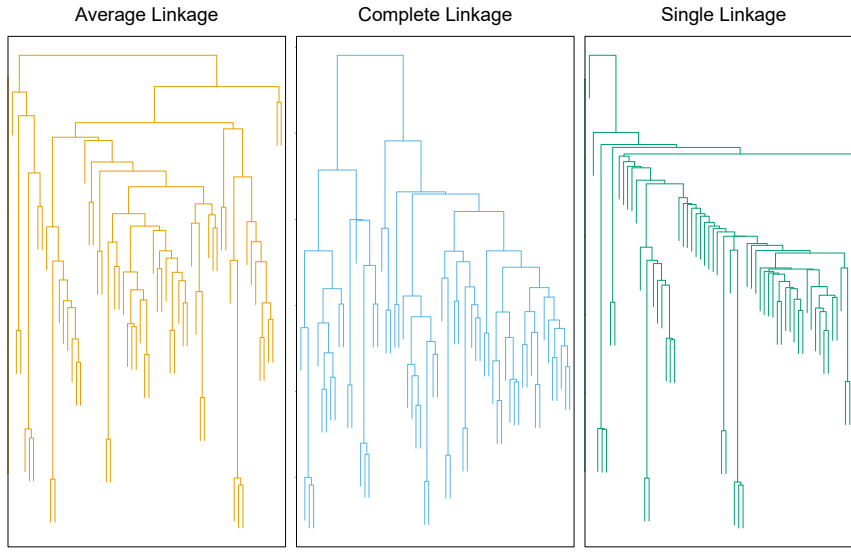


图 12.14。*Average, complete, and single linkage applied to an example data set. Average and complete linkage tend to yield more balanced clusters.*

刚才描述的。有些商品可能比其他商品购买得更频繁；例如，购物者可能一年买十双袜子，但很少买电脑。因此，像袜子这样的高频购买往往比像电脑这样的低频购买对购物者之间的不相似性，从而对最终获得的聚类产生更大的影响。这可能并不理想。如果在计算观察间不相似性之前将变量缩放至标准差为一，那么在进行层次聚类时，每个变量实际上将被赋予同等的重要性。如果变量是在不同的尺度上测量的，我们可能也希望将它们缩放至标准差为一；否则，特定变量的单位选择（例如厘米与千米）将极大地影响所获得的不相似性度量。不足为奇的是，在计算不相似性度量之前是否缩放变量这一决定是否明智取决于手头的应用。一个例子如图12.16所示。我们注意到，在进行聚类之前是否缩放变量的问题同样适用于K-均值聚类。

12.4.3 Practical Issues in Clustering

聚类在无监督环境下可以成为数据分析中一个非常有用的工具。然而，在执行聚类时会遇到一些问题。我们在此描述其中一些。

具有重大后果的小决定

为了进行聚类，必须做出一些决策。

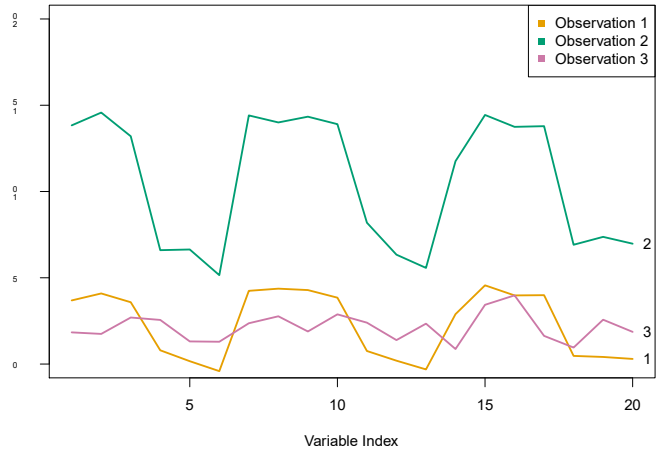


图12.15。*Three observations with measurements on 20 variables are shown. Observations 1 and 3 have similar values for each variable and so there is a small Euclidean distance between them. But they are very weakly correlated, so they have a large correlation-based distance. On the other hand, observations 1 and 2 have quite different values for each variable, and so there is a large Euclidean distance between them. But they are highly correlated, so there is a small correlation-based distance between them.*

- 应该先以某种方式对观测值或特征进行标准化吗？例如，也许变量应该被缩放至标准差为一。

- 在层次聚类的情况下，

  - 应使用哪种相异性度量？ - 应使用哪种链接方式？ - 为了获得聚类，我们应该在哪里切割树状图？

- 在K-均值聚类的情况下，我们应该在数据中寻找多少个聚类？

这些决策中的每一个都可能对所得结果产生重大影响。在实践中，我们会尝试几种不同的选择，并寻找具有最有用或最可解释解的方案。采用这些方法时，并不存在唯一正确答案——任何能揭示数据中某些有趣方面的解都值得考虑。

验证所获得的聚类

任何时候对数据集进行聚类，我们都会找到聚类。但我们真正想知道的是，找到的聚类是否代表了数据中的真实子群，还是仅仅是*clustering the noise*的结果。例如，如果我们获得一组独立的观测值，那么这些观测值是否也会显示出相同的聚类集？这是一个很难回答的问题。存在多种技术可以为聚类分配一个p值，以评估是否有更多

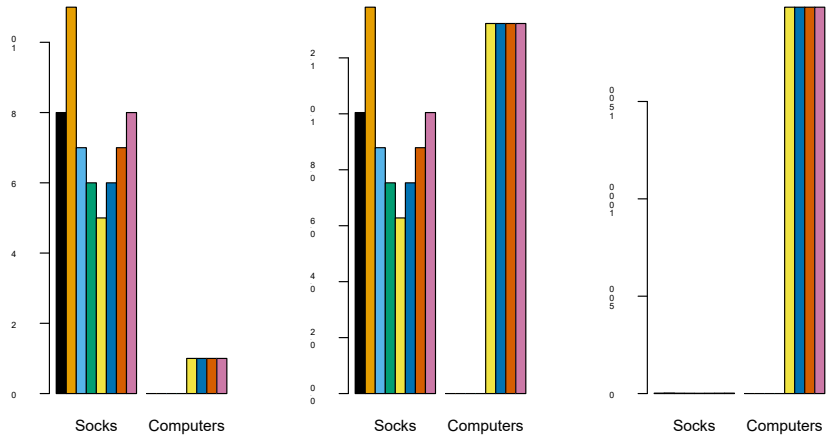


图12.16。*An eclectic online retailer sells two items: socks and computers.*左：*the number of pairs of socks, and computers, purchased by eight online shoppers is displayed. Each shopper is shown in a different color. If inter-observation dissimilarities are computed using Euclidean distance on the raw variables, then the number of socks purchased by an individual will drive the dissimilarities obtained, and the number of computers purchased will have little effect. This might be undesirable, since (1) computers are more expensive than socks and so the online retailer may be more interested in encouraging shoppers to buy computers than socks, and (2) a large difference in the number of socks purchased by two shoppers may be less informative about the shoppers' overall shopping preferences than a small difference in the number of computers purchased.*中：*the same data are shown, after scaling each variable by its standard deviation. Now the two products will have a comparable effect on the inter-observation dissimilarities obtained.*右：*the same data are displayed, but now the y-axis represents the number of dollars spent by each online shopper on socks and on computers. Since computers are much more expensive than socks, now computer purchase history will drive the inter-observation dissimilarities obtained.*

对于该聚类而言，其证据比偶然预期更为显著。然而，关于单一最佳方法尚未达成共识。更多详情可参阅ESL.[8]

聚类中的其他考虑因素

K-均值和层次聚类都会将每个观测值分配到一个簇中。然而，有时这可能并不合适。例如，假设大多数观测值实际上属于少数几个（未知的）子群，而一小部分观测值彼此之间以及与所有其他观测值都大不相同。那么，由于K-均值和层次聚类强制将*every*个观测值归入一个簇，所发现的簇可能会因为存在不属于任何簇的异常值而严重失真。混合模型是应对此类异常值存在的一种有吸引力的方法。这些方法相当于K-均值聚类的*soft*版本，并在ESL中进行了描述。

[8]ESL: *The Elements of Statistical Learning* by Hastie, Tibshirani and Friedman.

此外，聚类方法通常对数据的扰动不太稳健。例如，假设我们对n个观测值进行聚类，然后在随机移除一部分n个观测值后再次对观测值进行聚类。人们会希望得到的两组聚类结果相当相似，但通常情况并非如此！

一种温和的方法来解释聚类结果

我们已经描述了与聚类相关的一些问题。然而，如果使用得当，聚类可以是一种非常有用且有效的统计工具。我们提到，聚类执行过程中的一些小决策，例如数据如何标准化以及使用何种链接方式，都会对结果产生重大影响。因此，我们建议使用这些参数的不同选择进行聚类，并查看完整的结果集，以观察哪些模式会一致地显现出来。由于聚类可能不稳健，我们建议对数据子集进行聚类，以便了解所获得聚类的稳健性。最重要的是，我们必须注意如何报告聚类分析的结果。这些结果不应被视为关于数据集的绝对真理。相反，它们应构成开发科学假设和进一步研究的起点，最好是在独立数据集上进行。

12.5 实验：无监督学习

在本实验中，我们在多个数据集上演示PCA和聚类。与其他实验一样，我们在顶部层级导入一些库。这使代码更具可读性，因为浏览笔记本的前几行就能告诉我们该笔记本使用了哪些库。

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.datasets import get_rdataset
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from ISLP import load_data
```

我们还收集了本实验所需的新导入内容。

In [2]:
```
from sklearn.cluster import \
     (KMeans,
      AgglomerativeClustering)
from scipy.cluster.hierarchy import \
     (dendrogram,
      cut_tree)
from ISLP.cluster import compute_linkage
```

12.5.1 Principal Components Analysis

在本实验中，我们在计算环境R中对数据集USArrests执行PCA。我们使用`get_rdataset()`获取数据，该命令能够获取

`get_rdataset()`

来自许多标准 R 包的数据。数据集的各行包含按字母顺序排列的50个州。

```
In [3]: USArrests = get_rdataset('USArrests').data
        USArrests
```

```
Out[3]:              Murder  Assault  UrbanPop  Rape
         Alabama       13.2      236        58  21.2
          Alaska       10.0      263        48  44.5
         Arizona        8.1      294        80  31.0
             ...        ...      ...       ...   ...
       Wisconsin        2.6       53        66  10.8
         Wyoming        6.8      161        60  15.6
```

数据集的列包含四个变量。

```
In [4]: USArrests.columns
```

```
Out[4]: Index(['Murder', 'Assault', 'UrbanPop', 'Rape'],
              dtype='object')
```

我们首先简要检查数据。我们注意到这些变量的均值差异很大。

```
In [5]: USArrests.mean()
```

```
Out[5]: Murder      7.788
        Assault   170.760
        UrbanPop   65.540
        Rape       21.232
        dtype: float64
```

数据框有几种有用的方法可以计算按列汇总。我们还可以使用 `var()` 方法检查这四个变量的方差。

```
In [6]: USArrests.var()
```

```
Out[6]: Murder      18.970465
        Assault   6945.165714
        UrbanPop   209.518776
        Rape        87.729159
        dtype: float64
```

毫不意外，这些变量的方差也大相径庭。UrbanPop变量衡量的是各州居住在城市的百分比人口，这与每10万人中强奸案的数量不可同日而语。PCA寻找的是能解释数据集中大部分方差的衍生变量。如果我们不在执行PCA之前对变量进行缩放，那么主成分将主要由Assault变量驱动，因为它的方差远大于其他变量。因此，如果变量的测量单位不同或规模差异很大，建议在执行PCA之前将变量标准化，使其标准差为1。通常，我们也会将均值设为零。

这可以通过上面导入的 StandardScaler() 变换来完成缩放。我们首先 fit 缩放器，它会计算所需的均值和

standard deviations and then apply it to our data using the transform method. As before, we combine these steps using the fit_transform() method.

```
In [7]: scaler = StandardScaler(with_std=True,
                                with_mean=True)
        USArrests_scaled = scaler.fit_transform(USArrests)
```

Having scaled the data, we can then perform principal components analysis using the PCA() transform from the sklearn.decomposition package.

PCA()

```
In [8]: pcaUS = PCA()
```

(By default, the PCA() transform centers the variables to have mean zero though it does not scale them.) The transform pcaUS can be used to find the PCA scores returned by fit(). Once the fit method has been called, the pcaUS object also contains a number of useful quantities.

```
In [9]: pcaUS.fit(USArrests_scaled)
```

After fitting, the mean_ attribute corresponds to the means of the variables. In this case, since we centered and scaled the data with scaler() the means will all be 0.

```
In [10]: pcaUS.mean_
```

```
Out[10]: array([-0.,  0., -0.,  0.])
```

The scores can be computed using the transform() method of pcaUS after it has been fit.

```
In [11]: scores = pcaUS.transform(USArrests_scaled)
```

We will plot these scores a bit further down. The components_ attribute provides the principal component loadings: each row of pcaUS.components_ contains the corresponding principal component loading vector.

```
In [12]: pcaUS.components_
```

```
Out[12]: array([[ 0.53589947,  0.58318363,  0.27819087,  0.54343209],
                [ 0.41818087,  0.1879856 , -0.87280619, -0.16731864],
                [-0.34123273, -0.26814843, -0.37801579,  0.81777791],
                [ 0.6492278 , -0.74340748,  0.13387773,  0.08902432]])
```

The biplot is a common visualization method used with PCA. It is not built in as a standard part of sklearn, though there are python packages that do produce such plots. Here we make a simple biplot manually.

```
In [13]: i, j = 0, 1 # which components
         fig, ax = plt.subplots(1, 1, figsize=(8, 8))
         ax.scatter(scores[:,0], scores[:,1])
         ax.set_xlabel('PC%d' % (i+1))
         ax.set_ylabel('PC%d' % (j+1))
         for k in range(pcaUS.components_.shape[1]):
```

```
    ax.arrow(0, 0, pcaUS.components_[i,k], pcaUS.components_[j,k])
    ax.text(pcaUS.components_[i,k],
            pcaUS.components_[j,k],
            USArrests.columns[k])
```

请注意，此图是图12.1关于y轴的镜像。回忆一下，主成分仅在符号变化上不唯一，因此我们可以通过翻转第二组得分和载荷的符号来重现该图。我们还增加了箭头的长度以强调载荷。

In [14]:
```
scale_arrow = s_ = 2
scores[:,1] *= -1
pcaUS.components_[1] *= -1 # flip the y-axis
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(scores[:,0], scores[:,1])
ax.set_xlabel('PC%d' % (i+1))
ax.set_ylabel('PC%d' % (j+1))
for k in range(pcaUS.components_.shape[1]):
    ax.arrow(0, 0, s_*pcaUS.components_[i,k], s_*pcaUS.components_[
        j,k])
    ax.text(s_*pcaUS.components_[i,k],
            s_*pcaUS.components_[j,k],
            USArrests.columns[k])
```

主成分得分的标准差如下：

In [15]:
```
scores.std(0, ddof=1)
```

Out[15]:
```
array([1.5909, 1.0050, 0.6032, 0.4207])
```

每个分数的方差可以通过`explained_variance_`属性直接从`pcaUS`对象中提取。

In [16]:
```
pcaUS.explained_variance_
```

Out[16]:
```
array([2.5309, 1.01  , 0.3638, 0.177 ])
```

每个主成分解释的方差比例（PVE）存储为 `explained_variance_ratio_`：

In [17]:
```
pcaUS.explained_variance_ratio_
```

Out[17]:
```
array([0.6201, 0.2474, 0.0891, 0.0434])
```

我们看到第一个主成分解释了数据中62.0%的方差，第二个主成分解释了24.7%的方差，以此类推。我们可以绘制每个成分解释的PVE，以及累积PVE。首先，我们绘制解释的方差比例。

In [18]:
```
%%capture
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
ticks = np.arange(pcaUS.n_components_)+1
ax = axes[0]
ax.plot(ticks,
        pcaUS.explained_variance_ratio_,
        marker='o')
```

```
ax.set_xlabel('Principal Component');
ax.set_ylabel('Proportion of Variance Explained')
ax.set_ylim([0,1])
ax.set_xticks(ticks)
```

Notice the use of `%%capture`, which suppresses the displaying of the partially completed figure.

In [19]:
```
ax = axes[1]
ax.plot(ticks,
        pcaUS.explained_variance_ratio_.cumsum(),
        marker='o')
ax.set_xlabel('Principal Component')
ax.set_ylabel('Cumulative Proportion of Variance Explained')
ax.set_ylim([0, 1])
ax.set_xticks(ticks)
fig
```

The result is similar to that shown in Figure 12.3. Note that the method `cumsum()` computes the cumulative sum of the elements of a numeric vector. For instance: `cumsum()`

In [20]:
```
a = np.array([1,2,8,-3])
np.cumsum(a)
```

Out[20]:
```
array([ 1,  3, 11,  8])
```

12.5.2 Matrix Completion

我们现在重新进行在第12.3节中对{v*}数据进行的分析。

我们在第12.2.2节看到，在中心化数据矩阵**X**上求解优化问题（12.6）等价于计算数据的前M个主成分。我们使用缩放并中心化的`USArrests`数据作为
X 在下面。*singular value decomposition* (SVD)是一种通用的奇异值分解算法`svd()`，用于求解（12.6）。

In [21]:
```
X = USArrests_scaled
U, D, V = np.linalg.svd(X, full_matrices=False)
U.shape, D.shape, V.shape
```

Out[21]:
```
((50, 4), (4,), (4, 4))
```

`np.linalg.svd()` 函数返回三个分量，`U`、`D` 和 `V`。
矩阵 `V` 等价于主成分的载荷矩阵（不考虑一个不重要的符号翻转）。使用 `full_matrices=False` 选项确保对于高矩阵，`U` 的形状与 `X` 的形状相同。 `np.linalg.svd()`

In [22]:
```
V
```

Out[22]:
```
array([[-0.53589947, -0.58318363, -0.27819087, -0.54343209],
       [ 0.41818087,  0.1879856 , -0.87280619, -0.16731864],
       [-0.34123273, -0.26814843, -0.37801579,  0.81777791],
       [ 0.6492278 , -0.74340748,  0.13387773,  0.08902432]])
```

```
In [23]: pcaUS.components_
```

```
Out[23]: array([[ 0.53589947,  0.58318363,  0.27819087,  0.54343209],
                [ 0.41818087,  0.1879856 , -0.87280619, -0.16731864],
                [-0.34123273, -0.26814843, -0.37801579,  0.81777791],
                [ 0.6492278 , -0.74340748,  0.13387773,  0.08902432]])
```

矩阵 U 对应于 PCA 得分矩阵的 *standardized* 版本（每列标准化为平方和为一）。如果我们用 D 中相应的元素乘以 U 的每一列，就可以精确地恢复 PCA 得分（除了无意义的符号翻转）。

```
In [24]: (U * D[None,:])[:3]
```

```
Out[24]: array([[-0.9856,  1.1334, -0.4443,  0.1563],
                [-1.9501,  1.0732,  2.04  , -0.4386],
                [-1.7632, -0.746 ,  0.0548, -0.8347]])
```

```
In [25]: scores[:3]
```

```
Out[25]: array([[ 0.9856, -1.1334, -0.4443,  0.1563],
                [ 1.9501, -1.0732,  2.04  , -0.4386],
                [ 1.7632,  0.746 ,  0.0548, -0.8347]])
```

虽然可以使用`PCA()`估计器来完成这个实验，但这里我们使用`np.linalg.svd()`函数来演示其用法。

我们现在从 50×4 数据矩阵中随机省略 20 个条目。我们通过首先随机选择 20 行（州），然后在每一行中随机选择四个条目中的一个来实现这一点。这确保每一行至少有三个观测值。

```
In [26]: n_omit = 20
         np.random.seed(15)
         r_idx = np.random.choice(np.arange(X.shape[0]),
                                  n_omit,
                                  replace=False)
         c_idx = np.random.choice(np.arange(X.shape[1]),
                                  n_omit,
                                  replace=True)
         Xna = X.copy()
         Xna[r_idx, c_idx] = np.nan
```

这里，数组 {v*} 包含 20 个从 0 到 49 的整数；这表示被选中包含缺失值的状态（{v*} 的行）。而 {v*} 包含 20 个从 0 到 3 的整数，表示每个选定状态中包含缺失值的特征（{v*} 中的列）。

现在我们编写一些代码来实现算法12.1。首先，我们编写一个函数，该函数接收一个矩阵，并使用`svd()`函数返回矩阵的近似值。这一步在算法12.1的第2步中将会用到。

```
In [27]: def low_rank(X, M=1):
             U, D, V = np.linalg.svd(X)
             L = U[:,:M] * D[None,:M]
             return L.dot(V[:M])
```

为了执行算法的第1步，我们初始化 Xhat——即算法12.1中的$\tilde{\mathbf{X}}$——通过用非缺失项的列均值替换缺失值。这些值在沿行轴运行 np.nanmean() 后，存储在下面的 Xbar 中。我们进行复制，以便当我们赋值

下面的 Xhat 值，我们也不会覆盖 Xna 中的值。

np.nanmean()

In [28]:
```
Xhat = Xna.copy()
Xbar = np.nanmean(Xhat, axis=0)
Xhat[r_idx, c_idx] = Xbar[c_idx]
```

在我们开始第二步之前，我们先准备好衡量迭代进度的方法：

In [29]:
```
thresh = 1e-7
rel_err = 1
count = 0
ismiss = np.isnan(Xna)
mssold = np.mean(Xhat[~ismiss]**2)
mss0 = np.mean(Xna[~ismiss]**2)
```

这里 ismiss 是一个与 Xna 维数相同的逻辑矩阵；若对应矩阵元素缺失，则给定元素为 True。记号 ~ismiss 对该布尔向量取反。这很有用，因为它允许我们同时访问缺失和非缺失条目。我们将非缺失元素的平方均值存储在 mss0 中。我们将旧版本 Xhat 的非缺失元素的均方误差存储在 mssold（中，该值目前与 mss0）一致。我们计划将当前版本 Xhat 的非缺失元素的均方误差存储在 mss 中，然后将迭代算法 12.1 的步骤 2，直到由 (mssold - mss) / mss0 定义的 *relative error* 降至 thresh = 1e-7 以下。[9]

在算法12.1的第2(a)步中，我们使用low_rank()来近似Xhat；我们称此为Xapp。在第2(b)步中，我们利用Xapp来更新Xhat中在Xna缺失的元素的估计值。最后，在第2(c)步中，我们计算相对误差。这三个步骤包含在以下的while循环中：

In [30]:
```
while rel_err > thresh:
    count += 1
    # Step 2(a)
    Xapp = low_rank(Xhat, M=1)
    # Step 2(b)
    Xhat[ismiss] = Xapp[ismiss]
    # Step 2(c)
    mss = np.mean(((Xna - Xapp)[~ismiss])**2)
    rel_err = (mssold - mss) / mss0
    mssold = mss
    print("Iteration: {0}, MSS:{1:.3f}, Rel.Err {2:.2e}"
          .format(count, mss, rel_err))
```

[9] Algorithm 12.1 tells us to iterate Step 2 until (12.14) is no longer decreasing. Determining whether (12.14) is decreasing requires us only to keep track of mssold - mss. However, in practice, we keep track of (mssold - mss) / mss0 instead: this makes it so that the number of iterations required for Algorithm 12.1 to converge does not depend on whether we multiplied the raw data $\mathbf{X}$ by a constant factor.

```
Iteration: 1, MSS:0.395, Rel.Err 5.99e-01
Iteration: 2, MSS:0.382, Rel.Err 1.33e-02
Iteration: 3, MSS:0.381, Rel.Err 1.44e-03
Iteration: 4, MSS:0.381, Rel.Err 1.79e-04
Iteration: 5, MSS:0.381, Rel.Err 2.58e-05
Iteration: 6, MSS:0.381, Rel.Err 4.22e-06
Iteration: 7, MSS:0.381, Rel.Err 7.65e-07
Iteration: 8, MSS:0.381, Rel.Err 1.48e-07
Iteration: 9, MSS:0.381, Rel.Err 2.95e-08
```

我们看到，经过八次迭代后，相对误差已降至`thresh = 1e-7`以下，因此算法终止。此时，非缺失元素的均方误差等于0.381。

最后，我们计算 20 个插补值与实际值之间的相关性：

In [31]:
```
np.corrcoef(Xapp[ismiss], X[ismiss])[0,1]
```

Out[31]: 0.711

在本实验中，我们为了教学目的自行实现了算法12.1。然而，希望将矩阵补全应用于其数据的读者，可能会寻求更专业的Python实现。

12.5.3 Clustering

K-Means 聚类

估计器 `sklearn.cluster.KMeans()` 在 Python. 我们从一个简单的模拟示例开始，其中数据确实存在两个聚类：前25个观测值相对于接下来的25个观测值有一个均值偏移。

`Kmeans()`

In [32]:
```
np.random.seed(0);
X = np.random.standard_normal((50,2));
X[:25,0] += 3;
X[:25,1] -= 4;
```

我们现在使用 $K = 2$ 执行 K 均值聚类。

In [33]:
```
kmeans = KMeans(n_clusters=2,
                random_state=2,
                n_init=20).fit(X)
```

我们指定 `random_state` 以使结果可复现。50 个观测的聚类分配包含在 `kmeans.labels_` 中。

In [34]:
```
kmeans.labels_
```

Out[34]:
```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int32)
```

K-均值聚类完美地将观测值分成两组，即使我们没有向`KMeans()`提供任何分组信息。我们可以绘制数据，根据每个观测值的聚类分配来着色。

In [35]:
```
fig, ax = plt.subplots(1, 1, figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=kmeans.labels_)
ax.set_title("K-Means Clustering Results with K=2");
```

这里可以很容易地绘制观测值，因为它们是二维的。如果有两个以上的变量，那么我们可以转而进行PCA，并绘制前两个主成分得分向量来表示聚类。

在这个例子中，我们知道确实有两个聚类，因为数据是我们生成的。然而，对于真实数据，我们既不知道真实的聚类数量，也不知道它们是否以任何精确的方式存在。相反，我们本可以在此例中执行K-均值聚类，设定$K = $为3。

In [36]:
```
kmeans = KMeans(n_clusters=3,
                random_state=3,
                n_init=20).fit(X)
fig, ax = plt.subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=kmeans.labels_)
ax.set_title("K-Means Clustering Results with K=3");
```

当 $K = 3$ 时，K 均值聚类将两个聚类分开。我们使用了 `n_init` 参数以 20 次初始聚类分配运行 K 均值聚类（默认值为 10）。如果使用大于 1 的 `n_init` 值，则将在算法 12.2 的第 1 步中使用多个随机分配执行 K 均值聚类，并且 `KMeans()` 函数将仅报告最佳结果。这里我们比较使用 `n_init=1` 与 `n_init=20`。

In [37]:
```
kmeans1 = KMeans(n_clusters=3,
                 random_state=3,
                 n_init=1).fit(X)
kmeans20 = KMeans(n_clusters=3,
                  random_state=3,
                  n_init=20).fit(X);
kmeans1.inertia_, kmeans20.inertia_
```

Out[37]:
```
(78.06, 75.04)
```

注意，`kmeans.inertia_` 是簇内平方和的总和，我们通过执行 K-均值聚类 (12.17) 来寻求其最小化。

我们*strongly*建议始终使用较大的K均值聚类值`n_init`，例如20或50，否则可能会获得不理想的局部最优解。

在执行 K-均值聚类时，除了使用多个初始聚类分配外，通过 `random_state` 参数设置随机种子到 `KMeans()` 也很重要。这样，步骤1中的初始聚类分配可以复现，K-均值输出将完全可重复。

层次聚类

来自 `sklearn.clustering` 包的 `AgglomerativeClustering()` 类-age 实现了层次聚类。由于其名称较长，我们使用简写 `HClust` 来表示 *hierarchical clustering*。请注意，这不会改变

`AgglomerativeClustering()`

使用此方法时的返回类型，因此实例仍将是类 AgglomerativeClustering 的。在以下示例中，我们使用前一个实验的数据，以欧几里得距离作为不相似度度量，采用完全连接、单连接和平均连接聚类绘制层次聚类树状图。我们首先使用完全连接对观测值进行聚类。

In [38]:
```
HClust = AgglomerativeClustering
hc_comp = HClust(distance_threshold=0,
                 n_clusters=None,
                 linkage='complete')
hc_comp.fit(X)
```

这会计算整个树状图。我们同样可以轻松地改用平均连接或单连接进行层次聚类：

In [39]:
```
hc_avg = HClust(distance_threshold=0,
                n_clusters=None,
                linkage='average');
hc_avg.fit(X)
hc_sing = HClust(distance_threshold=0,
                 n_clusters=None,
                 linkage='single');
hc_sing.fit(X);
```

要使用预计算的距离矩阵，我们提供了一个额外的参数 metric="precomputed"。在下面的代码中，前四行计算了50×50 的成对距离矩阵。

In [40]:
```
D = np.zeros((X.shape[0], X.shape[0]));
for i in range(X.shape[0]):
    x_ = np.multiply.outer(np.ones(X.shape[0]), X[i])
    D[i] = np.sqrt(np.sum((X - x_)**2, 1));
hc_sing_pre = HClust(distance_threshold=0,
                     n_clusters=None,
                     metric='precomputed',
                     linkage='single')
hc_sing_pre.fit(D)
```

我们使用来自scipy.cluster.hierarchy的dendrogram()来绘制树状图。克。然而，dendrogram()期望的是聚类的所谓*linkage-matrix representation*，这并非由AgglomerativeClustering()提供，但可以计算得出。ISLP.cluster中的函数compute_linkage()为此目的提供了包。

dendrogram()

compute_linkage()

ISLP.cluster

现在我们可以绘制树状图。图底部的数字标识了每个观测值。dendrogram() 函数有一种默认方法来为树的不同分支着色，这暗示了在特定深度对树进行预定义的切割。我们倾向于通过将此阈值设置为无穷大来覆盖这一默认行为。由于我们希望许多树状图都采用这种行为，我们将这些值存储在一个字典 cargs 中，并使用 **cargs 表示法将其作为关键字参数传递。

In [41]:
```
cargs = {'color_threshold':-np.inf,
         'above_threshold_color':'black'}
linkage_comp = compute_linkage(hc_comp)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
```

```
dendrogram(linkage_comp,
           ax=ax,
           **cargs);
```

我们可能希望将树中高于和低于切割阈值的分支以不同颜色表示。这可以通过改变 `color_threshold` 来实现。让我们在高度为 4 处切割树，将合并高度大于 4 的连接线着为黑色。

In [42]:
```
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_comp,
           ax=ax,
           color_threshold=4,
           above_threshold_color='black');
```

要确定与树状图给定切割相关联的每个观测的聚类标签，我们可以使用来自 `scipy.cluster.hierarchy` 的 `cut_tree()`：

In [43]:
```
cut_tree(linkage_comp, n_clusters=4).T
```

Out[43]:
```
array([[0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 2, 0, 0, 0, 1, 1, 0, 0, 1,
        0, 0, 2, 0, 2, 2, 3, 2, 3, 3, 3, 3, 2, 3, 3, 3, 3, 2, 3,
        3, 3, 3, 2, 3, 3, 3, 3, 3, 3, 3, 3]])
```

这也可以通过向`HClust()`提供参数`n_clusters`来实现；然而，每次切割都需要重新计算聚类。同样，树也可以通过距离阈值进行切割，向`HClust()`提供参数`distance_threshold`，或向`cut_tree()`提供参数`height`。

In [44]:
```
cut_tree(linkage_comp, height=5)
```

Out[44]:
```
array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 0, 1, 1, 2, 1, 2, 2, 2, 2, 1, 2, 2, 2, 2, 1, 2,
        2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2]])
```

为了在对观测值执行层次聚类之前对变量进行缩放，我们像在PCA示例中一样使用`StandardScaler()`：

In [45]:
```
scaler = StandardScaler()
X_scale = scaler.fit_transform(X)
hc_comp_scale = HClust(distance_threshold=0,
                       n_clusters=None,
                       linkage='complete').fit(X_scale)
linkage_comp_scale = compute_linkage(hc_comp_scale)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_comp_scale, ax=ax, **cargs)
ax.set_title("Hierarchical Clustering with Scaled Features");
```

基于相关性的观测值间距离可用于聚类。两个观测值之间的相关性衡量了它们特征值的相似性。[10] 对于 n 个观测值，$n \times n$ 相关矩阵

[10] Suppose each observation has p features, each a single numerical value. We measure the similarity of two such observations by computing the correlation of these p pairs of numbers.

然后可以将其用作相似性（或亲和力）矩阵，即一减去相关矩阵所得的矩阵就是用于聚类的相异性矩阵。

注意，仅对至少具有三个特征的数据使用相关性才有意义，因为在两个特征上的任何两个观测值之间的绝对相关性始终为一。因此，我们将对一个三维数据集进行聚类。

In [46]:
```
X = np.random.standard_normal((30, 3))
corD = 1 - np.corrcoef(X)
hc_cor = HClust(linkage='complete',
                distance_threshold=0,
                n_clusters=None,
                metric='precomputed')
hc_cor.fit(corD)
linkage_cor = compute_linkage(hc_cor)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_cor, ax=ax, **cargs)
ax.set_title("Complete Linkage with Correlation-Based Dissimilarity
    ");
```

12.5.4 NCI60 Data Example

无监督技术常用于基因组数据分析。特别是PCA和层次聚类是流行的工具。我们在NCI60癌细胞系微阵列数据上说明这些技术，该数据由64个癌细胞系上的6830个基因表达测量值组成。

In [47]:
```
NCI60 = load_data('NCI60')
nci_labs = NCI60['labels']
nci_data = NCI60['data']
```

每个细胞系都标注了癌症类型。在进行主成分分析（PCA）和聚类时，我们并未利用这些癌症类型信息，因为这两种技术属于无监督方法。然而，在完成PCA和聚类后，我们将检查这些癌症类型与无监督技术所得结果的吻合程度。

数据包含64行和6830列。

In [48]:
```
nci_data.shape
```

Out[48]:
```
(64, 6830)
```

我们首先考察细胞系的癌症类型。

In [49]:
```
nci_labs.value_counts()
```

Out[49]:
```
label
NSCLC          9
RENAL          9
MELANOMA       8
BREAST         7
COLON          7
LEUKEMIA       6
OVARIAN        6
```

```
CNS             5
PROSTATE        2
K562A-repro     1
K562B-repro     1
MCF7A-repro     1
MCF7D-repro     1
UNKNOWN         1
dtype: int64
```

NCI60数据的主成分分析

我们首先对数据进行主成分分析，在此之前将变量（基因）标准化，使其标准差为一，尽管这里有人可能会合理争论，由于基因是以相同的单位测量，最好不对其进行标准化。

In [50]:
```
scaler = StandardScaler()
nci_scaled = scaler.fit_transform(nci_data)
nci_pca = PCA()
nci_scores = nci_pca.fit_transform(nci_scaled)
```

我们现在绘制前几个主成分得分向量，以便可视化数据。对应于给定癌症类型的观测（细胞系）将以相同的颜色绘制，这样我们就可以看到一个癌症类型内的观测在多大程度上彼此相似。

In [51]:
```
cancer_types = list(np.unique(nci_labs))
nci_groups = np.array([cancer_types.index(lab)
                       for lab in nci_labs.values])
fig, axes = plt.subplots(1, 2, figsize=(15,6))
ax = axes[0]
ax.scatter(nci_scores[:,0],
           nci_scores[:,1],
           c=nci_groups,
           marker='o',
           s=50)
ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
ax = axes[1]
ax.scatter(nci_scores[:,0],
           nci_scores[:,2],
           c=nci_groups,
           marker='o',
           s=50)
ax.set_xlabel('PC1'); ax.set_ylabel('PC3');
```

得到的图如图12.17所示。总体而言，对应于单一癌症类型的细胞系在最初几个主成分得分向量上确实倾向于具有相似的值。这表明来自同一癌症类型的细胞系往往具有相当相似的基因表达水平。

我们还可以绘制主成分解释的百分比方差以及累积百分比方差。这类似于我们之前为 `USArrests` 数据绘制的图表。

In [52]:
```
fig, axes = plt.subplots(1, 2, figsize=(15,6))
ax = axes[0]
ticks = np.arange(nci_pca.n_components_)+1
```

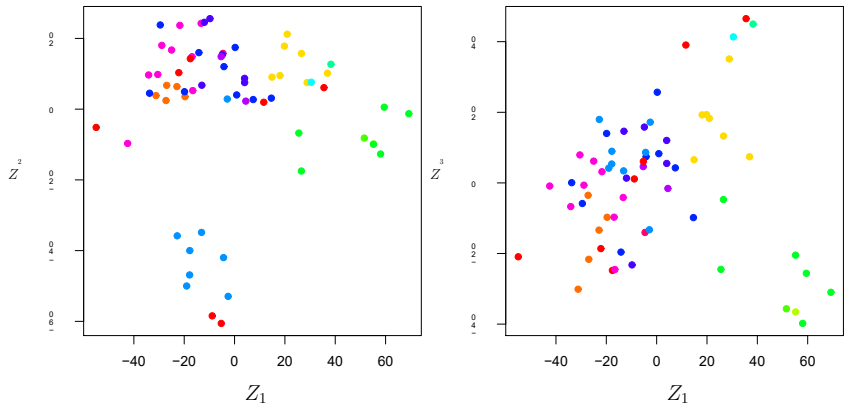


图12.17。 *Projections of the* NCI60 *cancer cell lines onto the first three principal components (in other words, the scores for the first three principal components). On the whole, observations belonging to a single cancer type tend to lie near each other in this low-dimensional space. It would not have been possible to visualize the data without using a dimension reduction method such as PCA, since based on the full data set there are* $\binom{6,830}{2}$ *possible scatterplots, none of which would have been particularly informative.*

```
ax.plot(ticks,
        nci_pca.explained_variance_ratio_,
        marker='o')
ax.set_xlabel('Principal Component');
ax.set_ylabel('PVE')
ax = axes[1]
ax.plot(ticks,
        nci_pca.explained_variance_ratio_.cumsum(),
        marker='o');
ax.set_xlabel('Principal Component')
ax.set_ylabel('Cumulative PVE');
```

生成的图表如图12.18所示。

我们看到，前七个主成分合起来解释了数据中大约40%的方差。这并不是一个很大的方差比例。然而，观察碎石图，我们发现虽然前七个主成分各自解释了相当数量的方差，但后续主成分所解释的方差显著减少。也就是说，在第七个主成分之后，图中出现了一个*elbow*。这表明检查超过七个左右的主成分可能收益甚微（尽管即使是检查七个主成分也可能很困难）。

对NCI60数据观测的聚类

我们现在对 NCI60 数据中的细胞系进行层次聚类，使用完全连接、单连接和平均连接方法。再次强调，目标在于探究观察结果是否聚集成不同的癌症类型。欧几里得距离被用作不相似度的度量。我们首先编写一个简短函数来生成这三个树状图。

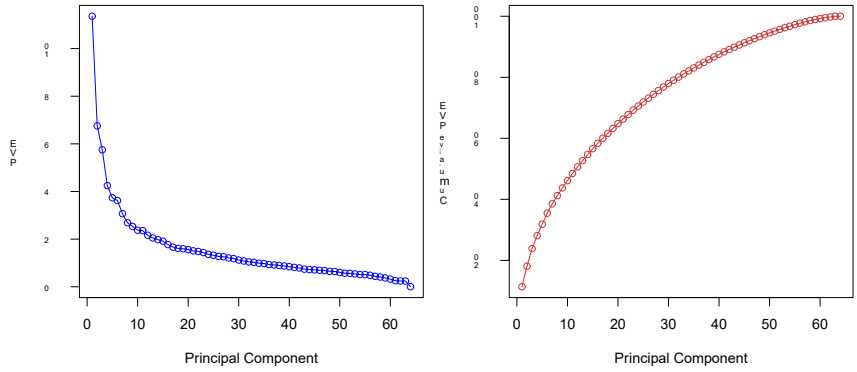


图12.18。 *The PVE of the principal components of the* `NCI60` *cancer cell line microarray data set. 左: the PVE of each principal component is shown. 右: the cumulative PVE of the principal components is shown. Together, all principal components explain 100,% of the variance.*

In [53]:
```
def plot_nci(linkage, ax, cut=-np.inf):
    cargs = {'above_threshold_color':'black',
             'color_threshold':cut}
    hc = HClust(n_clusters=None,
                distance_threshold=0,
                linkage=linkage.lower()).fit(nci_scaled)
    linkage_ = compute_linkage(hc)
    dendrogram(linkage_,
               ax=ax,
               labels=np.asarray(nci_labs),
               leaf_font_size=10,
               **cargs)
    ax.set_title('%s Linkage' % linkage)
    return hc
```

Let's plot our results.

In [54]:
```
fig, axes = plt.subplots(3, 1, figsize=(15,30))
ax = axes[0]; hc_comp = plot_nci('Complete', ax)
ax = axes[1]; hc_avg = plot_nci('Average', ax)
ax = axes[2]; hc_sing = plot_nci('Single', ax)
```

结果如图12.19所示。我们可以看到，连接方式的选择确实会影响所得到的结果。通常，单连接倾向于产生*trailing*聚类：非常大的聚类，单个观测值一个接一个地附加上去。另一方面，完全连接和平均连接倾向于产生更平衡、更有吸引力的聚类。因此，完全连接和平均连接通常比单连接更受青睐。显然，同一癌症类型内的细胞系确实倾向于聚集在一起，尽管聚类并不完美。在接下来的分析中，我们将使用完全连接层次聚类。

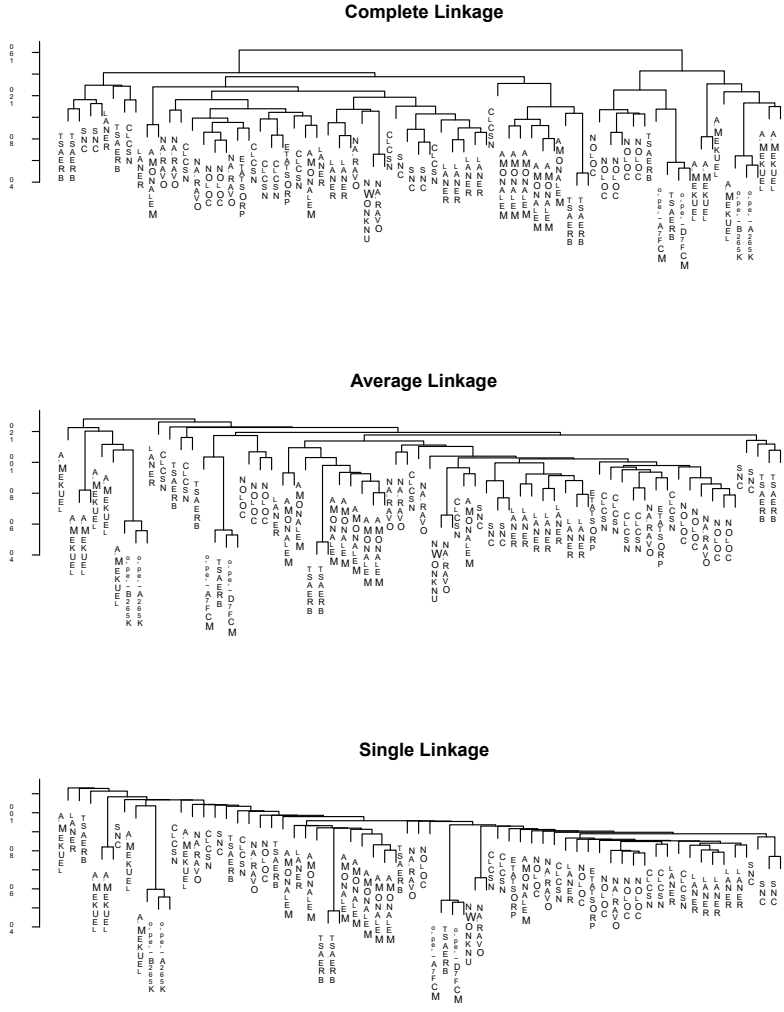


图 12.19. *The* NCI60 *cancer cell line microarray data, clustered with average, complete, and single linkage, and using Euclidean distance as the dissimilarity measure. Complete and average linkage tend to yield evenly sized clusters whereas single linkage tends to yield extended clusters to which single leaves are fused one by one.*

我们可以在某个高度切割树状图，以获得特定数量的簇，比如四个：

In [55]:
```
linkage_comp = compute_linkage(hc_comp)
comp_cut = cut_tree(linkage_comp, n_clusters=4).reshape(-1)
pd.crosstab(nci_labs['label'],
            pd.Series(comp_cut.reshape(-1), name='Complete'))
```

有一些明显的模式。所有白血病细胞系聚集在一个簇中，而乳腺癌细胞系则分布在三个不同的簇中。

我们可以在树状图上绘制一条切线，从而得到这四个聚类：

In [56]:
```
fig, ax = plt.subplots(figsize=(10,10))
plot_nci('Complete', ax, cut=140)
ax.axhline(140, c='r', linewidth=4);
```

`axhline()` 函数在任何现有坐标轴集合上绘制一条水平线。参数 `140` 在树状图上高度 140 处绘制一条水平线；这个高度会导致四个不同的聚类。很容易验证，得到的聚类与我们在 `comp_cut` 中得到的相同。

我们在第12.4.2节早些时候声称，K-均值聚类和通过树状图切割获得相同数量簇的层次聚类可能会产生非常不同的结果。如果将K-均值聚类执行$K=$次，这些NCI60层次聚类的结果与我们所得到的结果相比如何?

In [57]:
```
nci_kmeans = KMeans(n_clusters=4,
                    random_state=0,
                    n_init=20).fit(nci_scaled)
pd.crosstab(pd.Series(comp_cut, name='HClust'),
            pd.Series(nci_kmeans.labels_, name='K-means'))
```

Out[57]:

K-means	0	1	2	3
HClust				
0	28	3	9	0
1	7	0	0	0
2	0	0	0	8
3	0	9	0	0

我们看到，使用层次聚类和 K-均值聚类得到的四个簇有所不同。首先我们注意到，两种聚类中的标签是任意的。也就是说，交换簇的标识符不会改变聚类结果。我们在这里看到，K-均值聚类中的簇3与层次聚类中的簇2相同。然而，其他簇有所不同：例如，K-均值聚类中的簇0包含了层次聚类分配给簇0的部分观测值，以及层次聚类分配给簇1的所有观测值。

我们也可以对前几个主成分得分向量进行层次聚类，而不是对整个数据矩阵进行层次聚类，将这些前几个成分视为数据的噪声较少的版本。

In [58]:
```
hc_pca = HClust(n_clusters=None,
                distance_threshold=0,
                linkage='complete'
                ).fit(nci_scores[:,:5])
linkage_pca = compute_linkage(hc_pca)
fig, ax = plt.subplots(figsize=(8,8))
dendrogram(linkage_pca,
           labels=np.asarray(nci_labs),
           leaf_font_size=10,
           ax=ax,
           **cargs)
ax.set_title("Hier. Clust. on First Five Score Vectors")
pca_labels = pd.Series(cut_tree(linkage_pca,
                                n_clusters=4).reshape(-1),
                       name='Complete-PCA')
pd.crosstab(nci_labs['label'], pca_labels)
```

12.6 练习

Conceptual

1. 这个问题涉及到 K-均值聚类算法。



(a) 证明(12.18)。(b) 基于此恒等式，论证K均值聚类算法（算法12.2）在每次迭代中都会减少目标函数(12.17)。

2. 假设我们有四个观测值，我们据此计算得到一个相异度矩阵，如下所示：

$$\begin{bmatrix} & 0.3 & 0.4 & 0.7 \\ 0.3 & & 0.5 & 0.8 \\ 0.4 & 0.5 & & 0.45 \\ 0.7 & 0.8 & 0.45 & \end{bmatrix}.$$

例如，第一个和第二个观测之间的相异度为0.3，第二个和第四个观测之间的相异度为0.8。

(a) 基于这个不相似矩阵，勾画出使用完全连接法对这四个观测进行层次聚类所得到的树状图。确保在图上标出每次融合发生的高度，以及树状图中每个叶节点对应的观测。(b) 重复(a)，这次使用单连接聚类。(c) 假设我们切割(a)中得到的树状图，使得产生两个聚类。每个聚类中包含哪些观测？(d) 假设我们切割(b)中得到的树状图，使得产生两个聚类。每个聚类中包含哪些观测？

(e) 本章提到，在树状图的每次融合中，被融合的两个聚类的位置可以交换，而不会改变树状图的含义。画出一个与（a）中树状图等价的树状图，其中有两个或更多的叶节点被重新定位，但树状图的含义相同。

3. 在这个问题中，您将手动执行 K-均值聚类，其中 $K = 2$，在具有 $n = 6$ 个观测值和 $p = 2$ 个特征的小型示例上。观测值如下。

Obs.	X_1	X_2
1	1	4
2	1	3
3	0	4
4	5	1
5	6	2
6	4	0

((a)) 绘制观测数据。(b) 为每个观测随机分配一个聚类标签。你可以使用`np.random.choice()`函数来完成此操作。报告每个观测的聚类标签。(c) 计算每个聚类的质心。(d) 根据欧几里得距离，将每个观测分配到最近的质心。报告每个观测的聚类标签。(e) 重复(c)和(d)，直到得到的结果不再变化。(f) 在(a)的图中，根据获得的聚类标签为观测数据着色。

4. 假设对于特定数据集，我们使用单链和全链进行层次聚类。我们得到两个树状图。

(a) 在单链接聚类图的某一点上，簇 {1, 2, 3} 和 {4, 5} 融合。在完全链接聚类图上，簇 {1, 2, 3} 和 {4, 5} 也在某一点上融合。哪种融合会在树上出现得更高，还是它们会在相同高度融合，还是没有足够的信息来判断？(b) 在单链接聚类图的某一点上，簇 {5} 和 {6} 融合。在完全链接聚类图上，簇 {5} 和 {6} 也在某一点上融合。哪种融合会在树上出现得更高，还是它们会在相同高度融合，还是没有足够的信息来判断？

5. 用文字描述，如果基于袜子和电脑购买情况对图12.16中的八位购物者进行K-均值聚类，且$K =$为2，你预期会得到什么结果。给出三个答案，分别对应所显示的每一种变量缩放方式。并解释。

6. 我们在12.2.2节看到，主成分载荷和得分向量提供了对矩阵的近似，如(12.5)所示。具体来说，主成分得分和载荷向量解决了(12.6)中给出的优化问题。

现在，假设M主成分得分向量$z_{im},\ m = 1, \ldots, M$是已知的。利用(12.6)，解释前$M$个主成分载荷向量$\phi_{jm},\ m = 1, \ldots, M$中的每一个都可以通过执行$p$个独立的最小二乘线性回归来获得。在每次回归中，主成分得分向量是预测变量，而数据矩阵的其中一个特征是响应变量。

Applied

7. 在本章中，我们提到了使用基于相关性的距离和欧几里得距离作为层次聚类的相异度度量。事实证明，这两种度量几乎等价：如果每个观测值都已中心化，使其均值为零且标准差为一，并且如果我们用 r_{ij} 表示第 i 个和第 j 个观测值之间的相关性，那么 $1 - r_{ij}$ 这个量与第 i 个和第 j 个观测值之间的欧几里得距离的平方成正比。

在`USArrests`数据上，展示这种比例关系成立。

Hint: The Euclidean distance can be calculated using the `pairwise_distances()` *function from the* `sklearn.metrics` *module, and correlations can be calculated using the* `np.corrcoef()` *function.*

`pairwise_distances()`

8. 在第12.2.3节中，方程12.10给出了计算PVE的公式。我们还了解到，可以使用拟合的`PCA()`估计器的`explained_variance_ratio_`属性来获得PVE。

在 `USArrests` 数据上，通过两种方式计算 PVE：

(a) 使用拟合的 `PCA()` 估计器的 `explained_variance_ratio_` 输出，如同第12.2.3节所做的那样。(b) 直接应用方程12.10。载荷作为拟合的 `PCA()` 估计器的 `components_` 属性存储。使用这些载荷在方程12.10中以获得PVE。

这两种方法应该给出相同的结果。

Hint: You will only obtain the same results in (a) and (b) if the same data is used in both cases. For instance, if in (a) you performed `PCA()` *using centered and scaled variables, then you must center and scale the variables before applying Equation 12.10 in (b).*

9. 考虑 `USArrests` 数据。我们现在将对状态进行层次聚类。

(a) 使用完全链接和欧氏距离的层次聚类对州进行聚类。(b) 在树状图中切割出一个高度，使得形成三个不同的聚类。哪些州属于哪个聚类?

(c) 对状态使用完全连接和欧氏距离进行层次聚类,
after scaling the variables to have standard de- viation one。

(d) 变量缩放对所得层次聚类有何影响? 您认为在计算观测间的不相似度之前, 是否应该对变量进行缩放? 请为您的答案提供理由。

10. 在这个问题中, 你将生成模拟数据, 然后对数据执行PCA和K-均值聚类。

(a) 生成一个模拟数据集, 其中三个类别各包含20个观测值（即总共60个观测值）, 以及50个变量。
Hint: There are a number of functions in `Python` *that you can use to generate data. One example is the* `normal()` *method of the* `random()` *function in* `numpy`*; the* `uniform()` *method is another option. Be sure to add a mean shift to the observations in each class so that there are three distinct classes.*

(b) 对60个观测进行PCA, 并绘制前两个主成分得分向量。使用不同的颜色来区分三个类别中的观测。如果在这个图中三个类别显得分离, 则继续到部分(c)。如果不是, 则返回到部分(a)并修改模拟, 以便三个类别之间有更大的分离。直到三个类别在前两个主成分得分向量中显示出至少一些分离, 才继续到部分(c)。

(c) 对观测数据进行 K 均值聚类, 其中 $K = 3$。你在 K 均值聚类中得到的聚类与真实类标签相比如何?

Hint: You can use the `pd.crosstab()` *function in* `Python` *to compare the true class labels to the class labels obtained by clustering. Be careful how you interpret the results: K-means clustering will arbitrarily number the clusters, so you cannot simply check whether the true class labels and clustering labels are the same.*

(对)执行K-均值聚类, 其中$K =$为2。描述你的结果。

(e) 现在使用 $K = 4$ 进行 K 均值聚类, 并描述你的结果。

(f) 现在对前两个主成分得分向量执行 K-均值聚类, 其中 $K = 3$, 而不是对原始数据进行聚类。也就是说, 对60×2矩阵执行 K-均值聚类, 其中第一列是第一主成分得分向量, 第二列是第二主成分得分向量。评论结果。

(g) 使用 `StandardScaler()` 估计器, 对数据
after scaling each variable to have standard deviation one 进行 K 均值聚类, 其中 $K =$ 为 3。这些结果与 (b) 中得到的结果相比如何? 请解释。

11. 编写一个 Python 函数，按照算法12.1以及第12.5.2节所述执行矩阵补全。在每次迭代中，该函数应记录相对误差及迭代次数。迭代应持续到相对误差足够小或达到预设的最大迭代次数（为此最大次数设定一个默认值）。此外，应提供选项以打印每次迭代的进度。

在Boston数据上测试你的函数。首先使用StandardScaler()函数将特征标准化为均值为零、标准差为一。运行一个实验，其中你随机剔除从5%到30%、步长为5%的递增（且嵌套）数量的观测值。应用算法12.1，参数为$M = 1, 2, \ldots, 8$。显示近似误差作为缺失观测值比例的函数，以及M的值，并在10次实验重复中取平均。

12. 在12.5.2节中，算法12.1是通过np.linalg模块中的svd()函数实现的。然而，鉴于在实验中强调的svd()函数与PCA()估计量之间的联系，我们本可以改用PCA()来实现该算法。

编写一个函数来实现算法12.1，该算法利用PCA()而非svd()。

13. 在书籍网站www.statlearning.com上，有一个基因表达数据集（Ch12Ex13.csv），包含40个组织样本，对1,000个基因进行了测量。前20个样本来自健康患者，而后20个则来自患病组。

(a) 使用pd.read_csv()加载数据。你需要选择header = None。(b) 使用基于相关性的距离对样本进行层次聚类，并绘制树状图。基因是否将样本分成两组？你的结果是否取决于所使用的链接类型？(c) 你的合作者想知道哪些基因在两组之间差异最大。建议一种回答这个问题的方法，并在此应用。

13
多重检验



到目前为止，本教科书主要聚焦于*estimation*及其近亲*prediction*。在本章中，我们将转而关注假设检验，这对于进行*inference*至关重要。我们提醒读者，推理在第二章中已有简要讨论。

虽然第13.1节简要回顾了零假设、p值、检验统计量以及假设检验中的其他关键概念，但本章假设读者之前已经接触过这些主题。特别地，我们将不会聚焦于使用*why*或*how*来进行假设检验——这是一个可以（并且已经）写成整本书的主题！相反，我们假设读者有兴趣检验某一组特定的零假设，并且心中已有进行检验并获取p值的具体计划。

经典统计学中的许多重点都集中在检验单个原假设上，例如
H_0*: the expected blood pressure of mice in the control group equals the expected blood pressure of mice in the treatment group*。当然，我们可能希望发现两组平均血压之间*is*差异。但出于即将明了的原因，我们构建了一个对应于无差异的原假设。

在当代环境中，我们经常面临海量数据，因此可能希望检验大量零假设。例如，我们可能不仅想检验H_0，还想检验m个零假设，$H_{01}, \ldots, H_{0m}$，其中H_{0j}*: the expected value of the* j^{th} *biomarker among mice in the control group equals the expected value of the* j^{th} *biomarker among mice in the treatment group*。在进行*multiple testing*时，我们需要非常谨慎地解释结果，以避免错误地拒绝过多的零假设。

本章讨论了在大数据环境下进行多重检验的经典方法以及更为现代的方法。在13.2节中，我们强调了与多重检验相关的挑战。这些的经典解决方案

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0_13

第13.3节介绍了挑战，而更现代的解决方案则在第13.4节和第13.5节中。

特别是，第13.4节聚焦于错误发现率。错误发现率的概念可以追溯到20世纪90年代。在21世纪初，随着基因组学领域大规模数据集的涌现，这一概念迅速走红。这些数据集之所以独特，不仅因为其规模庞大，[1]还因为它们通常是为特定目的而收集的，*exploratory*：研究人员收集这些数据集是为了检验大量零假设，而非仅仅少数预先设定的零假设。时至今日，几乎在所有领域，庞大的数据集都是在没有预先设定零假设的情况下收集的。正如我们将看到的，错误发现率完美契合了这一现代现实。

本章自然聚焦于经典的统计技术——p值，用于量化假设检验的结果。在本书撰写之时（2020年），p值近期已成为社会科学研究界广泛评论的话题，以至于一些社会科学期刊甚至完全禁止使用p值！我们仅需说明，当被正确理解和应用时，p值为我们从数据中得出推断性结论提供了强大的工具。

13.1 假设检验快速回顾

假设检验提供了一个严格的统计框架，用于回答关于数据的简单“是或否”问题，例如：

1. 在线性回归中，Y对$X_1, \ldots, X_p$的真实系数β_j是否等于零？[2] 2. 对照组实验室小鼠与治疗组实验室小鼠的预期血压是否存在差异？[3]

在13.1.1节中，我们简要回顾了假设检验的步骤。13.1.2节讨论了假设检验中可能出现的不同类型的错误或误差。

13.1.1 Testing a Hypothesis

进行假设检验通常包括四个步骤。首先，我们定义零假设和备择假设。接下来，我们构建一个检验统计量，以总结反对零假设的证据强度。然后，我们计算一个p值，该值量化了获得

[1] Microarray data was viewed as “big data” at the time, although by today’s standards, this label seems quaint: a microarray dataset can be (and typically was) stored in a Microsoft Excel spreadsheet!

[2] This hypothesis test was discussed on page 76 of Chapter 3.

[3] The “treatment group” refers to the set of mice that receive an experimental treatment, and the “control group” refers to those that do not.

在零假设下，检验统计量的一个可比或更极端的值。最后，依据p值，我们决定是否拒绝零假设。接下来，我们将依次简要讨论这些步骤。

步骤1：定义零假设和备择假设

在假设检验中，我们将世界分为两种可能性：*null hypothesis*和*alternative hypothesis*。零假设，记为H_0，是对世界的信念的默认状态。[4] 例如，与本章前面提出的两个问题相关的零假设如下：

null hypothesis

alternative hypothesis

1. 线性回归中Y对$X_1, \ldots, X_p$的真实系数β_j等于零。2. 对照组和治疗组小鼠的预期血压没有差异。

零假设在构造上就显得乏味：它或许确实为真，但我们可能希望数据能告诉我们并非如此。

备择假设，记为 H_a，代表某种不同且出乎意料的情况：例如，两组小鼠的预期血压之间存在差异。通常，备择假设只是简单地假定原假设不成立：如果原假设陈述的是 *there is no difference between A and B*，那么备择假设陈述的则是 *there is a difference between A and B*。

需要注意的是，H_0 和 H_a 的处理是不对称的。H_0 被视为世界的默认状态，我们专注于使用数据来拒绝 H_0。如果我们拒绝 H_0，那么这提供了支持 H_a 的证据。我们可以将拒绝 H_0 视为对我们的数据做出一个 *discovery*：即，我们发现 H_0 不成立！相比之下，如果我们未能拒绝 H_0，那么我们的发现就更加模糊：我们将不知道我们未能拒绝 H_0 是因为我们的样本量太小（在这种情况下，在更大或更高质量的数据集上再次测试 H_0 可能会导致拒绝），还是因为我们未能拒绝 H_0 是因为 H_0 确实成立。

步骤 2：构造检验统计量

接下来，我们希望利用数据来寻找支持或反对零假设的证据。为此，我们必须计算一个 *test statistic*，记为 T，它概括了我们的数据与 H_0 一致的程度。我们构建 T 的方式取决于我们所检验的原假设的性质。

检验统计量

具体来说，令 $x_1^t, \ldots, x_{n_t}^t$ 表示治疗组中 n_t 只小鼠的血压测量值，令 $x_1^c, \ldots, x_{n_c}^c$ 表示对照组中 n_c 只小鼠的血压测量值，且 $\mu_t = \mathrm{E}(X^t)$，$\mu_c = \mathrm{E}(X^c)$。为检验 H_0：$\mu_t = \mu_c$，我们利用一个定义为如下的 *two-sample t-statistic*，[5]

two-sample t-statistic

[4] H_0 is pronounced “H naught” or “H zero”.

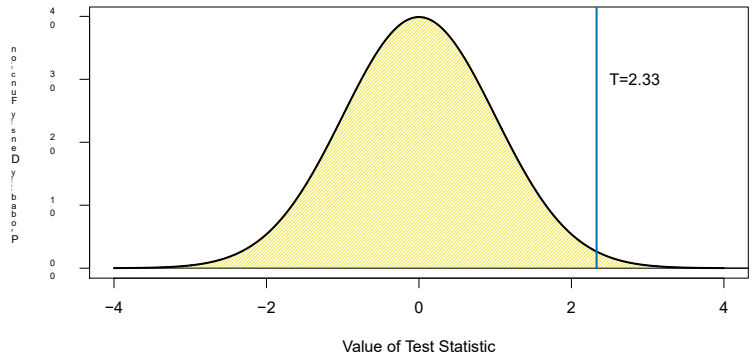


图13.1。*The density function for the* $N(0,1)$ *distribution, with the vertical line indicating a value of* 2.33. *1% of the area under the curve falls to the right of the vertical line, so there is only a 2% chance of observing a* $N(0,1)$ *value that is greater than* 2.33 *or less than* -2.33. *Therefore, if a test statistic has a* $N(0,1)$ *null distribution, then an observed test statistic of* $T = 2.33$ *leads to a p-value of 0.02.*

$$T = \frac{\hat{\mu}_t - \hat{\mu}_c}{s\sqrt{\frac{1}{n_t} + \frac{1}{n_c}}} \tag{13.1}$$

其中 $\hat{\mu}_t = \frac{1}{n_t}\sum_{i=1}^{n_t} x_i^t$、$\hat{\mu}_c = \frac{1}{n_c}\sum_{i=1}^{n_c} x_i^c$ 以及

$$s = \sqrt{\frac{(n_t - 1)s_t^2 + (n_c - 1)s_c^2}{n_t + n_c - 2}} \tag{13.2}$$

是两个样本合并标准差的估计量。[6] 这里，s_t^2 和 s_c^2 分别是治疗组和对照组血压方差的无偏估计量。T的较大（绝对值）值为 H_0： $\mu_t = \mu_c$ 提供了反对证据，因此也为 H_a： $\mu_t \neq \mu_c$ 提供了支持证据。

步骤 3：计算 p 值

在上一节中，我们指出，两样本t统计量的绝对值较大时，提供了反对H_0的证据。这就引出了一个问题：*how large is large?* 换言之，给定的检验统计量值提供了多少反对H_0的证据?

p-value 的概念为我们提供了一种形式化以及回答这个问题。p值定义为观察到等于或比观察统计量更极端的检验统计量的概率，*under the assumption that* H_0 *is in fact true*。因此，小的p值提供了*against* H_0的证据。

p-value

[5]The t-statistic derives its name from the fact that, under H_0, it follows a t-distribution.

[6]Note that (13.2) assumes that the control and treatment groups have equal variance. Without this assumption, (13.2) would take a slightly different form.

具体来说，假设检验统计量（13.1）中的 $T = 2.33$。那么，我们可以问：如果 H_0 确实成立，观察到 T 如此大的值的概率是多少？结果表明，在 H_0 下，（13.1）中 T 的分布近似服从 $N(0, 1)$ 分布[7]——即均值为0、方差为1的正态分布。该分布如图13.1所示。我们看到，$N(0, 1)$ 分布的绝大部分——98%——落在 -2.33 和 2.33 之间。这意味着在 H_0 下，我们预期只有2%的时间会看到 $|T|$ 如此大的值。因此，对应于 $T = 2.33$ 的 p 值为 0.02。

在H_0 (下检验统计量的分布，也称为检验统计量的*null distribution*)，将取决于具体类型是什么
正在检验零假设，以及所使用的检验统计量类型。通常，最常用的检验统计量在零假设下遵循一个著名的统计分布——例如正态分布、t分布、χ^2分布或F分布——前提是样本量足够大，并且其他一些假设成立。通常，用于计算检验统计量的R函数会利用这个零分布来输出一个p值。在13.5节中，我们将看到一种使用重采样来估计检验统计量零分布的方法；在许多当代环境中，这是一个非常有吸引力的选择，因为它利用了快速计算机的可用性，以避免对数据做出可能有问题假设。

null distribution

p 值也许是整个统计学中最常被使用和滥用的概念之一。特别是，有时人们会说 p 值是 H_0 成立的概率，即原假设为真的概率。这是不正确的！p 值唯一正确的解释是，如果我们把实验重复很多很多次，我们预期看到检验统计量 [8] 如此极端值的次数所占的比例，*provided* H_0 *holds*。

在第2步中，我们计算了一个检验统计量，并注意到统计量的较大（绝对值）值提供了反对H_0的证据。在第3步中，检验统计量被转换为p值，较小的p值提供了反对H_0的证据。那么，我们在第3步中将第2步的检验统计量转换为p值，究竟实现了什么呢？为了回答这个问题，假设一位数据分析师进行了一项统计检验，并报告了检验统计量$T = 17.3$。这是否提供了反对H_0的强有力证据？没有更多信息，这是无法知道的：特别是，我们需要知道

[7] More precisely, assuming that the observations are drawn from a normal distribution, then T follows a t-distribution with $n_t + n_c - 2$ degrees of freedom. Provided that $n_t + n_c - 2$ is larger than around 40, this is very well-approximated by a $N(0, 1)$ distribution. In Section 13.5, we will see an alternative and often more attractive way to approximate the null distribution of T, which avoids making stringent assumptions about the data.

[8] A *one-sided* p-value is the probability of seeing such an extreme value of the test statistic; e.g. the probability of seeing a test statistic greater than or equal to $T = 2.33$. A *two-sided* p-value is the probability of seeing such an extreme value of the *absolute* test statistic; e.g. the probability of seeing a test statistic greater than or equal to 2.33 or less than or equal to -2.33. The default recommendation is to report a two-sided p-value rather than a one-sided p-value, unless there is a clear and compelling reason that only one direction of the test statistic is of scientific interest.

		Truth	
		H_0	H_a
Decision	Reject H_0	Type I Error	Correct
	Do Not Reject H_0	Correct	Type II Error

表 13.1. *A summary of the possible scenarios associated with testing the null hypothesis H_0. Type I errors are also known as false positives, and Type II errors as false negatives.*

在H_0下，检验统计量的期望值应该是多少。这正是p值所给出的。换句话说，p值使我们能够将按某种任意且难以解释的尺度衡量的检验统计量转换为一个介于0和1之间、更易于解释的数值。

第4步：决定是否拒绝零假设

一旦我们计算出了对应于 H_0 的 p 值，接下来就需要决定是否拒绝 H_0。（我们通常不说“接受” H_0：相反，我们说的是“未能拒绝” H_0。）一个较小的 p 值表明，在 H_0 下，如此大的检验统计量值不太可能出现，从而提供了反对 H_0 的证据。如果 p 值足够小，那么我们就想拒绝 H_0（，因此，做出一个“发现”）。但多小才算足够小，以至于可以拒绝 H_0 呢?

事实证明，这个问题的答案很大程度上取决于观察者的角度，更具体地说，是数据分析师的角度。p值越小，反对H_0的证据就越强。在某些领域，如果p值低于0.05，通常会拒绝H_0；这意味着，如果H_0成立，我们预期看到如此小的p值的概率不超过5%。[9]然而，在其他领域，所需的证明标准要高得多：例如，在物理学的某些领域，只有当p值低于10^{-9}时，才会典型地拒绝H_0!

在图13.1所示的例子中，如果我们以0.05作为拒绝零假设的阈值，那么我们将拒绝零假设。相反，如果我们以0.01作为阈值，那么我们将无法拒绝零假设。这些概念将在下一节中正式阐述。

13.1.2 Type I and Type II Errors

如果零假设成立，则我们称其为*true null hypothesis*；否则，它是一个*false null hypothesis*。例如，如果我们测试H_0：$\mu_t = \mu_c$如第13.1.1节所述，如果治疗组小鼠和对照组小鼠的*population*平均血压确实没有差异，那么H_0为真；否则为假。当然，我们不知道*a priori*H_0是真的还是假的：这就是为什么我们需要进行假设检验!

真零假设

错误零假设

[9]Though a threshold of 0.05 to reject H_0 is ubiquitous in some areas of science, we advise against blind adherence to this arbitrary choice. Furthermore, a data analyst should typically report the p-value itself, rather than just whether or not it exceeds a specified threshold value.

表13.1总结了与检验零假设H_0相关的可能情景。一旦执行了假设检验，表中的*row*便是已知的（基于我们是否拒绝了H_0）；然而，我们无法知道自己处于哪一种*column*中。如果当H_0为假（即H_a为真）时我们拒绝了H_0，或者当H_0为真时我们没有拒绝它，那么我们得到了正确的结果。然而，如果当H_0实际上为真时我们错误地拒绝了H_0，那么我们就犯了一个*Type I error*。*Type I error rate* 定义为在给定情况下犯第一类错误的概率H_0成立，即错误拒绝H_0的概率。或者，如果当H_0实际上为假时我们没有拒绝H_0，那么我们就犯了一个*Type II error*。假设检验的*power*定义为概率在H_a成立的情况下不犯第二类错误的概率，即正确拒绝H_0的概率。

Type I error
Type I error rate
Type II error
power

理想情况下，我们希望第一类错误率和第二类错误率都很小。但在实践中，这很难实现！通常存在一种权衡：我们可以通过仅在非常确定H_0不成立时才拒绝它，来使第一类错误变小；然而，这会导致第二类错误增加。或者，我们可以在即使只有适度的证据表明H_0不成立时就拒绝它，从而使第二类错误变小，但这会导致第一类错误变大。在实践中，我们通常认为第一类错误比第二类错误更“严重”，因为前者涉及宣布一个不正确的科学发现。因此，当我们进行假设检验时，通常要求较低的第一类错误率——例如，至多$\alpha = 0.05$——同时尽量使第二类错误变小（或者等价地，使功效变大）。

事实证明，导致我们拒绝 H_0 的 p 值阈值与第一类错误率之间存在直接的对应关系。仅当 p 值低于 α 时拒绝 H_0，我们可以确保第一类错误率将小于或等于 α。

13.2 多重测试的挑战

在上一节中，我们看到，如果p值低于（比如）0.01，则拒绝H_0，这为我们提供了一种简单的方法，将H_0的第一类错误控制在0.01水平：如果H_0为真，那么我们拒绝它的概率不超过1%。但现在假设我们希望检验m个零假设，$H_{01}, \ldots, H_{0m}$。仅仅拒绝所有相应的p值低于（比如）0.01的零假设，这样可以吗？换句话说，如果我们拒绝所有p值低于0.01的零假设，那么我们应该预期犯多少第一类错误？

作为回答这个问题的第一步，设想一位股票经纪人，她希望通过让潜在客户相信她的交易能力来招揽新客户。

[10]There are parallels between Table 13.1 and Table 4.6, which has to do with the output of a binary classifier. In particular, recall from Table 4.6 that a false positive results from predicting a positive (non-null) label when the true label is in fact negative (null). This is closely related to a Type I error, which results from rejecting the null hypothesis when in fact the null hypothesis holds.

敏锐。她告诉1,024（$1{,}024 = 2^{10}$）位潜在新客户，她能准确预测苹果股价连续10天的涨跌。在这10天里，苹果股价的变动有2^{10}种可能。因此，她给每位客户发送这2^{10}种可能中的一种。绝大多数潜在客户会发现，这位股票经纪人的预测并不比随机猜测好（许多人甚至会认为比随机猜测还差）。但停摆的钟一天也有两次是准的，其中一位潜在客户会惊讶地发现，她的预测在这10天里全都正确！于是，这位股票经纪人赢得了一位新客户。

这里发生了什么？这位股票经纪人是否对苹果股价的涨跌有任何实际洞察？没有。那么，她如何能够连续10天完美预测苹果的股价呢？答案是她做了很多猜测，而其中一个恰好完全正确。

这与多重检验有何关系？假设我们抛掷1,024枚公平硬币[11]，每枚各十次。那么平均而言，我们预期会有一枚硬币每次都是反面。（任何单枚硬币连续十次出现反面的概率为$1/2^{10} =$的$1/1{,}024$。因此，如果我们抛掷1,024枚硬币，那么平均而言，我们预期会有一枚硬币全是反面。）如果其中一枚硬币全是反面，我们可能因此断定这枚特定的硬币不公平。实际上，针对这枚特定硬币公平性的零假设进行标准假设检验，会得到一个低于0.002的p值！[12]但得出硬币不公平的结论将是错误的：事实上，零假设成立，我们只是碰巧连续得到了十次反面。

这些例子说明了*multiple testing*的主要挑战：当测试大量零假设时，我们必然会偶然得到一些非常小的p值。如果我们对是否拒绝每个零假设做出决定，而不考虑我们进行了非常大量的检验这一事实，那么我们可能最终会拒绝大量真实的零假设——也就是说，犯下大量的第一类错误。

multiple testing

问题有多严重？回顾上一节，如果我们拒绝单个零假设，H_0，如果其 p 值小于，比如说，$\alpha = 0.01$，那么当 H_0 实际上为真时，犯错误拒绝的概率为1%。现在，如果我们检验 m 个零假设，$H_{01}, \ldots, H_{0m}$，全部为真，会怎样？拒绝任何单个零假设的概率为1%；因此，我们预期错误地拒绝大约 $0.01 \times m$ 个零假设。如果 $m = 10{,}000$，那么这意味着我们预期偶然错误地拒绝100个零假设！这是一个 *lot* 的第一类错误。

问题的关键如下：如果 p 值低于 α 就拒绝零假设，这会将错误拒绝*that null hypothesis*的概率控制在 α 水平。然而，如果我们对 m 个零假设都这样做，那么错误拒绝 *at least one of the m null hypotheses* 的机会就会高得多！

---

[11] A *fair coin* is one that has an equal chance of landing heads or tails.

[12] Recall that the p-value is the probability of observing data at least this extreme, under the null hypothesis. If the coin is fair, then the probability of observing at least ten tails is $(1/2)^{10} = 1/1{,}024 < 0.001$. The p-value is therefore $2/1{,}024 < 0.002$, since this is the probability of observing ten heads or ten tails.

	H_0 is True	H_0 is False	Total
Reject H_0	V	S	R
Do Not Reject H_0	U	W	$m-R$
Total	m_0	$m-m_0$	m

表13.2。 *A summary of the results of testing m null hypotheses. A given null hypothesis is either true or false, and a test of that null hypothesis can either reject or fail to reject it. In practice, the individual values of V, S, U, and W are unknown. However, we do have access to V + S = R and U + W = m − R, which are the numbers of null hypotheses rejected and not rejected, respectively.*

我们将在第13.3节中更详细地探讨此问题，并提出解决方案。

13.3 家庭式错误率

在以下各节中，我们将讨论在控制至少犯一次第一类错误的概率的同时检验多个假设。

13.3.1 What is the Family-Wise Error Rate?

回想一下，第一类错误率是指在H_0为真的情况下拒绝H_0的概率。*family-wise error rate* (FWER)将这一概念推广到集合-m 个零假设的检验，$H_{01}, \ldots, H_{0m}$，并被定义为犯 *at least one* 个第一类错误的概率。更正式地表述这一概念，考虑表13.2，它总结了进行 m 个假设检验时可能的结果。这里，V表示第一类错误的数量（也称为假阳性或假发现），S 表示真阳性的数量，U 表示真阴性的数量，而 W 表示第二类错误的数量（也称为假阴性）。那么，家庭-wise 错误率由下式给出：

family-wise error rate

$$\text{FWER} = \Pr(V \geq 1). \tag{13.3}$$

一种策略，若任何原假设的p值低于α (，即对每个原假设在水平α)上控制第一类错误，则拒绝该原假设，这导致FWER为

$$\begin{aligned} \text{FWER}(\alpha) &= 1 - \Pr(V = 0) \\ &= 1 - \Pr(\text{do not falsely reject any null hypotheses}) \\ &= 1 - \Pr\Big(\textstyle\bigcap_{j=1}^{m} \{\text{do not falsely reject } H_{0j}\}\Big). \end{aligned} \tag{13.4}$$

回忆基础概率论，如果两个事件 A 和 B 是独立的，那么 $\Pr(A \cap B) = \Pr(A)\Pr(B)$。因此，如果我们作出额外且相当强的假设，即 m 个检验是独立的，并且所有 m 个原假设均为真，那么

$$\text{FWER}(\alpha) = 1 - \prod_{j=1}^{m}(1-\alpha) = 1 - (1-\alpha)^m. \tag{13.5}$$

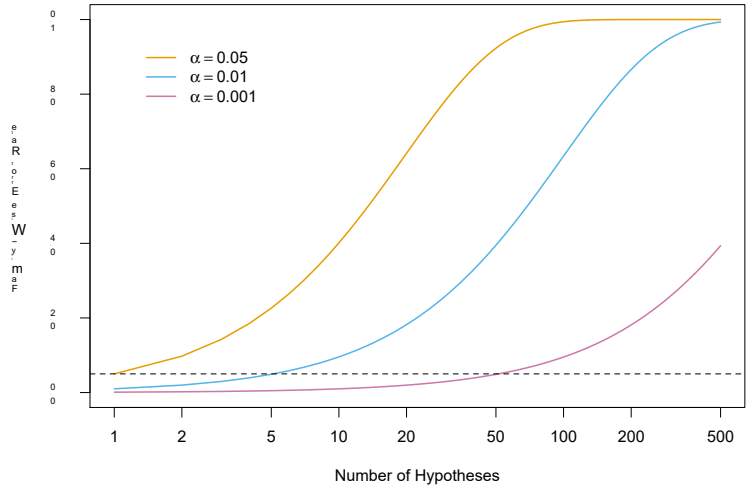


图13.2。 *The family-wise error rate, as a function of the number of hypotheses tested (displayed on the log scale), for three values of* α*:* $\alpha = 0.05$ *(orange),* $\alpha = 0.01$ *(blue), and* $\alpha = 0.001$ *(purple). The dashed line indicates* 0.05*. For example, in order to control the FWER at* 0.05 *when testing* $m = 50$ *null hypotheses, we must control the Type I error for each null hypothesis at level* $\alpha = 0.001$*.*

因此，如果我们只检验一个原假设，那么FWER(α) $= 1 - (1 - \alpha)^1 = \alpha$，所以第一类错误率与FWER相等。然而，如果我们进行$m = 100$次独立检验，那么FWER(α) $= 1 - (1 - \alpha)^{100}$。例如，取$\alpha = 0.05$会导致FWER为$1 - (1 - 0.05)^{100} = 0.994$。换句话说，我们几乎必然会犯至少一个第一类错误！

图13.2展示了(13.5)在不同m（假设数量）和α（I类错误）取值下的结果。我们看到，设定$\alpha = 0.05$会导致即使对于中等大小的m，FWER也很高。当$\alpha = 0.01$时，在FWER超过0.05之前，我们最多只能检验五个原假设。只有当取值非常小，例如$\alpha = 0.001$时，我们才能确保较小的FWER，至少对于中等大小的m而言。

我们现在简要回到第13.1.1节中的例子，其中我们考虑使用两样本t统计量检验形式为H_0: $\mu_t = \mu_c$的单个原假设。回顾图13.1，为了保证第一类错误不超过0.02，我们使用2.33的割点来决定是否拒绝H_0（即如果$|T| \geq 2.33$，则拒绝H_0）。现在，如果我们希望使用两样本t统计量检验10个原假设，而不是仅检验一个，该怎么办？我们将在第13.3.2节看到，通过仅拒绝那些p值低于0.002的原假设，我们可以保证FWER不超过0.02。这对应一个严格得多的割点3.09（即对于$j = 1, \ldots, 10$，只有当检验统计量$|T_j| \geq 3.09$时，才应拒绝H_{0j}）。换句话说，将FWER控制在水平α上，就拒绝任何给定原假设所需的证据而言，其门槛远高于仅将每个原假设的第一类错误控制在水平α上。

Manager	Mean, $\bar{x}$	Standard Deviation, s	t-statistic	p-value
One	3.0	7.4	2.86	0.006
Two	-0.1	6.9	-0.10	0.918
Three	2.8	7.5	2.62	0.012
Four	0.5	6.7	0.53	0.601
Five	0.3	6.8	0.31	0.756

表13.3. *The first two columns correspond to the sample mean and sample standard deviation of the percentage excess return, over* $n = 50$ *months, for the first five managers in the* `Fund` *dataset. The last two columns provide the* t*-statistic* ($\sqrt{n} \cdot \bar{X}/S$) *and associated* p*-value for testing* H_{0j}: $\mu_j = 0$*, the null hypothesis that the (population) mean return for the* j*th hedge fund manager equals zero.*

13.3.2 Approaches to Control the Family-Wise Error Rate

在本节中，我们简要概述了一些控制FWER的方法。我们将通过`Fund`数据集来说明这些方法，该数据集记录了2000名基金经理在50个月内的月度百分比超额回报。表13.3提供了前五名基金经理的相关汇总统计数据。

我们首先介绍Bonferroni方法和Holm的逐步向下程序，这两种是非常通用的控制FWER的方法，只要计算出{p}-值，无论零假设的形式、检验统计量的选择或{p}-值的（不）独立性如何，都可以应用。然后，我们简要讨论Tukey方法和Scheffé方法，以说明在某些情况下，采用更专门的控制FWER的方法可能更为可取。

邦费罗尼方法

如前一节所述，假设我们希望检验 $H_{01}, \ldots, H_{0m}$。令 A_j 表示对于第 j 个零假设，我们犯第一类错误的事件，其中 $j = 1, \ldots, m$。那么

$$\begin{aligned} \text{FWER} &= \Pr(\text{falsely reject at least one null hypothesis}) \\ &= \Pr(\cup_{j=1}^{m} A_j) \\ &\leq \sum_{j=1}^{m} \Pr(A_j). \end{aligned} \tag{13.6}$$

在(13.6)中，不等式的得出基于这样一个事实：对于任意两个事件A和B，$\Pr(A \cup B) \leq \Pr(A) + \Pr(B)$，无论$A$和$B$是否独立。*Bonferroni method*，即*Bonferroni correction*，将每个假设检验的拒绝阈值设为α/m，从而使得$\Pr(A_j) \leq \alpha/m$。方程13.6意味着

$$\text{FWER}(\alpha/m) \leq m \times \frac{\alpha}{m} = \alpha,$$

[13] Excess returns correspond to the additional return the fund manager achieves beyond the market's overall return. So if the market increases by 5% during a given period and the fund manager achieves a 7% return, their *excess return* would be $7\% - 5\% = 2\%$.

因此，此过程在 α 水平上控制 FWER。例如，为了在测试 $m = 100$ 个零假设时，将 FWER 控制在 0.1 水平，Bonferroni 过程要求我们将每个零假设的 Type I 错误率控制在 $0.1/100 = 0.001$ 水平，即拒绝所有 p-值低于 0.001 的零假设。

我们现在考虑表13.3中的Fund数据集。如果我们分别在每个基金经理处将第一类错误控制在$\alpha = 0.05$水平，那么我们会得出第一位和第三位经理具有显著非零超额回报的结论；换言之，我们将拒绝H_{01}：$\mu_1 = 0$和H_{03}：$\mu_3 = 0$。然而，如前几节所述，这一程序并未考虑到我们检验了多个假设这一事实，因此会导致FWER大于0.05。如果我们改为希望在0.05水平上控制FWER，那么使用Bonferroni校正，我们必须将每个个体经理的第一类错误控制在$\alpha/m = 0.05/5 = 0.01$水平。因此，我们只会拒绝第一位经理的原假设，因为所有其他经理的p值都超过0.01。Bonferroni校正让我们安心，因为我们没有错误地拒绝过多原假设，但这是有代价的：我们拒绝的原假设很少，因此通常会犯相当多的第二类错误。

Bonferroni校正是统计学中最著名且最常用的多重性校正方法。它的普遍性在很大程度上归因于其易于理解和实施简单的特点，以及无论m假设检验是否独立，它都能成功控制第一类错误的事实。然而，正如我们将要看到的，它通常既不是最强大的方法，也不是多重测试校正的最佳途径。特别是，Bonferroni校正可能相当保守，这意味着真实的FWER往往比名义（或目标）FWER要低得多；这一结果源自(13.6)中的不等式。相反，一个较为不保守的程序可能允许我们在控制FWER的同时拒绝更多零假设，从而减少第二类错误。

Holm的逐步向下过程

Holm's method,也称为Holm逐步向下过程或Holm–Bonferroni法，是Bonferroni程序的一种替代方法。Holm方法控制FWER，但它比Bonferroni法保守性更低，即它会拒绝更多的零假设，通常导致更少的第二类错误，从而具有更大的功效。该程序在算法13.1中进行了总结。证明该方法控制FWER的过程与(13.6)中Bonferroni方法控制FWER的论证相似，但稍微复杂一些。值得注意的是，在Holm程序中，我们用来拒绝每个零假设的阈值——步骤5中的$p_{(L)}$——实际上取决于p个值的$all\ m$值。（参见(13.7)中L的定义。）这与Bonferroni程序形成对比，在后者中，为了在α水平上控制FWER，我们拒绝任何p值低于α/m的零假设，而不考虑其他p值。Holm方法不对m假设检验作任何独立性假设，并且均匀地比Bonferroni方法更强大——它将

Holm's method

算法13.1 *Holm's Step-Down Procedure to Control the FWER*

1. 指定 α，即控制FWER的水平。
2. 计算m个零假设$H_{01}, \ldots, H_{0m}$的p值，$p_1, \ldots, p_m$。
3. 将{v*}值排序，使得$p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(m)}$。
4. 定义
$$L = \min \left\{ j : p_{(j)} > \frac{\alpha}{m+1-j} \right\}. \tag{13.7}$$
5. 拒绝所有零假设 H_{0j}，其中 $p_j < p_{(L)}$。

总是拒绝至少与Bonferroni一样多的零假设——因此应始终优先选择。

我们现在考虑将 Holm 方法应用于表 13.3 中 `Fund` 数据集的前五位基金经理，同时在 0.05 水平上控制 FWER。排序后的 p 值为 $p_{(1)} = 0.006, p_{(2)} = 0.012, p_{(3)} = 0.601, p_{(4)} = 0.756$ 和 $p_{(5)} = 0.918$。Holm 程序拒绝了前两个原假设，因为 $p_{(1)} = 0.006 < 0.05/(5+1-1) = 0.01$ 和 $p_{(2)} = 0.012 < 0.05/(5+1-2) = 0.0125$，但 $p_{(3)} = 0.601 > 0.05/(5+1-3) = 0.0167$，这意味着 $L = 3$。我们注意到，在这种情况下，Holm 比 Bonferroni 更有效：前者拒绝了第一位和第三位经理的原假设，而后者仅拒绝了第一位经理的原假设。

图13.3展示了在涉及m =10个假设检验的设定下，Bonferroni和Holm方法在三个模拟数据集上的应用，其中m_0 =2个零假设为真。每个面板显示了对应的十个p值，从最小到最大排序，并以对数刻度绘制。八个红色点代表错误的零假设，两个黑色点代表真实的零假设。我们希望在0.05水平上控制FWER。Bonferroni程序要求我们拒绝所有p值低于0.005的零假设；这由黑色水平线表示。Holm程序要求我们拒绝所有低于蓝线的零假设。蓝线始终位于黑线之上，因此Holm总是比Bonferroni拒绝更多的检验；两条线之间的区域对应于仅被Holm拒绝的假设。在左侧面板中，Bonferroni和Holm均成功拒绝了八个错误零假设中的七个。在中间面板中，Holm成功拒绝了所有八个错误零假设，而Bonferroni未能拒绝一个。在右侧面板中，Bonferroni仅拒绝了三个错误零假设，而Holm拒绝了所有八个。在这些例子中，Bonferroni和Holm均未犯任何第一类错误。

两个特殊情况：图基方法和谢弗方法



Bonferroni 方法和 Holm 方法几乎可以在任何我们希望控制 m 个零假设的 FWER 的情况下使用：它们

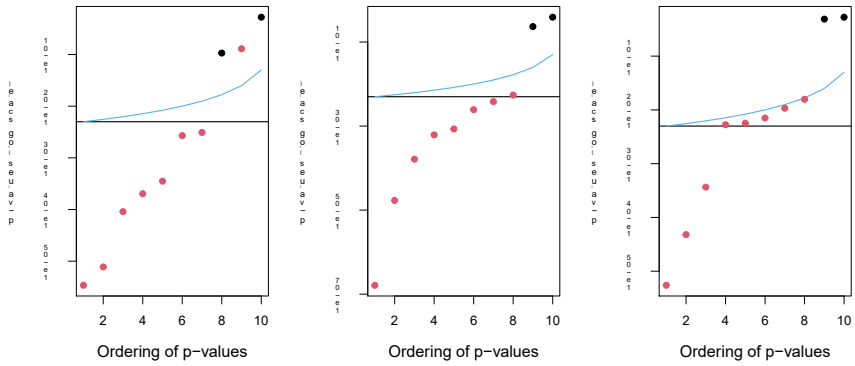


图13.3。 *Each panel displays, for a separate simulation, the sorted p-values for tests of* $m = 10$ *null hypotheses. The p-values corresponding to the* $m_0 = 2$ *true null hypotheses are displayed in black, and the rest are in red. When controlling the FWER at level* 0.05*, the Bonferroni procedure rejects all null hypotheses that fall below the black line, and the Holm procedure rejects all null hypotheses that fall below the blue line. The region between the blue and black lines indicates null hypotheses that are rejected using the Holm procedure but not using the Bonferroni procedure. In the center panel, the Holm procedure rejects one more null hypothesis than the Bonferroni procedure. In the right-hand panel, it rejects five more null hypotheses.*

对零假设的性质、所用检验统计量的类型或 p 值的（非）独立性不作任何假设。然而，在某些非常特定的情况下，我们可以通过使用更适合当前任务的方法来控制 FWER，从而实现更高的功效。Tukey 方法和 Scheffé 方法提供了两个这样的例子。

表13.3表明，对于Fund数据集，经理一和经理二的样本平均回报差异最大。这一发现可能会促使我们检验原假设H_0：$\mu_1 = \mu_2$，其中μ_j是第j位基金经理的（总体）平均回报。对H_0进行双样本t检验（13.1）得到的p值为0.0349，表明对H_0有微弱的反对证据。然而，这个p值具有误导性，因为我们是在考察了所有五位经理的回报之后才决定比较经理一和经理二的平均回报的；这实质上相当于进行了$m = 5 \times (5-1)/2 = 10$次假设检验，并选择了$p$值最小的那一次。这表明，为了在0.05的水平上控制FWER，我们应该对$m = 10$次假设检验进行Bonferroni校正，因此只应拒绝p值低于0.005的原假设。如果这样做，那么我们将无法拒绝经理一和经理二表现相同的原假设。

然而，在这种设定下，Bonferroni 校正实际上过于严格，因为它没有考虑到 $m = 10$ 个假设检验都或多或少相关这一事实：例如，经理二和经理五的平均回报相似，经理二和经理四也是如此；这保证了经理四和经理五的平均回报相似。换句话说，m 两两比较的 m p 值是 *not* 独立的。因此，应该有可能以一种

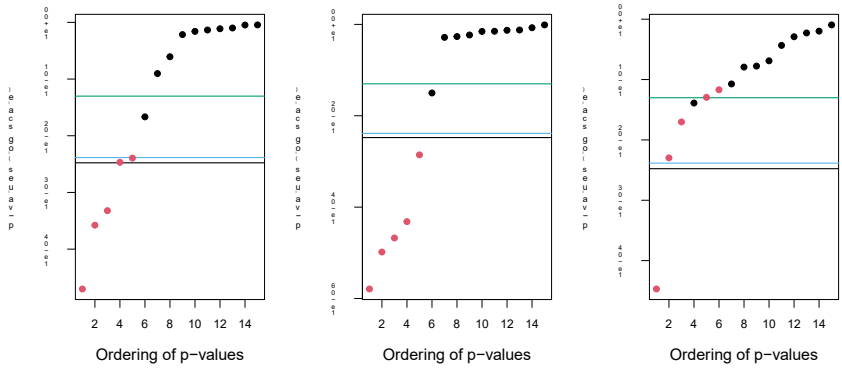


图 13.4. *Each panel displays, for a separate simulation, the sorted p-values for tests of* $m = 15$ *hypotheses, corresponding to pairwise tests for the equality of* $G = 6$ *means. The* $m_0 = 10$ *true null hypotheses are displayed in black, and the rest are in red. When controlling the FWER at level* 0.05*, the Bonferroni procedure rejects all null hypotheses that fall below the black line, whereas Tukey rejects all those that fall below the blue line. Thus, Tukey's method has slightly higher power than Bonferroni's method. Controlling the Type I error* 无 *adjusting for multiple testing involves rejecting all those that fall below the green line.*

较不保守。这正是 *Tukey's method* 背后的想法：当执行 $m = G(G-1)/2$ 对 G 均值的成对比较，它允许我们在 α 水平上控制 FWER，同时拒绝所有 p 值低于 α_T 的原假设，对于某些 $\alpha_T > \alpha/m$。

Tukey's method

图13.4展示了在图克方法在三个模拟数据集上的应用，设定中有$G = 6$个均值，且$\mu_1 = \mu_2 = \mu_3 = \mu_4 = \mu_5 \neq \mu_6$。因此，在形式为$H_0 : \mu_j = \mu_k$的$m = G(G-1)/2 = 15$个零假设中，有十个是真的，五个是假的。在每个面板中，真实的零假设以黑色显示，虚假的以红色显示。水平线表明，图克方法总是导致至少与邦弗朗尼方法一样多的拒绝。在左侧面板中，图克正确地比邦弗朗尼多拒绝了两个零假设。

现在，假设我们再次查看表13.3中的数据，并注意到经理一和经理三的平均回报率高于经理二、四和五。这可能促使我们检验零假设

$$H_0 : \frac{1}{2}(\mu_1 + \mu_3) = \frac{1}{3}(\mu_2 + \mu_4 + \mu_5). \tag{13.8}$$

(回想一下，μ_j 是第 j 位对冲基金管理者的总体平均收益。事实证明，我们可以使用 (13.1) 中提出的双样本 t 检验的一种变体来检验 (13.8)，得到的 p 值为 0.004。这表明有强有力的证据显示管理者一和管理者三与管理二、四、五之间存在差异。然而，存在一个问题：我们是在查看了表13.3中的数据之后才决定检验 (13.8) 中的原假设。从某种意义上说，这意味着我们进行了多重检验。在这种情况下，使用Bonferroni方法在 α 水平上控制FWER

将需要一个 p 值阈值 α/m，对于极其巨大的 m[14] 值。

Scheffé's method 正是为这种设置而设计的。它允许我们进可以计算出一个值 α_S，使得当 p 值低于 α_S 时拒绝 (13.8) 中的原假设 H_0，从而将第一类错误控制在水平 α。结果表明，对于 Fund 示例，为了将第一类错误控制在水平 $\alpha = 0.05$，我们必须设定 $\alpha_S = 0.002$。因此，尽管 p 值看似非常小，仅为 0.004，我们仍无法拒绝 (13.8) 中的 H_0。Scheffé 方法的一个重要优势在于，我们可以使用相同的阈值 $\alpha_S = 0.002$ 来对管理者任意划分为两组进行成对比较：例如，我们也可以检验 H_0: $\frac{1}{3}(\mu_1 + \mu_2 + \mu_3) = \frac{1}{2}(\mu_4 + \mu_5)$ 和 H_0: $\frac{1}{4}(\mu_1 + \mu_2 + \mu_3 + \mu_4) = \mu_5$，使用相同的阈值 0.002，而无需进一步针对多重检验进行调整。

Scheffé 方法

总之，{v*}和Bonferroni方法是非常通用的多重检验校正方法，可以在所有情况下应用。然而，在某些特殊情况下，可能有更强大的多重检验校正方法可用，以便在控制FWER的同时实现比使用Holm或Bonferroni更高的功效（即犯更少的第二类错误）。在本节中，我们展示了两个这样的例子。

13.3.3 Trade-Off Between the FWER and Power

一般来说，我们选择的FWER阈值与拒绝零假设的*power*之间存在权衡。回顾一下，功效定义为被拒绝的假零假设的数量除以假零假设的总数，即使用表13.2的符号表示为$S/(m - m_0)$。图13.5展示了一个模拟设置的结果，其中涉及m个零假设，90%为真，其余10%为假；功效作为FWER的函数显示。在这个特定的模拟设置中，当m =为10时，FWER为0.05对应于大约60%的功效。然而，随着m的增加，功效下降。当m =为500时，在FWER为0.05的情况下，功效低于0.2，因此我们只能成功拒绝20%的假零假设。

图13.5表明，当m取较小值（如5或10）时，控制FWER是合理的。然而，对于m = 100或m = 1,000的情况，试图控制FWER将使得几乎无法拒绝任何错误零假设。换句话说，功效将极其低。

为什么会出现这种情况？回想一下，使用表13.2中的符号，FWER定义为Pr($V \geq 1$)（13.3）。换句话说，在α水平上控制FWER保证了数据分析师在概率不超过α)的情况下拒绝*any*个真正的零假设，即出现任何假阳性。为了在m很大时兑现这一保证，数据分析师可能被迫拒绝极少的零假设，甚至可能完全不拒绝（因为如果$R = 0$，那么$V = 0$；见表13.2）。

[14] In fact, calculating the "correct" value of m is quite technical, and outside the scope of this book.

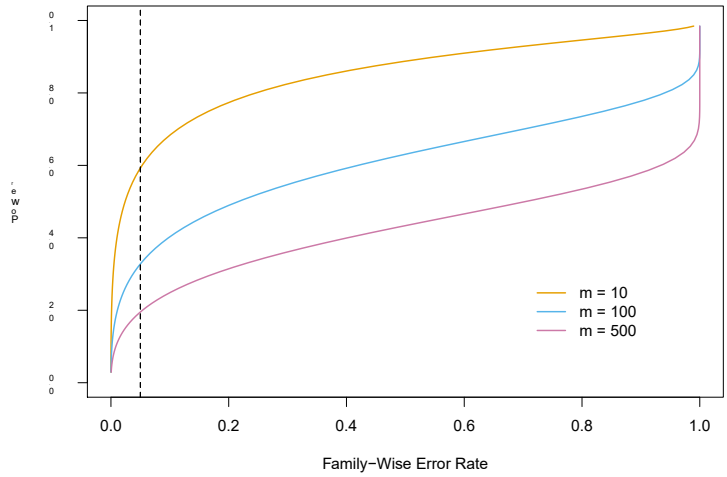


图13.5。*In a simulation setting in which 90% of the m null hypotheses are true, we display the power (the fraction of false null hypotheses that we successfully reject) as a function of the family-wise error rate. The curves correspond to* $m = 10$（橙色），$m = 100$（蓝色），*and* $m = 500$（紫色）。*As the value of m increases, the power decreases. The vertical dashed line indicates a FWER of* 0.05.

这在科学上并无趣味，且通常导致极低的统计功效，如图13.5所示。

在实践中，当m较大时，为了做出更多发现，即更多次拒绝原假设，我们可能愿意容忍一些假阳性。这就是我们接下来介绍的假发现率背后的动机。

13.4 错误发现率

13.4.1 Intuition for the False Discovery Rate

正如我们刚才讨论的，当 m 很大时，试图防止 *any* 假阳性（如FWER控制中）就过于严格了。相反，我们可能试图确保假阳性比率（V）与总阳性（$V + S = R$）的比足够低，以便大多数被拒绝的原假设并非假阳性。比率 V/R 被称为 *false discovery proportion*

false discovery proportion

(FDP)。

人们可能会忍不住要求数据分析师控制FDP：确保在被拒绝的原假设中，假阳性的比例不超过，比如说，20%。然而，在实践中，控制FDP对数据分析师来说是一项不可能完成的任务，因为她无法在任何特定数据集上确定哪些假设是真的，哪些是假的。这与数据分析师可以控制FWER的事实非常相似，即她可以保证对于任何预先指定的α，$\Pr(V \geq 1) \leq \alpha$，但她不能保证在任何特定数据集上$V = 0$（除非未能拒绝任何原假设，即设定$R = 0$）。

因此，我们转而控制定义的*false discovery rate* (FDR)[15]，当

错误发现速率

$$\text{FDR} = \text{E}(\text{FDP}) = \text{E}(V/R). \tag{13.9}$$

当我们将FDR控制在（例如）q =20%的水平时，我们尽可能多地拒绝零假设，同时保证在被拒绝的零假设中，假阳性的比例不超过20%，*on average*。

在(13.9)中FDR的定义里，期望是基于数据生成的总体来计算的。例如，假设我们对m个零假设在q =0.2的水平上控制FDR。这意味着如果我们重复这个实验无数次，并且每次都把FDR控制在q =0.2的水平，那么我们应当预期，平均而言，被拒绝的零假设中将有20%是假阳性。在某个具体数据集上，被拒绝假设中假阳性的比例可能高于或低于20%。

到目前为止，我们是从务实的角度来论证使用FDR的必要性，认为当m较大时，控制FWER过于严格，无法带来“足够多”的发现。使用FDR的另一个动机是，它与当代应用中数据采集的方式高度契合。随着各领域数据集规模持续增长，出于探索性而非验证性目的进行大量假设检验变得越来越普遍。例如，一位基因组研究人员可能对患有和未患有某种特定疾病的个体进行基因组测序，然后针对20,000个基因中的每一个，检验该基因中的序列变异是否与所关注的疾病相关。这相当于进行m =20,000次假设检验。该分析本质上是探索性的，因为研究人员并没有预设任何特定的假设；相反，她希望了解每个基因与疾病之间是否存在适度证据，并计划进一步研究任何有此证据的基因。她很可能会容忍在她将进一步研究的基因集中存在一定数量的假阳性；因此，FWER并非合适的选择。然而，需要对多重检验进行某种校正：对她来说，简单地研究p值小于（比如）0.05的*all*个基因并不是一个好主意，因为即使没有任何基因与疾病相关，我们也会预期有1,000个基因仅凭偶然就具有如此小的p值（因为$0.05 \times 20{,}000 = 1{,}000$）。将她的探索性分析的FDR控制在20%，可以保证——平均而言——她进一步研究的基因中假阳性不超过20%。

值得注意的是，与p值不同——对于p值，0.05的阈值通常被视为“阳性”结果的最低证据标准，而0.01甚至0.001的阈值则被视为更具说服力——对于FDR控制，并没有公认的标准阈值。相反，FDR阈值的选择通常取决于具体情境，甚至取决于数据集。例如，前一个例子中的基因组研究人员可能会寻求将FDR控制在10%的阈值，如果计划中的后续……

[15] If $R = 0$, then we replace the ratio V/R with 0, to avoid computing 0/0. Formally, $\text{FDR} = \text{E}(V/R|R > 0)\,\Pr(R > 0)$.

分析可能耗时或成本高昂。或者，如果她计划进行成本较低的后续分析，那么30%这一大得多的阈值可能更为合适。

13.4.2 The Benjamini–Hochberg Procedure

我们现在关注控制FDR的任务：即在保证FDR，E(V/R)，小于或等于某个预先指定的值q的同时，决定拒绝哪些零假设。为了做到这一点，我们需要某种方法将来自m个零假设的p值，$p_1, \ldots, p_m$，与期望的FDR值q联系起来。事实证明，可以使用算法13.2中概述的一个非常简单的程序来控制FDR。

算法13.2 *Benjamini–Hochberg Procedure to Control the FDR*

1. 指定q，即控制FDR的水平。2. 计算p值，$p_1, \ldots, p_m$，对于m零假设$H_{01}, \ldots, H_{0m}$。3. 将m p值排序，使得$p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(m)}$。4. 定义

$$L = \max\{j : p_{(j)} < qj/m\}. \tag{13.10}$$

5. 拒绝所有原假设 H_{0j}，对于这些假设 $p_j \leq p_{(L)}$。

算法13.2被称为*Benjamini–Hochberg procedure*。关键此过程的原理见(13.10)。例如，再次考虑Fund数据集中的前五位管理者，如表13.3所示。（在此例中，m =5，尽管在涉及更多零假设的设定中，我们通常控制FDR。）我们看到$p_{(1)}$ =0.006 <0.05 ×1/5，$p_{(2)}$ =0.012 <0.05 ×2/5，$p_{(3)}$ =0.601 >0.05 ×3/5，$p_{(4)}$ =0.756 >0.05 ×4/5，以及$p_{(5)}$ =0.918 >0.05 ×5/5。因此，要在5%水平上控制FDR，我们拒绝第一位和第三位基金经理表现不优于随机的零假设。

Benjamini–Hochberg procedure

只要 m p 值相互独立或仅轻微相关，那么 Benjamini–Hochberg 程序保证[16]

$$\text{FDR} \leq q.$$

换句话说，这一过程确保平均而言，被拒绝的原假设中假阳性的比例不超过 q。值得注意的是，无论有多少原假设为真，也无论那些为假的原假设的 p 值分布如何，这一点都成立。因此，Benjamini–Hochberg 过程为我们提供了一种非常简便的方法，在给定一组 m p 值的情况下，确定应拒绝哪些原假设，以便将 FDR 控制在任何预先指定的水平 q。

[16] However, the proof is well beyond the scope of this book.

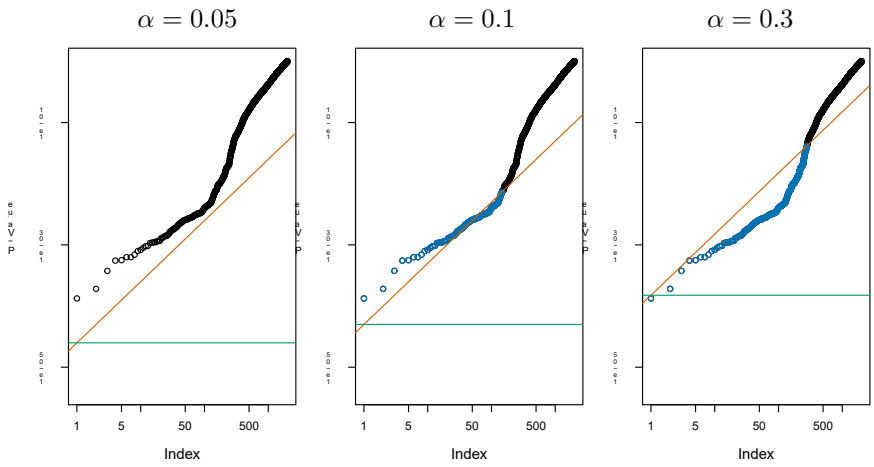


图 13.6. *Each panel displays the same set of* $m = 2{,}000$ *ordered p-values for the* Fund *data. The green lines indicate the p-value thresholds corresponding to FWER control, via the Bonferroni procedure, at levels* $\alpha = 0.05$（左）, $\alpha = 0.1$（中）, *and* $\alpha = 0.3$（右）. *The orange lines indicate the p-value thresholds corresponding to FDR control, via Benjamini–Hochberg, at levels* $q = 0.05$（左）, $q = 0.1$（中）, *and* $q = 0.3$（右）. *When the FDR is controlled at level* $q = 0.1$, *146 null hypotheses are rejected* (中)*; the corresponding p-values are shown in blue. When the FDR is controlled at level* $q = 0.3$, *279 null hypotheses are rejected* (右)*; the corresponding p-values are shown in blue.*

第13.3.2节的Bonferroni程序与Benjamini–Hochberg程序之间存在根本区别。在Bonferroni程序中，为了在水平α下控制m个零假设的FWER，我们只需拒绝那些p值低于α/m的零假设。这个α/m的阈值不依赖于数据的任何方面（除了m的值之外），当然也不依赖于p值本身。相比之下，Benjamini–Hochberg程序中使用的拒绝阈值更为复杂：我们拒绝所有p值小于或等于第L小的p值的零假设，其中L本身是所有m p值的函数，如（13.10）所示。因此，在进行Benjamini–Hochberg程序时，我们无法预先规划将使用什么阈值来拒绝p值；我们需要先看到数据。例如，在摘要中，当使用FDR阈值0.1且m =为100时，无法知道我们是否会拒绝对应于0.01的p值的零假设；答案取决于其他$m - 1$ p值。Benjamini–Hochberg程序的这一性质也为Holm程序所共有，后者同样涉及数据依赖的p值阈值。

图13.6展示了在Fund数据集上应用Bonferroni和Benjamini–Hochberg程序的结果，使用的是完整的$m = 2{,}000$名基金经理集合，其中前五位已在表13.3中展示。当使用Bonferroni将FWER控制在水平0.3时，仅有一个原假设被拒绝；也就是说，我们只能得出结论，有一位基金经理跑赢了市场。尽管事实上相当一部分

m =2,000位基金经理似乎在没有进行多重检验校正的情况下战胜了市场——例如，其中13位的p值低于0.001。相比之下，当FDR控制在0.3水平时，我们可以得出结论：279位基金经理战胜了市场：我们预期这些基金经理中不超过约$279 \times 0.3 = 83.7$位仅因运气而表现良好。因此，我们看到FDR控制比FWER控制宽松得多——也更强大——因为它允许我们拒绝更多的原假设，代价是实质上更多的假阳性。

Benjamini–Hochberg 方法自 20 世纪 90 年代中期以来一直存在。尽管自那时起已有大量论文提出了在特定场景下能表现更好的 FDR 控制替代方法，但 Benjamini–Hochberg 方法仍然是一种非常有用且广泛适用的方法。

13.5 重新采样方法估计p值和错误发现率

到目前为止，本章的讨论假设我们感兴趣的是使用检验统计量 T 来检验一个特定的原假设 H_0，该统计量在 H_0 下具有某些已知（或假定）的分布，例如正态分布、t 分布、χ^2 分布或 F 分布。这被称为 *theoretical null distribution*。我们通常依赖于为了获得与我们的检验统计量相关的p-值，需要理论零分布的存在性。实际上，对于我们可能感兴趣检验的大多数类型的零假设，只要愿意对数据做出严格的假设，理论零分布是可用的。

theoretical null distribution

然而，如果我们的零假设H_0或检验统计量T有些不同寻常，那么可能不存在理论上的零分布。或者，即使存在理论上的零分布，我们可能也不愿依赖它，或许是因为其成立所需的某些假设被违反了。例如，样本量可能太小。

在本节中，我们提出一个在此设定下进行推断的框架，该框架利用快速计算机的可用性来近似 T 的零分布，从而获得 p 值。虽然这个框架非常通用，但必须针对具体感兴趣的问题仔细实例化。因此，在下文中，我们考虑一个具体的例子，其中我们希望使用双样本 t 检验来检验两个随机变量的均值是否相等。

本节中的讨论比本章前面的部分更具挑战性，且若读者满足于使用理论零分布来计算其检验统计量的p-值，则可放心跳过本节。

13.5.1 A Re-Sampling Approach to the p-Value

我们回到13.1.1节的例子，其中我们希望检验一个随机变量 X 的均值是否等于另一个随机变量 Y的均值，即 H_0： $\mathrm{E}(X) = \mathrm{E}(Y)$，相对于备择假设 H_a： $\mathrm{E}(X) \neq \mathrm{E}(Y)$。给定来自 X 的 n_X 个独立观测值和来自 Y的 n_Y 个独立观测值，两样本 t 统计量形式为

$$T = \frac{\hat{\mu}_X - \hat{\mu}_Y}{s\sqrt{\frac{1}{n_X} + \frac{1}{n_Y}}} \tag{13.11}$$

其中 $\hat{\mu}_X = \frac{1}{n_X}\sum_{i=1}^{n_X} x_i$、$\hat{\mu}_Y = \frac{1}{n_Y}\sum_{i=1}^{n_Y} y_i$、$s = \sqrt{\frac{(n_X-1)s_X^2+(n_Y-1)s_Y^2}{n_X+n_Y-2}}$、$s_X^2$ 和 s_Y^2是两组方差的无偏估计量。T的较大（绝对）值提供了反对 H_0 的证据。

如果 n_X 和 n_Y很大，那么（13.11）中的 T近似服从 $N(0, 1)$ 分布。但如果 n_X 和 n_Y很小，那么在对 X 和 Y的分布没有强假设的情况下，我们不知道 T的理论零分布。[17] 在这种情况下，事实证明我们可以使用 *re-sampling* 方法近似 T的零分布，或者更具体地说，是一种*permutation*方法。

re-sampling
permutation

为此，我们进行一个思想实验。如果 H_0 成立，使得 $\mathrm{E}(X) = \mathrm{E}(Y)$，并且我们作更强的假设，即 X 和 Y的分布相同，那么 T的分布在将 X 的观测与 Y的观测交换时保持不变。也就是说，如果我们随机地将 X 中的一些观测与 Y中的观测交换，那么 *the test statistic* T *in* (13.11) *computed based on this swapped data has the same distribution as* T *based on the original data.* 这只有在 H_0 成立，并且 X 和 Y的分布相同时才为真。

这表明，为了近似得到 T的零分布，我们可以采取以下方法。我们对 $n_X + n_Y$个观测值进行 B 次随机置换，其中 B 取值较大，并在每次置换后计算（13.11）。我们令 $T^{*1}, \ldots, T^{*B}$ 表示在置换数据上（13.11）的值。这些值可以被视为在 H_0 下 T的零分布的一个近似。回想一下，根据定义，p 值是在 H_0 下观察到至少如此极端的检验统计量的概率。因此，为了计算 T的 p 值，我们可以简单地计算

$$p\text{-value} = \frac{\sum_{b=1}^{B} 1_{(|T^{*b}| \geq |T|)}}{B}, \tag{13.12}$$

置换数据集中检验统计量的值至少与原始数据上观测到的值一样极端的比例。该过程总结在算法13.3中。

---

[17] If we assume that X and Y are normally distributed, then T in (13.11) follows a t-distribution with $n_X + n_Y - 2$ degrees of freedom under H_0. However, in practice, the distribution of random variables is rarely known, and so it can be preferable to perform a re-sampling approach instead of making strong and unjustified assumptions. If the results of the re-sampling approach disagree with the results of assuming a theoretical null distribution, then the results of the re-sampling approach are more trustworthy.

算法 13.3 *Re-Sampling p-Value for a Two-Sample t-Test*

1. 在原始数据 $x_1, \ldots, x_{n_X}$ 和 $y_1, \ldots, y_{n_Y}$ 上计算（13.11）中定义的 T。

2. 对于$b = 1, \ldots, B$，其中B是一个大数（例如$B = 10{,}000$）：

   (a) 随机排列 $n_X + n_Y$ 个观测值。将前 n_X 个排列后的观测值称为 $x^*_1, \ldots, x^*_{n_X}$，并将其余的 n_Y 个观测值称为 $y^*_1, \ldots, y^*_{n_Y}$。(b) 对排列后的数据 $x^*_1, \ldots, x^*_{n_X}$ 和 $y^*_1, \ldots, y^*_{n_Y}$ 计算 (13.11)，并将结果称为 T^{*b}。

3. p 值由 $\frac{\sum_{b=1}^{B} 1_{(|T^{*b}| \geq |T|)}}{B}$ 给出。

我们在 `Khan` 数据集上尝试这个过程，该数据集包含2,308个基因在四种小圆血细胞肿瘤亚型中的表达测量值，这是一种通常见于儿童的癌症。此数据集是 `ISLR2` 包的一部分。我们将注意力限制在可获得最多观察值的两个亚型上：横纹肌肉瘤（$n_X = 29$）和伯基特淋巴瘤（$n_Y = 25$）。

对零假设为第11个基因在两组中的平均表达值相等的双样本t检验，得到$T = -2.09$。使用理论零分布，即t_{52}分布（因为$n_X + n_Y - 2 = 52$），我们得到p值为0.041。（注意，t_{52}分布与$N(0, 1)$分布几乎无法区分。）如果我们改用算法13.3，且$B = 10{,}000$，则得到的p值为0.042。图13.7展示了该基因的理论零分布、重采样零分布以及检验统计量的实际值（$T = -2.09$）。在这个例子中，我们看到使用理论零分布和重采样零分布得到的p值之间差异很小。

相比之下，图13.8展示了第877个基因的一组类似结果。在这种情况下，理论零分布与重采样零分布之间存在显著差异，导致它们的p值也有所不同。

一般来说，在样本量较小或数据分布更为偏斜的环境中（此时理论零分布准确性较低），重采样与理论{v*}值之间的差异往往更加显著。实际上，图13.8中重采样零分布与理论零分布之间存在较大差异，是因为第877个基因中的一个观测值与其他观测值相距甚远，导致分布极为偏斜。

13.5.2 A Re-Sampling Approach to the False Discovery Rate

现在，假设我们希望控制m个零假设$H_{01}, \ldots, H_{0m}$的错误发现率，在这种设定下，要么没有可用的理论零分布，要么我们只是倾向于避免使用理论零分布。



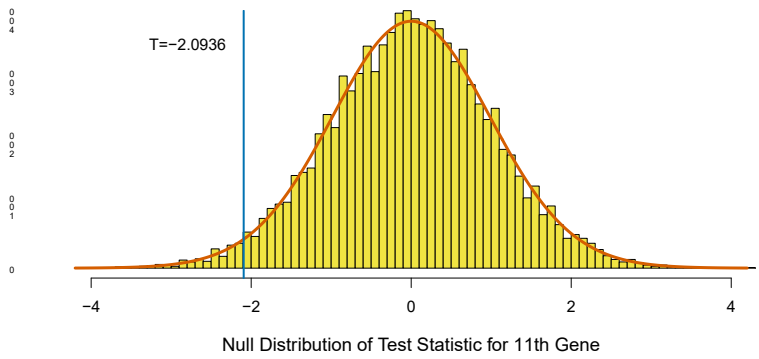


图13.7。 *The* 11*th gene in the* `Khan` *dataset has a test statistic of* $T = -2.09$. *Its theoretical and re-sampling null distributions are almost identical. The theoretical p-value equals* 0.041 *and the re-sampling p-value equals* 0.042.

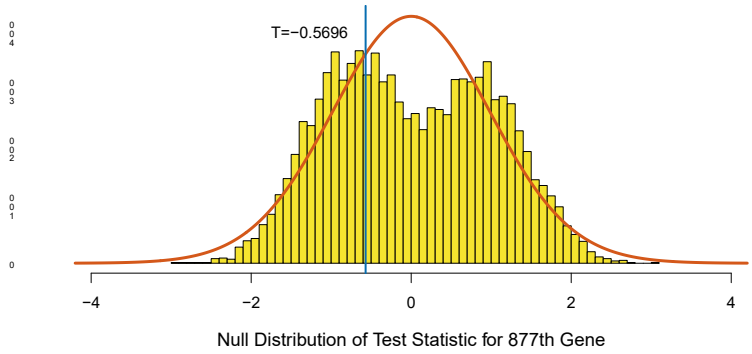


图13.8。 *The* 877*th gene in the* `Khan` *dataset has a test statistic of* $T = -0.57$. *Its theoretical and re-sampling null distributions are quite different. The theoretical p-value equals* 0.571, *and the re-sampling p-value equals* 0.673.

分布。如第13.5.1节所述，我们对每个假设利用双样本t统计量，从而得到检验统计量$T_1, \ldots, T_m$。我们可以简单地如第13.5.1节那样，为m个零假设分别计算p值，然后对这些p值应用第13.4.2节的Benjamini–Hochberg方法。然而，事实证明，我们可以采用更直接的方式，甚至无需计算p值即可完成。

回顾第 13.4 节，FDR 定义为 E(V/R)，使用表 13.2 中的记号。为了通过重采样估计 FDR，我们首先作如下近似：

$$\text{FDR} = E\left(\frac{V}{R}\right) \approx \frac{\text{E}(V)}{R}. \tag{13.13}$$

现在假设我们拒绝任何检验统计量的绝对值超过 c 的原假设。那么计算 (13.13) 式右侧分母中的 R 就很简单：$R = \sum_{j=1}^{m} 1_{(|T_j| \geq c)}$。

然而，(13.13) 式右侧的分子 E(V) 更具挑战性。这是与拒绝任何检验统计量绝对值超过 c 的原假设相关的预期假阳性数量。冒着陈述显而易见事实的风险，估计 V之所以具有挑战性，是因为我们不知道 $H_{01}, \ldots, H_{0m}$ 中哪些是真正为真的，因此我们不知道哪些被拒绝的假设是假阳性。为了克服这个问题，我们采用重抽样方法，在该方法中我们在 $H_{01}, \ldots, H_{0m}$ 下模拟数据，然后计算得到的检验统计量。超过 c 的重抽样检验统计量的数量提供了 V的估计。

更详细地说，对于双样本t统计量（13.11），针对每个零假设 $H_{01}, \ldots, H_{0m}$，我们可以如下估计E(V)。令$x_1^{(j)}, \ldots, x_{n_X}^{(j)}$和$y_1^{(j)}, \ldots, y_{n_Y}^{(j)}$表示与第$j$个零假设相关的数据，$j = 1, \ldots, m$。我们随机置换这些$n_X + n_Y$观测值，然后在置换后的数据上计算$t$统计量。对于这些置换后的数据，我们知道所有零假设$H_{01}, \ldots, H_{0m}$都成立；因此，超过阈值$c$的置换$t$统计量的数量（取绝对值后）提供了对E($V$)的估计。通过将置换过程重复$B$次（$B$取较大值）并对结果取平均，可以进一步改进这一估计。

算法13.4详细描述了这一过程。[18] 它提供了所谓的FDR的 *plug-in estimate*，因为(13.13)中的近似允许我们通过将R代入分母，并将E(V)的估计值代入分子来估计FDR。

我们将重采样方法应用于算法13.4中的FDR，以及使用理论p值的算法13.2中的Benjamini-Hochberg方法，应用于`Khan`数据集中的$m = 2{,}308$个基因。结果如图13.9所示。我们发现，对于给定数量的被拒绝假设，两种方法估计的FDR几乎相同。

我们在本节开始时指出，为了使用重采样方法控制m个假设检验的FDR，我们可以简单地按照第13.5.1节计算m个重采样p值，然后将第13.4.2节的Benjamini-Hochberg方法应用于这些p值。事实证明，如果我们将第j个重采样p值定义为

$$p_j = \frac{\sum_{j'=1}^{m} \sum_{b=1}^{B} 1_{(|T_{j'}^{*b}| \geq |T_j|)}}{Bm} \tag{13.14}$$

对于 $j = 1, \ldots, m$，而不是如(13.12)中那样，那么对这些重采样的 p 值应用Benjamini-Hochberg过程，*exactly* 等价于算法13.4。注意，(13.14)是(13.12)的一个替代方案，它在近似零分布时汇集了所有 m 假设检验的信息。

13.5.3 When Are Re-Sampling Approaches Useful?

在第13.5.1节和第13.5.2节中，我们考虑了使用两样本t统计量（13.11）来检验形式为H_0：E(X) = E(Y) 的原假设，对此我们

[18] To implement Algorithm 13.4 efficiently, the same set of permutations in Step 2(b)i. should be used for all m null hypotheses.

算法 13.4 *Plug-In FDR for a Two-Sample T-Te st*

1. 选择一个阈值 c，其中 $c > 0$。

2. 对于 $j = 1, \ldots, m$:

   (a) 计算$T^{(j)}$，即基于原始数据$x_1^{(j)}, \ldots, x_{n_X}^{(j)}$和$y_1^{(j)}, \ldots, y_{n_Y}^{(j)}$，针对零假设$H_{0j}$的双样本$t$统计量(13.11)。(b) 对于$b = 1, \ldots, B$，其中$B$是一个大数（例如$B = 10{,}000$）：i. 随机置换$n_X + n_Y$个观测值。称前$n_X$个观测值为$x_1^{*(j)}, \ldots, x_{n_X}^{*(j)}$，称剩余的观测值为$y_1^{*(j)}, \ldots, y_{n_Y}^{*(j)}$。ii. 在置换后的数据$x_1^{*(j)}, \ldots, x_{n_X}^{*(j)}$和$y_1^{*(j)}, \ldots, y_{n_Y}^{*(j)}$上计算(13.11)，并将结果称为$T^{(j),*b}$。

3. 计算 $R = \sum_{j=1}^{m} 1(|T^{(j)}| \geq c)$。

4. 计算 $\widehat{V} = \frac{\sum_{b=1}^{B} \sum_{j=1}^{m} 1\left(|T^{(j),*b}| \geq c\right)}{B}$。

5. 与阈值 c 相关的估计FDR为 $\widehat{V}/R$。

通过重新采样方法近似了零分布。我们发现，使用重新采样方法在图13.8中与使用理论p值方法得到的结果有显著差异，但在图13.7中则不然。

一般来说，有两种情况下重新采样方法特别有用:

1. 也许没有理论上的零分布可用。当你测试一个非典型的零假设 H_0，或使用非典型的检验统计量 T 时，可能就是这种情况。2. 也许一个理论上的零分布 *is* 是可用的，但其有效性所需的前提条件并不成立。例如，式(13.11)中的两样本 t 统计量仅在观测值呈正态分布时才遵循 $t_{n_X+n_Y-2}$ 分布。此外，只有当 n_X 和 n_Y 相当大时，它才遵循 $N(0, 1)$ 分布。如果数据非正态且 n_X 和 n_Y 较小，那么利用理论零分布得到的 p 值将无效（即它们不能正确控制第一类错误）。

一般来说，如果你能提出一种方法来重新采样或排列你的观测值，以生成符合零分布的数据，那么你就可以使用算法13.3和13.4的变体来计算p值或估计FDR。在许多现实世界的情境中，当没有现成的假设检验可用，或者当这些现成检验的关键假设被违反时，这提供了一种强大的假设检验工具。

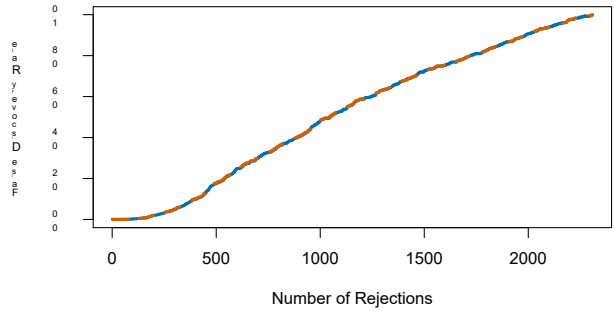


图13.9。 *For $j = 1, \ldots, m = 2{,}308$, we tested the null hypothesis that for the jth gene in the* `Khan` *dataset, the mean expression in Burkitt's lymphoma equals the mean expression in rhabdomyosarcoma. For each value of k from 1 to 2,308, the y-axis displays the estimated FDR associated with rejecting the null hypotheses corresponding to the k smallest p-values. The orange dashed curve shows the FDR obtained using the Benjamini–Hochberg procedure, whereas the blue solid curve shows the FDR obtained using the re-sampling approach of Algorithm 13.4, with $B = 10{,}000$. There is very little difference between the two FDR estimates. According to either estimate, rejecting the null hypothesis for the 500 genes with the smallest p-values corresponds to an FDR of around 17.7%.*

13.6 实验：多重检验

我们包含在之前实验中常见的导入。

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from ISLP import load_data
```

We also collect the new imports needed for this lab.

In [2]:
```
from scipy.stats import \
    (ttest_1samp,
     ttest_rel,
     ttest_ind,
     t as t_dbn)
from statsmodels.stats.multicomp import \
     pairwise_tukeyhsd
from statsmodels.stats.multitest import \
     multipletests as mult_test
```

13.6.1 Review of Hypothesis Tests

我们首先进行一些单样本t检验。首先，我们创建100个变量，每个变量包含10个观测值。前50个变量的均值为0.5，方差为1，而其余变量的均值为0，方差为1。

In [3]:
```
rng = np.random.default_rng(12)
X = rng.standard_normal((10, 100))
true_mean = np.array([0.5]*50 + [0]*50)
X += true_mean[None,:]
```

首先，我们使用来自 `scipy.stats` 模块的 `ttest_1samp()` 来检验 H_0: `ttest_1samp()` $\mu_1 = 0$，即第一个变量均值为零的零假设。

In [4]:
```
result = ttest_1samp(X[:,0], 0)
result.pvalue
```

Out[4]: 0.931

p 值为 0.931，这不足以在 $\alpha = 0.05$ 水平上拒绝零假设。在这种情况下，$\mu_1 = 0.5$，因此零假设为假。因此，当零假设为假时，我们未能拒绝零假设，从而犯了第二类错误。

我们现在测试 $H_{0,j}$： $\mu_j = 0$，对于 $j = 1, \ldots, 100$。我们计算这100个 p - 值，然后构造一个向量，记录第 j个 p-值是否小于或等于0.05，在这种情况下我们拒绝 H_{0j}，或者大于0.05，在这种情况下我们不拒绝 H_{0j}，对于 $j = 1, \ldots, 100$。

In [5]:
```
p_values = np.empty(100)
for i in range(100):
   p_values[i] = ttest_1samp(X[:,i], 0).pvalue
decision = pd.cut(p_values,
                  [0, 0.05, 1],
                  labels=['Reject H0',
                          'Do not reject H0'])
truth = pd.Categorical(true_mean == 0,
                       categories=[True, False],
                       ordered=True)
```

由于这是一个模拟数据集，我们可以创建一个类似于表13.2的2 × 2表格。

In [6]:
```
pd.crosstab(decision,
            truth,
     rownames=['Decision'],
     colnames=['H0'])
```

Out[6]:

H0	True	False
Decision		
Reject H0	5	15
Do not reject H0	45	35

因此，在水平 $\alpha = 0.05$ 下，我们拒绝了50个错误原假设中的15个，并且错误地拒绝了5个真实原假设。使用第13.3节的符号表示，我们有 $V = 5$, $S = 15$, $U = 45$ 和 $W = 35$。我们设定了 $\alpha = 0.05$，这意味着我们预期拒绝大约5%的真实原假设。这与上面的2 × 2表格一致，该表显示我们拒绝了50个真实原假设中的 $V = 5$个。

在上面的模拟中，对于错误的零假设，均值与标准差之比仅为 $0.5/1 = 0.5$。这相当于相当弱的信号，并导致了大量第二类错误。让我们改为用更强的信号模拟数据，使错误的零假设的均值与标准差之比等于 1。我们仅产生 10 个第二类错误。

In [7]:
```
true_mean = np.array([1]*50 + [0]*50)
X = rng.standard_normal((10, 100))
X += true_mean[None,:]
for i in range(100):
   p_values[i] = ttest_1samp(X[:,i], 0).pvalue
decision = pd.cut(p_values,
                  [0, 0.05, 1],
                  labels=['Reject H0',
                          'Do not reject H0'])
truth = pd.Categorical(true_mean == 0,
                       categories=[True, False],
                       ordered=True)
pd.crosstab(decision,
            truth,
            rownames=['Decision'],
            colnames=['H0'])
```

Out[7]:

H0	True	False
Decision		
Reject H0	2	40
Do not reject H0	48	10

13.6.2 Family-Wise Error Rate

回想（13.5），如果对于每个m独立假设检验，原假设都成立，那么FWER等于$1-(1-\alpha)^m$。我们可以使用这个表达式来计算FWER对于$m = 1, \ldots,$ 500和$\alpha = 0.05$、0.01以及0.001的情况。为了重现图13.2，我们绘制了这些α值下的FWER。

In [8]:
```
m = np.linspace(1, 501)
fig, ax = plt.subplots()
[ax.plot(m,
         1 - (1 - alpha)**m,
         label=r'$\alpha=%s$' % str(alpha))
         for alpha in [0.05, 0.01, 0.001]]
ax.set_xscale('log')
ax.set_xlabel('Number of Hypotheses')
ax.set_ylabel('Family-Wise Error Rate')
ax.legend()
ax.axhline(0.05, c='k', ls='--');
```

如前所述，即使对于m这样的中等值，如50，除非将α设置为非常低的值，如0.001，否则FWER会超过0.05。当然，将α设置为如此低的值的问题在于，我们很可能会犯许多II型错误：换句话说，我们的功效非常低。

我们现在对Fund数据集中的前五位经理分别进行单样本t检验，以检验第j位基金经理的平均收益等于零的原假设，即$H_{0,j}$：$\mu_j = 0$。

In [9]:
```
Fund = load_data('Fund')
fund_mini = Fund.iloc[:,:5]
fund_mini_pvals = np.empty(5)
for i in range(5):
```

```
    fund_mini_pvals[i] = ttest_1samp(fund_mini.iloc[:,i], 0).pvalue
fund_mini_pvals
```

```
Out[9]: array([0.006, 0.918, 0.012, 0.601, 0.756])
```

对于经理一和经理三，p值较低，而对于其他三位经理则较高。然而，我们不能简单地拒绝$H_{0,1}$和$H_{0,3}$，因为这样做未能考虑我们所进行的多重测试。相反，我们将采用Bonferroni方法和Holm方法来控制FWER。

为此，我们使用了来自 `statsmodels` 的 `multipletests()` 函数。模块（缩写为 `mult_test()`）。给定 p 值，对于 Holm 和 Bonferroni 等方法，函数输出 *adjusted p-values*，这可以被视为一组经过多重检验校正的新 p 值。如果给定假设的校正 p 值小于或等于 α，那么可以在保持 FWER 不超过 α 的情况下拒绝该假设。换句话说，对于此类方法，只需将由 `multipletests()` 函数得到的校正 p 值与所需 FWER 进行比较，即可确定是否拒绝每个假设。我们稍后会看到，我们也可以使用同一个函数来控制 FDR。

`multipletests()`

调整后的 p值

`mult_test()` 函数接收 p 值和一个 `method` 参数，以及一个可选的 `alpha` 参数。它返回决策（下文的 `reject`）以及调整后的 p 值（`bonf`）。

```
In [10]: reject, bonf = mult_test(fund_mini_pvals, method = "bonferroni")[:2]
reject
```

```
Out[10]: array([ True, False, False, False, False])
```

p 值 `bonf` 就是将 `fund_mini_pvalues` 乘以 5 并截断至小于或等于 1 的结果。

```
In [11]: bonf, np.minimum(fund_mini_pvals * 5, 1)
```

```
Out[11]: (array([0.03, 1.  , 0.06, 1.  , 1.  ]),
          array([0.03, 1.  , 0.06, 1.  , 1.  ]))
```

因此，使用Bonferroni方法，我们只能在控制FWER在0.05的情况下拒绝Manager One的原假设。

相比之下，使用Holm方法时，调整后的 p 值表明我们可以在0.05的FWER水平上拒绝经理一和经理三的零假设。

```
In [12]: mult_test(fund_mini_pvals, method = "holm", alpha=0.05)[:2]
```

```
Out[12]: (array([ True, False,  True, False, False]),
          array([0.03, 1.  , 0.05, 1.  , 1.  ]))
```

如前所述，经理一似乎表现特别好，而经理二的表现不佳。

```
In [13]: fund_mini.mean()
```

```
Out[13]: Manager1    3.0
         Manager2   -0.1
         Manager3    2.8
         Manager4    0.5
         Manager5    0.3
         dtype: float64
```

这两位管理者之间的表现是否存在有意义的差异的证据？我们可以通过使用配对的t-t执行*paired t-test*来检验这一点。
`ttest_rel()` 来自 `scipy.stats` 的函数：

东

`ttest_rel()`

```
In [14]: ttest_rel(fund_mini['Manager1'],
                   fund_mini['Manager2']).pvalue
```

```
Out[14]: 0.038
```

测试结果得出 p 值为 0.038，表明存在统计学显著差异。

然而，我们决定在检查数据并注意到经理一和经理二分别拥有最高和最低的平均表现后才进行这项测试。从某种意义上说，这意味着我们隐式地进行了 $\binom{5}{2} = 5(5-1)/2 = 10$ 次假设检验，而不仅仅是一次，正如第13.3.2节所讨论的。因此，我们使用了来自

`statsmodels.stats.multicomp` 为了应用Tukey方法以调整多重测试，此函数接受一个拟合的*ANOVA*回归-

`pairwise_tukeyhsd()`

方差分析

信号模型，本质上仅是一个线性回归，其中所有预测变量都是定性的。在此情况下，响应变量由每位经理实现的月度超额回报组成，而预测变量则指示每个回报对应的经理。

```
In [15]: returns = np.hstack([fund_mini.iloc[:,i] for i in range(5)])
         managers = np.hstack([[i+1]*50 for i in range(5)])
         tukey = pairwise_tukeyhsd(returns, managers)
         print(tukey.summary())
```

```
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====================================================
group1 group2 meandiff p-adj   lower  upper  reject
-----------------------------------------------------
     1      2     -3.1 0.1862 -6.9865 0.7865  False
     1      3     -0.2 0.9999 -4.0865 3.6865  False
     1      4     -2.5 0.3948 -6.3865 1.3865  False
     1      5     -2.7 0.3152 -6.5865 1.1865  False
     2      3      2.9 0.2453 -0.9865 6.7865  False
     2      4      0.6 0.9932 -3.2865 4.4865  False
     2      5      0.4 0.9986 -3.4865 4.2865  False
     3      4     -2.3  0.482 -6.1865 1.5865  False
     3      5     -2.5 0.3948 -6.3865 1.3865  False
     4      5     -0.2 0.9999 -4.0865 3.6865  False
-----------------------------------------------------
```

`pairwise_tukeyhsd()` 函数为每对管理者（`lower` 和 `upper`）之间的差异提供置信区间，以及一个

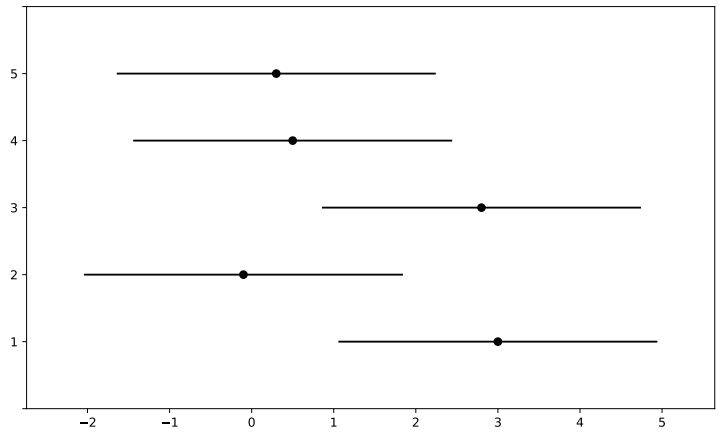


图13.10. *95% confidence intervals for each manager on the* `Fund` *data, using Tukey's method to adjust for multiple testing. All of the confidence intervals overlap, so none of the differences among managers are statistically significant when controlling FWER at level* 0.05.

p-值。所有这些量都已针对多重检验进行了调整。注意，经理一和经理二之间差异的p值已从0.038增加到0.186，因此不再有明显证据表明两位经理的表现存在差异。我们可以使用`tukey`的`plot_simultaneous()`方法绘制成对比较的置信区间。任何不重叠的区间对都表明在0.05的名义水平上存在显著差异。在这种情况下，如上表所示，没有差异被认为是显著的。

In [16]:
```
fig, ax = plt.subplots(figsize=(8,8))
tukey.plot_simultaneous(ax=ax);
```

结果可见于[19]图13.10。

13.6.3 False Discovery Rate

现在我们对`Fund`数据集中的所有2，000名基金经理进行假设检验。我们执行一个单样本t检验，原假设为$H_{0,j}$：μ_j ==0，即第j位基金经理的平均回报率为零。

In [17]:
```
fund_pvalues = np.empty(2000)
for i, manager in enumerate(Fund.columns):
    fund_pvalues[i] = ttest_1samp(Fund[manager], 0).pvalue
```

有太多的管理者需要考虑控制FWER。相反，我们专注于控制FDR：即被拒绝的原假设中实际为假阳性的预期比例。

[19] Traditionally this plot shows intervals for each paired difference. With many groups it is more convenient and equivalent to display one interval per group, as is done here. By "differencing" all pairs of intervals displayed here you recover the traditional plot.

`multipletests()` 函数（缩写为 `mult_test()`）可用于执行 Benjamini–Hochberg 过程。

In [18]:
```
fund_qvalues = mult_test(fund_pvalues, method = "fdr_bh")[1]
fund_qvalues[:10]
```

Out[18]:
```
array([0.09, 0.99, 0.12, 0.92, 0.96, 0.08, 0.08, 0.08, 0.08,
       0.08])
```

Benjamini–Hochberg 过程输出的 *q-values* 可以进行相互可以将其解释为我们拒绝某一特定原假设的最小FDR阈值。例如，q-值为0.1意味着我们可以在10%或更高的FDR下拒绝相应的原假设，但在低于10%的FDR下不能拒绝该原假设。 q-values

如果我们将FDR控制在10%，那么对于多少位基金经理，我们可以拒绝 $H_{0,j}$：$\mu_j = 0$?

In [19]:
```
(fund_qvalues <= 0.1).sum()
```

Out[19]:
```
146
```

我们发现，在2000名基金经理中，有146名的q值低于0.1；因此，我们可以得出结论，在10%的FDR下，有146名基金经理战胜了市场。这些基金经理中，仅有约15名（146的10%）可能是误报。

相比之下，如果我们改用Bonferroni方法来控制FWER在α =0.1水平上，那么我们本将无法拒绝任何原假设！

In [20]:
```
(fund_pvalues <= 0.1 / 2000).sum()
```

Out[20]:
```
0
```

图13.6展示了Fund数据集的排序后的p值，$p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(2000)}$，以及Benjamini–Hochberg程序的拒绝阈值。回想一下，Benjamini–Hochberg程序识别最大的p值，使得$p_{(j)} < qj/m$，并拒绝所有p值小于或等于$p_{(j)}$的假设。在下面的代码中，我们自己实现了Benjamini–Hochberg程序，以说明其工作原理。我们首先对p值进行排序。然后我们识别所有满足 $p_{(j)} < qj/m$ (`sorted_set_`)的p值。最后，`selected_`是一个布尔数组，指示哪些p值小于或等于`sorted_[sorted_set_]`中的最大p值。因此，`selected_`索引了被Benjamini–Hochberg程序拒绝的p值。

In [21]:
```
sorted_ = np.sort(fund_pvalues)
m = fund_pvalues.shape[0]
q = 0.1
sorted_set_ = np.where(sorted_ < q * np.linspace(1, m, m) / m)[0]
if sorted_set_.shape[0] > 0:
    selected_ = fund_pvalues < sorted_[sorted_set_].max()
    sorted_set_ = np.arange(sorted_set_.max())
else:
    selected_ = []
    sorted_set_ = []
```

我们现在重现图13.6的中间面板。

In [22]:
```
fig, ax = plt.subplots()
ax.scatter(np.arange(0, sorted_.shape[0]) + 1,
           sorted_, s=10)
ax.set_yscale('log')
ax.set_xscale('log')
ax.set_ylabel('P-Value')
ax.set_xlabel('Index')
ax.scatter(sorted_set_+1, sorted_[sorted_set_], c='r', s=20)
ax.axline((0, 0), (1,q/m), c='k', ls='--', linewidth=3);
```

13.6.4 A Re-Sampling Approach

这里，我们使用在13.5节中研究的Khan数据集，实施重采样方法进行假设检验。首先，我们将训练数据和测试数据合并，得到83位患者的2,308个基因的观察结果。

In [23]:
```
Khan = load_data('Khan')
D = pd.concat([Khan['xtrain'], Khan['xtest']])
D['Y'] = pd.concat([Khan['ytrain'], Khan['ytest']])
D['Y'].value_counts()
```

Out[23]:
```
2    29
4    25
3    18
1    11
Name: Y, dtype: int64
```

癌症有四类。对于每个基因，我们将第二类（横纹肌肉瘤）中的平均表达量与第四类（伯基特淋巴瘤）中的平均表达量进行比较。使用来自 scipy.stats 的 ttest_ind() 对第11个基因进行标准双样本 t 检验，得到一个
检验统计量为 -2.09，相应的 p 值为 0.0412，表明两种癌症类型之间平均表达水平存在差异的证据有限。 ttest_ind()

In [24]:
```
D2 = D[lambda df:df['Y'] == 2]
D4 = D[lambda df:df['Y'] == 4]
gene_11 = 'G0011'
observedT, pvalue = ttest_ind(D2[gene_11],
                              D4[gene_11],
                              equal_var=True)
observedT, pvalue
```

Out[24]:
```
(-2.094, 0.041)
```

然而，这个p值依赖于这样一个假设：在两组之间无差异的零假设下，检验统计量服从自由度为$29 + 25 - 2 = 52$的t分布。我们可以不采用这个理论零分布，而是将54名患者随机分为29人和25人的两组，并计算一个新的检验统计量。在组间无差异的零假设下，这个新的检验统计量应该与原始检验统计量具有相同的分布。重复这一

处理10000次允许我们近似检验统计量的零分布。我们计算观察到的检验统计量超过通过重采样获得的检验统计量的次数比例。

In [25]:
```
B = 10000
Tnull = np.empty(B)
D_ = np.hstack([D2[gene_11], D4[gene_11]])
n_ = D2[gene_11].shape[0]
D_null = D_.copy()
for b in range(B):
    rng.shuffle(D_null)
    ttest_ = ttest_ind(D_null[:n_],
                       D_null[n_:],
                       equal_var=True)
    Tnull[b] = ttest_.statistic
(np.abs(Tnull) > np.abs(observedT)).mean()
```

Out[25]: 0.0398

这个分数，0.0398，是我们基于重采样的 p 值。它与使用理论零分布得到的 p 值 0.0412 几乎相同。我们可以绘制基于重采样的检验统计量的直方图，以重现图 13.7。

In [26]:
```
fig, ax = plt.subplots(figsize=(8,8))
ax.hist(Tnull,
        bins=100,
        density=True,
        facecolor='y',
        label='Null')
xval = np.linspace(-4.2, 4.2, 1001)
ax.plot(xval,
        t_dbn.pdf(xval, D_.shape[0]-2),
        c='r')
ax.axvline(observedT,
           c='b',
           label='Observed')
ax.legend()
ax.set_xlabel("Null Distribution of Test Statistic");
```

基于重采样的零分布几乎与理论零分布完全相同，后者以红色显示。

最后，我们实现算法13.4中概述的插件重采样FDR方法。根据您计算机的速度，为Khan数据集中的所有2，308个基因计算FDR可能需要一段时间。因此，我们将在一个包含100个基因的随机子集上演示该方法。对于每个基因，我们首先计算观察到的检验统计量，然后生成10，000个重采样的检验统计量。这可能需要几分钟来运行。如果您时间紧迫，那么可以将B设置为一个较小的值（例如B=500）。

In [27]:
```
m, B = 100, 10000
idx = rng.choice(Khan['xtest'].columns, m, replace=False)
T_vals = np.empty(m)
Tnull_vals = np.empty((m, B))

for j in range(m):
    col = idx[j]
```

```
    T_vals[j] = ttest_ind(D2[col],
                          D4[col],
                          equal_var=True).statistic
    D_ = np.hstack([D2[col], D4[col]])
    D_null = D_.copy()
    for b in range(B):
        rng.shuffle(D_null)
        ttest_ = ttest_ind(D_null[:n_],
                           D_null[n_:],
                           equal_var=True)
        Tnull_vals[j,b] = ttest_.statistic
```

接下来，我们计算被拒绝的零假设数量 R、估计的假阳性数量 $\widehat{V}$ 以及估计的 FDR，对于算法 13.4 中的一系列阈值 c。阈值的选择基于来自 100 个基因的检验统计量的绝对值。

In [28]:
```
cutoffs = np.sort(np.abs(T_vals))
FDRs, Rs, Vs = np.empty((3, m))
for j in range(m):
   R = np.sum(np.abs(T_vals) >= cutoffs[j])
   V = np.sum(np.abs(Tnull_vals) >= cutoffs[j]) / B
   Rs[j] = R
   Vs[j] = V
   FDRs[j] = V / R
```

现在，对于任何给定的FDR，我们都能找出将被拒绝的基因。例如，当FDR控制在0.1时，我们在100个零假设中拒绝了15个。平均而言，我们预期这些基因中大约有一到两个（即15个中的10%）会是假发现。在FDR为0.2时，我们可以对28个基因拒绝零假设，其中我们预计大约六个会是假发现。

变量`idx`存储了我们随机选择的100个基因中包含哪些基因。让我们看看估计FDR小于0.1的基因。

In [29]:
```
sorted(idx[np.abs(T_vals) >= cutoffs[FDRs < 0.1].min()])
```

在FDR阈值为0.2时，虽然选出了更多的基因，但代价是预期假发现的比例更高。

In [30]:
```
sorted(idx[np.abs(T_vals) >= cutoffs[FDRs < 0.2].min()])
```

下一行生成图13.11，该图与图13.9相似，但仅基于基因的一个子集。

In [31]:
```
fig, ax = plt.subplots()
ax.plot(Rs, FDRs, 'b', linewidth=3)
ax.set_xlabel("Number of Rejections")
ax.set_ylabel("False Discovery Rate");
```

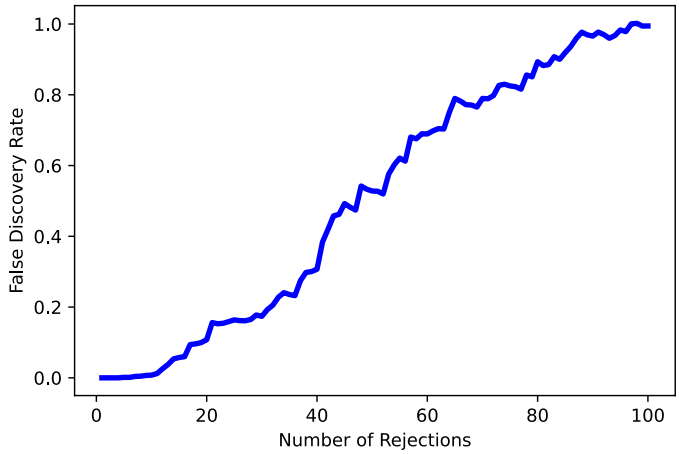


图 13.11. *The estimated false discovery rate versus the number of rejected null hypotheses, for 100 genes randomly selected from the* `Khan` *dataset.*

13.7 练习

Conceptual

1. 假设我们检验 m 个原假设，所有原假设均为真。我们在水平 α 下控制每个原假设的第一类错误。对于每个子问题，请证明你的答案。

(a) 总共预计会犯多少I型错误？(b) 假设我们进行的m次测试是独立的。这些m次测试相关的家族错误率是多少？
Hint: If two events A and B are independent, then $\Pr(A \cap B) = \Pr(A$) $\Pr(B)$.c) 假设m =为2，并且这两个测试的{p}值正相关，即一个值小则另一个也倾向于小，一个值大则另一个也倾向于大。这些m =为2的测试的家族错误率与(b)中m =为2的答案在性质上如何比较？
Hint: First, suppose that the two p-values are perfectly correlated. (d) 再次假设m =为2，但此时这两个测试的{p}值负相关，即一个值大则另一个倾向于小。这些m =为2的测试的家族错误率与(b)中m =为2的答案在性质上如何比较？
Hint: First, suppose that whenever one p-value is less than α, then the other will be greater than α. In other words, we can never reject both null hypotheses.

2. 假设我们检验 m 个假设，并在水平 α 上控制每个假设的第一类错误。假设所有 m p 值都是独立的，并且所有零假设都是真的。

(a) 令随机变量 A_j 等于1，如果第 j 个零假设被拒绝，否则为0。A_j 的分布是什么？(b) $\sum_{j=1}^{m} A_j$ 的分布是什么？(c) 我们将犯的第一类错误数量的标准差是多少？

3. 假设我们检验 m 个零假设，并对于 $j = 1, \ldots, m$，将第 j 个零假设的I型错误控制于水平 α_j。论证家庭-wise错误率不大于 $\sum_{j=1}^{m} \alpha_j$。

Null Hypothesis	p-value
H_{01}	0.0011
H_{02}	0.031
H_{03}	0.017
H_{04}	0.32
H_{05}	0.11
H_{06}	0.90
H_{07}	0.07
H_{08}	0.006
H_{09}	0.004
H_{10}	0.0009

表 13.4. *p-values for Exercise 4.*

4. 假设我们检验了m =10个假设，并得到了表13.4所示的p值。

(a) 假设我们希望在每个原假设上将第一类错误控制在α = 0.05的水平。我们会拒绝哪些原假设？(b) 现在假设我们希望在α = 0.05的水平上控制FWER。我们会拒绝哪些原假设？请证明你的答案。(c) 现在假设我们希望在q = 0.05的水平上控制FDR。我们会拒绝哪些原假设？请证明你的答案。(d) 现在假设我们希望在q = 0.2的水平上控制FDR。我们会拒绝哪些原假设？请证明你的答案。(e) 在FDR水平q = 0.2下被拒绝的原假设中，大约有多少是假阳性？请证明你的答案。

5. 对于这个问题，您将使用Bonferroni和Holm程序来构造导致一定数量拒绝的p值。

(a) 给出五个 p 值的例子（即五个介于0和1之间的数，针对本题目的目的，我们将它们解释为 p 值），对于这些值，Bonferroni方法和Holm方法同时

在控制FWER在0.1水平时，恰好拒绝一个零假设。

(b) 现在给出一个五个 p 值的例子，使得在水平 0.1 下，Bonferroni 方法拒绝一个原假设，而 Holm 方法拒绝多于一个原假设。

6. 对于图13.3中的三个面板，分别回答以下问题:

(a) 在水平 $\alpha = 0.05$ 下应用 Bonferroni 程序控制 FWER 时，会产生多少假阳性、假阴性、真阳性、真阴性、第一类错误和第二类错误？(b) 在水平 $\alpha = 0.05$ 下应用 Holm 程序控制 FWER 时，会产生多少假阳性、假阴性、真阳性、真阴性、第一类错误和第二类错误？(c) 使用 Bonferroni 程序在水平 $\alpha = 0.05$ 下控制 FWER 时，错误发现比例是多少？(d) 使用 Holm 程序在水平 $\alpha = 0.05$ 下控制 FWER 时，错误发现比例是多少？(e) 如果我们改用 Bonferroni 程序在水平 $\alpha = 0.001$ 下控制 FWER，那么 (a) 和 (c) 的答案会如何变化?

Applied

7. 本题使用 `ISLP` 包中的 `Carseats` 数据集。

(a) 对于数据集中除 `Sales` 外的每个定量变量，拟合一个线性模型来预测 `Sales`，使用该定量变量。报告与变量系数相关的 p 值。也就是说，对于形式为 $Y = \beta_0 + \beta_1 X + \epsilon$ 的每个模型，报告与系数 β_1 相关的 p 值。这里，Y 代表 `Sales`，X 代表其他定量变量之一。(b) 假设我们对 (a) 中获得的 p 值在水平 $\alpha = 0.05$ 上控制第一类错误。我们拒绝哪些原假设？(c) 现在假设我们对 p 值在水平 0.05 上控制 FWER。我们拒绝哪些原假设？(d) 最后，假设我们对 p 值在水平 0.2 上控制 FDR。我们拒绝哪些原假设?

8. 在这个问题中，我们将模拟来自$m = 100$位基金经理的数据。

```
rng = np.random.default_rng(1)
n, m = 20, 100
X = rng.normal(size=(n, m))
```

这些数据代表了n =位基金经理在20个月中的百分比回报率。我们希望检验每个基金经理的百分比回报率的总体均值等于零这一原假设。请注意，我们模拟数据的方式使得每位基金经理的百分比回报率总体均值确实为零；换句话说，所有m个原假设都是真的。

(a) 对每位基金经理进行单样本 t 检验，并绘制所得 p 值的直方图。(b) 若我们在显著性水平 $\alpha = 0.05$ 下对每个原假设控制第一类错误，那么我们会拒绝多少个原假设？(c) 若我们在显著性水平 0.05 下控制 FWER，那么我们会拒绝多少个原假设？(d) 若我们在显著性水平 0.05 下控制 FDR，那么我们会拒绝多少个原假设？(e) 现在假设我们“挑选”出在数据中表现最好的 10 位基金经理。若我们仅对这 10 位基金经理在显著性水平 0.05 下控制 FWER，那么我们会拒绝多少个原假设？若我们仅对这 10 位基金经理在显著性水平 0.05 下控制 FDR，那么我们会拒绝多少个原假设？(f) 解释为什么 (e) 中的分析具有误导性。

Hint: The standard approaches for controlling the FWER and FDR assume that 所有 *tested null hypotheses are adjusted for multiplicity, and that no "cherry-picking" of the smallest p-values has occurred. What goes wrong if we cherry-pick?*

索引

准确率，415 激活，400 激活函数，401 加法，11，94–98，110–111 可加性，305，306 调整后的 R^2，87，231，232，236–238 `Advertising` 数据集，15，16，19，69，71–73，77，78，80，82，83，85，87–90，95，96，109–111 凝聚聚类，525 赤池信息量准则，87，231，232，236–238 备择假设，76，559 方差分析，312 ANOVA，587 曲线下面积，155，486–487 参数，40 数组，42 属性，42 AUC，155 `Auto` 数据集，12，66，98–101，129，197，202–207，327，398 自相关，421 自回归，423 坐标轴，48

反向传播，429 后向逐步选择，87，234–235 n元词袋，415 词袋，414 袋装法，11，24，331，343–346，354，360–361 BART，343，350，353，354，362–363 基线，93，145，161 基函数，293–294，296 贝叶斯分类器，35–37，147 决策边界，148 误差，35–37 贝叶斯定理，146，250 贝叶斯方法，250–251，353 贝叶斯加性回归树，331，343，350，350，353，354，362–363 贝叶斯信息准则，87，231，232，236–238 本杰明–霍赫伯格过程，575–577 伯努利分布，172 最佳子集选择，231，246 偏差，31–34，74，90，159，405 偏差-方差分解，32

回填，307，328

© Springer Nature Switzerland AG 2023

G. James et al., *An Introduction to Statistical Learning*, Springer Texts in Statistics,
https://doi.org/10.1007/978-3-031-38747-0

权衡，31–34，38，111–112，157，159，163，164，242，254，263，266，301，336，376，385
双向，425
Bikeshare 数据集，12，167–172
二元，27，138
双标图，507，508
Bonferroni 方法，575–577，585
布尔，53，176
提升，11，24，331，343，347–350，354，361–362
自助法，11，201，212–214，343
Boston 数据集，12，67，117，122，133，199，227，287，327，364，556
自底向上聚类，525
箱线图，62
BrainCancer 数据集，12，472–474，476，482
分支，333
预热，352
C 指数，487
Caravan 数据集，12，184，366
Carseats 数据集，12，126，130，364
分类，2，27
删失数据，469–502
删失
独立，471
区间，471
左，471
机制，471
非信息性，471
右，471
时间，470
链式法则，429
通道，407
CIFAR100 数据集，406，409–411，448，449
分类，2，11，27，34–39，135–199，367–382
错误率，338
树，337–341，355–358
分类器，135
聚类分析，25–26
聚类，4，25–26，520–535
凝聚，525
自下而上，525
层次结构，521，525–535
K-均值，11，521–524
Cochran–Mantel–Haenszel 检验，475
系数，71
College 数据集，12，65，286，328
共线性，106–110
拼接，41
条件概率，35
置信区间，75–76，90，110，292
混杂，144
混淆矩阵，153，176
连续，2
等高线，246
等高线图，50
对比，94
便捷函数，53
卷积滤波器，407
卷积层，407
卷积神经网络，406–413
相关性，79，82–83，530
计数数据，167，170
Cox 比例风险模型，480，483–486
C_p，87，231，232，236–238
Credit 数据集，12，91，92，94，97，98，106–109
交叉熵，405
交叉验证，11，31，34，201–211，231，252，270
k-折，206–209
留一法，204–206
维度灾难，115，193，266
数据增强，411
数据框，55
数据集
Advertising，15、16、19、69、71–73、77、78、80、82、83、85、87–90、95、96、109–111
Auto，12、66、98–101、129、197、202–207、327、398
Bikeshare，12、167–172

Boston, 12, 67, 117, 122, 133, 19 9, 227, 287, 327, 364, 556 BrainCancer, 12, 472–474, 476, 482 Caravan, 12, 184, 366 Carseats, 12, 126, 130, 364 CIFAR100, 406, 409–411, 448, 4 49 College, 12, 65, 286, 328 Credit, 12, 91, 92, 94, 97, 98, 10 6–109 Default, 12, 136–139, 14 1– 144, 152–156, 160, 161, 225, 226, 466 Fund, 12, 567–570, 572 , 575, 576, 585, 588, 589 Heart, 339, 340, 344–347, 352, 353, 382 , 383 Hitters, 12, 332, 333, 336 , 338, 339, 366, 425, 426, 437, 44 6 IMDb, 413, 415, 416, 418, 420, 437, 458, 467 Income, 16–18, 21 –23 Khan, 12, 579–581, 583, 590 , 593 MNIST, 402–404, 406, 430, 431, 441, 444, 445, 448 NCI60, 4, 5, 12, 546, 548–550 NYSE, 12, 42 2–424, 466, 467 OJ, 12, 365, 398 Portfolio, 12 Publication, 1 2, 482–487 Smarket, 2, 3, 12, 17 3, 184, 196 USArrests, 12, 507, 508, 510, 512, 513, 515, 516, 518 , 519 Wage, 1, 2, 8, 9, 12, 290, 29 1, 293, 295, 297–300, 302– 306, 309, 315, 327 Weekly, 12, 196, 2 26

Default 数据集，12，136–139，14 1–144，152–156，160，161，225，226，466 自由度，30，266，295，296，301 树状图，521，525–530 密度函数，146 因变量，15 导数，296 ，300 检测器层，410 偏差，232 字典，66 降维，230，253–262 判别函数，149 判别方法，146–161 相异性 ，530–532 基于距离相关，530–532 ，554 欧几里得，509，522，523，529–532 双重下降，431–435 双指数分布，251 丢弃，406，431 虚拟变量，91–94，138，142，292 早停，430 有效自由度，301 特征分解，50 6，516 肘部，548 嵌入，418 嵌入层，419 集成，343–354 熵，337–33 9，363 轮次，430 误差不可约，17 ，30 率，34 可约，17 项，16 欧几里得距离，509，522，523，529–53 2，554 事件时间，470 异常，45 期望值，18 探索性数据分析，504 指数，173 指数族，173

dat 一个类型，42
dec视觉函数，387
decision tree, 11, 331–342
deep学习，399

F统计量，84

因子，92 阶乘，170 失败时间，470 错误发现比例，155，573 发现率，558，573–577，579–582 阴性，155，562 阳性，155，562，563 阳性率，155，156，382 族错误率，565–573，577 特征，15 特征图，406 特征选择，230 特征化，414 前馈神经网络，400 图，48 拟合，21 拟合值，101 展平，424 灵活，21 浮点，43 前向逐步选择，86，87，233–234，268 函数，40 Fund 数据集，12，567–570，572，575，576，585，588，589 Gamma，173 高斯（正态）分布，146，147，150，172，561 广义加性模型，5，24，162，289，290，305–309，319 广义线性模型，5，135，167–174，217 生成模型，146–161 基尼指数，337–339，345，346，363 全局最小值，427 梯度，428 梯度下降，427 Harrell 一致性指数，487 风险函数，476–478 基线，478 风险率，476 Heart 数据集，339，340，344–347，352，353，382，383 热图，50 辅助函数，311 异方差性，103，168 隐藏层，400 隐藏单元，400 层次聚类，525–530 树状图，525–528 反转，529 连接，529–530 层次原则，96 高维，86，234，263 合页损失，385 Hitters 数据集，12，332，333，336，338，339，366，425，426，437，446 留出集，202 Holm 方法，568，576，585 超几何分布，501 超参数，187 超平面，367–372 假设检验，76–77，84，103，558–583 IMDb 数据集，413，415，416，418，420，437，458，467 插补，515 Income 数据集，16–18，21–23 增量，60 自变量，15 可索引，186 指示函数，292 推断，17，18 内积，379，380 输入层，400 输入变量，15 积分，301 交互，70，89，95–98，110– 111，308 截距，71，72 插值，432 可解释性，229 反转，529 不可约误差，17，36，90，110 迭代器，312

联合分布，158

K-means 聚类，11，521–524 K-最近邻，135，164–167 分类器，11，36–37 回归，111–115

Kaplan–Meier 生存曲线，472–474，483 核，379–382，384，394 线性，380 非线性，377–382 多项式，380，382 径向，381–383，390 核密度估计器，159 关键字，46 Khan 数据集，12，579–581，583，590，593 结点，290，294，296–299

ℓ_1 范数, 244 ℓ_2 范数, 242 滞后, 422 拉普拉斯分布, 251 套索, 11, 24, 244–251, 265–266, 336, 385, 484 叶, 333, 526 学习率, 429 最小二乘, 5, 21, 71–72, 140, 141, 229 线, 73 加权, 103 水平, 92 杠杆, 104–106 似然函数, 141 线性, 2, 69–115 线性组合, 128, 230, 253, 505 线性判别分析, 5, 11, 135, 138, 147–155, 164–167, 377, 382 线性核, 380 线性模型, 20, 69–115 线性回归, 5, 11, 69–115, 172–173 多元, 80–90 简单, 70–80 链接函数, 172, 173 连接, 529–530, 548 平均, 529–530 质心, 529–530 完全, 526, 529–530 单一, 529–530 列表, 41 列表推导, 123 局部最小值, 427

局部回归，290 对数几率，145 对数秩检验，474–476，483 逻辑函数，139 逻辑回归，5，11，25，135，138–144，164–167，172–173，308–309，377，384–385 多项，145，163 多重，142–144 logit，140 损失函数，300，385 低维，262 LSTM RNN，420 主效应，96 多数投票，344 Mallow’s C_p，87，231，232，236–238 Mantel–Haenszel 检验，475 间隔，370，385 边际分布，158 马尔可夫链蒙特卡洛，353 矩阵补全，515 矩阵乘法，10 最大间隔分类器，367–372 超平面，370 最大似然，139–141，143，170 均方误差，28 网格，53 方法，43 小批量，429 误分类误差，35 随机缺失，515 缺失数据，56，515–520 混合选择，87 MNIST 数据集，402–404，406，430，431，441，444，445，448 模型评估，201 模型选择，201 模块，42 多重共线性，108，266 多项逻辑回归，145，163 多重检验，557–583 多任务学习，403 多元高斯，150

多元正态，150 朴素贝叶斯，135, 158–161, 164– 167 命名空间，116 自然样条，297, 298, 301, 317 NCI60 数据集，4, 5, 12, 546, 548– 550 负二项，173 阴性预测值，155, 156 神经网络，5, 399 节点内部，333 纯度，337–339 终端，333 噪声，21, 252 非线性，2, 11, 289–329 决策边界，377–382 核，377–382 非参数，20, 22–23, 111– 115, 193 正态（高斯）分布，146, 147, 150, 172, 476, 561 笔记本，40 零，152 分布，561, 578 假设，76, 559 模型，87, 231, 245 零率，186 NYSE 数据集，12, 422–424, 466, 467 奥卡姆剃刀，426 几率，140, 145, 195 OJ 数据集，12, 365, 398 独热编码，92, 126, 403 单标准误规则，240 一对多，384 一对一，384 一对其余，384 最优分离超平面，370 训练误差的乐观性，30 有序分类变量，315 正交，257, 506 基，125 袋外，345 离群值，103–104 输出变量，15 过度参数化，465 过度离散，172 过拟合, 21, 23, 25, 30–31, 88, 152, 233, 371 p值, 77, 82, 560–562, 578–579 调整后, 586 包, 42 参数, 71 参数化, 20–22, 111–115 偏最小二乘, 254, 260–262, 282 偏似然, 480 路径算法, 249 置换, 578 置换方法, 577–582 垂直, 257 泊松分布, 169, 172 泊松回归, 135, 167–173 多项式核, 380, 382 回归, 98–99, 289–292, 294– 295 池化, 410 总体回归线, 73 Portfolio 数据集, 12 阳性预测值, 155, 156 后验分布, 251 众数, 251 概率, 147 功效, 108, 155, 563 精确率, 155 预测, 17 区间, 90, 110 预测变量, 15 主成分, 505 分析, 11, 254–260, 504–515 载荷向量, 505, 506 缺失值, 515–520 方差解释比例, 510–515, 547 回归, 11, 254–260, 280– 282, 504, 515 得分向量, 506 碎石图, 514–515 先验分布, 251 概率, 146 概率密度函数, 477, 478

p投影，230
p比例风险假设，478

p运行，336 成本复杂度
，336 最弱环节，336

Publication 数据集，12，482–
487
P Python 对象和函数 %%capture，458
iloc[]，58 loc[]，57
AgglomerativeClustering()，543
anova()，313 anova_lm()，125，129
，312，313 axhline()，122，551
axline()，121，129，329 BART()，36
2 biplot，537 boot_SE()，223
boxplot()，62，66 bs()，315，327
BSpline()，315 clone()，222
columns.drop()，122
compute_linkage()，544
confusion_table()，176 contour()，
50 corr()，129，174
cost_complexity_pruning_path()，3
57 CoxPHFitter()，491
cross_val_predict()，270
cross_validate()，218，219，226
cumsum()，539 cut_tree()，545
data.frame()，227 Dataset，440
decision_function()，392
DecisionTreeClassifier()，354，35
5 DecisionTreeRegressor()，354 def
，121 dendrogram()，544 describe()
，62，66 dir()，116
drop()，179 dropna()，56，268，461
DTC()，*see* DecisionTreeClassifier
()
DTR()，*see* DecisionTreeRegressor
()
dtype，43 ElasticNetCV()，279
enumerate()，217
export_text()，356
export_tree()，365 fit()，118
，181，218 fit_transform()，1
19 for，59 GaussianNB()，182
GBR()，*see* GradientBoosting-
Regressor()，
461 get_influence()，121
get_dummies()
get_prediction()，120，314
get_rdataset()，535 glm()，31
3 glob()，437
GradientBoostingClassifier()
，361
GradientBoostingRegressor()
，354，361 GridSearchCV()，27
6 groupby()，490 hist()，62
iloc[]，58，59 import，42
imshow()，50，449 ISLP.bart，
362 ISLP.cluster，544 json，43
7 KaplanMeierFitter()，502
keras，437 KFold()，219
KMeans()，542，543 Kmeans()，
542 KNeighborsClassifier()，1
83 lambda，58 LDA()，
see LinearDiscriminant-
Analysis()，132
lifelines，490
legend()
LinearDiscriminantAnalysis()
，174，179 LinearGAM()，317
LinearRegression()，280

load_data(), 117 loc[], 58, 59, 177
log_loss(), 355 LogisticGAM(), 3
23 logrank_test(), 490 lowess(),
324 matplotlib, 48 max(), 66
mean(), 48 median(), 197 min(), 66
MNIST(), 444 ModelSpec(), 116–11
8, 122, 124, 267 MS(),
see ModelSpec(),
mult_test() *see* multipletests()
multipletests(), 586
multipletests(), 583, 589
multivariate_logrank_test(), 49
6 NaturalSpline(), 317, 319 ndim,
42 nn.RNN(), 461 normal(), 132, 28
6, 555 np, *see* numpy, 54, 180
np.allclose(), 190 np.any(), 54
np.arange(), 51 np.argmax(), 122
np.array(), 42 np.concatenate(),
133 np.corrcoef(), 46, 554
np.empty(), 224 np.isnan(), 268
np.ix_(), 53 np.linalg.svd(), 53
9 np.linspace(), 50
np.logspace(), 318 np.mean(), 47,
176 np.nan, 60 np.nanmean(), 541
np.percentile(), 228 np.power(),
219 np.random.choice(), 553
np.random.default_rng(), 46, 47
np.random.normal(), 45
np.sqrt(), 45
np.squeeze(), 457 np.std(), 47
np.sum(), 43 np.var(), 47
np.where(), 180 ns(), 317 numpy, 42,
555 os.chdir(), 55 outer(), 219
pairwise_distances(), 554
pairwise_tukeyhsd(), 587 pandas, 5
5 params, 175 partial(), 222, 269
PCA(), 280, 537, 540, 554 pd,
see pandas
pd.crosstab(), 555 pd.cut(), 315
pd.get_dummies(), 314
pd.plotting.scatter_matrix(), 62
pd.qcut(), 314, 315 pd.read_csv(),
55, 556 pd.Series(), 62 Pipeline(),
275 plot(), 48, 61, 356, 490
plot.scatter(), 120 plot_gam(), 32
1 plot_svm(), 398 PLSRegression(),
282 poly(), 125, 313, 327 predict(),
175, 178, 181, 216, 218, 323, 358
predict_survival_function(), 493
print(), 40 pvalues, 175 pygam, 307,
317 pytorch_lightning, 435 QDA(),
see QuadraticDiscriminant-
Analysis() QuadraticDiscriminantAnalysis(), 174,
181 random(), 555
RandomForestRegressor(), 354, 360
read_image(), 436 reindex(), 461
reshape(), 43

return, 198 RF(), *see* RandomForestRegressor() rng, *see* np.random.default_rng() rng.choice(), 60 rng.standard_normal(), 60 roc_curve(), 392 RocCurveDisplay.from_estimator(), 387 savefig(), 50 scatter(), 49, 61 scipy.interpolate, 315 score(), 218, 461 seed_everything(), 436 set_index(), 57 set_title(), 49 set_xlabel(), 49 set_xscale(), 198 set_ylabel(), 49 set_yscale(), 198 shape, 43 ShuffleSplit(), 219 sim_time(), 495 SimpleDataModule(), 441 SimpleModule.classification(), 446 SimpleModule.regression(), 442 skl, *see* sklearn.linear_model skl.ElasticNet(), 273, 277 skl.ElasticNet.path, 274 skl.ElasticNet.path(), 273 sklearn, 118, 181 sklearn.ensemble, 360 sklearn.linear_model, 267 sklearn.model_selection, 267 sklearn_selected(), 269 sklearn_selection_path(), 270 sklearn_sm(), 218 skm, *see* sklearn.model_selection skm.cross_val_predict(), 271 skm.KFold(), 271 skm.ShuffleSplit(), 272 slice(), 51, 462 sm, *see* statsmodels, 174, 192, 226 sm.GLM(), sm.Logit(), 174 sm.OLS(), 118, 129, 174, 319 StandardScaler(), 185, 438, 537, 555 statsmodels, 116, 173 std(), 186 Stepwise(), 269 str.contains(), 59 subplots(), 48 sum(), 43, 268 summarize(), 118, 129, 223, 226 summary(), 119, 322, 587 super(), 440 SupportVectorClassifier(), 387, 389–391, 393 SupportVectorRegression(), 394 SVC(), *see* SupportVectorClassifier(), 539 SVR(), *see* SupportVectorRegression(), 441 svd(), 495 sympy, 437 torch, 435 torchinfo, 436 torchmetrics, 436 torchvision, 436 ToTensor(), 444 train_test_split(), 186, 216 transform(), 118, 119 ttest_1samp(), 584 ttest_ind(), 590 ttest_rel(), 587 tuple, 43 uniform(), 555 value_counts(), 66 var(), 536 variance_inflation_factor(), 116, 124 VIF(), *see* variance_inflation_factor(), 355 zip(), 60, 312 where()

q-值，589
二次，98
二次判别分析，4, 135, 156–157, 164–167

定性, 2, 27, 91, 135, 167, 202 变量, 91–94 定量, 2, 27, 91, 135, 167, 202 径向核, 381, 383, 390 随机森林, 11, 331, 343, 346– 347, 354, 360–361 随机种子, 46 重采样, 577–582 召回率, 155 接收者操作特征曲线 (ROC), 154, 382–383 推荐系统, 516 修正线性单元, 401 循环神经网络, 416–427 递归二叉分裂, 334, 337, 338 可约误差, 17, 90 回归, 2, 11, 27 局部, 289, 290, 304–305 分段多项式, 294–295 多项式, 289–292, 299 样条, 289, 294 树, 331–337, 358–360 正则化, 230, 240, 406, 484– 486 ReLU, 401 重采样, 201–214 残差, 71, 81 图, 100 标准误, 75, 77–78, 88– 89, 109 学生化, 104 平方和, 71, 79, 81 残差, 263, 348 响应, 15 岭回归, 11, 240–244, 385, 484 风险集, 473 稳健, 374, 376, 535 ROC 曲线, 154, 382–383, 486– 487 R^2, 77–80, 88, 109, 238 地毯图, 314 尺度等变, 242 Scheffé 方法, 572 碎石图, 512, 514–515

肘部, 514 半监督学习, 27 敏感度, 153, 155, 156 分离超平面, 367–372 Seq2Seq, 425 序列, 41 收缩, 230, 240, 484–486 惩罚, 240 sigmoid, 401 信号, 252 签名, 45 奇异值分解, 539 松弛变量, 375 切片, 51 斜率, 71, 72 `Smarket` 数据集, 2, 3, 12, 173, 184, 196 平滑器, 308 平滑样条, 290, 300–303 软间隔分类器, 372–374 软阈值化, 250 softmax, 145, 405 稀疏, 244, 252 稀疏矩阵格式, 414 稀疏性, 244 特异性, 153, 155, 156 样条, 289, 294–303 三次, 296 线性, 296 自然, 297, 301 回归, 289, 294–299 平滑, 30, 290, 300–303 薄板, 22 标准误, 75, 101 标准化, 185 统计模型, 1 阶跃函数, 111, 289, 292–293 逐步模型选择, 11, 231, 233 随机梯度下降, 429 字符串, 41 字符串插值, 490 树桩, 349 子集选择, 230–240 子树, 336 监督学习, 25–27, 261 支持向量, 371, 376, 385 分类器, 367, 372–377 机, 5, 11, 24, 377–386

回归，386 生存分析，469–502 曲线，472，483 函数，472 时间，470 协同，70，89，95–98，110–111 系统，16

t分布，77, 165 t统计量，76 t检验单样本，583, 584, 588 配对，587 两样本，559, 570, 571, 577–581, 584, 590 测试误差，35, 37, 176 均方误差，28–32 观察值，28 集，30 统计量，559 理论零分布，577 时间序列，101 总平方和，79 追踪，102 训练，21 训练数据，20 误差，35, 37, 176 均方误差，28–31 变换器，311 树，331–342 基于树的方法，331 真阴性，155 真阳性，155 真阳性率，155, 156, 382 截断幂基，296 Tukey方法，571, 585, 587 调优参数，187, 240, 484 两样本*t*检验，474 第一类错误，155, 562–565 第一类错误率，563 第二类错误，155, 563, 568, 584

无监督学习，25–27, 255, 260, 503–552 `USArrests` 数据集，12, 507, 508, 510, 512, 513, 515, 516, 518, 519

验证集，202方法，202–204变量，15因变量，15虚拟变量，91–94，97–98重要性，346，360自变量，15指标，35输入，15输出，15定性，91–94，97–98选择，86，230，244方差，18，31–34，159膨胀因子，108–110，123变系数模型，305

`Wage` 数据集，1，2，8，9，12，290，291，293，295，297–300，302–306，309，315，327 弱学习器，343 最弱环节剪枝，336 `Weekly` 数据集，12，196，226 权重冻结，412，419 权重共享，418 加权最小二乘，103，304 权重，404 有放回，214 类内协方差，150 包装器，217